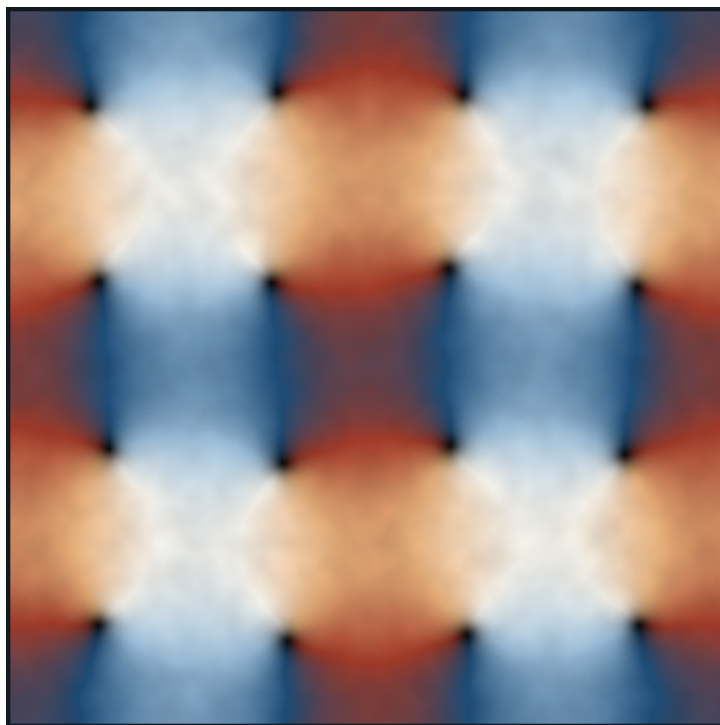




A TRIBUTE TO HENRI GODFRIN

Quantum Fluids in Lean 4

*Machine-checked mathematics, solver experiments
and the measurements of a master experimentalist*

Xavier Callens

SocrateAI Research

with the Lean 4 library `QuantumFluids`
and the rusty-SUNDIALS solver (`qf-pgpe`, `CVODE`)

October 2026 · first edition

Quantum Fluids in Lean 4: a tribute to Henri Godfrin.

Xavier Callens, SocrateAI Research. First edition, October 2026.

How to cite. X. Callens, *Quantum Fluids in Lean 4: a tribute to Henri Godfrin*, SocrateAI Research (2026), doi:[10.5281/zenodo.23272153](https://doi.org/10.5281/zenodo.23272153). This is the concept DOI of the book; it always resolves to the latest edition.

Licence. Text and figures: Creative Commons Attribution 4.0 International (CC BY 4.0). Lean, Python and Rust sources: MIT or Apache-2.0, at the reader's option.

Sources. Every chapter, figure script, number file, Lean file and audit log of this book is in the source archive deposited with it under the DOI above, and in the directory `book/` of <https://github.com/xaviercallens/SocrateAI-Scientific-QuantumFluids>. Lean 4 toolchain v4.34.0-rc2, Mathlib at commit 85e3a25. The Lean library `QuantumFluids` is also archived as software under doi:[10.5281/zenodo.22855581](https://doi.org/10.5281/zenodo.22855581); the solver is `rusty-SUNDIALS`, <https://github.com/xaviercallens/rusty-SUNDIALS>. Appendix B says how to rebuild every figure and rerun the Lean audit.

A note on the dedication. Henri Godfrin has not reviewed this book, and nothing in it speaks for him or for his collaborators. The book describes their published measurements; the proofs are proofs of mathematical statements, not of experimental data; every remaining error is the author's.

*To Henri Godfrin,
and to everyone who builds the cold apparatus
that lets a quantum fluid answer.*

Preface

This book has one idea and one dedication.

The idea is that a quantum fluid can be understood through *three instruments used together*. The first is the experiment: a measurement made at the bench, in a cryostat, on a neutron beam line, in a vacuum chamber of cold atoms. The second is a proof: a statement about the mathematics of the model, checked by a computer so that it cannot be wrong in the way an arithmetic slip or a misprinted coefficient can be wrong. The third is a simulation: a solver that integrates the equations of the model and lets us watch what the proof only asserts. Each instrument is blind where the others see. A proof says nothing about helium; an experiment does not tell you whether a printed formula is right; a simulation cannot distinguish a theorem from a coincidence. Used together, and with their limits stated, they give something firmer than any of them alone.

The dedication is to Henri Godfrin. His group at the Institut Néel in Grenoble does the hardest kind of physics: it cools matter to microkelvin temperatures, puts a single atomic layer of helium-3, or a bath of helium-4, in a neutron beam, and reads the excitation spectrum of a quantum fluid directly. The measurements of that programme — the roton-like collective mode of a two-dimensional Fermi liquid, and the dispersion relation of superfluid helium-4 together with its thermodynamic consequences — are the experimental thread of this book. Where the Lean library formalizes a statement that appears in his papers, the book says exactly which statement and what the proof does and does not show. Nothing here speaks for him: he has not reviewed this book, the proofs are of mathematics and not of his data, and every error that remains is ours.

The Lean 4 library is `QuantumFluids`, a collection of machine-checked theorems on the mathematical structure of quantum fluids; the solver is `rusty-SUNDIALS`, a Rust implementation of the SUNDIALS suite with a quantum-fluids module `qf-pgpe`. Both belong to a research programme that is run under strict discipline: predictions written before data are read, failures reported, claims retracted when they fail. That discipline shapes the book. Every chapter contains a box that says what is established and what is not, and the last chapter tells, plainly, what went wrong in the programme and how it was found. A tribute to an experimentalist should be written in the register of experimentalists: with the error bars in view.

The book is organized as ten chapters that can be read in order or separately. Chapter 1 introduces the three instruments and the measurements that motivate them; Chapter 2 is a primer on Lean and on the solver. Chapters 3 to 9 each take one physical theme — circulation and vortices, the specific heat of helium-4, the dispersion relation, the Kosterlitz–Thouless flow, friction, two-dimensional Fermi liquids, flow past an obstacle — and treat it with a measurement, a theorem and a computation. Chapter 10 is about trust. Every figure in the book is produced by a script that is part of the source, and every number quoted in the text comes from a file or a computation recorded alongside it.

How the book was made. The chapters were written separately, each from the same briefing, and combined at the end; the text, the Lean code and the figures were drafted with the assistance of an AI system and then checked: the Lean files compile against the pinned Mathlib, the solver code was run, and the sources of every number are listed in the per-chapter reports kept with the source. Corrections are welcome.

X. C.
October 2026

Contents

Preface	v
1 A Tribute: The Bench, the Proof and the Solver	1
1.1 The bench: two spectra and the cold that made them	2
1.2 What a neutron can see	4
1.3 From the curve to the calorimeter	6
1.4 Three instruments, one discipline	7
1.5 What the proof assistant says: five statements	8
1.6 What the solver shows: one pair, four checks	11
1.7 What the measurements hand to theory	16
Exercises	18
Notes and further reading	18
2 Two Languages: Lean 4 and rusty-SUNDIALS	19
2.1 Two ways to be wrong	19
2.2 Lean 4 in twenty minutes	20
2.3 A theorem about the equation the solver integrates	23
2.4 rusty-SUNDIALS in twenty minutes	26
2.5 The solver on problems with known answers	28
2.6 The plane wave: a theorem and three integrators	31
2.7 Where a theorem and a program part: aliasing	34
2.8 What each instrument can see	37
2.9 Notes and further reading	39

3	From the Wave Function to the Fluid: Circulation and Vortices	41
3.1	An observation: circulation comes in whole units	41
3.2	The wave function as a fluid	42
3.3	Quantized circulation	44
3.4	The vortex	46
3.4.1	A vortex is a zero of ψ	46
3.4.2	Structure of one vortex	46
3.4.3	A pair, and the reduced model	46
3.4.4	What a loop sees: scale-resolved winding	47
3.5	The solver: vortices in the projected Gross–Pitaevskii field	48
3.5.1	The engine and the imprint	48
3.5.2	Seeing the fluid in ψ	49
3.5.3	Counting quanta	50
3.5.4	Where the energy sits	53
3.5.5	The speed of a pair, and the lesson of the periodic box	54
3.5.6	Leapfrogging pairs, and the reduced model in CVODE	56
3.6	What this chapter establishes, and what it does not	58
	Exercises	59
	Notes and further reading	60
4	The Specific Heat of Helium-4 and Godfrin’s Series	61
4.1	A cubic law, and what bends it	61
4.2	From a dispersion relation to a heat capacity	62
4.2.1	A gas of phonons	62
4.2.2	The bending	63
4.2.3	A series in T	64
4.3	What the proof assistant says	65
4.3.1	Step one: the Bose integral	66
4.3.2	Step two: inverting the dispersion and counting states, without analysis	66
4.3.3	Step three: the six coefficients	68
4.3.4	Closing the links that were made by inspection	69
4.4	A confirmation, not a finding	70
4.5	What the solver says	72
4.5.1	The Bose integrals as initial-value problems	73
4.5.2	The thermal integral with no series at all	74
4.5.3	What numerics cannot see	76
4.6	The series meets the measured curve	77
4.6.1	Reproducing the paper’s table	77
4.6.2	Which excitations carry the heat	77
4.6.3	An error budget	78

4.6.4	A series that does not converge	79
4.7	What this chapter establishes, and what it does not	81
	Exercises	82
	Notes and further reading	83
5	Phonons, Rotons and the Dispersion Relation	85
5.1	A curve read off a spectrometer	85
5.2	Landau's reading of the curve	88
5.3	Three ways a phonon can fail to live	90
5.3.1	The sign of the curvature	90
5.3.2	Where the channel closes: three numbers, not one	90
5.3.3	Two rotons and the plateau	93
5.4	Bogoliubov's phonon, and what it cannot do	93
5.4.1	A phonon that turns into a particle	93
5.4.2	What is proved about it	94
5.4.3	Against helium: how far is the real curve from the weak-coupling one?	96
5.5	The solver as a spectrometer	96
5.5.1	Exciting every mode at once	97
5.5.2	The frame lesson	97
5.5.3	What the measurement shows	98
5.5.4	One function of one variable	100
5.5.5	Waves in space: the group velocity	101
5.5.6	A referee: CVOICE	103
5.5.7	Amplitude: the dispersion relation is a small-amplitude statement	105
5.6	What this establishes, and what it does not	105
	Exercises	106
	Notes and further reading	107
6	Vortices in Two Dimensions: the Kosterlitz–Thouless Flow	109
6.1	A jump fixed by constants	109
6.2	Stiffness, vortices and the argument of Kosterlitz and Thouless	110
6.3	What can be proved about the flow	112
6.4	The Coulomb gas inside the solver	117
6.5	The flow, integrated by CVOICE	118
6.6	The simulated field near the transition	123
6.7	What this chapter establishes, and what it does not	127
7	Friction, Diffusion and the Dissipative Vortex	131
7.1	A pair that opens, a pair that closes	131
7.2	The dissipative point vortex	132

7.3	What Lean proves about the model	133
7.4	The model in CVOICE	135
7.5	A vortex pair in a thermal field	136
7.6	Friction follows the temperature, not the normal density	142
7.7	The sideways force and the random kicks	145
7.8	The wind that was not	147
7.9	An estimator that was exact, and was not	150
7.10	What this chapter establishes, and what it does not	150
7.11	Exercises	151
7.12	Notes and further reading	152
8	Two-Dimensional Fermi Liquids: Zero Sound and the Roton Mode	153
8.1	What the neutrons saw	154
8.2	A Fermi liquid with one number	155
8.3	What Lean proves	156
8.4	The structure factor, the pole and the sum rule	158
8.5	Integrating the kinetic equation with CVOICE	160
8.6	Landau damping is dephasing	164
8.7	A plasma cousin, and the property that decides	166
8.8	Beyond Landau: the window at twice the Fermi momentum	169
8.9	What is established, and what is not	171
9	Flow Past an Obstacle: Reproducing a Published Experiment	173
9.1	A hole in the fluid	173
9.2	The deposited run and the model	174
9.3	Why a subsonic flow can dissipate	176
9.4	The reproduction	181
9.5	Counting vortices	184
9.6	The birth of a pair	187
9.7	What does not agree, and what we do not know	190
9.8	Exercises	192
9.9	Notes and further reading	193
10	Trust: Audit, Failure and the Road Ahead	195
10.1	A number that passed once	195
10.2	What each check can and cannot see	199
10.3	The audit of the library	199
10.4	What failed, plainly	203
10.5	Speed is a claim too	210
10.6	What a reader can reuse	211

10.7	What is still open	212
A	The Lean Library: a Complete Catalogue	215
A.1	The catalogue, module by module	217
A.1.1	BoseIntegral	217
A.1.2	ChargeLattice	218
A.1.3	CompactBoson	218
A.1.4	ContinuumWinding	219
A.1.5	DissipativeVortexDynamics	220
A.1.6	DualLength	220
A.1.7	Duality	221
A.1.8	EinsteinRelation	222
A.1.9	Fricke	223
A.1.10	FrictionKinetic	223
A.1.11	GP Galerkin	224
A.1.12	HeliumKinematics	225
A.1.13	KTFlow	226
A.1.14	LevelRankDuality	226
A.1.15	MadelungNSE	226
A.1.16	MadelungSplit	227
A.1.17	MatchingScreening	227
A.1.18	PairPolarisation	228
A.1.19	PhaseMixing	229
A.1.20	PhononSeries	229
A.1.21	PhononSpecificHeat	230
A.1.22	QHFricke	231
A.1.23	QuantizedCirculation	231
A.1.24	QuantumFluidsShell	232
A.1.25	QuasiPeriodicBound	233
A.1.26	RCFTDuality	233
A.1.27	RipsFloor	233
A.1.28	ScaleResolvedWinding	234
A.1.29	SectorDuality	234
A.1.30	SectorTemperature	234
A.1.31	ShellHamiltonian	235
A.1.32	SigmaRule	236
A.1.33	TopologicalProtection	237
A.1.34	Villani	238
A.1.35	VortexWinding	239
A.1.36	WassersteinCertificate	239

A.1.37	ZeroSound	240
A.2	Lean written for this book	240
A.2.1	Cho1_NeutronWindow	241
A.2.2	Cho2_Galerkin	241
A.2.3	Cho2_Primer	242
A.2.4	Cho3_PointVortexPair	242
A.2.5	Cho4_DosLink	243
A.2.6	Cho5_BogoliubovDispersion	245
A.2.7	Cho6_KTForward	246
A.2.8	Cho7_DissipativePairs	247
A.2.9	Cho8_ZeroSound2D	248
A.2.10	Cho9_FlowPast	248
B	Reproducibility: Rebuilding the Figures and Running the Audit	251
B.1	The machine and the software	251
B.2	Rebuilding a figure	252
B.3	CVODE: which build is imported	256
B.4	Running the Lean audit	257
B.5	Limits of what can be reproduced	259

List of Figures

1.1	The landmarks this book revisits	2
1.2	The curve the neutron has to find	5
1.3	One vortex pair seen by the solver and the proof	13
1.4	Where the speed of the pair comes from	15
2.1	Regions of absolute stability of the Adams and BDF formulas	29
2.2	CVODE on problems with closed-form solutions	30
2.3	The exact plane wave against three integrators	33
2.4	The same equation in the two languages	36
2.5	Theorems and engine, in rates and in time	36
3.1	One wave function, three faces	50
3.2	Counting quanta on the solver field	52
3.3	Where the energy of a vortex sits	53

3.4	The speed of a vortex pair in the periodic box	55
3.5	Two vortex pairs leapfrog	57
4.1	The phonon branch of superfluid helium-4 measured by neutron scattering . . .	63
4.2	The six Bose integrals of the Lean statement, computed by CVODE	74
4.3	The solver against the proved series	76
4.4	Which excitations carry the heat?	78
4.5	Eq. (22) against the integral of the measured dispersion	79
4.6	The series is asymptotic, not convergent	80
5.1	The dispersion relation of superfluid helium-4 at saturated vapour pressure . . .	86
5.2	Where the three-phonon channel closes	92
5.3	The solver measures Bogoliubov's dispersion relation	99
5.4	A spectrometer map of the simulated fluid, and the universal curve	100
5.5	A dispersive ring wave	102
6.1	The logarithmic interaction of vortices in the solver	118
6.2	The Kosterlitz flow, integrated with CVODE	121
6.3	The universal jump, the finite box and the essential singularity	122
6.4	The simulated Bose field near the transition	125
6.5	The two Lean statements about vortex positions, tested on the solver's configurations	126
7.1	CVODE against the Lean statements on dissipative vortex pairs	137
7.2	A zero-temperature control of the vortex imprint	139
7.3	Two antiparallel vortex pairs in a thermal projected Gross–Pitaevskii field	140
7.4	The friction in the field	141
7.5	Two theorems that were true and did not describe what they were applied to . . .	149
8.1	The zero-sound root and what it carries	159
8.2	CVODE against the closed forms of the kinetic equation	163
8.3	The Fermi circle as the dynamical variable	165
8.4	Vlasov–Poisson: the same density wave on two backgrounds	168
8.5	The (q, ω) plane in free-gas units	170
9.1	The wake at $t = 50 \tau$	174
9.2	The flow is locally supersonic long before the first vortex	180
9.3	The force on the obstacle	185
9.4	How many vortices?	188
9.5	The first vortex pair	189
10.1	The energy estimator of the friction fails a known-answer test	198
10.2	The library as audited for this book	201

10.3	Every criterion the ledger records, by campaign	205
10.4	Time per step of the projected Gross–Pitaevskii integrator in several engines . . .	210

A Tribute: The Bench, the Proof and the Solver

On 29 March 2012 *Nature* published a spectrum. A neutron beam at the Institut Laue–Langevin (ILL) in Grenoble had been scattered by a single atomic layer of liquid helium-3 adsorbed on graphite, a film that is two-dimensional for every purpose of the physics, and from the neutrons that came out the authors had read the energy of the density waves of the film as a function of their wave number [1]. The curve had a minimum. It was a *roton-like* minimum, a feature long familiar from superfluid helium-4, and here it was found in a Fermi liquid; and the density waves, the collective mode of the film, which enter the band of particle–hole excitations and are strongly damped there, were found to reappear beyond twice the Fermi wave number as a well-defined excitation. Nine years later a second paper with the same first author reported the same kind of measurement on bulk superfluid helium-4, at the ILL spectrometer IN5: the dispersion relation of the Landau elementary excitations, phonon, maxon and roton, and the thermodynamic consequences that follow from it [2].

A spectrum is the most complete thing one can ask a quantum fluid for. Landau saw in 1941 that at low temperature the thermodynamics of liquid helium is that of a gas of elementary excitations [3]; the energy of an excitation as a function of its wave number, the *dispersion relation*, is therefore the one curve from which the rest follows, and a neutron beam is the instrument that can draw it. Drawing it takes a sample cooled until it is a quantum fluid, a beam line, and a command of the kinematics of the collision good enough to read a curve out of a cloud of counts. That is a craft. This book tries to honour it in the only way a text can: by saying exactly what the measurements are, why they are hard, and what they settled.

It then does something that the measurements themselves do not ask for: it reads them with three instruments used together. The *bench* is the experiment, a number about nature together with the procedure and the error bar that make it a number. The *proof* is a statement about the mathematics of a model, checked by the proof assistant Lean 4, so that it cannot be wrong in the way a misprinted coefficient can be wrong. The *solver* is rusty-SUNDIALS, a Rust implementation of the SUNDIALS suite of solvers [4] with a module for quantum fluids; with it we integrate the equations of a model and watch what a proof only asserts. None of the three can do the work of the others. This chapter says what each can do, shows each at work on an example, and closes with an account of what the two measurements above hand over to theory and of how a companion made of proofs and simulations can be of use to it without presuming to speak for it. The order is the bench (Sections 1.1 and 1.2), the chain from a curve to a calorimeter (Section 1.3), the method (Section 1.4),

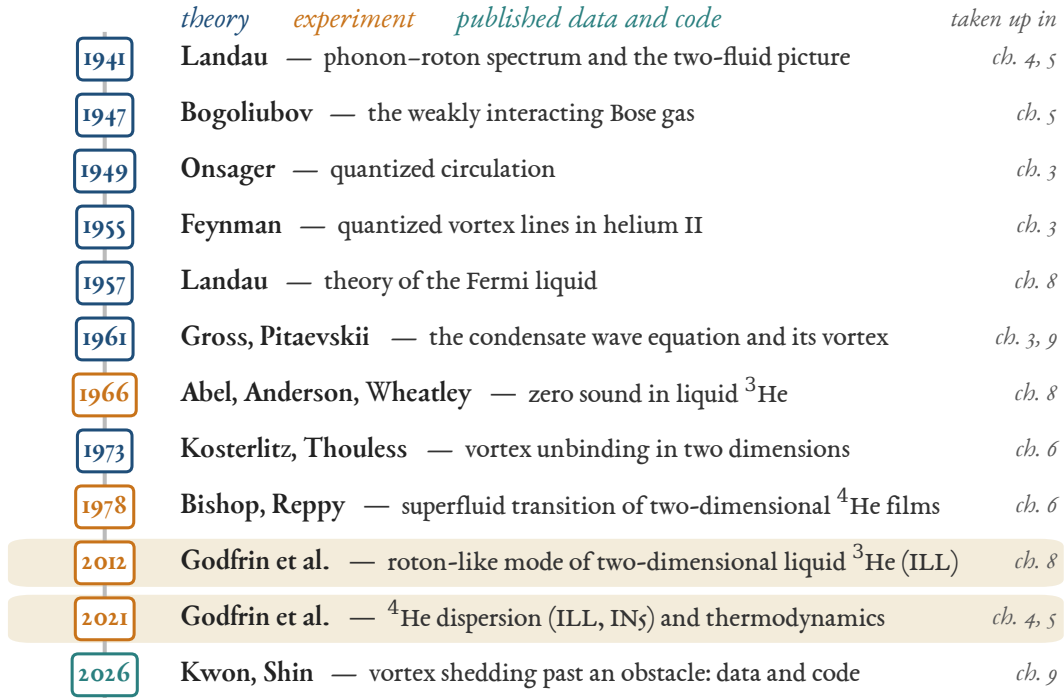


Figure 1.1: The landmarks this book revisits. Blue: theory; orange: experiment; teal: published data and code. The two measurements of Godfrin et al. that run through the book sit on the gold band. Rows are in chronological order and equally spaced (the figure is not to scale in time); the grey tags name the chapters, by their titles, that take each landmark up. The date given for Landau’s Fermi-liquid paper is that of the English translation in the bibliography. Script: figures/ch01_timeline.py.

the proof (Section 1.5), the solver (Section 1.6) and the handover (Section 1.7). Figure 1.1 places the two measurements among the landmarks that the book revisits.

1.1 The bench: two spectra and the cold that made them

Both papers have Henri Godfrin of the Institut Néel in Grenoble as first author, with collaborators at the ILL and elsewhere. The first box says in a few lines what the group in which he works does; it is the whole of what this book will say about the group, and it comes from the institute’s own pages.

GODFRIN THE LABORATORY

Henri Godfrin is a CNRS Director of Research at the Institut Néel, Grenoble, and a permanent researcher of its Ultra-low Temperature group (UBT). The group does condensed-matter experiments down to the *microkelvin* range: nuclear adiabatic-demagnetization cryostats that

reach about 100 μK , dilution refrigerators with a base temperature of 8 mK, neutron scattering for the structure and excitations of quantum matter, quantum fluids and solids (^3He , ^4He) and amorphous matter, and microwave optomechanics. The group is part of the European Microkelvin Platform.

Two of its published measurements are the experimental thread of this book.

GODFRIN 2012: A ROTON COLLECTIVE MODE IN A TWO-DIMENSIONAL FERMI LIQUID

H. Godfrin et al., *Nature* **483**, 576–579 (2012), published 29 March 2012 [1]. Inelastic neutron scattering at the ILL (spectrometer IN6) on two-dimensional liquid ^3He : a monolayer adsorbed on graphite pre-plated with a solid monolayer of ^4He . The collective density mode of the film, strongly damped inside the particle–hole band, reappears at wave numbers larger than twice the Fermi wave number as a well-defined excitation whose dispersion shows a roton-like minimum. The authors are at CNRS (France), Aalto University, Oak Ridge National Laboratory, SUNY Buffalo and Johannes Kepler University Linz.

GODFRIN 2021: THE DISPERSION OF HELIUM-4 AND ITS THERMODYNAMICS

H. Godfrin et al., *Phys. Rev. B* **103**, 104516 (2021), arXiv:2012.09067 [2]. Inelastic neutron scattering on superfluid ^4He at the ILL spectrometer IN5: the dispersion relation of the Landau elementary excitations (phonon, maxon, roton) over wave numbers up to a few $\text{\AA}^{-1}$, and the thermodynamic consequences. Equation (22) of the paper gives the phonon specific heat as the series $C_V = AT^3 + CT^5 + DT^6 + ET^7 + KT^8 + LT^9$ for a dispersion $\varepsilon = \hbar ck(1 + \alpha_2 k^2 + \dots + \alpha_6 k^6)$ with $\alpha_1 = 0$; the paper remarks that earlier published versions of this series contain errors. A review of the excitation spectra of quantum fluids by the same first author with E. Krotscheck followed in 2022 [5].

Both are spectroscopies of a quantum fluid. In both the quantity measured is an energy as a function of a wave number, and in both what makes the number worth having is that it is directly the object that theory is built from: the elementary excitations of Landau’s picture of helium-4, and, for the film, a collective mode of the kind that Landau’s theory of the Fermi liquid calls zero sound (Chapter 8).

None of the difficulties of such a measurement is exotic, and together they are severe. The sample is nearly nothing: a neutron is a weak probe, which is why it can be used through the walls of a cryostat at all, and a film of a few atomic layers presents very little matter to the beam, while the substrate and the sample cell scatter too. The sample is a strong absorber of the probe: helium-3 has an absorption cross section of 5333 barn for neutrons of speed 2200 m/s, growing in inverse proportion to the speed for slower neutrons [6], which is why helium-3 gas is the working medium of neutron detectors; a helium-3 target eats its own beam, and the cold neutrons that match its excitations are absorbed more strongly than thermal ones. And the sample must be cold on the scale

of what is measured: the roton of helium-4 costs 8.6 K (Figure 1.2), and only a sample far below that has a roton gas empty enough for the spectrum to show single excitations. The dilution refrigerators and demagnetization stages of the first box are the unglamorous half of every spectrum in this book. A fourth difficulty is geometry, and it is the subject of the next section.

What the two measurements settled can be said briefly: that a roton-like collective mode exists in a two-dimensional Fermi liquid and is long lived there, and that the excitation curve of helium-4, from the phonon to the roton, can be measured and then carried through to the thermodynamics. What they left open is the subject of Section 1.7.

1.2 What a neutron can see

A neutron of wavelength λ has wave number $k = 2\pi/\lambda$ and kinetic energy $E = \hbar^2 k^2 / 2m_n = (81.80 \text{ meV } \text{\AA}^2) / \lambda^2$: at $\lambda = 5 \text{ \AA}$ that is 3.27 meV and a speed of 791 m/s. These are the right sizes for a quantum fluid: a wavelength of a few ångström, comparable with the spacing of the atoms, and an energy comparable with those of the excitations (the roton of helium-4 costs 0.74 meV, 23 per cent of the energy of such a neutron). A photon of that wavelength carries 2.48 keV, about 760 000 times more; a photon of the neutron's energy has a wavelength of 0.38 mm. This coincidence of scales is why the neutron is the probe of excitations in condensed matter [7, 8].

In a collision the neutron changes its wave vector from $\mathbf{k}_i$ to $\mathbf{k}_f$, and the sample receives the wave-vector transfer $\mathbf{Q} = \mathbf{k}_i - \mathbf{k}_f$ and the energy $\hbar\omega = E_i - E_f$. The number of neutrons that emerge per unit solid angle and energy is proportional to $(k_f/k_i) S(\mathbf{Q}, \omega)$, where S is the dynamic structure factor of the sample, the space-time Fourier transform of its density-density correlation function. In a fluid at low temperature that supports a well-defined collective excitation, $S(\mathbf{Q}, \omega)$ has a ridge at $\hbar\omega = \varepsilon(Q)$; the dispersion relation *is* that ridge. A time-of-flight spectrometer such as IN5, a direct-geometry instrument for cold neutrons whose choppers define the incident pulses [9], sorts the neutrons that leave the sample by flight time and direction, and so records the ridge for many Q at once.

Figure 1.2(a) puts a real curve in front of these formulas. It is the dispersion relation of superfluid ^4He at saturated vapour pressure, from the table that the authors of [2] distribute as an ancillary file of the arXiv version of their paper: 1727 rows between $Q = 0$ and $3.6 \text{ \AA}^{-1}$, an uncertainty being given for 34 of them. It is the authors' processed curve, not raw instrument counts, and we use it only as a curve. It has the three regions of Landau's spectrum: the phonon branch at small Q , the maxon, a maximum of 1.19 meV (13.8 K) at $Q = 1.11 \text{ \AA}^{-1}$, and the roton, the minimum of 0.741 meV (8.6 K) at $Q = 1.92 \text{ \AA}^{-1}$, followed by the rise to a plateau.

Energy and momentum conservation limit what a given incident wavelength can reach. The three vectors $\mathbf{k}_i$, $\mathbf{k}_f$ and $\mathbf{Q}$ form a triangle, so $|k_i - k_f| \leq Q \leq k_i + k_f$, together with $E_f = E_i - \varepsilon$ this bounds the energy that can be left in the sample at a wave-vector transfer Q ,

$$\varepsilon \leq \frac{\hbar^2}{2m_n} (2k_i Q - Q^2) = E_i - \frac{\hbar^2}{2m_n} (k_i - Q)^2,$$

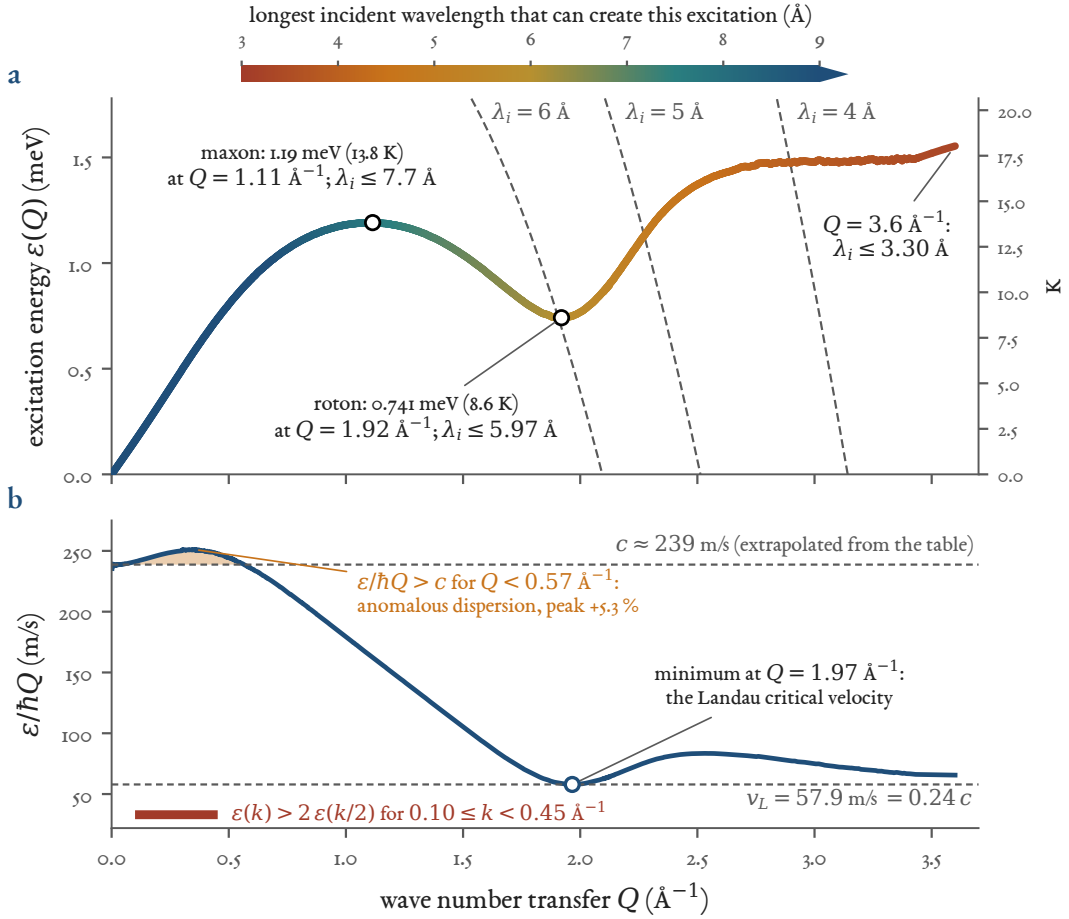


Figure 1.2: The curve the neutron has to find. **(a)** Dispersion relation of superfluid ^4He at saturated vapour pressure, from the table of [2] (author-processed values, not raw counts; open circles: maximum and minimum of the table). The *colour is not data*: it is the longest incident wavelength with which a neutron could create the excitation at that point, from inequality (1.1) (statement 5 of Section 1.5); a point above a dashed line, the kinematic limit of a neutron of wavelength $\lambda_i = 6, 5$ or $4 \text{ \AA}$, cannot be reached with it. These are limits of kinematics, not instrument settings. **(b)** Phase velocity $\varepsilon/\hbar Q$ of the same table; c is extrapolated from $\varepsilon/\hbar Q = c(1 + aQ^2)$ fitted on $0 < Q \leq 0.2 \text{ \AA}^{-1}$. Shaded: $\varepsilon/\hbar Q > c$. Red bar: $\varepsilon(k) > 2\varepsilon(k/2)$ on a cubic-spline interpolation of the table (below $0.1 \text{ \AA}^{-1}$ the table's rounding, $5 \times 10^{-5} \text{ meV}$, is comparable with the effect). Script: figures/ch01_bench.py.

a downward parabola with its apex E_i at $Q = k_i$: the dashed lines of the figure. Solved for the neutron instead of the sample, the same inequality reads $k_i \geq Q/2 + m_n \varepsilon/\hbar^2 Q$, or, multiplied by $\hbar/m_n$,

$$v_n \geq \frac{\hbar Q}{2m_n} + \frac{\varepsilon}{\hbar Q}. \quad (1.1)$$

The first term is the speed at which a neutron can just deliver the wave-vector transfer Q by reversing its direction without losing energy; the second is the price of the energy ε paid at that wave number, the phase velocity of the excitation. Section 1.5 gives this inequality in Lean; it is what colours the curve of Figure 1.2(a). To create the roton a neutron must be at least as fast as 663 m/s, that is $\lambda_i \leq 5.97 \text{ \AA}$ (and $E_i \geq 2.30 \text{ meV}$); to reach the last row of the table, $Q = 3.6 \text{ \AA}^{-1}$, it must have $\lambda_i \leq 3.30 \text{ \AA}$; to reach $Q = 3 \text{ \AA}^{-1}$, $\lambda_i \leq 3.88 \text{ \AA}$. At the other end, as $Q \rightarrow 0$ the condition becomes $v_n \geq c$: the neutron must outrun sound, $\lambda_i \leq 16.6 \text{ \AA}$. For comparison, the ILL lists the incident wavelengths of IN5 as 1.8 to 20 $\text{\AA}$ [9], a range that contains all of these; what was set for which part of the curve in the 2021 measurement is not something this book describes.

Panel (b) of the figure divides the table by Q . The phase velocity $\varepsilon/\hbar Q$ tends at small Q to the speed of sound, which the extrapolation puts at 239 m/s. It then *rises*, by 5.3 per cent to 251 m/s at $Q = 0.35 \text{ \AA}^{-1}$, falls back through c at $0.57 \text{ \AA}^{-1}$, and goes on down to a minimum of 57.9 m/s, $0.24c$, at $Q = 1.97 \text{ \AA}^{-1}$, beside the roton. The rise is the *anomalous dispersion* of helium-4 at low pressure, and its consequence is kinematic: a phonon may then decay into two (Section 1.5). The minimum is the Landau critical velocity: an object moving through the fluid at a speed below it cannot create any excitation, because creating one of wave number Q and energy ε takes a speed of at least $\varepsilon/\hbar Q$ — the very quantity that appears as the second term of (1.1), seen from the other side. Real flows lose their superfluidity earlier, typically by nucleating vortices [10], so v_L is the bound of an idealized argument and not a measured threshold.

1.3 From the curve to the calorimeter

The second paper does more than tabulate a curve. It follows Landau's chain from the dispersion relation to the thermodynamics, and the chain is short enough to write down.

LANDAU'S CHAIN, FROM $\varepsilon(k)$ TO C_V

At temperature T the fluid is a gas of bosonic excitations, so its energy per volume V and its specific heat are

$$\frac{E}{V} = \int \frac{d^3k}{(2\pi)^3} \frac{\varepsilon(k)}{e^{\varepsilon(k)/k_B T} - 1}, \quad C_V = \frac{\partial E}{\partial T}.$$

For phonons, $\varepsilon = \hbar ck$, the integral is the Bose integral $\int_0^\infty t^3/(e^t - 1) dt = \pi^4/15$ and

$$C_V = A T^3, \quad A = \frac{2\pi^2 k_B^4 V}{15 \hbar^3 c^3},$$

the T^3 law. A correction $\alpha_n k^n$ to the dispersion changes the density of states $k^2 dk/d\varepsilon$ and with it the coefficient of the next power of T ; for $\varepsilon = \hbar ck(1 + \alpha_2 k^2 + \dots + \alpha_6 k^6)$ the result is the series of Eq. (22) of the paper, $C_V = AT^3 + CT^5 + DT^6 + ET^7 + KT^8 + LT^9$, with no T^4 term because $\alpha_1 = 0$. Four of the six coefficients (A, C, E, L) bring the even zeta values

	can establish	cannot establish	a case from this programme
The bench	a number about nature, with its error bar and the procedure that produced it	that a printed formula is algebraically right; why the number is what it is	a six-term specific-heat series whose predecessors in the literature contained errors (Section 1.3)
The proof	that a statement follows from its hypotheses for all values, without arithmetic slips	that the hypotheses describe helium; that a correct theorem is new	the ‘dual length’ theorem about the Bogoliubov spectrum: correct, and withdrawn as a finding on 2026-09-20
The solver	what a model does for given parameters, with conservation laws as a check	why; that a coincidence is not a theorem; that the model is nature	a vortex pair that moved 23 per cent faster than the point-vortex formula on a torus, until the proof explained why (Section 1.6)

Table 1.1: What each instrument can and cannot establish, with one case of each from the programme behind this book.

$\zeta(4)$, $\zeta(6)$, $\zeta(8)$, $\zeta(10)$, which are rational multiples of powers of π ; the other two bring $\zeta(7)$ and $\zeta(9)$, which have no known closed form, and this is why a zeta value is written out in D and K and in no other coefficient.

A six-term series whose coefficients are built from products of the α_n and from zeta values is not something a reader can re-derive on a train, and the paper itself remarks that earlier published versions of this series contain errors. A measured curve cannot check an algebraic identity, and a calorimeter cannot either: the series is a mathematical consequence of the model dispersion, and a mathematical consequence is what a proof assistant can certify. Section 1.5 says what was certified, and Chapter 4 follows the whole chain. Which of the series’ truncations describes the measured curve at a given temperature is a different question, about a measured function; it is not a theorem and the proof assistant does not touch it.

1.4 Three instruments, one discipline

Each instrument is blind where another sees (Table 1.1). A measurement says nothing about whether a printed coefficient is algebraically right, and a calorimeter cannot find a misprinted exponent. A proof says nothing about helium: it holds for every model that satisfies its hypotheses, and whether helium does is not a mathematical question. A simulation shows what a model does and can be wrong in ways that conservation laws catch, but it cannot say *why*, and it cannot tell a theorem from a coincidence.

Used together they check one another only if their roles are kept apart, and this book keeps four

rules. A theorem is never presented as a measurement, nor a simulation as one. Every statement says which instrument supports it. An analogy between two systems is not a result: before a finding is carried over we name the property it depends on and check that the other system has it (the programme calls this rule LL-15, after the lesson in which it was learned). And failures are published. Three of them concern this chapter.

The ‘dual length’ $\epsilon^2 / (\hbar^2 c^2 k^3)$ of the Bogoliubov spectrum had a correct Lean proof and was first presented as a finding. A literature check on 2026-09-20 showed it to be a repackaging of known relations whose bound is dimensionally forced and, outside a narrow window of wave numbers, weaker than Onsager’s inequality; it was withdrawn (RETRACTIONS.md, R2). A proof assistant could not have told us that, and a literature search did. Second, the pair speed of Section 1.6 was at first read, in a forked analysis session, as an imprint transient; it is the mean flow that a periodic box forces on a pair (ledger CLAIM-o82). A simulation reported the excess faithfully and could not explain it; the explanation came from the theorem of Section 1.5. Third, a proposal to certify the zero-sound parameters of two-dimensional helium-3 stopped at the literature gate, before any code, because the numbers it needed were not to be found in open sources (ledger CLAIM-o30 and o31); Section 1.7 comes back to it. In each case what mattered was less the instrument than the mechanism that was in place before the claim was tested.

1.5 What the proof assistant says: five statements

Lean 4 is a programming language and a proof assistant: a statement written in it is accepted only if a small kernel has checked a complete proof of it [11]. The library QuantumFluids [12] collects such statements about quantum fluids on top of Mathlib [13]; at the time of writing it indexes 310 theorems. Chapter 2 is the primer. Here are five statements, chosen so that each touches something in this chapter. Each box gives the statement exactly as it stands in the library, or, for the last, in the new file written for this book; the paragraph after it says in words what it asserts and what it does not.

LEAN 4 STATEMENT 1 BOSEINTEGRAL

```
noncomputable def debyeEnergy (V kB hbar c I T : ℝ) : ℝ :=
  V / (2 * π ^ 2) * (kB * T) ^ 4 / (hbar * c) ^ 3 * I

theorem debye_specific_heat (V kB hbar c T : ℝ) :
  HasDerivAt (fun T => debyeEnergy V kB hbar c (π ^ 4 / 15) T)
    (2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3) T
```

Statement 1, the leading term of Eq. (22). With the Bose integral set to $I = \pi^4/15$, the phonon energy of a volume V has the temperature derivative $2\pi^2 k_B^4 V T^3 / (15 c^3 \hbar^3)$: the coefficient A of Eq. (22), exactly. The value of the integral is the theorem QuantumFluids.BoseIntegral.mellin_bose_four, $\int_0^\infty t^3 / (e^t - 1) dt = \pi^4/15$, which Mathlib did not have and which the

library derives from the Mellin transform. The theorem `QuantumFluids.PhononSpecificHeat.phonon_specific_heat` assembles all six terms; the printed coefficients A, C, D, E, K, L and the printed inverse series were derived independently by computer algebra and then proved. The result is a confirmation of the paper, and the programme records it as nothing more: no discrepancy was found. What is not proved is as important: that the coefficients typed into the capstone are the coefficients of the density-of-states polynomial proved in `PhononSeries` is checked by inspection, not by a theorem, and that the truncated series approximates the specific heat computed from the measured curve is a statement about a measured function.

LEAN 4 STATEMENT 2 HELIUMKINEMATICS

```
def phononDisp (c a k : ℝ) : ℝ := c * k * (1 + a * k ^ 2)

theorem three_phonon_open_iff {c a k1 k2 : ℝ} (hc : 0 < c) (h1 : 0 < k1) (h2 :
  ↪ 0 < k2) :
  phononDisp c a k1 + phononDisp c a k2 ≤ phononDisp c a (k1 + k2) ↔ 0 ≤ a
```

Statement 2, when a phonon can decay. A phonon of wave number $k_1 + k_2$ can decay into two collinear phonons of wave numbers k_1 and k_2 , conserving energy and momentum, only if $\varepsilon(k_1) + \varepsilon(k_2) \leq \varepsilon(k_1 + k_2)$. For the model dispersion $\varepsilon = ck(1 + ak^2)$ with $c > 0$ this holds exactly when $a \geq 0$: the dispersion must bend upward. It is one of the kinematic statements in the analysis of the neutron data of the 2021 paper that the module `HeliumKinematics` formalizes, and the proof is of the mathematical statement, not of the experiment. Figure 1.2(b) is where it meets the measurement: the table’s phase velocity exceeds c at small Q , and the symmetric split $\varepsilon(k) \geq 2\varepsilon(k/2)$ holds on the table for $0.10 \leq k < 0.45 \text{ \AA}^{-1}$. The theorem says nothing about the table beyond the cubic model, and nothing about decay rates.

LEAN 4 STATEMENT 3 ZEROSOUND

```
theorem zero_sound_2d_iff (F : ℝ) :
  (∃ s, 1 < s ∧ 1 + F * (1 - s / √(s ^ 2 - 1)) = 0) ↔ 0 < F
```

Statement 3, zero sound in two dimensions. Landau’s collisionless kinetic equation for a Fermi liquid with a single Landau parameter F has, in two dimensions, a collective mode with phase velocity $s v_F$ when $1 + F(1 - s/\sqrt{s^2 - 1}) = 0$. The mode is undamped when $s > 1$, outside the particle–hole continuum. The theorem says that such a root exists if and only if $F > 0$; the proof exhibits it, $s = (1 + F)/\sqrt{1 + 2F}$. This is textbook physics [14]; what is new is only that it is machine-checked. The two-dimensional case is in the library *because* the 2012 measurement is on a two-dimensional film, where the logarithmic three-dimensional formula does not apply. It is not a statement about that film: nothing is claimed about the value of F for it, about the roton-like minimum, or about the damped branch $s < 1$ (Chapter 8).

LEAN 4 STATEMENT 4 VORTEXWINDING

```

noncomputable def pdiff (a b : ℝ) : ℝ := toIocMod two_pi_pos (-π) (b - a)

theorem loop_sum_eq_mul (θ : ℕ → ℝ) (n : ℕ) (hclosed : θ n = θ 0) :
  ∃ m : ℤ, ∑ i ∈ Finset.range n, pdiff (θ i) (θ (i + 1)) = (m : ℝ) * (2 * π) := by
  refine {·} ∑ i ∈ Finset.range n, toIocDiv two_pi_pos (-π) (θ (i + 1) - θ i), ?_
  have h : ∀ i, pdiff (θ i) (θ (i + 1))
    = (θ (i + 1) - θ i)
      - ((toIocDiv two_pi_pos (-π) (θ (i + 1) - θ i) : ℤ) : ℝ) * (2 * π) := by
    intro i; rw [pdiff_eq, zsmul_eq_mul]
  simp_rw [h]
  rw [Finset.sum_sub_distrib, Finset.sum_range_sub (fun i => θ i), hclosed, ← Finset.
    ↪ sum_mul]
  push_cast
  ring

```

Statement 4, why a computed winding number is an integer. Here θ_i are the phases of a field at the points of a closed loop, and pdiff is the principal value in $(-\pi, \pi]$ of a phase step. The theorem says that the sum of the principal steps around a closed loop is an integer multiple of 2π . The proof, shown in full, is one idea: each principal step differs from the plain difference of the phases by a multiple of 2π (`QuantumFluids.VortexWinding.pdiff_eq`), and the plain differences telescope to $\theta_n - \theta_0 = 0$ because the loop is closed. This is why a numerically computed winding number is an integer and not a decimal: the standard phase-winding vortex detector of Gross–Pitaevskii codes relies on it. Multiplied by $\hbar/m$ it gives `QuantumFluids.Circulation.circulation_quantized`: the circulation around a closed loop of sample points is an integer multiple of the quantum $\kappa = \hbar/m$. The same module states exactly when the standard algorithm fails: if a phase step is exactly π , the two orientations of an edge contribute 2π instead of cancelling (`QuantumFluids.VortexWinding.pdiff_add_rev_eq_two_pi_iff`). Not proved: that the integer is the topological charge of the continuum field the samples came from; that is a statement about sampling, and the library’s `ContinuumWinding` module addresses it for continuous loops.

LEAN 4 STATEMENT 5 NEUTRONWINDOW (NEW IN THIS BOOK)

```

variable {V : Type*} [NormedAddCommGroup V]

noncomputable def energy (hbar m : ℝ) (k : V) : ℝ := hbar ^ 2 * ||k|| ^ 2 / (2 * m)

theorem min_incident_wavevector (hbar m : ℝ) (hbar0 : 0 < hbar) (hm : 0 < m) (ki kf : V
  ↪ ) (hQ : 0 < ||ki - kf||) :
  ||ki - kf|| / 2 + m * (energy hbar m ki - energy hbar m kf) / (hbar ^ 2 * ||ki - kf||)
  ↪ ≤ ||ki||

```

Statement 5, what a neutron can reach. This is inequality (1.1). For a neutron that leaves a sample with wave vector $\mathbf{k}_f$ after arriving with $\mathbf{k}_i$, and has given the sample the energy $E_i - E_f$ at the

wave-vector transfer $Q = \|\mathbf{k}_i - \mathbf{k}_f\| > 0$, the incident wave number is at least $Q/2 + m(E_i - E_f)/\hbar^2 Q$. It was written for this book, in the file `book/lean/Ch01_NeutronWindow.lean`, together with the two lemmas it rests on, the triangle inequality for $\mathbf{Q}$ and the parabola bound on the energy transfer; it compiles against the pinned Mathlib with no error and no `sorry`, its three theorems depend only on the standard axioms `propext`, `Classical.choice` and `Quot.sound`, and two altered statements (the bound raised by $1/10$, or the recoil term $Q/2$ replaced by Q) are rejected by the same proof, as they must be. The proof is the reverse triangle inequality and a line of algebra. It is the *necessary* direction only: that every point under the parabola is reachable needs a triangle with three prescribed sides, and that collinear scattering, forward or backward, attains the bound so that it cannot be improved; both are elementary and neither is formalized here. Nothing in it is a statement about an instrument, and nothing is a statement about helium; it is the geometry of a collision, and it is what draws the colour of Figure 1.2(a).

1.6 What the solver shows: one pair, four checks

The simplest object that is topological in the sense of statement 4 is a vortex and an antivortex in a two-dimensional superfluid; the cover of this book shows sixteen vortices computed with the same engine. In a model of a Bose condensate by a complex field $\psi(\mathbf{r}, t)$ the velocity of the fluid is the gradient of the phase, $\mathbf{v} = (\hbar/m)\nabla \arg \psi$, so that the circulation around a closed loop is $(\hbar/m)$ times the total change of phase, which must be a multiple of 2π : statement 4, again. The vortex is a zero of ψ around which the phase winds by $\pm 2\pi$ [15–18]. Chapter 3 develops the theory; here we only use the object to show the solver at work, and to compare what it reports with what the proof says.

The model is the projected Gross–Pitaevskii equation on a periodic square [19, 20],

$$i \partial_t \psi = \mathcal{P} \left[-\frac{1}{2} \nabla^2 \psi + |\psi|^2 \psi \right],$$

with $\mathcal{P}$ the projector on the Fourier modes below a cutoff, in units $\hbar = m = g = 1$ and with background density 1, so that the speed of sound $c = 1$ and the healing length $\xi = 1$ are the units of speed and length, and ξ/c the unit of time. The engine is the crate `qf-pgpe` of rusty-SUNDIALS and its Python extension `qf_pgpe` [21]; it advances the field by an integrating-factor fourth-order Runge–Kutta scheme, the linear part exactly and the cubic term by RK4, and, as the crate’s README reports, its tests integrate the same right-hand side with the CVODE solver as an independent reference (Chapter 2 introduces the solver itself; CVODE is not used in the computations of this chapter). The box has side $L = 64\xi$ and 128^2 points. At $t = 0$ a vortex of charge +1 is imprinted at (26, 32) and an antivortex at (38, 32), with the periodic imprint of the engine.

RUSTY-SUNDIALS A VORTEX PAIR IN TEN TIME UNITS

```

import numpy as np, qf_pgpe

s = qf_pgpe.Pgpe(128, 64.0)      # 128^2 points, box 64 xi; g = 1
pos0 = np.array([[26., 32.], [38., 32.]]) # vortex, antivortex
c0 = s.imprint_v2(s.uniform(), pos0, np.array([1, -1]))
c = s.run(c0, 10.0)              # 1000 RK4 steps in one call
for name, f in (("energy", s.energy), ("norm", s.norm)):
    print(f"{name:6s} t=0 {f(c0):.9f} t=10 {f(c):.9f} "
          f"relative drift {abs(f(c) - f(c0)) / f(c0):.1e}")
pos, q = s.detect(c)
print("vortices at t=10:", pos.round(3).tolist(), q.tolist())

def winding(psi, loop):          # loop: grid points, first = last
    th = np.angle([psi[i, j] for i, j in loop])
    d = np.diff(th)
    d -= 2 * np.pi * np.ceil((d - np.pi) / (2 * np.pi)) # pdiff
    return d.sum() / (2 * np.pi), np.abs(d).max()

def ring(i0, j0, h):            # ccw square, half-side h
    up = [(i0 + h, j0 + t) for t in range(-h, h)]
    left = [(i0 + h - t, j0 + h) for t in range(2 * h)]
    down = [(i0 - h, j0 + h - t) for t in range(2 * h)]
    right = [(i0 - h + t, j0 - h) for t in range(2 * h)]
    return up + left + down + right + [(i0 + h, j0 - h)]

psi = np.asarray(s.psi(c))
cases = {"around the vortex": pos[0],
        "around the antivortex": pos[1],
        "empty fluid": (47., 40.)}
for name, centre in cases.items():
    i0, j0 = (int(round(x / 0.5)) for x in centre)
    w, step = winding(psi, ring(i0, j0, 7))
    print(f"{name:22s} winding {w:+.15f} max step {step:.3f}")

```

Output of this box, run as it stands (wall-clock time 9.4 s, of which 5.0 s of CPU, on a shared 8-core machine with a load average of 18.5; phase steps in radians):

```

energy t=0 2069.113131399 t=10 2069.113131436 relative drift 1.8e-11
norm t=0 4096.000000000 t=10 4096.000000033 relative drift 8.0e-12
vortices at t=10: [[26.123, 32.902], [37.877, 32.902]] [1, -1]
around the vortex winding +1.000000000000000 max step 0.216
around the antivortex winding -1.000000000000000 max step 0.216
empty fluid winding +0.000000000000000 max step 0.030

```

Figure 1.3 shows the field at $t = 10$ three ways.

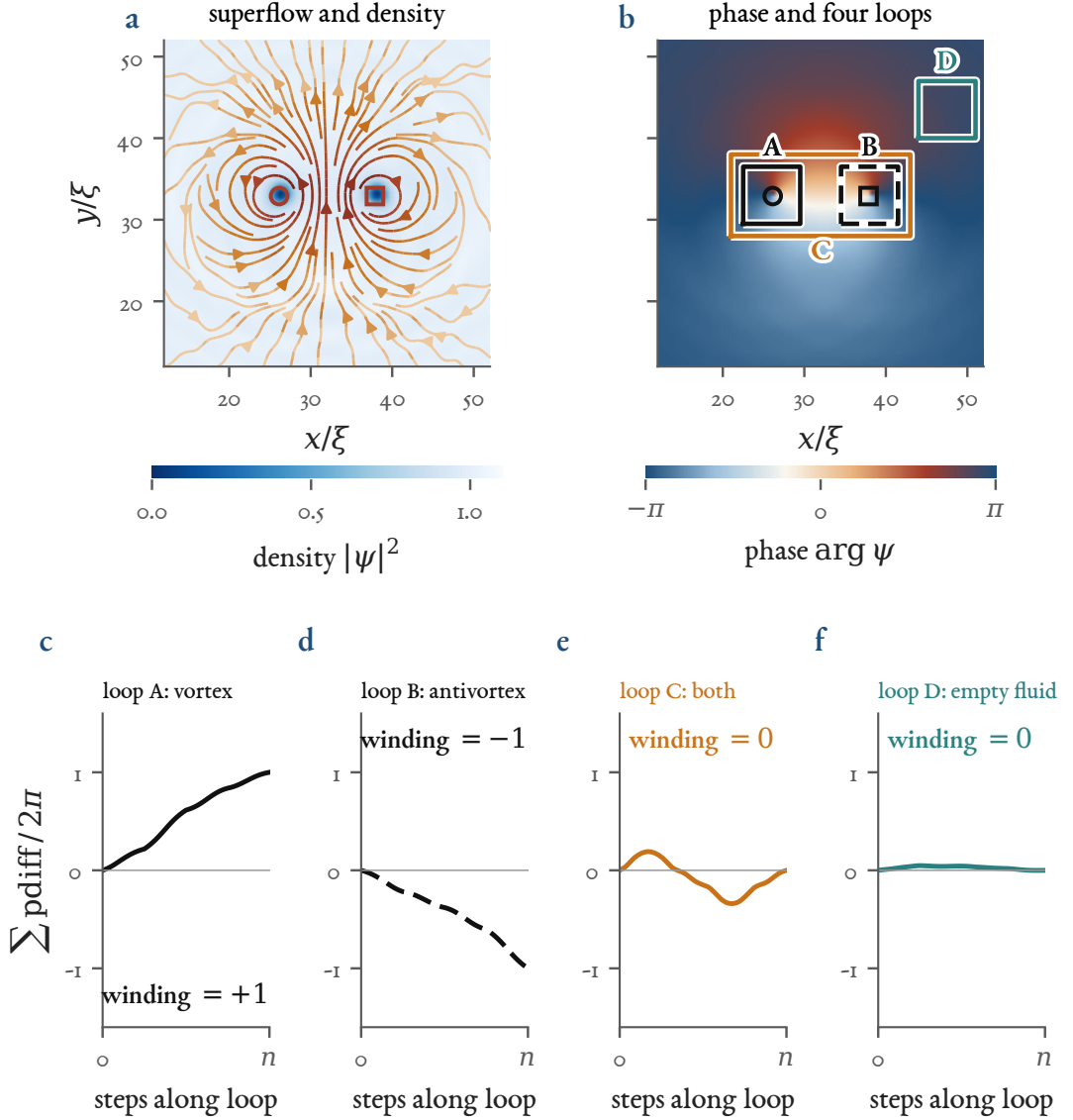


Figure 1.3: One vortex pair seen by the solver and the proof. **(a)** Density (colour) and superflow (streamlines, darker = faster) of the field at $t = 10$; the cores of the vortex (circle) and of the antivortex (square) are the two dark dots, 11.75ξ apart. **(b)** Phase $\arg \psi$ and four closed lattice loops: A around the vortex, B around the antivortex, C around both, D in empty fluid. **(c–f)** Running sum of the principal phase differences (pdiff) along each loop, in units of 2π : it ends at +1, -1, 0, 0. Engine: qf_pgpe, 128^2 points, $L = 64 \xi$, $t = 10$. Script: figures/ch01_triptych.py.

Check 1: the proof meets the field. The sums of principal phase differences around the loops A, B, C and D, in units of 2π , come out as $+1$, -1 , 0 and 0 , integers to within 1.4×10^{-16} (the double-precision rounding of the sums; the box above prints them to fifteen decimals), as statement 4 says they must. The theorem fails only if a phase step reaches exactly π ; the largest step along any of the loops of the figure is 0.216 rad, 0.07π .

Check 2: the continuum. The integer of the proof is the sum of *sampled* phases. The circulation of the velocity field itself is another thing, and the one that physics is about. Integrated along the same four loops by Gauss–Legendre quadrature on the exact band-limited field of the engine, $\oint \mathbf{v} \cdot d\mathbf{l}/\kappa$ is $+1.000000000000$, -1.000000000000 , -5.30×10^{-17} and -1.44×10^{-17} , with $\kappa = 2\pi\hbar/m$. (A trapezoid rule on the grid values gives 0.9966 , 0.9977 , 0.9987 and 0.9994 for loops around the vortex of half-side 2 , 2.5 , 3.5 and 4.5ζ , the deficit shrinking as the loop grows, as a quadrature error of the grid should; the exact integral does not depend on the loop.) In this instance the sampled loop sum is the continuum circulation: the statement that `QuantizedCirculation` leaves to sampling holds here, to rounding error.

Check 3: the conservation laws. Over the ten time units of the box above the relative drift of the energy is 1.8×10^{-11} , of the norm 8.0×10^{-12} and of the momentum 2.0×10^{-11} ; over the sixty time units of the next check, 8.6×10^{-11} , 4.3×10^{-11} and 5.5×10^{-11} . They are the first numbers to look at when one asks whether the engine integrates the model it claims to integrate; they are necessary checks, not sufficient ones (the order of the scheme is tested against CVODE in the crate).

Check 4: the speed of the pair, and what the proof explains. A pair of opposite vortices at a distance d in the plane moves at $1/d$ (in these units) perpendicular to the line joining them. In a periodic box the periodic images of the two vortices lower this speed, and the standard formula for point vortices on a torus, the Weiss–McWilliams sum [22], includes them. We tracked the cores for 60 time units, each time unit, and fitted the displacement of their midpoint on $t \geq 20$, after the imprint has relaxed. The measured speed is 0.0932 (fits that start at $t = 10$, 20 and 30 give 0.0936 , 0.0932 and 0.0928 , a half-range of 0.4 per cent, which we take as the uncertainty), against 0.0850 for the planar formula at the mean separation $d = 11.76\zeta$ (standard deviation over the window 0.02) and 0.0757 for the torus formula: 23 per cent more than the point-vortex prediction.

The missing piece is not in the point-vortex formula. A pair carries the momentum $P \approx 2\pi d$ (per unit density, in these units), and in a periodic box that momentum cannot go anywhere: the whole fluid moves, with the mean velocity $u = P/N$ (N the norm), which the formula for point vortices, built on a Green function of zero mean, does not contain. Adding the measured $u = 0.0177$ gives 0.0934 , within 0.3 per cent of what the solver does, which is inside the spread of the fits (Figure 1.4).

The proof says that this mean flow is not an accident of the imprint. The phase of ψ must be single-valued, so by statement 4 the circulation around each of the two non-contractible cycles of the torus is 2π times an integer. Around a vertical cycle at a position x between the vortices the

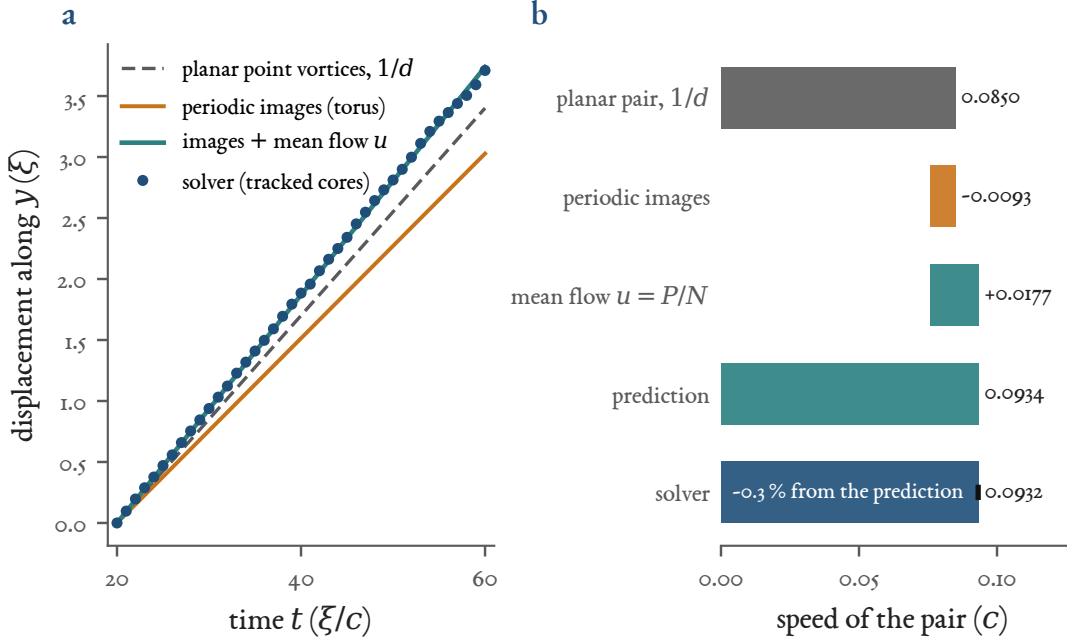


Figure 1.4: Where the speed of the pair comes from. **(a)** Displacement of the midpoint of the two cores along y from $t = 20$, tracked each time unit in the solver (dots), against the planar formula $1/d$, the formula for point vortices on a torus (periodic images), and the torus formula plus the mean flow $u = P/N$ of the whole fluid. **(b)** The same as a budget of the speed in units of c : planar pair, correction of the periodic images, mean flow, the resulting prediction, and the solver. Script: `figures/ch01_budget.py`.

integer differs by one from its value outside, because the circulation around a single core is 2π ; so the average of v_y over the box is $(2\pi/L^2)(n_{\text{in}}d + n_{\text{out}}(L - d))$ with $n_{\text{in}} - n_{\text{out}} = \pm 1$, and its smallest magnitude is $2\pi d/L^2$. The mean of the superfluid velocity of a neutral pair on a torus cannot be made zero. We checked the integers on the field at $t = 10$, by exact line integrals around the cycles, in units of 2π : +1 along the vertical cycle between the cores, 0 and 0 along the two vertical cycles outside them, 0 and 0 along the two horizontal ones, each within 5.2×10^{-14} of the integer. They give $u = 2\pi 11.75/L^2 = 0.0180$, while the momentum of the field gives 0.0177, the difference, 1.6 per cent, being the average of $(1 - n)v_y$ over the box: the density n is not uniform (it falls to 0.016 at the cores and carries sound waves), so the density-weighted and the plain mean velocities differ. The programme's own $T = 0$ sweep over pair sizes shows the same thing at every size [23]: the ratio of the measured speed to the torus formula grows in the box frame from 1.06 at $d = 6\xi$ to 1.49 at $d = 16\xi$ (the paper's table; Chapter 7 compares it with the saved file of the sweep), and is 1.004 to 1.005 for $d = 8$ to 16ξ once the mean flow is subtracted. That subtraction is the correction of the early reading of the excess as an imprint transient, mentioned in Section 1.4 (ledger CLAIM-082).

A last honest line. The measured speed is -0.3 per cent from the prediction, which is inside the spread of the fits: no residual is resolved. A pair of finite size is a solitary wave of a compressible field

(the Jones–Roberts family [24]) and its speed departs from the point-vortex value by a correction of the order of $(\zeta/d)^2$; the programme measured that correction at $T = 0$, with longer runs, as +0.4 per cent for $d \geq 8\zeta$ and larger for tighter pairs, which is as large as our uncertainty, so we neither see it nor exclude it. The run took 181 s of wall-clock time on a shared 8-core machine whose load average was 20.2 at its end; the figure is not a benchmark.

1.7 What the measurements hand to theory

The two measurements hand three things to anyone who works on quantum fluids.

A curve. The dispersion relation of helium-4 from phonon to roton is a table, and a table can be put in front of any theory of the liquid. This book does not compute helium. What the library does is to make explicit which mathematical statements stand between that table and the thermodynamic functions: the Bose integral and the zeta values, the inversion of the dispersion series, the density of states, the kinematics of decay and of the collision itself. Each of them is a place where a printed formula could be wrong without any measurement being able to say which step was at fault.

A chain, and the confirmation of one of its links. Landau’s chain from the curve to the calorimeter has a six-term analytic form in the paper, and the form has been derived a second time, independently, and proved. A confirmation is a small thing and the programme records it as small. It is, however, the kind of thing an experimentalist can use: a series that enters an analysis is now one whose algebra is certified, so that a disagreement with a calorimeter, should one appear, is a disagreement about the model and not about the arithmetic.

A film, and a question. The two-dimensional helium-3 film has a collective mode that lives long and has a roton-like minimum. Statement 3 of Section 1.5 is the part of the story that is a theorem: in Landau’s single-parameter description an undamped zero sound exists in two dimensions if and only if $F > 0$. To use it on the film one needs the Landau parameters of the film, and the programme did not find an open numerical table of them; it stopped twice, at the literature gate, rather than put a number into the theorem that it could not source. That is not a result about the film, and the roton-like minimum, which lies at a wave number where a single Landau parameter is not the right tool, is outside what the theorem says. It is a map of what is missing.

How can a companion made of proofs and simulations honour a measurement without speaking for the people who made it? We have tried to follow five rules, and they are offered as a standard against which the book can be judged.

- *Say precisely which statement is used.* Where the library formalizes a statement of a paper, the book names the equation and says whether what was proved is the mathematics of the statement or something about the experiment. It is always the former.
- *Prove mathematics, not physics.* A theorem of this book is about a model, or about arithmetic, or about the geometry of a collision. It is never a statement about helium.
- *Leave the measurement its authority.* The only measured curve in this chapter is the authors’ table. Our additions to it are kinematics and arithmetic, and they are labelled as such in the figure.

- *Be useful to the reader of the paper.* The right test of a companion is whether an experimentalist who reads the paper would rather have it than not: a certified series, a stated failure condition of a standard algorithm, a picture of what a neutron can reach.
- *Put the failures next to the successes.* A tribute that hid its retractions would be a poor one to a craft whose first rule is to report the error bar.

The book makes no claim about the data behind the two papers beyond the table described above, and none about the published version of the 2021 paper beyond its Eq. (22). It has not been reviewed by the people whose measurements it describes.

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved (Lean 4.34.0-rc2, Mathlib v4.34.0-rc2, standard axioms): the five statements of Section 1.5; the first four belong to the library `QuantumFluids`, the fifth is new and lives in `book/lean/Ch01_NeutronWindow.lean`. They are theorems about a Debye-type energy, about a cubic model dispersion, about Landau's single-parameter kinetic equation, about sums of principal phase differences and about the triangle of a collision; none is a statement about helium or about an instrument. What is not proved: the link between the density-of-states polynomial and the coefficients typed into `QuantumFluids.PhononSpecificHeat.phonon_specific_heat` (by inspection), the validity of the series for the measured curve, the reachability of points under the neutron parabola, and that a sampled winding is the continuum charge in general.

Measured: nothing new. The curve of Figure 1.2 is the authors' processed table; the speed of sound $c \approx 239$ m/s is our extrapolation of it, and v_L , the phonon excess and the kinematic limits are arithmetic on it. They are not the settings of any experiment.

Simulated: one vortex pair in a periodic box of a classical-field model. It reproduces the integer windings, the quantum of circulation and the conservation laws, and moves at the speed predicted from the torus formula plus the mean flow to 0.3 per cent; the residual is not decomposed.

Only an analogy: that the pair of a Bose-condensate model says anything about a vortex in helium-4 or in a Fermi liquid. The model has no roton and no Fermi statistics. The mean flow of a periodic box is a property of the box, not of helium. The theorem on zero sound is not applied to the 2012 film.

Failed or corrected (programme record): the 'dual length' claim, withdrawn 2026-09-20; the early reading of the pair-speed excess as an imprint transient, corrected as the mean flow (CLAIM-082); the zero-sound certification proposal, stopped at the literature gate (CLAIM-030, 031).

Exercises

Exercise 1. The roton of Figure 1.2 has $Q_0 = 1.92 \text{ \AA}^{-1}$ and $\varepsilon_0 = 0.741 \text{ meV}$. With $\hbar^2/2m_n = 2.072 \text{ meV \AA}^2$, use (1.1) to find the longest wavelength and the least energy of a neutron that can create it. Then find the longest wavelength that reaches $Q = 3 \text{ \AA}^{-1}$, where $\varepsilon = 1.478 \text{ meV}$, and explain why the answer for $Q \rightarrow 0$ does not depend on Q . *Hint:* multiply $k_i \geq Q/2 + m_n \varepsilon / \hbar^2 Q$ by $\hbar/m_n$; for the energy, $E_i = (\hbar^2/2m_n)k_i^2$; as $Q \rightarrow 0$ the first term of (1.1) vanishes.

Exercise 2. Take the closed loop of four phases $\theta = (0, \pi, 0, \pi)$ (so that $\theta_4 = \theta_0 = 0$). Compute the sum of the principal differences. It is an integer multiple of 2π , as the theorem `QuantumFluids.VortexWinding.loop_sum_eq_mul` requires; which integer? Is it the winding of the smooth loop that the four samples were taken from? Which statement of the same module explains what has happened? *Hint:* $\text{pdiff}(0, \pi) = \pi$ and $\text{pdiff}(\pi, 0) = \pi$, because the principal interval is $(-\pi, \pi]$; a sampled loop cannot tell a wind of 4π from an oscillation, and the failure condition is exactly a step of π .

Exercise 3. A neutral vortex pair of separation $d < L/2$ sits in a periodic square of side L (units $\hbar/m = 1$, density 1). Show that the average of v_y over the box is $(2\pi/L^2)(n_{\text{in}}d + n_{\text{out}}(L - d))$, where the integers n_{in} and n_{out} are the windings of the phase along vertical cycles inside and outside the pair and $n_{\text{in}} - n_{\text{out}} = \pm 1$, and that its least magnitude is $2\pi d/L^2$. Evaluate it for $L = 64$, $d = 11.75$, and say why the speed of the pair in the box differs from the torus point-vortex speed by about that amount. *Hint:* $\oint v_y dy = 2\pi n(x)$ at fixed x by `QuantumFluids.VortexWinding.loop_sum_eq_mul`; integrate over x ; the choice $n_{\text{out}} = 0$ gives the least magnitude since $L - d > d$.

Notes and further reading

The two measurements are [1] and [2]; for the excitation spectra of quantum fluids in general see the review [5]. Landau's own papers are [3] for helium-4 and [25] for the Fermi liquid; Baym and Pethick [14] treat the latter, Pitaevskii and Stringari [26] the Bose condensate, and Donnelly [10] the quantized vortices of helium II. For neutron scattering, Squires [7] is the standard introduction and Glyde [8] treats helium specifically. SUNDIALS is described in [4], Lean in [11], Mathlib in [13]; the library and the solver of this book are [12] and [21], and the transport of vortices in the same engine is the subject of [23]. The periodic point-vortex velocity used in Section 1.6 is that of Weiss and McWilliams [22], and the compressible solitary pair is due to Jones and Roberts [24]. The reference run of Kwon and Shin [27, 28], which the same engine reproduces, is the subject of Chapter 9. The records of what failed are in the programme's `RETRACTIONS.md`, `LEDGER.md` and `LL.md`.

Two Languages: Lean 4 and rusty-SUNDIALS

2.1 Two ways to be wrong

The first run of the registered controls of the programme’s solver for the projected Gross–Pitaevskii equation (PGPE), recorded in an amendment dated 22 September 2026, reported that the total momentum of an isolated, periodic, two-dimensional fluid had changed by 8.54 in 20 time units, one per cent of its value, and had done so identically at two different time steps. Momentum cannot change in such a system, since translating the periodic box leaves the equation unchanged; and a drift that does not depend on the step is not a fault of the time stepping but of the equation being stepped. The same run, which tests the norm (control K1), the energy (K2), the momentum (K3) and an exact plane-wave solution (K4), had found the norm conserved to 4.2×10^{-10} and the plane wave reproduced to 7.0×10^{-10} : those two controls passed, and neither could have seen the fault. It sat in a sentence of the programme’s own pre-registration, which said that a cutoff at two thirds of the largest wave number makes the cubic term of the equation free of aliasing. That is the rule for a quadratic nonlinearity [29]; for a cubic term the cutoff has to sit at one half. The aliased scheme is not translation invariant, and the momentum control saw it.¹

A different kind of fault is the subject of a remark in the 2021 paper by Godfrin and his co-authors on the dispersion relation of superfluid helium-4 [2]. Equation (22) of that paper gives the phonon specific heat as a series, $C_V = AT^3 + CT^5 + DT^6 + ET^7 + KT^8 + LT^9$, and the authors note that earlier published versions of the series contain errors. The neutron measurement fixes the dispersion relation; the step from the dispersion to the series is algebra, and a formula of that length is exactly where a coefficient or a sign can be wrong without anyone noticing.

GODFRIN A PRINTED SERIES, AND A REMARK ABOUT IT

The inelastic neutron-scattering measurement of Godfrin, Beauvois, Sultan, Krotscheck, Dawidowski, Fåk and Ollivier on superfluid ^4He (spectrometer IN5, Institut Laue–Langevin; Phys.

¹Amendment A1 of the programme’s pre-registration (docs/designs/PGPE_BKT_PREREG.md, 2026-09-22) and the stored first run data/generated/pgpe/known_answers.json: $N = 64$, $L = 32$, a random state of energy 3 per particle, $t = 20$, momentum drift 8.5358 at $\Delta t = 0.005$ and 8.5358 at 0.01. The numbers of this paragraph are quoted from those files, not recomputed.

Rev. B 103, 104516 (2021)) gives the Landau phonon–maxon–roton dispersion (the authors’ published table runs from 0 to $3.6 \text{ \AA}^{-1}$) and the thermodynamic consequences. Its Eq. (22) is the series above, for a dispersion $\varepsilon = \hbar ck (1 + \alpha_2 k^2 + \dots + \alpha_6 k^6)$ with $\alpha_1 = 0$, and the paper remarks that earlier versions of this series contain errors. The programme re-derived the series independently and proved all six coefficients in Lean. That is a *confirmation of the printed formula*, not a finding, and a proof of a mathematical statement, not of the experiment; Chapter 4 tells it.

The two faults call for two different instruments, and this chapter is a primer on both. The second fault, an algebraic slip, is the business of a proof assistant: a computer checks the derivation, and a wrong coefficient cannot pass. The first fault is not. The programme had a Lean theorem about the cubic term, and the theorem was true; it was about the *exact* Galerkin convolution on an additive group such as the integer lattice, while the program evaluated a different function, a fast Fourier transform on a periodic grid, which coincides with the convolution only when no triple of wave vectors wraps around the grid. A proof checks a statement against its hypotheses and a simulation checks a program against an expectation; neither checks that the program computes the object the statement is about.

THE DIVISION OF LABOUR

A theorem is about a *function*; a run is of a *program*. Lean can establish that the function has a property for every state and every parameter. The solver can show that the program reproduces an exact solution, an independent integrator or a conservation law to a stated precision, in a stated case. Each instrument is blind where the other sees, and the instructive cases are those in which they disagree.

We follow one equation through both languages. Section 2.2 is Lean in twenty minutes; Section 2.3 states the theorems that concern the equation; Section 2.4 is the solver in twenty minutes; Sections 2.5 to 2.7 are three experiments, the last of which is the fault above, reproduced and measured; Section 2.8 sorts out which instrument can see what.

2.2 Lean 4 in twenty minutes

Lean 4 [11] is at once a programming language and a proof assistant. In it a statement is a *type*, a proof of the statement is a *term* of that type, and a small program, the *kernel*, checks that the term has the type it is claimed to have. The kernel, together with the axioms it is told to assume, is what a reader has to trust; the elaborator and the tactics that build terms are not trusted, because whatever they produce is re-checked. Mathlib [13] is the community library of mathematics on which the programme builds. The programme’s own library, QuantumFluids, adds 37 modules with 310 theorems and lemmas (413 declarations in all) on the mathematics of quantum fluids, compiled

with Lean 4.34.0-rc2 against the matching Mathlib; its sources are in `lean_src/`. Every statement quoted from it in this book is copied from there and cited by module and name. The Lean written for this chapter is marked *new* in the title of its box, lives in `book/lean/`, and was compiled with zero errors and no sorry; the `#print axioms` lines quoted below are those of the compiler.

The box below contains the whole vocabulary needed to read a Lean statement.

LEAN 4 NEW: CH02_PRIMER

```
#check (2 : ℕ)           -- 2 : ℕ
#check (2 + 2 = 4)       -- 2 + 2 = 4 : Prop
example : 2 + 2 = 4 := rfl -- a proof is a term of that type

theorem and_swap' (p q : Prop) (hp : p) (hq : q) : q ∧ p :=
  (hq, hp)

theorem two_add_two : (2 : ℝ) + 2 = 4 := by norm_num

theorem one_add_nilpotent_inv {R : Type*} [CommRing R]
  (e : R) (h : e ^ 2 = 0) : (1 + e) * (1 - e) = 1 := by
  linear_combination (-1 : R) * h
```

The first two lines ask Lean for the type of an expression: 2 is a natural number, and $2 + 2 = 4$ is a proposition, an element of the type `Prop`. The line `example : 2 + 2 = 4 := rfl` proves it: both sides reduce to the same numeral, so reflexivity of equality is a proof. The theorem `and_swap'` says that if two propositions hold then they hold in the other order; its proof is the pair `(hq, hp)`, a term written by hand. The theorem `two_add_two` is about real numbers, and its proof is a *tactic*: `norm_num` searches for a proof and hands the result to the kernel.

The last theorem is the pattern the library uses at scale. In any commutative ring R with an element e satisfying $e^2 = 0$, the element $1 + e$ has the inverse $1 - e$. The tactic `linear_combination (-1 : R) * h` is a *certificate*: it asserts that the goal, minus -1 times the hypothesis h , is an identity of commutative rings, and `ring` checks that identity. The certificate was supplied from outside and the kernel checks it; nothing about how it was found has to be believed. The module `PhononSeries` uses exactly this pattern for the series of the previous section: its certificates, computed by computer algebra, run to several thousand characters per line (see `QuantumFluids.PhononSeries.kPow2_eq`) and are checked by the kernel like the toy above. Truncating a power series at order e^8 becomes the statement $e^8 = 0$ in a commutative ring, with no analysis needed.

What the kernel refuses. A checker is only worth having if it can say no. With the coefficient of the certificate mistyped (`(1 : R)` instead of `(-1 : R)`), the same tactic refuses; and so does `norm_num` on a false numerical statement. The compiler's reply to the file `book/lean/Ch02_NegativeControl.lean`, which exists only to be refused (file path shortened, and the hygiene mark that Lean prints after `inst` omitted):

```

Ch02_NegativeControl.lean:9:2: error: ring failed, ring expressions not equal
R : Type u_1
inst : CommRing R
e : R
h : e ^ 2 = 0
⊢ -(e ^ 2 * 2) = 0
Ch02_NegativeControl.lean:12:29: error: unsolved goals
⊢ False

```

The residual $-2e^2$ is the evidence: with the wrong sign the goal minus the certificate is not zero as a ring identity (the goal needed $-e^2$ and the certificate supplied $+e^2$), and no proof is produced. The second error is `norm_num` reducing $2 + 2 = 5$ to `False`.

What an axiom audit looks like. A Lean file can end with a line `#print axioms` for each theorem, which lists the axioms the proof ultimately depends on; the library files carry such lines (written by a script), and Appendix A tabulates the audit of every module. The file of the box above prints

```

'QuantumFluids.Ch02Primer.and_swap'' does not depend on any axioms
'QuantumFluids.Ch02Primer.two_add_two' depends on axioms: [propext, Classical.choice
, Quot.sound]
'QuantumFluids.Ch02Primer.one_add_nilpotent_inv' depends on axioms: [propext, Quot.
sound]

```

The three names are the standard foundation of classical mathematics in Lean and Mathlib (propositional extensionality, the axiom of choice, soundness of quotients); `and_swap'` needs no axiom at all, and the certificate needs two. An incomplete proof shows up as a fourth name, `sorryAx`: when the first draft of this chapter's Lean file contained a proof that did not compile, the compiler reported the error and replaced the proof by `sorry`, and the audit lines of the two theorems that depended on it read `[propext, sorryAx, Classical.choice, Quot.sound]`. That line is the reason the audit exists.

A program is not a theorem. Lean also runs programs. The three commands

```

#eval (List.range 6).map (fun n => n ^ 2)
#eval (0.1 + 0.2 : Float)
#eval ((0.1 + 0.2 : Float) == 0.3)

```

print, in that order,

```

[0, 1, 4, 9, 16, 25]
0.300000
false

```

`Float` is the machine's double-precision number, the same 64-bit format as `float64` in numpy and `f64` in Rust, and in it the sum of 0.1 and 0.2 is not 0.3. In the same file the theorem $(1 / 10 + 2 / 10 : \mathbb{R}) = 3 / 10$ is proved by `norm_num`, because in the field of real numbers the equation is

exact. A theorem about $\mathbb{R}$ is a statement about mathematics; what a program returns is a statement about a finite machine. Nothing in the library speaks about the rounding of any program, and that is why the solver of this book is checked by experiment and the equation by proof.

2.3 A theorem about the equation the solver integrates

The projected Gross–Pitaevskii equation [19, 20] is the classical-field description of a Bose fluid used throughout the programme. In units $\hbar = m = 1$, on a periodic box of side L ,

$$i \partial_t \psi = P \left[-\frac{1}{2} \nabla^2 \psi + g |\psi|^2 \psi \right], \quad (2.1)$$

where P keeps the Fourier modes with $|k| \leq k_{\text{cut}}$ and discards the others. Writing $\psi(x, t) = \sum_{k \in \Lambda} a_k(t) e^{ik \cdot x}$ with $k = 2\pi n/L$, $n \in \mathbb{Z}^2$, and Λ the finite set of retained modes, the equation becomes a system of ordinary differential equations,

$$i \dot{a}_k = \omega_k a_k + g N_k(a), \quad \omega_k = \frac{1}{2} |k|^2, \quad N_k(a) = \sum_{\substack{k_1, k_2, k_3 \in \Lambda \\ k_1 + k_3 = k + k_2}} a_{k_1} \overline{a_{k_2}} a_{k_3}, \quad (2.2)$$

for $k \in \Lambda$. This is the system that the Lean module `GPgalerkin` is about, with an arbitrary additive group in place of $\mathbb{Z}^2$ and an arbitrary finite set Λ (nothing in what follows depends on which modes are retained), and with the amplitudes called ψ_k and the integer vectors n as the elements of the group (the factor $2\pi/L$ sits in ω). The box below quotes its definition of N and two of its theorems.

LEAN 4 GPgALERKIN

```
-- variable {G : Type*} [AddCommGroup G] [DecidableEq G]

noncomputable def nL (Λ : Finset G) (ψ : G → ℂ) (k : G) : ℂ :=
  ∑ k1 ∈ Λ, ∑ k2 ∈ Λ, ∑ k3 ∈ Λ,
    if k1 + k3 = k + k2 then ψ k1 * (starRingEnd ℂ) (ψ k2) * ψ k3 else 0

theorem pairing_eq_sum (Λ : Finset G) (ψ : G → ℂ) :
  pairing Λ ψ =
    ∑ q ∈ diffs Λ, dens Λ ψ q * (starRingEnd ℂ) (dens Λ ψ q) := by
-- (proof abridged) rearrangement of finite sums

theorem mass_rate_zero (Λ : Finset G) (ψ : G → ℂ) (ω : G → ℝ)
  (g : ℝ) :
  ∑ k ∈ Λ,
    ((starRingEnd ℂ) (ψ k) *
      ((ω k : ℂ) * ψ k + (g : ℂ) * nL Λ ψ k)).im = 0 := by
-- (proof abridged) termwise, conj ψ_k * (ω_k ψ_k + g N_k) is
```

```
-- the real number  $\omega_k |\psi_k|^2$  plus  $g * (\text{conj } \psi_k * N_k)$ ; the
-- second parts sum to  $g * \text{pairing } \Lambda \psi$ , whose imaginary part is
-- 0 (pairing_im_zero): pairing  $\Lambda \psi$  is a sum of squared moduli
```

The symbols: $\text{Finset } G$ is a finite subset of the group G ; $\sum k \in \Lambda$, $\text{f } k$ is the finite sum; $\text{starRingEnd } \mathbb{C}$ is complex conjugation; .im is the imaginary part; $(\omega_k : \mathbb{C})$ is a real number regarded as a complex one. The definition nl is the sum of Equation (2.2), with the condition $k_1 + k_3 = k + k_2$ written as an *if*. The theorem `QuantumFluids.GPGalerkin.pairing_eq_sum` says that the quartic interaction sum $Q = \sum_k \overline{\psi_k} N_k$ equals $\sum_q |A_q|^2$, where $A_q = \sum_{a-b=q} \psi_a \overline{\psi_b}$ is the Fourier transform of the density $|\psi|^2$; hence Q is real and non-negative (`QuantumFluids.GPGalerkin.pairing_nonneg`), which is the defocusing sign of the interaction, for every Λ . The theorem `QuantumFluids.GPGalerkin.mass_rate_zero` is the algebraic core of the conservation of the norm: for $i\dot{\psi}_k = G_k$ one has $d|\psi_k|^2/dt = 2 \text{Im}(\overline{\psi_k} G_k)$, so the theorem says that the *rate* of change of $\sum_k |\psi_k|^2$ is zero. The whole conservation law is the statement that a certain sum is real, and the abridged proof says why. (All thirteen theorems of the module depend on the three standard axioms and on nothing else, as the audit lines at the end of the file show.)

What the theorem does not say is as important. It does not say that the system has a solution, or that the solution is unique; it does not say that $\sum_k |\psi_k|^2$ is constant along a solution (that needs the chain rule, which the library does not formalize); and it says nothing about the numbers that any integrator returns. The companion statement for the energy, `QuantumFluids.GPGalerkin.energy_rate_zero`, is of the same kind: an algebraic core, with the same limits, resting on the gradient identity `QuantumFluids.GPGalerkin.grad_identity`.

The library has no statement about the third conservation law of this equation, momentum. The programme's pre-registration of the solver (its section 4) said otherwise: it stated that the invariants of the controls K_1 to K_3 were theorems of `GPGalerkin` and that the projector did not change them. Two of the three were there as algebraic cores only, and the third was not there at all. The fault of Section 2.1 was in the second half of that sentence. The statements written for this chapter repair the first half; Section 2.7 measures the second.

LEAN 4 NEW: CHO2_GALERKIN

```
-- variable {G : Type*} [AddCommGroup G] [DecidableEq G]
-- nl : as in GPGalerkin, copied verbatim

noncomputable def planeWave (a₀ : ℂ) (ω t : ℝ) : ℂ :=
  a₀ * Complex.exp (-(Complex.I * ω) * t)

noncomputable def planeState (k₀ : G) (a₀ : ℂ) (Ω t : ℝ) :
  G → ℂ :=
  fun x => if x = k₀ then planeWave a₀ Ω t else 0
```

```

noncomputable def galerkinField (Λ : Finset G) (ω : G → ℝ)
  (g : ℝ) (ψ : G → ℂ) (k : G) : ℂ :=
  -Complex.I * ((ω k : ℂ) * ψ k + (g : ℂ) * n1 Λ ψ k)

theorem planeState_solves_galerkin (Λ : Finset G) (k0 : G)
  (hk0 : k0 ∈ Λ) (ω : G → ℝ) (g : ℝ) (a0 : ℂ) (k : G)
  (t : ℝ) :
  HasDerivAt
    (fun s : ℝ =>
      planeState k0 a0 (ω k0 + g * ||a0|| ^ 2) s k)
    (galerkinField Λ ω g
      (planeState k0 a0 (ω k0 + g * ||a0|| ^ 2) t) k) t := by
  -- (proof abridged) n1 of a one-mode state is |a|2 a on that
  -- mode and 0 elsewhere (n1_single); on the mode, |planeWave|
  -- is constant and planeWave' = -i Ω planeWave

```

The exact plane wave. The box above states the solution on which the programme's fourth known answer (K4) rests. A single occupied mode k_0 with amplitude $a_0 e^{-i\Omega t}$, where $\Omega = \omega_{k_0} + g|a_0|^2$, solves the Galerkin system on every finite mode set that contains k_0 , for every dispersion ω , every coupling g and every amplitude. The theorem evaluates the vector field of Equation (2.2) on the plane-wave state, component by component, and equates it to the time derivative of that component. It is a statement about $\mathbb{R}$ and $\mathbb{C}$ and about no program; Section 2.6 asks three integrators to reproduce it.

LEAN 4 NEW: CHO2_GALERKIN

```

theorem momentum_rate_zero (Λ : Finset G) (ψ : G → ℂ)
  (ω : G → ℝ) (g : ℝ) (p : G →+ ℝ) :
  ∑ k ∈ Λ, p k * ((starRingEnd ℂ) (ψ k) *
    ((ω k : ℂ) * ψ k + (g : ℂ) * n1 Λ ψ k)).im = 0 := by
  -- (proof abridged) the four-fold sum over (a, b, c, d) with
  -- b + d = a + c of p a * conj ψ_a ψ_b conj ψ_c ψ_d; the
  -- renamings a<->c, b<->d, (a,b,c,d)->(b,a,d,c); additivity:
  -- p a + p c = p (a + c) = p (b + d) = p b + p d; so it is real

theorem no_momentum_on_torus (N : ℕ) [NeZero N]
  (p : (ZMod N × ZMod N) →+ ℝ) (x : ZMod N × ZMod N) :
  p x = 0 := by
  -- (proof abridged) N • x = 0 in the torus, so N * p x = 0 in ℝ

```

Momentum, and the hypothesis that carries it. The box above is the new statement behind the fault of Section 2.1. Here $p : G \rightarrow+ \mathbb{R}$ is an *additive map*: $p(k + k') = p(k) + p(k')$. On

$G = \mathbb{Z}^2$ these are exactly the linear combinations of the two components of the wave vector, so `momentum_rate_zero` says that the rate of change of the total momentum $\sum_k p(k) |\psi_k|^2$ vanishes, for every finite Λ and every state. The constant weight $p \equiv 1$ is not additive (the norm has its own theorem and proof); the momentum theorem uses, besides the resonance condition $k_1 + k_3 = k + k_2$, *only* that p is additive. The abridged proof is short. The nonlinear part of the momentum rate is the imaginary part of the four-fold sum $S_a = \sum p(a) \overline{\psi_a} \psi_b \overline{\psi_c} \psi_d$ over quadruples with $b + d = a + c$. Renaming $a \leftrightarrow c$ leaves the summand unchanged, so $S_a = S_c$, and likewise $S_b = S_d$ when the weight is $p(b)$ or $p(d)$; additivity and the resonance condition give $p(a) + p(c) = p(b) + p(d)$, so $S_a + S_c = S_b + S_d$ and therefore $S_a = S_b$; and complex conjugation maps the summand at (a, b, c, d) to the summand at (b, a, d, c) , so $\overline{S_a} = S_b = S_a$. A number equal to its conjugate is real. In the file the three renamings are three short chains of `Finset.sum_comm`, and the last step is one `linear_combination`; a corollary instantiates $G = \mathbb{Z} \times \mathbb{Z}$ and $p(k) = k_1$.

The second theorem of the box says why the hypothesis is not decoration. On the periodic grid $\mathbb{Z}_N \times \mathbb{Z}_N$, where the engine's wave vectors live, every additive map to the real numbers is zero: there is no additive momentum, so `momentum_rate_zero` has no content there. This does *not* prove that momentum fails to be conserved by a wrapped-around scheme (that is a measurement, Section 2.7); it says that the proof has nothing to stand on. It follows (`no_additive_wavenumber` in the file) that the wave number itself, $k \mapsto k$, is not an additive function modulo N , which is the formal form of “aliasing breaks translation invariance”.

2.4 rusty-SUNDIALS in twenty minutes

What CVODE does. CVODE integrates $y' = f(t, y)$, $y(t_0) = y_0$, by linear multistep methods: the new value is built from several previous values and from f , with a step h and an order q that the solver changes as it goes. Two families are offered. The Adams–Moulton methods (orders 1–12) are meant for smooth, non-stiff problems and the backward differentiation formulas (BDF, orders 1–5) for stiff ones. Both are implicit, the new value appears on both sides, and in the Rust implementation both are solved by the same Newton iteration with a dense LU factorization of $I - \gamma J$; when the user supplies no Jacobian, as in the Python module, J is built column by column by finite differences, at the price of n extra evaluations of f per Jacobian for a system of dimension n . The solution is kept as a Nordsieck array of scaled derivatives. After each step the solver estimates the local error from the size of the Newton correction, weights it component by component with $1/(\text{rtol } |y_i| + \text{atol})$, accepts or rejects the step, and chooses h and q for the next one. The user states a tolerance, not a step. What the tolerance controls is the *local* error of each step; the error of the final answer is not guaranteed to be of that size, and Section 2.5 shows how far it can stray.

What rusty-SUNDIALS is. rusty-SUNDIALS is the programme's Rust implementation of this algorithm, translated, in the words of its source files, from the C code of the SUNDIALS suite [4]. The Python module `rusty_sundials` exposes one class, `CvodeSolver(method, rtol, atol, max_steps)`, with one method, `solve(rhs, t0, y0, t_out)`, which calls the user's Python

function `rhs(t, y)` and returns the pair (t, y) at the end of the interval and nothing else; a trajectory is obtained by many calls, and every call starts the integration afresh. The Rust source calls the order selection of its BDF path a simple heuristic, so step counts need not match those of the C code, and its repository's README speaks of Lean 4 specifications of the solver that this book neither uses nor re-checks. Nothing in this book is a proof about the Rust code. It is tested: against closed-form solutions, against a second integrator of the same equation, and against the programme's numpy implementation of the engine. The word “verified” in the title of the software paper [21] is used in that sense: its claims are agreements with reference answers, listed there with their precision.

RUSTY-SUNDIALS CCODE FROM PYTHON

```
import math
from rusty_sundials import CcodeSolver

calls = 0
def rhs(t, y):
    # x'' = -x written as a first-order system (x, v)
    global calls
    calls += 1
    return [y[1], -y[0]]

T = 20 * math.pi          # ten periods; the exact solution is (cos T, -sin T)
for method in ("adams", "bdf"):
    calls = 0
    # arguments: method, rtol, atol, max_steps
    solver = CcodeSolver(method, 1e-8, 1e-10, 100000)
    t, y = solver.solve(rhs, 0.0, [1.0, 0.0], T)
    err = max(abs(y[0] - math.cos(T)), abs(y[1] + math.sin(T)))
    print(f"{method:5s} rhs calls {calls:6d} error {err:.2e}")

adams rhs calls    1549  error 3.76e-07
bdf   rhs calls    5217  error 2.53e-06
```

Output of `figures/ch02_snippets.py`, run with the module described below.

The box above is the whole interface in use: ten periods of the harmonic oscillator, both methods, the exact answer known. The number of calls of `rhs`, which includes the columns of the finite-difference Jacobian, is the *cost* used throughout this chapter; unlike wall time it does not depend on the load of a shared machine. Both runs asked for a relative tolerance of 10^{-8} , and neither delivered a final error of that size: 3.8×10^{-7} and 2.5×10^{-6} .

A defect, and a stale build. Until it was repaired, `Method : Adams` never left order 1: the coefficients that define the higher orders were placeholders, so every Adams run was implicit Euler with

an error estimate of the wrong kind, and under a per-step error control a first-order method has a global error that falls only as the square root of the tolerance. The defect was first seen as exactly that square-root convergence on a cosmological distance integral, and it also blocks the programme's order test of the PGPE engine against CVODE (Section 2.6), which cannot finish without the repair: the Adams path fails after about 10^6 evaluations. The repair implements the order and step selection of the C library (rusty-SUNDIALS, docs/CVODE_ADAMS_FIX.md, dated 2026-09-28). The Python module in the programme's virtual environment was built on 26 September, before it, and still has the defect. This chapter's first probe, run with that module, showed the old signature at once; every CVODE result below comes from a build of the checked-out sources (commit 5db8041, release line v1.1.6.0), and the compute script refuses to run with a module whose Adams path takes more than 5 000 evaluations on $y' = -y$. Panel (d) of Figure 2.2 shows the two builds side by side.

What qf-pgpe is. The quantum-fluids crate qf-pgpe integrates Equation (2.1) on a periodic grid by an integrating-factor Runge–Kutta scheme of order four (IF-RK4): the linear part $-ik^2/2$ is applied exactly in Fourier space, the cubic part by classical RK4, the projector is a sharp circle at $k_{\text{cut}} = k_{\text{max}}/2$, and the arrays follow numpy's FFT conventions so that the Python programme and the Rust engine exchange fields without conversion. A Python extension, qf_pgpe, runs a whole time interval in one call. The same right-hand side is available to external integrators as a system of $2n_{\text{modes}}$ real equations (the functions rhs, pack and unpack of the engine): that is how CVODE enters, as a second integrator, written independently, of the *identical* equation. On the programme's benchmark the single-threaded Rust step is 4.3, 4.2, 2.7 and 2.5 times faster than the numpy step at $N = 64, 128, 256$ and 512 [21]; those numbers were measured on a 2013 four-core laptop processor shared with an unrelated job, and are quoted, not repeated here. The dense Jacobian of the Python module needs n evaluations and n^2 numbers of memory for $n = 2n_{\text{modes}}$: for the 128^2 grid of the thermal sweep ($\approx 3\,200$ modes) about 330 MB. CVODE is therefore used here as a referee on a 16×16 grid (49 modes, $n = 98$), not as the production integrator. No GPU code is involved anywhere in this book.

2.5 The solver on problems with known answers

Before asking CVODE anything about a quantum fluid we ask it questions to which we know the answers. Figure 2.1 is the theory behind the questions. For the test equation $y' = \lambda y$ a multistep formula with the step frozen advances y by an amplification factor determined by $z = h\lambda$; the *region of absolute stability* is the set of z for which the factors do not grow [30, 31]. For the Adams–Moulton formulas of order three and higher the region is a small lobe around the origin, with intercepts on the negative real axis of $-6.00, -3.00, -1.84$ and -1.18 for orders 3 to 6 (computed here, not copied); the trapezoidal rule and implicit Euler are stable on the whole left half-plane. For BDF the unstable region is a bounded lobe on the right, and every point outside it is stable. BDF of orders 1 and 2 are stable on the whole left half-plane; orders 3, 4 and 5 are stable in a wedge of half-angle $\alpha = 86.0^\circ, 73.4^\circ$ and 51.8° around the negative real axis and leave out a wedge of the left half-plane

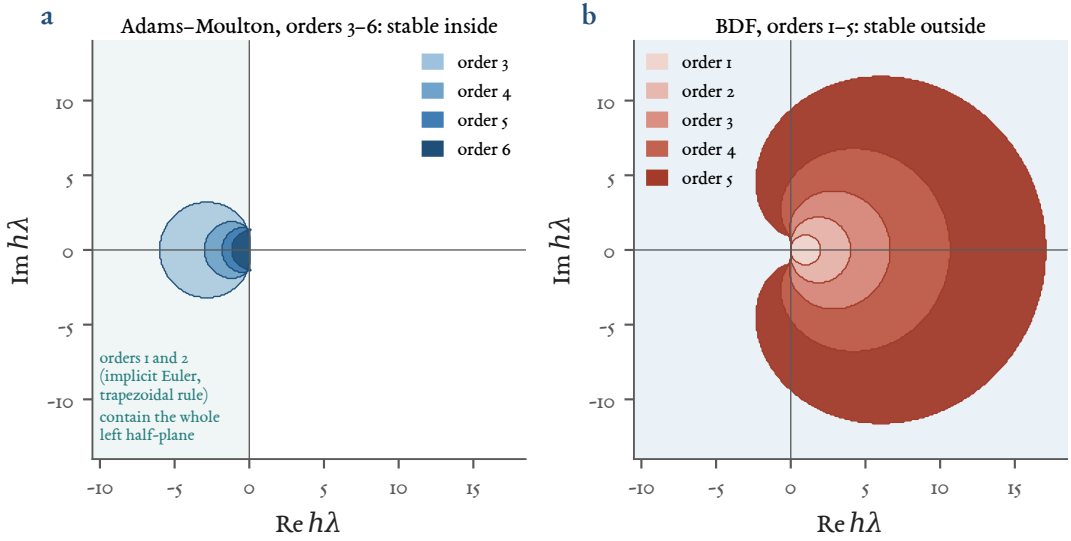


Figure 2.1: Regions of absolute stability, computed by a root test on a grid of $z = h\lambda$ for the fixed-step formulas to which CVODE's two families reduce when step and order are frozen. (a) Adams-Moulton, orders 3-6: stable inside the lobes; orders 1 and 2 (implicit Euler, trapezoidal rule) are stable on the whole shaded left half-plane. (b) BDF, orders 1-5: the filled lobes are the *unstable* regions, everything else is stable. The coefficients were checked against their order conditions (exactness on polynomials), and the computed half-angles $90^\circ, 90^\circ, 86.0^\circ, 73.4^\circ, 51.8^\circ$ of the stable wedge of BDF orders 1-5 agree with an independent boundary-locus computation ($86.03^\circ, 73.35^\circ, 51.84^\circ$), and the real-axis ends of panel (a) were refined by bisection. CVODE changes step and order at run time: this figure is a guide to the families, not a law for a run. Script `figures/ch02_stability.py`.

next to the imaginary axis, which widens with the order. A problem is *stiff* when its eigenvalues have large negative real parts: then any step that resolves the solution has $h\lambda$ far outside an Adams lobe, and only a formula whose region contains the far left half-plane (BDF, or the Adams formulas of orders 1 and 2) can take it with a step that accuracy, not stability, chooses.

Figure 2.2 reports what the Python module does on three problems with closed-form solutions. Panel (a) is ten periods of the harmonic oscillator, 19 tolerances from 10^{-3} to 10^{-12} for each method. BDF gives a clean line: the error falls smoothly as the tolerance is tightened. Adams is cheaper, by a factor of about seven where it matters (the cheapest Adams run with a final error below 10^{-8} used 1 880 evaluations, the cheapest BDF run 12 769), and erratic: the run with $\text{rtol} = 10^{-5}$ used 704 evaluations and ended with an error of 4.1×10^{-5} , the run with 10^{-6} used 3 327 and ended with 5.0×10^{-4} , twelve times worse at 4.7 times the cost. Local error control does not promise a monotone global error, and it does not deliver one.

Panel (b) is the stiffness test. The Prothero-Robinson problem has the smooth solution $y = \cos t$ for every λ , and a stiffness that grows with λ . At $\lambda = 1$ Adams is cheaper than BDF (255 evaluations against 470); at $\lambda = 10^5$ it is thirteen times dearer (5 565 against 418), and BDF's cost has not

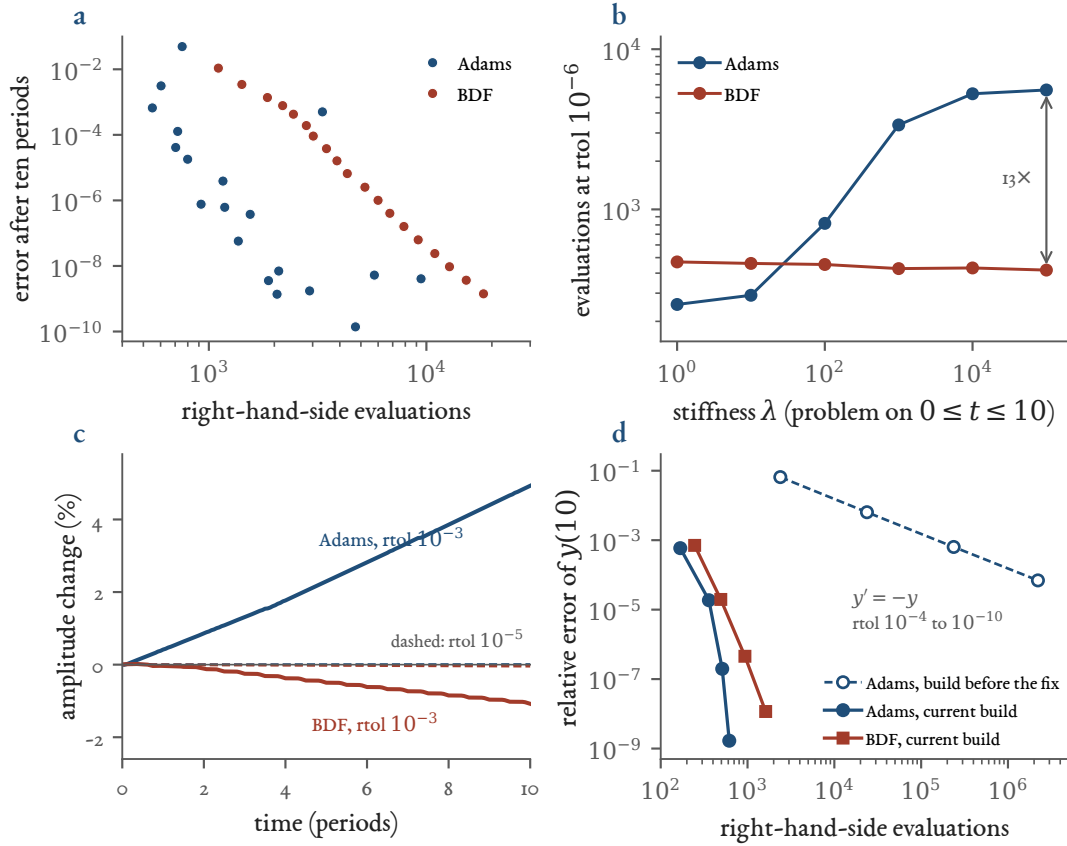


Figure 2.2: CVODE through the Python module on problems with closed forms; cost is the number of calls of the user’s right-hand side, including the finite-difference Jacobian. (a) Ten periods of $x'' = -x$: error against cost, 19 tolerances each. (b) Prothero–Robinson problem $y' = -\lambda(y - \cos t) - \sin t$ [32] on $0 \leq t \leq 10$ at $\text{rtol} = 10^{-6}$: cost against stiffness λ . (c) Amplitude $|(x, v)|$ of the oscillator, exactly 1, in independent runs to each time: at $\text{rtol} = 10^{-3}$ (full lines) and 10^{-5} (dashed). (d) $y' = -y$: the module built on 26 September (before the Adams repair) and the current build. Script figures/ch02_workprecision.py; data figures/ch02_numbers.json, key B_cvode.

moved. A fixed-step stability bound would make the Adams cost grow in proportion to λ ; it saturates instead. We did not trace why (a plausible reason is that the order selection falls back to the A-stable orders 1 and 2, but the Python module does not report the order). What the measurement shows is the textbook division of labour with its cost attached: the non-stiff method is the cheaper one on the non-stiff problem, and an order of magnitude dearer on the stiff one.

Panel (c) matters for a Hamiltonian equation like Equation (2.1): the oscillator has a conserved amplitude, as the PGPE has a conserved norm. After ten periods at $\text{rtol} = 10^{-3}$ the BDF solution has lost 1.1 % of its amplitude (0.043 % at 10^{-5}), as a dissipative method does on an oscillation (the stability theory is in [31]), and in all 19 runs of panel (a) it ended with an amplitude below 1. Adams errs in either direction depending on the tolerance (negative in 11 of the 19 runs); at $\text{rtol} = 10^{-3}$ it gained 4.9 %, at 10^{-5} 0.004 %. The integrator has a bias of its own, which competes with the physics being simulated.

Panel (d) is the defect of the previous section made visible. With the old module the error of Adams on the simplest problem of all, $y' = -y$, falls by a factor $10^{0.5}$ per decade of tolerance while the cost rises by the same factor, the signature of a first-order method: at $\text{rtol} = 10^{-10}$ it needs 2.2×10^6 evaluations for an error of 6.9×10^{-5} , where the current build needs 616 for 1.7×10^{-9} . Every Adams run made with a build that predates the repair was a run of implicit Euler.

2.6 The plane wave: a theorem and three integrators

The known answer is the one proved in Section 2.3. We take a plane wave on a periodic box of side $L = 16$, $g = 1$, density 1, wave number $k = 2\pi \cdot 3/L = 1.178$, so that $\Omega = k^2/2 + g = 1.694$ (the one-mode form of the theorem is $\dot{a} = -i(k^2/2 + g|a|^2)a$ with $|a| = 1$), and integrate to $t = 10$. The box below shows the call in the Rust engine.

RUSTY-SUNDIALS QF-PGPE AGAINST THE EXACT SOLUTION

```
import numpy as np, qf_pgpe

N, L, g, m = 32, 16.0, 1.0, 3
engine = qf_pgpe.Pgpe(N, L, g, 0.01)      # grid, box, coupling, time step
x = np.arange(N) * L / N
X, Y = np.meshgrid(x, x, indexing="ij")
k = 2 * np.pi * m / L
psi0 = np.exp(1j * k * X)                 # one mode, density 1
c = engine.run(np.ascontiguousarray(engine.modes(psi0)), 10.0)
omega = 0.5 * k ** 2 + g                   # omega = k^2/2 + g |a0|^2
exact = psi0 * np.exp(-1j * omega * 10.0)
err = np.abs(engine.psi(c) - exact).max()
print(f"plane wave, t = 10: max |psi - exact| = {err:.2e}")
```

```
plane wave, t = 10: max |psi - exact| = 1.12e-08
```

The error is 1.1×10^{-8} at $\Delta t = 0.01$. Panel (a) of Figure 2.3 shows how it arises. For $\Delta t = 0.02, 0.01, 0.005$ the error at $t = 10$ is $1.8 \times 10^{-7}, 1.1 \times 10^{-8}$ and 7.0×10^{-10} , in the numpy engine and in the Rust engine alike (the two final states differ by 4×10^{-14} at most, relative to the amplitude of the occupied mode); the ratios are 15.9 and 16.0. The error grows in proportion to t and is a phase error: over the same run the norm of the state, hence the amplitude of the mode, changes by only 7×10^{-12} (relative) at $\Delta t = 0.01$; its dependence on the step is that of a fourth-order scheme. At the programme's registered configuration (known answer K4: $N = 64, L = 32, \Delta t = 0.005, t = 10$, acceptance 10^{-9}) both engines give 7.0×10^{-10} , the value of the stored first run. With $g = 0$ the error is 1.1×10^{-14} , rounding level.

The order of the scheme, by an independent integrator. The programme's registered control K2 asks that the energy drift of the scheme fall as Δt^4 . The Rust port tests the order more directly, against a solution of the *same* right-hand side by CVODE, which shares with the engine nothing but that function. On a 16×16 grid (49 modes) with a smooth initial state, CVODE (Adams, $\text{rtol} = 10^{-12}$, $\text{atol} = 10^{-14}$) needs 482 evaluations in 95 steps, and the engine's error at $\Delta t = 0.04, \dots, 0.0025$ falls from 1.2×10^{-4} to 1.8×10^{-9} with successive ratios 15.99, 16.00, 16.00 and 15.97 (a fourth-order scheme gives 16). The fitted order is 3.999. The reference run needs the Adams repair: with the old path it does not finish.

CVODE on the same wave. Panel (c) puts the integrators on one axis, for the 16×16 plane wave. CVODE's error is not monotone in its tolerance: Adams with $\text{rtol} = 10^{-4}$ spent 1 530 evaluations and ended with an error of 1.3×10^{-3} , with 10^{-6} it spent 788 and ended with 2.5×10^{-4} , with 10^{-8} 701 and 1.8×10^{-6} , with 10^{-10} 1 143 and 2.0×10^{-8} ; BDF at 10^{-8} spent 2 364 for 2.9×10^{-5} . At comparable error the engine's scheme needs about three times the evaluations of CVODE-Adams at its tightest tolerance ($\approx 3\,400$ for 2×10^{-8} , by interpolation) and, unlike CVODE, its error is monotone in the step. An evaluation of the engine's nonlinear term and one of CVODE's full right-hand side are not the same amount of work, and which integrator is cheaper is not the point. The point is that all three reproduce the exact solution of the theorem to the precision each promises, which is what “the program computes the function” means in this case.

What does not conserve what. Lean proves that the vector field has zero norm rate. Panel (d) shows what integrators do with a conserved quantity. On a smooth state with interacting modes (the order-test state, evolved to $t = 5$) the relative change of the norm under IF-RK4 at $\Delta t = 0.02, 0.01, 0.005$ is $-1.5 \times 10^{-9}, -9.8 \times 10^{-11}$ and -6.3×10^{-12} : it falls by a factor 15–16 per halving, so it is truncation error, not conservation. For CVODE-Adams at $\text{rtol} = 10^{-4}$ to 10^{-10} it is $+1.2 \times 10^{-6}, -3.4 \times 10^{-8}, +3.3 \times 10^{-10}$ and -1.2×10^{-12} , of either sign. For BDF it is $-1.2 \times 10^{-5}, -2.3 \times 10^{-7}, -7.9 \times 10^{-9}$ and -1.9×10^{-10} , always a loss. No integrator conserves

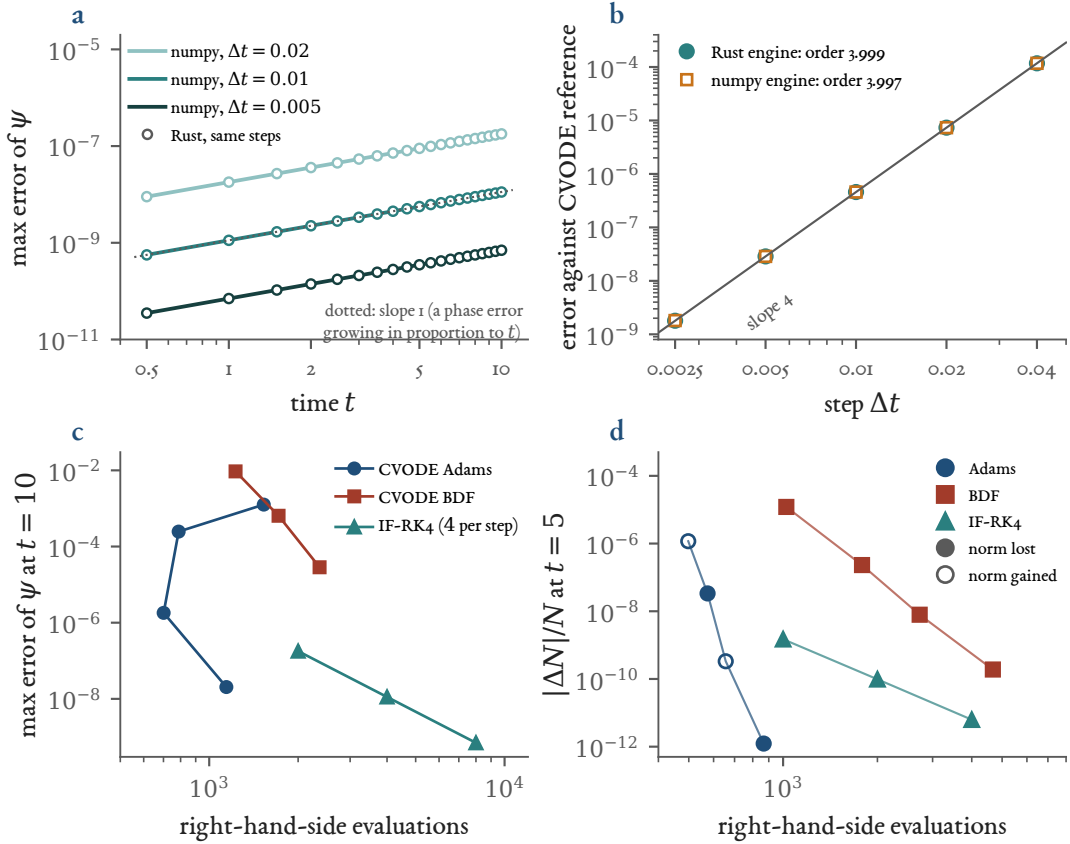


Figure 2.3: The exact plane wave against three integrators. (a) Error of the numpy IF-RK4 engine (lines) and of the Rust engine (circles) against $\psi_0 e^{-i\Omega t}$, $N = 32$, three steps; the error grows $\propto t$. (b) Order of the IF-RK4 step measured against a CVODE (Adams, $\text{rtol} = 10^{-12}$) solution of the same right-hand side on a 16×16 grid, $t = 1$: the registered Rust example (re-run; its output is identical to the stored file) gives order 3.999, the same test through the Python module and the numpy engine 3.997; the errors of the two routes agree to 0.8%. (c) Cost against accuracy on the 16×16 plane wave. (d) Drift of the norm for a state with interacting modes (a smooth three-mode superposition, $t = 5$); filled symbols: norm lost, open: norm gained. Script figures/ch02_planewave.py; keys A_order, C_planewave, D_invariants.

the norm; each loses or gains it at a rate set by its own tolerance. The theorem is about the equation, and is true; the invariant is conserved by the program to a stated precision and no further.

2.7 Where a theorem and a program part: aliasing

The engine does not evaluate N_k of Equation (2.2). It evaluates $P[\text{FFT}(|\psi|^2 \psi)]$ on an $N \times N$ grid: transform to the grid, cube pointwise, transform back, keep the modes inside the projector. On the grid a wave vector is a class modulo N , so a triple with $k_1 - k_2 + k_3 = k + Nm$ for an integer vector $m \neq 0$ is added to the same output mode as the triples with $m = 0$. The engine's cubic term is a sum over all triples whose sum is congruent to k ; the Lean N_k is a sum over the triples whose sum is k . They are the same function exactly when no triple wraps.

The box below is the check: a literal transcription of the Lean definition for $G = \mathbb{Z}^2$, no wrap-around, and a comparison with the numpy engine's cubic term, rescaled to the units of the Lean statement ($a = c/N^2$ for the unnormalized FFT convention of the engine). The transcription was written for this chapter and is not checked by Lean; its consistency is tested below.

RUSTY-SUNDIALS THE LEAN DEFINITION OF THE CUBIC TERM, TRANSCRIBED, AGAINST THE ENGINE

```
def lean_nl(Lam, A):
    """Literal transcription of QuantumFluids.GPGalerkin.nl on Z^2:
        nl psi k = sum over k1, k2, k3 in Lam with k1 + k3 = k + k2 of
            psi k1 * conj(psi k2) * psi k3          (no wrap-around)
        Lam: (m, 2) integer array of modes, A: (m,) amplitudes psi.
        Returns (table, B) with table[kx + B, ky + B] = nl psi (kx, ky)."""
    m = len(Lam)
    B = 3 * int(np.abs(Lam).max()) + 1          # sums reach 3 R
    size = 2 * B + 1
    re = np.zeros(size * size)
    im = np.zeros(size * size)
    for i in range(m):                          # k1 = Lam[i]
        k = Lam[i] - Lam[:, None, :] + Lam[None, :, :]    # k1 - k2 + k3
        idx = ((k[..., 0] + B) * size + (k[..., 1] + B)).ravel()
        v = (A[i] * np.conj(A[:, None]) * A[None, :]).ravel()
        re += np.bincount(idx, weights=v.real, minlength=size * size)
        im += np.bincount(idx, weights=v.imag, minlength=size * size)
    return (re + 1j * im).reshape(size, size), B
```

and, in the driver, the engine's term in the same units:

```
a = np.zeros((N, N), dtype=complex)
a[s.P] = A          # psi_k, the units of the Lean statement
c = a * N ** 2      # the engine's unnormalized FFT amplitudes
```

```
# the engine's cubic term, in the units of nl:
```

```
nl_eng = s.nonlin(c) / (-1j * g * N ** 2)
```

```
cutoff 0.500  modes 197  max|engine-nl|/max|nl| = 7.85e-16  differing: 0
```

```
cutoff 0.500  modes 197  max|engine-nl|/max|nl| = 1.15e-03  differing: 4
```

```
cutoff 0.667  modes 357  max|engine-nl|/max|nl| = 1.53e-01  differing: 348
```

The three lines are the cases of Figures 2.4 and 2.5: $N = 32$, $L = 16$, a random complex state on the retained modes (seed fixed); the first with the four extreme modes empty, the second with all modes filled, the third at the larger cutoff. Function `lean_nl` and driver `bridge_case` of `figures/ch02_compute.py`, run by `figures/ch02_snippets.py`.

With the cutoff at one half of the largest wave number, the engine's cubic term equals the Lean definition to 7.9×10^{-16} , rounding, on a state whose four extreme modes are empty. With those modes filled, four modes of the 197 differ, by 1.1×10^{-3} at most (relative to the largest value of N_k). At two thirds, 348 of 357 modes differ, by up to 15 % of the largest value. The geometry of Figure 2.4(a,b) explains all three. A triple reaches $|k_1 - k_2 + k_3| \leq 3R$ if the retained modes lie in a disc of radius R (in index units); the periodic images of that reach-disc, displaced by N along the axes, intrude into the retained disc when $3R + R > N$. So the engine's function equals N_k for *every* state when $4R < N$, that is, $k_{\text{cut}} < k_{\text{max}}/2$. At $4R = N$, which is the engine's default cutoff, the discs touch at four points: the four modes $\pm(N/4, 0)$, $\pm(0, N/4)$, each fed by a single wrapped triple, for the mode $(-N/4, 0)$ the triple $k_1 = k_3 = -k_2 = (N/4, 0)$, as the data show. For a quadratic nonlinearity the reach is $2R$, the condition is $3R \leq N$, and one recovers the two-thirds rule [29]; for the cubic term of the PGPE that rule is wrong, as the programme found.

Panel (a) of Figure 2.5 puts the theorems against the engine. For the literal definition both rates vanish to rounding (below 10^{-17} relative to the sum of the moduli of the terms) in all three cases, as `QuantumFluids.GPGalerkin.mass_rate_zero` and `momentum_rate_zero` say they must. That the transcription is the Lean object is tested by the identity of `QuantumFluids.GPGalerkin.pairing_eq_sum`, which the transcription satisfies: $Q = 1.403114631989707$ against $\sum_q |A_q|^2 = 1.4031146319897068$, with imaginary part 2.8×10^{-17} . For the engine, the mass rate is zero to rounding even where triples wrap: by Parseval on the grid

$$\sum_k \overline{a_k} P[\text{FFT}(|\psi|^2 \psi)]_k = \sum_x |\psi(x)|^4,$$

a real number, wrapped or not (an argument, not a Lean theorem). The momentum rate is not zero: 1.6×10^{-17} with the extreme modes empty, 2.3×10^{-5} with them filled, 4.1×10^{-3} at the larger cutoff. The theorem and the program agree exactly where the theorem's hypothesis holds, and part company where it does not: there the grid is $\mathbb{Z}_N^2$, and `no_momentum_on_torus` says that no additive momentum exists on it.

Integrated in time (panel (b) of Figure 2.5), one smooth random state ($N = 32$, $L = 16$, energy

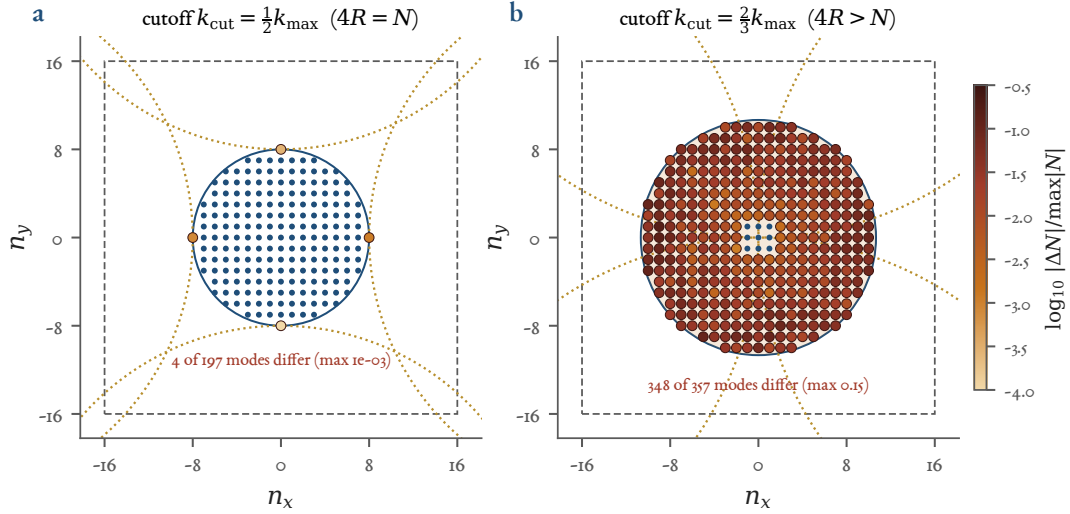


Figure 2.4: The same equation in the two languages: where the engine’s cubic term and the literal Lean definition coincide. Retained modes in the index plane ($N = 32$), coloured by the difference between the two on a random state with all modes filled; blue: equal to rounding. Dashed square: the Nyquist box; dotted arcs: the periodic images of the disc of radius $3R$ reached by the triple sums; orange shading: the modes the images cover. The cutoffs are $\frac{1}{2}k_{\max}$ (a) and $\frac{2}{3}k_{\max}$ (b). Script figures/ch02_bridge.py; key E_bridge.

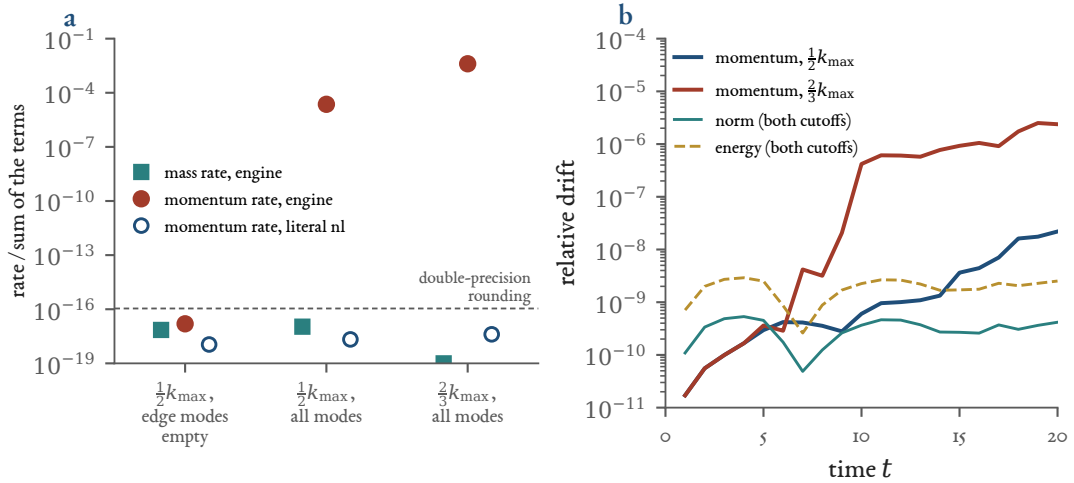


Figure 2.5: Theorems and engine, in rates and in time. (a) Instantaneous rates for the engine and for the literal definition, in the three cases of the text: of the mass, $|\sum_k \text{Im}(\bar{\psi}_k G_k)| / \sum_k |\bar{\psi}_k G_k|$, and of the momentum, the same with the weight $p(k) = k_x$ in numerator and denominator; the Lean theorems say that the literal definition gives zero. The point on the lower edge is exactly 0. (b) Drift of the invariants along an IF-RK4 run ($\Delta t = 0.01$) from one smooth random state at $N = 32$, for the two cutoffs; norm and energy are the same for both. Script figures/ch02_bridge.py; key E_bridge.

property	Lean (a statement for every Λ and state)	registered control and independent integrators	what it caught here
norm rate = 0	<code>mass_rate_zero</code> (library), any Λ	K1: drift 4.2×10^{-10} ; engine rate at most 1.1×10^{-17} in all cases	nothing: blind to aliasing
exact plane wave	<code>planeState_solves_galerkin</code> (new)	K4: 7.0×10^{-10} ; numpy, Rust and CVODE agree	nothing: blind to aliasing
momentum rate = 0	<code>momentum_rate_zero</code> (new), needs an additive p	K3: drift 8.5 in the first run; rate 4.1×10^{-3} at $\frac{2}{3}k_{\max}$	the wrong cutoff rule
order of IF-RK4	none in the library	K2 (energy drift $\propto \Delta t^4$); error against CVODE: order 3.999	the Adams defect: the reference run cannot finish without the repair
the program computes the function	none: a statement about a program	transcription of nl against the engine	4 modes differ at $\frac{1}{2}k_{\max}$, 348 at $\frac{2}{3}k_{\max}$

Table 2.1: Which instrument can establish what. The first two controls pass with or without aliasing; the third separates the cutoffs.

0.6 per particle, seed 3) evolved by IF-RK4 at $\Delta t = 0.01$ with the two cutoffs has at $t = 10$ a relative norm drift of 3.7×10^{-10} and an energy drift of 2.3×10^{-9} , the same at both cutoffs, and a momentum drift (relative to $\sum_k |k| |c_k|^2$) of 6.0×10^{-10} at $\frac{1}{2}k_{\max}$ and 4.2×10^{-7} at $\frac{2}{3}k_{\max}$; at $t = 20$ the momentum drifts are 2.2×10^{-8} and 2.4×10^{-6} , norm and energy 4.2×10^{-10} and 2.5×10^{-9} for both. Norm and energy do not see the aliasing; momentum does, separating the two cutoffs by two to three orders of magnitude. This is a modest reproduction of the programme's control, on a smooth state and a 32^2 grid where the recorded failure (8.5, one per cent) was a random high-energy state on 64^2 . The drift at the one-half cutoff is not zero either; we did not separate the truncation error of the scheme from the effect of the four edge modes, which the evolving state may populate.

2.8 What each instrument can see

Table 2.1 sorts the statements of this chapter by the instrument that can establish them. The rows for the norm and the plane wave are the reason the programme runs controls of several kinds: a proof of the right theorem cannot detect that the program evaluates another function, and the control that can, the momentum, is one that the norm and the plane wave are blind to.

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

- **Proved** (Lean, kernel-checked, standard axioms only). For every finite mode set in every additive group: the mass-rate identity and the non-negativity of the quartic interaction (library, `GP Galerkin`); the exact plane wave solves the Galerkin system; the momentum-rate identity for every additive weight; no non-zero additive map exists on $\mathbb{Z}_N \times \mathbb{Z}_N$

(new, book/lean/Ch02_Galerkin.lean). These are statements about the mathematical function N_k and about rates at an instant: not about existence of solutions, not about conservation along a solution, not about any program.

- **Measured or simulated.** The engine's cubic term equals a transcription of N_k to 8×10^{-16} when no triple wraps (one random state per case, $N = 32$); three integrators reproduce the exact plane wave to the precisions stated; the order of the IF-RK4 step is 4.00 (fitted 3.999 and 3.997); CVODE's costs and errors on the stated problems, with the build described above.
- **Not established.** That the transcription equals the Lean object is a correspondence made by hand and tested by one identity, not proved; nothing proves the Rust code; agreement of rusty-SUNDIALS with the C library is not claimed; the stability regions are fixed-step guides; the Adams cost on the stiff problem is measured but its cause is not traced; the momentum drift at the one-half cutoff mixes truncation error and edge-mode aliasing, which we did not separate.
- **An analogy, not a result.** The division into "Lean sees" and "the solver sees" is a classification of this chapter's cases, not a theorem about methods.
- **What failed.** The pre-registered rule $k_{\text{cut}} = \frac{2}{3}k_{\text{max}}$ was wrong for a cubic term, and the momentum control caught it. The pre-registration's sentence that the three conservation laws were library theorems was wrong for one of them and incomplete for the other two. The Adams path of rusty-SUNDIALS was implicit Euler until it was repaired, and the module in the programme's virtual environment still is. The first draft of this chapter's Lean file contained a proof that did not compile, and its audit line carried `sorryAx`.

Exercise 1. In a commutative ring with $e^3 = 0$ show that $(1 + e)(1 - e + e^2) = 1$ by `linear_combination`, then change the sign of the e^2 term and read the reply. *Hint.* The goal minus the coefficient times the hypothesis must be an identity of rings: $(1 + e)(1 - e + e^2) - 1 = e^3$, so the coefficient is 1.

Exercise 2. In the first box of Section 2.4 replace the oscillator by the Prothero–Robinson problem with $\lambda = 10^4$: right-hand side `[-lam * (y[0] - math.cos(t)) - math.sin(t)]`, initial value `[1.0]`, end time 10, tolerances `1e-6` and `1e-8`. Count the calls for both methods and compare with Figure 2.2(b); then set `max_steps` to 300 and read the reply. *Hint.* With the build used here the counts are 5 270 (Adams) and 431 (BDF). With `max_steps = 300` BDF still finishes, and Adams stops with `maximum_steps exceeded (300) at t=1.77`: `max_steps` counts internal steps, and Adams needs more than 300 of them for the first 1.8 time units.

Exercise 3. (a) On a grid with $N = 16$ the retained set at $k_{\text{cut}} = \frac{1}{2}k_{\text{max}}$ contains $(\pm 4, 0)$ and $(0, \pm 4)$. Find the single wrapped triple that feeds the mode $(-4, 0)$, and use the geometry of Figure 2.4 to say for which cutoffs the cubic term of the engine equals N_k for every state when

$N = 16$ (answer in index units). (b) Show that the engine's mass rate $\sum_k \text{Im}(\overline{a_k} G_k)$ vanishes for every state, wrapped or not, and explain why the same argument cannot give momentum. *Hint.* (a) Look for $k_1 + k_3 - k_2 = (-4, 0) + (16, 0)$ with all three modes of norm 4; the condition is $4R < N$. (b) Parseval on the grid turns $\sum_k \overline{a_k} P[\text{FFT}(|\psi|^2 \psi)]_k$ into $\sum_x |\psi(x)|^4$; for momentum you need a function of k that is additive modulo N , and `no_momentum_on_torus` says there is none.

2.9 Notes and further reading

For Lean, *Theorem Proving in Lean 4* [33] introduces the language and *Mathematics in Lean* [34] the use of Mathlib; the kernel and the elaborator are described in [11]. For the numerics, Hairer, Nørsett and Wanner [35] treat Adams methods and Hairer and Wanner [31] stiffness, BDF and absolute stability, with Dahlquist's paper on A-stability [30] as the source of the notion; Prothero and Robinson [32] define the stiff test problem used here; the SUNDIALS suite is described in [4]. The pseudo-spectral treatment of the projected equation and the aliasing rules are in [20] and [29]. The solver is described, with its cross-checks and its performance, in the software paper [21], the Lean library in [12]. Everything in this chapter can be regenerated: the numbers by `figures/ch02_compute.py` (parts A to E, into `figures/ch02_numbers.json`), the code boxes by `figures/ch02_snippets.py`, the figures by the four `figures/ch02_*.py` scripts, the Lean files with `lake env lean` on the three files `book/lean/Ch02_*.lean`; the Python module of the current build is made by `book/rust/ch02_build_py.sh` (the copy used here, with its checksum, is in `/mnt/data/xdev-cache/rs_py_5db8041`). The next chapters use the two languages without re-introducing them; Chapter 10 returns to the question of how far each instrument can be trusted.

From the Wave Function to the Fluid: Circulation and Vortices

3.1 An observation: circulation comes in whole units

In 1961 W. F. Vinen detected single quanta of circulation in superfluid ^4He with a vibrating wire; the quantum had the size that had been anticipated, h/m , with m the mass of a helium atom [36, 37]. Eighteen years later Yarmchuk, Gordon and Packard recorded photographically the positions of the quantized vortex lines in rotating superfluid helium and observed stationary arrays [38]. What these experiments measure is a number that no classical fluid has. The circulation $\Gamma = \oint \mathbf{u} \cdot d\mathbf{l}$ of the velocity around a closed curve drawn in the liquid takes only the values $0, \pm\kappa, \pm2\kappa, \dots$, where

$$\kappa = \frac{h}{m}. \quad (3.1)$$

The number is remarkable for what it does not contain. It is built from Planck's constant and the mass of one atom; no interaction strength, no density, no temperature enters. For ^4He , with the exact SI value of h and the atomic mass $m = 4.002603254 \text{ u}$ (NIST), it is $\kappa = 9.969 \times 10^{-8} \text{ m}^2/\text{s}$ (computed in `figures/ch03_compute.py`). In a vessel rotating at angular velocity Ω the quantized lines must on average imitate solid-body rotation, whose circulation per unit area is 2Ω ; Feynman's counting [16] then gives an areal density $2\Omega/\kappa$ of lines, which is 2.0×10^3 per cm^2 at $\Omega = 1 \text{ rad/s}$.

How can a complex wave function know about circulation? A wave function has a phase, and the velocity of the fluid it describes is the gradient of that phase. A gradient has no circulation, unless the phase cannot be defined everywhere, and a single-valued wave function can fail to define a phase only where it vanishes. A vortex is such a point. That is the chapter in three sentences; the pages that follow make each precise in the way of this book, as a statement a proof assistant has checked, as a computation the solver has run and, where an experiment exists, as a number to compare with it.

WHAT THE CHAPTER ESTABLISHES

1. **Circulation is quantized:** around a closed loop it is $w\kappa$ with w an integer; sampled, this is a theorem about sums of principal phase differences (Lean, Section 3.3).
2. **A vortex is a zero of ψ with a winding number**, which the detector of every simulation code finds once the sampling resolves the phase (Lean, Section 3.3); the solver shows how

fine that is (Section 3.5.3).

3. **Vortices are carried by the flow of the others:** a pair of separation d moves at $\hbar/(md)$ in the plane; in a periodic box the integers that wind the phase around the torus add a uniform flow, 21% of that speed at $d \approx 12\zeta$. The solver reproduces the corrected speed to about a percent for $d \geq 6\zeta$ and follows two leapfrogging pairs against a point-vortex model integrated with CVODE (Sections 3.5.5 and 3.5.6).

GODFRIN THE OTHER KIND OF EXCITATION

The experiments of Henri Godfrin and his collaborators probe quantum fluids through their excitations: inelastic neutron scattering at the Institut Laue–Langevin measured the dispersion relation of the phonon–maxon–roton excitations of superfluid ^4He (Phys. Rev. B **103**, 104516 (2021), spectrometer IN5) [2] and the density waves of two-dimensional liquid ^3He (Nature **483**, 576 (2012)) [1]. In the linear theory such excitations are small oscillations about the uniform state, with a velocity that is the gradient of a single-valued phase: irrotational, without circulation. The vortices of this chapter are the other kind of excitation, states whose phase is *not* single valued around a loop, which no small-amplitude wave can create or remove. Both live in the same ψ . Chapter 5 treats the first kind; the superfluid transition of a two-dimensional ^4He film, measured by Bishop and Reppy [39] and described by the unbinding of vortex–antivortex pairs [40], is where the two meet (Chapter 6). Nothing here claims that those experiments observed vortices.

3.2 The wave function as a fluid

The model of the superfluid used throughout the book is the Gross–Pitaevskii equation [17, 18],

$$i\hbar \partial_t \psi = -\frac{\hbar^2}{2m} \nabla^2 \psi + g |\psi|^2 \psi, \quad (3.2)$$

with $g > 0$ the strength of a contact repulsion. For a dilute Bose gas at zero temperature it is the equation of the condensate; for liquid helium it is a model with the right structure (a complex order parameter, a compressible fluid, quantized circulation) and not the right numbers, since the core of a vortex in the liquid is of atomic size. The uniform state of density n_0 has two natural scales: the healing length $\zeta = \hbar/\sqrt{mg n_0}$ and the speed of sound $c = \sqrt{g n_0/m}$. In the units $\hbar = m = g = n_0 = 1$ used by the solver both are 1, so is the unit of time ζ/c , and the quantum of circulation is $\kappa = 2\pi$.

Write the wave function in polar form, $\psi = \sqrt{n} e^{iS}$, with $n = |\psi|^2$ the density and S the phase, and define the velocity

$$\mathbf{u} = \frac{\hbar}{m} \nabla S. \quad (3.3)$$

Substituting into (3.2) and separating the real and imaginary parts gives two real equations, found by Madelung in 1927 [41]:

$$\partial_t n + \nabla \cdot (n \mathbf{u}) = 0, \quad (3.4)$$

$$m(\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u}) = -\nabla(gn + Q), \quad Q = -\frac{\hbar^2}{2m} \frac{\nabla^2 \sqrt{n}}{\sqrt{n}}. \quad (3.5)$$

The first is conservation of mass. The second is Euler's equation for an inviscid compressible fluid of pressure $P = gn^2/2$, plus a *quantum pressure* Q , which is all that remains of $\hbar$ once ψ is written this way, and plus a constraint that no classical fluid obeys: $\mathbf{u}$ is a gradient, so $\nabla \times \mathbf{u} = 0$ wherever S is smooth. All the vorticity of a quantum fluid sits where S is not smooth. The dictionary is short: density $n = |\psi|^2$, velocity $\mathbf{u} = (\hbar/m)\nabla S$, pressure $gn^2/2$, quantum pressure Q (local, important only at scales $\lesssim \xi$), and vortex cores where $\psi = 0$ and S is undefined.

What the proof assistant says about it is limited and exact. The library does not contain (3.4) and (3.5), which are textbook and derived above in two lines. What `MadelungSplit` proves is the algebra underneath: the squared gradient of $\psi = a e^{iS}$ splits into an amplitude part and a phase part, so the kinetic energy density of ψ is the sum of the quantum-pressure term and the classical kinetic energy of the fluid.

LEAN 4 MADELUNGSPPLIT

```
-- namespace QuantumFluids.Madelung
/-- **Kinetic-energy density split with physical constants.** With `rho = a^2` and
    ↪ superfluid
velocity components `u_i = (hbar/m) d_i S`, the kinetic energy density
`(hbar^2/2m) |grad psi|^2` equals the quantum-pressure term `(hbar^2/2m) |grad a|^2`
plus the hydrodynamic term `(m/2) rho |u|^2`. -/
theorem kinetic_density_split (h m a ga gS : ℝ) (hm : m ≠ 0) :
  h ^ 2 / (2 * m) * (ga ^ 2 + a ^ 2 * gS ^ 2)
    = h ^ 2 / (2 * m) * ga ^ 2 + m / 2 * a ^ 2 * (h / m * gS) ^ 2
```

Here ga and gS stand for the components of ∇a and ∇S with $a = \sqrt{n}$. The statement of which this is the bookkeeping, `QuantumFluids.Madelung.madelung_gradient_split`, is proved for an amplitude–phase wave function on $\mathbb{R}^n$ under the hypothesis that a and S are differentiable at the point; that hypothesis is genuine, and it is exactly what fails at a vortex core, where $a = 0$ and S is undefined. The module `MadelungNSE` restates the Madelung velocity in the vocabulary of a Lean formalisation of the Navier–Stokes problem statement and proves $\text{div } \mathbf{u} = (\hbar/m)\Delta S$ there (`QuantumFluids.MadelungNSE.divergence_madelungVelocity`): bookkeeping, not a result about Navier–Stokes, which puts the two fluids in one formalism. That (3.4) and (3.5) look like the classical equations is an analogy; Section 3.6 says what the quantum fluid has that the classical one has not.

3.3 Quantized circulation

The circulation of the Madelung velocity (3.3) around a closed curve C on which $\psi \neq 0$ is

$$\Gamma(C) = \oint_C \mathbf{u} \cdot d\mathbf{l} = \frac{\hbar}{m} \oint_C \nabla S \cdot d\mathbf{l} = \frac{\hbar}{m} \Delta_C S, \quad (3.6)$$

that is, $\hbar/m$ times the change of the phase around C . The wave function must return to itself after one turn, so S does so modulo 2π : $\Delta_C S = 2\pi w$ with w an integer, and $\Gamma = w \hbar/m = w\kappa$. This is the argument of Onsager [15] and Feynman [16], and the integer w is the *winding number* of ψ along C . It is topological: neither g nor n nor the dynamics enters, which is why the same quantum appears in a liquid and in a dilute gas.

Neither a simulation nor a photograph provides S along a curve, only ψ at points. The method of essentially every code that finds vortices, including this book's, is the following. Take points $\mathbf{x}_0, \dots, \mathbf{x}_n = \mathbf{x}_0$ along the loop, read the phases $\theta_i = \arg \psi(\mathbf{x}_i)$, replace each step $\theta_{i+1} - \theta_i$ by its principal value $\text{pdiff}(\theta_i, \theta_{i+1}) \in (-\pi, \pi]$, and add. Two questions arise. Is the sum a multiple of 2π whatever the sampling? And is the multiple the winding number of the continuous field? The library answers the first in `VortexWinding` and `QuantizedCirculation`, the second in `ContinuumWinding`.

Layer one: an identity of sums. The principal difference differs from the true step by an integer multiple of 2π (`QuantumFluids.VortexWinding.pdiff_eq`), and on a closed loop the true steps telescope to $\theta_n - \theta_0 = 0$. Hence the sum of principal differences is an integer multiple of 2π (`QuantumFluids.VortexWinding.loop_sum_eq_mul`), and multiplying by $\hbar/m$ gives the following.

LEAN 4 QUANTIZEDCIRCULATION

```
-- namespace QuantumFluids.Circulation ; pdiff comes from QuantumFluids.VortexWinding
noncomputable def kappa (hbar m : ℝ) : ℝ := 2 * π * hbar / m

noncomputable def circulation (hbar m : ℝ) (S : ℕ → ℝ) (n : ℕ) : ℝ :=
  (hbar / m) * ∑ i ∈ Finset.range n, pdiff (S i) (S (i + 1))

/-- **Circulation is quantized.** Around any closed loop the circulation is an integer
    ↪ multiple of
the quantum `kappa`. -/
theorem circulation_quantized (hbar m : ℝ) (S : ℕ → ℝ) (n : ℕ) (hclosed : S n = S 0) :
  ∃ q : ℤ, circulation hbar m S n = (q : ℝ) * kappa hbar m := by
  obtain (q, hq) := loop_sum_eq_mul S n hclosed
  refine (q, ?_)
  unfold circulation kappa
  rw [hq]
  ring
```

The proof is the four lines shown; the content is in `QuantumFluids.VortexWinding.loop_sum_eq_mul`, whose proof is the telescoping argument just given (not shown). The same module proves `QuantumFluids.Circulation.abs_circulation_ge`, that a nonzero circulation has magnitude at least κ (no fraction of a quantum), and `QuantumFluids.Circulation.circulation_quantum_attained`, that κ itself is attained, by an explicit four-point loop whose last step wraps around by 2π , the branch behaviour that `pdiff` exists to handle. The bound is therefore sharp. What neither theorem says is that the integer has anything to do with the field: for an arbitrary sequence of phases the sum is just as much a multiple of 2π .

Layer two: the integer is the right one. The next theorem closes that gap for a loop of field values.

LEAN 4 CONTINUUMWINDING

```
-- namespace QuantumFluids.ContinuumWinding ; I = unit interval, [0,1]
/-- The `i`-th of `n` uniform sample points of the unit interval. -/
noncomputable def sample (n i : ℕ) : I := Set.projIcc 0 1 zero_le_one ((i : ℝ) / n)

/-- **The phase-winding vortex detector is correct.** For a continuous, nowhere-
    ↪ vanishing, closed loop
of field values, the sampled principal-branch loop sum of `arg` equals `2π` times the
    ↪ degree of the
loop, for every fine enough uniform sampling. -/
theorem detector_correct (c : C(I, ℂ)) (hne : ∀ t, c t ≠ 0) (hclosed : c 1 = c 0) :
  ∃ w : ℤ, ∃ n₀ : ℕ, 0 < n₀ ∧ ∀ n ≥ n₀,
    ∑ i ∈ Finset.range n, pdiff (Complex.arg (c (sample n i))) (Complex.arg (c (
      ↪ sample n (i + 1))))
    = (w : ℝ) * (2 * π)
```

In words: if the loop $t \mapsto c(t)$ is continuous, closed and never zero, there are an integer w (its degree) and a size n_0 such that every uniform sampling with $n \geq n_0$ points gives exactly $2\pi w$. The proof idea (abridged): the angle of c is a closed loop on the circle $\mathbb{R}/2\pi\mathbb{Z}$; lift it to $\mathbb{R}$ along the covering map (Mathlib’s path lifting); the lift is uniformly continuous on a compact interval (Heine–Cantor), so for large n consecutive samples differ by less than π , every principal difference *is* the true difference of the lift, the sum telescopes, and it equals the total change of the lift, $2\pi w$.

Two things the theorem does not give are what the solver supplies. It does not give n_0 , which depends on how fast $c/|c|$ turns; the file says so. And its hypothesis `hne` is where a vortex enters: on a loop through a zero of ψ the lift, the degree and the theorem cease to exist. The failure mode of the standard algorithm is also a theorem: traversing an edge forth and back cancels exactly, except when the phase difference is exactly π , where both directions return $+\pi$ and add up to 2π (`QuantumFluids.VortexWinding.pdiff_add_rev_eq_two_pi_iff`); then two neighbouring plaquettes of a grid fail to cancel their shared edge and a spurious winding can appear. No true phase step between neighbouring samples may reach π .

3.4 The vortex

3.4.1 A vortex is a zero of ψ

Let a loop C have winding $w \neq 0$. If ψ did not vanish anywhere on the disc bounded by C , shrinking C continuously to a point would keep the winding an integer-valued continuous function, hence constant, and a point has winding zero. So ψ vanishes somewhere inside. This is standard topology and it is *not* in the library: the pinned Mathlib has no argument principle for general continuous maps, as the module `ContinuumWinding` says. What the library has is the discrete substitute of Section 3.4.4 below.

3.4.2 Structure of one vortex

Around a singly quantized vortex at the origin the phase is the azimuth, $S = \theta$, so $\mathbf{u} = \hbar/(mr) \hat{\theta}$ *exactly*, whatever the density. The wave function is $\psi = \sqrt{n_0} f(r) e^{i\theta}$ where, in units of ξ ,

$$f'' + \frac{f'}{r} - \frac{f}{r^2} + 2f(1 - f^2) = 0, \quad f(0) = 0, \quad f(\infty) = 1. \quad (3.7)$$

A boundary-value solution (`figures/ch03_gpvortex.py`) gives $f \simeq 0.825 r/\xi$ near the axis and $n = n_0 f^2 \simeq n_0 (1 - \xi^2/2r^2)$ far from it. The velocity decays like $1/r$ and the density disturbance like $1/r^2$: the flow of a vortex is long ranged, its core short ranged. By the split of the previous section the energy inside a disc of radius $R \gg \xi$ about the core then has a flow part $\int \frac{1}{2} m n_0 u^2 d^2x = \pi(\hbar^2 n_0/m) \ln(R/\xi) + \text{const.}$, which grows without bound with R , and a quantum-pressure part confined to the core. A single vortex therefore has an energy that depends on the size of the system.

3.4.3 A pair, and the reduced model

A vortex–antivortex pair of separation d has no net circulation, so the logarithm is cut off at d :

$$E = 2\pi \frac{\hbar^2 n_0}{m} \ln \frac{d}{\xi} + 2E_c, \quad (3.8)$$

with E_c the energy of a core. Its momentum, the Kelvin impulse of the pair, is $P = m n_0 \kappa d = 2\pi \hbar n_0 d$. Hamilton's equations for a localized object then give its speed by one line.

THE PAIR MOVES AT $\hbar/(\kappa m d)$

Eliminating d between $E = 2\pi(\hbar^2 n_0/m) \ln(d/\xi) + 2E_c$ and $P = 2\pi \hbar n_0 d$ gives $E(P) = 2\pi(\hbar^2 n_0/m) \ln(P/2\pi \hbar n_0 \xi) + 2E_c$, and the speed of the pair is

$$U = \frac{dE}{dP} = \frac{2\pi \hbar^2 n_0/m}{2\pi \hbar n_0 d} = \frac{\hbar}{m d} = \frac{\kappa}{2\pi d}.$$

Helmholtz's picture gives the same answer: each vortex is carried by the velocity that the other induces at its position [42], $\kappa/2\pi d$, perpendicular to the line joining them.

The second derivation has a Lean statement. The point-vortex law, in which a vortex of circulation Γ at w induces the velocity $i\Gamma/(2\pi(\overline{z-w}))$ at z (complex notation), is textbook and was not in the library, so a small module was written for this chapter, `Ch03_PointVortexPair.lean` (new in this book, not part of the `QuantumFluids` library), which proves what the law implies for a pair.

LEAN 4 CHO3_POINTVORTEXPAIR (NEW)

```
-- namespace QuantumFluids.Ch03PointVortexPair
/-- Velocity, as the complex number `u + i v`, induced at `z` by a point vortex of
    ↪ circulation `Γ` at `w`. -/
noncomputable def induced (Γ : ℝ) (z w : ℂ) : ℂ :=
  Complex.I * (Γ : ℂ) / (2 * (Real.pi : ℂ) * (starRingEnd ℂ) (z - w))

/-- **Speed of a pair carrying one quantum of circulation**: with `Γ = κ = 2πħ/m` the
    ↪ speed is `ħ / (m d)`. -/
theorem pair_speed_quantum (ħ m : ℝ) (hħ : 0 < ħ) (hm : 0 < m) {z w : ℂ} (h : z ≠ w) :
  ||induced (2 * Real.pi * ħ / m) z w|| = ħ / (m * ||z - w||)
```

Besides `QuantumFluids.Ch03PointVortexPair.pair_speed_quantum` the module proves three facts that make up the point-vortex law for a pair:

- `QuantumFluids.Ch03PointVortexPair.induced_mul_conj`: the induced velocity times $\overline{(z-w)}$ is exactly $i\Gamma/2\pi$, so the velocity is perpendicular to the line joining the two points and has modulus $|\Gamma|/2\pi|z-w|$;
- `QuantumFluids.Ch03PointVortexPair.pair_velocities_equal`: a vortex of circulation $-\Gamma$ and one of circulation $+\Gamma$ induce the *same* velocity on each other, so a pair translates rigidly;
- `QuantumFluids.Ch03PointVortexPair.separation_const`: along any differentiable motion obeying the two induced-velocity equations the separation is constant.

Nothing in the module concerns the Gross–Pitaevskii equation, the periodic box or more than two vortices; whether the Gross–Pitaevskii vortex obeys the law is what the solver is asked in Section 3.5. The module compiles with zero errors and uses only the standard axioms (Section 3.6). A real pair is not made of point vortices anyway: it is a solitary wave of the compressible field [24], whose speed departs from $\hbar/(md)$ at small d , by a correction that the programme's friction study puts at order $(\xi/d)^2$ [23]; Section 3.5.5 measures where it starts to matter.

3.4.4 What a loop sees: scale-resolved winding

A loop of radius R around one member of the pair sees winding ± 1 ; a loop that encloses both sees 0. The winding as a function of the scale of the loop is a step at $R = d$, and a dipole is invisible from far away. The library proves this for any region of a grid, in the discrete form in which a code computes it: a region is a finite family of plaquettes, each with four corner phases, whose interior edges come

in reversed pairs. If the charges of the plaquettes sum to zero the winding along the boundary is zero whatever the size of the region; if exactly one plaquette is charged, the boundary winding is its charge at every scale.

LEAN 4 SCALERESOLVEDWINDING

```
-- namespace QuantumFluids.ScaleResolvedWinding ; variable {m : ℕ} (Rg : Region m)
/-- **A dipole is invisible from outside.** If the charges inside sum to zero, the
    ↪ boundary winding
is zero, whatever the size of the region. -/
theorem dipole_invisible (hcut : ∀ e ∈ Rg.interior, Rg.step e ≠ π) (q : Fin m → ℤ)
  (hq : ∀ p, Rg.winding p = (q p : ℝ) * (2 * π)) (hneutral : ∑ p, q p = 0) :
  Rg.boundarySum = 0
```

The hypothesis `hcut` is again the branch-point condition: no interior edge may carry a phase step of exactly π . The proof is a discrete Stokes theorem: the boundary sum equals the sum of the plaquette loop sums (`QuantumFluids.ScaleResolvedWinding.stokes`), because every interior edge is traversed once in each direction.

3.5 The solver: vortices in the projected Gross–Pitaevskii field

3.5.1 The engine and the imprint

The engine is `qf-pgpe`, the Rust quantum-fluids crate of the rusty-SUNDIALS workspace, called through its Python extension `qf_pgpe` (Chapter 2). It integrates the projected Gross–Pitaevskii equation $i \partial_t \psi = \mathcal{D}[-\frac{1}{2} \nabla^2 \psi + |\psi|^2 \psi]$ [19, 20] in the units of Section 3.2 on a periodic square of side $L = 64 \xi$ with 128×128 grid points ($\Delta x = 0.5 \xi$). The projector $\mathcal{D}$ keeps the 3209 Fourier modes with $|\mathbf{k}| \leq k_{\text{cut}} = \pi/\xi$, so the field is a band-limited function that can be evaluated at any point from its mode sum. The integrator is the integrating-factor Runge–Kutta scheme of order four with $\Delta t = 0.01$. Vortices are placed by the engine’s `imprint_v2`: a product of Jacobi ϑ_1 functions makes the phase periodic on the torus, the density is the Bernoulli profile of the imprinted flow, and the phase jump that a pair would otherwise leave across the box edge is removed by a uniform counterflow (the origin of Section 3.5.5). They are found by `detect`: the principal-difference winding of every grid plaquette, then a sub-grid refinement of the position, the detector that `ContinuumWinding` is about.

RUSTY-SUNDIALS A VORTEX PAIR IN `qf_pgpe`

```
import numpy as np, qf_pgpe
s = qf_pgpe.Pgpe(128, 64.0) # n, L ; g = 1, dt = 0.01, k_cut = pi
pos0 = np.array([26.13, 30.71], [38.13, 30.71]); q = np.array([1, -1]) # separation
12, + on the left
```

```

c = s.imprint_v2(s.uniform(), pos0, q)      # periodic theta-function phase, Bernoulli
      density
for k in range(41):                          # t = 0, 1, ..., 40
    if k > 0:
        c = s.run(c, 1.0)                  # 100 IF-RK4 steps inside one Rust call
        dp, dq = s.detect(c)                # plaquette windings -> sub-grid positions
        and charges
    E, N, P = s.energy(c), s.norm(c), s.momentum(c)

```

Condensed from `figures/ch03_run.py`, which also tracks each vortex between snapshots. Output of the run used below (positions of the + and – vortex, energy E , norm N , momentum P_y):

t= 0	+(26.16, 30.72)	-(38.13, 30.72)	E=2069.120127	N=4096.000000	Py=72.9991
t=20	+(26.22, 32.58)	-(38.04, 32.58)	E=2069.120127	N=4096.000000	Py=72.9991
t=40	+(26.25, 34.46)	-(38.02, 34.47)	E=2069.120127	N=4096.000000	Py=72.9991

Energy, norm and momentum are constants of the motion of the projected equation, and the run bears this out: over 40 time units E changes by a relative 6×10^{-11} , N by 3×10^{-11} and P_y by 4×10^{-11} . All timings in this chapter come from a shared eight-core machine whose load average was between 14 and 21 when the solver runs started; they are not benchmarks. A run of 40 time units took 41 to 77 s on it.

3.5.2 Seeing the fluid in ψ

Figure 3.1 shows one snapshot of the pair at $t = 20$: the same complex array ψ read as a phase, as a density and as a velocity. The cores are the dark dots of panel (a); around each of them the phase runs through all its values once, in opposite senses for the two; the streamlines of $\mathbf{u} = \text{Im}(\bar{\psi} \nabla \psi) / |\psi|^2$ circle each core and join, between the vortices, into the jet that carries the pair upward.

Three checks make the reading quantitative. First, the grid-level detector finds exactly 1 plaquette of charge +1, 1 of charge –1 and 0 of any other charge, and the same on a grid refined four-fold by spectral interpolation. The positions it reports are within 0.005ζ of exact zeros of the band-limited field: minimizing $|\psi|^2$ from the detected position finds points where $|\psi|^2 \leq 2 \times 10^{-29}$, so the cores are zeros of ψ , as Section 3.4.1 requires. The largest principal phase step over all grid edges is 0.81π (0.71π on the refined grid), so no edge is near the branch point of Section 3.3.

Second, panel (c): the relative rms difference between the Madelung velocity of the solver and the periodic point-vortex velocity (with the uniform flow of Section 3.5.5) is 1.7% in the shell 1.5–3 ζ about the cores and 2.2% in the shell 3–6 ζ ; without the uniform flow the second number is 8.1%. The phase of one Gross–Pitaevskii vortex is the azimuth exactly, which is why the point-vortex law holds so close to the core; I did not isolate the origin of the remaining two percent. Farther out the difference is 0.013 rms beyond 6 ζ , against a point-vortex speed of only 0.042, and it is not vortex flow: 99.9% of its energy is curl-free and its rms equals that of the density ripple, 0.012 (the sound speed is 1). It is the sound radiated while the imprinted state relaxes, visible as rings in panel (b) and absent from the point-vortex model.

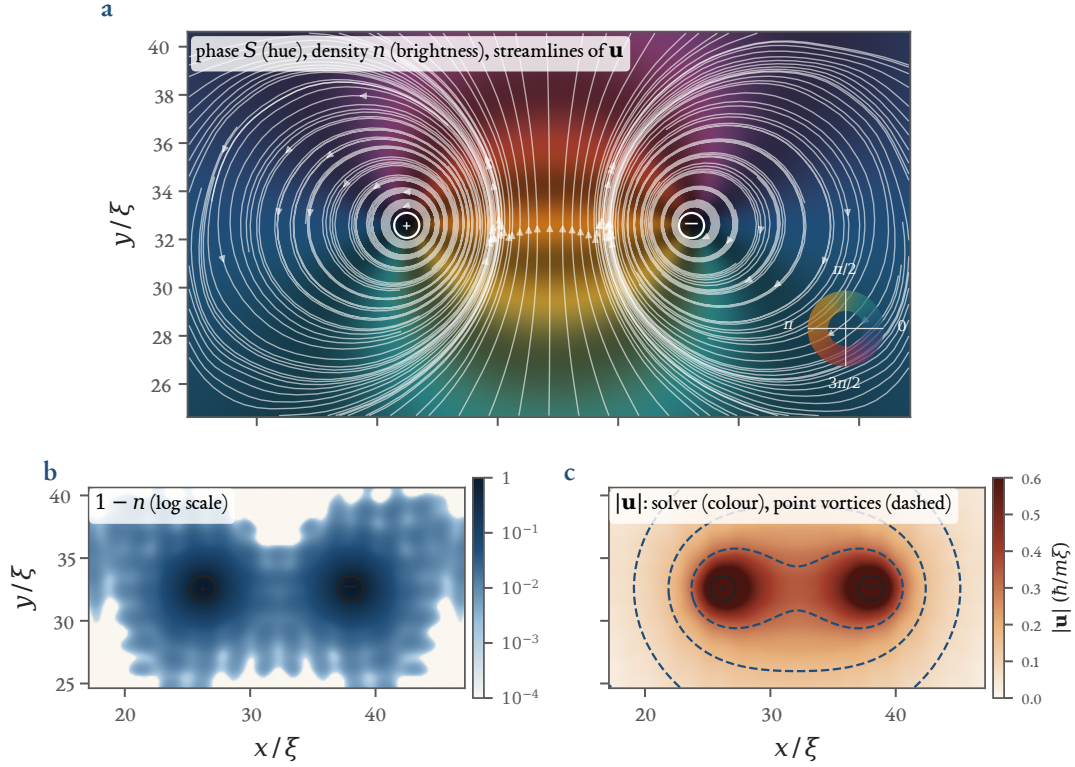


Figure 3.1: One wave function, three faces (qf-pgpe, $t = 20$, pair of separation $d \approx 12 \xi$, + vortex on the left). **(a)** Phase of ψ (hue; global phase e^{-it} removed), density (brightness), streamlines of the Madelung velocity (white). **(b)** Density deficit $1 - n$, logarithmic; the faint rings are radiated sound. **(c)** Speed $|\mathbf{u}|$: solver (colour) and periodic point vortices with their uniform flow (dashed contours at 0.04, 0.08, 0.16, 0.32).

Third, the first Madelung equation (3.4). Evaluating $\partial_t n$ and $\nabla \cdot (n\mathbf{u})$ with the engine's own right-hand side (no time stepping, no tracking), the continuity equation holds to a relative rms residual of 6.8% of $|\partial_t n|$ outside 3ξ from the cores. The residual is not noise: for the *projected* equation it equals $-2 \operatorname{Im}(\bar{\psi}(1 - \mathcal{P})[|\psi|^2 \psi])$, the part of the cubic term that the projector removes, an identity the script checks to 8×10^{-14} (largest residual 0.012). The engine conserves the total norm, not the local continuity equation.

3.5.3 Counting quanta

Now the quantization of Section 3.3 on the solver field. Around the + vortex, at its detected position, take circles of radius R from 0.15ξ to 27.8ξ (39 loops), evaluate ψ and $\nabla \psi$ at 1600 points of each circle from the exact mode sum (not the grid), and compute the two quantities that the theory ties together: the circulation integral Γ/κ with the Madelung velocity, and the winding found by the sum of principal differences of $\arg \psi$.

RUSTY-SUNDIALS TWO WAYS TO COUNT THE QUANTA

```

def pdiff(a, b):
    d = np.mod(b - a + np.pi, 2 * np.pi) - np.pi      # in [-pi, pi)
    return np.where(d == -np.pi, np.pi, d)

def loop_winding(ev, cx, cy, r, M=720):
    p = ev.psi(x, y); th = np.angle(p)
    st = pdiff(th, np.roll(th, -1))
    return st.sum() / (2 * np.pi), np.abs(p).min(), np.abs(st).max()

def loop_circulation(ev, cx, cy, r, M=720):
    p, px, py = ev.psi_grad(x, y)
    n = np.abs(p) ** 2
    ux = np.imag(np.conj(p) * px) / n; uy = np.imag(np.conj(p) * py) / n
    tx, ty = -np.sin(t), np.cos(t)
    return float(np.sum((ux * tx + uy * ty) * r * (2 * np.pi / M)))

```

Docstrings omitted. `pdiff` is the numpy form of `QuantumFluids.VortexWinding.pdiff`; `ev` evaluates the band-limited field and its gradient at any point from the mode sum.

The result, Figure 3.2(a,b), is as sharp as the theorem allows. For every loop that encloses only the $+$ vortex (R from 0.15 to 10.3ξ) $\Gamma/\kappa = 1$ to within 4×10^{-16} ; for every loop that encloses both ($R > d + 1.4 \xi$) it is zero to within 3×10^{-16} . The circulation of the solver is quantized in units of κ to the accuracy of the quadrature, and the pair is invisible from outside, as `QuantumFluids.ScaleResolvedWinding.dipole_invisible` says it must be. The step from 1 to 0 takes place across the other core, where what limits the numbers is the quadrature and not the quantization: within 0.2ξ of that core the integrand is sharply peaked and 1600 points no longer resolve it ($|\Gamma/\kappa - \text{integer}| = 5 \times 10^{-12}, 3 \times 10^{-6}$ and 2×10^{-3} at distances 0.2, 0.1 and 0.05ξ). The detector, which only compares phases at consecutive samples, returns the right integer on all 39 loops, including those three.

Panel (c) answers the question that `QuantumFluids.ContinuumWinding.detector_correct` leaves open: how large is n_0 ? Sample a circle with M equally spaced points, for every divisor M of 7200 and from one of six generic starting angles, and ask for the smallest M_0 from which every finer sampling gives the right winding; the worst of the six angles is plotted. A bare vortex needs $M_0 = 3$, since its step $2\pi/M$ must stay below π . If the other core lies *inside* the loop, M_0 stays at 3 at every distance tested, down to $\delta = 0.05 \xi$. If it lies *outside*, M_0 grows as the loop approaches it, from 3 at $\delta = 8 \xi$ to 36 at $\delta = 0.05 \xi$, as $\delta^{-0.51}$. The reading is the hypothesis of the Lean theorems in quantitative form: the detector goes wrong exactly when the *true* phase change between neighbouring samples reaches π . Along an arc of length $\ell = 2\pi R/M$ passing a zero at distance δ , the phase of that zero changes by up to $\pi - 4\delta/\ell$, adding to the loop's own step $2\pi/M$ if the zero has the opposite charge and lies outside the loop, and subtracting from it if the zero lies inside.

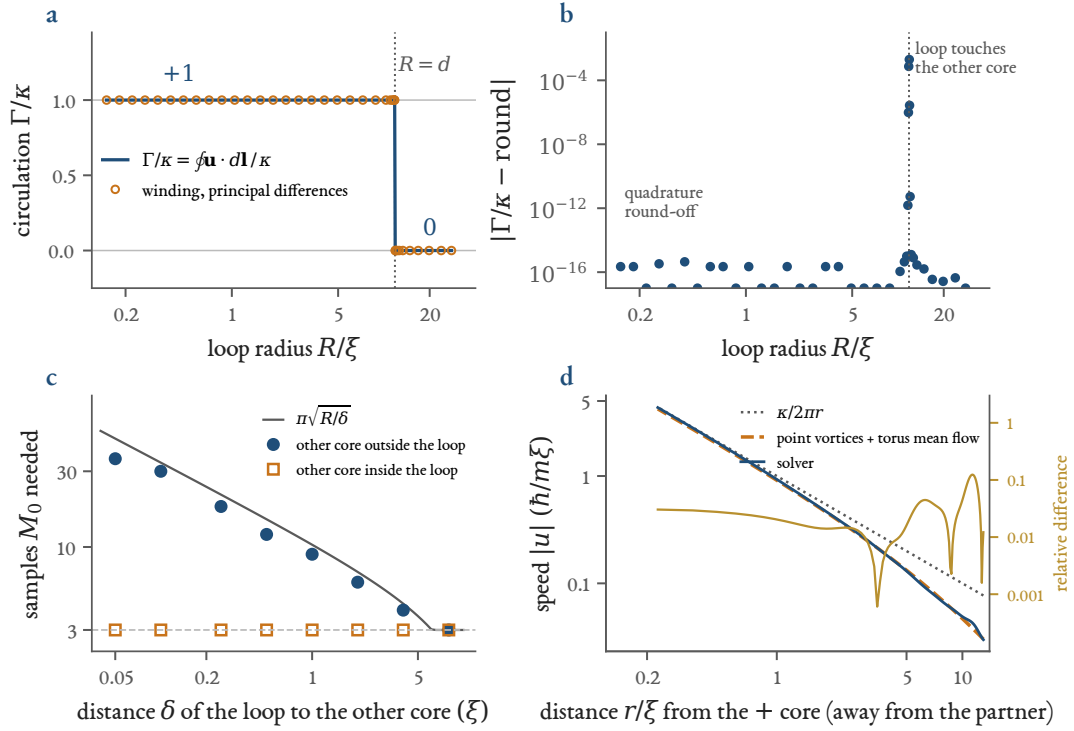


Figure 3.2: Counting quanta on the solver field ($d \approx 12 \xi$, $t = 20$, circles about the $+$ vortex). **(a)** Circulation Γ/κ (line) and detector winding (circles) against loop radius. **(b)** Distance of Γ/κ from the nearest integer. **(c)** Smallest number of samples M_0 from which the principal-difference sum is right for every finer sampling, for a loop at distance δ from the other core, outside (blue) or inside (orange) the loop; worst of six starting angles. **(d)** Speed along the line through both cores, on the side of the $+$ vortex away from its partner, against the distance r to the core: solver, periodic point vortices, and $\kappa/2\pi r$ (dotted); gold, right axis: relative difference of the first two.

The sum reaches π once $M \lesssim \pi\sqrt{R/\delta}$, and the measured M_0 are 0.75 to 0.91 times that. This derivation is mine, checked against these numbers, and is not in the library. So n_0 is finite, as the theorem says, but not a number the theorem could know: it depends on the distance to the nearest zero of ψ and on whether that zero helps or opposes the winding being measured.

The torus has its own windings. A loop does not have to be contractible. On the periodic grid the sums of principal differences along each of the 128 rows and 128 columns are again integers (to 7×10^{-16}), by the same theorem: all row windings are 0, and the column windings are 1 for the 24 columns between the two vortices and 0 elsewhere. The integer pair measured outside that strip is the *sector* of the state. The circulation of $\mathbf{u}$ around a cycle of the torus is 2π times it, so the *mean* velocity over the box is fixed by the sector and the vortex positions, not by dynamics. For this pair the column windings give $\langle u_y \rangle = 2\pi(24 \times 0.5 \xi)/L^2 = 0.0184$, the integral of the Madelung velocity over the grid gives 0.0182, and the position formula of Section 3.5.5 gives 0.0181; they

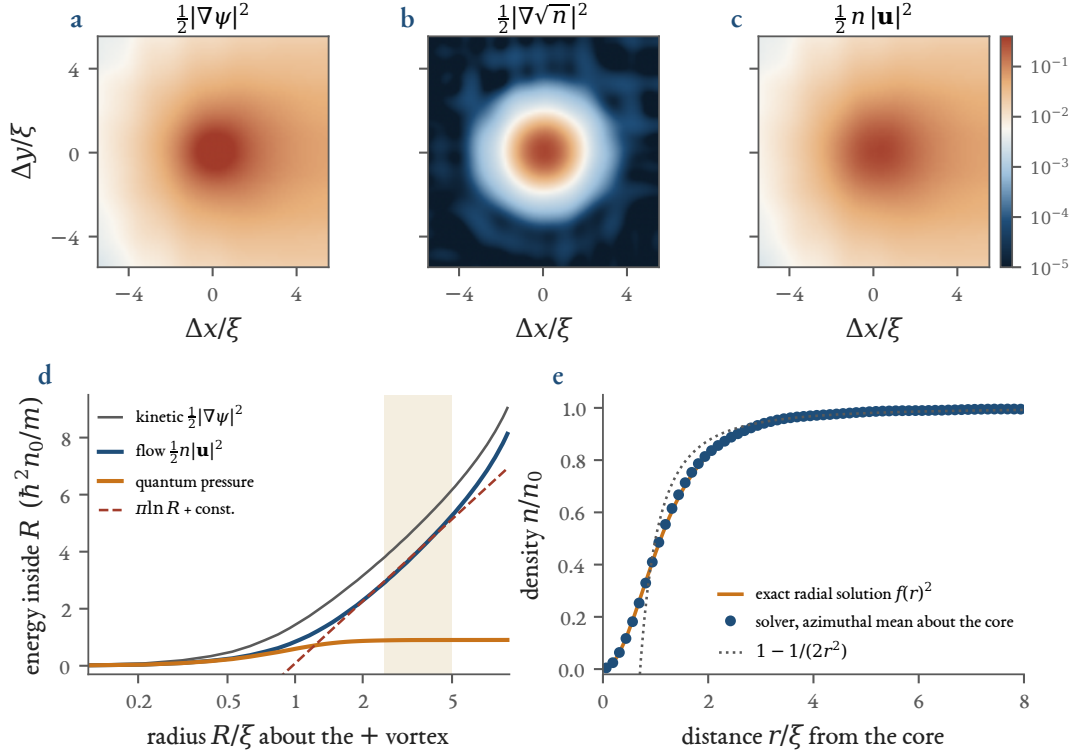


Figure 3.3: Where the energy of a vortex sits ($d \approx 12\xi$, $t = 20$, refined grid). **(a–c)** Kinetic, quantum-pressure and flow energy density near the + vortex, one logarithmic scale. **(d)** Energy inside radius R about the core: the flow part follows $\pi \ln R$ in the shaded window and keeps growing, the quantum-pressure part saturates inside 3ξ . **(e)** Azimuthal mean density against the exact radial solution of (3.7).

agree to the width of one grid column. This small uniform flow has a large effect.

3.5.4 Where the energy sits

The identity of `QuantumFluids.Madelung.kinetic_density_split` can be evaluated on the field: split $\frac{1}{2}|\nabla\psi|^2$ at every point into $\frac{1}{2}|\nabla\sqrt{n}|^2$ and $\frac{1}{2}n|\mathbf{u}|^2$, on the refined 512² grid, and integrate. The pointwise difference between the kinetic density and the sum of the two parts is 5×10^{-16} of its maximum (on the solver grid itself, 4×10^{-16}): an algebraic identity holding to rounding error, not a measurement. What is measured is the partition. The integral of $\frac{1}{2}|\nabla\psi|^2$ is 17.76 in units of $\hbar^2 n_0/m$ (the Parseval sum of the Fourier amplitudes gives 17.76); of this, 15.86 (89.3%) is flow and 1.90 (10.7%) is quantum pressure. The total energy of the pair above the uniform state is 21.12; the remainder, $\frac{1}{2} \int (n - 1)^2 d^2x$, is the interaction energy of the density disturbance.

Figure 3.3 shows where the two parts live. The quantum pressure is a ring around the core: inside 3ξ it holds 0.90 and inside 6ξ 0.90, so it has saturated, whereas the flow energy inside the same

radii is 3.48 and 6.02 and keeps growing. Fitting the flow energy against $\ln R$ for $2.5 \leq R/\xi \leq 5$ gives a slope of 3.40 about the $+$ vortex and 3.40 about the $-$ vortex. The vortex's own azimuthal flow $\hbar/(mr)$ contributes 3.01, to be compared with $\pi\bar{n} = 3.04$ for the measured mean density $\bar{n} = 0.966$ on the circle $R = 3.75\xi$; the partner's flow and the uniform flow inside the disc contribute 0.39, the cross term -0.001 , and the three add up to the measured slope. Panel (e) puts the core against its exact profile: they differ by at most 0.010 in density over $r \leq 8\xi$ ($n(1\xi) = 0.447$ against 0.445). I did not isolate the origin of this one-percent difference.

3.5.5 The speed of a pair, and the lesson of the periodic box

The Lean module of Section 3.4.3 says that a point-vortex pair of separation d moves at $\hbar/(md)$. Does the Gross–Pitaevskii pair in a periodic box of side 64ξ do so? The solver was run for six separations, $d = 4, 6, 8, 12, 16, 20\xi$ (40 time units each; $d = 12\xi$ is the run of the previous figures); the speed is the slope of the position of the pair's centre over $10 \leq t \leq 40$, the first ten time units containing the relaxation of the imprint. Three predictions can be made, each with one more ingredient than the last.

1. *The plane law*, $v = \hbar/(md)$.
2. *Point vortices on the torus*. Each vortex is carried by the velocity of the other and of all its periodic images. In Fourier space the velocity of vortices of charges q_j at $\mathbf{x}_j$ is $\hat{\mathbf{u}}(\mathbf{k}) = i(k_y, -k_x) \hat{\rho}(\mathbf{k})/|\mathbf{k}|^2$ with $\hat{\rho}(\mathbf{k}) = (\kappa/L^2) \sum_j q_j e^{-i\mathbf{k} \cdot \mathbf{x}_j}$, and the term $\mathbf{k} = 0$ is absent: this is the field with *zero mean flow*. The programme's earlier code has the same field in closed form (the Weiss–McWilliams pair function); the two agree to 6×10^{-15} on the test configuration of the box below.
3. *Plus the uniform flow that the windings of the torus force*. With positions taken in $[0, L)$, the circulation of the zero-mean-flow field around the x -cycle just above the line $y = 0$ is $-(\kappa/L) \sum_j q_j y_j$, and around the y -cycle just above $x = 0$ it is $+(\kappa/L) \sum_j q_j x_j$, which are not integer multiples of κ . The wave function requires 2π times an integer, and the integers measured in Section 3.5.3 are zero, so a uniform flow must make up the difference:

$$\bar{\mathbf{u}} = \frac{\kappa}{L^2} \left(\sum_j q_j y_j, -\sum_j q_j x_j \right). \quad (3.9)$$

For a pair of separation d it has magnitude $\kappa d/L^2$ and the direction of the pair's own motion; it is the impulse of the pair divided by $m n_0 L^2$. In units of the plane speed it is $\bar{u}/(\hbar/md) = 2\pi d^2/L^2$, that is 21% at $d = 12\xi$ and 61% at $d = 20\xi$.

RUSTY-SUNDIALS THE PERIODIC POINT-VORTEX MODEL

```
def sector_flow(pos, q, L=L):
    pos = np.atleast_2d(pos).astype(float); q = np.asarray(q, float)
    return (KAPPA / L ** 2) * np.array([np.sum(q * pos[:, 1]), -np.sum(q * pos[:, 0])])
```

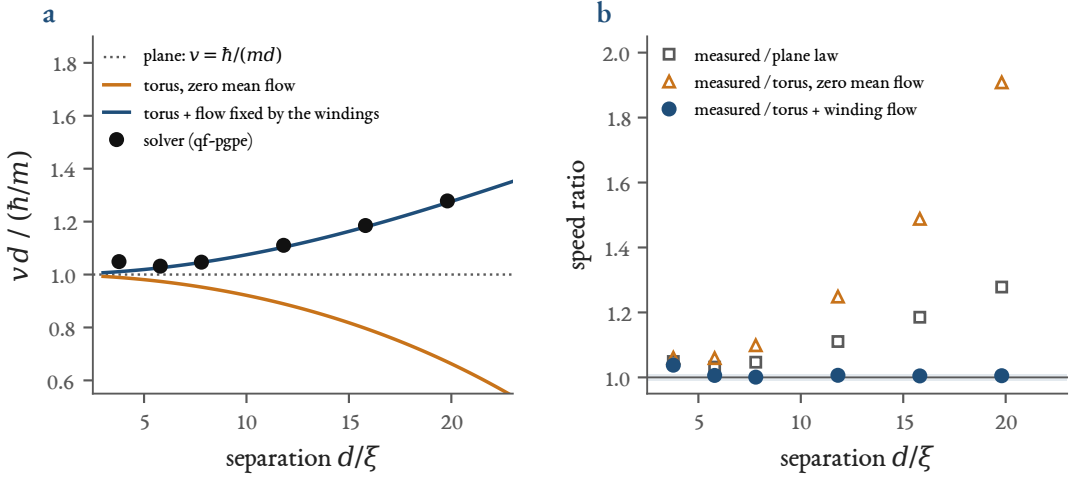


Figure 3.4: The speed of a vortex pair in the periodic box against its separation (six solver runs of 40 time units). **(a)** $v d / (\hbar/m)$: the plane law ($= 1$), point vortices on the torus with zero mean flow (orange), the same plus the uniform flow (3.9) fixed by the windings (blue), and the measurements (black). **(b)** Ratio of the measured speed to each prediction; the band is $\pm 1\%$.

d/ξ	measured	plane $1/d$	torus, zero flow	torus + winding flow	measured / plane	zero flow	+ flow
3.8	0.2778	0.2648	0.2619	0.2677	1.049	1.061	1.038
5.8	0.1781	0.1726	0.1681	0.1770	1.032	1.059	1.006
7.8	0.1342	0.1282	0.1221	0.1341	1.047	1.099	1.001
11.8	0.0940	0.0847	0.0753	0.0934	1.110	1.249	1.006
15.8	0.0750	0.0633	0.0504	0.0746	1.185	1.488	1.005
19.8	0.0645	0.0505	0.0338	0.0642	1.278	1.908	1.005

Table 3.1: The measured speed of the pair against the three predictions (units $\hbar/m\xi$ per unit time; d is the separation averaged over $10 \leq t \leq 40$; the predictions are evaluated at the tracked positions and averaged over the same window).

```
def vortex_velocity(pos, q, with_sector=True, L=L):
    pos = np.atleast_2d(pos).astype(float); q = np.asarray(q, float)
    v = pv_velocity(pos, q, L)
    if with_sector:
        v = v + sector_flow(pos, q, L)
    return v
```

`pv_velocity` is the programme's zero-mean-flow field (`exploration/pgpe/transport_estimators.py`); the Fourier-series version `fourier_velocity` of `figures/ch03_pv.py` agrees with it to 6×10^{-15} on a test configuration (+ at (26.30, 30.70), − at (38.10, 31.20); recorded in `ch03_numbers.json`).

Figure 3.4 and Table 3.1 give the result. The measured speed exceeds the plane law by 3.2 to

4.9% for $d \leq 8\zeta$ and by 11%, 19% and 28% at $d = 12, 16, 20\zeta$. The torus point-vortex speed with zero mean flow is too small, and increasingly so with d : the measured speed is 1.25, 1.49 and 1.91 times it at $d = 12, 16, 20\zeta$, so the discrepancy is not a core effect, which would shrink with d . With the uniform flow (3.9) the ratios are 1.006, 1.001, 1.006, 1.005 and 1.005 for $d \geq 6\zeta$: the prediction reproduces the solver to about one percent, with no fitted parameter. At $d \approx 4\zeta$ the measured speed is 3.8% above it; there the pair is a solitary wave of the compressible field rather than two point vortices [24]. That is the sign and size of the discrepancy, not a derivation of it.

The same effect appeared in the programme's study of friction on vortices, where it was first misread: the ratios of the box-frame speed to the zero-mean-flow prediction there (1.06, 1.06, 1.10 and 1.49 at $d = 4, 6, 8$ and 16ζ , in its table of $T = 0$ kinematics [23]; different runs and imprint position) are the numbers measured here. The lesson is general. A vortex pair carries impulse, and a periodic box has no infinity to which to assign it, so the fluid as a whole moves; the amount is a property not of the dynamics but of the integers that wind the phase around the torus, whose existence `QuantumFluids.VortexWinding.loop_sum_eq_mul` guarantees.

The field momentum is conserved ($P_y = 73.00$, relative change 4×10^{-11}) but is 98.3% of the impulse $2\pi d = 74.26$ at $t = 20$. The difference is the density deficit: the momentum is $\int n u_y d^2x$, and the integral of u_y alone over the box, $L^2 \times 0.0182$, exceeds it by $\int (1 - n) u_y d^2x$, which is 2.0% of it. The separation shrinks by 1.6% over the run, so the impulse of the vortices falls while the field momentum does not; the balance must be carried by the sound field.

3.5.6 Leapfrogging pairs, and the reduced model in CVODE

Two pairs of the same sense, one behind the other, give a motion familiar from classical hydrodynamics as the leapfrogging of vortex rings. The rear pair is drawn forward by the flow of the front pair, narrows as it accelerates, passes through the front pair, which has widened and slowed, and then widens in its turn while the other narrows. The solver was started with four vortices $+, -, +, -$ at the corners of a square of side 10.3ζ and run for 120 time units; Figure 3.5 shows the wave function at six times and the tracked vortices.

The reduced model is the periodic point-vortex model of Section 3.5.5: four vortices obeying $\dot{\mathbf{x}}_a = \mathbf{u}_a(\mathbf{x}) + \bar{\mathbf{u}}$, with $\mathbf{u}_a$ the zero-mean-flow field of the other three and their images and $\bar{\mathbf{u}}$ the uniform flow (3.9), constant here because $\sum_j q_j \mathbf{x}_j$ is conserved. It is integrated with CVODE from the positions detected at $t = 0$.

RUSTY-SUNDIALS CVODE INTEGRATES THE POINT-VORTEX MODEL

```
def rhs(t, y):
    nfe[0] += 1
    return vortex_velocity(np.array(y).reshape(n, 2), q).ravel().tolist()

solver = CvodeSolver(method, rtol, atol, 1_000_000)
y = np.asarray(pos0, float).ravel().tolist()
```

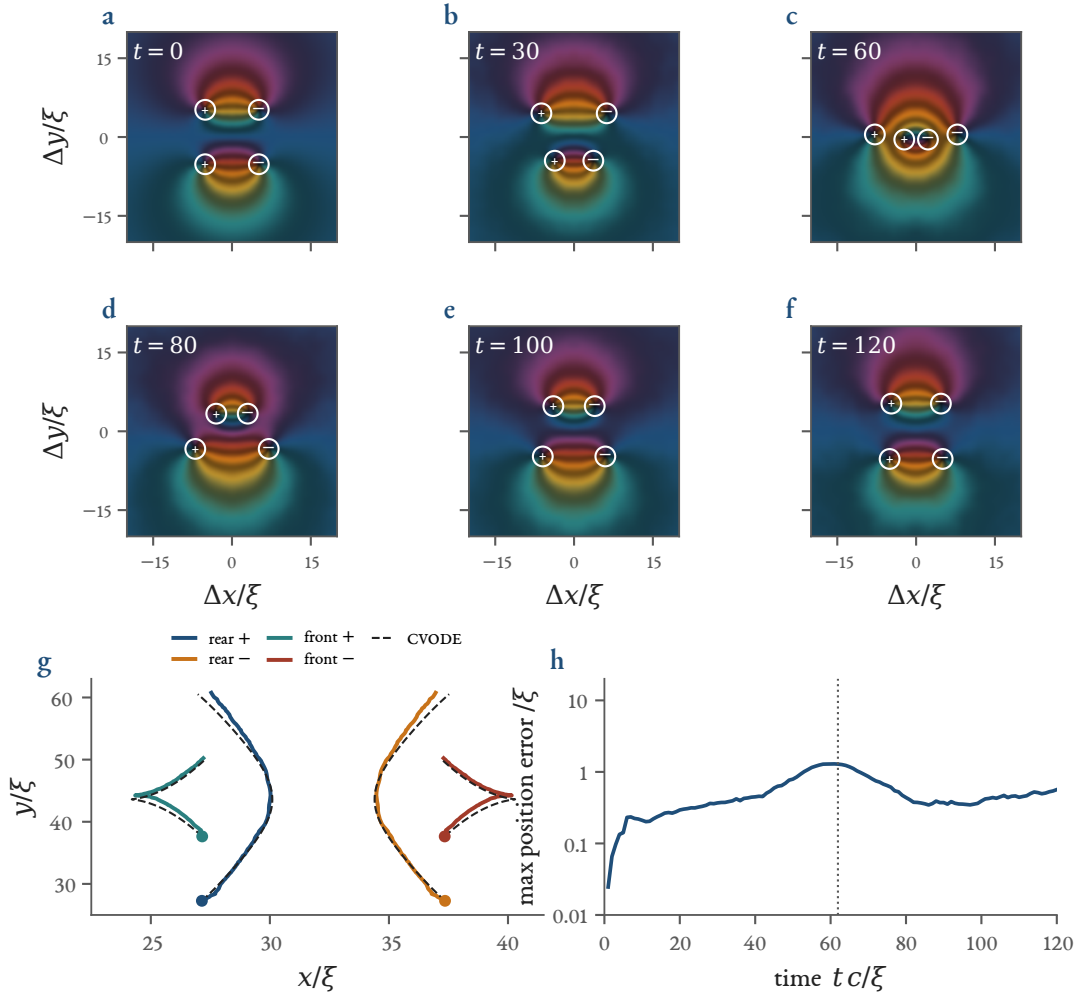


Figure 3.5: Two vortex pairs leapfrog (qf-pgpe, $L = 64 \xi$, 120 time units). **(a–f)** The wave function at six times (colour code of Figure 3.1), the view re-centred on the four vortices. **(g)** Tracked vortices (solid) and the periodic point-vortex model integrated with CVODE (dashed) from the detected positions at $t = 0$ (dots). **(h)** Largest distance between a tracked vortex and its model trajectory; dotted: the time at which the rear pair passes the front pair in the solver.

```
out = [np.array(y).reshape(n, 2)]
for t0, t1 in zip(t_grid[:-1], t_grid[1:]):
    _, y = solver.solve(rhs, float(t0), y, float(t1))
    out.append(np.array(y).reshape(n, 2))
```

Excerpt of `integrate_cvmode` (`figures/ch03_pv.py`; `rhs` advances the state, the loop restarts CVODE at each output time). Run with the rusty-SUNDIALS build of commit 5db8041, `rusty_sundials.__file__` recorded in `ch03_numbers.json`. Result for the four-vortex run: 5 028 right-hand-side calls in 16 s for 120 time units on the

loaded machine (the right-hand side is a Python function called through the binding); an independent integration of the same equations with SciPy's DOP853 at tolerance 10^{-12} differs from the CVODE trajectory by at most $4 \times 10^{-7} \zeta$ over the whole run, and the dipole moment $\sum_j q_j \mathbf{x}_j$, conserved by the equations, is conserved by CVODE to 1×10^{-13} .

Against the solver the model does well, and one cause of the difference can be measured. The rear pair narrows from 10.2ζ to 4.44ζ in the solver (4.30ζ in the model) and the front pair widens to 15.8ζ (16.1ζ); the rear pair passes the front pair at $t = 61.9$ in the solver and at $t = 58.2$ in the model, 3.8 time units (6%) earlier; by $t = 120$ the roles are exchanged, the former rear pair being 10.5ζ ahead in the solver. The largest distance between a tracked vortex and its model trajectory is 0.21, 0.29 and 0.45ζ at $t = 10, 20, 40$, rises to 1.29ζ at the passage, where a small error in timing becomes a large error in position because the vortices move fastest, and is 0.39 and 0.57ζ at $t = 100, 120$.

Where does the difference come from? Part of it is the imprint: the imprinted state relaxes, and the error is already 0.22ζ at $t = 8$. Restarting the model from the tracked positions at a later time tests this (with DOP853, which takes seconds). Started at $t = 0, 10, 20, 40$ the model passes the front pair at $t = 58.2, 59.4, 60.7, 61.5$ against 61.9 in the solver, and its largest error over the rest of the run is 1.29, 1.17, 0.68, 0.40ζ ; ten time units after a restart at $t = 20$ it is 0.04ζ . So most of the passing-time discrepancy is the first twenty to forty time units of relaxation, not an inability of point vortices to leapfrog. What is left (a restart at $t = 40$: 0.4ζ at worst, a passing time within 0.5 time units) is what point vortices lack, the core structure and the compressibility of the field; I did not separate the two. The closest approach inside a pair is 4.44ζ , where the speed excess of the previous subsection lies between 0.6% and 3.8%. The impulse $\sum_j q_j \mathbf{x}_j$ is conserved exactly in the model and falls by 4.3% over the run in the solver, while the total field momentum is conserved to 1×10^{-10} and the energy to 2×10^{-10} .

3.6 What this chapter establishes, and what it does not

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved in Lean (standard axioms `propext`, `Classical.choice`, `Quot.sound`: by the programme's compile audit for `VortexWinding`, `MadelungSplit` and `MadelungNSE`; re-compiled for this chapter for `QuantizedCirculation`, `ContinuumWinding` and `ScaleResolvedWinding`). Sums of principal phase differences around a closed loop are integer multiples of 2π , so the discrete circulation is $q\kappa$, a nonzero one has $|\Gamma| \geq \kappa$, and κ is attained; for a continuous nowhere-vanishing closed loop the detector returns $2\pi w$ for every fine enough sampling, n_0 unquantified; the kinetic energy density of $a e^{iS}$ splits into an amplitude and a phase term; $\operatorname{div} \mathbf{u} = (\hbar/m)\Delta S$ in a Navier–Stokes vocabulary; a dipole is invisible to the discrete winding of any enclosing region. *New here* (`book/lean/Ch03_PointVortexPair.lean`, zero errors, same three axioms): the

point-vortex law implies that a $\pm\Gamma$ pair translates rigidly at $\hbar/(md)$ for $\Gamma = \kappa$, at constant separation.

Not in Lean, and textbook: the Madelung equations; the vortex profile (3.7); the energy and impulse of a pair and $U = dE/dP$; that a Gross–Pitaevskii vortex moves with the velocity induced by the others; that a nonzero winding implies a zero of ψ inside.

Simulated (qf-pgpe at 128^2 , $k_{\text{cut}} = \pi/\xi$, zero temperature, one initial condition per case, up to 120 time units; CVODE for the reduced model). Circulation is an integer to 4×10^{-16} and the detector is right on all 39 loops; 89.3% of the kinetic energy of the pair is flow; the pair speed equals the torus point-vortex prediction with the winding-fixed flow to about 1% for $d \geq 6\xi$ (3.8% above it at $d \approx 4\xi$); the model follows the leapfrog to 1.29ξ at worst over 120 time units, with a 6% error in the passing time, mostly the relaxation of the imprint.

Analogy only. The Madelung equations have the form of those of a classical inviscid fluid, and vortex pairs in the quantum fluid move like pairs in the classical one. LL-15: what the two systems must share for this to transfer is a velocity that is a gradient away from a few singular points that the flow carries; the quantum fluid adds the single-valuedness of ψ (quantized circulation, no continuous vorticity), the point-vortex model lacks the compressibility of the field. Nothing here is a prediction for the liquid helium of Section 3.1: Gross–Pitaevskii is a model of it.

What failed or was wrong on the way. (i) The first compilation of the new Lean file failed on three proofs (a simplifier normal form for the conjugate of a difference, an unsolved imaginary part, the same form inside a norm); the version shown is the one that compiled. (ii) My first guess for the sampling threshold n_0 (the number of samples needed to resolve a π step across the distance to the other core) was wrong twice over: loops started towards that core needed no extra samples, and with generic starting angles the threshold depends on whether the other zero is inside or outside the loop and grows as $\delta^{-1/2}$. (iii) In the programme’s friction study the excess of the measured pair speed over the zero-mean-flow prediction (up to 49%) was first read as an imprint transient; it is the uniform flow of the torus (CLAIM-o82 of the programme’s ledger). (iv) The first CVODE run of the reduced model used a stale build of the Python module, older than the repair of commit 5db8041 of rusty-SUNDIALS: it needed 269 733 right-hand-side calls and differed from DOP853 by $8 \times 10^{-4}\xi$, against 5 028 calls and $4 \times 10^{-7}\xi$ for the repaired build, with which the run was redone.

Exercises

Exercise 1. *Two samples are not enough.* A vortex has phase $\theta(\varphi) = \varphi$ along a circle centred on it. (a) For $n = 2$ samples, at $\varphi = 0$ and π , compute the sum of principal differences for the counter-clockwise and for the clockwise traversal. (b) Explain with `QuantumFluids.VortexWinding.pdiff_add_rev_eq_two_pi_iff` why both give the same answer, and why `QuantumFluids.ContinuumWinding.detector_correct` says nothing about

this sampling. (c) For which n is the sum right for every starting angle? *Hint:* the principal value of a step of exactly π is $+\pi$ in both directions; for (c) all steps $2\pi/n$ must be below π .

Exercise 2. *The mean flow of the torus.* (a) Show from (3.9) that $\bar{u}/(\hbar/md) = 2\pi d^2/L^2$ and evaluate it for $d = 6, 12, 16\zeta$, $L = 64\zeta$. (b) Derive (3.9) for one pair, the $+$ vortex at (x_1, y) and the $-$ vortex at (x_2, y) . *Hint:* the circulation of the zero-mean-flow field around a row $y = \text{const}$ is piecewise constant in y (it changes by $\mp\kappa$ across a vortex), has zero mean over y , and the wave function requires an integer multiple of 2π . (c) In which direction, and how fast, does the pair with the $+$ vortex *below* the $-$ vortex, at the same x , move? *Hint:* the box has fourfold symmetry; check the direction with `vortex_velocity`.

Exercise 3. *Run it.* With `qf_pgpe`, imprint the pair of Section 3.5.1 with $d = 12\zeta$ in a box of side $L = 96\zeta$ with 192^2 grid points (the same Δx), run for 40 time units and measure the speed as in Section 3.5.5. Predict it first from (3.9) and the zero-mean-flow field (`vortex_velocity` with `L=96`); what changes between the two boxes, and by how much? *Hint:* the uniform flow scales as L^{-2} , and so, to leading order, does the correction from the images. For comparison, my run of this recipe (61 s on the loaded machine) gave $v = 0.0889$ against the prediction 0.0885, where the same pair in the 64ζ box moves at 0.0940.

Notes and further reading

The hydrodynamic form of the Schrödinger equation is Madelung's [41]; the quantization of circulation was proposed by Onsager [15] and applied by Feynman to rotating helium [16]. It was detected in single quanta in Vinen's vibrating-wire experiment [36] (Zieve's retrospective [37] is a good entry point) and the arrays of lines were photographed by Yarmchuk, Gordon and Packard [38]. For the Gross–Pitaevskii vortex see [17, 18] and the books of Pitaevskii and Stringari [26] and, for the liquid, Donnelly [10]; vortices are carried by the flow of the others in Helmholtz's analysis [42], and the pair as a solitary wave of the compressible field is the subject of Jones and Roberts [24]. The projected method is reviewed in [20]; the engine is [21], the library [12], and the friction study that identified the mean flow of the torus [23]. The pairs of this chapter are what unbinds in Chapter 6, what moves with dissipation in Chapter 7, and what a moving obstacle sheds in Chapter 9, in the simulations of Kwon and Shin [27].

The Specific Heat of Helium-4 and Godfrin's Series

4.1 A cubic law, and what bends it

Halve the temperature of a mole of superfluid helium-4, from 0.2 K to 0.1 K, and its specific heat falls by a factor of almost eight. In the table that accompanies the 2021 paper of Godfrin and his collaborators [2], the specific heat at 0.2 K is $6.513 \times 10^{-4} \text{ J K}^{-1} \text{ mol}^{-1}$ and at 0.1 K it is $8.260 \times 10^{-5} \text{ J K}^{-1} \text{ mol}^{-1}$. The ratio is 7.88, which is 1.4 per cent below $2^3 = 8$. Divide the specific heat by T^3 and the result moves only from 0.0826 at 0.1 K to 0.0780 $\text{J mol}^{-1} \text{K}^{-4}$ at 0.5 K, a drift of a few per cent across a factor five in temperature. That near-constancy is the T^3 law of a gas of phonons. The drift is where the physics of this chapter lives: it is the bending of the dispersion relation, seen thermally.

A word on provenance, because this book insists on it. These numbers are not calorimeter readings: the paper *integrates the measured dispersion relation* (neutron scattering, Chapter 5) over the thermal occupation of the excitations and tabulates the result, in an open ancillary file. Calorimetry measured the same quantity directly, long before: the letter of Phillips, Waterfield and Hoffer in 1970 is entitled *Calorimetric evidence for positive phonon dispersion in liquid helium-4* [43], and Greywall's 1978 paper (with an erratum in 1980) *Specific heat and phonon dispersion of liquid ^4He* [44]. The specific heat of this liquid is a thermodynamic window onto the shape of $\varepsilon(k)$: a calorimeter and a neutron spectrometer measure one quantum fluid in two languages, the first counting how much heat a mole absorbs per kelvin, the second how much energy an excitation of wavevector k carries, and statistical mechanics is the dictionary between them.

GODFRIN WHAT THE PAPER CONTAINS

Experiment. Inelastic neutron scattering on superfluid ^4He at the ILL spectrometer IN5 gave the dispersion relation of the Landau excitations (phonon–maxon–roton) [2]. The open ancillary data of the paper tabulate the result at saturated vapour pressure as a processed table, not raw spectra: 1727 numeric rows, only 34 of them with an uncertainty, for k from 0 to $3.6 \text{ \AA}^{-1}$, spaced by $0.002 \text{ \AA}^{-1}$ up to $3.44 \text{ \AA}^{-1}$ and by $0.05 \text{ \AA}^{-1}$ beyond; below $0.15 \text{ \AA}^{-1}$ the energies are ultrasound-based.

Thermodynamic consequences. The paper computes the specific heat from the measured

curve numerically, and analytically from a low-temperature series. Its Eq. (22) gives the phonon specific heat

$$C_V = A T^3 + C T^5 + D T^6 + E T^7 + K T^8 + L T^9$$

for a dispersion $\varepsilon = \hbar c k (1 + \alpha_2 k^2 + \dots + \alpha_6 k^6)$ with $\alpha_1 = 0$, and remarks that earlier published versions of this series contain errors.

What this chapter does with it. It does not use or modify the measurement. It takes Eq. (22) as a mathematical statement and asks three instruments whether it is right.

The formula is a polynomial in T with six terms. Each coefficient is an explicit expression in the speed of sound c , the molar volume V and the dispersion coefficients α_j , with powers of π in some and values of the Riemann zeta function in others. Printed formulas of that kind are easy to get wrong and hard to check by eye. This chapter checks this one three ways: by a machine-checked proof in Lean 4 (Section 4.3); by two derivations that do not use Lean, one symbolic and one in exact rational arithmetic (Section 4.4); and by numerical integration with the CVODE solver of rusty-SUNDIALS, which does not know the series at all (Section 4.5). The verdict can be stated now, and it is the book's rule to state it early: *every coefficient of Eq. (22) is correct, and so is the inverse series it is built on.* This is a confirmation, not a finding. A published formula is expected to be right, and this one is. What is new is that it is now *proved* — as a statement about mathematics, not about helium — and that the assumptions under which it holds are written down, which Sections 4.6 and 4.7 use to say what the series can and cannot do.

4.2 From a dispersion relation to a heat capacity

4.2.1 A gas of phonons

Below about half a kelvin the thermal excitations of liquid ^4He are overwhelmingly phonons, rotons being exponentially rare (the heat map of Section 4.6 shows where the roton takes over). Their number is not conserved, so they carry no chemical potential, and a mole of liquid of molar volume V holds, in a shell dk of wavevectors, $(V/2\pi^2) k^2 dk$ modes. The energy of the gas is

$$E(T) = \frac{V}{2\pi^2} \int_0^\infty \frac{\varepsilon(k)}{e^{\varepsilon(k)/k_B T} - 1} k^2 dk, \quad C_V = \frac{dE}{dT}. \quad (4.1)$$

For the linear dispersion $\varepsilon = \hbar c k$ (Debye's) the substitution $x = \hbar c k/k_B T$ gives

$$E = \frac{V}{2\pi^2} \frac{(k_B T)^4}{(\hbar c)^3} \int_0^\infty \frac{x^3}{e^x - 1} dx = \frac{\pi^2 V (k_B T)^4}{30 (\hbar c)^3}, \quad C_V = A T^3, \quad A = \frac{2\pi^2 k_B^4 V}{15 \hbar^3 c^3},$$

because $\int_0^\infty x^3/(e^x - 1) dx = \pi^4/15$. With $c = 238.3 \text{ m s}^{-1}$ and $V = 27.5793 \text{ cm}^3 \text{ mol}^{-1}$, the values quoted in the paper for saturated vapour pressure, this is $A = 0.083091 \text{ J mol}^{-1} \text{ K}^{-4}$, against

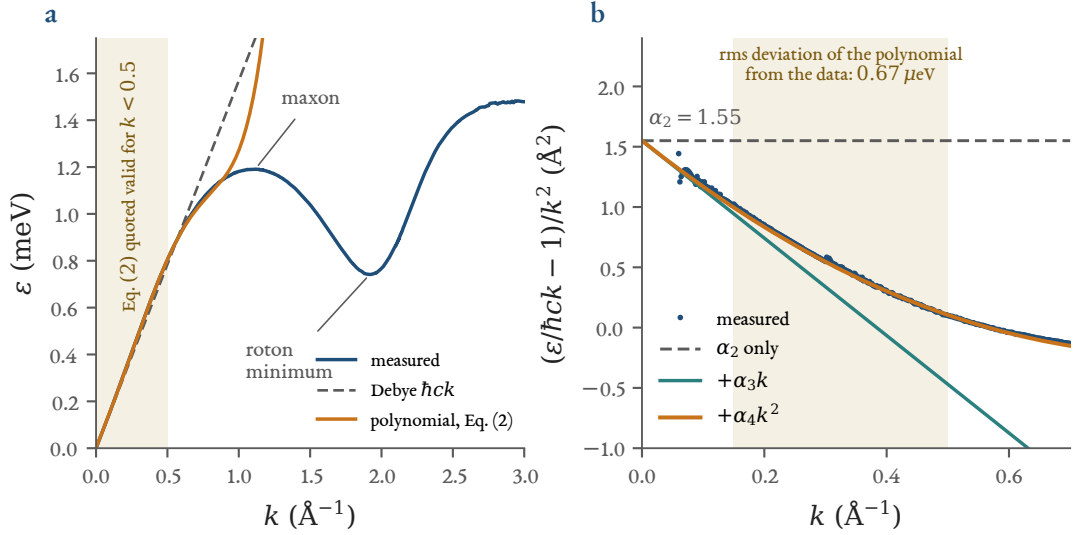


Figure 4.1: The phonon branch of superfluid ^4He at saturated vapour pressure ($T < 0.1$ K), measured by neutron scattering (ancillary data of [2], CC BY 4.0). **(a)** The measured curve (blue), the linear Debye dispersion (dashed grey) and the polynomial of Eq. (2) of the paper with the coefficient set quoted there (orange); maxon at $k = 1.114 \text{ \AA}^{-1}$, roton minimum at $k = 1.92 \text{ \AA}^{-1}$, $\varepsilon/k_B = 8.6$ K in this table. **(b)** The same data divided by $\hbar ck$ and k^2 : the intercept is α_2 , the slope α_3 , the curvature α_4 . The rms difference between the polynomial and the table for $0.15 \leq k \leq 0.5 \text{ \AA}^{-1}$ is $0.67 \mu\text{eV}$.

the printed 0.0831. Only two properties of helium enter the cubic law, the sound velocity and the molar volume: a measurement of C_V/T^3 at low temperature is, in this model, a measurement of c .

The natural unit of wavevector at temperature T is the thermal wavevector $k_B T/\hbar c$. With the numbers above, $\hbar c/k_B = 18.20 \text{ K \AA}$, so a kelvin corresponds to $0.0549 \text{ \AA}^{-1}$. At 0.1 K half of the heat capacity is carried by phonons with $k < 0.024 \text{ \AA}^{-1}$, that is, wavelengths longer than $257 \text{ \AA}$: they live far to the left of everything in the dispersion curve that makes helium interesting.

4.2.2 The bending

Real helium is not linear. The paper writes the phonon branch as a power series

$$\varepsilon(k) = \hbar c k (1 + \alpha_2 k^2 + \alpha_3 k^3 + \alpha_4 k^4 + \alpha_5 k^5 + \alpha_6 k^6), \quad (4.2)$$

with no linear term in the bracket ($\alpha_1 = 0$); α_j has the dimension of length^j . For saturated vapour pressure it quotes $c = 238.3 \text{ m s}^{-1}$, $\alpha_2 = 1.55 \text{ \AA}^2$, $\alpha_3 = -4.04 \text{ \AA}^3$, $\alpha_4 = 2.30 \text{ \AA}^4$ and $\alpha_5 = \alpha_6 = 0$, and says that this set describes the measured dispersion for $k < 0.5 \text{ \AA}^{-1}$. Figure 4.1 compares it with the open table: within the shaded range the polynomial and the data differ by $0.67 \mu\text{eV}$ rms and by at most $2.2 \mu\text{eV}$, and at $k = 1 \text{ \AA}^{-1}$ the polynomial lies 7.6 per cent above the

data: beyond its range it stops describing helium and becomes a sequence of numbers (that is not a criticism, the range is quoted with the coefficients; it is the first hint of what a series in T can and cannot do).

The sign of α_2 matters physically. For $\alpha_2 > 0$ the energy at a given k lies above the linear Debye value, so there are fewer thermal phonons than in a Debye gas and C_V/T^3 *falls* as the temperature rises: this is the slow drift of the first paragraph. (The same sign, the “anomalous” dispersion, keeps the collinear decay of a phonon into two phonons energetically open: this is `QuantumFluids.HeliumKinematics.three_phonon_open_iff`, for a dispersion $ck(1 + ak^2)$; Chapter 5 treats it.)

4.2.3 A series in T

To turn (4.1) into a series one needs the integral over energy, not over wavevector. Write $u = \varepsilon/\hbar c$, which has the dimension of an inverse length, so that $u = k(1 + \alpha_2 k^2 + \dots)$, and invert the series:

$$k(u) = u - \alpha_2 u^3 - \alpha_3 u^4 + (3\alpha_2^2 - \alpha_4)u^5 + (7\alpha_2\alpha_3 - \alpha_5)u^6 + \eta u^7 + O(u^8), \quad (4.3)$$

with $\eta = -12\alpha_2^3 + 8\alpha_2\alpha_4 + 4\alpha_3^2 - \alpha_6$. This is the inverse series printed in the paper, here for $\alpha_1 = 0$ (the paper prints it for general α_1). The density of states in u is $k^2 dk/du = \sum_n g_n u^n$ with

$$\begin{aligned} g_2 &= 1, & g_3 &= 0, & g_4 &= -5\alpha_2, & g_5 &= -6\alpha_3, & g_6 &= 7(4\alpha_2^2 - \alpha_4), \\ g_7 &= 8(9\alpha_2\alpha_3 - \alpha_5), & g_8 &= -3(55\alpha_2^3 - 30\alpha_2\alpha_4 - 15\alpha_3^2 + 3\alpha_6). \end{aligned} \quad (4.4)$$

Each monomial $g_n u^n$ gives a thermal integral that is a Bose integral,

$$\int_0^\infty \frac{u^m}{e^{\hbar c u/k_B T} - 1} du = \left(\frac{k_B T}{\hbar c}\right)^{m+1} m! \zeta(m+1), \quad (4.5)$$

so that

$$E(T) = \frac{V}{2\pi^2} \sum_n g_n (n+1)! \zeta(n+2) \frac{(k_B T)^{n+2}}{(\hbar c)^{n+1}}, \quad (4.6)$$

$$C_V = \sum_n \frac{V k_B^{n+2}}{2\pi^2 (\hbar c)^{n+1}} g_n (n+2)! \zeta(n+2) T^{n+1}. \quad (4.7)$$

The term $n = 2$ is Debye's. The term $n = 3$ would give a T^4 term, but $g_3 = 0$ because there is no α_1 : that is why Eq. (22) has no $B T^4$. The terms $n = 4, 5, 6, 7, 8$ give $C T^5, D T^6, E T^7, K T^8, L T^9$, and the result is Table 4.1.

Table 4.1: The six coefficients of Eq. (22). Column g_n : bracket of the density of states (4.4); column I_n : the Bose integral $\int_0^\infty t^{n+1}/(e^t - 1) dt = (n+1)! \zeta(n+2)$ (all six values are proved in Lean, Section 4.3); last two columns: the closed form evaluated with the coefficient set of the paper (units $\text{J mol}^{-1} \text{K}^{-p}$, p the power of T ; values from `figures/ch04_common.py`), and the number printed in the paper.

	T^p	g_n	I_n	computed	printed
A	T^3	1	$\pi^4/15$	0.083091	0.0831
C	T^5	$-5\alpha_2$	$8\pi^6/63$	-0.054809	-0.0548
D	T^6	$-6\alpha_3$	$720 \zeta(7)$	0.065345	0.0653
E	T^7	$7(4\alpha_2^2 - \alpha_4)$	$8\pi^8/15$	0.060370	0.0603
K	T^8	$8(9\alpha_2\alpha_3 - \alpha_5)$	$40320 \zeta(9)$	-0.26247	-0.262
L	T^9	$-3(55\alpha_2^3 - 30\alpha_2\alpha_4 - 15\alpha_3^2 + 3\alpha_6)$	$128\pi^{10}/33$	0.14086	0.141

Written out, with ζ the Riemann zeta function, the six coefficients are

$$\begin{aligned}
 A &= \frac{2\pi^2 k_B^4 V}{15 c^3 \hbar^3}, & C &= -\frac{40 \pi^4 \alpha_2 k_B^6 V}{21 c^5 \hbar^5}, \\
 D &= -\frac{15120 \alpha_3 k_B^7 V \zeta(7)}{\pi^2 c^6 \hbar^6}, & E &= \frac{224 \pi^6 k_B^8 V (4\alpha_2^2 - \alpha_4)}{15 c^7 \hbar^7}, \\
 K &= \frac{1451520 k_B^9 V \zeta(9) (9\alpha_2\alpha_3 - \alpha_5)}{\pi^2 c^8 \hbar^8}, \\
 L &= -\frac{640 \pi^8 k_B^{10} V (55\alpha_2^3 - 30\alpha_2\alpha_4 - 15\alpha_3^2 + 3\alpha_6)}{11 c^9 \hbar^9}.
 \end{aligned}$$

Three features are structural, not matters of presentation. First, π appears in A, C, E, L and $\zeta(7)$, $\zeta(9)$ in D and K : even zeta values at $2k$ are rational multiples of π^{2k} (Bernoulli numbers), odd ones are not known in closed form, and (4.7) contains $\zeta(n+2)$, which is even for n even and odd for n odd. Second, the coefficients are not one-to-one with the α_j : E contains α_2 and α_4 , L contains $\alpha_2, \alpha_3, \alpha_4$ and α_6 , so a measurement of E gives a combination of dispersion coefficients, not α_4 . Third, the printed numbers of Table 4.1 follow from the printed formulas at the quoted coefficients to within 0.18 per cent (the quoted α_j are themselves rounded, so a few parts in 10^3 carry no information). That is a check of the paper's arithmetic; it is not yet a check of its formulas, which is where an error of an earlier series would have lived.

4.3 What the proof assistant says

The library `QuantumFluids` has three modules for this chapter, `BoseIntegral`, `PhononSeries` and `PhononSpecificHeat`, with a design note recording how the work was done (the file `PHONON_SERIES_A_TO_L.md` in the `docs/designs` directory of the repository). Chapter 2 explains

how to read Lean; here it is enough to know that a `theorem` is a statement together with a proof that the kernel has checked, and that the statements below are copied from the library unchanged. For this book the three modules were compiled, in dependency order, with Lean 4.34.0-rc2 against the library's Mathlib: no errors, no sorry. The only axioms used are the three of classical mathematics (`propext`, `Classical.choice`, `Quot.sound`) for `BoseIntegral` and `PhononSpecificHeat`, and just `propext` and `Quot.sound` for `PhononSeries`, which is pure ring theory.

4.3.1 Step one: the Bose integral

Every coefficient of Eq. (22) rests on one identity, $\int_0^\infty t^n / (e^t - 1) dt = n! \zeta(n + 1)$. Mathlib has the Gamma integral and $\zeta(2k)$ in closed form, but not this integral, so the library proves it.

LEAN 4 BOSEINTEGRAL

```
-- The Bose factor `1/(e^t - 1)`, complex-valued so that Mathlib's `mellin` applies. -/
noncomputable def bose (t : ℝ) : ℂ := ((1 / (exp t - 1)) : ℝ) : ℂ

theorem bose_integral_nat {n : ℕ} (hn : 1 ≤ n) :
  mellin bose (n + 1) = (Nat.factorial n : ℂ) * riemannZeta (n + 1)
-- statement only; the proof is in lean_src/BoseIntegral.lean
```

In words. Mathlib's `mellin f s` is by definition $\int_0^\infty t^{s-1} f(t) dt$, so `mellin bose (n + 1)` is the integral $\int_0^\infty t^n / (e^t - 1) dt$. The first theorem, `QuantumFluids.BoseIntegral.bose_integral_nat`, says it equals $n! \zeta(n + 1)$ for every $n \geq 1$: this is (4.5) with the temperature scaled out. A companion theorem, `QuantumFluids.BoseIntegral.debye_specific_heat`, says that the energy $\pi^2 V (k_B T)^4 / 30 (\hbar c)^3$ of the Debye gas, whose integral is the case $n = 3$ (`QuantumFluids.BoseIntegral.mellin_bose_four`: `mellin bose 4 =  $\pi^4/15$`), has the derivative $A T^3$ with $A = 2\pi^2 k_B^4 V / 15 c^3 \hbar^3$, the coefficient printed in the paper. *Proof idea (abridged)*. For $t > 0$ one has $1/(e^t - 1) = \sum_{m \geq 0} e^{-(m+1)t}$, the geometric series behind Bose statistics (`QuantumFluids.BoseIntegral.hasSum_bose`); Mathlib's `hasSum_mellin` integrates a series of exponentials term by term, each term giving $\Gamma(s) / (m + 1)^s$; the sum is $\Gamma(s) \zeta(s)$, and $\Gamma(n + 1) = n!$.

4.3.2 Step two: inverting the dispersion and counting states, without analysis

The next module is where the printed series meet algebra. The claim “ $k(u)$ is the inverse series up to $O(u^8)$ ” involves a truncation, and a truncation normally brings questions of convergence with it. The library avoids them. *Equal up to $O(e^8)$* is stated as an identity in an arbitrary commutative ring R with an element e such that $e^8 = 0$. Taking R to be the ring of power series modulo e^8 recovers the usual statement; nothing about limits is needed, and the theorem is pure ring theory.

LEAN 4 PHONONSERIES

```

variable {R : Type*} [CommRing R]

/-- The inverse series for `alpha_1 = 0`, the case the paper's Eq. (22) treats. -/
def kInv0 (a2 a3 a4 a5 a6 e : R) : R :=
  -12*a2^3*e^7 + 3*a2^2*e^5 + 7*a2*a3*e^6 + 8*a2*a4*e^7 - a2*e^3 + 4*a3^2*e^7 - a3*e^4
  ↪ - a4*e^5 - a5*e^6 - a6*e^7 + e

/-- Its formal derivative with respect to `e`. -/
def kInv0Deriv (a2 a3 a4 a5 a6 e : R) : R :=
  -84*a2^3*e^6 + 15*a2^2*e^4 + 42*a2*a3*e^5 + 56*a2*a4*e^6 - 3*a2*e^2 + 28*a3^2*e^6 -
  ↪ 4*a3*e^3 - 5*a4*e^4 - 6*a5*e^5 - 7*a6*e^6 + 1

theorem density_of_states (a2 a3 a4 a5 a6 e : R) (h : e ^ 9 = 0) :
  kInv0 a2 a3 a4 a5 a6 e ^ 2 * kInv0Deriv a2 a3 a4 a5 a6 e
  = e ^ 2 - 5 * a2 * e ^ 4 - 6 * a3 * e ^ 5 + 7 * (4 * a2 ^ 2 - a4) * e ^ 6
    + 8 * (9 * a2 * a3 - a5) * e ^ 7
    - 3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6) * e ^ 8
  -- statement only; the proof is in lean_src/PhononSeries.lean

```

In words. The module proves two theorems. The first, `QuantumFluids.PhononSeries.dispersion_kInv`, says that the inverse series printed in the paper, for general α_1 and through $(\omega/c)^7$ with the coefficient η , really inverts $u = k(1 + \alpha_1 k + \dots + \alpha_6 k^6)$: substitute it in and you get e back, modulo e^8 (it is stated for the series `kInv` of general α_1 , of which `kInv0` is the case $\alpha_1 = 0$). The second, shown above, `QuantumFluids.PhononSeries.density_of_states`, says that $k^2 dk/du$, computed from the series (4.3) for $\alpha_1 = 0$ and its derivative, equals the polynomial whose coefficients are the brackets (4.4), modulo e^9 . The missing term e^3 is the missing BT^4 . *Proof idea (abridged)*. The powers $k^2, \dots, k^7$ of the series are truncated one at a time, each step a statement of the form `linear_combination (r) * h` with $h : e^8 = 0$. This tactic asks the kernel to check that (left side – right side) – $r \cdot e^N$ vanishes identically as a polynomial ($N = 8$ or 9); r is the *certificate*. Here is the shape of one such proof, with its certificate shortened:

```

theorem kSq0_eq (a2 a3 a4 a5 a6 e : R) (h : e ^ 9 = 0) :
  kInv0 a2 a3 a4 a5 a6 e ^ 2 = kSq0 a2 a3 a4 a5 a6 e := by
  unfold kInv0 kSq0
  linear_combination (144*a2^6*e^5 - 72*a2^5*e^3 - 168*a2^4*a3*e^4 - 192*a2^4*a4*e^5 + ...)
  ↪ * h

```

(proof abridged: the certificate has 42 terms; the six certificates for the powers $k^2, \dots, k^7$ of the general inverse series have 202, 156, 116, 82, 58 and 40 terms.) The certificates were produced by a computer-algebra script and are *checked, not trusted*: had the script made an error the kernel would reject the proof. The design note records this module as the first generated proof file in the library.

4.3.3 Step three: the six coefficients

The last module assembles the two ingredients. The energy carried by the monomial $g u^n$ of the density of states is, from (4.6), $\frac{V}{2\pi^2} g I k_B^{n+2} T^{n+2} / (hc)^{n+1}$ with $I = \int_0^\infty t^{n+1} / (e^t - 1) dt$ the Bose integral. The library calls it `QuantumFluids.PhononSpecificHeat.energyTerm`.

LEAN 4 PHONONSPECIFICHEAT

```

noncomputable def energyTerm (V kB hbar c g : ℝ) (n : ℕ) (I T : ℝ) : ℝ :=
  V / (2 * π ^ 2) * g * I * kB ^ (n + 2) / (hbar * c) ^ (n + 1) * T ^ (n + 2)

theorem phonon_specific_heat (V kB hbar c a2 a3 a4 a5 a6 z7 z9 T : ℝ) (hh : hbar ≠ 0) (
  ↪ hc : c ≠ 0) :
  HasDerivAt (fun T =>
    energyTerm V kB hbar c 1 2 (π ^ 4 / 15) T
  + energyTerm V kB hbar c (-5 * a2) 4 (8 * π ^ 6 / 63) T
  + energyTerm V kB hbar c (-6 * a3) 5 (720 * z7) T
  + energyTerm V kB hbar c (7 * (4 * a2 ^ 2 - a4)) 6 (8 * π ^ 8 / 15) T
  + energyTerm V kB hbar c (8 * (9 * a2 * a3 - a5)) 7 (40320 * z9) T
  + energyTerm V kB hbar c (-3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6
  ↪ )) 8
    (128 * π ^ 10 / 33) T)
  ( 2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3
  + -(40 * (π ^ 4 * a2 * kB ^ 6 * V)) / (21 * (c ^ 5 * hbar ^ 5)) * T ^ 5
  + -(15120 * (a3 * kB ^ 7 * V * z7)) / (π ^ 2 * c ^ 6 * hbar ^ 6) * T ^ 6
  + 224 * π ^ 6 * kB ^ 8 * V * (4 * a2 ^ 2 - a4) / (15 * c ^ 7 * hbar ^ 7) * T ^ 7
  + 1451520 * kB ^ 9 * V * z9 * (9 * a2 * a3 - a5) / (π ^ 2 * c ^ 8 * hbar ^ 8) * T
  ↪ ^ 8
  + -(640 * (π ^ 8 * kB ^ 10 * V * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 *
  ↪ a6)))
    / (11 * (c ^ 9 * hbar ^ 9)) * T ^ 9) T
  -- statement only; the proof is in lean_src/PhononSpecificHeat.lean

```

In words. A theorem proved earlier in the module, `QuantumFluids.PhononSpecificHeat.bose_integral_values`, gives the six Bose integrals of Table 4.1 in closed form: $\int_0^\infty t^3 / (e^t - 1) dt = \pi^4/15$, then $8\pi^6/63$, $720\zeta(7)$, $8\pi^8/15$, $40320\zeta(9)$, $128\pi^{10}/33$ (as mellin bose 4, ..., mellin bose 10). The even ones use Mathlib's $\zeta(2k)$ through Bernoulli numbers (the library computes $B_6 = \frac{1}{42}$, $B_8 = -\frac{1}{30}$, $B_{10} = \frac{5}{66}$, because Mathlib's table stops at B_4 , and from them $\zeta(6) = \pi^6/945$, $\zeta(8) = \pi^8/9450$, $\zeta(10) = \pi^{10}/93555$); the odd ones stay symbolic. The capstone `QuantumFluids.PhononSpecificHeat.phonon_specific_heat` says: *the derivative with respect to T of the sum of the six energy terms — with the density-of-states brackets of density_of_states and the six Bose integrals — is exactly the six printed closed forms A, C, D, E, K, L*. The two odd zeta values enter as real parameters z7, z9; `bose_integral_values` is where they are tied to `riemannZeta 7` and `riemannZeta 9`, which keeps the capstone in $\mathbb{R}$ and makes the absence of a closed form visible in the type. *Proof idea (abridged)*. Each term differentiates by the power rule; `HasDerivAt.add` is applied five times; the two sides then differ

only by rearrangement, which `field_simp` and `ring close`.

4.3.4 Closing the links that were made by inspection

Reading the three boxes again, one sees what the design note itself records as the weak points: the chain from the dispersion to Eq. (22) is assembled by *retyping*. The brackets $1, -5\alpha_2, -6\alpha_3, \dots$ in the capstone were copied from `density_of_states`; no theorem says that they are the coefficients of the polynomial `kInv02·kInv0Deriv`. `kInv0Deriv` was typed in, not computed as a derivative. And the six values of I in the capstone were retyped from `bose_integral_values`. None of these is likely to hide an error, but it is exactly the kind of gap that a “machine-checked” label can paper over.

The file `lean/Ch04_DosLink.lean` of this book removes the retypings. It is *new Lean, written for this book and not part of the library*: it imports the three modules above, lives in the namespace `QuantumFluids.DosLink`, and compiles in about a minute against the same `Mathlib`, with no error and no sorry; its `#print axioms` lines show the three axioms of classical mathematics and nothing else.

LEAN 4 NEW: Ch04_DosLink

```

/-- The inverse series with the dispersion coefficients as constant polynomials,
    ↪ evaluated at `X`. -/
noncomputable def kPoly (a2 a3 a4 a5 a6 : ℝ) : ℝ[X] :=
  kInv0 (C a2) (C a3) (C a4) (C a5) (C a6) X

theorem derivative_kPoly (a2 a3 a4 a5 a6 : ℝ) :
  derivative (kPoly a2 a3 a4 a5 a6) = kInv0Deriv (C a2) (C a3) (C a4) (C a5) (C a6) X

/-- The density-of-states polynomial `k(e)^2 * dk/de`, with `dk/de` the formal
    ↪ derivative. -/
noncomputable def dosPoly (a2 a3 a4 a5 a6 : ℝ) : ℝ[X] :=
  kPoly a2 a3 a4 a5 a6 ^ 2 * derivative (kPoly a2 a3 a4 a5 a6)

theorem dos_coeffs (a2 a3 a4 a5 a6 : ℝ) :
  (dosPoly a2 a3 a4 a5 a6).coeff 2 = 1 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 3 = 0 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 4 = -5 * a2 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 5 = -6 * a3 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 6 = 7 * (4 * a2 ^ 2 - a4) ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 7 = 8 * (9 * a2 * a3 - a5) ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 8 = -3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 +
    ↪ 3 * a6)

theorem phonon_specific_heat_end_to_end (V kB hbar c a2 a3 a4 a5 a6 T : ℝ) (hh : hbar ≠ 0) (hc0 : c ≠ 0) :
  HasDerivAt (fun T =>
    energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 2) 2 (mellin bose 4).re T
  + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 4) 4 (mellin bose 6).re T
  + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 5) 5 (mellin bose 7).re T
  + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 6) 6 (mellin bose 8).re T

```

```

+ energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 7) 7 (mellin bose 9).re T
+ energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 8) 8 (mellin bose 10).re T
(⋯) T      -- abridged: the six closed forms of `phonon_specific_heat`, not repeated
-- statements only; the proofs are in lean/Ch04_DosLink.lean

```

In words. `QuantumFluids.DosLink.kPoly` is the inverse series `kInv0` with the dispersion coefficients as constant polynomials, evaluated at the variable X of $\mathbb{R}[X]$; `QuantumFluids.DosLink.derivative_kPoly` says that Mathlib's formal derivative of it *is* `kInv0Deriv`, which is thus no longer typed in. `QuantumFluids.DosLink.dosPoly` is $k^2 \cdot k'$ as an honest polynomial, and `QuantumFluids.DosLink.dos_coeffs` says that its coefficients in degrees 2 to 8 are exactly the brackets (4.4), the absent degree 3 included. `QuantumFluids.DosLink.phonon_specific_heat_end_to_end` is Eq. (22) with nothing retyped: the brackets are read off `dosPoly`, the six Bose integrals are the real parts of `mellin bose 4`, ..., `mellin bose 10`, and the two odd zeta values are $\zeta(7)$ and $\zeta(9)$ themselves.

Proof idea (abridged). Apply `QuantumFluids.PhononSeries.density_of_states` not in $\mathbb{R}$ but in the ring $\mathbb{R}[X]/(X^9)$, where $e = X$ satisfies $e^9 = 0$. It says that $k^2 k'$ and the polynomial of brackets are equal there, that is, that X^9 divides their difference, and a polynomial divisible by X^9 has vanishing coefficients below degree 9 (`Polynomial.X_pow_dvd_iff`). The end-to-end theorem then follows from `QuantumFluids.PhononSpecificHeat.phonon_specific_heat` and `QuantumFluids.PhononSpecificHeat.bose_integral_values`.

The file goes one step further. It defines the energy of the monomial $g u^n$ as the integral itself, written with Mathlib's Mellin transform (`energyMonomial`); proves, in `QuantumFluids.DosLink.energyMonomial_eq`, that for $T > 0$ it equals `energyTerm` evaluated with the Bose integral `Re mellin bose (n + 2)`; and restates Eq. (22), in `QuantumFluids.DosLink.phonon_specific_heat_integral`, for the sum of the six integrals.

What is still outside Lean. What these theorems say is exact and limited: the derivative of the thermal energy of the *truncated* density of states $\sum_{n=2}^8 g_n u^n$, integrated against the Bose factor over $(0, \infty)$, is Eq. (22). That the truncated sum may stand for the true density of states under the integral — that the series is an asymptotic expansion of the specific heat of a given dispersion curve, with the roton branch and the upper limit dropped — is an analytical statement that the library does not contain. Section 4.6 computes what it amounts to.

4.4 A confirmation, not a finding

Lean is one derivation. Two others were made without it, for the reason that two thermometers matter: a proof assistant checks a *statement*, so it can confirm a derivation that started from the wrong statement just as faithfully as from the right one.

A symbolic derivation. The script `exploration/godfrin/derive_cv_series.py` derives everything from scratch with the computer-algebra system `sympy`: it reverts the series order by order,

forms $d(k^3/3)/du$, integrates term by term with the Bose integral (4.5), differentiates, and compares each result with the printed closed form by the exact test `simplify(derived - printed) == 0`, which leaves no room for tuning. I re-ran it for this book (32 s on the shared machine); its output ends:

```
A (T^3): MATCHES the printed closed form      C (T^5): MATCHES the printed closed form
D (T^6): MATCHES the printed closed form      E (T^7): MATCHES the printed closed form
K (T^8): MATCHES the printed closed form      L (T^9): MATCHES the printed closed form
T^4 coefficient (paper: absent because alpha_1 = 0): 0
INVERSE SERIES: all 7 printed coefficients match
C_V SERIES: ALL SIX MATCH
```

(“All 7” counts the seven coefficients of the printed inverse series for general α_1 , through $(\omega/c)^7$.)

Exact rational arithmetic. The third derivation is this chapter’s own, written to be as unlike the other two as possible: no symbols, no proof, only fractions. With the coefficient set of the paper as the exact fractions $\alpha_2 = 31/20$, $\alpha_3 = -101/25$, $\alpha_4 = 23/10$, the script `figures/ch04_series_ext.py` reverts the series by fixed-point iteration in Python’s `fractions` and reads off the density-of-states coefficients, $g_4 = -31/4$, $g_5 = 606/25$, $g_6 = 5117/100$, $g_7 = -56358/125$, $g_8 = 3527061/8000$: the brackets (4.4) at those α ’s, obtained by a program that has never seen them. Multiplying by the Bose integrals gives the coefficient of each power of T ; the six that Eq. (22) predicts agree with the closed forms to a relative 9×10^{-16} (double-precision rounding), and the coefficient of T^4 is exactly zero. The same script continues past L , giving $c_{10} = 1.039$ and $c_{11} = -2.585$ (in $\text{J mol}^{-1}\text{K}^{-p}$): Lean does not cover those, the exact arithmetic does, and Section 4.6 uses them.

What kind of check this is. An earlier pass in the programme had only reproduced the six printed *numbers* from the six printed *formulas* (the last column of Table 4.1). That tests the arithmetic of the paper, not its formulas, and a formula is where an error of an earlier series would live. The programme’s rule LL-15 (name the property a claim depends on, and check that property rather than a neighbour of it) asks for a derivation from the dispersion relation, and that is what the three above are. They agree with Eq. (22) in all six coefficients, with the inverse series, and with the absence of a T^4 term.

Checkers must be able to fail. A proof checker that cannot reject anything proves nothing. When the Lean proofs were first written, four deliberately wrong versions were fed to it — $-12\alpha_2^3$ in η changed to $-11\alpha_2^3$, the 15120 of D to 15121, the $9\alpha_2\alpha_3$ of K to $8\alpha_2\alpha_3$, the 55 of L to 54 — and all four were rejected (design note). I repeated two of them for this book, against the library modules compiled as above (the files `lean/Ch04_NegativeControl_L54.lean` and `lean/Ch04_NegativeControl_D15121.lean` are meant to fail; paths shortened in what follows, and the dagger that Lean prints after inaccessible names removed). Changing 55 to 54 in `QuantumFluids.PhononSeries.density_of_states`, with the certificate of the true theorem, gives:

```

NegControlL54.lean:15:2: error: ring failed, ring expressions not equal
R : Type u_1
inst : CommRing R
a2 a3 a4 a5 a6 e : R
h : e ^ 9 = 0
⊢ -(a2 ^ 3 * e ^ 8 * 3) = 0

```

and changing the 15120 of D to 15121 in the capstone `QuantumFluids.PhononSpecificHeat.phonon_specific_heat` gives:

```

NegControlD15121.lean:22:52: error: unsolved goals
[... 48 more lines: the goal that `ring` could not close, a polynomial
identity in V, kB, T, hbar, c, pi, a2..a6, z7, z9 ...]

```

In the first case ring prints the residue: the two sides differ by $-3a_2^3e^8$, exactly what changing 55 to 54 in the term $-3(55a_2^3 - \dots)e^8$ must leave behind. In the second the message is a polynomial identity that ring could not close, too long to show; what matters is that it was not closed. A proof that fails, with a residue one can read, is a checker doing its work.

The outcome rule, and a lapse. The programme's discipline is to fix, before running a check, what each outcome will be called. Here that was *not* done: the derivation script was written and run first and the rule below afterwards, and the design note records a process lapse. Because the comparison is a symbolic identity, with no tolerance to tune and no analysis choice (the coefficients to compare are the paper's own), the lapse cannot have shaped the result; it is recorded all the same. The rule that binds what follows: a mismatch would have been a candidate misprint of the preprint, to be settled against the published version before anything is said about the paper; a full match is a confirmation, and *is not to be dressed up as a finding*. It was a full match, for the arXiv version; the version of record was not compared. Eq. (22) is a correct consequence of Eq. (2), as a statement of mathematics. That is a good thing to know about a series whose predecessors were wrong, and it is all it is.

4.5 What the solver says

The solver is a witness for a different question. Lean says that, for the polynomial dispersion (4.2), the coefficients of the series in T are what Eq. (22) says. A solver that has never heard of a series can be asked whether the *integral itself* behaves as that series says. Two experiments follow. Neither is a proof: each carries a tolerance below which it is blind, and each concerns the model polynomial and not helium. All integrals use CVODE of rusty-SUNDIALS through its Python module (`rusty_sundials.CvodeSolver`), with the Adams–Moulton option, the natural one for smooth non-stiff problems.

Which build of the solver. My first runs used the module installed in the project's virtual environment (built 2026-09-26) and gave plausible numbers that were wrong: at relative tolerance 10^{-6}

its Adams method returned the Debye integral $\int_0^{70} x^3 / (e^x - 1) dx$ with a relative error 6.5×10^{-4} after 15 532 right-hand-side calls, 649 times the tolerance, and its BDF gave up (“too many error test failures at one step”) on 38 of 160 end points when asked for the six Bose integrals of the next subsection (figures/ch04_solver_probe.py). The maintainers’ note docs/CVODE_ADAMS_FIX.md in the rusty-SUNDIALS repository describes the cause: in that build Method:Adams was implicit Euler with a Milne error estimate, a first-order method whose error scales like $\sqrt{\text{rtol}}$, as observed; it was fixed on 2026-09-28. Every number of this chapter comes from a build of rusty-SUNDIALS commit 5db8041 (sha256 of rusty_sundials.so beginning obdb1b4a504f), and the scripts refuse to run with any other: Adams on $y' = -y$ over $[0, 10]$ at relative tolerance 10^{-8} must use fewer than 5000 right-hand-side calls (it uses 511). With that build the Debye integral costs 260 calls at relative tolerance 10^{-6} and returns a relative error 2.8×10^{-7} , and no end point defeats Adams or BDF (0 give-ups). A numerical result with the right shape is not evidence that the tool is right; a check against a closed form is.

4.5.1 The Bose integrals as initial-value problems

The integral $\int_0^X t^m / (e^t - 1) dt$ is the value at $x = X$ of the solution of $y' = x^m / (e^x - 1)$, $y(0) = 0$. Scaling by $m!$ makes all six components of order one ($y_m \rightarrow \zeta(m+1)$), which lets one scalar absolute tolerance serve all of them. The comparison is with the closed forms *proved in Lean* (Table 4.1), not with a numerically evaluated $m! \zeta(m+1)$.

RUSTY-SUNDIALS CVODE: THE SIX BOSE INTEGRALS

```
def rhs_bose(x, y):
    # y_m' = x^m / (m! (e^x - 1)), y_m(0) = 0
    if x <= 0.0:
        return [0.0]*len(ms)
    e = np.expml(x)
    return [x**m/(fact[m]*e) for m in ms]
# ms = [3, 5, 6, 7, 8, 9]; excerpt of figures/ch04_cvode_compute.py
solver = rs.CvodeSolver("adams", rtol, atol, 200_000) # method, rtol, atol, max_steps
_, y = solver.solve(rhs_bose, 0.0, [0.0]*len(ms), 80.0) # x from 0 to X = 80
```

requested rtol (atol = 10^{-2}rtol)	10^{-4}	10^{-6}	10^{-8}	10^{-10}	10^{-12}
largest error of the six values	5.1×10^{-4}	2.4×10^{-5}	2.0×10^{-7}	3.3×10^{-9}	2.4×10^{-11}
right-hand-side calls	231	383	528	688	848

Relative error against $\pi^4/15$, $8\pi^6/63$, $720\zeta(7)$, $8\pi^8/15$, $40320\zeta(9)$, $128\pi^{10}/33$ of bose_integral_values; SciPy’s adaptive quadrature at relative tolerance 10^{-13} reaches 1×10^{-16} .

The largest error is between 5.1 and 33 times the requested tolerance over the whole range from 10^{-4} to 10^{-12} (the global error of a multistep method exceeds its local tolerance); at the tightest tolerance the six closed forms $\pi^4/15$ to $128\pi^{10}/33$ are matched to 2.4×10^{-11} (Figure 4.2b).

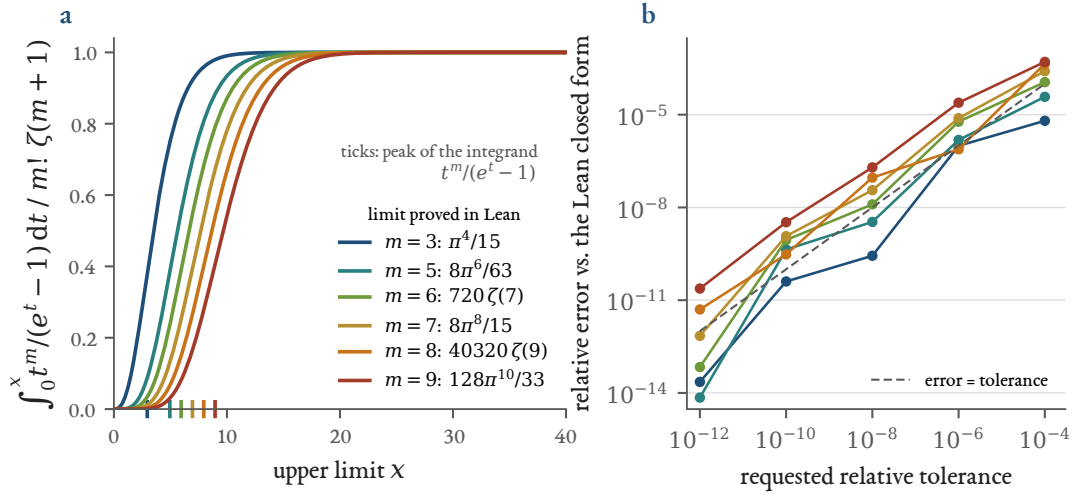


Figure 4.2: The six Bose integrals of Lean’s `bose_integral_values` computed by CVODE (Adams). **(a)** The cumulative integrals $\int_0^x t^m / (e^t - 1) dt$ divided by their limits; the integrand peaks near $t = m$ (ticks), so the higher-order terms of the series draw their weight from higher energies. **(b)** Relative error against the closed forms proved in Lean, versus the requested relative tolerance; the dashed line is error equal to tolerance.

Panel (a) carries a physical point for later: the integrand $t^m / (e^t - 1)$ peaks near $t = m$, so the term of order T^m draws its weight from phonons of energy about $m k_B T$, the L term from nine thermal energies, not one: a series in T is a series in progressively less thermal phonons.

4.5.2 The thermal integral with no series at all

Now the experiment that matters. For the model dispersion, let the solver integrate over wavevector the thermal integral that *defines* the specific heat, with one ODE component per temperature, and compare with the truncated series. There is one technical step, and it is a use of the first Lean theorem. Written naively, the answer is of order $A T^3$ and the interesting part, the series’ corrections, is a small difference of order T^2 (relative) hidden inside it; a solver with relative tolerance 10^{-12} then sees the corrections only to 10^{-12} of the whole. But the Debye law is *proved*: $\int_0^\infty (V k_B / 2\pi^2) k^2 f(\hbar c k / k_B T) dk = A T^3$ exactly (`debye_specific_heat`). So subtract it inside the integrand, and give the solver only the difference:

$$\begin{aligned} \frac{C_V(T)}{A T^3} &= 1 + \int_0^{k_{\max}} \frac{V k_B k^2}{2\pi^2 A T^3} [f(x_{\text{poly}}) - f(x_D)] dk, \\ x_{\text{poly}} &= \frac{\hbar c k (1 + \alpha_2 k^2 + \alpha_3 k^3 + \alpha_4 k^4)}{k_B T}, \quad x_D = \frac{\hbar c k}{k_B T}, \end{aligned} \quad (4.8)$$

with $f(x) = x^2 e^x / (e^x - 1)^2$ and $k_{\max} = 2.5 \text{ \AA}^{-1}$ (beyond it the Debye integrand contributes less than 10^{-14} of $A T^3$ for every $T \leq 1 \text{ K}$). The right-hand side of the ODE is the integrand of (4.8) evaluated at all the temperatures at once.

RUSTY-SUNDIALS CVMODE: THE SPECIFIC HEAT OF THE MODEL DISPERSION, NO SERIES

```
def rhs_model(k, y):
    # one component per temperature in T_all (68 of them)
    if k <= 0.0:
        return [0.0]*len(T_all)
    xp = cc.THETA*cc.u_poly(k)/T_all      # eps/k_B T, polynomial dispersion
    xd = cc.THETA*k/T_all                 # eps/k_B T, Debye dispersion
    return (cc.PREF*k*k*(cc.bose_weight(xp) - cc.bose_weight(xd))/ft(T_all)).tolist()
# excerpt of figures/ch04_cvmode_compute.py; cc.PREF = V k_B/(2 pi^2), ft(T) = A T^3
y, info = solve_ode(rhs_model, [0.0]*len(T_all), [0.0, cc.KMAX_MODEL],
                    1e-12, 1e-14, "model")
```

68 temperatures (41 geometrically spaced from 0.005 to 1 K, the reference temperature 0.02 K, and the 26 of the measured table), Adams, relative tolerance 10^{-12} , absolute 10^{-14} in units of $A T^3$: 2 370 right-hand-side calls, 3 s on the shared 8-core machine at load average about 2.0. Against a 30-digit mpmath quadrature of the same integrand at seven temperatures (0.005 to 1 K) the absolute error is 4×10^{-15} at 0.1 K and at most 4×10^{-13} (at 0.005 K). Without the subtraction, at the same tolerances: 5×10^{-14} at 0.1 K.

The subtraction is worth having: at relative tolerance 10^{-8} it reduces the absolute error of the result from 2×10^{-8} to 6×10^{-13} at 0.005 K and from 4×10^{-11} to 4×10^{-12} at 0.1 K. The proved Debye law is not decoration here, it is what lets the solver see the series.

Figure 4.3a is the test. If the closed forms are the coefficients, then the residual after A is $C T^5 (1 + O(T))$, that after $A + C$ is $D T^6 (1 + O(T))$, and so on: a ladder of straight lines in logarithmic axes, with slopes 2, 3, 4, 5, 6 (the powers of T minus 3) and a last line of slope 7. The lines are straight, and the fitted slopes agree with the expected ones to within about 0.15 (the slope of a sum of two power laws is not exactly that of the leading one). Over the range where each residual lies above a thousand times the solver's error and $T \leq 0.06 \text{ K}$, fitted slopes against the expected ones are:

truncated after	A	$A+C$	$A+C+D$	$\dots +E$	$\dots +K$	$\dots +L$
next term	$C T^5$	$D T^6$	$E T^7$	$K T^8$	$L T^9$	$c_{10} T^{10}$
expected slope	2	3	4	5	6	7
fitted slope	1.98	3.01	3.91	4.98	6.15	6.91
residual / next term at the lowest T	0.994	1.005	0.977	0.995	1.103	0.952

The solid lines meet the dashed ones as $T \rightarrow 0$: at the lowest temperature of each line the residual is within about 10 per cent of the next term (last row), and the solver's absolute error, in units of $A T^3$, is 4×10^{-15} at 0.1 K (grey squares in the figure). The absence of a T^4 term, the exactness of A, C, D, E, K and the order of L are thus visible in a computation that never used a series. The one

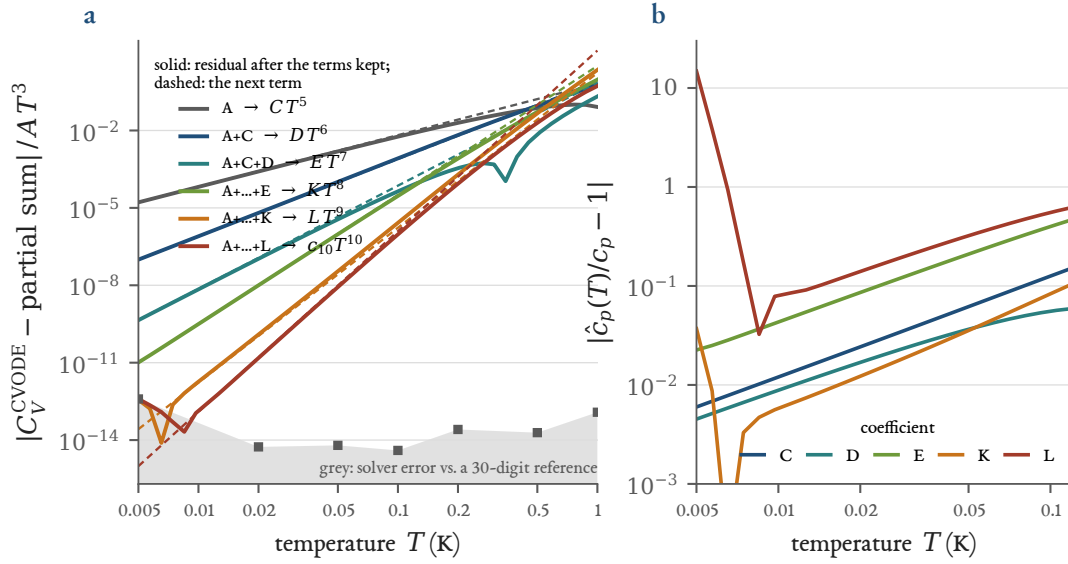


Figure 4.3: The solver against the proved series. **(a)** The absolute difference between the CVODE value of $C_V/A T^3$ for the model dispersion and the truncation of Eq. (22) named on each line, versus T (solid), with the next term of the series (dashed): each line should have the slope of the term it does not contain. The grey squares are the solver's own absolute error, measured against a 30-digit reference at seven temperatures. The top line (only A) is Debye's law; the bottom line (after L) is compared with a coefficient, c_{10} , that Lean does not contain and exact rational arithmetic supplies. **(b)** The relative error of the coefficient that the numerical data themselves suggest, $\hat{c}_p(T) = (C_V - \sum_{q < p} c_q T^q)/T^p$ with the coefficients c_q ($q < p$) of Eq. (22), against c_p .

coefficient *not* confirmed by Lean here, c_{10} , is confirmed by the solver to the same standard — as it should be, since exact rational arithmetic computed it.

4.5.3 What numerics cannot see

A solver is blind below its tolerance, and the series makes that blindness worse. Figure 4.3b asks how well the numerical data alone would determine each coefficient. The quantity $\hat{c}_p(T) = (C_V - \sum_{q < p} c_q T^q)/T^p$ tends to c_p as $T \rightarrow 0$, but only linearly (the first neglected term enters as $c_{p+1}T/c_p$), and the smaller T , the more of $\hat{c}_p$ is solver noise divided by T^p . At 0.02 K, with data whose absolute error in units of $A T^3$ is only 5×10^{-15} , the coefficients C, D, E, K, L come out with relative errors of 2.4, 1.7, 8.6, 1.2 and 14 per cent against the Lean values; at 0.05 K of 6.2, 3.7, 21, 3.6 and 32 per cent. This is the inverse problem of an experimenter who fits C_V/T^3 to a polynomial: the forward map from the dispersion to the series is explicit and well conditioned, the inverse map is not.

It also explains why two of the four wrong versions of the previous section could not have been caught numerically. Replacing 15120 by 15121 changes $C_V/A T^3$ at 0.1 K by 5.2×10^{-8}

and replacing 55 by 54 by 4.3×10^{-8} , whereas the first term the series does not contain, $c_{10}T^{10}$, contributes 1.3×10^{-6} , that is 24 and 29 times more. Both effects are far above the solver's error (4×10^{-15}), but cannot be separated from the next term of the series without already knowing it. The coarser slip $8\alpha_2\alpha_3$ for $9\alpha_2\alpha_3$ in K changes $C_V/A T^3$ by 3.5×10^{-6} , 2.8 times the neighbouring term; it would turn the slope-6 line of Figure 4.3a into a line of slope 5, and the solver would catch it. Lean rejected all of them. The two instruments are not redundant: the solver sees a whole function, its slopes and gross slips; the proof assistant sees every coefficient to the last digit.

4.6 The series meets the measured curve

Everything so far concerned a *model*, the polynomial (4.2). The specific heat of helium is the integral (4.1) of the *measured* dispersion. This section puts the two next to each other, with one rule: nothing is tuned. The coefficients, c and V are those quoted in the paper, and the dispersion is the open table.

4.6.1 Reproducing the paper's table

The open ancillary files of the paper contain, besides the dispersion, a table of the specific heat from 0.01 to 1.3 K with three columns: total, “phonon” and “roton”. I recomputed it from the dispersion table with CVODE: the table interpolated by a cubic spline in k , the thermal integral written as an initial-value problem in k with one component per temperature (as in Section 4.5), integrated first to the maxon wavevector $k_M = 1.114 \text{ \AA}^{-1}$ (the maximum of the table's $\varepsilon(k)$: the phonon column) and then to the end of the table at $3.6 \text{ \AA}^{-1}$ (the total), for the 26 temperatures 0.05, 0.10, ..., 1.30 K. Adams, relative tolerance 10^{-8} , absolute 10^{-11} in units of $A T^3$; 26 778 right-hand-side evaluations in about 23 s on the shared machine (load average ≈ 19). Against the tabulated total the relative difference is at worst 2.6×10^{-3} (at 0.75 K) and 1.3×10^{-4} in the median; the phonon column agrees to 3.6×10^{-3} ; SciPy's adaptive quadrature gives the same integral as CVODE to 3×10^{-7} . The table carries four significant digits, which would show differences of order 10^{-4} ; the larger differences are real but small compared with the table's own uncertainty column (0.86 per cent at 0.7 K), and I did not track them down (the interpolation between grid points and the treatment of the ultrasound-based region below $0.15 \text{ \AA}^{-1}$ are candidates). It is a reproduction of a tabulated number from its own open data, nothing more; but it tells us that the integral computed below is the same quantity as the table's.

4.6.2 Which excitations carry the heat

Figure 4.4 is built from the same table. It shows the structure that the series ignores. At 0.1 K half of the specific heat is carried by phonons with $k < 0.024 \text{ \AA}^{-1}$, wavelengths longer than $257 \text{ \AA}$, and nine tenths by $k < 0.043 \text{ \AA}^{-1}$; at 0.5 K the numbers are 0.119 and $0.211 \text{ \AA}^{-1}$. The ridge is a straight line of unit slope in these logarithmic axes, as the Debye law says ($k \propto T$), bending

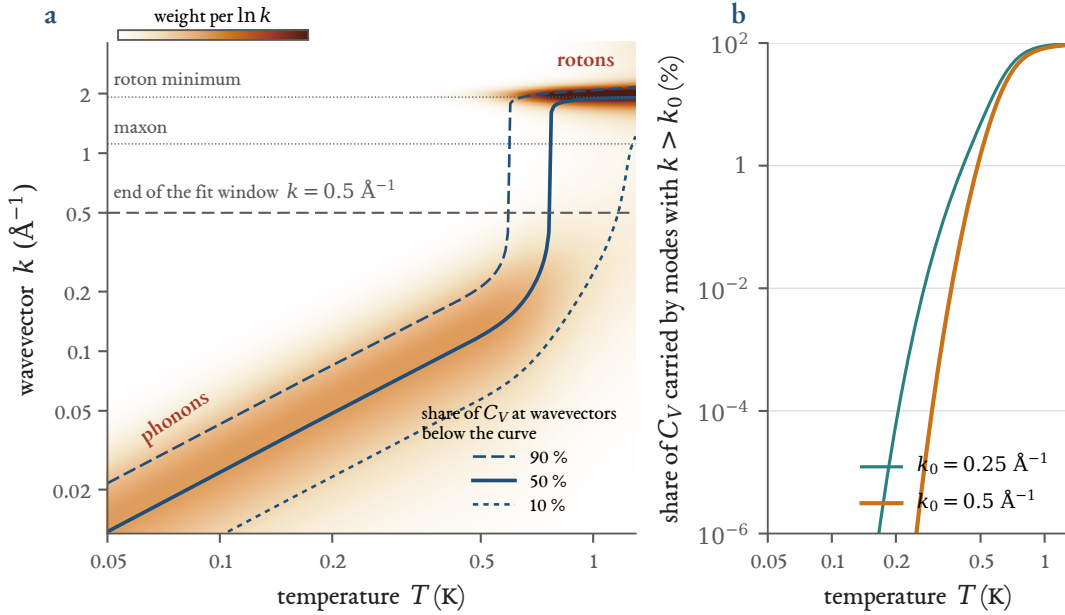


Figure 4.4: Which excitations carry the heat? **(a)** The distribution of the specific heat over the wavevector k of the excitation, $w(k, T) = (V k_B / 2\pi^2) k^2 f(\epsilon/k_B T) / C_V(T)$ with $f(x) = x^2 e^x / (e^x - 1)^2$, per unit $\ln k$, computed from the measured dispersion table (integration limit $3.6 \text{\AA}^{-1}$); white is no weight. The phonon ridge climbs along $k \propto T$; at about 0.8 K the median wavevector jumps to the roton minimum. Curves: wavevectors below which 10, 50 and 90 per cent of C_V sit. **(b)** The share of C_V carried by modes with $k > k_0$, for $k_0 = 0.25$ and $0.5 \text{\AA}^{-1}$.

slowly because $\alpha_2 > 0$. Then the weight switches: in my integration the roton part of the integral overtakes the phonon part at 0.77 K, and the median wavevector jumps to the roton minimum at $k = 1.92 \text{\AA}^{-1}$. Panel (b) quantifies what the polynomial's range of validity would suggest. If the polynomial is only trusted below $0.5 \text{\AA}^{-1}$, the fraction of the specific heat that falls outside it is 0.00015 per cent at 0.3 K, 1.5 per cent at 0.5 K, 33.5 per cent at 0.7 K and 82.4 per cent at 1 K. So by this criterion the polynomial should serve almost to 0.5 K and fail within a few tenths of a kelvin above. We now see whether the series does.

4.6.3 An error budget

Figure 4.5 separates two sources of error that are usually mixed. The six-term series is an approximation to the integral of the polynomial, and the polynomial is an approximation to helium. The solver gives the exact integral of the polynomial, so the two can be told apart. Up to about 0.3 K everything is excellent: the series differs from the exact polynomial integral by 0.11 per cent at 0.3 K, and the polynomial differs from the measured phonon integral by 0.06 per cent. At 0.5 K the picture has changed: the model error is 0.14 per cent and the truncation error of the six terms is 2.1 per cent,

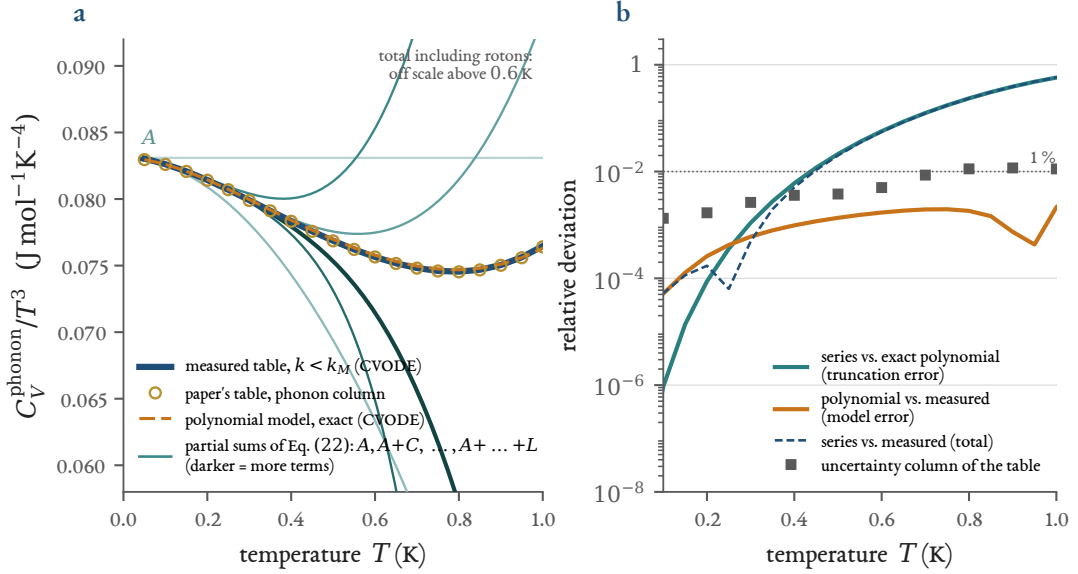


Figure 4.5: Eq. (22) against the integral of the measured dispersion. **(a)** C_V/T^3 of the phonon part ($k < k_M$): the integral of the measured table (blue, CVODE), the paper's own table column (circles), the exact integral of the polynomial model (orange dashed, CVODE), and the partial sums of Eq. (22) (teal, darkening with the number of terms). The total including rotons rises above the frame beyond 0.6 K. **(b)** Three relative deviations: the six-term series against the exact integral of the *same* polynomial (the series' own truncation error, teal); that polynomial against the measured table (the model error, orange); the series against the measured table (the total, blue dashed). Grey squares: the uncertainty column of the paper's table, relative to its total C_V .

more than a factor of ten larger. At 0.7 K the truncation error has reached 12 per cent; above that the series is useless as an approximation. The six-term series first departs from the exact polynomial integral by more than one per cent at 0.45 K. For scale, the uncertainty column of the paper's table is 0.13 per cent of the total at 0.1 K, 0.26 per cent at 0.3 K and 0.38 per cent at 0.5 K (grey squares). So the limit on the series is not the polynomial's range of validity, as the heat map suggested — at 0.5 K only 1.5 per cent of the heat is carried outside it — but the *series* itself: as a series in T it runs out of accuracy earlier than the model it expands.

4.6.4 A series that does not converge

The exact-rational script of Section 4.4 does not stop at L . For the paper's coefficient set the coefficients of T^p for $p = 3, 5, 6, 7, 8, 9$ are, relative to A : 1, -0.66 , 0.79 , 0.73 , -3.16 , 1.70 , and the next ones, beyond Lean, are 13 ($p = 10$), -31 , -29 , 313 , -399 , growing. Figure 4.6a shows the consequence. At $T = 0.2$ K the error of the partial sum falls as terms are added, reaches 2.8×10^{-7} at order $p = 19$, and then grows without bound; at 0.3 K the best is 1.3×10^{-4} (order 19); at 0.5 K it is 7.6×10^{-3} (order 6); at 0.8 K it is 9.1×10^{-2} (order 6). So the six terms of Eq. (22), through T^9 , stop well short of the best truncation at 0.2 and 0.3 K, and overshoot it from 0.5 K on: at 0.5 K

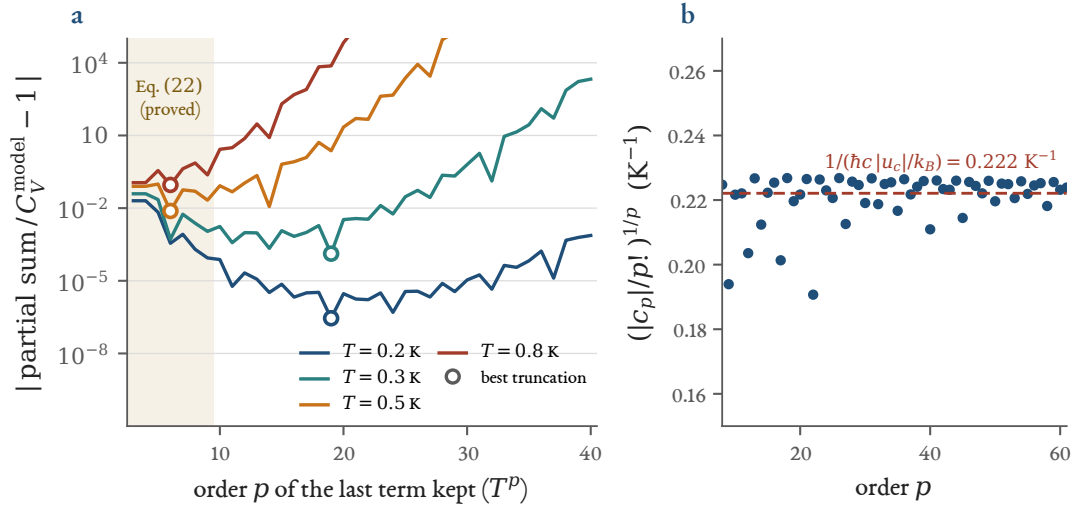


Figure 4.6: The series continues to every order and is asymptotic, not convergent. **(a)** Relative error of the partial sum through T^p against the exact (CVODE) integral of the polynomial model, for four temperatures, with the coefficients of Eq. (22) and beyond from exact rational arithmetic; circles mark the best truncation. The gold band is the range proved in Lean. **(b)** The root test $(|c_p|/p!)^{1/p}$ of the coefficients c_p of T^p ; the dashed line is $1/(\hbar c|u_c|/k_B)$, computed from the critical points of $u(k)$ alone.

the error after $A + C + D (T^6)$ is 7.6×10^{-3} , smaller than the 2.1×10^{-2} after all six terms. The six terms of Eq. (22) are the beginning of an *asymptotic* expansion, not of a convergent one that could be extended at will; its best truncation depends on the temperature.

Where does the divergence come from? Two factors. The Bose integrals contribute $(n + 2)!$ to the coefficient of T^{n+1} (they are Γ functions), a factorial. And the density of states $g(u) = \sum g_n u^n$ has a finite radius of convergence: $k(u)$ is the inverse function of $u(k)$ and has a branch point at every critical value of $u(k)$, where $du/dk = 0$, so g_n grows like R^{-n} with R the distance from the origin to the nearest singularity on the principal sheet. The polynomial $u(k) = k(1 + \alpha_2 k^2 + \alpha_3 k^3 + \alpha_4 k^4)$ has degree five, so $du/dk = 0$ is a quartic equation, with two complex-conjugate pairs of roots; the pair with the smaller critical value sits at $k_c = -0.134 \pm 0.316 i \text{ \AA}^{-1}$ and gives $u_c = -0.084 \pm 0.233 i \text{ \AA}^{-1}$, $|u_c| = 0.247 \text{ \AA}^{-1}$, that is an energy $\hbar c|u_c|/k_B = 4.50 \text{ K}$. Together: the coefficient of T^p behaves like $p! (\hbar c|u_c|/k_B)^{-p}$, and the root test $(|c_p|/p!)^{1/p}$ should tend to $1/(\hbar c|u_c|/k_B) = 0.222 \text{ K}^{-1}$. Figure 4.6b shows that it does: the sequence is 0.222 at $p = 20$, 0.211 at $p = 40$ and 0.223 at $p = 60$. The same reasoning gives the optimal order, where the factorial overtakes the geometric decay, $p^* \approx \hbar c|u_c|/k_B T$, and the best accuracy, $\exp(-\hbar c|u_c|/k_B T)$ times a power of T . The order p^* at which the terms (taken as the largest of three consecutive ones, to suppress accidental zeros) are smallest is 46 at 0.1 K (the estimate gives 45), 20 at 0.2 K (23), 15 at 0.3 K (15) and 10 at 0.4 K (11); and fitting the size of those smallest terms for $0.08 \leq T \leq 0.6 \text{ K}$ to $a T^s e^{-b/T}$ gives $b = 4.47 \text{ K}$ and $s = -3.7$, against the predicted $b = 4.50 \text{ K}$. The 4.50 K scale is a

property of the *polynomial* — the complex critical points are an artefact of truncating the dispersion at α_4 — and not of helium, whose measured dispersion has a real maxon at $\varepsilon/k_B = 13.8$ K and a roton minimum at 8.6 K; it is an illustration of how a truncated polynomial and a Bose integral conspire, not a statement about the real fluid. What the figure says is limited: below about 0.3 K the six-term series represents the integral of the polynomial excellently, above 0.5 K it does not, and the transition is set by the mathematics of the series, not by the measured curve.

4.7 What this chapter establishes, and what it does not

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved in Lean (statements about mathematics). (i) $\int_0^\infty t^n / (e^t - 1) dt = n! \zeta(n+1)$ for all $n \geq 1$, and its six values at $s = 4, 6, 7, 8, 9, 10$. (ii) The inverse series printed in the paper inverts $u = k(1 + \alpha_1 k + \dots + \alpha_6 k^6)$ modulo u^8 , and $k^2 dk/du$ equals the density-of-states polynomial modulo u^9 . (iii) The derivative with respect to T of the term-by-term energy is the six closed forms A, C, D, E, K, L of Eq. (22). (iv) New in this book: the density-of-states brackets are literally the coefficients of $k^2 \cdot k'$, with k' the formal derivative, and the capstone can be stated with them and with the Mellin transforms, nothing retyped, and with the energy of each monomial written as its integral (Section 4.3.4).

Derived independently, not in Lean. The same coefficients by computer algebra, and by exact rational arithmetic, which also gives the terms beyond L .

Computed here with rusty-SUNDIALS (CVODE, Adams, commit 5db8041). The Bose integrals to the tolerance requested; the specific heat of the model dispersion with no series, whose residual after each truncation has the slope of the next term; the integral of the measured dispersion table, reproducing the paper's table to 2.6×10^{-3} (relative, maximum over 0.05–1.3 K).

Measured by others and used as given. The dispersion relation $\varepsilon(k)$ (neutron scattering, open ancillary data) and the paper's numerical specific heat, which is itself an integral of that dispersion. I did not measure anything, and I did not compare with calorimetry: the calorimetric data are not in the open files.

Not established, and not claimed. (a) That Eq. (22) describes liquid helium at any temperature: it is a statement about the polynomial (4.2). Section 4.6 shows what the six terms do, for the model and against the table. (b) That the series converges: for the model it does not, and the divergence is computed here, not proved in Lean. The legitimacy of term-by-term integration, and the asymptotic character of the expansion for a measured curve, are analytical statements that the library does not contain. (c) Anything about $\alpha_1 \neq 0$ (the T^4 term and the α_1 -dependence of the other coefficients): the Lean files treat C_V for $\alpha_1 = 0$ only. (d) Anything about errata in the paper. The check found no discrepancy between the printed closed forms and the derived ones; the formulas checked are those of the arXiv version, and the version of

record was not compared. (e) The picture of a gas of non-interacting phonons, and the density of states $Vk^2 dk/2\pi^2$, are modelling assumptions of the physics; no theorem here proves that liquid helium is such a gas.

What failed or was left unexplained. The first version of the new Lean file forgot open Real: Lean then read π as a fresh variable (an auto-bound implicit), and the capstone failed with a type mismatch between that variable and Real.pi — the checker rejecting a statement about the wrong object, as it should. My first runs used a stale build of the solver, whose Adams method was implicit Euler: plausible numbers, wrong by up to hundreds of times the tolerance; they were discarded, and every number of the chapter was recomputed with the fixed build (Section 4.5). The derivation check of 2026-09-21 was not pre-registered; the design note records that lapse.

Exercises

Exercise 1. *The missing T^4 term.* Keep $\alpha_1 \neq 0$ and set $\alpha_2 = \alpha_3 = \dots = 0$, so that $\varepsilon = \hbar ck(1 + \alpha_1 k)$. (a) Invert $u = k(1 + \alpha_1 k)$ exactly and show that the density of states is $k^2 dk/du = u^2 - 4\alpha_1 u^3 + 15\alpha_1^2 u^4 - 56\alpha_1^3 u^5 + \dots$. (b) Use (4.7) with $n = 3$ to show that C_V acquires a term $B T^4$ with $B = -240 \zeta(5) \alpha_1 V k_B^5 / (\pi^2 \hbar^4 c^4)$. (c) Explain why Eq. (22) has no such term, and which line of density_of_states is the T^4 term's absence.

Hint. (a) $k = (-1 + \sqrt{1 + 4\alpha_1 u})/2\alpha_1$, and $g = d(k^3/3)/du$. (b) The factor is $(n+2)! \zeta(n+2) = 5! \zeta(5)$ and $g_3 = -4\alpha_1$. (c) $g_3 = 0$ when $\alpha_1 = 0$; the missing e^3 in the right-hand side. The expression in (b) agrees with the B printed for $\alpha_1 \neq 0$ in the Supplemental Material of the arXiv version of the paper; the Lean files do not cover it.

Exercise 2. *Where do 5, 28, 165 come from?* Set $\alpha_3 = \alpha_4 = \alpha_5 = \alpha_6 = 0$. Show, by Lagrange inversion, that $g_{2n+2} = (-1)^n \binom{3n+2}{n} \alpha_2^n$. Check it against the brackets of C , E and L ($-5, 28 = 7 \cdot 4, -165 = -3 \cdot 55$), and predict the coefficient $1001 \alpha_2^4$ of u^{10} .

Hint. For $u = k(1 + \alpha_2 k^2)$ one has $[u^m] k^r = \frac{r}{m} [k^{m-r}] (1 + \alpha_2 k^2)^{-m}$. Take $r = 3, m = 2n+3$, and note $g_{2n+2} = \frac{2n+3}{3} [u^{2n+3}] k^3$. (The formula was checked symbolically through $n = 4$ while writing this chapter.)

Exercise 3. *Optimal truncation.* At fixed T the terms $c_p T^p$ of the series (4.7) first shrink and then grow. Using $(|c_p|/p!)^{1/p} \rightarrow 1/(\hbar c|u_c|/k_B)$ with $\hbar c|u_c|/k_B = 4.50$ K (Section 4.6), show that the smallest term sits near $p^* \approx 4.50$ K/ T and that the best accuracy of the truncated series is of order $\exp(-4.50$ K/ T) up to a power of T . Compare with Figure 4.6 and Section 4.6: at $T = 0.2$ K the smallest terms sit at order 20 while the estimate gives 23.

Hint. Stirling: $\ln(p! x^p) \approx p \ln(px/e)$ with $x = T/4.50$ K; minimize over p .

Notes and further reading

The phonon–roton spectrum is Landau’s [3]; textbook accounts of the quasiparticle picture are in [10, 26]. The measurement and the analytical and numerical thermodynamics of this chapter are those of Godfrin and collaborators [2]; the wider survey of excitation spectra of quantum fluids is [5]. Calorimetric determinations of the phonon dispersion of liquid ^4He are [43, 44]. For asymptotic series and the idea of optimal truncation (Section 4.6) see [45]. The Lean library is [12], built on Mathlib [13] and Lean 4 [11]; the history of the three modules, including the derivation script and the generator of the certificates, is in `docs/designs/PHONON_SERIES_A_TO_L.md` of the repository. The solver is CVODE of the SUNDIALS family [4], in its rusty-SUNDIALS port. Every figure is made by a script `figures/ch04_*.py` from `ch04_cvode_raw.json`, `ch04_cvode_results.npz` and `ch04_numbers.json`; every computed number in the text is read from the last of these by `figures/ch04_numbers_tex.py`.

Phonons, Rotons and the Dispersion Relation

5.1 A curve read off a spectrometer

Keep a cell of liquid helium below a tenth of a kelvin, send cold neutrons of known energy through it, and note which of them come out having left energy behind. A neutron that has created exactly one excitation of the liquid — one phonon, one roton — is found to have lost a definite energy ε while its direction changed by an amount that fixes a definite momentum transfer $\hbar k$. Plot the number of such neutrons against (k, ε) and, at these temperatures, you get a thin bright line. That line *is* the dispersion relation of the liquid: the energy of the one-excitation states as a function of their wave number. Godfrin and his collaborators measured it for superfluid ^4He on the time-of-flight spectrometer IN5 at the Institut Laue–Langevin [2], and the authors' processed curve is distributed with the paper as an open table of 1 727 rows (34 of them with an uncertainty), one every $0.002 \text{ \AA}^{-1}$ up to $3.44 \text{ \AA}^{-1}$ and four sparser entries out to $3.6 \text{ \AA}^{-1}$. Figure 5.1a plots it.

The curve does three things that no free particle does. For small k it is a straight line, $\varepsilon = \hbar ck$, with the sound speed c that ultrasonics measures: these are the *phonons*. Near $1.1 \text{ \AA}^{-1}$ it turns over at a maximum of about 1.19 meV, the *maxon*. It then falls to a minimum of 0.741 meV — 8.60 kelvin — at $1.92 \text{ \AA}^{-1}$, the *roton*, and climbs again to a flat shoulder. Landau proposed a spectrum of this kind in 1941 and, in 1947, corrected it to a single curve in which phonons and rotons are the two ends of one dispersion relation [3, 46]. The name is historical: no rotation takes place, and the Godfrin–Krotscheck review describes the minimum as the sign of an incipient short-range order that ends, under enough pressure, in solid helium [5]. According to the same review, Feynman and Cohen suggested inelastic neutron scattering as the way to see the curve [47], and the first evidence for the roton minimum came from cold neutrons in 1957 [48].

GODFRIN WHAT WAS MEASURED

Experiment. Inelastic neutron scattering on superfluid ^4He at the ILL spectrometer IN5, giving the dispersion relation of the Landau elementary excitations (phonon–maxon–roton) from the smallest wave numbers reachable up to a few $\text{\AA}^{-1}$ [2].

What the authors did with it. They derived the thermodynamic consequences of the

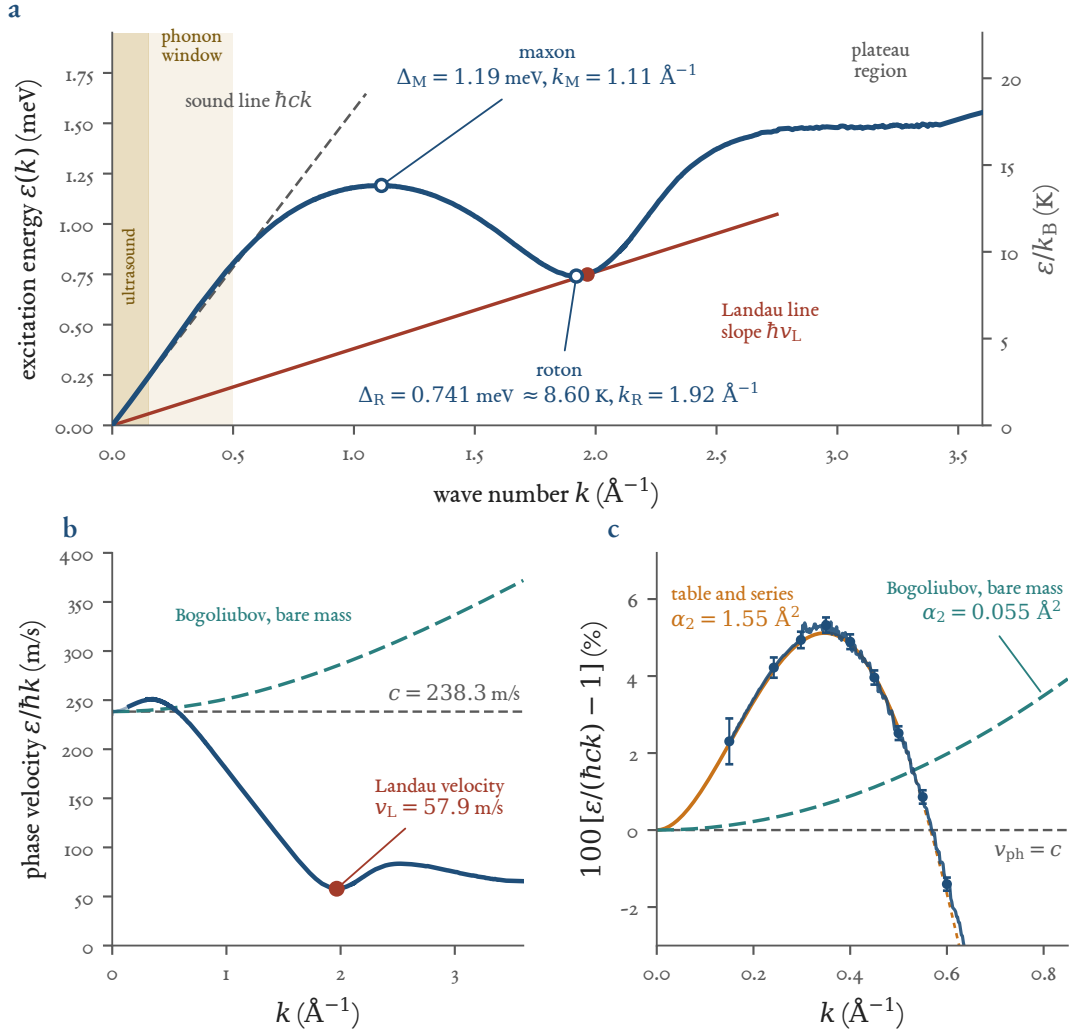


Figure 5.1: The dispersion relation of superfluid ^4He at saturated vapour pressure, from the authors' processed table that accompanies Godfrin *et al.* [2] (arXiv ancillary file; not raw counts). **a**, $\varepsilon(k)$ with the sound line $\hbar ck$ (grey, $c = 238.3 \text{ m s}^{-1}$) and the Landau line through the origin that touches the curve at the roton (red; its slope is $\hbar v_L$). Open circles: the maximum (maxon) and the minimum (roton) of the table. The strip below $0.15 \text{ \AA}^{-1}$ is, according to the caption of the paper's own table, ultrasound data; $0.15\text{--}0.3 \text{ \AA}^{-1}$ combines ultrasound and neutrons; above $0.3 \text{ \AA}^{-1}$ the values are neutron data. **b**, the phase velocity $\varepsilon/\hbar k$. Landau's critical velocity is its minimum, 57.9 m s^{-1} at $1.97 \text{ \AA}^{-1}$, i.e. $0.243 c$. Teal, dashed: the Bogoliubov formula with the bare atomic mass and the measured c , which never goes below c . **c**, the phonon window, $\varepsilon/(\hbar ck) - 1$ in per cent: table (with the uncertainties the file gives), the quartic series of Equation (5.4) (orange; solid inside its stated range $k < 0.5 \text{ \AA}^{-1}$) and the same Bogoliubov curve (teal). The phase velocity exceeds c by up to 5.4 per cent near $0.35 \text{ \AA}^{-1}$. Script: figures/ch05_curve.py.

measured curve; their Eq. (22) is the specific-heat series of Chapter 4.

What this chapter does with it. It reads the published curve (the open table of the arXiv version) as data, and asks which statements about excitations are *kinematic*, that is, consequences of the shape of the curve alone. The Lean library proves several of them as mathematics; the solver shows a different dispersion relation, Bogoliubov's, obeying the same statements.

A word on what the table is. It is the authors' own processed curve, not a list of raw counts; the raw data are a different object, and nothing in this book depends on them. Its composition matters for what can be inferred from it, and the caption of the table in the arXiv version says so: below $0.15 \text{ \AA}^{-1}$ the entries are ultrasound data, from 0.15 to $0.3 \text{ \AA}^{-1}$ ultrasound and neutron data are combined, and above $0.3 \text{ \AA}^{-1}$ they are neutron data. The printed version of the table quotes uncertainties of 0.001 to 0.002 meV for k between 0.2 and $2 \text{ \AA}^{-1}$. (This chapter quotes the arXiv version throughout; it has not been compared with the journal version of record.)

From arrival times to energies

How does one get from the clock of a detector to a point on the curve? The incident neutrons have a known energy E_i . For each detector angle one records the time t_{el} at which elastically scattered neutrons arrive (the aluminium of the cell does the elastic scattering; helium scatters none), and the time t_{in} of the helium peak. If t_s is the moment at which the neutron reaches the sample, the energy lost to the excitation is

$$\varepsilon = E_i \left[1 - \left(\frac{t_{\text{el}} - t_s}{t_{\text{in}} - t_s} \right)^2 \right], \quad (5.1)$$

which is Eq. (13) of the arXiv version of the paper; the wave number follows from the scattering angle. Its companion, Eq. (12), expresses the same energy through the flight distance D and the two arrival times, $\varepsilon = \frac{1}{2} m_n D^2 [(t_{\text{el}} - t_s)^{-2} - (t_{\text{in}} - t_s)^{-2}]$, and the two are one formula once $D = v_i(t_{\text{el}} - t_s)$ and $E_i = \frac{1}{2} m_n v_i^2$ are used. That is a one-line algebraic fact, and it is proved in the Lean library of this programme [12]: `QuantumFluids.HeliumKinematics.tof_eq12_eq_eq13`. The analysis therefore rests on two instrumental parameters only (E_i and t_s), which is why the energy scale could be fixed *at one point*: the authors calibrated it at the roton, using the value $\Delta_R = 0.7418 \pm 0.001 \text{ meV}$ that a triple-axis spectrometer had given, and they remark that if a better roton gap becomes available the energies should be corrected proportionally. Equation (5.1) shows why proportionally: at fixed arrival times, changing E_i by a factor λ changes every ε by the same factor, which the library states as `QuantumFluids.HeliumKinematics.tofEnergy_rescale`.

It is worth saying how small the quantities are that the next sections will use, because that is what the measurement achieved. At $k = 0.3 \text{ \AA}^{-1}$ the bending of the phonon branch lifts the energy by only 23 micro-electronvolts above the sound line, and the excess that decides the three-phonon channel (Section 5.3) is at most 14.5 micro-electronvolts. At the incident energy used for the low wave numbers the energy resolution of the spectrometer is 0.07 meV (full width at half maximum)

according to the paper [2]: the effects are a third and a fifth of the width of a single line, and the table quotes an uncertainty of 1.0 micro-electronvolt near this wave number. Two things make that possible, both stated in the paper: the resolution function is well described by a Gaussian, so that the centre of a line can be found to a small fraction of its width, and the analysis fits the lines pixel by pixel on a detector of 384 tubes with 241 pixels each, averaging typically more than 70 values into each tabulated wave-number bin. A curve whose interesting features are a fraction of the width of the instrument's line is a curve that has been measured with great care, not read off.

Two consequences, one practical and one structural, are worth stating before any physics is done. The practical one: the roton gap of the table (0.741 meV at the minimum of the curve; the calibration input was 0.7418 meV) is tied to the calibration, so it cannot be read as an independent measurement. The structural one: whatever one concludes from the curve that does not change when $\varepsilon \rightarrow \lambda\varepsilon$ is immune to the largest systematic uncertainty of the experiment. Zero crossings of energy differences, such as those of Section 5.3, are of that kind.

THE SHAPE DECIDES

Every statement of this chapter that is a theorem concerns the *shape* of the curve $\varepsilon(k)$: whether it bends up or down away from its tangent at the origin, whether $\varepsilon/\hbar k$ has a minimum below the sound speed, whether $\varepsilon(k_1 + k_2)$ exceeds $\varepsilon(k_1) + \varepsilon(k_2)$. None of them needs a microscopic theory of the liquid, which is why a proof assistant can state them, a published table can be asked for their numerical values, and a simulated field can be tested against them.

5.2 Landau's reading of the curve

Why does the shape of this curve matter so much? Because it decides whether the liquid is a superfluid. Landau's argument fits in a paragraph. Let a heavy body move through the liquid at velocity $\mathbf{V}$, the liquid being at rest and at zero temperature. The body can only slow down by creating an excitation of the liquid, of momentum $\hbar\mathbf{k}$ and energy $\varepsilon(k)$. Conservation of energy and momentum, with the body so heavy that its recoil energy is negligible, demands $\varepsilon(k) = \hbar\mathbf{k} \cdot \mathbf{V}$, which has a solution only if $V \geq \varepsilon(k)/\hbar k$ for some k . Below

$$v_L = \min_{k>0} \frac{\varepsilon(k)}{\hbar k} \quad (5.2)$$

no excitation can be created and the body moves without friction. The critical velocity of Landau is the *smallest value of the phase velocity*.

For a gas of free particles, $\varepsilon = \hbar^2 k^2 / 2m$, the phase velocity $\hbar k / 2m$ tends to zero with k , so $v_L = 0$: an ideal Bose gas is not a superfluid. The library proves exactly this, in the form in which it is used: for every $v > 0$ there is a wave number at which $\hbar k^2 / k < v$ (`QuantumFluids.HeliumKinematics.landau_velocity_parabolic_zero`). The same statement from the other side: if the dispersion lies on or above the sound line ck , then every phase velocity is at least c , so $v_L \geq c$ (`QuantumFluids.HeliumKinematics.landau_velocity_ge_of_above_sound_line`). A lin-

P (bar)	c (m/s)	Δ_R (meV)	k_R ($\text{\AA}^{-1}$)	v_L (m/s)	v_L/c
0.00	238.3	0.7413	1.920	57.9	0.243
0.51	242.6	0.7381	1.924	57.5	0.237
1.02	246.5	0.7357	1.920	57.2	0.232
2.01	253.9	0.7301	1.932	56.5	0.223
5.01	274.0	0.7138	1.966	54.8	0.200
10.01	302.3	0.6882	1.982	52.0	0.172
24.08	361.9	0.6256	2.048	46.1	0.127

Table 5.1: The Landau velocity against pressure, from the seven-pressure table that accompanies the paper (neutron data from $0.15 \text{ \AA}^{-1}$ to about $2.2 \text{ \AA}^{-1}$). c is the ultrasonic sound velocity quoted in the paper; Δ_R and k_R are the minimum of the tabulated curve; $v_L = \min_{k>1 \text{ \AA}^{-1}} \varepsilon/\hbar k$ always lies inside the tabulated range. The sound speed rises by 52 per cent over this pressure range while the Landau velocity falls.

ear start is therefore essential, and a curve that dips *below* the sound line must have a critical velocity below c . Helium does dip below it: Figure 5.1b shows the phase velocity of the table rising a little above c , peaking at 5.4 per cent above the ultrasonic value (normalising instead by the 238.5 m/s that the table itself gives at $0.01 \text{ \AA}^{-1}$ would make it 5.3 per cent: the first rows are rounded to 0.1 micro-electronvolt, so the table fixes c only to a few tenths of a per cent), and then falling nearly linearly to a minimum

$$v_L^{\text{table}} = 57.9 \text{ m s}^{-1} = 0.243 c \quad \text{at } k = 1.97 \text{ \AA}^{-1}, \quad (5.3)$$

just beyond the roton: the Landau line of Figure 5.1a is the chord from the origin that touches the curve from below. If the roton is approximated by Landau's own quadratic form, $\varepsilon \simeq \Delta_R + \hbar^2(k - k_R)^2/2\mu_R m_4$, the minimisation can be done in closed form (Exercise 1); with $\mu_R = 0.141$ and the gap and wave number of the table it gives 57.9 m s^{-1} , within 0.01 per cent of the table value.

The seven-pressure table that accompanies the paper allows the same computation under pressure (Table 5.1). As the pressure rises to 24.08 bar the roton deepens (Δ_R falls to 0.626 meV) and moves outward (k_R rises to $2.05 \text{ \AA}^{-1}$) while the sound speed rises by 52 per cent, to 362 m/s; the Landau velocity therefore *falls*, from 57.9 to 46.1 m/s, and its ratio to the sound speed is nearly halved, from 0.243 to 0.127. The sound speed, which is the Landau velocity of a weakly interacting superfluid and in any case an upper bound for it, grows with the density of the liquid; the quantity that actually sets the ceiling in helium, the roton, moves the other way.

One caution, in the spirit of this book. Equation (5.2) is the threshold of one process, the creation of a single elementary excitation. It does not claim that every flow of ^4He is dissipationless up to v_L : a flow can become dissipative earlier by other routes, the nucleation of vortices above all, which is the subject of Chapters 3, 6 and 9. What the table does give is the *ceiling*, and its value is set by the roton and not by the sound speed.

5.3 Three ways a phonon can fail to live

5.3.1 The sign of the curvature

At low wave number the curve is written as a series about the sound line,

$$\varepsilon(k) = \hbar ck (1 + \alpha_2 k^2 + \alpha_3 k^3 + \alpha_4 k^4 + \dots), \quad (5.4)$$

with no linear term, $\alpha_1 = 0$ [2]. At saturated vapour pressure the ultrasonic measurement of Rugar and Foster gives $\alpha_2 = 1.55 \pm 0.01 \text{ \AA}^2$ when α_1 is taken to vanish [49], and the paper uses $\alpha_2 = 1.55 \text{ \AA}^2$, $\alpha_3 = -4.04 \text{ \AA}^3$ and $\alpha_4 = 2.30 \text{ \AA}^4$ to describe the curve for $k < 0.5 \text{ \AA}^{-1}$. In the paper's notation $\alpha_2 = -\gamma$, so positive α_2 is *anomalous* dispersion: the curve bends upward away from the sound line, as in the first part of Figure 5.1c.

Anomalous dispersion has a kinematic meaning. A phonon of wave number $k_1 + k_2$ may split into two collinear phonons of wave numbers k_1 and k_2 only if the energy it gives up is at least what the two products carry, $\varepsilon(k_1 + k_2) \geq \varepsilon(k_1) + \varepsilon(k_2)$. For the straight line $\varepsilon = \hbar ck$ this holds with equality at every k_1, k_2 : the decay is exactly resonant, and the first correction decides. With $\varepsilon = \hbar ck(1 + ak^2)$, $\hbar = 1$, the library has the identity and the equivalence that settle it:

LEAN 4 HELIUMKINEMATICS

```
def phononDisp (c a k : ℝ) : ℝ := c * k * (1 + a * k ^ 2)

theorem three_phonon_open_iff {c a k1 k2 : ℝ} (hc : 0 < c) (h1 : 0 < k1) (h2 :
  ↪ 0 < k2) :
  phononDisp c a k1 + phononDisp c a k2 ≤ phononDisp c a (k1 + k2) ↔ 0 ≤ a
```

The idea of the proof is two lines. The energy excess of the decay is $\varepsilon(k_1 + k_2) - \varepsilon(k_1) - \varepsilon(k_2) = 3ca k_1 k_2 (k_1 + k_2)$ (QuantumFluids.HeliumKinematics.three_phonon_excess, proved by ring), and for positive c, k_1, k_2 the prefactor is positive, so the sign of the excess is the sign of a . This is the statement behind the remark of the paper that phonon damping by three-phonon processes is possible up to a critical wave number when the dispersion is anomalous, and why it disappears at high pressure: the authors report that the dispersion becomes normal, $\alpha_2 \leq 0$, at about 20 bar [2], and for normal dispersion the equivalence says the channel is closed. What it proves is the *mathematical* implication, for the model $\varepsilon = \hbar ck(1 + ak^2)$; the sign of a in helium is measured, not proved.

5.3.2 Where the channel closes: three numbers, not one

The quartic series is good only for $k < 0.5 \text{ \AA}^{-1}$, and the decay channel closes inside that range, so the interesting question is *where*. Here the library earns its place, because each way of asking the question gives a different number, and the three are easy to conflate. Write $\varepsilon = \hbar ck(1 + \alpha_2 k^2 + \alpha_3 k^3 + \alpha_4 k^4)$.

Three identities, each proved by the kernel, give

$$\frac{\varepsilon(k)}{\hbar k} - c = c k^2 (\alpha_2 + \alpha_3 k + \alpha_4 k^2), \quad (5.5)$$

$$\frac{d\varepsilon}{\hbar dk} - c = c k^2 (3\alpha_2 + 4\alpha_3 k + 5\alpha_4 k^2), \quad (5.6)$$

$$\varepsilon(k) - 2\varepsilon(k/2) = c k^3 \left(\frac{3}{4}\alpha_2 + \frac{7}{8}\alpha_3 k + \frac{15}{16}\alpha_4 k^2 \right). \quad (5.7)$$

The first (`QuantumFluids.HeliumKinematics.phase_velocity_excess`) says where the curve crosses back through the sound line: $v_{\text{ph}} = c$. The second (`QuantumFluids.HeliumKinematics.group_velocity_excess`) is the group velocity: a phonon can emit a very soft phonon, the limit $q \rightarrow 0$ of the split $k \rightarrow q + (k - q)$, only while its group velocity exceeds c — a Cherenkov condition. The third is the symmetric split $k \rightarrow k/2 + k/2$, whose energy excess is the last to change sign; its statement is short enough to quote:

LEAN 4 HELIUMKINEMATICS

```
def seriesDisp (c α2 α3 α4 k : ℝ) : ℝ := c * k * (1 + α2 * k ^ 2 + α3 * k ^ 3
  ↪ + α4 * k ^ 4)

theorem symmetric_split_excess (c α2 α3 α4 k : ℝ) :
  seriesDisp c α2 α3 α4 k - 2 * seriesDisp c α2 α3 α4 (k / 2)
  = c * k ^ 3 * (3 / 4 * α2 + 7 / 8 * α3 * k + 15 / 16 * α4 * k ^ 2)
```

It is a polynomial identity (the proof is `unfold; ring`), so it holds for *every* choice of coefficients, which is its use: the decay threshold of any set of measured α_j is read off a quadratic. Each right-hand side of Equations (5.5) to (5.7) is $c k^n$ times a quadratic in k , so each threshold is the smaller root of an explicit quadratic. With the coefficients above they are $k_g = 0.404$, $k_{\text{sym}} = 0.455$ and $k_{\text{ph}} = 0.566 \text{ \AA}^{-1}$ (the other roots, near $1 \text{ \AA}^{-1}$, lie far outside the range of the series).

Figure 5.2 shows them, and shows the table agreeing. Computed straight from the table, without the series, the symmetric split changes sign at $k = 0.453 \text{ \AA}^{-1}$, against 0.455 from the series; the maximum of the excess is 14.5 micro-electronvolts at $0.31 \text{ \AA}^{-1}$, about three per cent of the energy $\varepsilon(k)$ there, and the series reproduces the table to 2.2 micro-electronvolts at most for $0.25 \leq k \leq 0.5 \text{ \AA}^{-1}$. This agreement is a consistency check and not an independent confirmation: the coefficients of the series were fitted by the authors to the very curve that the table tabulates. What it does confirm is the kinematics: the soft end of the channel shuts first, near $0.404 \text{ \AA}^{-1}$, then the symmetric split, and the two should not be mistaken for the sound-line crossing at $0.566 \text{ \AA}^{-1}$, which closes nothing.

The calibration remark of Section 5.1 pays off here, with a distinction. The symmetric-split threshold is the zero of a function that is homogeneous of degree one in the energy scale, so it does not move at all if the roton-gap calibration is revised. The other two compare the curve with the

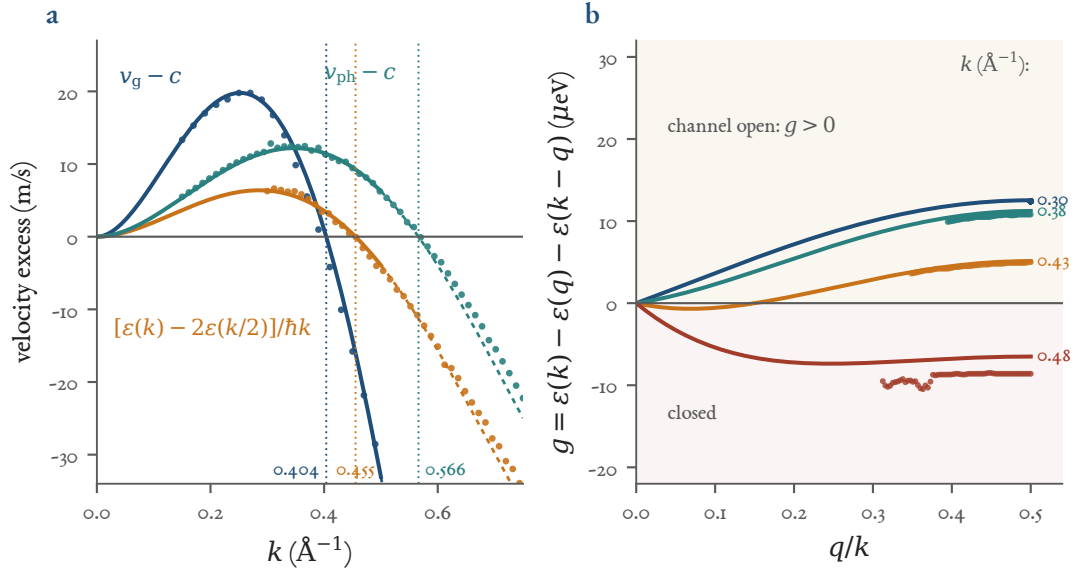


Figure 5.2: Where the three-phonon channel closes. **a**, the three excess velocities of Equations (5.5) to (5.7) computed from the quartic series (lines; dotted beyond its stated range) and directly from the table (dots; the group velocity is a local linear fit over $\pm 0.07 \text{\AA}^{-1}$). The zero crossings, in $\text{\AA}^{-1}$, are 0.404 ($v_g = c$, soft emission), 0.455 (symmetric split) and 0.566 ($v_{ph} = c$). **b**, the energy balance $g = \varepsilon(k) - \varepsilon(q) - \varepsilon(k-q)$ of a collinear decay $k \rightarrow q + (k-q)$ against q/k for four wave numbers (lines: series; dots: table, where both q and $k-q$ are at least $0.15 \text{\AA}^{-1}$). For $k = 0.43 \text{\AA}^{-1}$, between the first and the second threshold, the channel is closed at the soft end and open for near-symmetric splits. Script: `figures/ch05_thresholds.py`.

sound speed measured by ultrasonics, and are invariant only if that speed were rescaled with the curve; for a proportional change of the energy scale by the 2.1×10^{-3} that the paper quotes as the relative systematic uncertainty, they move by at most $0.002 (k_g)$ and $0.005 \text{ \AA}^{-1} (k_{ph})$.

5.3.3 Two rotons and the plateau

There is a third kind of decay, with the same logic and a different threshold. Above the roton the curve flattens into the shoulder that the authors, following the literature, call Pitaevskii's plateau, the energy at which one excitation can decay into two rotons [2, 5]. Two rotons of wave number k_R carry a total momentum between 0 (anti-parallel) and $2k_R$ (parallel), so the plateau, which is the locus of the decay, cannot extend beyond $2k_R$ in the collinear configuration and can in principle extend down to zero wave number through non-collinear ones, as the review explains [5]. The geometric core of this is the triangle inequality, and it is a theorem for any inner-product space: `QuantumFluids.HeliumKinematics.two_roton_momentum_le`, with the extremes attained by `QuantumFluids.HeliumKinematics.two_roton_parallel` and `QuantumFluids.HeliumKinematics.two_roton_antiparallel`. The plateau is not used quantitatively in this book, and nothing is claimed about the data there.

5.4 Bogoliubov's phonon, and what it cannot do

5.4.1 A phonon that turns into a particle

There is one quantum fluid in which the whole dispersion relation can be derived on a page, and it is the one the solver simulates. Take the Gross–Pitaevskii equation with $\hbar = m = 1$,

$$i \partial_t \psi = -\frac{1}{2} \nabla^2 \psi + g |\psi|^2 \psi, \quad (5.8)$$

and the uniform solution $\psi_0 = \sqrt{n} e^{-i\mu t}$ with $\mu = gn$. Write $\psi = e^{-i\mu t} (\sqrt{n} + \delta)$ and keep terms linear in δ : $i \partial_t \delta = -\frac{1}{2} \nabla^2 \delta + gn(\delta + \delta^*)$. A plane-wave pair $\delta = u e^{i(kx - \omega t)} + v^* e^{-i(kx - \omega t)}$ turns this into a two-by-two eigenvalue problem whose determinant gives

$$\omega^2 = \frac{1}{2} k^2 (\frac{1}{2} k^2 + 2gn) \iff \varepsilon(k) = \sqrt{c^2 \hbar^2 k^2 + (\hbar^2 k^2 / 2m)^2}, \quad c^2 = \frac{gn}{m}, \quad (5.9)$$

the formula Bogoliubov found in 1947 [50] (see also [26, 51]). It is a phonon, $\varepsilon \simeq \hbar c k$, below the wave number

$$k_* = \frac{2mc}{\hbar}, \quad (5.10)$$

and a free particle, $\varepsilon \simeq \hbar^2 k^2 / 2m$, above it; the wave number k_* is the inverse of the healing length of the condensate up to a numerical factor (with $\xi = \hbar / mc$, the convention of the solver, $k_* \xi = 2$). The phonon *turns into a particle* as k increases, and the whole curve depends on the single wave

number k_* : for $c, m, k > 0$,

$$\frac{\varepsilon(k)}{ck} = \sqrt{1 + (k/k_*)^2} \quad (5.11)$$

(`QuantumFluids.BogoliubovDispersion.phase_velocity_universal`). Everything that follows about Bogoliubov's curve is a statement about this one function. These excitations are waves of density and of phase in a single-valued ψ : the velocity they carry, $\hbar \nabla S / m$ for $\psi = \sqrt{\rho} e^{iS}$, is the gradient of a single-valued phase, so it is irrotational and carries no circulation. They are the first of the two kinds of excitation of the same field distinguished in Chapter 3; the second kind, the vortices, is not in the linearised equation at all.

5.4.2 What is proved about it

Several properties of Equation (5.9) are used below, and each is a theorem in the new Lean module written for this chapter, the file `Ch05_BogoliubovDispersion`, whose namespace is `QuantumFluids.BogoliubovDispersion`; the module is new and is not part of the library `QuantumFluids`. Its definitions are the Bogoliubov energy and the curvature coefficient, and its first theorem is the dictionary between the solver's parameters and the formula, $\varepsilon^2 = \frac{1}{2}k^2(\frac{1}{2}k^2 + 2gn)$ when $c^2 = gn$ and $m = 1$ (`QuantumFluids.BogoliubovDispersion.bogEps_sq_gp`), which is the expression the solver's known answer `K5` is tested against in Section 5.5. The module proves:

- ε is strictly increasing on $[0, \infty)$ (`QuantumFluids.BogoliubovDispersion.bogEps_strictMono0n`): there is no maxon, no roton, no stationary point at all;
- $\varepsilon(k_1) + \varepsilon(k_2) < \varepsilon(k_1 + k_2)$ for all $k_1, k_2 > 0$ (`QuantumFluids.BogoliubovDispersion.bogEps_superadditive`): the collinear decay of a phonon into two is open at *every* wave number, not only below a critical one — the spectrum is anomalous throughout, in the sense of Section 5.3, which is the kinematic precondition of what the literature calls Beliaev damping (its rate is not computed here);
- $\varepsilon(k) - ck$ is strictly increasing (`QuantumFluids.BogoliubovDispersion.bogEps_sub_sound_strictMono0n`): the energy of an excitation outruns the sound line at every wave number, the derivative-free form of the statement that the group velocity exceeds c ;
- the low- k expansion has the curvature coefficient $\alpha_2 = 1/(8m^2c^2)$, with the explicit error band below;
- the Landau velocity of the spectrum is c , exactly.

The last two are worth displaying. The first is the quantitative form of the expansion, a two-sided bound whose proof is the elementary inequality $\sqrt{1+2t} \leq 1+t$ and its refinement $\sqrt{1+2t} \geq 1+t-t^2/2$:

LEAN 4 BOGOLIUBOV DISPERSION (NEW FOR THIS BOOK)

```
noncomputable def bogEps (c m k : ℝ) : ℝ := Real.sqrt (c ^ 2 * k ^ 2 + (k ^ 2
  ↪ / (2 * m)) ^ 2)
```

```

noncomputable def alpha2 (c m : ℝ) : ℝ := 1 / (8 * m ^ 2 * c ^ 2)

theorem abs_bogEps_sub_phononDisp_le {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk
  ↪ : 0 ≤ k) :
  |bogEps c m k - phononDisp c (alpha2 c m) k| ≤ c * (alpha2 c m) ^ 2 * k ^
  ↪ 5 / 2

theorem landau_velocity_eq {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  IsGLB ((fun k => bogEps c m k / k) '' Set.Ioi 0) c

```

Here `phononDisp` is the definition of the library repeated verbatim in the new file, so that the module can be compiled alone. In words: the Bogoliubov energy differs from $ck(1 + \alpha_2 k^2)$ by at most $c\alpha_2^2 k^5/2$, so the curvature coefficient of the phonon branch is $1/(8m^2c^2) > 0$ and the error of the quadratic correction is of order k^5 (`QuantumFluids.BogoliubovDispersion.alpha2_limit` states the limit); and the infimum of the phase velocity ε/k over $k > 0$ is exactly c and is not attained, because $\varepsilon/k = \sqrt{c^2 + k^2/4m^2} > c$ at every $k > 0$ (`QuantumFluids.BogoliubovDispersion.sound_lt_phase_velocity`). By Landau's criterion the critical velocity of this superfluid is therefore its sound speed and nothing less: no excitation of the Bogoliubov spectrum is cheaper per unit momentum than a phonon. The proof of the upper half of the band is short enough to show, in the form in which it was compiled; the auxiliary identity `eps_arg` that it uses $(c^2k^2 + (k^2/2m)^2 = (ck)^2(1 + 2\alpha_2k^2))$, by `field_simp; ring`) is not shown (proof abridged):

LEAN 4 BOGOLIUBOVDISPERSION (NEW FOR THIS BOOK)

```

theorem bogEps_le_phononDisp {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk : 0 ≤ k
  ↪ ) :
  bogEps c m k ≤ phononDisp c (alpha2 c m) k := by
have ha : 0 < alpha2 c m := by unfold alpha2; positivity
unfold bogEps phononDisp
rw [Real.sqrt_le_iff]
refine (<b>by positivity, ?</b>)
rw [eps_arg hc hm] -- (ck)^2 (1 + 2t), t = alpha2 k^2
have : (c * k) ^ 2 * (1 + 2 * (alpha2 c m * k ^ 2)) ≤ (c * k) ^ 2 * (1 +
  ↪ alpha2 c m * k ^ 2) ^ 2 := by
  apply mul_le_mul_of_nonneg_left _ (sq_nonneg _)
  nlinarith [sq_nonneg (alpha2 c m * k ^ 2)] -- 1+2t <= (1+t)^2
calc _ ≤ _ := this
  _ = _ := by ring

```

The lower half is the same argument with $(1 + t - t^2/2)^2 \leq 1 + 2t$, valid when $1 + t - t^2/2 \geq 0$; when that quantity is negative the lower bound is itself negative and holds trivially, because $\varepsilon \geq 0$. All of it was compiled with Lean 4 and Mathlib on the machine that ran the solver, with no sorry

and only the three standard axioms.

5.4.3 Against helium: how far is the real curve from the weak-coupling one?

Bogoliubov’s derivation assumes a dilute gas whose interaction can be replaced by one coupling constant. Liquid helium is neither. Still the *shape* of Equation (5.9) can be compared with the measured curve with no free parameter, because the formula has only two inputs, the mass and the sound speed, and both are known: the bare mass of the ^4He atom and $c = 238.3 \text{ m s}^{-1}$. With them the wave number of the problem is $k_* = 2mc/\hbar = 3.00 \text{ \AA}^{-1}$, which is *inside* the measured range, and the curvature coefficient of the phonon branch is $\alpha_2^{\text{Bog}} = 1/(2k_*^2) = 0.055 \text{ \AA}^2$, against the measured $1.55 \text{ \AA}^2$: the real curve bends away from the sound line 28 times faster. Figure 5.1b,c show the comparison; the Bogoliubov phase velocity climbs without limit while the measured one turns over at $k \approx 0.35 \text{ \AA}^{-1}$ and falls to a quarter of c .

The two-sided reading is the point of this book’s rule about analogies. The comparison measures *how far helium is from the hypotheses that Bogoliubov’s formula needs*; it does not test the formula, which is a theorem about the Gross–Pitaevskii equation, and it does not say that a gas with those parameters would behave like helium. A property that the roton depends on is not a property of the Gross–Pitaevskii equation with a contact interaction: a monotonically increasing ε has no minimum, whatever the value of g . The roton needs structure on the scale of the interatomic distance. Feynman’s account is that the energy of the excitation is, at best, $\hbar^2 k^2 / 2mS(k)$, with $S(k)$ the static structure factor, and that this is an *upper bound* to the true excitation energy [47, 52]; the minimum of ε sits where $S(k)$ has its first peak, the signature of the short-range order of the liquid. In the weak-coupling theory $S(k) = (\hbar^2 k^2 / 2m) / \varepsilon(k)$ and the bound is saturated; in helium it is not, and the paper itself remarks that the Bijl–Feynman spectrum is “clearly very far from the experimental result” [2].

A different mechanism produces a roton minimum in a Bose gas, and the difference is the lesson of this section. Give the atoms a non-local interaction whose Fourier transform $V(k)$ becomes negative in a window, as for dipolar atoms, and the Bogoliubov spectrum becomes $\omega^2 = \epsilon_k(\epsilon_k + 2nV(k))$ with $\epsilon_k = \hbar^2 k^2 / 2m$: it can have a roton minimum, and such rotons have been seen in a dipolar quantum gas [53, 54]. That is a mean-field roton produced by the sign of an interaction in k -space; the helium roton is the signature of short-range order. The two share a name and a shape, and the properties that depend on the mechanism — what the roton mass is, how it responds to pressure, what it does at the liquid–solid transition — are not transferable from one to the other without checking (Exercise 3).

5.5 The solver as a spectrometer

The projected Gross–Pitaevskii equation is the one quantum fluid of this book whose dispersion relation is known in closed form, and that makes it a test bench: if the excitations of the simulated field do not obey Equation (5.9), the instrument is wrong. The instrument is `qf-pgpe`, the Rust

engine of rusty-SUNDIALS that Chapter 2 describes. It integrates $i \partial_t \psi = P[-\frac{1}{2} \nabla^2 \psi + g|\psi|^2 \psi]$ on a doubly periodic box with an integrating-factor fourth-order Runge–Kutta step (the linear part exact, the cubic part by RK4) and a sharp circular cutoff P at $k_{\text{cut}} = k_{\text{max}}/2$, which is the condition under which the cubic term does not alias into the retained modes. The crate’s own documentation records agreement with the numpy engine of the programme to 10^{-10} over twenty time units [21]. Units are $\hbar = m = n_0 = 1$ and $g = 1$, so that $c = 1$, $\xi = \hbar/mc = 1$ and $k_* = 2$. The box is $L = 64 \xi$ on a 128×128 grid, which retains 3209 wave vectors with $|k| \leq \pi/\xi$.

5.5.1 Exciting every mode at once

A time-of-flight spectrometer covers hundreds of scattering angles at once, and the simulation can do the same. Start from the uniform condensate $\psi = 1$ and add a weak random complex amplitude, of size $\eta = 10^{-6}$, to every retained mode; evolve for $T = 300$; every 0.4 time units record the amplitude $c_k(t)$ of every mode. In the linear regime each mode then oscillates as a superposition of the two normal modes of the pair $\pm k$, so that, with the proper frame (next paragraph),

$$s_k(t) = \alpha_k e^{-i\omega_k t} + \beta_k e^{+i\omega_k t}, \quad (5.12)$$

two complex amplitudes and one frequency per mode. For each mode the frequency is the value of ω that minimises the residual of the best linear fit of the two amplitudes; the residual is evaluated in closed form, and a golden-section search, started from a one-step linear-prediction estimate, finds the minimum in 28 evaluations. The half-records give a model-free error proxy: the two halves of the record are fitted separately and the difference is reported. One run yields 3208 measurements of the dispersion relation.

5.5.2 The frame lesson

The record is kept in the *condensate frame*, $s_k = c_k c_0^*/|c_0|$, where c_0 is the amplitude of the $k = 0$ mode, and the reason is an episode that the programme recorded as a failure. In its pre-registered list of known answers for the engine, the Bogoliubov test (called K5) failed at the first attempt, by factors of 5, 2.5 and 1.2 in the frequency, before any physics was run [55]. The cause was not the engine. The equation has no chemical potential in it, so the condensate rotates in the laboratory frame as $e^{-i\mu t}$ with $\mu = gn_0$, and every mode amplitude inherits that phase: the measured frequencies were $\mu \pm \omega$. The record of this run shows it directly. For the mode (16, 0), $k = 1.571$, the Bogoliubov frequency is $\omega = 1.9974$, and the laboratory-frame record has its two spectral peaks at -2.9975 and $+0.9974$, which are $-(\mu + \omega) = -2.9974$ and $-\mu + \omega = +0.9974$ to within the 10^{-3} resolution of the peak position. Reading the larger peak as the excitation frequency gives about 3 where the answer is 2; reading the smaller gives about 1. Dividing by the condensate’s phase fixed the test. The same logic holds for the experiment: a neutron’s energy loss $E_i - E_f$ is measured against the ground state of the liquid, and the chemical potential of the liquid never appears. A frequency is a statement about a frame.

RUSTY-SUNDIALS THE BROADBAND PULSE AND THE CONDENSATE FRAME

```

import numpy as np, qf_pgpe
eng = qf_pgpe.Pgpe(N, L, g, dt, 0.5)          # n, L, g, dt, k_cut/k_max

def initial_pulse(eng, N, eta, seed):          # (docstring removed)
    rng = np.random.default_rng(seed)
    mask = mode_set(eng, N)
    c = np.array(eng.uniform(), dtype=complex)
    xi = (rng.normal(size=(N, N)) + 1j * rng.normal(size=(N, N))) / np.sqrt(2.0)
    add = eta * N ** 2 * xi * mask
    add[0, 0] = 0.0
    return c + add, mask

def record_condensate_frame(run_step, c0, mask, n_samples, N, keep_lab=None):
    c = c0                                     # (docstring removed)
    nm = int(mask.sum())
    out = np.empty((n_samples, nm), dtype=np.complex128)
    lab = np.empty(n_samples, dtype=np.complex128) if keep_lab is not None else None
    for j in range(n_samples):
        if j > 0:
            c = run_step(c)                   # eng.run(c, 0.4): 20 steps
            c = np.asarray(c)
            ph = np.conj(c[0, 0]) / abs(c[0, 0]) # condensate frame: remove exp(-i mu t)
            out[j] = c[mask] * ph / N ** 2
            if lab is not None:
                lab[j] = c[keep_lab] / N ** 2
    return out, lab

```

The code is the code that was run (figures/ch05_common.py; the main run is figures/ch05_dispersion_run.py). The main run, 15 000 steps of a 128^2 grid, took 65 seconds of wall time, plus 12 seconds of fitting, on the shared eight-core machine of this project while its load average was between ten and thirty-six; the figure is indicative of the order of magnitude and not a benchmark.

5.5.3 What the measurement shows

Figure 5.3a shows the result: all 3208 frequencies, over almost a decade and a half in k , lie on Bogoliubov's curve, bending from the sound line to the free-particle parabola across k_* . The deviations (Figure 5.3b) have a median of 0.60 parts per million, 99 per cent of the modes are within 8.5 ppm and the worst is 26 ppm. The error is not uniform in k : the median is 3.8 ppm below $k = 0.4$ and 0.33 ppm above $k = 2.5$, because the slowest modes complete only a few periods in the record; the half-record difference, which is blind to the model, has a median of 1.2 ppm and says the same. So this is a statement about the fit as much as about the engine, and it is a good one: the engine's frequencies equal the closed form to a few parts in 10^7 .

Three further checks sit in the same figure. The second run, with $g = 2$, gives a median of 0.42

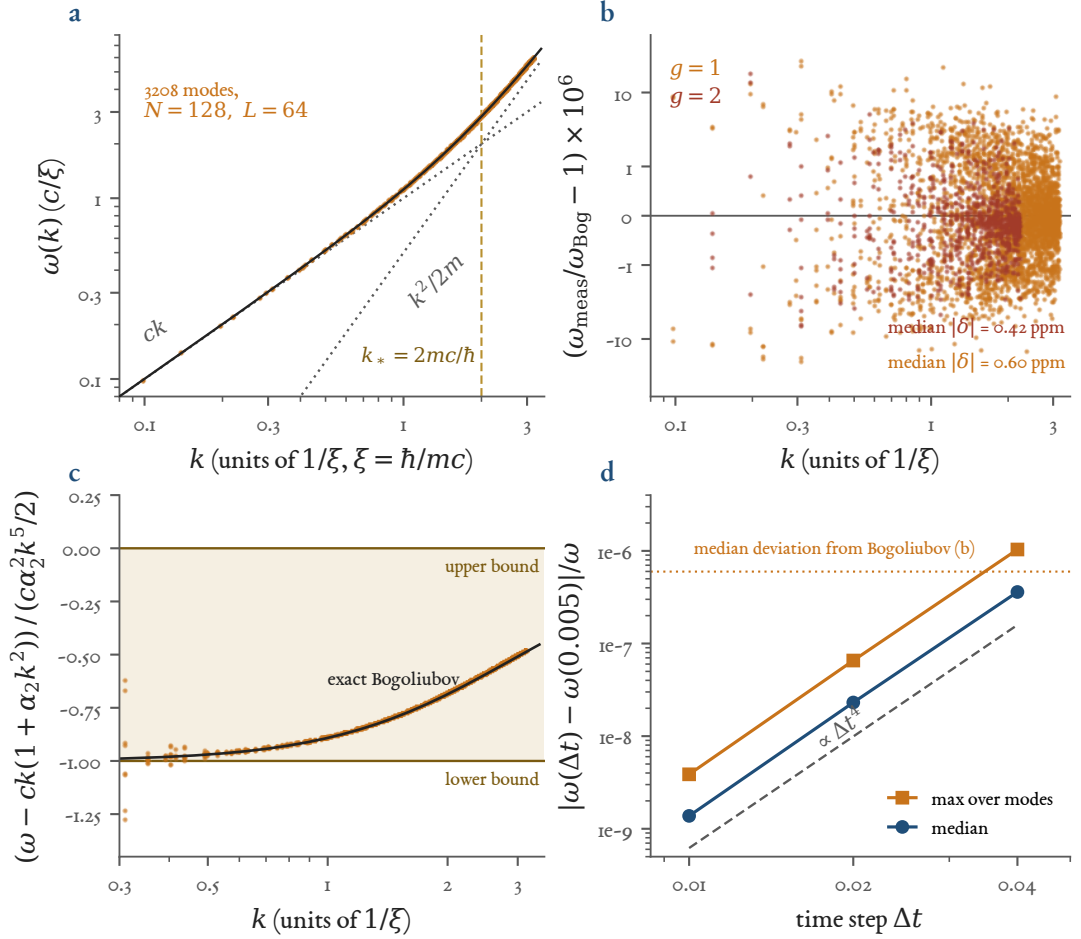


Figure 5.3: The solver measures Bogoliubov's dispersion relation. **a**, the frequencies of all 3208 modes of the 128^2 run (orange) on the Bogoliubov curve (black) between its two asymptotes (dotted). **b**, the relative deviation of every mode from ω_{Bog} , in parts per million, for $g = 1$ and for a second run with $g = 2$ ($c = \sqrt{2}$; the wave number is in units of $1/\xi = mc/\hbar$). **c**, the band proved in Lean: the quantity $(\omega - ck(1 + \alpha_2 k^2))/(c\alpha_2^2 k^5/2)$ lies in $[-1, 0]$ for the Bogoliubov spectrum, and the measured points follow the exact curve (black) from the lower edge, which is asymptotically sharp as $k \rightarrow 0$. Below $k \approx 0.5$ the noise of the fit, magnified by the k^{-5} of the normalisation, takes some points out of the band; the band is a theorem about the formula, the points are measurements. **d**, the time-step ladder: differences between the frequencies obtained with $\Delta t = 0.04, 0.02, 0.01$ and those of the same pulse with $\Delta t = 0.005$; they fall by a factor 15.6 and then 16.8 per halving, the fourth-order law of the scheme. Script: `figures/ch05_dispersion_run.py` (data), `figures/ch05_solver_fig.py` (figure).

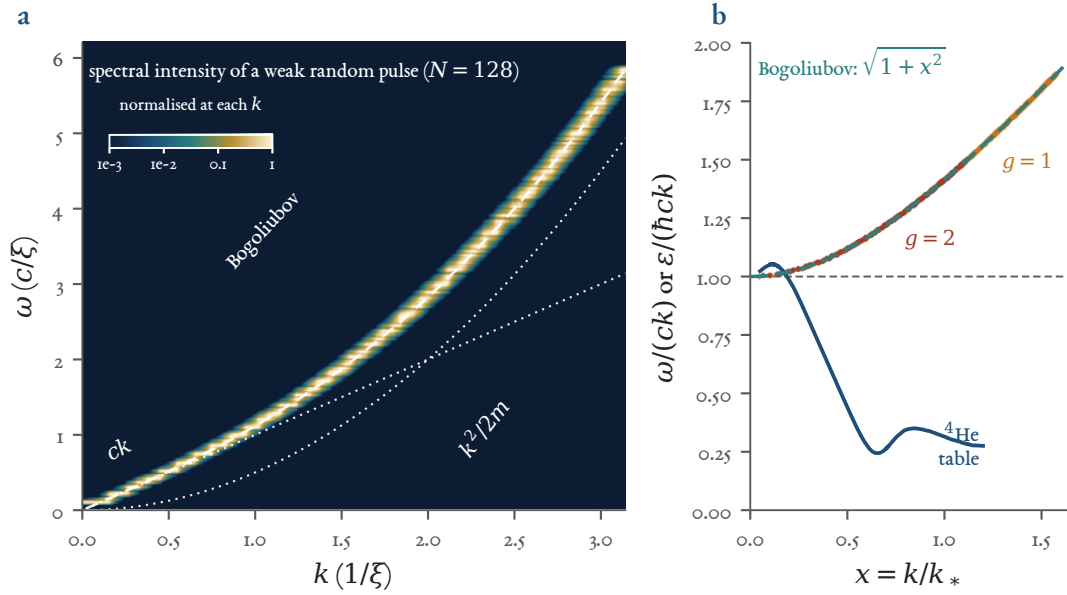


Figure 5.4: A spectrometer map of the simulated fluid, and the universal curve. **a**, the power spectrum of the record of every mode, folded onto positive frequency, smeared in k with a Gaussian of width $0.03/\xi$ (a display resolution, like the finite angular resolution of an instrument), and normalised at each k . A bright ridge follows the Bogoliubov curve (white, dashed) from the sound line to the free-particle parabola (dotted). The intensity along the ridge is set by the random pulse and has no physical meaning; only the position of the ridge does. **b**, the phase velocity in units of c against $x = k/k_*$, $k_* = 2mc/\hbar$: the $g = 1$ run (orange) and the $g = 2$ run (red) fall on the single curve $\sqrt{1 + x^2}$ (teal) of Equation (5.11), to within 2.6×10^{-5} and 1.7×10^{-5} ; the measured curve of ^4He (blue, with the bare atomic mass and $c = 238.3$ m/s in k_*) leaves it at $x \approx 0.1$ and has no part in common with it beyond. Script: figures/ch05_solver_fig.py.

ppm over 796 modes: the dictionary $c^2 = gn_0$ of `QuantumFluids.BogoliubovDispersion.bogEps_sq_gp` is right. The Lean theorem `QuantumFluids.BogoliubovDispersion.abs_bogEps_sub_phononDisp_le` meets data in the band of Figure 5.3c: for the 3128 modes with $k \geq 0.5$ all of the points lie inside $[-1, 0]$, between -0.976 and -0.488 , and the largest distance from the exact curve is 0.009 in these units. The ladder of Figure 5.3d is the integrator's own order: the median difference to the finest run is 3.6×10^{-7} , 2.3×10^{-8} and 1.4×10^{-9} for $\Delta t = 0.04$, 0.02, 0.01, a drop of 15.6 and 16.8 per halving. At the step used for the main run, $\Delta t = 0.02$, the time-stepping error is therefore a factor 26 smaller than the fit's own scatter.

5.5.4 One function of one variable

Equation (5.11) says that the whole curve is a function of one variable, and the two runs test it. In the coordinates $x = k/k_*$ and $y = \omega/(ck)$ the $g = 1$ and the $g = 2$ runs, whose sound speeds differ by $\sqrt{2}$ and whose cutoffs sit at different x , collapse on $\sqrt{1 + x^2}$ to within 2.6×10^{-5} and

1.7×10^{-5} (Figure 5.4b). Helium, in the same coordinates with the bare atomic mass, does not: it leaves the universal curve at $x \approx 0.1$, rises to 1.054 and falls to 0.25 at the roton. The two curves have the same two inputs and nothing else in common.

Figure 5.4a is the same data seen as an experimentalist sees $S(Q, \omega)$: a map in the (k, ω) plane, with a bright line where the excitations are. It is, if one likes, a numerical neutron-scattering experiment. It shows no maxon and no roton, only a ridge that never turns over, and the point of the figure is that contrast: the line a neutron spectrometer draws for helium (Figure 5.1a) has structure that a contact-interaction Gross–Pitaevskii field cannot produce. Two caveats keep the analogy honest. The intensity along the ridge is an artefact of our random pulse; the physical weight of the single-excitation peak is the static structure factor $S(k) = (\hbar^2 k^2 / 2m) / \varepsilon(k)$ of Bogoliubov’s theory, and a classical field at zero temperature has no way to produce it. And the lines of the map are as sharp as the window allows, because the field is integrated for three hundred time units without damping; what limits the width of the excitation peaks in real helium — the decay channels of Section 5.3 — is not computed here.

5.5.5 Waves in space: the group velocity

The frequency of one wave number is one way to see the dispersion; a wave packet is another, and it measures a different quantity, the group velocity. A localised density pulse contains all wave numbers. Each travels at its own group velocity $v_g = d\omega/dk = (k + k^3/2)/\omega$ (units $c = m = \hbar = 1$), so that the stationary-phase argument puts the wave number k at the radius $r = v_g(k) t$: a snapshot of the density is a map of the dispersion relation read through its derivative. Since $\varepsilon(k) - ck$ is increasing (QuantumFluids.BogoliubovDispersion.bogEps_sub_sound_strictMonoOn), the mean group velocity between any two wave numbers exceeds c , and the ripples run ahead of the front. Figure 5.5 shows it. The local wave number of the ripples, measured in the range $r = 11$ to 28ξ (wave numbers 0.8 to $3.1/\xi$), follows the stationary-phase prediction with a median relative deviation of 3.4 per cent (largest: 13 per cent at the edges of the window, where the phase of the analytic signal is least reliable). No fit and no free parameter enters: the dispersion relation has been read off a single snapshot, and it is the same one that was read off 3208 time series.

Panel c is the comparison with the helium of Section 5.1, in the universal coordinate x . The group velocity of Bogoliubov’s fluid exceeds c at every wave number and grows without bound ($3.19 c$ at the cutoff of the simulation). The group velocity of helium, computed from the table by a local linear fit, rises above c by 8.5 per cent near $0.27 \text{ \AA}^{-1}$ and returns to c at $0.40 \text{ \AA}^{-1}$ (the series of Section 5.3 gives 0.404); it vanishes at the maxon, is negative between maxon and roton — excitations whose energy increases while their momentum decreases — and vanishes again at the roton. A pulse launched in helium would therefore send ahead of the sound front only the waves with k below about $0.40 \text{ \AA}^{-1}$, and the waves between the maxon ($1.10 \text{ \AA}^{-1}$) and the roton would travel backwards. Both statements are consequences of the shape of the table, and neither has been seen in a simulation here, since the engine cannot produce this curve.

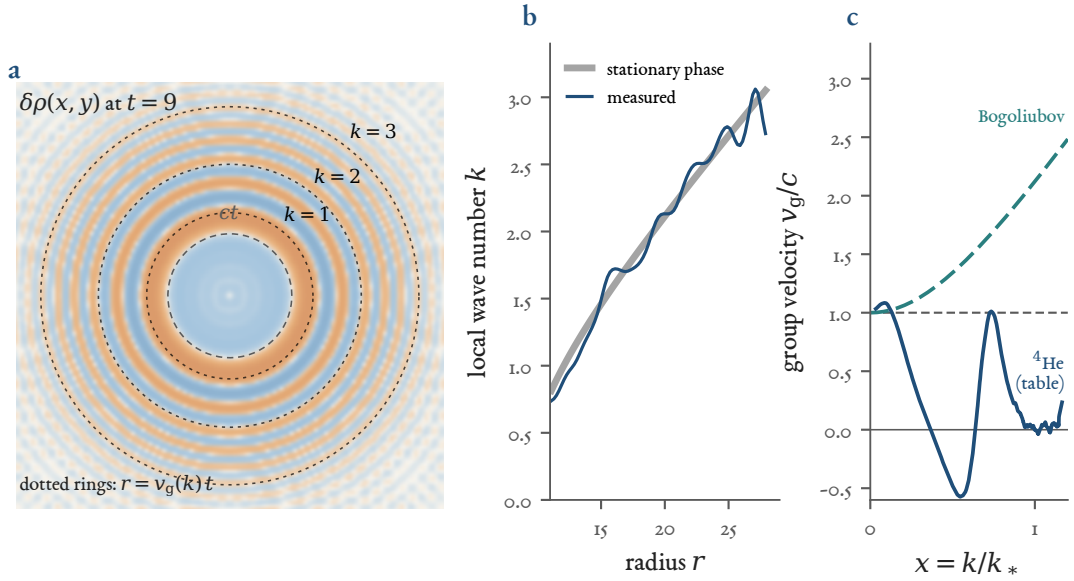


Figure 5.5: A dispersive ring wave. **a**, the density deviation $\delta\rho$ (symmetric logarithmic colour scale) at $t = 9$ after a Gaussian density bump of width 0.6ξ and height 0.02 at the centre of the box (zero initial velocity; 128^2 grid, $L = 64 \xi$, bicubic interpolation for display). The ring at $r = ct$ is the sound front; outside it the ripples are the short waves that run ahead, each wave number k at the radius $v_g(k)t$ (dotted circles for $k = 1, 2, 3$). **b**, the local wave number of the azimuthally averaged profile (blue; phase of the analytic signal of $\sqrt{r} \delta\rho$) against the stationary-phase prediction $v_g(k_s) = r/t$ (grey). **c**, the group velocity in units of c against $\chi = k/k_*$: Bogoliubov (teal, dashed) rises monotonically to $3.19c$ at the cutoff; for ^4He (blue, local linear fit to the table) it reaches only $1.08c$ near $0.27 \text{ \AA}^{-1}$, vanishes at the maxon ($1.10 \text{ \AA}^{-1}$), turns negative ($-0.57c$ at $1.63 \text{ \AA}^{-1}$) and returns to zero at the roton. Script: figures/ch05_rings.py.

5.5.6 A referee: CVODE

The engine is a fixed-step scheme of known order (Figure 5.3d); an integrator of a different kind is a useful second opinion. The projected equation is an ordinary differential system for $2n_{\text{modes}}$ real numbers, and CVODE, the variable-order solver of rusty-SUNDIALS, integrates it through its Python module `rusty_sundials` with the right-hand side supplied from Python. The interface returns the end state only, so a time series is built by chaining one call per sample interval; the Adams method is the sensible choice for a non-stiff problem.

RUSTY-SUNDIALS THE SAME TRAJECTORY UNDER CVODE

```
# abridged from figures/ch05_cvode.py
from rusty_sundials import CcodeSolver
scale = float(N ** 2)

def rhs(_t, y):
    # dc/dt of the projected GPE; y = (Re c, Im c)
    / N^2
    c = py.unpack(np.asarray(y) * scale) # py: numpy engine, same equation, same modes
    return (py.pack(py.rhs(c)) / scale).tolist()

solver = CcodeSolver("adams", rtol, atol, 10_000_000)
y = (py.pack(c0) / scale).tolist()
for j in range(1, ns):
    _, y = solver.solve(rhs, (j - 1) * dts, y, j * dts) # one call per sample interval
```

This cross-check has a history that belongs to the record. The software paper of the engine makes the same comparison, the engine against a CVODE reference on a box of 16×16 points with 49 modes, and reports that it depends on a fix of the library, “the Adams order was stuck at one”: without the fix the solver collapses its step on this oscillatory Hamiltonian problem and fails after about 10^6 right-hand-side evaluations [21]. The module installed in the project’s virtual environment predates the fix (it was installed on 26 September 2026), and Table 5.2 shows the difference from the outside: on a box of 8×8 points the older build needs 2, 14 and 92 times as many right-hand-side evaluations as the current one at tolerances 10^{-4} , 10^{-6} and 10^{-8} . At the same tolerance it is also less accurate (at 10^{-6} its frequencies differ from the engine’s by 1.7×10^{-3} , those of the current build by 1.7×10^{-5}). Its cost grows roughly as the inverse square root of the tolerance (by factors of 7 and 9 for each hundredfold tightening, where an exact square-root law gives ten) and its error falls by about 11 and 10 over the same steps, which is how a first-order method behaves when the step is set by an error that is quadratic in the step. The first attempt at this section was run on the older build and was abandoned for that reason; everything below uses the current build.

With the current build CVODE does what a referee should (Table 5.3). The box is that of the software paper’s reference, 16×16 points with 49 modes, integrated here for 20 time units. As the tolerance is tightened from 10^{-4} to 10^{-10} the frequencies that CVODE gives approach those of the finest run of the engine, the difference falling from 7.8×10^{-4} to 3.3×10^{-9} ; the last two rows are

tolerance (rtol)	older build		current build	
	right-hand sides	max $ \Delta\omega /\omega$	right-hand sides	max $ \Delta\omega /\omega$
10^{-4}	1 361	1.9×10^{-2}	599	3.8×10^{-4}
10^{-6}	9 730	1.7×10^{-3}	718	1.7×10^{-5}
10^{-8}	91 519	1.7×10^{-4}	992	3.5×10^{-7}

Table 5.2: The same experiment on two builds of the Python module. A box of 8^2 grid points (13 modes, 26 unknowns), $T = 4$, the same pulse. The older build is the module of the project’s virtual environment (installed on 26 September 2026); the current one was built from the rusty-SUNDIALS tree (commit 5db8041) that contains the fix of the Adams order.

tolerance (rtol)	right-hand sides	wall (s)	max $ \Delta\omega /\omega$	max $ \Delta s / s $
10^{-4}	6 229	4.8	7.8×10^{-4}	7.7×10^{-2}
10^{-6}	6 948	5.4	2.7×10^{-5}	1.6×10^{-3}
10^{-8}	8 313	7.0	2.7×10^{-7}	1.8×10^{-5}
10^{-10}	10 158	7.6	3.3×10^{-9}	8.0×10^{-7}

Table 5.3: CVODE against the Rust engine. The same random pulse ($\eta = 10^{-4}$) on a box of 16^2 grid points with 49 retained modes (98 real unknowns), integrated to $T = 20$ by CVODE’s Adams method (atol = 10^{-3} rtol) through the Python interface and by the engine ($\Delta t = 0.01$ and a four times smaller step as reference). Columns: right-hand-side evaluations, wall time on the shared machine, the largest relative difference between the frequencies read from the CVODE record and from the fine-step engine record, and the largest difference between the two mode records, relative to the rms amplitude of each mode. The engine’s own frequencies differ from its four-times-finer run by at most 5.3×10^{-7} , and the engine took 0.3 s for this record. Both records go through the same fit; the frequencies of this short record, in which the modes complete between 1.3 and 6.4 periods, differ from Bogoliubov’s formula by up to 3.3×10^{-4} , which cancels in the difference between the two records.

below the 5.3×10^{-7} by which the engine at its production step differs from that finest run. The finest run is not exact either: for a scheme of fourth order the error of the run with the four times smaller step is about $1/255$ of that difference, that is 2.1×10^{-9} , which is of the order of the last difference, so the comparison cannot resolve more. Two independent integrators, one of fixed step and fourth order and one of variable order and step, agree on the dispersion relation of the projected equation: the frequencies that the same fit reads from their records differ by 3.3×10^{-9} at most. What the referee cannot do is scale. Every right-hand-side evaluation is a call from CVODE into Python, where the numpy engine evaluates the equation and the state is converted to and from a list, and both the number of evaluations and the cost of each one grow with the box: for two time units at tolerance 10^{-6} the 26-unknown box needs 365 evaluations and 0.2 seconds, the 98-unknown box 710 and 0.5 seconds, and the 394-unknown box 2 291 and 9.4 seconds (wall time on the shared machine). The main run has 6 418 unknowns, 16 times as many as the largest box tried here, and was followed for three hundred time units instead of two; it was not attempted through this interface, and for it the evidence is the closed-form reference of Figure 5.3. CVODE stays what the software

check	result	where
frequencies of all modes against Bogoliubov's formula ($g = 1$)	median 0.60 ppm over 3208 modes; worst 26 ppm (lowest k)	Figure 5.3a,b
second coupling, $g = 2$	median 0.42 ppm over 796 modes	Figure 5.3b
collapse on $\sqrt{1+x^2}$	within 2.6×10^{-5} ($g = 1$) and 1.7×10^{-5} ($g = 2$)	Figure 5.4b
Lean error band, $k \geq 0.5$	all 3128 modes inside; largest distance from the exact curve 0.009	Figure 5.3c
order of the time step	differences fall by 15.6 and 16.8 per halving (fourth order: 16)	Figure 5.3d
group velocity from a snapshot	median 3.4 per cent over $k = 0.8\text{--}3.1/\zeta$	Figure 5.5b
CVODE against the engine	at rtol 10^{-10} : frequencies within 3.3×10^{-9} , records within 8.0×10^{-7}	Table 5.3

Table 5.4: The checks of this chapter at a glance. Each is an agreement between the engine and a closed form or a second integrator; none is a statement about helium.

paper made it, an independent reference on small systems and a regression guard for the library.

5.5.7 Amplitude: the dispersion relation is a small-amplitude statement

Everything above is at the amplitude $\eta = 10^{-6}$. The same engine, started from a single standing wave $\psi = 1 + \epsilon \cos kx$ and read in the same way, shows the limit of the concept. For the mode $k = 1.571/\zeta$ the relative frequency shift is -2.7×10^{-5} , -2.6×10^{-3} and -2.2×10^{-2} at $\epsilon = 0.01$, 0.1 and 0.3: it is quadratic in the amplitude, negative (the frequency softens), and reaches 2.2 per cent at $\epsilon = 0.3$. For the longest mode of that run, $k = 0.393/\zeta$, the two-frequency description of Equation (5.12) fails already at $\epsilon = 0.1$ (residual 0.19 of the signal): a nearly dispersionless wave drives its own harmonics nearly resonantly and the single-frequency picture does not survive, which is plausible but has not been tested further here. The dispersion relation $\omega(k)$ is the $\epsilon \rightarrow 0$ limit of a family, and the measurements above are taken in that limit.

5.6 What this establishes, and what it does not

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved (in Lean, as mathematics). The kinematic identities of the library module `HeliumKinematics` used in Sections 5.1 to 5.3 (the time-of-flight formulas, the Landau-velocity statements, the three-phonon excess, the three thresholds), which formalise statements of the paper and of the Godfrin–Krotscheck review; and, in the new module of this chapter, the properties of Bogoliubov's formula listed in Section 5.4. These are theorems about formulas. They say nothing about whether helium obeys them. The new module compiles with no

sorry; its 15 #print axioms lines list only propext, Classical.choice and Quot.sound.

Measured or simulated. The 3208 frequencies of the projected-GPE field equal Bogoliubov's formula to a median of 0.60 ppm; the $g = 2$ run, the time-step ladder, the Lean band, the ring-wave group velocity and the CVOICE referee agree as described above. The thresholds 0.404, 0.455 and $0.566 \text{ \AA}^{-1}$ are consequences of the series coefficients that the paper uses; the table reproduces the symmetric one (0.453), but the series was fitted to that table, so the agreement is a consistency check. The Landau velocity of 57.9 m/s and the group-velocity landmarks are properties of the *published, processed* curve, whose energy scale was calibrated at one point (the roton gap) and whose lowest wave numbers are ultrasound data.

Analogy only. The comparison of helium with Bogoliubov's formula with the bare atomic mass measures the distance from weak coupling; it is neither a test of the formula nor a theory of helium. The Gross–Pitaevskii field has no roton, no maxon and no negative group velocity; a roton in a dipolar gas is a different mechanism that shares a name. A property that depends on the mechanism has to be checked in each system (Exercise 3).

Not done, or failed. (i) Nothing here uses the raw ILL data, and nothing here reproduces the experiment: the solver cannot make a roton. (ii) Damping and lifetimes are not computed: the kinematic channels are open or closed, but the *rates* of the three-phonon decays, and the widths of the lines, are not. (iii) Landau's velocity is a threshold for one process; it is not the measured critical velocity of any flow. (iv) The first Bogoliubov known answer of the engine failed on a frame error and was repaired (Section 5.5); the CVOICE referee ran only on boxes of up to 394 unknowns, and its first version, on the older build of the module, had to be abandoned. (v) An earlier hypothesis of this programme, that the dimensionless ratio $\ell(k) = \varepsilon^2 / (\hbar^2 c^2 k^3)$ cannot fall below its Bogoliubov value, was measured on exactly this table and found violated at the roton by a factor of about twenty at saturated vapour pressure; its significance was then withdrawn (the programme's retraction R2), because $\ell = v_{\text{ph}}^2 / (c^2 k)$ and any fluid with a roton violates it. What remains is the textbook statement that the roton lies far below Bogoliubov's curve because helium is strongly correlated.

Exercises

Exercise 1. Near the roton write $\varepsilon \simeq \Delta + a(k - k_R)^2$ with $a = \hbar^2 / 2\mu_R m_4$. Show that $\varepsilon / \hbar k$ is minimal at $k_L = \sqrt{k_R^2 + \Delta/a}$ and that $\hbar v_L = 2a(k_L - k_R)$. With $\Delta = 0.7418 \text{ meV}$, $k_R = 1.918 \text{ \AA}^{-1}$, $\mu_R = 0.141$ and $\hbar^2 / 2m_4 = 0.522 \text{ meV \AA}^2$, evaluate v_L and compare it with the value read from the table. *Hint.* Differentiate $(\Delta + a(k - k_R)^2) / k$; the numerator of the derivative factorises as $a(k - k_R)(k + k_R) - \Delta$. The result is 58.0 m/s.

Exercise 2. Prove `QuantumFluids.BogoliubovDispersion.bogEps_superadditive` on paper: $\varepsilon(k_1 + k_2) > \varepsilon(k_1) + \varepsilon(k_2)$ for Bogoliubov's spectrum. Then say which step of your

argument fails for the helium table, and for which wave numbers. *Hint.* Use $\varepsilon(k) = ks(k)$ with $s = \sqrt{c^2 + k^2/4m^2}$ increasing, so that $(k_1 + k_2)s(k_1 + k_2) > k_1s(k_1) + k_2s(k_2)$. For helium $s = \varepsilon/k$ is the phase velocity, which increases only up to $k = 0.35 \text{ \AA}^{-1}$ (Figure 5.1b); beyond it the argument gives no information, and the actual closing of the symmetric channel, at $0.455 \text{ \AA}^{-1}$, needs the finer structure of Equation (5.7).

Exercise 3. For a non-local interaction with Fourier transform $V(k)$, take the Bogoliubov spectrum $\omega^2 = \epsilon_k(\epsilon_k + 2nV(k))$ with $\epsilon_k = k^2/2$ ($\hbar = m = 1$) and $nV(0) = c^2 = 1$. (a) Show that $\omega/k = \sqrt{k^2/4 + nV(k)}$ and conclude that the Landau velocity is $\min_k \sqrt{k^2/4 + nV(k)}$, which is below c as soon as $k^2/4 + nV(k) < 1$ somewhere. (b) Take $nV(k) = 1 - 3k^2e^{-k^2}$ and compute $\omega(k)$. Which of the Lean theorems of Section 5.4 no longer hold? *Hint.* Part (b) takes a few lines of Python; the curve has a maximum $\omega \simeq 0.350$ at $k \simeq 0.56$ and a minimum $\omega \simeq 0.304$ at $k \simeq 0.84$, and $\min \omega/k = 0.349$ at $k \simeq 0.90$. Strict monotonicity, the Landau value c and `QuantumFluids.BogoliubovDispersion.sound_lt_phase_velocity` fail; none of them is a property of the Gross–Pitaevskii equation as such, but of the contact interaction.

Notes and further reading

The measured curve and the thermodynamics built on it are in the paper of Godfrin and his collaborators [2], and the general account of the excitations of quantum fluids is the review of Godfrin and Krotscheck [5]; for the neutron-scattering literature, Glyde’s book and review are the standard references [8, 56], and Cowley and Woods [57] is one of the classic neutron measurements on which, in the paper’s account, the earlier quantitative knowledge of the curve rests. Landau’s spectrum is in [3, 46], Bogoliubov’s in [50] (a modern derivation in [26, 51]), Feynman’s account of the roton in [47, 52]. The Lean library and the module `HeliumKinematics` used here are described in [12]; the engine and its verification in [21]. The ultrasonic determination of the curvature α_2 is that of Rugar and Foster [49], and the theory of phonon–phonon interactions that gives anomalous dispersion its consequences is reviewed by Maris [58]. For rotons in dipolar gases, which are the mean-field relatives mentioned above, see [53, 54]. Chapter 4 is the thermodynamic counterpart of this chapter: the same curve, integrated over a thermal distribution; Chapter 8 meets the word “roton” again, in a two-dimensional Fermi liquid.

Vortices in Two Dimensions: the Kosterlitz–Thouless Flow

6.1 A jump fixed by constants

A thin film of liquid helium-4 is adsorbed on a substrate that is made to oscillate, and the film is cooled. Bishop and Reppy [39] read two things from the response of the oscillating substrate: the superfluid mass of the film and the dissipation. Both change near a temperature T_c , and the way they change was compared with a theory five years old. Kosterlitz and Thouless [40] had proposed that a two-dimensional superfluid does not lose its superfluidity gradually, as a bulk superfluid does, but abruptly, through the unbinding of pairs of vortices; Nelson and Kosterlitz [59] had drawn the consequence that the superfluid density must *jump* at the transition, by an amount fixed by constants of nature. The abstract of the letter reports that the superfluid mass and the dissipation, analysed with the dynamic theory of Ambegaokar, Halperin, Nelson and Siggia [60], support the Kosterlitz–Thouless picture, and that the predicted jump, $\rho_s(T_c^-) = 8\pi k_B (m/h)^2 T_c$, is in good agreement with estimates from the experiment.

Look at the right-hand side. Apart from the transition temperature, everything in it is a constant of nature: Boltzmann’s constant, Planck’s constant, and the mass m of a helium atom. With $m = 4.002\,603\,254\,13$ atomic mass units (the NIST value for ^4He) and the CODATA values of the constants shipped with SciPy, the number is

$$\frac{\rho_s(T_c^-)}{T_c} = 8\pi k_B \left(\frac{m}{h}\right)^2 = 3.49 \times 10^{-9} \text{ g cm}^{-2} \text{ K}^{-1},$$

computed by `figures/ch06_constants.py`. Theory says that the superfluid areal mass density just below the transition is this number times the temperature at which the transition happens, whatever the substrate, the thickness of the film, or the strength of the interatomic forces; the last sentence of the abstract is a comparison of that number with the experiment. In the units of this book, with the thermal wavelength $\lambda_T^2 = 2\pi\hbar^2/(mk_B T)$ and the superfluid number density $n_s = \rho_s/m$, the same statement is

$$n_s \lambda_T^2 = 4 \quad \text{at } T_c^-, \quad n_s \lambda_T^2 \rightarrow 0 \quad \text{above } T_c. \quad (6.1)$$

The superfluid density, measured in units of the thermal wavelength squared, falls from 4 to 0 at the

transition.

This chapter asks why. Why should a two-dimensional superfluid lose its stiffness abruptly, and why at a value fixed by $\hbar$, m and k_B alone? The answer runs through vortices, and through a flow equation that Kosterlitz derived in 1974 [61]: the flow of the stiffness and of the vortex density as one looks at the film on larger and larger scales. We use the three instruments of the book on it. The proof assistant gives exact statements about the flow, and we shall find that one of them, as the library states it, says less than it seems to (Section 6.3). The solver does two jobs: CVODE integrates the flow and checks it against the theorems, and the Rust engine `qf-pgpe` measures, in a simulated Bose field, the logarithmic interaction between vortices on which everything rests (Section 6.4) and the transition itself (Section 6.6). The experiment is Bishop and Reppy's, as their abstract reports it; we re-analyse no film data.

GODFRIN TWO DIMENSIONS IN GRENOBLE

Godfrin and his collaborators measured the excitations of two-dimensional liquid helium-3, a single atomic layer on graphite, by inelastic neutron scattering at the Institut Laue–Langevin [1]; the collective density mode of that film reappears at wave numbers larger than twice the Fermi wave number as a well-defined excitation, and its dispersion has a roton-like minimum. The same programme includes measurements of the static structure factor of two-dimensional ^3He adsorbed on graphite [62]. Chapter 8 returns to that film.

What is measured there is a Fermi liquid. It shares its dimension with the helium-4 film of Bishop and Reppy, but not the property that the Kosterlitz–Thouless mechanism needs: a complex order parameter whose phase can wind around a point. This chapter therefore concerns the other isotope, the Bose film. The programme's rule (LL-15) is to name the property a result depends on and to check that the other system has it before an analogy is used; here the shared dimension is not that property.

6.2 Stiffness, vortices and the argument of Kosterlitz and Thouless

Phase stiffness. Write the order parameter of the film as $\psi = \sqrt{n} e^{i\theta}$. At low temperature the density is nearly uniform, and the part of the energy that depends on the phase is $\frac{J}{2} \int (\nabla\theta)^2 d^2r$, with the stiffness $J = \hbar^2 n_s / m$ (an energy). Thermal phase waves are harmonic oscillators; each carries $k_B T / 2$, and together they make the phase wander logarithmically with distance, $\langle (\theta(r) - \theta(0))^2 \rangle = (k_B T / \pi J) \ln(r/a)$. The first-order correlation function then decays as a power law,

$$g_1(r) = \frac{\langle \psi^*(0) \psi(r) \rangle}{n} \sim r^{-\eta}, \quad \eta = \frac{k_B T}{2\pi J} = \frac{1}{2\pi K} = \frac{1}{n_s \lambda_T^2}, \quad K \equiv \frac{J}{k_B T} = \frac{n_s \lambda_T^2}{2\pi}. \quad (6.2)$$

There is no long-range order, as the theorems of Mermin and Wagner [63] and of Hohenberg [64] demand, but there is a phase that is stiff: a state of quasi-long-range order with $n_s \neq 0$. The

dimensionless stiffness K is the variable of this chapter. The last equality in (6.2), $\eta n_s \lambda_T^2 = 1$, is the *spin-wave relation* (the programme's ledger and Chapter 10 call it the Josephson relation); it will be tested on the simulated field in Section 6.6. At the jump (6.1) one has $K = 2/\pi$ and $\eta = 1/4$.

Vortices and the free-energy argument. The phase can also wind: around a point, θ may increase by $2\pi q$ with q an integer, and the fluid circulates with the speed $\hbar q/(mr)$ (Chapter 3). The flow energy of one vortex in a film of size R is $\pi J q^2 \ln(R/a) + E_c$, where a is the core size and E_c the energy of the core. A vortex can be placed in about R^2/a^2 different places, so its entropy is $k_B \ln(R^2/a^2) = 2k_B \ln(R/a)$ and the free energy of one free vortex is

$$F = (\pi J - 2k_B T) \ln \frac{R}{a} + E_c. \quad (6.3)$$

For $\pi J > 2k_B T$ this grows with the size of the film: free vortices are improbable and the stiffness survives. For $\pi J < 2k_B T$ it becomes ever more negative: free vortices proliferate and destroy the phase coherence. The border, $K = J/k_B T = 2/\pi$, is $n_s \lambda_T^2 = 4$. This is the argument of Kosterlitz and Thouless [40] (Berezinskii had independently analysed the low-temperature state of two-dimensional systems with a continuous symmetry [65]). It gives the right number only if J is the stiffness the fluid has *at the scale of the film*, after everything smaller has been taken into account, and that is where the difficulty lies.

Pairs and screening. Vortices of opposite sign attract through the same logarithm: a pair at separation d has the energy

$$E_{\text{pair}}(d) = 2\pi J \ln \frac{d}{a} + 2E_c, \quad (6.4)$$

and Section 6.4 measures the coefficient $2\pi J$ in the solver. At any $T > 0$ such pairs are present in the superfluid phase, but they are small. The Boltzmann weight of a pair is $(d/a)^{-2\pi K} = (d/a)^{-n_s \lambda_T^2}$, the number of places it can sit grows as $d \, dd$, and so the mean square size $\langle d^2 \rangle \propto \int d^{3-n_s \lambda_T^2} dd$ is finite exactly when $n_s \lambda_T^2 > 4$. The threshold of the free-energy argument returns as the point where the polarisability of the gas of pairs diverges. A pair is an electric dipole of the two-dimensional Coulomb gas of vortices; a gas of dipoles polarises in the field of a test vortex and screens it, which lowers the stiffness that is felt at large distances, $J \rightarrow J_R < J$. Screening is not small near the transition, and it feeds back, since the pair sizes depend on the stiffness. Kosterlitz's renormalisation group [61] organises this self-consistency: integrate out the pairs of size between a and ae^l , and follow the stiffness $K(l)$ and the vortex fugacity $y(l)$ (the Boltzmann weight of a core, $y \sim e^{-E_c/k_B T}$) as l grows. To leading order in y ,

$$\frac{dK^{-1}}{dl} = 4\pi^3 y^2, \quad \frac{dy}{dl} = (2 - \pi K) y. \quad (6.5)$$

The first equation is the screening: pairs polarise and soften the stiffness. The second says that a vortex is *relevant*, its fugacity grows with scale, when $K < 2/\pi$, and *irrelevant* when $K > 2/\pi$. We write $u = 1/K$, so that

$$\frac{du}{dl} = 4\pi^3 y^2, \quad \frac{dy}{dl} = \left(2 - \frac{\pi}{u}\right)y, \quad (6.6)$$

which is the form used by the Lean module and by the solver below. The coefficient $4\pi^3$ depends on how the fugacity is normalised; none of the conclusions do (Section 6.3 and Exercise 1).

The picture of (6.6) is drawn in Figure 6.2a. The line $y = 0$ is a line of fixed points: a film without vortices keeps its stiffness at every scale. For $u < \pi/2$ the fixed points are stable, for $u > \pi/2$ unstable. The point $(u, y) = (\pi/2, 0)$, $K = 2/\pi$, is the end of the stable segment and the universal point. A trajectory that starts at small u and small y (a cold film, few vortices) flows to the line of fixed points at some $u_* < \pi/2$: the renormalised stiffness is $K_R = 1/u_* > 2/\pi$, the film is superfluid, and it is the renormalised value, not the bare one, that is measured. A trajectory that starts with more vortices or a smaller stiffness is carried away: y grows, K falls to zero, and the film is normal. The boundary between the two behaviours is the *separatrix* that enters the fixed point $(\pi/2, 0)$. A film that is just superfluid has $K_R = 2/\pi$, a film that is just normal has $K_R = 0$: that is the universal jump, and it is the reason why $n_s \lambda_T^2 = 4$ at T_c^- . Above the transition the flow runs past the fixed point and takes a long time to leave its neighbourhood; the correlation length, which is the scale at which the flow reaches $y \sim 1$, diverges as $\xi \sim \exp(b/\sqrt{T - T_c})$, an essential singularity (Section 6.5).

THE CHAIN OF THE ARGUMENT

- (i) A two-dimensional superfluid is a stiff phase, $K = n_s \lambda_T^2 / 2\pi$. (ii) A free vortex costs $(\pi J - 2k_B T) \ln(R/a)$: it is cheap exactly when $K < 2/\pi$. (iii) Bound pairs soften the stiffness, and the flow (6.6) says by how much. (iv) The flow has a separatrix that ends at $K = 2/\pi$; the renormalised stiffness is above $2/\pi$ on one side of it and zero on the other. (v) Hence $n_s \lambda_T^2 = 4$ at T_c^- , in every film.

6.3 What can be proved about the flow

The library module `KTFLOW` takes the flow (6.6) as two functions u, y of the scale l with derivative hypotheses, and proves four statements about it (the fourth, `KTFLOW.kt_units`, is shown in words only). The statements are copied from `lean_src/KTFLOW.lean`; the proofs are omitted except where shown.

LEAN 4 KTFLOW

```
noncomputable def f (u : ℝ) : ℝ := 2 * u - π * log u
noncomputable def H (u y : ℝ) : ℝ := f u - 2 * π ^ 3 * y ^ 2
```

```

variable (u y : ℝ → ℝ)
  (hu : ∀ l, HasDerivAt u (4 * π ^ 3 * (y l) ^ 2) l)
  (hy : ∀ l, HasDerivAt y ((2 - π / u l) * y l) l)
  (hpos : ∀ l, 0 < u l)

theorem kt_invariant (l : ℝ) : H (u l) (y l) = H (u 0) (y 0)

theorem kt_trapped (h0 : u 0 < π / 2) (hH : f (π / 2) < H (u 0) (y 0)) : ∀ l,
  ↪ 0 ≤ l → u l < π / 2

theorem kt_fugacity_bounded (h0 : u 0 < π / 2) (hH : f (π / 2) < H (u 0) (y 0))
  ↪ ) (l : ℝ) (hl : 0 ≤ l) :
  y l ^ 2 ≤ y 0 ^ 2

```

In words, `KTFlow.kt_invariant`: the function $H(u, y) = 2u - \pi \ln u - 2\pi^3 y^2$ is constant along every solution. This is exact, not a statement near the fixed point, and the proof is a line of calculus: $dH/dl = (2 - \pi/u) 4\pi^3 y^2 - 4\pi^3 y (2 - \pi/u) y = 0$. Its consequences are geometric. The level sets of H are the flow lines, so the phase portrait of Figure 6.2a is a contour plot. At the fixed point $u = \pi/2$ the function $f(u) = 2u - \pi \ln u$ has its minimum, $f(\pi/2) = \pi - \pi \ln(\pi/2) = 1.723$; expanding f to second order there, with $x = u - \pi/2$, gives $f \simeq f(\pi/2) + (2/\pi)x^2$, and the level sets become the hyperbolas $x^2 - \pi^4 y^2 = \text{const}$ of the textbook picture. The textbook picture is the quadratic approximation of the exact level sets, and the exact boundary between the two behaviours is the curve $y = \pm \sqrt{(f(u) - f(\pi/2))/(2\pi^3)}$, bent away from the straight lines $y = \pm(\pi/2 - u)/\pi^2$.

`KTFlow.kt_trapped`: if the flow starts on the superfluid side, $u(0) < \pi/2$, with H above its value at the fixed point, then $u(l) < \pi/2$ for all $l \geq 0$: the renormalised stiffness never falls to $2/\pi$. The whole proof is short enough to be read.

LEAN 4 KTFlow

```

theorem kt_trapped (h0 : u 0 < π / 2) (hH : f (π / 2) < H (u 0) (y 0)) : ∀ l,
  ↪ 0 ≤ l → u l < π / 2 := by
  intro l hl
  by_contra hcon
  push Not at hcon
  have hc : ContinuousOn u (Set.Icc 0 l) := fun x _ => (hu x).continuousAt.
    ↪ continuousWithinAt
  obtain (c, _, hc') := intermediate_value_Icc hl hc {h0.le, hcon}
  have := f_ge_H u y hu hy hpos c
  rw [hc'] at this
  linarith

```

If u reached $\pi/2$ at some l , then, since u is continuous and starts below $\pi/2$, the intermediate value theorem produces a scale c with $u(c) = \pi/2$; but H is conserved, so $H(u(0), y(0)) = H(u(c), y(c)) \leq f(u(c)) = f(\pi/2)$, against the hypothesis. `KTFlow.kt_fugacity_bounded` adds that on this side the fugacity never grows, $y(l)^2 \leq y(0)^2$, and the stiffness only decreases (u is monotone: `KTFlow.u_monotone`). `KTFlow.kt_units` is the dictionary with the units of this book: $K > 2/\pi$ in Kosterlitz–Thouless units is $2\pi K > 4$, that is, $n_s \lambda_T^2 > 4$. All four theorems depend only on the standard axioms (`propext`, `Classical.choice`, `Quot.sound`); the copy of the module in `lean_src` carries no `#print axioms` lines, so we compiled a copy with them added: it checks without error in about half a minute on the shared machine (`facts/ch06_report.md`). That such an invariant exists is no surprise (the programme’s literature review records that Prokofev and Svistunov checked its size independence numerically in a lattice model [66]; we have not re-read that paper). What is formal here is the trapping, which uses no linearisation.

A hypothesis that is too generous

Read the statements once more, and look at the quantifiers: `hu`, `hy` and `hpos` are required for *every real* l , negative ones included. A renormalisation trajectory is a solution for $l \geq 0$ only: $l < 0$ would mean scales finer than the microscopic cutoff, and nothing guarantees that the flow can be continued there. Is the hypothesis harmless? We integrated the flow backwards in l (the Python interface of CVODE, in the current build, does not integrate backwards when asked for an output time before the initial time: it marches away from it until its step limit stops it, so we integrated the time-reversed equations forward; `figures/ch06_backward.py`). For a trapped trajectory, $(u_0, y_0) = (0.9, 0.041)$ with H above $f(\pi/2)$, u decreases with decreasing l and reaches 0, with y running away, at $l = -0.663$, the time given by the exact quadrature of the scalar equation introduced below; at 0.02 before that time CVODE has $u = 0.21$, $y = 0.23$ and soon gives up. A trajectory below the separatrix does the same, at $l = -0.226$. (A trajectory on the disordered side, $(1.8, 0.004)$, can be continued to $l = -50$ and beyond, so the library’s hypotheses are not empty there.) The reason is that on the superfluid side u only increases, so backwards it decreases while $y^2 = (f(u) - H)/2\pi^3$ grows, and $u' = 4\pi^3 y^2$ does not get smaller. That is a theorem:

LEAN 4 CH06_KTFORWARD (NEW IN THIS BOOK)

```
theorem kt_eternal_trivial (h0 : u 0 < π / 2) : y 0 = 0
-- under the hypotheses hu hy hpos of KTFlow above (the flow equations for
  ↪ EVERY real l, u > 0)
```

Under the library’s hypotheses a solution that starts on the superfluid side has no vortices at all, $y(0) = 0$. The proof: backwards in l , u is below $u(0) < \pi/2$, where f is decreasing, so $u'(l) = 2(f(u) - H) \geq c := 4\pi^3 y(0)^2$; a function with $u' \geq c > 0$ on $l \leq 0$ is below $u(0) + cl$ and so negative before $l = -u(0)/c$, against `hpos`. The three library theorems are true; on the superfluid side, however, their hypotheses are met only by the line of fixed points (once $y(0) = 0$,

uniqueness for the linear equation of y gives $y \equiv 0$ and then u is constant: a standard step that we did not formalise), where the conclusion is trivial, and the trapped trajectories that matter, with $y \neq 0$, are not covered. We found this only because the statement was set beside a numerical trajectory. It is a case of the lesson of Chapter 7 and Chapter 10: a hypothesis can make a theorem true and nearly empty.

The repair asks for the equations only on $l \geq 0$, with right-hand derivatives at $l = 0$. The proofs go through with the one-sided mean value theorems of Mathlib in place of the two-sided ones. The following statements are those of the new module `book/lean/Ch06_KTForward.lean` (not part of the library; it compiles against the same Mathlib with the standard axioms, Section 6.7); in the module, `kt_invariant`, `kt_trapped`, `u_monotone0n` and `kt_fugacity_bounded` are the forward-time versions of the library theorems of the same names; the box shows the invariant and the statements that are new.

LEAN 4 Ch06_KTForward (NEW IN THIS BOOK)

```
variable (u y : ℝ → ℝ)
(hu : ∀ l, 0 ≤ l → HasDerivWithinAt u (4 * π ^ 3 * (y l) ^ 2) (Set.Ici 0) l)
(hy : ∀ l, 0 ≤ l → HasDerivWithinAt y ((2 - π / u l) * y l) (Set.Ici 0) l)
(hpos : ∀ l, 0 ≤ l → 0 < u l)

theorem kt_invariant (l : ℝ) (hl : 0 ≤ l) : H (u l) (y l) = H (u 0) (y 0)

theorem kt_escape (l : ℝ) (hl : 0 ≤ l) :
  u 0 + 2 * (f (π / 2) - H (u 0) (y 0)) * l ≤ u l

theorem kt_escape_time (hH : H (u 0) (y 0) < f (π / 2)) :
  ∃ l, 0 ≤ l ∧ π / 2 ≤ u l ∧
    l ≤ max 0 ((π / 2 - u 0) / (2 * (f (π / 2) - H (u 0) (y 0))))

theorem kt_dichotomy (h0 : u 0 < π / 2) (hne : H (u 0) (y 0) ≠ f (π / 2)) :
  (∀ l, 0 ≤ l → u l < π / 2) ∨ ∃ l, 0 ≤ l ∧ π / 2 ≤ u l
```

What they say. The invariant reduces the planar flow to one equation for u alone, $u' = 2(f(u) - H_0)$ (`kt_scalar`), with $H_0 = H(u(0), y(0))$. Because f has its global minimum at $\pi/2$ (the lemma `f_gt_fc` is the inequality $\ln x < x - 1$ for $x = 2u/\pi \neq 1$), $u' \geq 2(f(\pi/2) - H_0)$ whatever the solution does: `kt_escape`. Below the separatrix, $H_0 < f(\pi/2)$, the right-hand side is a positive slope: the stiffness cannot be held, u reaches $\pi/2$ no later than $l_1 = (\pi/2 - u_0) / (2(f(\pi/2) - H_0))$ (`kt_escape_time`), and $K = 1/u$ tends to zero (`kt_stiffness_vanishes`). Together with the trapping this is a dichotomy: for $u(0) < \pi/2$ and $H_0 \neq f(\pi/2)$, either the stiffness stays above $2/\pi$ for ever or it falls to $2/\pi$ in finite RG time, and the line between the two cases is the exact level set $H = f(\pi/2)$, not its linearisation. In the language of the experiment these are two of the three pieces of the jump: renormalised stiffness above $2/\pi$ on one side of the separatrix, zero

on the other. The third piece, that the renormalised stiffness tends to $2/\pi$ as $H_0 \downarrow f(\pi/2)$ (the continuity of the root of $f(u_*) = H_0$), is *not* proved; Section 6.5 shows it numerically. Nor is the existence of non-trivial solutions on $[0, \infty)$: it is standard ordinary-differential-equation theory, and the CVODE trajectories of Section 6.5 are the witnesses.

The positions of the vortices

The flow describes an average over pairs. What can be said, exactly, about the pairs of a configuration? Two modules of the library answer, without any model of the vortex gas. Let $\rho(k) = \sum_+ e^{ik \cdot p} - \sum_- e^{ik \cdot m}$ be the vortex charge density at the wavevector k of N positive vortices at p_i and N negative ones at m_j .

LEAN 4 MATCHINGSCREENING

```

noncomputable def wave (k r : E) : ℂ := exp (I * ((inner ℝ k r : ℝ) : ℂ))

noncomputable def rho {N : ℕ} (k : E) (p m : Fin N → E) : ℂ :=
  ∑ i, wave k (p i) - ∑ j, wave k (m j)

theorem norm_rho_le_matching {N : ℕ} (k : E) (p m : Fin N → E) (σ : Fin N ≈
  ↪ Fin N) :
  ||rho k p m|| ≤ ||k|| * ∑ i, ||p i - m (σ i)||

```

LEAN 4 PAIRPOLARISATION

```

theorem rho_polarisation {N : ℕ} (k : E) (p m : Fin N → E)
  (hk : ∀ i, |inner ℝ k (p i - m i)| ≤ 1) :
  ||rho k p m - I * ∑ i, ((inner ℝ k (p i - m i) : ℝ) : ℂ) * wave k (m i)||
  ≤ ∑ i, (inner ℝ k (p i - m i)) ^ 2

```

`MatchingScreening.norm_rho_le_matching`: for *any* way σ of pairing the positive vortices with the negative ones, $|\rho(k)| \leq |k| \sum_i |p_i - m_{\sigma(i)}|$. The proof is three steps of geometry: e^{ia} is 1-Lipschitz in a , so each pair contributes at most $|k||p_i - m_{\sigma(i)}|$, and the triangle inequality adds them. The cheapest pairing, the optimal-transport cost W between the two signs, is therefore at least $|\rho(k)|/|k|$: the charge density at a long wavelength is a certified lower bound on how far apart the vortices of opposite sign are (`MatchingScreening.matching_lower_bound`, and `MatchingScreening.norm_rho_le_matching_torus` for the periodic box, where each separation is the minimum-image one and k lies in the reciprocal lattice). `PairPolarisation.rho_polarisation` is the dielectric picture of the previous section made exact. For N pairs $d_i = p_i - m_i$, when $|k \cdot d_i| \leq 1$ for every pair, the charge density is the divergence of a polarisation density up to a second-order remainder: $\rho(k) = ik \cdot P(k) + R$ with $P(k) = \sum_i d_i e^{ik \cdot m_i}$ and $|R| \leq \sum_i (k \cdot d_i)^2$. This is the statement that bound charge is $ik \cdot P$, in a form that can be applied to a list of vortex

positions. Both modules were compiled with their `#print axioms` lines added (standard axioms for every declaration), and Section 6.6 applies them to vortex positions produced by the solver.

6.4 The Coulomb gas inside the solver

Everything above rests on the logarithm (6.4). Does the engine of the book have it? In the projected Gross–Pitaevskii field of `qf-pgpe` ($\hbar = m = 1$, mean density $n = 1$, $gn = 1$, so the healing length is $\xi = 1$; the field lives on a periodic box of side L , and Chapter 3 describes how a vortex is imprinted), we imprint a vortex–antivortex pair of separation d into the uniform condensate and read the excess energy of the field from the engine’s own energy: no time stepping, no temperature. On the torus there are two corrections to (6.4). The point-vortex energy of a neutral pair is not $2\pi \ln d$ but $\pi H_{\text{WM}}(d)$, where H_{WM} is the periodic point-vortex pair energy of Weiss and McWilliams [22] for a box rescaled to 2π (we use the closed form as implemented in the programme’s `vortex_thermometer.py`, and did not re-derive it; its agreement with the engine, below, is the check we make); and the phase must be single-valued, which a net dipole moment d forbids unless a uniform counterflow $v = 2\pi d/L^2$ is added, of kinetic energy $\frac{1}{2}nv^2L^2 = 2\pi^2d^2/L^2$ (the imprint does this). So

$$\Delta E(d) = \pi H_{\text{WM}}(d) + \frac{2\pi^2d^2}{L^2} + C_L, \quad C_L = 2\pi \ln \frac{L}{2\pi a} + 2E_c, \quad (6.7)$$

with one constant per box that contains the core energy. Figure 6.1 compares this with the engine.

For d between 10 and 28 ξ (ΔE from 19.8 to 27.9 in units $\hbar^2n/m$) in the box $L = 64 \xi$, and between 12 and 56 ξ (ΔE from 20.8 to 32.2) in $L = 128 \xi$, the engine follows (6.7) with a single constant per box to a root-mean-square deviation of 0.043 and 0.033 (largest deviations 0.071 and 0.064): about half a per cent of the range of the energy in the root-mean-square, and less than one per cent at worst. The coefficient of the logarithm is tested more sharply by the constants: doubling the box changes C_L by $2\pi \ln 2 = 4.355$ if the coupling is $2\pi\hbar^2n/m$, and the fitted constants differ by 4.351 (the fit windows $d \geq 4, 6, 8 \xi$ give 4.368, 4.349, 4.351). The naive slope of ΔE against $\ln d$ over the window $d = 4$ to 8ξ is 1.07 (box 64) and 1.04 (box 128) times 2π : the plane law is a good first guess, but the cores and the torus put a few per cent on it, and the comparison with the torus law is the meaningful one. The Coulomb-gas coupling of the programme’s projected field is therefore $2\pi\hbar^2n_s/m$ with $n_s = n$ at zero temperature, to within 0.3% in the constants for every window tried, and every instrument built on the point-vortex picture (the matching of Section 6.6, the pair statistics of Chapter 7) starts from a law that the engine itself obeys. The imprint is not the relaxed vortex, so the constants themselves carry a core-mismatch energy; no claim is made about E_c .

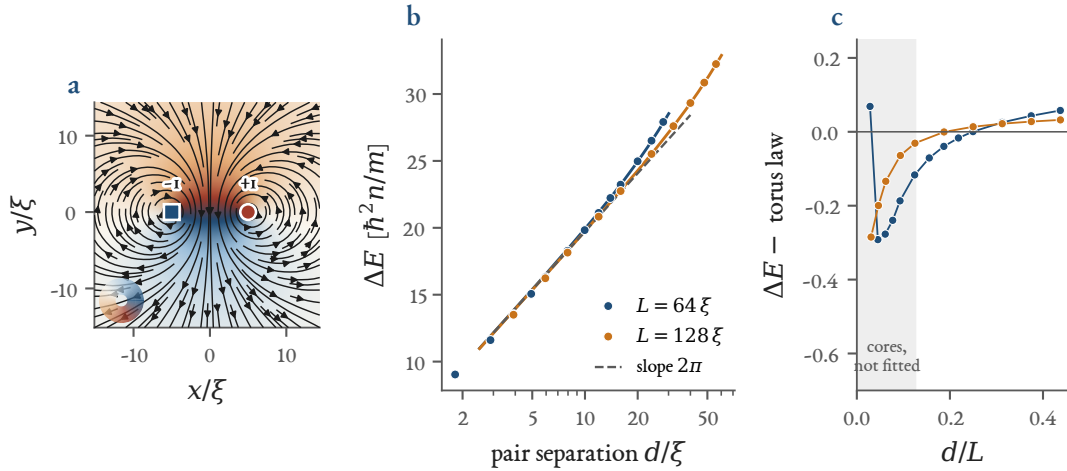


Figure 6.1: The logarithmic interaction of vortices in the solver. **a** A vortex–antivortex pair (red disc: +1, blue square: −1), separation $d = 10\xi$, imprinted into a uniform condensate by qf-pgpe ($L = 64\xi$); colour: phase of the field (the wheel at lower left), lines: streamlines of the superfluid velocity. **b** Excess energy of the imprinted pair against the separation, for $L = 64\xi$ (blue) and $L = 128\xi$ (orange); solid lines: the torus point-vortex law (6.7) with one fitted constant per box; dashed: the plane law with slope 2π . **c** Residual of the torus law; the shaded band is the core region $d < 8\xi$ that was not fitted. Source: figures/ch06_loglaw.py, ch06_coulomb.py.

6.5 The flow, integrated by CVODE

RUSTY-SUNDIALS CVODE ON THE KOSTERLITZ FLOW

The flow (6.6) is handed to CVODE through the Python module `rusty_sundials`. Its `solve` returns only the end state, so a trajectory is a chain of solves, each restarting the integrator from the previous end point (the drift reported below therefore includes the restarts). The excerpt is verbatim from `figures/ch06_flow_compute.py` (A_COEF is $4\pi^3$; H is the invariant of `KTFlow.kt_invariant`; the last part is the body of the loop of the function chain, one link of the chain per pass).

```
def f(u): return 2 * u - pi * math.log(u)
def H(u, y): return f(u) - 2 * pi ** 3 * y * y

def make_rhs(a=A_COEF):
    def rhs(l, s):
        u, y = s
        return [a * y * y, (2 - pi / u) * y]
    return rhs

def solver(method, rtol, atol=1e-13):
```

```

return CvodeSolver(method=method, rtol=rtol, atol=atol, max_steps=500000)

while l < lmax - 1e-12:
    step = dl
    if adaptive:
        step = min(dl, 0.05 / max(1.0, abs(rhs(l, s)[0])))
    l1 = min(l + step, lmax)
    try:
        _, s = sv.solve(rhs, l, s, l1)
    except RuntimeError:
        status = "solver_stop"; break
    l = l1
    L.append(l); U.append(s[0]); Y.append(s[1])
    if s[0] > ustop or abs(s[1]) > ystop:
        status = "left"; break

```

The module used for every number of this section is the build of rusty-SUNDIALS commit 5db8041 (sha256 0bdb1b4a504f..., recorded in figures/ch06_numbers.json). The script refuses to run with any other: it first integrates $y' = -y$ on $[0, 10]$ with Adams at $\text{rtol} = 10^{-8}$ and stops unless that needs fewer than 5000 right-hand-side calls (511 here). The printed record of the run (figures/ch06_flow_compute.log), last lines:

```

adams_0.0001 2.2990858496996225e-05 0.020871856137751443 0.0
adams_1e-06 1.1391289778117653e-07 0.020866424221486657 0.1
adams_1e-08 2.703142154558691e-09 0.020866528666865003 0.3
adams_1e-10 1.9197976541818207e-11 0.02086653416968831 0.1
bdf_0.0001 6.019662951661786e-06 0.020866911425781565 0.2
bdf_1e-06 2.1762584490048198e-07 0.020866208945461207 0.8
bdf_1e-08 1.7298834809054142e-08 0.020866528311566324 1.0
bdf_1e-10 6.256666296167168e-10 0.020866533765439677 1.3
control [0.013523778528752306, 0.004224805752023997, 0.0009541671101744864,
        0.011222398376601106, 0.2505532326600892, 0.01841533553023922]

```

(columns: method and rtol ; the largest drift of H over six trajectories to $l = 40$; the smallest distance of those trajectories from $u = \pi/2$; the seconds of the run on the shared machine. Last line: the drift of the same six trajectories of a flow whose coefficient $4\pi^3$ was changed by 5% in du/dl only, the negative control.)

A first-order integrator hiding in plain sight. Our first attempt used the rusty_sundials module that sits in the programme's virtual environment, and it was slow in a way that smelled wrong: one link of 0.25 units of l cost 12 898 evaluations of the right-hand side, and 20 units cost 353 469. We counted. For a smooth, slowly varying flow at $\text{rtol} = 10^{-9}$, a variable-order Adams method should need a few hundred. The module built from commit 5db8041 needs 64 and 354; on $y' = -y$ over $[0, 10]$ at $\text{rtol} = 10^{-8}$ the old module needs 237 022 and the new

one 511, and the old one’s error is 6.4×10^{-4} against 2.0×10^{-7} (figures/cho6_cvode_probe.py, ch06_cvode_probe_venv.json and _5db8041.json). The old module had been built before a repair to the library: its Adams method never left order one (it was implicit Euler with a Milne-style error estimate), and a first-order method under local error control has a global error that falls only like the square root of the tolerance (docs/CVODE_ADAMS_FIX.md of rusty-SUNDIALS). Its answers are far less accurate than the tolerance that was asked for, and each costs a thousand times more work; nothing in its output says so, and a user who does not count calls would not know. Our first full run with it had not finished the first of its six parts after ten minutes of wall-clock time on the shared machine when we stopped it to count. The lesson is the programme’s usual one: a solver result needs a cost figure next to it, and the script an environment check.

The portrait. Figure 6.2a shows 43 trajectories integrated with Adams at $\text{rtol} = 10^{-9}$, drawn over the level sets of the exact invariant. Of the 43 starting points, 20 satisfy the hypothesis of the trapping theorem ($u_0 < \pi/2$, $H_0 > f(\pi/2)$). All 20 stay below $u = \pi/2$ for the whole run to $l = 60$, u is monotone along each, and $y^2 \leq y_0^2$ throughout, as `KTFlow.kt_trapped`, `KTFlow.u_monotone` and `KTFlow.kt_fugacity_bounded` require; their end points lie on the line of fixed points at the root u_* of $f(u_*) = H_0$, as the invariant says they must: to 2.1×10^{-9} or better for all but three of them, which start within 4×10^{-3} of the separatrix in H and have not converged at $l = 60$ (the worst differs from the root by 3.3×10^{-4}). The 18 starting points on the superfluid side *below* the separatrix all cross $u = \pi/2$ (18 of 18), as the new escape theorems require. And 9 starting points lie between the exact separatrix and the textbook straight line $y = (\pi/2 - u)/\pi^2$: the linearised picture sends them to the disordered phase; the exact invariant, the theorem and CVODE all keep them on the superfluid side (9 of 9 trapped).

The invariant as a test of the integrator. The theorem says H is constant; the numerical H is therefore a direct measurement of the global error of the integrator, with no reference solution needed. Figure 6.2b: with Adams the largest drift over six trajectories falls from 2.3×10^{-5} at $\text{rtol} = 10^{-4}$ to 1.9×10^{-11} at 10^{-10} , about as fast as the tolerance; with BDF from 6.0×10^{-6} to 6.3×10^{-10} , slower at tight tolerance. The negative control shows the test has teeth: with the coefficient of du/dl wrong by 5% the same six trajectories drift by 9.5×10^{-4} to 0.25, that is, by 5×10^7 to 1.3×10^{10} times the drift of the correct flow at the tightest tolerance. (The drift measured here includes the restart at every link; a single call to a long interval does better, and we report the drift that a user of this interface gets.)

The escape bound. For nine starting points below the separatrix (Figure 6.2c) we measure the first-passage time of u through $\pi/2$ by root-finding on single CVODE solves, and compare it with two things: the exact value of the quadrature $\int_{u_0}^{\pi/2} du / (2(f(u) - H_0))$, which the scalar reduction `kt_scalar` makes available, and the Lean bound l_1 . CVODE and the quadrature agree to 2.6×10^{-11} (relative), and the crossing time is below the bound in all 9 cases, at 0.74 to 0.98 of it. The bound uses the smallest slope of u on the way, $2(f(\pi/2) - H_0)$, which is attained only at

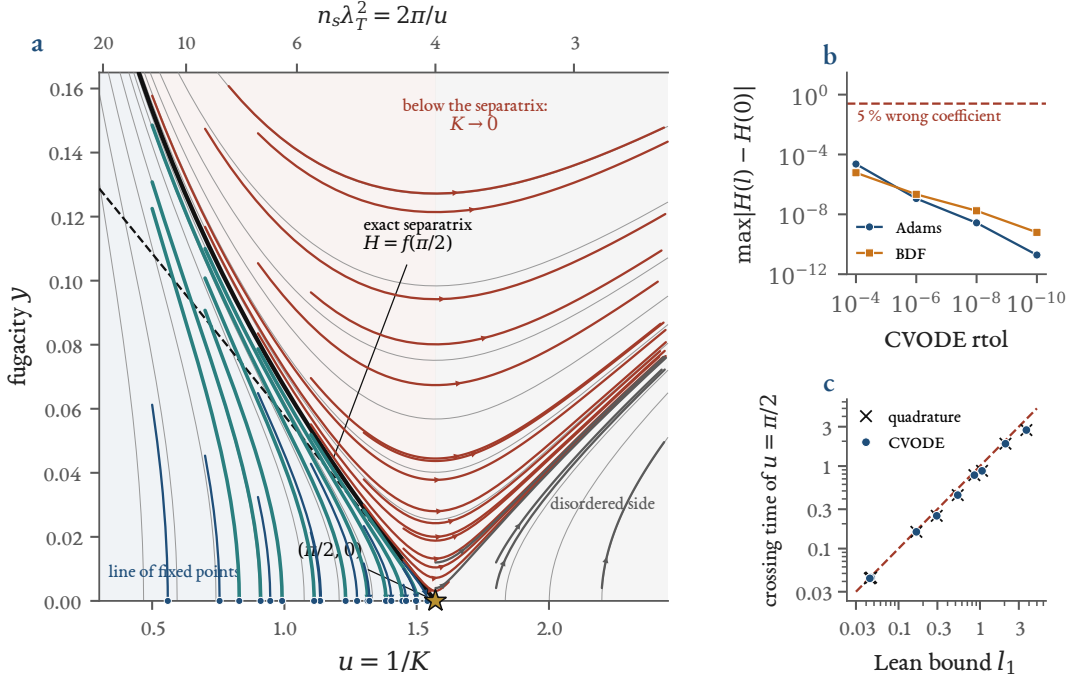


Figure 6.2: The Kosterlitz flow, integrated with CVODE (rusty-SUNDIALS). **a** Trajectories in the plane $(u = 1/K, y)$ over the level sets of the exact invariant H (thin grey lines; by `KTFlow.kt_invariant` they are the flow lines). Blue: starting points that satisfy the hypothesis of `KTFlow.kt_trapped`; teal: those that lie between the exact separatrix (black) and the textbook line (dashed); red: below the separatrix; grey: starting beyond $u = \pi/2$. Dots on the axis: the roots u_* of $f(u_*) = H_0$. Star: the fixed point $(\pi/2, 0)$, $K = 2/\pi$; the upper axis gives $n_s \lambda_T^2 = 2\pi/u$. **b** Largest drift of H over six trajectories ($l \leq 40$) against the CVODE tolerance for Adams and BDF, and for a flow with a 5% wrong coefficient (dashed red). **c** Crossing time of $u = \pi/2$ below the separatrix, CVODE (dots) and the exact quadrature of the scalar equation (crosses), against the bound l_1 of `kt_escape_time` (dashed: equality). Source: `figures/ch06_flow_compute.py`, `ch06_portrait.py`.

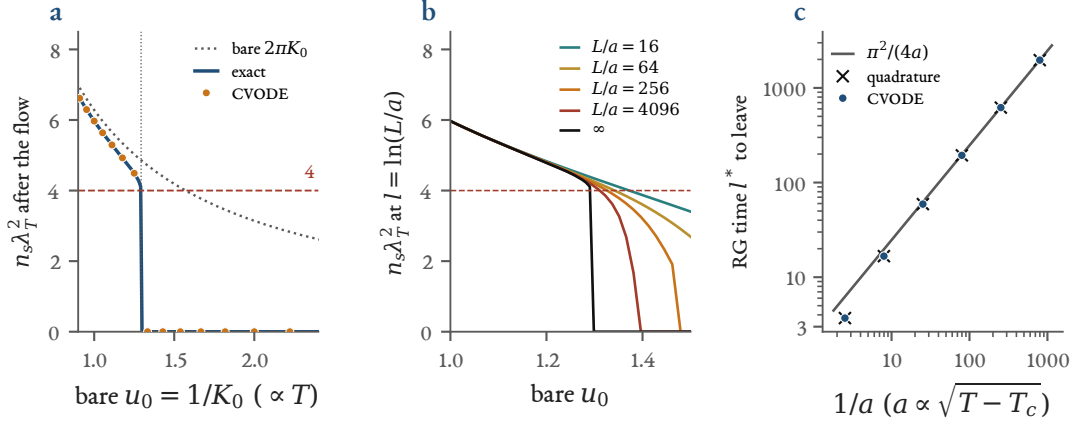


Figure 6.3: The universal jump, the finite box and the essential singularity in the flow (CVODE, Adams). **a** Renormalised $n_s \lambda_T^2 = 2\pi K_R$ against the bare $u_0 = 1/K_0$ at fixed $y_0 = 0.03$ (a toy: the bare fugacity is not tied to the temperature): the root of $f(u_*) = H_0$ (blue) and the end of a CVODE chain to $l = 80$ (orange dots). The renormalised stiffness stays above 4 on the superfluid side and is 0 beyond u_{0c} ; the dotted curve is the bare $2\pi K_0$. **b** The same flow stopped at $l = \ln(L/a)$, for four box sizes, and the limit $L \rightarrow \infty$: the jump is rounded, and the crossing of the level 4 moves to larger u_0 (higher temperature) in a smaller box. **c** The scale l^* at which a flow just above the separatrix leaves the critical region, against $1/a$ with $a^2 = (\pi/2)(f(\pi/2) - H_0)$, $a \propto \sqrt{T - T_c}$: CVODE (dots), quadrature (crosses), and the asymptote $\pi^2/(4a)$ (line): $\zeta = e^{l^*} \sim e^{\pi^2/4a}$, the essential singularity. Source: figures/ch06_flow_compute.py, ch06_jump.py.

$u = \pi/2$, so it is tight when the start is far below the separatrix and the slope is nearly the same all the way (ratios near 1 for the largest $f(\pi/2) - H_0$), and loose near the separatrix. This is one of the places where the proof assistant and the solver check each other: the theorem is quantitative, and the solver tests the number.

The jump. Figure 6.3a puts the two halves of the theorem together for a one-parameter family of flows at fixed $y_0 = 0.03$ (a toy, the bare fugacity is not tied to the bare stiffness here; the aim is the shape). On the superfluid side the end point of the flow is the root u_* of $f(u_*) = H_0$ and the renormalised stiffness is above the universal value; the end of a CVODE chain to $l = 80$ agrees with the root (for $K_0 = 0.8$: $K_R = 0.715$ from both). Past the critical bare stiffness, $K_{0c} = 0.7734$ ($u_{0c} = 1.293$), y grows and K falls to zero. Just above K_{0c} the renormalised stiffness is $K_R = 0.654$ ($n_s \lambda_T^2 = 4.11$) at the nearest grid point, and tends to the universal value $2/\pi$ with a square-root cusp: $f(u) \simeq f(\pi/2) + (2/\pi)(u - \pi/2)^2$ gives $K_R - 2/\pi \simeq (4/\pi^2)\sqrt{\pi\epsilon/2}$ for $H_0 = f(\pi/2) + \epsilon$. This continuity is the third piece of the jump, the one that the theorems do not prove. The jump itself, from 4 to 0, is what Bishop and Reppy tested.

A finite box. A film of size L cannot flow further than $l = \ln(L/a)$. Figure 6.3b stops the flow there. The jump is rounded, and the apparent transition, the crossing of $n_s \lambda_T^2 = 4$, moves to a larger

bare u_0 , that is, a higher temperature: by 6.1%, 3.8%, 2.6% and 1.4% of u_{0c} for $L/a = 16, 64, 256$ and 4096. The reason is the approach to the fixed point along the separatrix. With $x = u - \pi/2 < 0$ the scalar equation becomes $x' = (4/\pi)x^2(1 + O(x))$, so $x \simeq -\pi/(4l)$ and

$$n_s \lambda_T^2(l) = \frac{2\pi}{u} \simeq 4 + \frac{2}{l} \quad \text{along the critical trajectory} \quad (6.8)$$

(Exercise 2): the critical film still looks stiffer than 4 at any finite scale, and only logarithmically so. CVODE agrees: $l(n_s \lambda_T^2 - 4)$ along the separatrix is 1.78 at $l = 10$, 1.96 at $l = 10^2$ and 1.9993 at $l = 10^4$, tending to 2, with the invariant conserved to 1.9×10^{-13} over the whole run.

Above the transition. For a flow that starts just below the separatrix, $H_0 = f(\pi/2) - \epsilon$, the scalar equation near the fixed point is $x' = (4/\pi)(x^2 + a^2)$ with $a^2 = (\pi/2)\epsilon$, and the time to cross from $-X$ to X is $(\pi/4a) 2 \arctan(X/a) \rightarrow \pi^2/(4a)$: the flow lingers at the fixed point for a time that diverges as $1/\sqrt{\epsilon}$, and since ϵ is linear in the distance from the critical temperature, the correlation length $\xi \sim e^{l^*}$ diverges as $\exp(b/\sqrt{T - T_c})$. Figure 6.3c: the time for the flow to reach $u = 2$ from $u_0 = 0.8$, by quadrature and by CVODE, divided by $\pi^2/(4a)$ is 0.59, 0.95 and 0.998 for $\epsilon = 10^{-1}, 10^{-3}$ and 10^{-6} ; the difference $l^* - \pi^2/(4a)$ settles at -3.07 , a constant that depends on where one chooses to stop. CVODE and the quadrature differ by 5.6×10^{-6} (relative) at the smallest ϵ .

The flow equations are a model; everything so far says that the model is internally consistent with the theorems and that CVODE integrates it correctly. Whether a fluid obeys it is another question, and the next section asks it of the solver's own fluid.

6.6 The simulated field near the transition

The engine of Section 6.4 is a classical field: the projected Gross–Pitaevskii equation, started from a random state of chosen total energy, thermalises, and its temperature is *measured* from the trajectory (equipartition of the modes near the cutoff), not imposed. It is a weakly interacting Bose gas with a cutoff, not helium; the Kosterlitz–Thouless mechanism needs only a stiff phase and vortices, and the field has both (this is the property the analogy with the film depends on, in the sense of the programme's rule LL-15), but a number measured here is a number of this model. The *heating ladder* of the programme's second round starts from two equilibrated cold states (energy per particle $e = 0.90$, evolved to $t = 4000$, two seeds), adds random phonons until the total energy reaches the target, and raises e in nine steps, $e = 1.00, 1.05, \dots, 1.40$, in a periodic box of side $L = 64 \xi$; each run has a transient of 500 time units and is averaged over $t = 500$ to 1500 (data/generated/pgpe/r2/, files II_*.json). The steps above $e = 1.20$ were added after the log of the first runs showed that the crossing lay above the range that had been pre-registered (amendment R2-A4 of the programme, written before any verdict was computed). A smaller box, $L = 32 \xi$, with five energies and three seeds, was run to see the size dependence; those runs start from random states and last 4000 time units, with the last 1000 sampled, so the comparison of

Table 6.1: The heating ladder, $L = 64 \xi$: seed means of T , superfluid fraction, $n_s \lambda_T^2$, the exponent η of g_1 , the product $\eta n_s \lambda_T^2$ (equal to 1 for a Gaussian phase field, Equation (6.2)), and the number of vortices (all runs, both signs). Source: figures/ch06_pgpe_numbers.json.

e	T	n_s/n	$n_s \lambda_T^2$	η	$\eta n_s \lambda_T^2$	N_v
1.00	0.571	0.809	8.90	0.126	1.12	19
1.05	0.622	0.767	7.75	0.145	1.12	32
1.10	0.668	0.725	6.82	0.164	1.12	47
1.15	0.722	0.703	6.11	0.192	1.18	66
1.20	0.775	0.647	5.24	0.230	1.20	90
1.25	0.818	0.533	4.09	0.297	1.22	117
1.30	0.862	0.370	2.70	0.447	1.21	147
1.35	0.906	0.357	2.48	0.540	1.34	179
1.40	0.966	0.255	1.66	0.660	1.09	212

the two boxes inherits a difference of protocol. For each run we recompute from the saved per-run records (figures/ch06_pgpe_data.py, which agrees with the programme’s own analysis to the last digit)

- T , from equipartition of the high-wavenumber modes, in units $\hbar^2/(m\xi^2)$ with $k_B = 1$;
- n_s/n , from the transverse and longitudinal current correlators at the smallest wavevectors, $n_s/n = 1 - \langle |J_T|^2 \rangle / \langle |J_L|^2 \rangle$, and from it $n_s \lambda_T^2 = (n_s/n) 2\pi/T$ (for $n = 1$, $\lambda_T^2 = 2\pi/T$);
- η , from a fit of $g_1(r) \sim r^{-\eta}$ over a window that excludes the noise floor;
- the vortices, by the winding rule of `VortexWinding` (the sum of the principal-value phase differences around a plaquette is an exact multiple of 2π).

Only runs that pass the programme’s equilibrium admission test (the longitudinal and transverse current fluctuations must not exceed what an equilibrium state allows) enter; all 18 runs of the $L = 64$ ladder and all 15 of the $L = 32$ series pass. Table 6.1 gives the seed means.

Figure 6.4d shows the stiffness. It falls through the Nelson–Kosterlitz level $n_s \lambda_T^2 = 4$ between $e = 1.25$ and $e = 1.30$; interpolating the seed means, the crossing is at $T_{\text{BKT}}(L=64) = 0.821$, where $\eta = 0.307$ (the infinite-size value is $1/4$) and $n_s/n = 0.52$. The two seeds do not agree on where the crossing is: taken separately they give 0.809 and 0.836; this is the honest error bar of the number. At $e = 1.25$ the two seeds read $n_s \lambda_T^2 = 4.63$ and 3.56, on either side of 4. In the smaller box the crossing is at $T_{\text{BKT}}(L=32) = 0.893$ (seeds: 0.877 to 0.912, which does not overlap the range of the larger box), and the fall of the stiffness is gentler: the slope $|d(n_s \lambda_T^2)/dT|$ at the crossing is 32 for $L = 64$ and 13 for $L = 32$. Both signs, a crossing at a higher temperature and a softer fall in the smaller box, are what the flow predicts when it is stopped at the scale of the box: a film of size L sees the stiffness $K(l)$ at $l = \ln(L/a)$, and along the critical trajectory this exceeds $2/\pi$ by an amount that shrinks only as $1/l$ (Figure 6.3b and Exercise 2). The toy flow of that figure has two parameters that we did not fit to the field, and the agreement is one of sign and not of number: between $L/a = 32$ and 64 the toy moves the crossing, in temperature, by 1%, and the field between $L = 32 \xi$ and 64ξ by 9%.

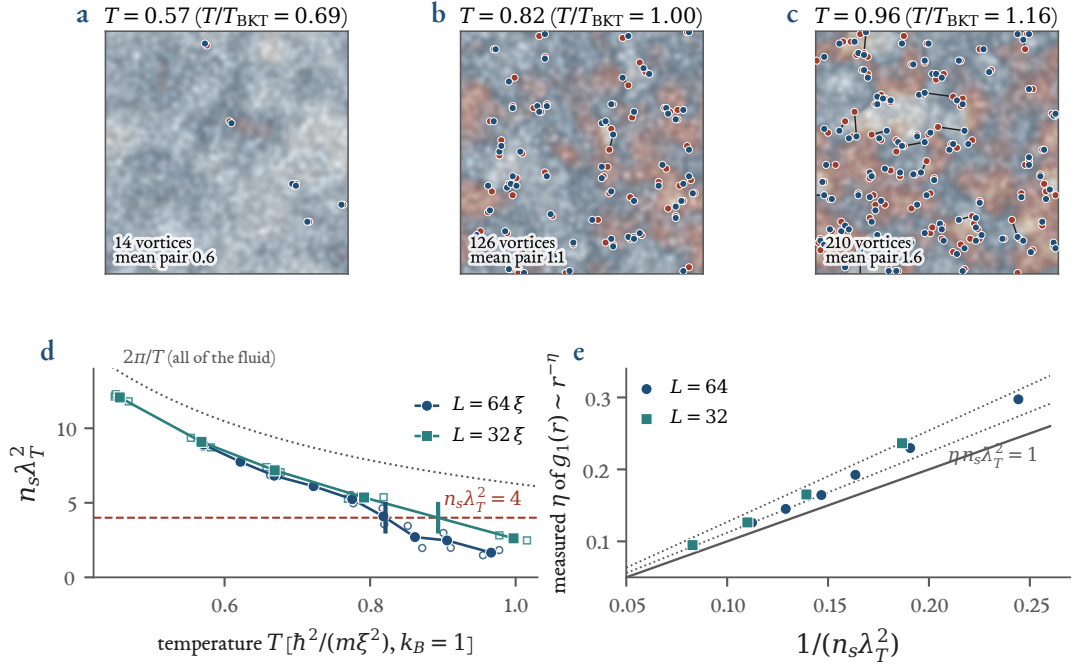


Figure 6.4: The simulated Bose field near the transition. **a–c** Final states of the $L = 64 \xi$ heating ladder at $T = 0.57, 0.82$ and 0.96 (in units of $T_{\text{BKT}}(64) = 0.82$: $0.69, 1.00, 1.16$): vortices (+1 red, -1 blue) found by the winding rule, the optimal pairing of positive to negative vortices as black bonds, and the coarse-grained phase as background hue; positions are on the grid, so separations below 0.5ξ are not resolved. **d** $n_s \lambda_T^2$ against T : single runs (open symbols) and seed means (filled) for $L = 64$ (blue) and $L = 32$ (teal); the red dashed line is the universal level 4, the ticks mark the interpolated crossings, the dotted line is $2\pi/T$, the value if the whole fluid were superfluid. **e** Measured η against $1/(n_s \lambda_T^2)$ for the energies with $n_s \lambda_T^2 > 4$; the grey line is the spin-wave relation $\eta n_s \lambda_T^2 = 1$, the dotted lines are 12 and 27% above it. Source: figures/ch06_field.py.

What does not come out as the theory says. Three things. First, the classical field at the crossing has $n\lambda_T^2 = 2\pi/T_{\text{BKT}} = 7.65$, against $\ln(380/\tilde{g}) = 5.94$ for the weakly interacting gas without a cutoff (Prokofev, Ruebenacker and Svistunov [67]; $\tilde{g} = 1$ here, which is not small): 29 % more. The programme’s literature review suggested that this is the ultraviolet logarithm of a classical field with a sharp cutoff; the estimate it gives for our cutoff, $T_{\text{BKT}} = 0.811$ and $n\lambda_T^2 = 7.75$, is close to the measured values, but that was a hypothesis when this chapter was written, not a test, and we do not use it. Second, the spin-wave relation. For the six energies with $n_s \lambda_T^2 > 4$ the product $\eta n_s \lambda_T^2$ is 1.12 to 1.22 (Figure 6.4e), and for $L = 32$ it is 1.14 to 1.27: a systematic excess of 12 to 27% over 1, which is outside the scatter of the seeds. The programme’s larger boxes ($L = 128$ and 192) show an excess that is also positive at every size and not monotone in L ; its origin is unknown to us, and Chapter 10 lists it among the open items. Third, the round-1 value $T_{\text{BKT}} \approx 0.72$, interpolated between states that were started by a quench, is withdrawn: those states were not equilibrated (at

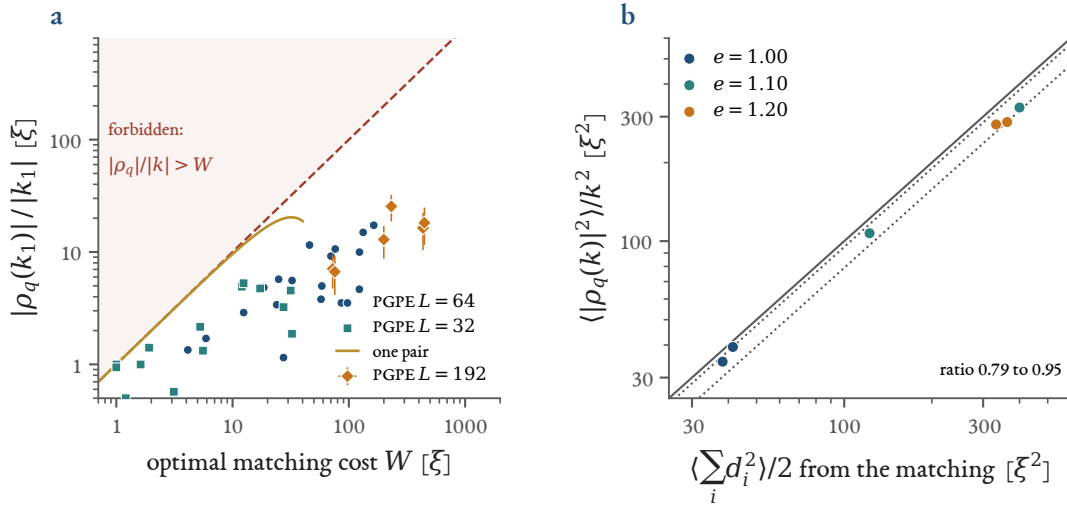


Figure 6.5: The two Lean statements about vortex positions, tested on the solver’s configurations. **a** $|\rho_q(k_1)|/|k_1|$ at the smallest wavevector against the optimal matching cost W : 33 final fields of the $L = 64$ and $L = 32$ series (symbols), the six $L = 192$ runs (mean and standard deviation over 100 snapshots), and one isolated pair of separation d in a box of 64ξ (gold curve). The red dashed line is the bound of `MatchingScreening.norm_rho_le_matching_torus`: no point may lie above it, and none does. **b** The polarisability plateau $\langle |\rho_q(k)|^2 \rangle / k^2$ (mean over the shells $m^2 = 4, \dots, 16$) against $\langle \sum_i d_i^2 \rangle / 2$ from the optimal matching, for the six $L = 192$ runs; the dotted lines are 0.79 and 0.95 times the diagonal. Source: `figures/ch06_certificates.py`, `ch06_screening.py`.

the same vortex count they had half the stiffness of the heated ones), and 0.821 replaces it.

Vortex positions against the two Lean statements. Figure 6.4a–c show what the optimal pairing looks like. At the lowest temperature almost all vortices are in tight pairs (the mean matched separation is 0.6ξ , at the resolution of the grid); at T_{BKT} there are 126 vortices in the box with a mean matched separation of 1.1ξ ; at $T = 0.96$ the pairs are 1.6ξ long and the bonds begin to reach across the box. We now apply the two statements of Section 6.3 to every configuration at hand: the final fields of the two series (33 configurations) and the saved positions of six $L = 192$ runs (100 snapshots each, $t = 13\,510$ to $14\,500$, the last window of the programme’s longest runs; five of the six are equilibrated, the sixth, $e = 1.10$ seed 11, still carries one pair of the size of the box). For each configuration we find the optimal matching W with minimum-image distances, and evaluate $|\rho(k)|/|k|$ on the reciprocal lattice (all $|n_x|, |n_y| \leq 6$).

The matching bound. There is no violation in any of the 33 configurations, nor in any of the 600 snapshots of the $L = 192$ runs, on any of the wavevectors tried. That the inequality holds is no surprise, since it is a theorem; what the data show is how much room it leaves. For a thermal gas of tight pairs the bound is loose: the ratio $\tau = |\rho(k_1)|/(|k_1|W)$ is 0.04 to 1.00 for the fields (the large values are the configurations with one or two pairs) and 0.04 to 0.11 for the $L = 192$ runs,

because the pairs cancel each other's charge at a long wavelength while the matching cost counts every one of them. The bound is saturated only by an isolated pair whose separation is small against the wavelength, $\tau = \sin(k_1 d/2)/(k_1 d/2) \rightarrow 1$ (Exercise 3). It is a certificate, not a detector: it bounds the total pairing length from below whatever the configuration (a charge density of order one at the smallest wavevector requires the vortices of opposite sign to be, in total, at least $L/2\pi$ apart), and a small value certifies nothing.

The polarisation form. At the four shortest wavevectors the premise $|k \cdot d_i| \leq 1$ of `PairPolarisation.rho_polarisation` holds for every pair in 32 of the 33 field configurations, and in all of them the inequality $|\rho - i \sum_i (k \cdot d_i) e^{ik \cdot m_i}| \leq \sum_i (k \cdot d_i)^2$ holds, with the pairs taken from the optimal matching. The typical size of the remainder is 0.12 of $|\rho|$ (median), and the bound on it is 0.55 of $|\rho|$: in a thermal gas of pairs the long-wavelength vortex charge density is, to about 12 % (median over the configurations), the divergence of a polarisation density. That is the dielectric picture in numbers. Its consequence for the stiffness is Kosterlitz and Thouless's $\varepsilon - 1 \propto \sum d^2$, and Figure 6.5b checks it on the six $L = 192$ runs: the long-wavelength charge fluctuations per k^2 are 0.79 to 0.95 of the sum of $d_i^2/2$ over the matched pairs, always below it; the programme reads the deficit as the screening of dipoles by dipoles, which we did not test separately. We re-derived these numbers with independent code, because the file stored in the repository for them (`data/generated/pgpe/dielectric/primary_C4.json`) still holds the values from before a correction, about 0.009 (a factor k^2 applied by mistake, as `PGPE_DIELECTRIC_RESULTS.md` records); our values match the corrected table of that document. What the dielectric analysis does *not* give is a mode-by-mode relation between the transverse current and the detected charges: the pre-registered test of that failed in five of six runs, the coherence being 0.06 to 0.28.

6.7 What this chapter establishes, and what it does not

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved in Lean (kernel-checked; every declaration depends only on `propext`, `Classical.choice`, `Quot.sound`). In the library: the exact invariant and the trapping of the Kosterlitz flow, the monotonicity of u , the bound on the fugacity and the unit conversion ($K > 2/\pi \iff n_s \lambda_T^2 > 4$) for solutions defined for every real l ; the bound of the vortex charge density by every pairing of the vortices (plane and torus); the polarisation form of the charge density of pairs. In the new module of this book (`book/lean/Ch06_KTForward.lean`, 18 declarations audited, no sorry): the same statements for solutions on $l \geq 0$ only, the escape bound and escape time below the separatrix, the vanishing of K there, the dichotomy at the separatrix, and the theorem that the library's hypotheses leave only fixed points on the superfluid side. *Not* formalised: that non-trivial solutions on $[0, \infty)$ exist; that $K_R \rightarrow 2/\pi$ as the invariant tends to $f(\pi/2)$ from above; the flow equations themselves, which are Kosterlitz's leading-order equations, taken as given.

Computed (CVODE from rusty-SUNDIALS commit 5db8041; qf - pgpe as installed on the machine, its build commit not recorded; one shared, loaded machine). CVODE integrates the flow with the invariant conserved to 1.9×10^{-11} (Adams, $\text{rtol} = 10^{-10}$, chain of solves), reproduces the exact first-passage times to 2.6×10^{-11} and keeps all 20 trapped starting points trapped; the engine qf - pgpe follows the torus point-vortex law for the energy of an imprinted pair to a root-mean-square deviation of 0.043 ($L = 64$) and 0.033 ($L = 128$) in units of $\hbar^2 n/m$, and the coefficient $2\pi\hbar^2 n/m$ of the logarithm to 0.3%. The classical field has an $n_s \lambda_T^2$ that crosses 4 at $T_{\text{BKT}}(64) = 0.821$ (seeds 0.809 to 0.836) and $T_{\text{BKT}}(32) = 0.893$; these come from runs of the programme’s numpy engine (two seeds for $L = 64$, three for $L = 32$), recomputed here from the raw records, not re-run. The two Lean statements about vortex positions were tested on 33 configurations and 600 snapshots and are never violated.

Measured. The only experiment of this chapter is Bishop and Reppy’s, as its abstract reports it: no film data were analysed, and nothing here is a statement about helium except that jump formula and the number $3.49 \times 10^{-9} \text{ g cm}^{-2} \text{ K}^{-1}$ computed from it. The simulated field is a classical Bose gas with a cutoff, in units where $\tilde{g} = 1$ is not small; the match with a film is the shared mechanism (a stiff phase with vortices), not a quantitative identity. The Godfrin measurements are of a different system (a Fermi liquid) and enter only through the dimension.

Analogy, not result. The toy flow of Figure 6.3 gives the sign of the finite-size shift of the transition but about one per cent where the field shows nine; its bare parameters were not fitted.

Failed, withdrawn or open. (i) The round-1 transition temperature $T_{\text{BKT}} \approx 0.72$ of the programme, from quench-started states, is withdrawn in favour of 0.821 (the quench states were not equilibrated). (ii) The spin-wave relation $\eta n_s \lambda_T^2 = 1$ is violated by a systematic 12–27% at $L = 32$ and 64, and by a positive and non-monotone amount at larger sizes; unexplained. (iii) The programme’s pre-registered mode-by-mode relation between the transverse current and the detected vortex charges failed (five of six runs); only the polarisability plateau holds (ratios 0.79 to 0.95). (iv) $n \lambda_T^2$ at the crossing is 29% above the weak-coupling value $\ln 380$; the cutoff explanation was not tested. (v) Seeds: two (or three) per point; the transition temperature is known to ± 0.014 ($L = 64$) at best. (vi) *Found while writing this chapter:* the hypotheses of the library theorems KTFLOW cover only fixed points on the superfluid side (the library file is unchanged; the repaired statements live in the book’s module); the Python CVODE module of the project’s environment predates the Adams repair and costs a thousand times more per result, and the current build’s solve does not handle an output time before the initial time; and the file `primary_C4.json` stores pre-correction values of the polarisability ratio.

Exercises

Exercise 1. Show that for $du/dl = c y^2$ and $dy/dl = (2 - \pi/u) y$, with any constant $c > 0$, the function $2u - \pi \ln u - \frac{c}{2} y^2$ is conserved. Expand it near $(\pi/2, 0)$ and show that the level sets

are the hyperbolas $x^2 - (\pi c/4) y^2 = \text{const}$, $x = u - \pi/2$, with straight separatrices $y = \pm x/\pi^2$ for $c = 4\pi^3$. What changes in the statement of the trapping theorem if c is replaced by another constant? *Hint*: differentiate; $f''(\pi/2) = 4/\pi$, so $f(u) \simeq f(\pi/2) + (2/\pi)x^2$.

Exercise 2. Prove (6.8): along the separatrix $n_s \lambda_T^2(l) = 4 + 2/l + o(1/l)$, and more precisely $l(n_s \lambda_T^2 - 4) = 2 - \frac{2}{3}(\ln l)/l + O(1/l)$. *Hint*: on the separatrix $H = f(\pi/2)$ and the scalar equation gives $x' = 2(f(\pi/2 + x) - f(\pi/2)) = (4/\pi)x^2 - (16/3\pi^2)x^3 + \dots$; integrate $d(1/x)/dl = -x'/x^2$, first with the leading term ($1/x \simeq -4l/\pi$), then once more to get the logarithm; then $2\pi/u = 4/(1 + 2x/\pi)$. Compare with the values of $l(n_s \lambda_T^2 - 4)$ quoted in Section 6.5: the expression $2 - \frac{2}{3}(\ln l)/l$ differs from them by 0.0074 at $l = 100$ and by 5×10^{-5} at $l = 10^4$.

Exercise 3. For one vortex–antivortex pair of separation d , show that $|\rho(k)|/|k| = 2 \sin(kd/2)/k$ when $k \parallel d$, hence that the ratio $\tau = |\rho(k)|/(|k|W)$ of `MatchingScreening.norm_rho_le_matching` equals $\sin(kd/2)/(kd/2)$. Evaluate it for the smallest wavevector $k_1 = 2\pi/L$ of a box of side L and a pair of size $d = L/2$, and compare with the gold curve of Figure 6.5a. Why can τ be close to 1 only when $kd \ll 1$, and what does that say about reading a small charge structure factor as evidence for short pairs? *Hint*: $|e^{i\varphi} - 1| = 2|\sin(\varphi/2)|$.

Notes and further reading

The unbinding of vortex pairs as the mechanism of the two-dimensional transition is due to Kosterlitz and Thouless [40]; Berezinskii analysed the low-temperature state of two-dimensional systems with a continuous symmetry independently [65], and the absence of long-range order they build on is the theorem of Mermin and Wagner [63] and of Hohenberg [64]. The flow equations are Kosterlitz's [61], the universal jump is Nelson and Kosterlitz's [59], and the experiment on helium films is Bishop and Reppy's [39], analysed with the dynamic theory of Ambegaokar, Halperin, Nelson and Siggia [60, 68]. Minnhagen's review [69] covers the Coulomb gas, Kosterlitz's equations and the experiments on helium films, in the language used here. The same crossover was later studied in a trapped atomic gas [70]; the critical phase-space density of the weakly interacting gas is the subject of [66, 67]; classical-field simulations of the pairing and unbinding of vortices are those of Simula and Blakie [71] and of Foster, Blakie and Davis [72]; Hasenbusch's Monte Carlo study of the XY model at the transition [73] derives and tests the finite-size behaviour of the helicity modulus there. For point vortices on a torus see Weiss and McWilliams [22]. The records of the programme behind this chapter are, in the repository, `LEDGER.md` (CLAIM-072 for KTFLOW, CLAIM-070 and CLAIM-085 for the vortex-position statements and the dielectric analysis), `docs/designs/PGPE_R2_RESULTS.md` (the ladder), `PGPE_DIELECTRIC_RESULTS.md` and `PGPE_ONSAGER_RESULTS.md`. The friction and diffusion of a single vortex in the same field are the subject of Chapter 7.

Friction, Diffusion and the Dissipative Vortex

7.1 A pair that opens, a pair that closes

In an oblate sodium condensate two vortices of the same sign circle each other, and the pair is seen to open up: its separation grows with time, and grows faster in a warmer cloud [74]. Read through a dissipative point-vortex model, the opening rate is a friction coefficient α between 0.01 and 0.03 over the 200–450 nK explored. In a rubidium condensate in a hard-walled optical trap, a crystal of vortices placed by optical tweezers and then released expands for the same reason; fitting the expansion to a scaling law gives $\alpha = 3.3(1) \times 10^{-3}$, while the random wandering of the same vortices is about a hundred times larger than the theory that ties it to the friction predicts, a surplus that the authors consider most likely due to vibrations of the trap walls [75]. Two things are read from the motion of a vortex: how hard it is dragged, and how hard it is kicked.

Both come from one fact: a vortex in a warm superfluid is not alone. Around it is a gas of thermal excitations, the normal component of Landau’s two-fluid description [3], and the vortex and the gas exchange momentum, which is called *mutual friction* [10, 76]. The gas acts on the vortex with a force along its velocity relative to the gas and a force across it, and, when the gas is also the heat bath of the vortex, a third effect, random kicks, has to be added: three numbers, α , α' and η (Section 7.2). This chapter follows the model through the three instruments of the book, in a classical field that has no walls, no reservoir and no noise put in by hand, so that the bath of a vortex is the field’s own thermal phonons.

The programme’s record on these questions is mixed, and it is told as it stands in the ledger. What survives: the friction of a vortex pair in the field, $\alpha = 0.0062(4)$ at $T/T_{\text{BKT}} = 0.14$, read by two independent estimators, and a transverse coefficient that is zero to about one per cent. What the programme’s own pre-registered tests refuted: the reading of the friction as proportional to the normal density with a coefficient that belongs to the vortex; the explanation of a stalled single pair by the phonon wind of a closed box; and, earlier, a known-answer gate that had seemed to pass on a defective instrument. What the decision rule written before the data leaves undecided: the Einstein relation between diffusion and friction. And one finding is a bias in the very estimator that Lean proves exact in the model: a lesson about hypotheses, not about Lean.

7.2 The dissipative point vortex

Work in units $\hbar = m = 1$, so that the circulation quantum is $\kappa = 2\pi$, in the frame where the normal component is at rest. A vortex of charge $q = \pm 1$ is carried by the superfluid velocity $\mathbf{v}_s$ that the other vortices induce at its position; the gas adds a drag along the velocity of the vortex relative to the gas and a force across it. In the two-fluid equations of Hall, Vinen, Bekharevich and Khalatnikov these are the coefficients α and α' [77], and for a weakly damped vortex they give

$$\dot{\mathbf{r}}_i = (1 - \alpha') \mathbf{v}_{s,i} - \alpha q_i \hat{\mathbf{z}} \times \mathbf{v}_{s,i}. \quad (7.1)$$

The second term is perpendicular to $\mathbf{v}_s$ and *dissipative*: it carries energy from the vortices to the gas, and α is the friction. The factor $1 - \alpha'$ only rescales the speed with which the vortex follows the superfluid and dissipates nothing.

The structure of (7.1) is clearest in terms of an energy. For point vortices on the plane $H = -\sum_{i<j} q_i q_j \ln r_{ij}^2$ (on the periodic box of the simulations, $\ln r^2$ is replaced by the Weiss–McWilliams periodic Green function [22]; nothing below depends on it), and $\mathbf{v}_{s,i} = -\frac{1}{2} q_i \hat{\mathbf{z}} \times \nabla_i H$. Since $q_i^2 = 1$, the friction term is $-\frac{\alpha}{2} \nabla_i H$, and the whole configuration obeys

$$\dot{\mathbf{r}} = A(\nabla H) - \frac{\alpha}{2} \nabla H, \quad \langle \mathbf{v}, A\mathbf{v} \rangle = 0 \text{ for every } \mathbf{v}, \quad (7.2)$$

a skew part A , which contains α' and moves the vortices along level sets of H , plus a gradient flow that moves them down H . Friction therefore pushes the members of an opposite-sign pair together, because their energy $\ln d^2$ grows with the separation d , and pushes the members of a same-sign pair apart, because their energy $-\ln d^2$ falls as d grows: the first is the dipole that shrinks in the computer, the second the pair that opens in the sodium cloud.

THREE NUMBERS DESCRIBE A VORTEX IN A WARM FLUID

α (friction): the vortex is dragged and energy flows from the vortices to the gas. α' (transverse coefficient): the gas pushes the vortex sideways, without dissipation. η (diffusion): the vortex is also kicked at random, $\langle |\Delta \mathbf{r}|^2 \rangle = 4\eta t$. If the gas is the heat bath of the vortex, an Einstein relation $\eta = \alpha T / (2\pi\rho)$ ($k_B = 1$) ties the last to the first.

The sideways force. Thouless, Ao and Niu derived the total non-dissipative transverse force on a vortex as the Magnus force proportional to the superfluid density, the core states not changing it [78]: the normal fluid exerts no extra sideways force and $\alpha' = 0$. Iordanskii [79] and later Sonin [80] find a transverse force from the scattering of thermal excitations, which the programme registered as $\alpha' = \rho_n / \rho$.

The random kicks. Add independent white noise of diffusion constant η to (7.1). Mehdi, Hope, Szigeti and Bradley derive $\eta = \alpha k_B T / (2\pi\hbar\rho_0)$ (ρ_0 the two-dimensional number density) from a

stochastic projected Gross–Pitaevskii theory and single out its measurement as an important test of stochastic point-vortex theory [81]. A model of this form had been used for helium films by Ambegaokar, Halperin, Nelson and Siggia, with the noise motivated by fluctuation–dissipation arguments and not derived [60].

GODFRIN THE EXCITATIONS ARE THE BATH

Phys. Rev. B **103**, 104516 (2021) reports inelastic neutron scattering on superfluid ^4He at the ILL spectrometer IN5: the dispersion relation of the Landau elementary excitations, phonon, maxon and roton, and the thermodynamic consequences [2]. Those excitations are the normal component of Landau’s picture [3], and in the kinetic theory of mutual friction the friction on a vortex is their scattering by the vortex [76]. What such a measurement fixes, for helium, is what sets the scale of every number below: the energy $\varepsilon(k)$ of a thermal excitation, hence the population of the bath. In our classical field the model must supply the bath itself, and Section 7.6 shows how little it resembles a quantum gas. The statements of his papers that the library formalises are in Chapters 4 and 5, not here.

7.3 What Lean proves about the model

Three modules of the library concern this chapter, `DissipativeVortexDynamics`, `Einstein-Relation` and `FrictionKinetic` (a fourth, `QuasiPeriodicBound`, appears once); a small new module written for the book adds statements to the first. The mathematics is elementary calculus, and the modules say so. The point is that the estimators used on the data are *identities of the model*, with the hypotheses under which they hold written where a machine can read them. Every theorem quoted here depends only on the three standard axioms `propext`, `Classical.choice` and `Quot.sound`: the library modules carry no `#print axioms` lines, so this was checked on 2026-10-09 by compiling scratch copies with one such line per theorem (Lean 4.34.0-rc2, the pinned Mathlib; no errors, no sorry).

The first theorem is general: any dynamics of the form (7.2), on any real inner-product space (any number of vortices, plane or torus), with A pointwise skew.

LEAN 4 DISSIPATIVEVORTEXDYNAMICS

```
variable {X : Type*} [NormedAddCommGroup X] [InnerProductSpace ℝ X] [CompleteSpace X]
theorem energy_dissipation (H : X → ℝ) (gradH : X → X) (hH : ∀ x, HasGradientAt H (
  ↪ gradH x) x)
  (A : X → X) (hA : ∀ v, inner ℝ v (A v) = 0) (γ : ℝ) (r : ℝ → X)
  (hr : ∀ t, HasDerivAt r (A (gradH (r t)) - γ • gradH (r t)) t) (t : ℝ) :
  HasDerivAt (fun t => H (r t)) (-γ * ||gradH (r t)|| ^ 2) t := by
```

(Proof abridged: eleven lines in the module, the chain rule and then hA.)

In words: along any solution, $\frac{d}{dt}H = -\gamma\|\nabla H\|^2$, whatever the skew part. For (7.1), $\gamma = \alpha/2$

and $\|\nabla H\|^2 = 4 \sum_i |\mathbf{v}_{s,i}|^2$, hence

$$\frac{dH}{dt} = -2\alpha \sum_i |\mathbf{v}_{s,i}|^2, \quad \alpha_{\text{energy}} = -\frac{\Delta H}{2 \int \sum_i |\mathbf{v}_{s,i}|^2 dt}. \quad (7.3)$$

The right-hand side estimates α from the positions of the tracked vortices alone, needs no pairing of them, and is blind to α' (`DissipativeVortexDynamics.dissipation_indep_of_skew`). This is the programme's *energy estimator*. Its hypothesis, written in the statement, is `hr`: the path has a derivative at every time. We shall come back to it.

For the plane dipole the model can be written in coordinates. Let $\mathbf{p}$ be the $+$ vortex, $\mathbf{m}$ the $-$ vortex, $\mathbf{d} = \mathbf{p} - \mathbf{m}$ and $r2 = |\mathbf{d}|^2$; the superfluid velocity at either vortex is $(d_y, -d_x)/r2$. The hypotheses `hpx`, `hpy`, `hmx`, `hmy` below are the model (7.1) for the two vortices, and `hpos` says $r2 \neq 0$.

LEAN 4 DISSIPATIVEVORTEXDYNAMICS

```
variable (px py mx my : ℝ → ℝ) (α α' : ℝ)

/-- squared separation -/
noncomputable def r2 (t : ℝ) : ℝ := (px t - mx t) ^ 2 + (py t - my t) ^ 2

include hpx hpy hmx hmy hpos

theorem dipole_sq_law (t : ℝ) (ht : t ∈ Icc (0 : ℝ) T) :
  r2 px py mx my t = r2 px py mx my 0 - 4 * α * t := by

theorem centre_velocity_identity (t : ℝ) (ht : t ∈ Icc (0 : ℝ) T) :
  ∃ cx' cy' : ℝ, HasDerivAt (fun s => (px s + mx s) / 2) cx' t ∧ HasDerivAt (fun s =>
    (py s + my s) / 2) cy' t ∧
    cx' * (py t - my t) - cy' * (px t - mx t) = 1 - α' ∧
    cx' * (px t - mx t) + cy' * (py t - my t) = 0 ∧
    (cx' ^ 2 + cy' ^ 2) * r2 px py mx my t = (1 - α') ^ 2 := by
```

Statements only (the four hypotheses are lines 88–96 of the module); the proofs are in the module.

First, $|\mathbf{d}|^2 = |\mathbf{d}_0|^2 - 4\alpha t$ for as long as the pair exists, *whatever* α' : a dipole is a clock whose dial is $|\mathbf{d}|^2$, calibrated in units of 4α , with a lifetime below $|\mathbf{d}_0|^2/4\alpha$. The proof is the computation $\frac{d}{dt}|\mathbf{d}|^2 = -4\alpha$ and the fact that a function with vanishing derivative on an interval is constant (the theorems `DissipativeVortexDynamics.r2_hasDerivAt` and `DissipativeVortexDynamics.dipole_lifetime_bound` are the two steps). Second, the centre of the pair moves perpendicular to its axis at speed $(1 - \alpha')/|\mathbf{d}|$, whatever α . The shrinking of the pair measures α , its translation measures α' , and the two are read from orthogonal observables.

A same-sign pair: new Lean

The simulations of this chapter imprint dipoles; the experiment that opened it watched the other pair, two vortices of the *same* sign. The book's new module `Ch07_DissipativePairs` (in `book/lean/`, not part of the library) proves the corresponding law with the same method: for $\mathbf{d} = \mathbf{p}_1 - \mathbf{p}_2$ the model gives $\dot{\mathbf{d}} = [2(1 - \alpha') \hat{\mathbf{z}} \times \mathbf{d} + 2\alpha \mathbf{d}] / |\mathbf{d}|^2$, so $\frac{d}{dt} |\mathbf{d}|^2 = +4\alpha$.

LEAN 4 CHO7_DISSIPATIVEPAIRS (NEW, NOT IN THE LIBRARY)

```
theorem corot_sq_law (t : ℝ) (ht : t ∈ Icc (0 : ℝ) T) :
  r2 x1 y1 x2 y2 t = r2 x1 y1 x2 y2 0 + 4 * α * t := by
```

Statement only; the hypotheses are the four equations of the model for two charges of equal sign (lines 41–51 of the file), the proof is that of the dipole law above. The file compiled on 2026-10-09 with no error and no sorry, and the theorem uses only the three standard axioms.

Friction makes opposite-sign pairs shrink and same-sign pairs spiral apart; the new theorem formalises a derivation, not a result about any fluid.

7.4 The model in CVODE

The point-vortex equations are smooth, small and not stiff until a pair is a few core sizes across, and they have closed forms: the right first test of an integrator, and a way of seeing what the theorems say. CVODE [4] is reached from Python through `rusty_sundials.CvodeSolver(method, rtol, atol, max_steps)`, whose method `solve(rhs, t0, y0, t_out)` returns the state at `t_out` (Chapter 2); the binding builds a fresh solver at every call, so a table of outputs is a chain of calls.

Figure 7.1, computed with the listing below, puts CVODE next to the theorems. Panel c is `DissipativeVortexDynamics.dipole_sq_law` and `QuantumFluids.DissipativePairs.corot_sq_law` as measured: with BDF at relative tolerance 10^{-8} the largest relative deviation of $|\mathbf{d}|^2$ from the law over 0.95 of the lifetime is between 1.8×10^{-8} and 2.9×10^{-8} for the six combinations of α and α' (dipoles) and 4.6×10^{-6} to 1.1×10^{-5} for the same-sign pairs, which turn about ten times (an integrator conserves nothing, and the error of a rotating pair accumulates over its turns); the slope does not depend on α' . Panel d: the displacement of the centre follows the closed form (the integral of the centre identity, done by hand; it is not a Lean theorem) to 2.1×10^{-5} over a track of length 311.

Panel e is the solver's side of the same coin. On this non-stiff problem Adams needs between a quarter and a third of the work of BDF at the same tolerance in a single call (93–372 right-hand-side evaluations over the four tolerances, against 413–1198); the chain of 60 outputs that a figure needs costs 7 to 21 times more, because the method restarts at low order at every call, and its error is not monotone in the tolerance (for BDF 1.2×10^{-5} at 10^{-4} and 1.1×10^{-5} at 10^{-6}). These numbers are for the build of `rusty-SUNDIALS` at commit 5db8041 (six commits after v1.1.5.0), not for the

module installed in the shared virtual environment, whose Adams method is the one before the repository's fix: on $y' = -y$ up to $t = 10$ at $\text{rtol} = 10^{-8}$ the stale module needed 237 022 right-hand-side evaluations and the fixed build 511. The figure script runs this probe first and refuses the stale module.

RUSTY-SUNDIALS THE DISSIPATIVE DIPOLE IN CVODE

```
def rhs_dipole(a, ap):
    def rhs(t, y):
        px, py, mx, my = y; dx = px - mx; dy = py - my; r2 = dx * dx + dy * dy
        return [((1 - ap) * dy - a * dx) / r2, (-(1 - ap) * dx - a * dy) / r2,
                ((1 - ap) * dy + a * dx) / r2, (-(1 - ap) * dx + a * dy) / r2]
    return rhs
# ...
def integrate(rhs, y0, tgrid, rtol, atol=1e-10, method="bdf"):
    s = CvodeSolver(method=method, rtol=rtol, atol=atol, max_steps=200000)
    t = float(tgrid[0]); y = list(y0); out = [list(y)]
    for tn in tgrid[1:]:
        t, y = s.solve(rhs, t, y, float(tn)); out.append(list(y))
    return np.array(out)
```

Excerpt of book/figures/ch07_dipole.py, as run (rhs_corot is its mirror image for two + vortices). The figure took 2.5 s of wall-clock on the shared 8-core machine (load average 3.5–3.8), with rusty-SUNDIALS commit 5db8041 (see below).

7.5 A vortex pair in a thermal field

The instrument

The field is the projected Gross–Pitaevskii equation $i\partial_t\psi = \mathcal{D}[-\frac{1}{2}\nabla^2\psi + g|\psi|^2\psi]$ on a periodic square box of side $L = 64$, with $\hbar = m = g = 1$ and mean density $n = 1$ (healing length $\xi = 1$, sound speed $c = 1$), 128×128 grid points and a sharp cutoff $|k| \leq k_c = \pi$, half the grid wavenumber [19, 20]. The energy is conserved (to 10^{-7} – 10^{-6} over every run of the campaign); nothing is damped and no noise is added. The temperature of a base state comes from an equipartition thermometer and its normal fraction from the current correlators [23]. The base state of this section is the campaign's $e = 0.60$ state: $T = 0.115$, that is $T/T_{\text{BKT}} = 0.14$ with $T_{\text{BKT}}(L=64) = 0.821$ from a heating ladder, and $\rho_n/\rho = 0.027$.

A pair is written into the state by multiplying ψ by the Jacobi-theta phase of the vortex configuration, made single-valued on the torus by a uniform phase gradient, and by the Bernoulli amplitude $[1 + |\mathbf{v}|^2/2]^{-1/2}$ of the imprinted flow. Vortices are found by the plaquette phase-winding rule that VortexWinding proves correct (Chapter 3), refined to sub-grid precision, followed by continuity (same sign, nearest detection within 3ξ) and recorded every time unit. A single pair in a periodic box

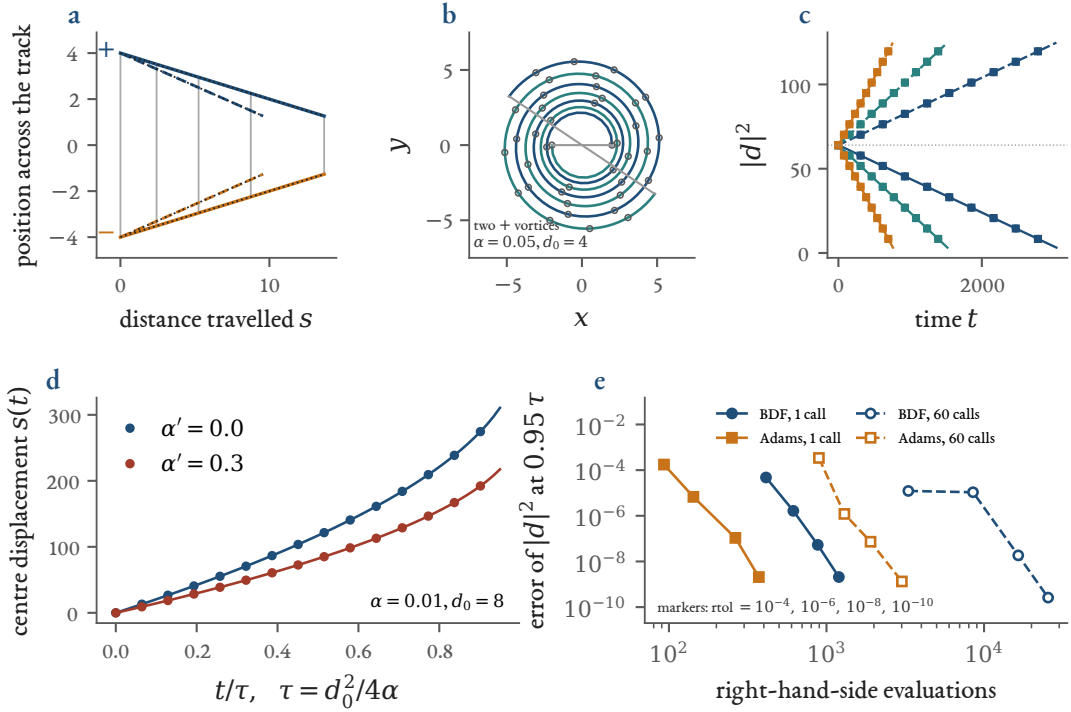


Figure 7.1: CVODE (rusty_sundials) against the statements of Section 7.3. **a** A dipole shrinking and translating: + (blue) and − (orange) vortices in the frame of the track, $\alpha' = 0$ (solid) and 0.3 (dashed), exact curves dotted, pair axis in grey at five times; $\alpha = 0.2$ is ten times a thermal value, chosen so that the funnel is visible. **b** Two + vortices spiral apart ($\alpha = 0.05, d_0 = 4$, 600 time units, 3.4 turns); grey circles: closed form. **c** $|d|^2$ against time: dipoles (solid) fall with slope -4α , same-sign pairs (dashed) rise with $+4\alpha$, for $\alpha = 0.005, 0.01, 0.02$ (blue, teal, orange); filled markers $\alpha' = 0$, open $\alpha' = 0.3$. **d** Displacement of the centre of a dipole against the closed form $s = (1 - \alpha')(d_0 - d)/2\alpha$. **e** Work and precision: right-hand-side evaluations against the error of $|d|^2$ at 0.95 of the lifetime ($\alpha = 0.01, \alpha' = 0.1$) for BDF and Adams at $\text{rtol} = 10^{-4}, \dots, 10^{-10}$, in one call (filled) and in a chain of 60 outputs (open).

carries impulse and drives a mean flow, so the simulations imprint *two antiparallel pairs*, a +− pair at $x = L/4$ and a −+ pair at $x = 3L/4$ of equal separation d_0 : zero net charge, dipole moment and impulse, the two pairs travelling in opposite directions.

RUSTY-SUNDIALS TWO ANTIPARALLEL PAIRS IN A THERMAL FIELD (QF-PGPE)

```
c = np.ascontiguousarray(s.imprint_v2(np.ascontiguousarray(c0), pos, q)); E0 = s.energy(c); t = 0.0; last = pos.copy()
# ...
dp, dq = s.detect(c); dp = np.asarray(dp); dq = np.asarray(dq); cur = np.full_like(last, np.nan)
```

```

for i in range(len(q)):
    cand = np.nonzero(dq == q[i])[0]
    if len(cand):
        d = dp[cand] - last[i]; d -= L * np.round(d / L); r = np.hypot(d[:, 0], d[:, 1])
        j = int(np.argmin(r))
        if r[j] <= a.r_track:
            cur[i] = dp[cand[j]]
# ...
c = np.ascontiguousarray(s.run(c, 1.0)); t += 1.0

```

Excerpt of book/figures/ch07_pairrun.py, as run (s is qf_pgpe.Pgpe(128, 64.0), c the projected Fourier amplitudes, last the previous positions). The run was advanced in five chunks of 100 to 500 time units, each under the shared-machine lock, and checkpointed: 928 s of wall-clock for 1500 time units on one thread, at load average 7–18 on 8 cores.

The control that caught the first instrument

An instrument has to be tested where the answer is known. At $T = 0$ a pair in a uniform condensate has no excitation to rub against, and the model says its separation must not shrink. The campaign's first known-answer gate did not use this control: it was a single dipole at $T/T_{\text{BKT}} \approx 0.14$ with the imprint of the earlier rounds and a biased vortex detector, and it *passed*, narrowly: four fits gave $\alpha = 0.0093, 0.0062$ ($d_0 = 8$) and $0.0028, 0.0022$ ($d_0 = 12$), median 0.0045 , inside the window $[0.004, 0.045]$ registered from an interpolation of the values published by Shukla, Brachet and Pandit [77]. That the friction fell with the size of the pair was not predicted, and was noted. The $T = 0$ control was run next, and it failed.

Figure 7.2 shows why. With the first imprint the pair, which cannot lose energy to a bath that does not exist, closed from 10.03 to 8.66 in 400 time units: an *apparent* friction $\alpha = 0.0100$ from the slope of d^2 , more than the thermal value measured later at $T/T_{\text{BKT}} = 0.14$ (0.0062). The peak density just after the imprint is 1.084 against 1.007 with the corrected one, and at $t = 20$ the field momentum is 0.75 of $2\pi nd$ against 0.96. Two defects were responsible: the theta-function phase of a single pair is periodic on the torus only if $\sum_i q_i \mathbf{r}_i \in L\mathbb{Z}^2$, otherwise it jumps by $2\pi d/L$ across a boundary and the projection turns the jump into a sheet of sound; and the usual per-vortex amplitude $[r^2/(r^2 + 2)]^{1/2}$ has a $1/r^2$ tail four times the Bernoulli depletion, whose excess leaves as sound and pushes the vortices (cf. [82]). With the corrected imprint the pair stays at $9.96 \rightarrow 9.81$ ($|\alpha_{\text{apparent}}| < 10^{-5}$).

The ledger records the consequence without softening: the pass of the first gate *is withdrawn*, and the stalled single pair that had motivated the hypothesis of Section 7.8 lost its standing as evidence. The replacement is the corrected instrument at $T = 0.115$ with a window fixed in advance: $\alpha_{\text{energy}} = 0.00705 \pm 0.00012$ in $[0.0028, 0.0112]$. The lesson is which known answer to ask for first: the $T = 0$ control is the cheapest, and the one a theorem makes decisive, since with no bath there is nothing to dissipate into.

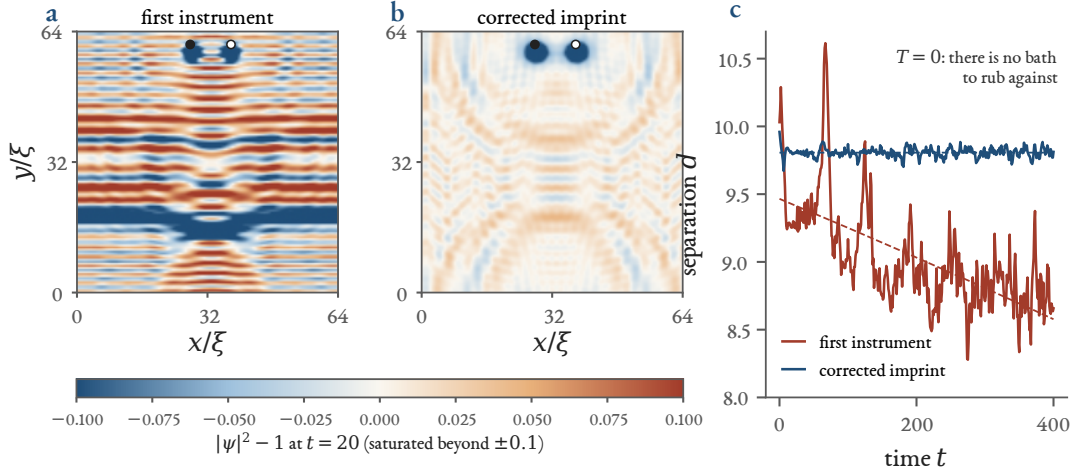


Figure 7.2: A $T = 0$ control. A single pair of separation 10 is imprinted in a uniform condensate ($L = 64$) and $|\psi|^2 - 1$ is shown 20 time units later. **a** The imprint of the first instrument leaves a phase step along a whole line of the box and a density tail, which become a sheet of sound. **b** The corrected imprint leaves a quiet wake. **c** Separation against time over 400 time units, from the archived tracks of the two gate runs (first attempt: failed; second: passed); dashed, a straight line fitted to d^2 .

The field, and what is measured in it

Figure 7.3 shows what the instrument sees. At $t = 0$ each pair is a dipole: between its members the superfluid velocity is a jet of speed about $1/d$, the two jets point in opposite directions, and each vortex rides on the jet of its partner. Over 1500 time units the two pairs travel about 2.5 times round the box in opposite directions, at the speeds 0.104 and 0.111 (the planar $1/d$ would give about 0.1). The torus point-vortex model, evaluated at the tracked positions, gives 0.102 and 0.111: the measured speed is 1.018 and 0.998 of the model's, the regression coefficient $1 - \alpha'$ below being 1.007 ± 0.006 (jackknife over eight blocks of the track), so that the centre moves at the model velocity with no sideways force within the errors, as the centre identity of Section 7.3 says it should. (The planar formula $1/d$ is 6 and 7 per cent below the model for the two pairs: the model adds the periodic images and the second pair.) The separations go from 9.7 to 8.8, between 8.4 and 11.4 along the way: the run reached the planned $t = 1500$ with all four vortices tracked throughout. The detected position of a core jitters by 0.31 per coordinate (the zero-lag offset of the residuals, $c_0 = 0.20$, for the archived runs of this arm). A least-squares slope of d^2 reads $\alpha = 0.0033$ (mean of the two pairs; separately $+0.0022$ and $+0.0044$): to be compared with the estimators below, not trusted over them.

The estimators the programme registered are the energy identity (7.3) and a two-coefficient regression: for each tracked vortex the displacement over a lag of 10 time units is regressed on $\int \mathbf{v}_{s,i} dt$ and $-q_i \int \hat{\mathbf{z}} \times \mathbf{v}_{s,i} dt$, with $\mathbf{v}_{s,i}$ the torus point-vortex velocity induced by the *other tracked vortices*; the coefficients are $1 - \alpha'$ and α , and the mean square of the residuals against lag gives

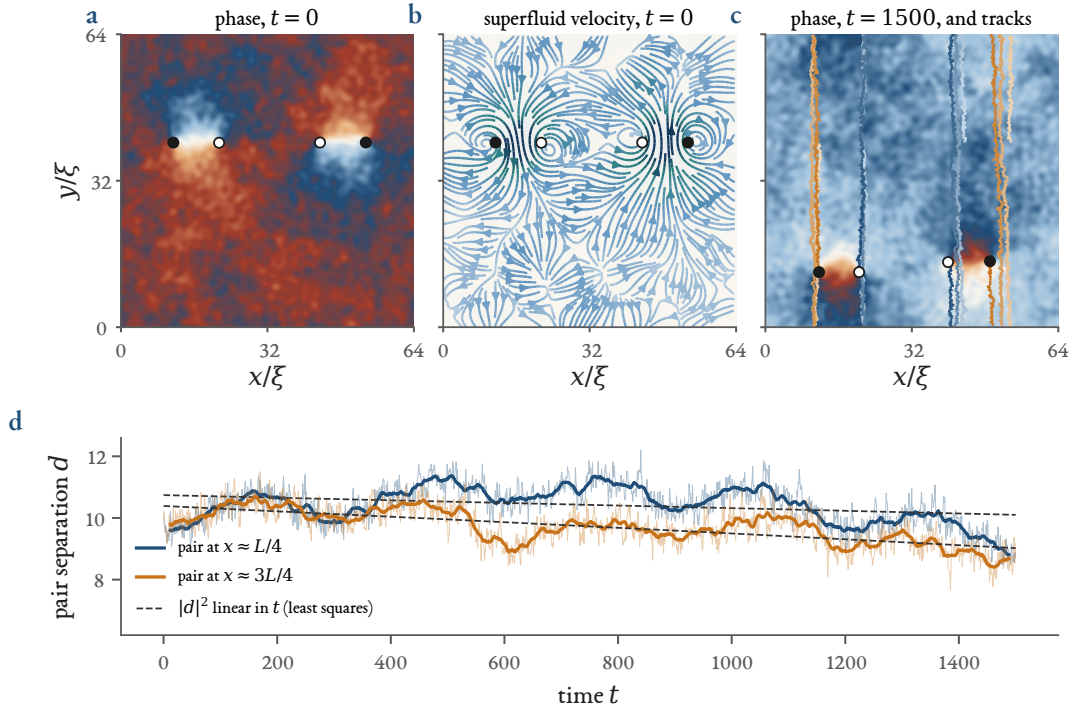


Figure 7.3: Two antiparallel vortex pairs in a thermal projected-GP field ($T = 0.115$, $T/T_{\text{BKT}} = 0.14$, $L = 64$, $d_0 = 10$), evolved with the Rust engine for 1500 time units (one run, seed 7; relative energy drift 6.8×10^{-8}). **a** Phase of ψ just after the imprint; open circles are + vortices, filled circles – vortices. **b** The superfluid velocity $\text{Im}(\psi^* \nabla \psi) / |\psi|^2$ coarse-grained over 3ξ : the jets between the members of each pair and the return flow around them; the thermal phonons are the faint background. **c** Phase at the end of the run with the four tracks (colour: time, drawn on the periodic box). **d** Separations of the two pairs against time; dashed, least-squares fits of d^2 against t .

η . Both are in the Rust extension, `qf_pgpe.analyse_tracks(tracks, 64.0)`, which takes a list of (times, positions, charges) and returns the estimates with block-jackknife errors. It agrees with the Python code of the campaign to 1×10^{-14} (relative) on the eight archived tracks of this arm (Python: 29 s; Rust: 1.2 s, on the loaded machine) and reproduces the campaign’s numbers: $\alpha_{\text{energy}} = 0.00617 \pm 0.00041$, $\alpha_{\text{regression}} = 0.00645 \pm 0.00037$, $1 - \alpha' = 1.0100 \pm 0.0015$.

Figure 7.4a is the theorem at work, with its caveat. For every run $H(t)$ and $I(t)$ are computed from the tracked positions only, and the identity says that they lie on a line of slope $-\alpha$. The eight archived runs (light blue) do, within the thermal scatter of the detected positions, but with slopes that differ from run to run: over the same 1500 time units their single-run α_E has a mean of 0.0057 and a standard deviation of 0.0020, 35 per cent of the mean (panel b). The fresh run (red) gives $\alpha_E = 0.0056$ and $\alpha_R = 0.0037$ from the registered estimators (they fit the two halves of every track separately and pool them, so that block-jackknife errors can be given; one straight line through

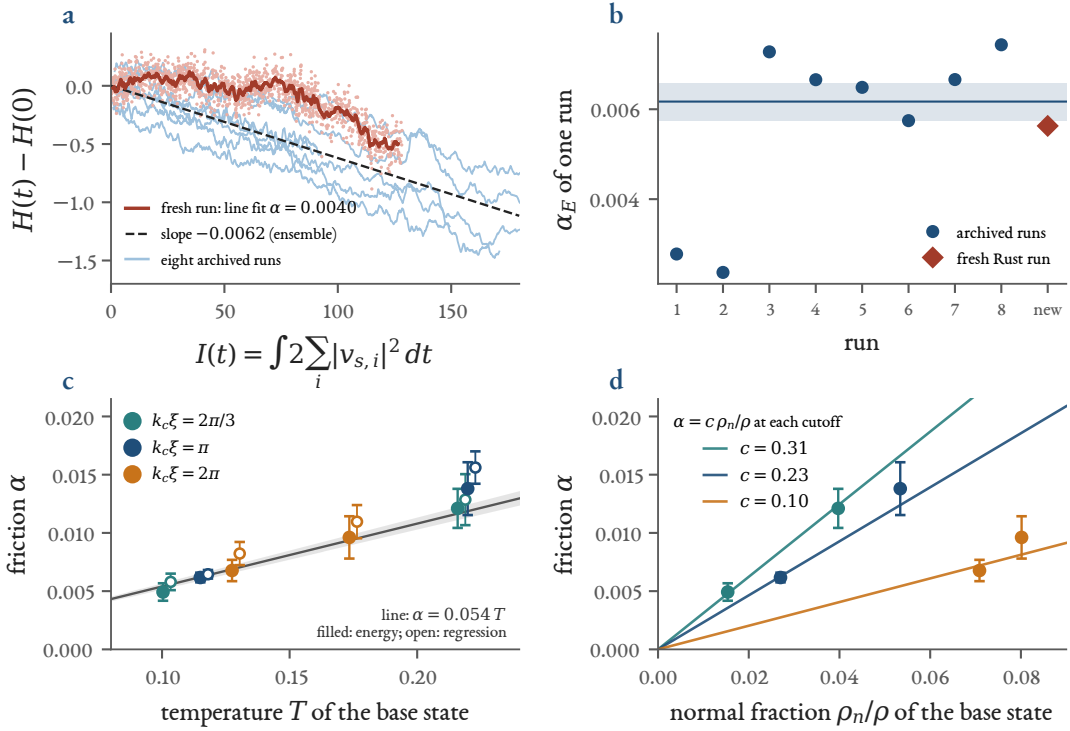


Figure 7.4: The friction in the field. **a** The energy identity on the tracks: $H(t) - H(0)$ against $I(t) = \int 2 \sum_i |\mathbf{v}_{s,i}|^2 dt$ falls along a line of slope $-\alpha$ (red: the fresh run, samples and means of 25 consecutive samples; light blue: the eight archived runs, the same means over the same duration; dashed: the slope of the ensemble). **b** Single-run estimates α_E at $T = 0.115$: the eight archived runs and the fresh one, against the ensemble mean and its jackknife error (band). **c** The friction of the six arms of the cutoff scan against the temperature of the base state (filled: energy estimator; open, shifted right: regression); the line is $\alpha = 0.054 T$ (band: one standard error). **d** The same data against the normal fraction: within each cutoff the points are proportional to ρ_n/ρ , with a coefficient c that changes by a factor 3 from one cutoff to the next.

the whole track gives 0.0040). That puts the run on the ensemble mean (0.02 standard deviations away, number 3 of the nine single runs counted from the lowest). It is one seed, drawn once and not replaced. The same run analysed at its checkpoint $t = 1000$, while the last chunk was waiting for a processor, had given $\alpha_E = -0.0013$ and $\alpha_R = -0.0016$: the estimate of one track moved by 0.0070 in 500 time units, and nothing was decided on that interim look. A pair at this temperature should lose $4\alpha t \approx 37$ in d^2 over the run, that is, d should fall from 10 to 7.9; but the separation of one pair wanders by up to 2.6 in the 25-unit means within the run (Figure 7.3d), so a single track cannot isolate the drift. That is why the campaign used six to eight runs per temperature and quoted jackknife errors over runs; and it is the reason that the identity, which is exact in the model, is a way of *estimating* α and not a way of seeing it in one track.

Table 7.1 gives the friction at the three temperatures. The three criteria registered for it passed:

T	T/T_{BKT}	ρ_n/ρ	runs	α (energy)	α (regression)	$\alpha_{d_0=8}/\alpha_{d_0=12}$
0	0	0	1	4.1×10^{-4} (floor)	2.0×10^{-4}	—
0.115	0.14	0.027	8	0.0062 ± 0.0004	0.0064 ± 0.0004	0.91
0.220	0.27	0.053	6	0.0138 ± 0.0023	0.0156 ± 0.0014	0.76
0.353 [†]	0.43	0.095	6	0.0205 ± 0.0138	0.0242 ± 0.0065	1.04

Table 7.1: Friction of a vortex pair in the closed field ($L = 64$, $k_c = \pi$; two antiparallel pairs of sizes 8 and 12, tracks of up to 2000 time units), the archived campaign re-analysed with the Rust estimators. Errors are block jackknives over runs. [†]A thermal vortex is present in more than 5% of the samples of this base state; the error is large and the base is kept with that flag. The energy estimator is low by the diffusion bias of Section 7.9.

the two pair sizes agree (ratios 0.91, 0.76, 1.04, inside the registered $[0.7, 1.43]$ at two of three temperatures), α increases with T , and the two estimators agree within 5, 13 and 18 per cent. At $0.14 T_{\text{BKT}}$ the value is a factor two below an interpolation of the values of Shukla, Brachet and Pandit [77], whose own transition temperature is an estimate; at 0.27 and 0.43 no closed-field value existed before, and these lie in the range of the oblate-condensate measurements [74]. One more observation was *not* registered, and is the subject of the next section: $\alpha/(\rho_n/\rho) = 0.23, 0.26, 0.22$.

7.6 Friction follows the temperature, not the normal density

A kinetic reading, a proportionality, and a prediction

In the projected field the bath is a finite set of modes $k \in K$ (3208 of them at $k_c = \pi$ in this box). For excitations of occupation n_k and transport cross-section $\sigma_{\text{tr}}(k)$ (a length, in two dimensions), a vortex moving slowly through them feels the drag $F = -Dv$ with $D = \frac{1}{2} \int \frac{d^2k}{(2\pi)^2} (-\partial n / \partial \epsilon) k^2 c_g \sigma_{\text{tr}}$, $c_g = d\epsilon/dk$, while the Landau normal density of the same gas is $\rho_n = \frac{1}{2} \int \frac{d^2k}{(2\pi)^2} (-\partial n / \partial \epsilon) k^2$: two integrals of the same weight, the first with one extra factor $c_g \sigma_{\text{tr}}$. A force balance on a weakly damped vortex gives $\alpha = D/(\rho_s \kappa)$. The finite sums are what Lean sees: with nonnegative weights w_k and drag factors $f_k = c_g \sigma_{\text{tr}}$, $D = \sum_k w_k f_k$ and $\rho_n = \sum_k w_k$, so that $\alpha/(\rho_n/\rho)$ is $\rho/(\rho_s \kappa)$ times the w -weighted mean of f , and that mean lies between the smallest and the largest f over the bath.

LEAN 4 FRICTIONKINETIC

```

variable {ι : Type*}

/-- The friction coefficient of the kinetic model: `α = D/(ρ_s κ)` with `D = Σ w f`. -/
noncomputable def alpha (K : Finset ι) (w f : ι → ℝ) (ps κ : ℝ) : ℝ := (Σ k ∈ K, w k *
  ↪ f k) / (ps * κ)

/-- The Landau normal density of the same bath: `ρ_n = Σ w`. -/
noncomputable def rhoN (K : Finset ι) (w : ι → ℝ) : ℝ := Σ k ∈ K, w k

theorem coeff_eq_weighted_mean (K : Finset ι) (w f : ι → ℝ) (p ps κ : ℝ) (hpn : rhoN K
  ↪ w ≠ 0) (hp : p ≠ 0) :
  alpha K w f ps κ / (rhoN K w / p) = p / (ps * κ) * ((Σ k ∈ K, w k * f k) / Σ k ∈ K,
  ↪ w k) := by

theorem weighted_mean_mem_Icc (K : Finset ι) (w f : ι → ℝ) (hw : ∀ k ∈ K, 0 ≤ w k) (
  ↪ hpos : 0 < Σ k ∈ K, w k)
  (m M : ℝ) (hm : ∀ k ∈ K, m ≤ f k) (hM : ∀ k ∈ K, f k ≤ M) :
  (Σ k ∈ K, w k * f k) / (Σ k ∈ K, w k) ∈ Set.Icc m M := by

```

Statements only; the proofs are in the module.

The two theorems are `FrictionKinetic.coeff_eq_weighted_mean` and `FrictionKinetic.weighted_mean_mem_Icc`; for a Rayleigh–Jeans bath with phonon dispersion the weights are equal (`FrictionKinetic.rayleigh_jeans_mean`). Two things follow, and the programme drew only the first. The law $\alpha \propto \rho_n$ is an *identity* of the kinetic model, for any bath. And its coefficient is not a number about the vortex: it is a mean of $c_g \sigma_{\text{tr}}$ *over the modes of the bath*. The module says so itself: “No claim about the value or the k -dependence of σ_{tr} is made; that is the measurement.”

At $k_c = \pi$ the measurement gave $\alpha / (\rho_n / \rho) = 0.23, 0.26, 0.22$ over a factor 3.5 in ρ_n , and the first paper read that coefficient as the mode-averaged transport cross-section of the vortex, $\approx 1.4\zeta$, “the geometric size of the core” [23]. That reading assumes that the coefficient belongs to the vortex and not to the bath; the check is to change the bath. The programme registered the test before running it, at two further cutoffs ($k_c = 2\pi/3$ and 2π , same instrument, $T = 0$ known answer reproduced at each): FL2, the coefficient differs between the three cutoffs by less than 20%, as a property of the vortex should; against a rival, the Born law of long-wavelength phonon scattering extended to the cutoff, which makes it proportional to k_c (0.67 : 1 : 2).

Both predictions fail

The coefficient is $0.31 \pm 0.03, 0.232 \pm 0.014$ and 0.102 ± 0.011 at $k_c \zeta = 2.1, 3.1, 6.3$ (weighted means over the arms of each cutoff, recomputed here from the six arms): the spread $(\max - \min) / \text{mean}$ is 0.98 against the registered 0.2, so FL2 fails; the Born rival fails too ($\chi^2 = 204$ for 2 degrees of freedom: the coefficient *falls* with the cutoff, and the Born law makes it rise); one common coefficient for the six arms fails ($\chi^2 = 76$ for 5). Proportionality to ρ_n holds within

each cutoff where two temperatures exist, as the identity says it must. The implied mode-averaged cross-section $c \kappa \rho_s / \rho$ is 1.9ζ , 1.4ζ and 0.59ζ (ledger values, each arm's own ρ_s): not a property of the vortex. The “geometric core size” reading is **withdrawn**.

What does not move with the cutoff is the friction itself. Figure 7.4c shows the six arms on one line, $\alpha/T = 0.0540 \pm 0.0026$ ($\chi^2 = 1.28$ for 5 degrees of freedom, against 76 for a common c), or 0.0598 ± 0.0023 with the regression estimator ($\chi^2 = 5.15$). In the first paper the three temperatures were taken at one cutoff, where ρ_n/ρ grows in proportion to T , so that a proportionality to either could not be told from the other; the cutoff scan broke the degeneracy (Figure 7.4c,d). The temperature law was found *after* the data of five arms were read. It was registered, as amendment FL-A1, as a hypothesis, with its prediction for the arm still running fixed in advance: $\alpha = 0.0094 \pm 0.0010$ at $k_c = 2\pi$, $T = 0.173$, against 0.0077 if the friction followed ρ_n with that cutoff's coefficient. The arm gave 0.0096 ± 0.0018 : inside the window registered for the temperature law, outside the other. The honest reading is smaller than the verdict: the error is almost twice the one assumed when the window was written, so the ledger puts this arm alone at 0.2σ from one value and 1.0σ from the other, and the discrimination rests on the comparison across cutoffs. The law has been tested over $T = 0.10$ – 0.22 and three cutoffs, in one box at one coupling; not above $T = 0.22$, where thermal vortices enter.

What transfers, and what does not

LL-15, the programme's rule, says that an analogy between two systems is not a result: name the property a finding depends on and check that the other system has it. The law $\alpha \propto T$ depends on a property of the *bath*. In a classical field every mode carries the energy T , so $n_k = T/\varepsilon_k$ and anything linear in the occupation is linear in T ; if $\int^\infty \sigma_{tr} dk$ converges (for $k\zeta \gg 1$ the weight of D makes the integrand $\propto \sigma_{tr}(k)$), the friction is also independent of the cutoff, while ρ_n grows logarithmically with it. Does a quantum gas have the property? We computed Landau's formula $\rho_n = \frac{1}{2}L^{-2} \sum_k k^2 (-\partial n/\partial \varepsilon)$ for free Bogoliubov quasiparticles, $\varepsilon_k^2 = k^2 + k^4/4$, on the actual mode set of each of the six arms (figures/ch07_bath.py), once with the equipartition occupation $n_k = T/\varepsilon_k$ of the field and once with the Bose occupation $1/(e^{\varepsilon_k/T} - 1)$ of a gas at the same temperature. The equipartition sum gives 64–84 per cent of the measured normal fraction of the arm (the rest is interaction); the Bose sum is smaller than the equipartition one by a factor 6 to 48.

So the field does not have it. At $T = 0.115$ the equipartition occupation of the mode at the cutoff, $k = \pi$ with $\varepsilon = 5.85 = 51 T$, is 0.020 and the Bose occupation is 8×10^{-23} ; at $k = 1$ the ratio is 1.7×10^3 ; only at $k \lesssim 0.2$, $\varepsilon \approx 1.75 T$, do the two agree within a factor 3. Of the equipartition Landau sum at $k_c = \pi$, $T = 0.115$, only 0.25 % comes from modes with $\varepsilon < T$ (2.2 % from $\varepsilon < 3T$): the normal density of the classical field is carried by modes that a quantum gas at that temperature would not populate, and its normal fraction is about thirty times that of the Bose bath (0.0227 against 0.00075). So the temperature of the classical field is a parameter of the model, which maps onto a physical temperature only through the cutoff [83], and the friction law of this section is a law of *this* model; the earlier advice to compare with quantum gases at equal ρ_n/ρ

rather than at equal T/T_c is withdrawn in the same sense. What a quantum bath would give, for helium, needs the excitation energies that Godfrin’s neutron measurements fix and the cross-section of a vortex for those excitations, which this programme has not measured. The direct measurement, a monochromatic Bogoliubov wave scattered by one vortex at $T = 0$, is the obvious next experiment; it is not in this chapter.

7.7 The sideways force and the random kicks

A prediction that failed, and a coefficient that is zero

The registered prediction for α' was the Iordanskii one: a weighted regression of $\hat{\alpha}'$ on ρ_n/ρ over the temperatures, $T = 0$ included, with a slope in $[0.6, 1.4]$; “no transverse force” was a 2σ upper bound on the slope below 0.3. In the frame of the simulation box the values are *negative*: $\hat{\alpha}' = -0.0100 \pm 0.0015$, -0.0154 ± 0.0029 , -0.0093 ± 0.0106 at the three bases of Table 7.1 (+0.0014 at $T = 0$), a slope of -0.31 ± 0.05 , 18σ from the Iordanskii window. The registered prediction is **refuted**, and the rule returns “no transverse force”. A negative α' has no theory behind it, though, and it was not left there: two kinematic facts about a compressible pair on a torus, both measured at $T = 0$, explain most of it.

First, a single pair carries field momentum $2\pi nd$, which in a periodic box is a mean superfluid velocity $u = 2\pi d/L^2$ of the whole fluid, absent from the periodic point-vortex velocity: in a $T = 0$ sweep of single pairs the ratio of measured to torus point-vortex speed rises in the box frame from 1.05 at $d = 6$ to 1.46 at $d = 16$ (the saved file `pair_speed_T0.json`; the programme’s note quotes 1.06 and 1.49), and in the fluid frame, which the ledger reports and which is not re-derived here, the pair moves at the point-vortex speed to +1% at $d = 6$ and +0.4% for $d \geq 8$ (CLAIM-o82). For two antiparallel pairs $|u| \leq 2 \times 10^{-3}$ and the pairs move in opposite directions, so the correction is below 0.0002: decisive for single pairs only. Second, a pair of finite size is a solitary wave of the compressible field, whose speed departs from the point-vortex one (the Jones–Roberts correction) by +3–4% at $d = 3$ –4, +1% at $d = 6$ and +0.4% for $d \geq 8$. Subtracting this baseline at the pair sizes the fits use gives $\alpha' = -0.0060 \pm 0.0015$, -0.0112 ± 0.0031 , -0.0055 ± 0.0106 (CLAIM-o82), with an uncertainty of the same order from the mapping between fit weights and pair sizes: α' **is zero within about 1%**, and the Iordanskii values +2.7%, +5.3%, +9.4% are excluded. The “vortices move faster than the point-vortex velocity” of the first analysis was mostly the compressibility of tight pairs at $T = 0$. This is a classical field at one cutoff; Shukla, Brachet and Pandit read a nonzero α' , without error bars, in the truncated field [77], and with a different arrangement (antiparallel pairs, the $T = 0$ baseline subtracted) we do not adjudicate. In the cutoff scan the box-frame $\hat{\alpha}'$ stays small and negative (-0.017 and -0.031 at $k_c = 2\pi/3$, -0.005 and -0.004 at 2π): reported, unexplained.

T/T_{BKT}	γ	η	K	R_E	quoted
0.14	0.71	$(3.7 \pm 0.7) \times 10^{-4}$	53.2	3.2 ± 0.6	no
0.27	0.82	$(10.8 \pm 3.5) \times 10^{-4}$	27.1	2.1 ± 0.7	yes
0.43	1.28	$(18.3 \pm 3.6) \times 10^{-4}$	16.1	1.4 ± 0.3	no

Table 7.2: The random kicks: the local exponent γ of $\langle |e|^2 \rangle - c_0$ against the lag (lags 20–400), the diffusion constant η from $\langle |e|^2 \rangle = c_0 + 4\eta\tau$, the stiffness $K = 2\pi\rho_s/T$, and $R_E = \eta K/\alpha$ with errors from η alone (α held fixed). Rule E2 quotes η only where $\gamma \in [0.8, 1.2]$.

The Einstein relation, as an identity

Take the stochastic pair of Section 7.2: drift $\mathbf{b} = -2\alpha \mathbf{d}/|\mathbf{d}|^2$ on the separation and isotropic noise of diffusion constant $D = 2\eta$ (the separation is the difference of two walkers). Its probability current for a density p is $\mathbf{J} = \mathbf{b}p - D\nabla p$, and the equilibrium pair gas of the Coulomb-gas picture of the transition [40] has $p \propto |\mathbf{d}|^{-K}$ with $K = 2\pi\rho_s/T$.

LEAN 4 EINSTEINRELATION

```

/-- The pair distribution `|d|^{-K}`, as a function of the two coordinates. -/
noncomputable def p (K x y : ℝ) : ℝ := (x ^ 2 + y ^ 2) ^ (-K / 2)

noncomputable def Jx (α D K x y : ℝ) : ℝ :=
  -2 * α * x / (x ^ 2 + y ^ 2) * p K x y - D * (-K * x / (x ^ 2 + y ^ 2) * p K x y)

theorem zero_flux_iff (α D K : ℝ) :
  (∀ x y : ℝ, x ^ 2 + y ^ 2 ≠ 0 → Jx α D K x y = 0 ∧ Jy α D K x y = 0) ↔ D * K = 2 *
  ↪ α := by

theorem einstein_vortex (α η ρ T : ℝ) (hp : 0 < ρ) (hT : 0 < T) :
  (∀ x y : ℝ, x ^ 2 + y ^ 2 ≠ 0 → Jx α (2 * η) (2 * π * ρ / T) x y = 0 ∧ Jy α (2 * η)
  ↪ (2 * π * ρ / T) x y = 0)
  ↔ η = α * T / (2 * π * ρ) := by

```

Statements only; the proofs are in the module.

The current vanishes at every separation if and only if $DK = 2\alpha$; in vortex units the pair gas can be in detailed balance iff $\eta = \alpha T/(2\pi\rho)$, the relation of Mehdi et al. [81]. Three things the programme measures separately, the shrinking of a pair (α), the random walk of its separation (D) and the stiffness (K), are tied by one identity, and $R_E = \eta K/\alpha$ is 1 exactly when the gas can be in equilibrium at that stiffness. The Lean statement adds not the physics, which is Einstein's of 1905, but the fact that the test statistic of the pre-registration is an identity of the model with the equilibrium it presupposes made explicit. That equilibrium is the hypothesis easy to read past: $|\mathbf{d}|^{-K}$ is an *assumption* about the state of the gas, and if the bath is not in equilibrium on the time scales sampled, $R_E \neq 1$ is not a violation of the identity.

Table 7.2 gives the measurement. The diffusion constant is read from the residuals of the re-

gression, $\langle |e|^2 \rangle(\tau) = c_0 + 4\eta\tau$, where the offset $c_0 \approx 0.2\text{--}1.2$ is the thermal jitter of the detected cores; at $T = 0$ the residual is bounded at 0.005. Rule E2, fixed before the analysis, quotes η only where the local exponent on lags 20–400 lies in $[0.8, 1.2]$: that admits one temperature of three, $T/T_{\text{BKT}} = 0.27$, where $R_E = 2.1 \pm 0.7$; the two unquoted temperatures would give 3.2 and 1.4. The decision rule needed two admitted temperatures: **by the rule the Einstein relation is undecided**. Informally, and against the rule, the vortex diffuses at about 1.4 to 3 times the Einstein value, the excess falling with temperature: far from the hundredfold surplus that Neely et al. measure in a trap with vibrating walls [75], and not cleanly one. A closed field has no walls to blame. (At $T/T_{\text{BKT}} = 0.43$ the error of α is 67% and would dominate the error of R_E .)

The sub-diffusive exponent at the lowest temperature, 0.71, invites a reading as a regular island: bounded quasi-periodic motion about the point-vortex trajectory. Lean makes the logic of the test precise: a finite sum of cosines has increments bounded by twice the sum of its amplitudes, so every mean of squared increments is at most $4(\sum_k |a_k|)^2$ at any lag (`QuasiPeriodicBound.msd_bound`), and `QuasiPeriodicBound.not_qp_of_msd_large` turns this into a refutation, whatever the frequencies and phases. The test is one-sided: growth refutes an island, a plateau never proves one. On lags ≥ 100 the exponents at the two cooler temperatures are 1.24 and 1.10, which is growth: the sub-diffusion is a short-lag regime of the jitter, not an island.

7.8 The wind that was not

The impulse $2\pi\rho_s d$ of a shrinking pair has to go somewhere. In a periodic box it goes into the phonons, which drift at $u = 2\pi\rho_s(d_0 - d)/(\rho_n L^2)$ along the pair, and friction, which acts on the velocity of the vortex relative to the normal fluid, is reduced: $\dot{d} = -2\alpha(1/d - u)$. This is the programme’s hypothesis W, and Lean has a theorem for it.

LEAN 4 DISSIPATIVE VORTEX DYNAMICS

```
theorem wind_stall {a b c : ℝ} (ha : 0 < a) (hab : a < b) (hc : 0 < c) (d : ℝ → ℝ)
  (hd : ∀ t, HasDerivAt d (-c * (d t - a) * (d t - b) / d t) t) (h0 : b ≤ d 0) :
  ∀ t, 0 ≤ t → b ≤ d t := by
```

```
example {a b c : ℝ} (hab : a < b) :
  (∀ t : ℝ, HasDerivAt (fun _ : ℝ => a) (-c * (a - a) * (a - b) / a) t) ∧ ¬ (b ≤ a)
  ↪ := by
refine (<fun t => ?_, not_le.mpr hab)
simp using hasDerivAt_const t a
```

The second statement is the module’s negative control: the hypothesis $b \leq d(0)$ cannot be dropped. The first is proved by a “fence” argument: a solution that went below b would have to cross a level $b - \varepsilon$ with positive derivative, which the sign of the drive forbids.

With $a + b = d_0$ and $ab = 1/w$, $w = 2\pi\rho_s/(\rho_n L^2)$, the wind equation is $\dot{d} = -2\alpha w(d - a)(d - b)/d$, and when $wd_0^2 > 4$ it has two positive zeros: a pair starting at or above b never falls

below it. The library states the identification of the two forms in a comment; the book's new module makes it a theorem and states the stall for the equation as derived.

LEAN 4 CHO7_DISSIPATIVEPAIRS (NEW, NOT IN THE LIBRARY)

```
theorem wind_stall_at_b {α w d0 : ℝ} (hα : 0 < α) (hw : 0 < w) (hd0 : 0 < d0) (h4 : 4 <
  ↳ w * d0 ^ 2)
  (d : ℝ → ℝ) (hd : ∀ t, HasDerivAt d (-(2 * α) * (1 / d t - w * (d0 - d t))) t)
  (h0 : stallPoint w d0 ≤ d 0) : ∀ t, 0 ≤ t → stallPoint w d0 ≤ d t := by
```

stallPoint w d0 is $[d_0 + \sqrt{d_0^2 - 4/w}]/2$. The module also proves that the roots are real, ordered and positive with $a + b = d_0$, $ab = 1/w$, that for $wd_0^2 < 4$ the right-hand side has no zero, and a negative control (the hypothesis on the starting point cannot be dropped); all with the three standard axioms.

For the box of this chapter ($L = 64$, $\rho_n/\rho = 0.027$) and $d_0 = 12$: $w = 0.055$, $wd_0^2 = 7.9 > 4$, and in the plane formula, which neglects the torus's image forces (with them the paper estimates nearer 9), the stall point is $b = 10.2$ (the lower root is $a = 1.8$), approached exponentially at the linearised rate $\lambda = 2\alpha w(b-a)/b = 5.6 \times 10^{-4}$, a time constant of about 1800 time units, with the measured $\alpha = 0.0062$ (CVODE, Figure 7.5a). A pair of $d_0 = 8$ should not stall ($wd_0^2 = 3.5 < 4$). Two single pairs of $d_0 \approx 12$ at $T = 0.115$, run for 4000 time units, do not annihilate, while the zero-impulse pairs of the same temperature shrink steadily: the four pairs of the two archived runs with $d_0 = 10$ go from 9.4–10.1 to 7.3–8.4 in 1500 time units. In the archived data one pair falls from 11.7 to 8.8 by $t \approx 1800$ and then wanders between 8.8 and 10.2 (the late slope of d^2 is 4% of the early one); the other falls to about 9.8 and re-expands to 11.3, back toward the stall point, so that by the letter of the registered criterion (late slope below 25% of the early one) it does not stall, though in substance it stops shrinking. The momentum of the modes with $|k| > 1$ follows the impulse shed by the pair, with regression slopes 0.69 and 0.51 against the registered window $[0.5, 1.2]$ (0.82 after the 25-unit running means of Figure 7.5b, which remove the attenuation caused by the jitter of the detected separation), and falls back when the pair re-expands. So far hypothesis W is supported; metastable counterflow states exist in the Fourier-truncated Gross–Pitaevskii equation [84], and here the counterflow would be generated by the pair itself.

Then the control that the programme had registered as decisive was run. In an $L = 96$ box at the same temperature ($\rho_n/\rho = 0.029$) the same pair has $wd_0^2 = 3.2 < 4$: the equation has no zero (QuantumFluids.DissipativePairs.wind_no_zero), the box criterion $\rho_n L^2 / (2\pi\rho_s) = 44.3$ exceeds $d_0^2/4 = 36$, and no stall is predicted. A pair that keeps shrinking supports the mechanism; a pair that stalls near the $L = 64$ separation refutes it. With a tie-break of two further runs written in advance, the four runs ended at $d = 9.3, 9.6, 8.3, 9.5$ (wind equation: 8.0–8.9; free shrinking: 5.3–6.5), and the late shrinking rate was 0, 10 and 37% of the early one in runs 2–4, where the mechanism predicted about 50%. Only one of four meets the no-stall criterion, and the rule (at most one: refuted) gives the verdict: **W2 is refuted; the pairs stall at $L = 96$ as at $L = 64$, the stall is not the box's, and the explanation by the phonon wind of a closed box is withdrawn.** The

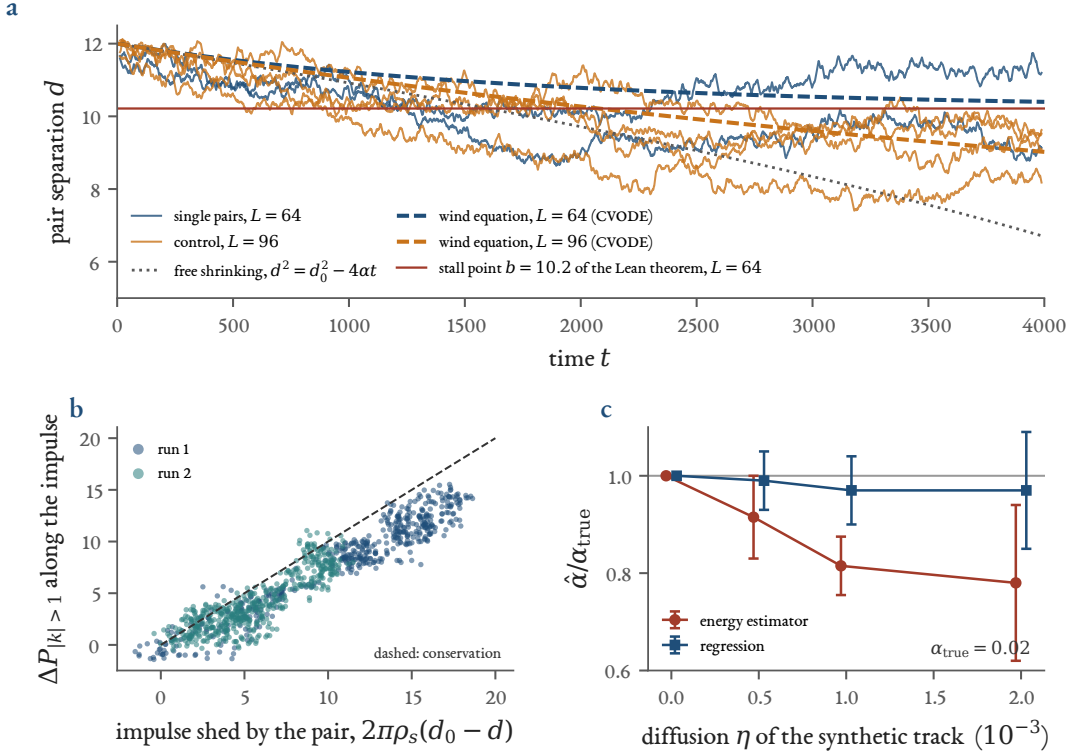


Figure 7.5: Two theorems that were true and did not describe what they were applied to. **a** Separation of single pairs at $L = 64$ (blue, two runs) and in the $L = 96$ control (orange, four runs), 25-unit running means, against the wind equation integrated with CVODE with the measured $\alpha = 0.0062$ and no free parameter (thick dashed), free shrinking (dotted) and the stall point b of the Lean theorem (red). **b** The wind seen directly: momentum of the phonon band $|k| > 1$ projected on the initial impulse against the impulse shed by the pair, for the two $L = 64$ runs; dashed, conservation of momentum between the pair and the phonons. **c** The energy estimator of α on synthetic tracks of known $\alpha = 0.02$ against the diffusion η of the track: exact at $\eta = 0$ (the theorem), biased low when $\eta > 0$; the regression estimator is not.

parameter-free wind curve of Figure 7.5a shows that a drift of the normal component is *consistent* with the data (all six pairs end within 0.2–1.5 of the curve of their box, and 1.6 to 4.6 above free shrinking); consistent is not causal. The observation is kept (a reproducible, bath-driven stall of a single pair at low temperature, with a $1/f^{0.5}$ separation spectrum), the explanation is not, and the graded test that could separate a residual wind from the real cause, a stall map in L , d_0 and T , is registered and has not been run. Every friction measurement on a single pair in a closed box at low temperature is affected, which is why the production runs use the zero-impulse geometry.

This is a clean example of what a proof assistant does and does not do. Lean certified the *implication*: if the pair obeys the wind equation it cannot fall below b , and for $L = 96$ the equation

has no zero. The experiment tested the *premise*, that the equation describes the pair, and refuted it. Nothing in the kernel could have said so.

7.9 An estimator that was exact, and was not

A second implementation is an instrument. The registered estimators were ported to Rust (crate `qf-pgpe`, module `transport`), first for speed, and the port agrees with the Python code on real tracks to 1×10^{-14} (Section 7.5). Run on the registered synthetic gate Go (known $\alpha = 0.02$, $\alpha' = 0.10$, $\eta = 2 \times 10^{-3}$, eight tracks of 2000 time units) over *many seeds*, it showed what the single seed of the gate had not.

With no diffusion the energy estimator returns the true value, exactly. With diffusion it is low, by 8.5, 18.5 and 22% at $\eta = 5 \times 10^{-4}$, 10^{-3} , 2×10^{-3} , while the regression estimator is low by only 1 to 3% (Figure 7.5c). The means over 8 seeds of eight tracks each are 0.0183 ± 0.0017 , 0.0163 ± 0.0012 , 0.0156 ± 0.0032 for the energy estimator and 0.0198 ± 0.0012 , 0.0194 ± 0.0014 , 0.0194 ± 0.0024 for the regression estimator, for a true $\alpha = 0.02$ and no detection noise (Rust example `g0_scan --eta-scan` of `qf-pgpe`, run for this chapter); the ledger records the bias as independent of the detection noise (CLAIM-098). The gate's criterion (energy estimator within 15%) passes on 2 of 12 seeds (CLAIM-098); the registered gate had passed on one lucky seed. In the real arms the regression estimator exceeds the energy one by 5 to 21%, in exactly this direction.

Why did the theorem not warn us? Because it is not about this. Its hypothesis is `hr`: the path has a derivative at every instant and obeys the flow (`DissipativeVortexDynamics.energy_dissipation`). A diffusing vortex has no derivative; the identity belongs to the *deterministic* model, and the data are not a path of that model. The cause of the bias is not proven (the programme's note suspects that the stochastic part of $H(t)$ is correlated with the cumulative regressor $\int 2S dt$, which depends on the same noisy path); what is proven is that the identity applies where its hypothesis holds, and there it is exact ($\alpha_{\text{energy}} = 0.0200$ at $\eta = 0$). This is LL-15 applied to the programme's own tool: a conclusion travels with its preconditions or not at all. No registered verdict changes (they compare arms with the same estimator), but the absolute α of Table 7.1 are low by about 5–20% (0.0062, 0.014, 0.02 become 0.0064, 0.0156, 0.0242 with the regression estimator, Table 7.1), the temperature law reads $0.060 T$ rather than $0.054 T$, and the gate's energy criterion is not a reliable known answer; the regression criterion is.

7.10 What this chapter establishes, and what it does not

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved in Lean (standard axioms only; checked on 2026-10-09). The energy identity for any flow of the form (7.2); for the plane dipole, the law $|\mathbf{d}|^2 = |\mathbf{d}_0|^2 - 4\alpha t$ and the centre velocity $(1 - \alpha')/|\mathbf{d}|$, and, in the book's new module, the law $+4\alpha t$ of the same-sign pair;

the zero-current characterisation $DK = 2\alpha$; the kinetic identity for $\alpha/(\rho_n/\rho)$; the stall of the wind equation and, new, its stall at the root b ; the bound on quasi-periodic mean-square displacements (Sections 7.3 and 7.6 to 7.8). These are statements about *models*, elementary and not new (the new theorems are in book/Lean/, not in the library), and none is a statement of Godfrin’s papers.

Simulated, in a classical projected field (one box, $mg = 1$, cutoffs $2\pi/3, \pi, 2\pi, T < 0.45 T_{\text{BKT}}$). $\alpha = 0.0062(4), 0.014(2), 0.02(1)$ at $T/T_{\text{BKT}} = 0.14, 0.27, 0.43$ by two estimators, the energy one low by 5–20%; $\alpha \simeq 0.054 T$ (energy) or $0.060 T$ (regression) over three cutoffs; $\alpha' = 0$ within about 1% once the $T = 0$ kinematics of a compressible pair on a torus are accounted for; the diffusion 1.4–3 times the Einstein value where it can be read; a bath-driven stall of a single pair; one fresh run of 1500 time units, one seed, inside the scatter of single runs. CVODE reproduces the exact pair laws to 3×10^{-8} (dipole) and 10^{-5} (rotating pair).

Analogy only, not established. Anything about helium, atomic gases, or the fluid whose excitations Godfrin measured: the classical-field bath is not a Bose bath (Section 7.6), the temperature of the field is a parameter of the model, and $\sigma_{\text{tr}}(k)$ of a vortex has not been measured here. The Einstein relation is undecided by the registered rule.

Failed, withdrawn or corrected. The first known-answer gate passed on a defective imprint and is withdrawn (CLAIM-075). The reading $\alpha \propto \rho_n$ with a vortex-owned coefficient and the “geometric core size” 1.4ζ are withdrawn: the coefficient depends on the cutoff, and the registered predictions of FL2 and of the Born law failed. The registered Iordanskii-like prediction for α' is refuted at 18σ . The phonon-wind mechanism is refuted by the $L = 96$ control while its Lean theorem stays true. The energy estimator is biased low by the diffusion, which no theorem was about. The temperature law was formulated after the data of five arms were read and is confirmed, by the registered window, on a sixth arm whose error is larger than was assumed.

7.11 Exercises

Exercise 1. Show that two vortices of the *same* sign obey $|\mathbf{d}|^2 = |\mathbf{d}_0|^2 + 4\alpha t$ and rotate with angular velocity $2(1 - \alpha')/|\mathbf{d}|^2$ about their fixed centre, and find how long a pair with $d_0 = 4\zeta$ and $\alpha = 0.02$ takes to double its separation. Then reproduce panel b of Figure 7.1: the listing of Section 7.4 needs only a new right-hand side, for two vortices of the same sign. *Hint.* With $\mathbf{d} = \mathbf{p}_1 - \mathbf{p}_2$ the velocities induced at the two vortices are $\pm \hat{\mathbf{z}} \times \mathbf{d}/|\mathbf{d}|^2$; the radial component of $\dot{\mathbf{d}}$ is $2\alpha/d$, so $d \dot{d} = 2\alpha$; the answer to the first question is $3d_0^2/(4\alpha) = 600$.

Exercise 2. Generate synthetic dipole tracks with the Langevin generator of the repository (exploration/pgpe/transport_estimators.py, function `langevin`) for $\alpha = 0.02$ and $\eta = 0$, then $\eta = 10^{-3}$, and apply the energy and the regression estimators through

`qf_pgpe.analyse_tracks`. Explain, from the hypotheses of `DissipativeVortexDynamics.energy_dissipation`, why the first case must be exact and why the second need not be. *Hint*. Look at `hr`: what regularity does a Brownian path have? Expect $\alpha_{\text{energy}} \approx 0.016$ at $\eta = 10^{-3}$ over eight tracks of 2000 time units.

Exercise 3. (LL-15.) For $\varepsilon_k = \sqrt{k^2 + k^4}/4$ at $T = 0.115$ compute $n_{\text{RJ}}/n_{\text{Bose}}$, with $n_{\text{RJ}} = T/\varepsilon_k$ and $n_{\text{Bose}} = 1/(e^{\varepsilon_k/T} - 1)$, at $k = 0.2, 1, \pi$. Then compute the fraction of the equipartition Landau sum $\frac{1}{2}L^{-2} \sum_k k^2 T/\varepsilon_k^2$ carried by modes with $\varepsilon < T$, on the 3208 modes of the disk $|k| \leq \pi, L = 64$. Which statements of Section 7.6 depend on the equipartition occupation, and which would survive in a Bose bath? *Hint*. The first part is three lines of code; the second is a fraction of a per cent.

7.12 Notes and further reading

The programme's two papers on the subject are [23], on friction, the transverse coefficient, the diffusion and the stall of a single pair (Zenodo, version 2.2, October 2026, doi:10.5281/zenodo.23262143), and [83], on the cutoff scan and the temperature law (doi:10.5281/zenodo.23262132); the engine is in [21], the Lean library in [12]. The static half of the first paper, the box-scale pair, equilibration and the polarisability of the pair gas, bears on the Kosterlitz–Thouless side of Chapter 6 and is not treated here. For mutual friction in helium start with Donnelly's book [10] and the review of Sergeev [76]; for measurements in atomic gases, [74] and [75], and, in a Fermi superfluid, where the friction and the vortex Hall angle were measured, [85]; for the diffusion, [81], and for the stochastic point-vortex model as used for helium films, [60]; for the transverse force, [78], [79] and [80]. The point-vortex flow on the torus is in [22], and the integrability of few-vortex flows in the survey [86].

Two-Dimensional Fermi Liquids: Zero Sound and the Roton Mode

Send neutrons through a sheet of liquid helium-3 that is one atom thick, and ask, for each momentum $\hbar q$ handed to the sheet, how much energy $\hbar\omega$ the neutrons leave behind. A gas of free fermions answers with a band: the energies of all the ways of lifting one atom out of the Fermi sea into a state above it. A superfluid of bosons, helium-4, answers with a single thin line, the phonon–roton branch that Landau predicted and that neutrons later traced (Chapter 5). The two-dimensional Fermi liquid studied by Godfrin and his collaborators answers with both, one after the other. At small q the band is accompanied by a collective density oscillation, of the kind Landau called zero sound, lying just above it; at intermediate q the oscillation runs into the band and is strongly broadened; and at wave vectors larger than twice the Fermi wave vector a well-defined excitation with a roton-like minimum reappears, in a regime where Landau’s theory has nothing to say [1, 87]. The third of these gives the chapter its title.

This chapter does not reproduce that measurement, and the reason is part of what it has to teach. The third regime lies outside anything the three instruments of this book can treat exactly. The first two do not: they belong to Landau’s theory of a Fermi liquid, here collisionless, in two dimensions, with one dimensionless interaction parameter F . For that model the proof assistant can say, once and for all, when an undamped zero-sound wave exists and where it sits; pencil and paper can say how the weight of the neutron spectrum is shared between the wave and the band; and CVODE, integrating the equation of motion in time, can be held against the pencil to 10^{-6} or better and made to show how a density wave dies out although nothing is dissipated. The last section turns to the missing regime, proves the one piece of kinematics that is exact about it, and says where the model stops.

One caution runs through everything below. The two-dimensional Landau parameters of the film measured by Godfrin and his collaborators are *not known to this book*: no open numerical table of them was found, and the programme’s two literature searches came back empty. The parameter F is therefore left free, every curve is a function of F , and no number here is a prediction for the film.

8.1 What the neutrons saw

GODFRIN WHAT WAS MEASURED

Experiment. Inelastic neutron scattering at the Institut Laue–Langevin, Grenoble, on two-dimensional liquid ^3He : a monolayer of ^3He adsorbed on graphite preplated with a monolayer of solid ^4He [1, 87]. The 2012 paper is H. Godfrin, M. Meschke, H.-J. Lauter, A. Sultan, H. M. Böhm, E. Krotscheck and M. Panholzer, *Nature* **483**, 576–579 (published 29 March 2012); an earlier paper on the same measurements is [87].

What the abstracts report. (i) The particle–hole excitations of the two-dimensional Fermi liquid and, above the particle–hole band, the zero-sound mode; at low wave vectors this mode lies very close to the band, contrary to bulk ^3He . (ii) At intermediate wave vectors the collective mode enters the band, where it is strongly broadened by Landau damping. (iii) At wave vectors larger than twice the Fermi wave vector the collective density mode reappears as a well-defined, roton-like excitation, with a dispersion relation “quite similar to that of superfluid ^4He ”. The dynamics of Fermi liquids at high wave vectors had been believed “essentially incoherent”; the authors find “unexpected collective behaviour” beyond the scope of Landau’s theory and interpret the spectra with a dynamic many-body theory. The authors’ copy of the 2012 paper (open archive HAL, hal-00920422) names the instrument: the time-of-flight spectrometer IN6 of the ILL.

What this chapter does not have: any number from those spectra (energies, wave vectors, densities) and the film’s Landau parameters. Where it says what a measurement decided, it quotes the abstracts and goes no further.

Why was this hard, and what did it decide? A single adsorbed layer holds very little matter compared with a bulk sample, so the scattering signal is small, and a collective mode has to be recognised on top of a particle–hole band that is broad. The three regimes of the box are three questions put to the same spectrum: where is the collective mode while Landau’s theory applies, what happens when it meets the band, and does anything survive beyond. The expected answer to the third was “no”: a mode that has entered the band is damped, and nothing in the theory brings it back. The measurement decided that the answer is “yes” for a two-dimensional Fermi liquid, which is a statement about nature that no model in this chapter could have supplied.

It helps to hold the (q, ω) plane in mind. Lift one fermion out of a Fermi sea of radius k_F and give it momentum $\hbar q$: the energies it can absorb fill a band, bounded above by $\hbar v_F q + \hbar^2 q^2 / 2m$ and, for $q < 2k_F$, below by zero. A density wave can be a sharp excitation only if its dispersion lies outside the band; inside, it can decay into pairs and is broadened. At small q Landau’s theory says where such a wave sits, and at larger q the line it draws enters the band. The abstract of the 2012 paper names the wave vector $2k_F$, at which the band itself changes character because its lower edge lifts off $\omega = 0$; Section 8.8 proves that and draws the plane in Figure 8.5.

8.2 A Fermi liquid with one number

Landau's picture keeps what survives of the free Fermi gas [14, 25]. The low-lying states are quasi-particles with momentum near the Fermi circle $|\mathbf{p}| = p_F$; the only state variable of the linear theory is how far the Fermi circle has been pushed outwards (in energy units) in the direction θ , at the place x and the time t , a function $\nu(\theta; x, t)$. A density wave is a Fermi circle that breathes in a travelling pattern. The quasi-particles interact through the Landau function, whose dimensionless form is $F(\theta - \theta') = N(0)f(\theta - \theta')$, with $N(0)$ the density of states at the Fermi level. We keep one number from it, the isotropic spin-symmetric part $F \equiv F_0^s$, and set the rest to zero. That is a model with four honest costs: with $F_1^s = 0$ the effective mass equals the bare mass, which is not a good description of helium; the spin-antisymmetric channel is dropped; there are no collisions, so the theory describes $\omega\tau \gg 1$, the regime in which zero sound exists, at zero temperature; and the curvature of the quasi-particle spectrum is dropped, so everything is a statement about $q \ll k_F$ and $\hbar\omega \ll E_F$.

For a perturbation along x the linearised, collisionless Landau kinetic equation reads

$$\partial_t \nu + v_F \cos \theta \partial_x [\nu + F \langle \nu \rangle] = 0, \quad \langle \nu \rangle = \frac{1}{2\pi} \int_0^{2\pi} \nu d\theta, \quad \delta n = N(0) \langle \nu \rangle. \quad (8.1)$$

The first term is free streaming: the part of the Fermi circle at angle θ is carried along x with velocity $v_F \cos \theta$. The second is the mean-field push, F times the density deviation, the same for every θ . Free streaming plus a force from the mean field is also the structure of the Vlasov equation of a plasma, a likeness that Section 8.7 uses and puts to a test.

A plane wave $\nu \propto e^{i(qx - \omega t)}$, with $s = \omega/(v_F q)$, turns (8.1) into $(s - \cos \theta) \nu = F \cos \theta \langle \nu \rangle$; dividing by $s - \cos \theta$ and averaging over the angle leaves the dispersion relation

$$1 + F \Omega(s) = 0, \quad \Omega(s) = \left\langle \frac{\cos \theta}{\cos \theta - s} \right\rangle = 1 - s \left\langle \frac{1}{s - \cos \theta} \right\rangle, \quad (8.2)$$

where Ω is the angular average of the response of free quasi-particles. On a circle the average is algebraic; on a sphere it is a logarithm:

$$\begin{aligned} \Omega_{2D}(s) &= 1 - \frac{s}{\sqrt{s^2 - 1}} \quad (s > 1), & \Omega_{2D}(s) &= 1 + i \frac{s}{\sqrt{1 - s^2}} \quad (0 < s < 1), \\ \Omega_{3D}(s) &= 1 - \frac{s}{2} \ln \frac{s+1}{s-1} \quad (s > 1). \end{aligned} \quad (8.3)$$

For $s > 1$ the wave is faster than every quasi-particle and Ω is real. For $0 < s < 1$ some quasi-particles move along x at exactly the phase velocity, $v_F \cos \theta = \omega/q$; they surf the wave and exchange energy with it, and Ω acquires an imaginary part (its sign fixed by $\omega \rightarrow \omega + i0$). That is Landau damping, and the strip $0 < \omega < v_F q$ is the particle-hole continuum.

An undamped wave needs a real root with $s > 1$. In both dimensions $\Omega(s) < 0$ for $s > 1$,

Table 8.1: The model's numbers (closed forms). s_0 : the two-dimensional zero-sound root (Lean: `QuantumFluids.ZeroSound.zero_sound_2d_iff`); s : the three-dimensional root of $g_3(s) = 1/F$, for contrast; $q_c/k_F = 2(s_0 - 1)$: where the line $\omega = s_0 v_F q$ meets the free-gas band edge $v_F q + \hbar q^2/2m$; W , “pole share”: the weight of the line in the structure factor and its share of the f -sum rule (Section 8.4; exactly $\frac{8}{9}, \frac{80}{81}, \frac{624}{625}$ at $F = 1, 4, 12$); “amplitude”: the undamped part of the density after a released wave (Section 8.5).

F	s_0 (2D)	s (3D)	q_c/k_F	W	pole share	amplitude
$\frac{1}{2}$	1.0607	1.0051	0.121	0.1768	0.7500	0.5000
1	1.1547	1.0444	0.309	0.1925	0.8889	0.6667
4	1.6667	1.4076	1.333	0.1481	0.9877	0.8889
12	2.6000	2.1487	3.200	0.0960	0.9984	0.9600

so $\Omega(s) = -1/F$ can be solved only if $F > 0$: a repulsive interaction pushes the wave out of the continuum; an attractive one leaves nothing sharp. In two dimensions the root can be written down; squaring (8.2) and using $(1 + F)^2 - F^2 = 1 + 2F$ gives

$$s_0 = \frac{1 + F}{\sqrt{1 + 2F}}, \quad s_0^2 - 1 = \frac{F^2}{1 + 2F} \quad (F > 0), \quad (8.4)$$

and $s_0^2 = \frac{F}{2} + \frac{3}{4} + \frac{1}{4(1+2F)}$ shows the strong-coupling limit at a glance. Weak coupling: $s_0 - 1 \simeq F^2/2$ (the ratio is 0.9980 at $F = 10^{-3}$), so the wave leaves the edge of the continuum only quadratically in F . Strong coupling: $s_0 \rightarrow \sqrt{F/2}$. In three dimensions the same limits are $s - 1 \simeq 2e^{-2-2/F}$ and $s \rightarrow \sqrt{F/3}$: the sphere lets the wave hug the edge exponentially closely, the circle only quadratically (Table 8.1). Within the model, “very close to the particle-hole band”, in the words of the 2010 abstract, means a small F : $s_0 - 1$ is 0.0607 at $F = \frac{1}{2}$ and 1.6000 at $F = 12$. That is the model’s way of speaking about the abstract’s sentence; it does not determine the film’s F .

Two further facts about (8.2). At strong coupling the zero-sound speed approaches the hydrodynamic first-sound speed $v_F \sqrt{(1 + F)/2}$ (from $c_1^2 = (n/m) \partial\mu/\partial n$, $\partial\mu/\partial n = (1 + F)/N(0)$); the model has no collisions and does not compute the crossover, the two formulas merely meet. And in the static limit $\Omega(0) = 1$, so the compressibility is $1/(1 + F)$ times its free value and $F > -1$ is the Pomeranchuk bound, on either side of which Section 8.6 sees the model change its behaviour.

8.3 What Lean proves

The library module `ZeroSound` states the existence criterion in both dimensions. Its header says what it is and what it is not: textbook physics [14], machine-checked, with the two-dimensional case included because the measurement of Section 8.1 is on a film, where the logarithm of three dimensions does not apply.

LEAN 4 ZERO SOUND

```

theorem zero_sound_2d_iff (F : ℝ) :
  (∃ s, 1 < s ∧ 1 + F * (1 - s / √(s ^ 2 - 1)) = 0) ↔ 0 < F

-- (proof abridged) for F > 0 the root is (1 + F) / √(1 + 2 * F); key step:
--   have e : ((1 + F) / √(1 + 2 * F)) ^ 2 - 1 = (F / √(1 + 2 * F)) ^ 2 := by
--     rw [div_pow, div_pow, hq2]; field_simp; ring

```

In words: an undamped two-dimensional zero-sound wave exists *if and only if* $F > 0$. If a root $s > 1$ exists then $s > \sqrt{s^2 - 1}$, so $\Omega(s) < 0$, and $0 = 1 + F\Omega(s)$ forces $F > 0$. If $F > 0$, the displayed identity gives $\sqrt{s_0^2 - 1} = F/\sqrt{1 + 2F}$, hence $s_0/\sqrt{s_0^2 - 1} = (1 + F)/F$ and $1 + F(1 - (1 + F)/F) = 0$. That a helium film obeys the equation is physics, and whether its F is positive is a question for the film.

In three dimensions there is no closed form, so the root is pinned instead.

LEAN 4 ZERO SOUND

```

noncomputable def g3 (s : ℝ) : ℝ := s / 2 * log ((s + 1) / (s - 1)) - 1

theorem zero_sound_iff (F : ℝ) : (∃ s, 1 < s ∧ g3 s = 1 / F) ↔ 0 < F

```

For $c = 1/F > 0$ the function g_3 is continuous on $(1, \infty)$ (`QuantumFluids.ZeroSound.continuousOn_g3`) and obeys $g_3(s) \geq \frac{1}{2} \ln \frac{2}{s-1} - 1$ (`QuantumFluids.ZeroSound.g3_ge`) and $g_3(s) \leq 1/(s-1)$ (`QuantumFluids.ZeroSound.g3_le`); at $s = 1 + 2e^{-(c+2)}$ the first bound gives $g_3 \geq c$, at $s = 1 + 2/c$ the second gives $g_3 \leq c/2$, and the intermediate value theorem delivers a root between. The module says what it does not prove: uniqueness of the three-dimensional root (that needs strict monotonicity of g_3), anything about F_1^s or finite temperature, and the damped branch $s < 1$.

Uniqueness *is* easy in two dimensions, and the book adds it in a new module, `Ch08_ZeroSound2D`, written for this chapter and compiled with the pinned toolchain, with no sorry (axioms in Section 8.9); everything in it is elementary.

LEAN 4 CH08_ZERO SOUND2D (NEW IN THIS BOOK)

```

noncomputable def omega2 (s : ℝ) : ℝ := 1 - s / √(s ^ 2 - 1)

noncomputable def s0 (F : ℝ) : ℝ := (1 + F) / √(1 + 2 * F)

theorem zero_sound_2d_unique {F s : ℝ} (hs : 1 < s)
  (h : 1 + F * omega2 s = 0) :
  0 < F ∧ s = s0 F

```

Together with `QuantumFluids.ZeroSound.zero_sound_2d_iff` this says: for $F > 0$ there is exactly one undamped zero-sound root, namely s_0 , and for $F \leq 0$ there is none. The proof squares the equation: $Fs = (1 + F)\sqrt{s^2 - 1}$ gives $s^2(1 + 2F) = (1 + F)^2$, and s is positive. (The statement `QuantumFluids.ZeroSound2D.zero_sound_2d_unique` lives in the new file; the library is unchanged.)

A theorem about the root invites a check of the number. The root can be *followed* instead of solved: along $u = \ln F$ the variable $w = \ln(s - 1)$ obeys the differential equation obtained by differentiating $1 + F\Omega(s) = 0$,

$$\frac{dw}{du} = \frac{s + 1}{s + \sqrt{s^2 - 1}} \quad (2D), \quad \frac{dw}{du} = -\frac{g_3(s)}{(s - 1)g'_3(s)} \quad (3D). \quad (8.5)$$

CVODE (the Rust crate `cvoid` of `rusty-SUNDIALS`, BDF, relative tolerance 1×10^{-12}) integrates (8.5) from $F = 0.05$ to $F = 1000$ and returns 81 points. Against the Lean closed form the largest relative error in s is 1.0×10^{-11} in two dimensions; against a 40-digit `mpmath` root it is 3.1×10^{-11} in three; the original equations are satisfied to 2.0×10^{-11} and 6.2×10^{-11} (open circles in Figure 8.1a, which also shows the circle against the sphere). This checks the solver against the theorem; the theorem needs no checking.

8.4 The structure factor, the pole and the sum rule

A neutron does not see ν ; it sees the dynamic structure factor $S(q, \omega)$. In the model the density response is $\chi = -N(0)\Omega/(1 + F\Omega)$, and at zero temperature $S = -\pi^{-1} \text{Im } \chi$ for $\omega > 0$; write $S(q, \omega) = N(0)\hat{S}(s)$ (the neutron-scattering constants are dropped). Inside the continuum $\text{Im } \Omega = s/\sqrt{1 - s^2}$ and $|1 + F\Omega|^2 = [(1 + F)^2 - (1 + 2F)s^2]/(1 - s^2)$, so

$$\hat{S}(s) = \frac{\text{Im } \Omega}{\pi|1 + F\Omega|^2} = \frac{s\sqrt{1 - s^2}}{\pi[(1 + F)^2 - (1 + 2F)s^2]} = \frac{s\sqrt{1 - s^2}}{\pi(1 + 2F)(s_0^2 - s^2)}, \quad 0 < s < 1. \quad (8.6)$$

(The closed form agrees with the definition to 1.1×10^{-15} on a grid of s and F .) For $F > 0$ the denominator vanishes at s_0 , just outside the interval, and the continuum is joined by a δ -function: near s_0 one has $1 + F\Omega \simeq F\Omega'(s_0)(s - s_0)$ with $\Omega'(s) = (s^2 - 1)^{-3/2}$, so the line has the weight

$$W = \frac{1}{F^2 \Omega'(s_0)} = \frac{(s_0^2 - 1)^{3/2}}{F^2} = \frac{F}{(1 + 2F)^{3/2}}. \quad (8.7)$$

How much of the spectrum does the line carry? There is an exact answer. The function $G(s) = \Omega/(1 + F\Omega)$ is analytic in the upper half plane for $F > -1$, and at large $|s|$ one has $\Omega \simeq -1/(2s^2)$, hence $G \simeq -1/(2s^2)$. Its Hilbert-transform representation $G(s) = \pi^{-1} \int \text{Im } G(s') (s' - s)^{-1} ds'$, with $\text{Im } G$ odd and equal to $\pi\hat{S}$ for $s > 0$, gives $G \simeq -2s^{-2} \int_0^\infty s' \hat{S}(s') ds'$, and comparing the

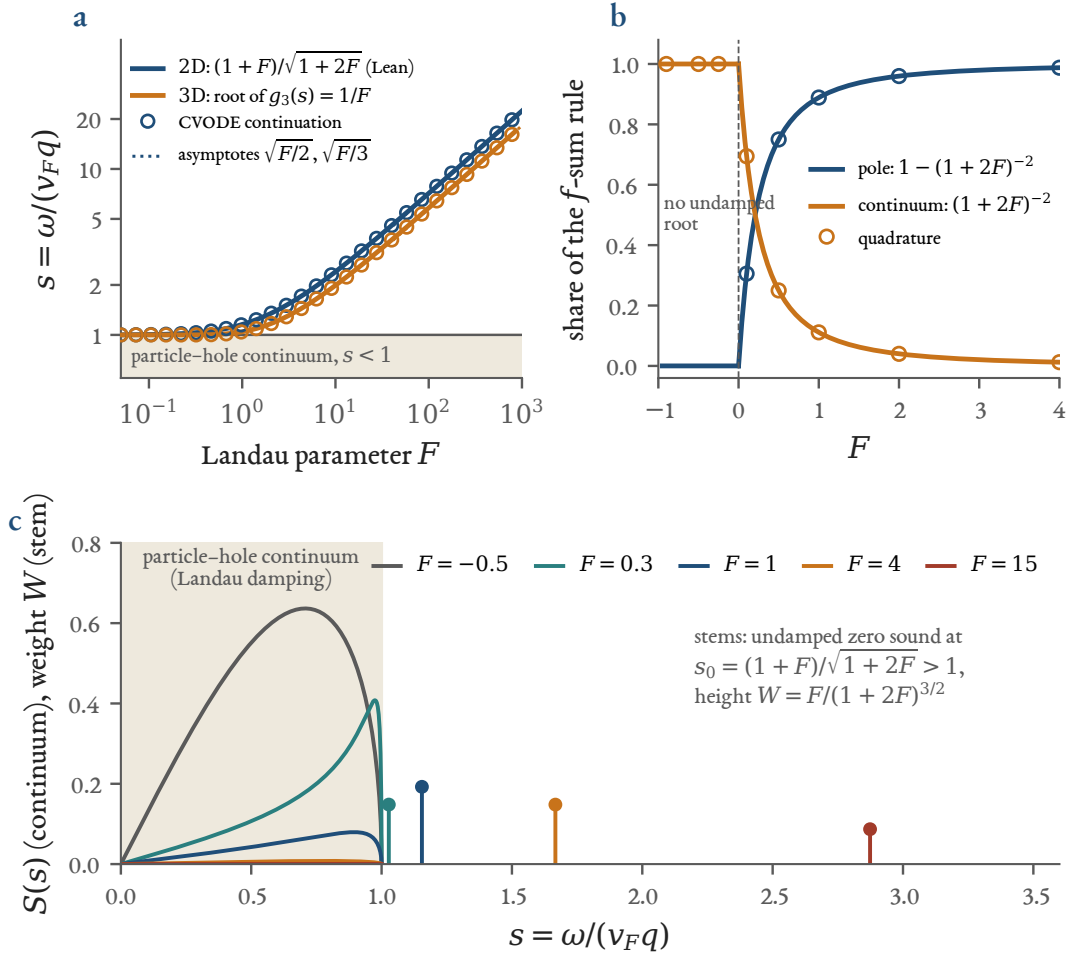


Figure 8.1: The zero-sound root and what it carries. **a**, the root $s(F)$: two dimensions (blue, the Lean closed form), three dimensions (orange, root of $g_3(s) = 1/F$); open circles, CVODE continuation of (8.5) (81 points, relative tolerance 1×10^{-12}); dotted, the asymptotes $\sqrt{F/2}$ and $\sqrt{F/3}$; shaded, the particle-hole continuum $s < 1$. **b**, shares of the first moment $\int s \hat{S} ds = \frac{1}{4}$: the pole carries $1 - (1+2F)^{-2}$ (blue; Lean: `QuantumFluids.ZeroSound2D.pole_fraction`), the continuum the rest (orange); open circles, quadrature of (8.6). For $F \leq 0$ there is no pole. **c**, the structure factor (8.6) for five values of F (a curve and its stem have one colour); a stem at s_0 has height W of (8.7) and stands for a δ -function. Script: `figures/ch08_zerosound.py`.

two,

$$\int_0^\infty s \hat{S}(s) ds = \frac{1}{4} \quad (F > -1), \quad (8.8)$$

which is the f -sum rule $\int \omega S d\omega = nq^2/2m$ of the model (in two dimensions $n = N(0)E_F$ and $E_F = \frac{1}{2}mv_F^2$). It holds for every F , so a line that exists takes its weight from the band. The pole contributes $s_0 W = F(1+F)/(1+2F)^2$, and since $4F(1+F) = (1+2F)^2 - 1$ its share is

$$\frac{s_0 W}{1/4} = 1 - \frac{1}{(1+2F)^2} \quad (F > 0). \quad (8.9)$$

The identities behind (8.7) and (8.9) are the new Lean theorems below. What Lean does *not* prove is that W is the strength of a δ -function in S , or that the first moment is $\frac{1}{4}$: those are the residue theorem and the f -sum rule, which the module does not contain. They are checked numerically: the continuum quadrature plus the pole gives $\frac{1}{4}$ to 6.3×10^{-15} at eight values of F between -0.9 and 15 , and the continuum alone gives $\frac{1}{4}(1+2F)^{-2}$ for $F > 0$ and exactly $\frac{1}{4}$ for $F \leq 0$.

LEAN 4 CHO8_ZERO_SOUND2D (NEW IN THIS BOOK)

```
theorem pole_fraction {F : ℝ} (hF : 0 < F) :
  4 * s0 F * (1 / (F ^ 2 * deriv omega2 (s0 F)))
  = 1 - 1 / (1 + 2 * F) ^ 2
```

This is (8.9), with the residue written $1/(F^2 \Omega'(s_0))$. Two further theorems of the new file carry the pieces: `QuantumFluids.ZeroSound2D.omega2_hasDerivAt` proves $\Omega'(s) = (s^2 - 1)^{-3/2}$ as a derivative of the function `omega2`, and `QuantumFluids.ZeroSound2D.pole_weight` turns that into (8.7). The proof of `QuantumFluids.ZeroSound2D.pole_fraction` is then algebra: the two square roots collapse because $s_0^2 - 1 = F^2/(1+2F)$ (`QuantumFluids.ZeroSound2D.s0_sq_sub_one`).

Read as physics, Figure 8.1b says that a weak repulsion ($F \rightarrow 0^+$) gives the line almost no weight, about $4F$ of the sum rule, and leaves it hugging the band, while a strong one pushes it out and gives it almost everything: at $F = 4$ the line carries $80/81$. Whether that says anything about the film the model cannot tell.

8.5 Integrating the kinetic equation with CVODE

Equation (8.1) goes onto a computer almost verbatim. Take units $v_F q = 1$. For the initial states used below ν is even in θ , so only $\theta \in (0, \pi)$ is needed, with N midpoint nodes $\theta_j = (j + \frac{1}{2})\pi/N$ and $\langle \cdot \rangle \rightarrow \frac{1}{N} \sum_j$ (a smooth periodic integrand integrated this way is exponentially accurate for times up to order N). With $c_j = \cos \theta_j$ and the real and imaginary parts of ν_j as unknowns, the system $\dot{\nu}_j = -ic_j(\nu_j + F \langle \nu \rangle)$ is linear with a constant matrix, which is also its Jacobian and is handed to CVODE as such.

RUSTY-SUNDIALS CVODE ON THE LANDAU KINETIC EQUATION

Excerpts of the right-hand side and the driver, as run (crate `cvode 6.4.0`, rusty-SUNDIALS commit `5db8041`; book/rust/ch08_kinetic); the closure `jac` fills the $2N \times 2N$ matrix $\begin{pmatrix} 0 & A \\ -A & 0 \end{pmatrix}$, $A_{ij} = c_i \delta_{ij} + c_i F/N$.

```
let rhs = move |_t: f64, y: &[f64], yd: &mut [f64]| -> Result<(), String> {
    let mr = y[..n].iter().sum::<f64>() / nf;
    let mi = y[n..].iter().sum::<f64>() / nf;
    for j in 0..n {
        yd[j] = c_rhs[j] * (y[n + j] + f * mi);
        yd[n + j] = -c_rhs[j] * (y[j] + f * mr);
    }
    Ok(())
};
...
let builder = Cvode::builder(method).rtol(rtol).atol(atol)
    .max_steps(5_000_000);
let builder = if std::env::var("CH08_NOJAC").is_ok() { builder }
    else { builder.jacobian(jac) };
let mut cv = builder.build(rhs, 0.0, SerialVector::from_slice(&y0))
    .expect("build");
...
let (t, y) = cv.solve(tout, Task::Normal)
    .unwrap_or_else(|e| panic!("solve to {tout}: {e:?}"));
```

One output ($F = 4$, $N = 64$, $0 \leq t \leq 60$, the density mode $m = \langle v \rangle$ recorded every 0.1; the line is the driver's own, copied from `figures/ch08_statsline.txt`):

```
STATS F=4 ic=uniform N=64 method=bdf rtol=1e-8 atol=1e-10 tend=60 steps=5963
    rhs_evals=15015 wall_s=13.607
```

Shared eight-core machine, load average 12–35 while these runs were made: wall times are not benchmarks (the same $F = 0$ problem took 7.1 s and 12.7 s in two runs of one batch).

There are exact answers to compare with. For $F = 0$ the nodes decouple and, for the initial state $v \equiv 1$ (a density wave suddenly released), $m(t) = \langle e^{-it \cos \theta} \rangle = J_0(t)$. For $F = -\frac{1}{2}$ the Laplace transform of the density is $2/(p + \sqrt{p^2 + 1})$, whose inverse is $2J_1(t)/t$. For any F the Laplace transform of m is $\hat{m}(p) = (p^2 + 1)^{-1/2}/[1 + F(1 - p/\sqrt{p^2 + 1})]$, which is Ω continued to imaginary frequency; its singularities are, for $F > 0$, the poles $p = \pm is_0$ (residue $F/(1 + 2F)$ each) and, for every F , the cut $|\operatorname{Im} p| < 1$ of the square root. Inverting,

$$m(t) = \frac{2F}{1 + 2F} \cos(s_0 t) + \frac{2}{\pi} \int_0^1 \frac{(1 + F)\sqrt{1 - y^2}}{(1 + F)^2 - (1 + 2F)y^2} \cos(yt) dy \quad (F > 0), \quad (8.10)$$

Table 8.2: CVODE (BDF, $N = 64$, relative tolerance 10^{-8} , absolute 10^{-10}) on the kinetic equation, $0 \leq t \leq 60$, against the closed forms. “Released”: $\nu \equiv 1$; “kick”: $\nu = \cos \theta$ (not run at $F = 0$). Energy drift: $\max_t |E(t)/E(0) - 1|$ for the conserved form (8.11). Steps for the released runs.

F	max error, released	max error, kick	energy drift	steps
$-\frac{1}{2}$	4.0×10^{-9}	1.4×10^{-9}	3.2×10^{-8}	3 910
0	1.7×10^{-8}	—	1.1×10^{-7}	3 947
$\frac{1}{2}$	6.5×10^{-8}	2.1×10^{-8}	1.5×10^{-7}	4 133
1	1.2×10^{-7}	3.1×10^{-8}	2.5×10^{-7}	4 370
4	3.1×10^{-7}	9.0×10^{-8}	6.5×10^{-7}	5 963
12	1.0×10^{-6}	1.8×10^{-7}	2.0×10^{-6}	7 935

and the same integral alone for $F \leq 0$; at $t = 0$ the two parts are $2F/(1 + 2F)$ and $1/(1 + 2F)$ and add to one. A second initial state, $\nu = \cos \theta$ (a density *kick*), gives a purely imaginary m with $-\text{Im } m(t)/2 = \int_0^\infty \hat{S}(s) \sin(st) ds$: the time signal is the sine transform of the structure factor (8.6), the δ -function contributing $W \sin(s_0 t)$. Its slope at $t = 0$ is the f -sum rule, $\dot{m}(0) = -i \langle \cos^2 \theta \rangle = -\frac{1}{2}i$ (the solver’s difference quotient over 0.1 of $\text{Im } m$ is -0.4996 at $F = -\frac{1}{2}$ and -0.4944 at $F = 12$).

Figure 8.2a is that formula made visible, and Table 8.2 gives the numbers. CVODE agrees with the closed forms at every F tried: the largest deviation over $0 \leq t \leq 60$ is 4.0×10^{-9} at $F = -\frac{1}{2}$ and 1.7×10^{-8} at $F = 0$, and grows to 1.0×10^{-6} at $F = 12$, where the solution oscillates fastest and more steps accumulate more error; for the kick the deviations of $-\text{Im } m/2$ from the sine transform are 1.4×10^{-9} and 1.8×10^{-7} .

The sharpest test against Lean is the pole. Subtract the exactly known band part of (8.10) from the CVODE signal; what remains must be one undamped cosine. A least-squares fit of $A \cos(\omega t + \varphi)$ over $0 \leq t \leq 60$ gives, at $F = 4$, $\omega = 1.666666667$ against the Lean root $s_0 = 5/3 = 1.666666667$ and $A = 0.888889$ against $2F/(1 + 2F) = 8/9$; at $F = \frac{1}{2}$, $\omega = 1.060660172$ against $s_0 = 1.060660172$. Over $F = \frac{1}{2}, 1, 4, 12$ the frequency agrees with s_0 to a relative 1.8×10^{-10} to 6.4×10^{-10} and the amplitude with $2F/(1 + 2F)$ to 7.6×10^{-8} to 5.2×10^{-7} (Figure 8.2c). The time integration of the kinetic equation lands on the number that Lean proved to exist, ten digits deep.

Two choices about the solver were not free, and a reader reproducing this should know them. *BDF, not Adams*. At $F = 0$, $N = 64$, output every 0.1, the BDF option reproduces J_0 to 5.0×10^{-7} , 1.7×10^{-8} and 1.7×10^{-9} at relative tolerances 10^{-6} , 10^{-8} , 10^{-10} ; the Adams option of the same crate gives 1.1×10^1 , 1.9×10^{-1} and 4.0×10^{-4} , the first two of the order of the signal itself or larger. The defect is intermittent. With only $N = 4$ nodes at tolerance 10^{-8} , Adams is accurate (1.1×10^{-11} to 1.5×10^{-7}) when the solution is requested every 0.01, 1, 5, 10, 20 or 60 time units, and wrong (3.3×10^{-3} ; 4.7×10^{-2} with the Jacobian differenced by CVODE) when it is requested every 0.1, while BDF stays between 3.3×10^{-8} and 1.2×10^{-6} for all seven spacings. We

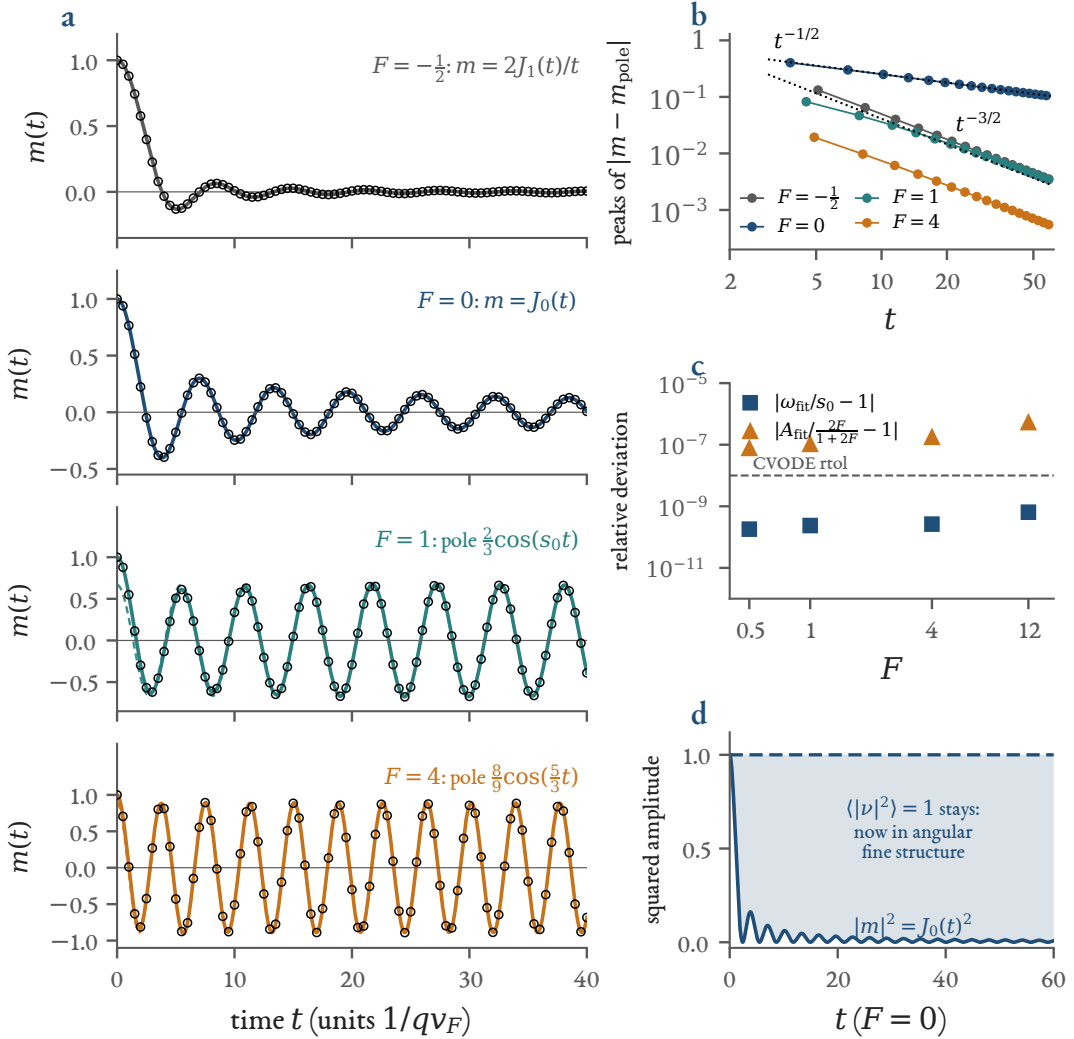


Figure 8.2: CVODE against the closed forms. Units $v_F q = 1$, BDF, $N = 64$, relative tolerance 10^{-8} .

a, the density amplitude $m(t)$ after a released density wave, for $F = -\frac{1}{2}$ (exactly $2J_1(t)/t$), $F = 0$ ($J_0(t)$), $F = 1$ and $F = 4$ (the sum (8.10) of an undamped line, dashed, and a band that dephases); open circles, CVODE every 0.5 time units; lines, closed forms. **b**, peaks of the part of m that is not the pole: the band's contribution decays algebraically, as $t^{-1/2}$ at $F = 0$ and $t^{-3/2}$ otherwise. **c**, deviation of the fitted late-time frequency from the Lean root s_0 (blue) and of the fitted amplitude from $2F/(1+2F)$ (orange), after the exactly known band part has been subtracted. **d**, $F = 0$: the squared density amplitude (solid) against the squared norm of the Fermi-circle distortion (dashed); the shading is the part of the distortion no longer visible in the density. Script: figures/ch08_timedomain.py.

did not look for the cause; the chapter's runs use BDF. *The crate, not the installed Python module.* The module `rusty_sundials` in the book's virtual environment (file dated 2026-09-26, the day before the repository's tight-tolerance fix) fails on this system at tolerances 10^{-8} and 10^{-10} ("too many error test failures at one step"), and its Adams option converges only as the square root of the tolerance, the signature of a first-order method (1.9×10^{-4} , 1.9×10^{-5} , 1.9×10^{-6} for $N = 32$ to $t = 0.5$); a build of commit 5db8041 passes the same test with either method (errors 1.9×10^{-7} to 1.4×10^{-10} for BDF, 1.7×10^{-7} to 1.7×10^{-10} for Adams). Everything in this chapter was computed with the Rust crates directly, at that commit. Last, $N = 96$ instead of 64 at $F = 4$ changes the result by 4.9×10^{-8} , within tolerance, with 6 000 steps against 5 963.

8.6 Landau damping is dephasing

At $F = 0$, Figure 8.2a shows a density that dies out, $m = J_0(t) \rightarrow 0$, in a system without any dissipation. Where did it go? The equation (8.1) conserves the quadratic form

$$E = \langle |\nu|^2 \rangle + F |\langle \nu \rangle|^2, \quad \frac{dE}{dt} = 0. \quad (8.11)$$

At $F = 0$ this is the squared norm of the Fermi-circle distortion, and $\langle |\nu|^2 \rangle = 1$ for all time (the solver keeps it to 1.1×10^{-7} up to $t = 60$) while $|m|^2 = J_0(t)^2$ has fallen to 0.0084 (Figure 8.2d). The amplitude of the distortion is intact; it has moved from the angular average, which the density sees, into angular structure of ever finer scale. Landau damping is dephasing, not dissipation: each direction oscillates at its own frequency $v_F q \cos \theta$, and the average over directions loses coherence.

Figure 8.3 shows it directly. It plots the Fermi circle itself, $r = 1 + \epsilon \operatorname{Re} \nu(\theta, t)$, at the place where the density wave has its maximum, for $F = 0$ and $F = 4$ (the profile on a fine angular grid is reconstructed from the CVODE solution by the exact Duhamel formula $\nu = e^{-ict} [\nu_0 - iFc \int_0^t e^{ict'} m(t') dt']$, $c = \cos \theta$, and agrees with the solver's node values to 7.3×10^{-8} and 3.7×10^{-5}). At $F = 0$ the ripples multiply in proportion to t , because the phase $t \cos \theta$ changes with θ at the rate $t \sin \theta$; near $\theta = 0$ and π it does not change, to first order. Those are the quasi-particles that move along the wave vector at the speed of the wave: they do not dephase, and they are the resonance of Section 8.2. At $F = 4$ the same filaments are present and, on top of them, a large smooth distortion of the circle, flattened at $\theta = 0, \pi$ and bulging at $\pm\pi/2$ at one moment and the reverse half a period later: zero sound, a collective mode of the whole circle, with a maximum $|\nu|$ of 3.40 at $t = 12$.

The decay law of what is left in the density comes from the edge of the continuum. The part of m that is not the pole is the cosine transform of a spectral density that vanishes like $(1 - y)^{1/2}$ at the edge $y = 1$ for every $F \neq 0$, and diverges like $(1 - y)^{-1/2}$ at $F = 0$, where the denominator in (8.10) vanishes at $y = 1$; the transforms decay as $t^{-3/2}$ and $t^{-1/2}$. The solver reproduces both: the fitted slopes of the envelope of the non-pole part over $10 \leq t \leq 60$ are -0.500 at $F = 0$ and -1.497 at $F = -\frac{1}{2}$ (Figure 8.2b), and -1.479 at $F = 4$; at $F = 1$, where the asymptotics sets in

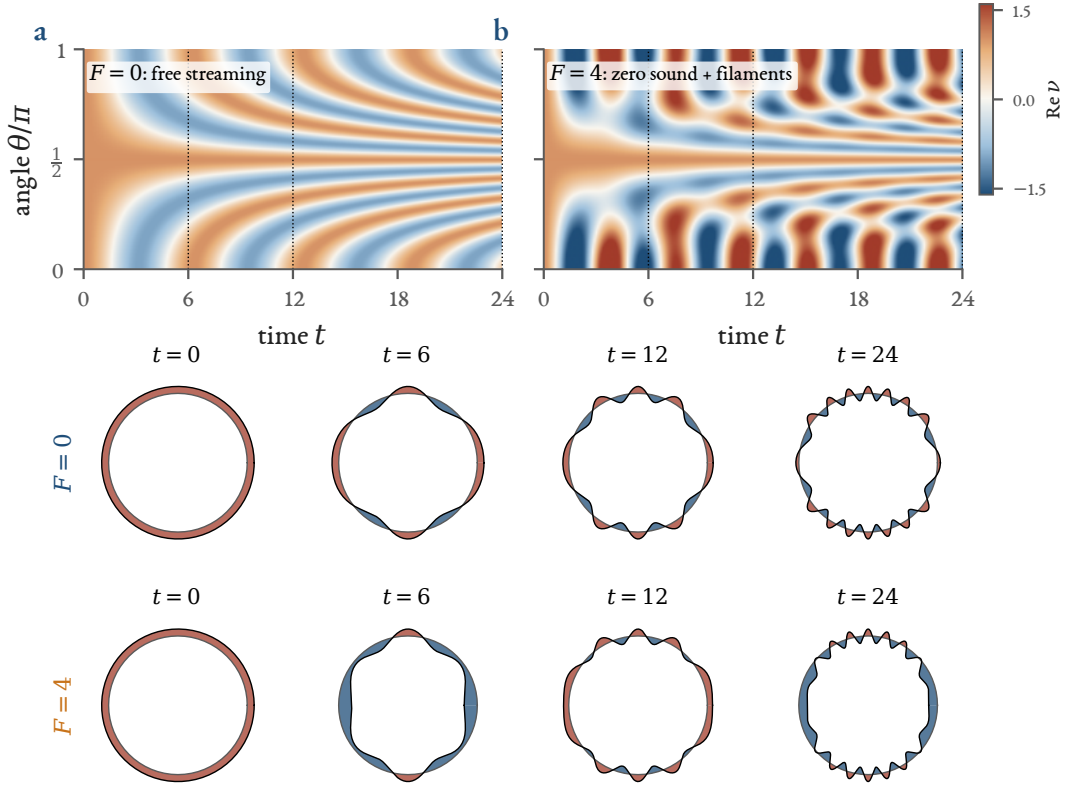


Figure 8.3: The Fermi circle as the dynamical variable. Top: $\text{Re } \nu(\theta, t)$ after a released density wave, $F = 0$ (free streaming: a fan of stripes, pitch shrinking as $1/t$ except at $\theta = 0, \pi$) and $F = 4$ (zero sound at $s_0 = 5/3$: large smooth lobes near $\theta = 0$ and π on top of the same fan); the colour scale saturates at ± 1.6 . Below: the circle $r = 1 + \epsilon \text{Re } \nu$, $\epsilon = 0.10$ (deformation magnified, the same ϵ in every panel; red outward, blue inward) at $t = 0, 6, 12, 24$; the wave vector points to the right. At $F = 0$ the circle keeps $\langle |\nu|^2 \rangle = 1$ while the density average decays: the amplitude has moved into the ripples. Script: `figures/ch08_fermisurface.py`.

later, it is -1.350 in this window.

The library holds the field-free core of this picture. The module `PhaseMixing` proves, with no regularity assumed, that the k -th density mode of free transport $e^{ik(x-vt)}g(v)$ tends to zero (`QuantumFluids.PhaseMixing.phase_mixing`, an application of the Riemann–Lebesgue lemma), and that for the unit Maxwellian it is exactly $e^{-k^2 t^2/2}$ (`QuantumFluids.PhaseMixing.maxwellian_mode`, quoted in Section 8.7); for the circle the analogue is J_0 , which is not in the library. That module is subsumed by Bedrossian’s Lean formalisation of the nonlinear Mouhot–Villani theorem [88, 89] and claims nothing beyond the linear statement.

Finally, (8.11) is the energy of the Pomeranchuk bound. Because $|\langle \nu \rangle|^2 \leq \langle |\nu|^2 \rangle$, one has $E \geq \min(1, 1 + F) \langle |\nu|^2 \rangle$: for $F > -1$ the form is positive definite and every solution of the single-wave model stays bounded. Below the bound it is indefinite and the model has a growing solution:

with $s = iy$, (8.2) reads $1 + F(1 - y/\sqrt{1 + y^2}) = 0$, solved by $y = r/\sqrt{1 - r^2}$, $r = 1 - 1/|F|$, once $F < -1$; at $F = -2$, $y = 1/\sqrt{3} = 0.577350$. CVODE at $F = -2$ grows as e^{yt} with the fitted rate 0.577350 over $10 \leq t \leq 14$, E conserved to 3.2×10^{-6} . This is a simulation of an elementary instability, not a theorem: the library contains no stability theorem for the Landau equation (Section 8.9, item 3).

8.7 A plasma cousin, and the property that decides

Equation (8.1) resembles the Vlasov equation of a plasma, and the rusty-SUNDIALS repository has a solver for one: the crate `qf-vlasov1d`, one space and one velocity dimension, electrons on a neutralising background (units of the plasma frequency),

$$\partial_t f + v \partial_x f - E \partial_v f = 0, \quad \partial_x E = 1 - \int f dv. \quad (8.12)$$

It advances x by an exact Fourier shift and v by a cubic semi-Lagrangian step (a natural spline, a documented departure from its Python original), in Strang splitting, on 32×512 cells with $|v| \leq 6$ and time step 0.1. Two backgrounds $f_0(v)$ of equal density are used: a Maxwellian, and a degenerate Fermi–Dirac profile $f_0 \propto [1 + e^{(|v|-v_0)/w}]^{-1}$ with $v_0 = 1$, $w = 0.1$, a sea with two smeared edges that is the one-dimensional Fermi surface. The Maxwellian is perturbed in density, $f_0(1 + a \cos kx)$; the degenerate sea by displacing its edges, $v_0 \rightarrow v_0(1 + a \cos kx)$, so that the perturbation lives where a Fermi gas can have one.

RUSTY-SUNDIALS QF-VLASOVID ON TWO BACKGROUNDS

From the driver book/rust/ch08_vlasov: the two backgrounds on the velocity grid, and the evolution (excerpt).

```
Bg::Maxwell => maxwellian(v, 0.0),
Bg::Fermi { v0, w } => 1.0 / (1.0 + ((v.abs() - v0) / w).exp()),
...
let g = Grid::new(2.0 * PI / k, 32, nv, 6.0);
let f0 = initial(&g, bg, a);
run(f0, &g, dt, t_end, observe, field_on,
    Vec::<(f64, fn(&[f64]) -> Vec<f64>)>::new());
```

The linear theory is a dispersion relation of the type (8.2): $1 - k^{-2} \int f'_0(v) (v - \omega/k)^{-1} dv = 0$ along Landau's contour. For the Maxwellian it is the textbook relation with the plasma dispersion function; at $k = \frac{1}{2}$ it has the root $\omega = 1.415661889 - 0.153359467i$ (our mpmath calculation), which agrees with the programme's certified root of its earlier kinetic benchmark ($1.415661889 - 0.153359467i$ to the digits printed) to 4.4×10^{-16} . Figure 8.4 shows both runs. With the Maxwellian the density perturbation is sheared into filaments of ever finer velocity structure,

Table 8.3: qf-vlasov1d against linear theory (k in inverse Debye lengths; $n_v = 512$): fits of the signed density mode over $5 \leq t \leq 40$ (Maxwellian: the window ends when the mode falls below 3×10^{-5} of its maximum) and the degenerate profile $v_0 = 1, w = 0.1$. Degenerate theory: continued root at $k = 0.5, 0.8$, weak-damping formula at $k = 0.3$. Rates below about 10^{-4} are at the resolution of the fit (residual 3.6×10^{-3} of the signal at $k = 0.5$).

	k	ω_{theory}	ω_{solver}	$ \gamma _{\text{theory}}$	$ \gamma _{\text{solver}}$	rel. diff. γ
Maxwellian	0.3	1.1598	1.1597	0.0126	0.0127	0.78 %
	0.5	1.4157	1.4156	0.1534	0.1535	0.09 %
	0.8	1.7999	1.8003	0.5346	0.5345	0.02 %
degenerate	0.3	1.0489	1.0492	1.2×10^{-9}	2.6×10^{-6}	—
	0.5	1.1337	1.1339	8.6×10^{-5}	5.9×10^{-5}	—
	0.8	1.3342	1.3341	1.1×10^{-2}	1.1×10^{-2}	0.70 %

pitch $2\pi/(kt)$, and the field decays as $e^{\gamma t}$ with $\gamma = -0.1534$. With the degenerate sea the same density wave rides on the two Fermi edges and does not decay. Least-squares fits of the solver's signed density mode over $5 \leq t \leq 40$ (amplitude, rate, frequency and phase free) give Table 8.3: for the Maxwellian the rates agree with the roots within 0.78 % at $k = 0.3$, where the mode has lost only 40 % of its amplitude by $t = 40$, and to better than 0.09 % from $k = \frac{1}{2}$ up, with frequencies within 0.036 %; going from $n_v = 128$ to 256 to 512 moves the fitted rates by at most 1.2×10^{-6} .

The degenerate background tells the other half. Its roots are $\omega = 1.0489, 1.1337, 1.3342$ at $k = 0.3, 0.5, 0.8$, with damping rates $1.2 \times 10^{-9}, 8.6 \times 10^{-5}$ and 1.1×10^{-2} : exponentially small at small k because the phase velocity ω/k lies several widths w beyond the edge v_0 , where the resonant fraction is $\sim e^{-(\omega/k-v_0)/w}$. The solver's frequencies are 1.0492, 1.1339, 1.3341, and its damping rate at $k = 0.8$, large enough to measure, is 1.1×10^{-2} against 1.1×10^{-2} . At $k = 0.3$ and 0.5 the true rates are below the resolution of the fit; what the solver supports is $|\gamma| < 10^{-4}$, three orders of magnitude below the Maxwellian 0.1534 at $k = \frac{1}{2}$. The field-off control is the Lean statement in numbers: with the field switched off, the density mode of the Maxwellian run is $e^{-k^2 t^2/2}$ to 3.7×10^{-9} over $0 \leq t \leq 40$, down to a floor of 1.9×10^{-10} set by the cut at $|v| = 6$, and the free-streaming recurrence, an artefact of any velocity grid at $t = 2\pi/(k\Delta v) = 536$, is far outside the window.

LEAN 4 PHASEMIXING

```
theorem maxwellian_mode (k t : ℝ) :
  ∫ v : ℝ, cexp (-(k * v * t : ℝ) : ℂ) * I) *
    ((rexp (-v ^ 2 / 2) / √(2 * π) : ℝ) : ℂ)
  = cexp (-(k * t) ^ 2 / 2 : ℝ))
```

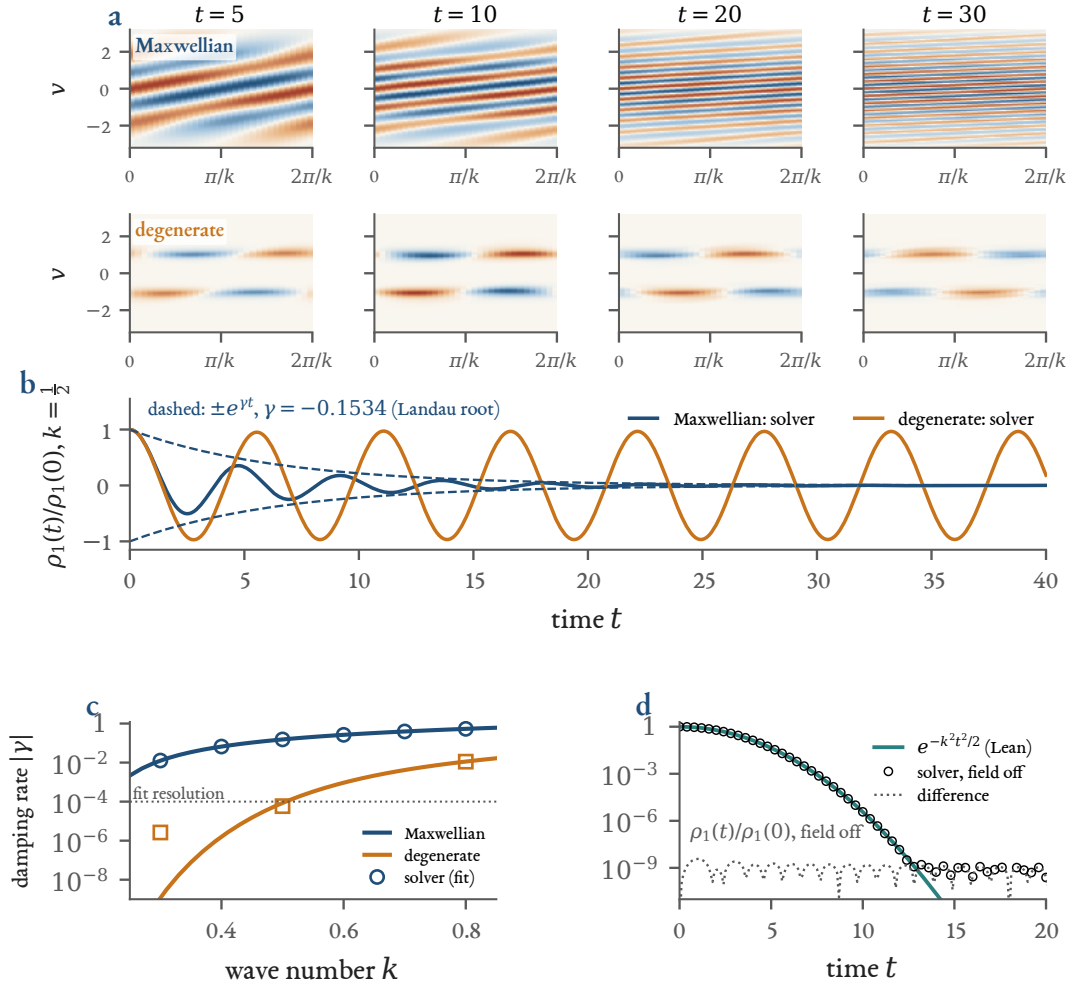


Figure 8.4: Vlasov–Poisson (qf-vlasov1d): the same density wave on two backgrounds. **a**, the perturbation $f(x, v, t) - \bar{f}_0(v)$ at $k = \frac{1}{2}$, $a = 0.05$, $t = 5, 10, 20, 30$ (colour scale per row): for the Maxwellian (top) filaments of velocity pitch $2\pi/kt$; for the degenerate sea (bottom) a pair of edge waves. **b**, the density mode $\rho_1(t)/\rho_1(0)$ at $k = \frac{1}{2}$, $a = 0.01$: damped (blue; dashed, the envelope $\pm e^{\gamma t}$ with the Landau root) and undamped (orange). **c**, damping rate against k : linear theory (lines) and solver fits (symbols); the degenerate curve is the weak-damping formula, and solver rates below about 10^{-4} are at the resolution of the fit. **d**, the field-off control: the density mode of free streaming (circles) against $e^{-k^2 t^2/2}$ (line; Lean: QuantumFluids.PhaseMixing.maxwellian_mode); dotted, the difference, at the floor set by cutting the Maxwellian at $|v| = 6$. Script: figures/ch08_vlasov.py.

AN ANALOGY IS NOT A RESULT

The finding is “a collective mode of a collisionless kinetic equation is damped by phase mixing”. The tempting transfer: the plasma shows Landau damping, so Fermi-liquid zero sound is Landau damped. What the finding depends on is not the form of the equation, which the two systems share, but the *support* of the velocity distribution relative to the phase velocity of the wave. A Maxwellian has unbounded support, so some particles always move at the phase velocity and the mode is damped at a rate fixed by $f'_0(\omega/k)$; a degenerate gas has bounded support, and a wave faster than the edge has no resonant particles. The degenerate run shows precisely this, and a Fermi circle has the property: $s > 1$ in (8.2) is its statement. What does *not* transfer is any number: the plasma’s interaction is a long-range force of strength $1/k^2$ where the Fermi liquid has the contact parameter F , the roots solve different equations, and the rate 0.1534 is a fact about electrons on a Maxwellian background and nothing about helium.

8.8 Beyond Landau: the window at twice the Fermi momentum

What does the model say about the third regime of Section 8.1? Nothing, and it is worth seeing why. The model’s collective mode is the straight line $\omega = s_0 v_F q$, derived for $q \ll k_F$ with a single parameter F and no scale at which a minimum could form. Against the free-gas band of Figure 8.5a the line enters the edge $v_F q + \hbar q^2/2m$ at $q_c = 2k_F(s_0 - 1)$, which is $0.121 k_F$ at $F = \frac{1}{2}$, $0.309 k_F$ at $F = 1$, $1.333 k_F$ at $F = 4$ and $3.200 k_F$ at $F = 12$ (for $F \geq 3 + 2\sqrt{3} = 6.464$ it stays above the edge up to $q = 2k_F$); beyond q_c the model’s answer is a broadened line. That is the second regime of the abstracts, in outline: the zero-sound mode enters the band and is damped. It is not a quantitative claim about the film, whose F , effective mass and k_F are not known here; and for the larger values of F the entry point is at $q_c \gtrsim k_F$, outside the range $q \ll k_F$ for which the straight line was derived.

The one exact statement about the third regime that survives in this framework is kinematic and holds for the free gas: the band has a lower edge, and the edge lifts off $\omega = 0$ at $q = 2k_F$.

LEAN 4 CHO8_ZERO_Sound2D (NEW IN THIS BOOK)

```

theorem ph_gap {E : Type*} [NormedAddCommGroup E]
  [InnerProductSpace ℝ E] {m kF : ℝ} (hm : 0 < m)
  (hkF : 0 ≤ kF) (k q : E) (hk : ||k|| ≤ kF)
  (hq : 2 * kF < ||q||) :
  0 < ||q|| * (||q|| - 2 * kF) / (2 * m) ∧
  ||q|| * (||q|| - 2 * kF) / (2 * m)
  ≤ (||k + q|| ^ 2 - ||k|| ^ 2) / (2 * m)

```

A hole of momentum k inside the Fermi sphere ($\|k\| \leq k_F$) and a particle of momentum $k + q$ cost the energy $(\|k + q\|^2 - \|k\|^2)/2m$ (with $\hbar = 1$). The gap theorem says that for $q > 2k_F$ every such pair costs at least $q(q - 2k_F)/2m > 0$. It rests on `QuantumFluids.ZeroSound2D.ph_`

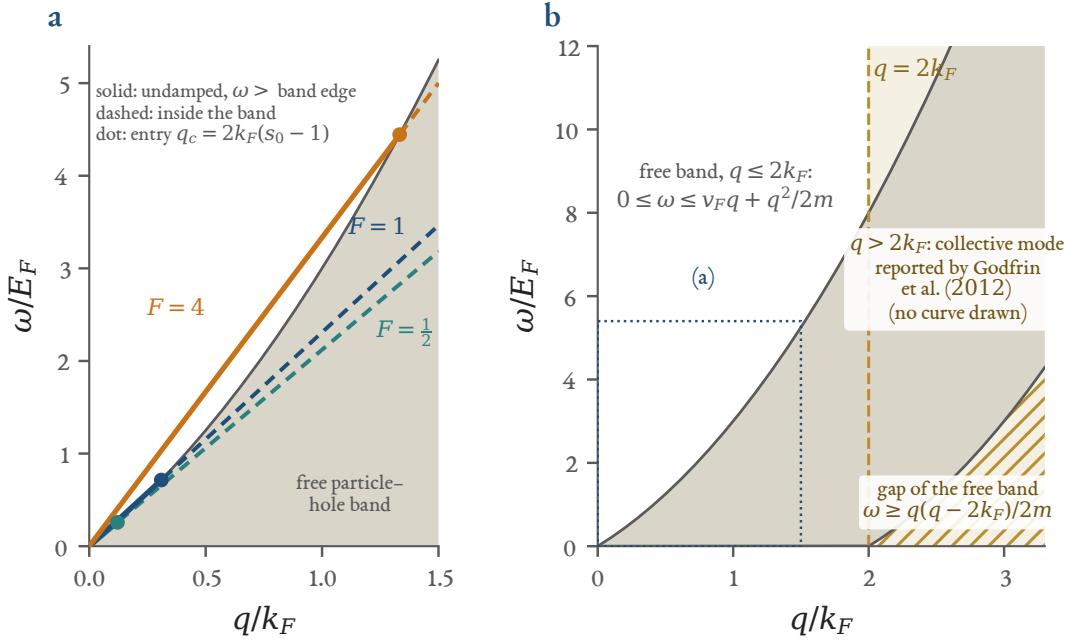


Figure 8.5: The (q, ω) plane, in free-gas units (q/k_F , ω/E_F with $E_F = \hbar^2 k_F^2/2m$, so that $v_F q = 2(q/k_F)E_F$). **a**, the Landau line $\omega = s_0 v_F q$ for $F = \frac{1}{2}, 1, 4$ against the upper edge $v_F q + \hbar q^2/2m$ of the free particle-hole band (grey): solid where the line is above the edge (undamped), dashed once inside (damped), dot at the entry $q_c = 2k_F(s_0 - 1)$. **b**, the whole free band up to $3.3 k_F$: its lower edge is 0 for $q \leq 2k_F$ and $\hbar q(q - 2k_F)/2m$ for $q > 2k_F$ (hatched, the gap; Lean: `QuantumFluids.ZeroSound2D.ph_gap`); gold, the momentum transfers $q > 2k_F$ at which Godfrin *et al.* report the reappearing collective mode. No curve is drawn there. Script: `figures/ch08_window.py`.

energy_window, which puts the cost between $(q^2 - 2k_F q)/2m$ and $(q^2 + 2k_F q)/2m$ in *any* real inner-product space, hence for the film and for the bulk liquid alike; the proof is the Cauchy–Schwarz inequality $|\langle k, q \rangle| \leq k_F q$ and nothing else. And `QuantumFluids.ZeroSound2D.ph_edge_attained` says the lower bound is reached, by the hole $k = -(k_F/\|q\|)q$ on the Fermi surface, whose partner $k + q$ lies outside it as soon as $\|q\| \geq 2k_F$. For $q < 2k_F$ the lower edge is zero, because the bound is negative and a pair at the Fermi surface can cost arbitrarily little.

This is the mathematics of the number $2k_F$ in the 2012 abstract: the collective mode is said to reappear at momentum transfers larger than twice the Fermi momentum, and $2k_F$ is where the free band opens a window below itself. The coincidence is noted; it is not claimed as the explanation. The abstracts say that the reappearing excitation lies “beyond the particle-hole band”, with a dispersion “quite similar to that of superfluid ^4He ”, and that a dynamic many-body theory interprets the spectra. Whether the observed mode sits in the free band’s window, how the interaction pushes it, and why it has a minimum are exactly what a theory of pair fluctuations beyond the Landau parameter must answer, and what this book’s tools do not reach; the Godfrin–Krotscheck survey [5] is the place to

read how the question is treated. In the helium-4 chapters the roton is a feature of a measured curve (Chapter 5); here it is a feature that a measurement found in a system where the standard theory did not expect it.

8.9 What is established, and what is not

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved (Lean, standard axioms only). The library: `QuantumFluids.ZeroSound.zero_sound_2d_iff` and `QuantumFluids.ZeroSound.zero_sound_iff` (an undamped zero-sound root $s > 1$ exists iff $F > 0$; 2D with the explicit root, 3D by pinning and the intermediate value theorem) and `QuantumFluids.PhaseMixing.maxwellian_mode`. New in this book, in `book/lean/Ch08_ZeroSound2D.lean` (namespace `QuantumFluids.ZeroSound2D`): uniqueness of the 2D root, the derivative of Ω_{2D} , the residue weight and its share of the first moment, and the free-gas particle–hole window. The file compiles with the pinned Lean 4 and Mathlib with no sorry; `#print axioms` reports `propext`, `Classical.choice`, `Quot.sound` for each of its nine theorems. These are statements about mathematics; that a helium film obeys them is physics.

Derived and checked numerically, not in Lean. The structure factor (8.6); the residue as the strength of a δ -function; the f -sum rule (8.8) (the quadrature closes it to 6.3×10^{-15}); the exact solutions J_0 and $2J_1(t)/t$ and the formula (8.10); the conserved form (8.11).

Simulated. CVODE (BDF) on the kinetic equation (errors 4.0×10^{-9} to 1.0×10^{-6} against the closed forms) and on the root continuation; `qf-vlasov1d`.

Measured, by others. Everything about helium, by Godfrin and his collaborators [1, 87], quoted from their abstracts. This book has no spectra and no film parameters; every number here is a function of the free F .

Only an analogy. Plasma against Fermi liquid. What they share (free streaming plus a mean field, dephasing) is real, and the degenerate run confirms the property that decides whether a mode is damped (bounded support); the plasma's numbers say nothing about helium.

Not claimed, and not possible here. The roton; any finite- q or finite-temperature statement; F_1^s and the effective mass; collisions and the crossover from zero to first sound; a comparison with the film. Past the line the model draws, the chapter says only what is free-gas kinematics.

What failed or was withdrawn. (1) The installed Python module of CVODE failed on this problem, and the Adams option of the crate gave wrong answers at one output spacing (Section 8.5); reported, not investigated. (2) A first estimate of the Vlasov damping rates from the spacing of field maxima returned a positive rate for the weakly damped degenerate run ($+8.3 \times 10^{-5}$ at $k = 0.5$); it was replaced by least-squares fits, and rates below about 10^{-4} are not claimed. (3) The programme's attempt to formalise a Pomeranchuk–Penrose stability

theorem for the 2D Landau equation was stopped at its literature gate (CLAIM-030); the library has none. (4) The programme's two searches for the 2D Landau parameters of the film found no open table (CLAIM-030, CLAIM-031); the ledger names the 2010 paper as a likely source, and only its abstract, which gives no parameter, was read for this chapter.

Exercises

Exercise 1. Show that squaring $1 + F \Omega_{2D}(s) = 0$ gives $s^2(1 + 2F) = (1 + F)^2$, and that the positive root solves the unsquared equation exactly when $F > 0$. Expand s_0 for small F to order F^3 and verify $s_0^2 = \frac{F}{2} + \frac{3}{4} + \frac{1}{4(1+2F)}$. *Hint.* $(1 + F)^2 - F^2 = 1 + 2F$; the unsquared equation needs $Fs/(1 + F) > 0$, which is where the sign of F enters. Answer: $s_0 - 1 = F^2/2 - F^3 + O(F^4)$.

Exercise 2. Derive (8.7) from $\Omega'(s) = (s^2 - 1)^{-3/2}$ and (8.4), then (8.9). For which F does the zero-sound line carry 90 % of the first moment? Show also that (8.8) follows from $G(s) \simeq -1/(2s^2)$ at large $|s|$. *Hint.* $s_0 W = F(1 + F)/(1 + 2F)^2$ and $4F(1 + F) = (1 + 2F)^2 - 1$; the first answer is $(1 + 2F)^2 = 10$, i.e. $F = (\sqrt{10} - 1)/2 = 1.081$.

Exercise 3. Prove that $E = \langle |\nu|^2 \rangle + F \langle \nu \rangle^2$ is constant along solutions of (8.1) for a single wave vector, and that for $F > -1$ every solution is bounded. Then find the growth rate for $F < -1$ and compare it with the CVODE run at $F = -2$ of Section 8.6. *Hint.* Differentiate both terms: only the cross terms $\pm 2F \operatorname{Re}[i m^* \langle \cos \theta \nu \rangle]$ survive, and they cancel; $\langle \cos \theta \rangle = 0$ removes the Fm term from $\dot{m}$. For the rate put $s = iy$ and use $\langle (\cos^2 \theta + y^2)^{-1} \rangle = 1/(y\sqrt{1 + y^2})$.

Notes and further reading

Landau's theory of the Fermi liquid is in his 1957 paper [25]; its textbook presentation, with the kinetic equation, the Landau parameters and zero sound, is Baym and Pethick [14]. Zero sound was observed in liquid ^3He by Abel, Anderson and Wheatley in 1966 [90]. The measurements this chapter is organised around are [1, 87], and the Godfrin–Krotscheck survey of excitation spectra of quantum fluids [5] puts them in context. Landau damping as a theorem (nonlinear, Vlasov–Poisson) is due to Mouhot and Villani [88], its Lean 4 formalisation to Bedrossian [89]; the library module used here, `PhaseMixing`, is only the linear field-free core [12]. The solver is the CVODE of SUNDIALS [4] as ported to Rust in `rusty-SUNDIALS` (commit 5db8041); the proof assistant is Lean 4 with `Mathlib` [11, 13].

Flow Past an Obstacle: Reproducing a Published Experiment

9.1 A hole in the fluid

At $t = 24.2 \tau$ the density of the fluid at one point of the axis, thirteen healing lengths downstream of the centre of the obstacle, is 9×10^{-5} of its bulk value. One tenth of a time unit later there are two vortices at that point, of opposite circulation, half a healing length apart. Nothing in the first twenty-four time units looked like dissipation: the flow was a smooth potential flow at 0.55 of the speed of sound, the force on the obstacle had risen quietly to $7.1 \mu/\xi$, and no vortex had been counted. These numbers do not come from a laboratory. They come from a numerical experiment, the example run that Junhwan Kwon and Yong-il Shin deposited with their paper on vortex shedding behind a penetrable obstacle in a two-dimensional superfluid [27, 28], and which we have reproduced, with an independent scheme, in the Rust engine of rusty-SUNDIALS. This chapter is about what that sentence means, what it cost, and what it leaves open.

Two remarks before anything else. First, *experiment* means a computer experiment here. Laboratory experiments of this kind exist: a laser beam moved through a Bose–Einstein condensate gave an early measurement of a critical velocity [91], the periodic shedding of vortex dipoles from a moving penetrable obstacle [92] and a von Kármán vortex street [93]; we reproduce none of them. Second, nothing in this chapter reproduces or tests a measurement of Henri Godfrin, whose experiments are neutron-scattering measurements on helium (Chapters 5 and 8). The chapter belongs in a book dedicated to his work for another reason: it is where the book’s three instruments are turned on somebody else’s published result, and where the question “do we agree?” must be answered with numbers, the disagreements stated as plainly as the agreements, much as one checks an instrument against another laboratory’s published number.

The three instruments appear in the following roles. The *experiment* is the deposited run, the data we try to match: the wave function at six times, the force on the obstacle every tenth of a time unit, the output of the authors’ vortex detector every five time units. The *proof assistant* is used for what it is good at, the small exact statements on which a comparison silently leans: what Landau’s criterion says about the uniform fluid, when a flow is locally supersonic, why the reference’s velocity ramp is

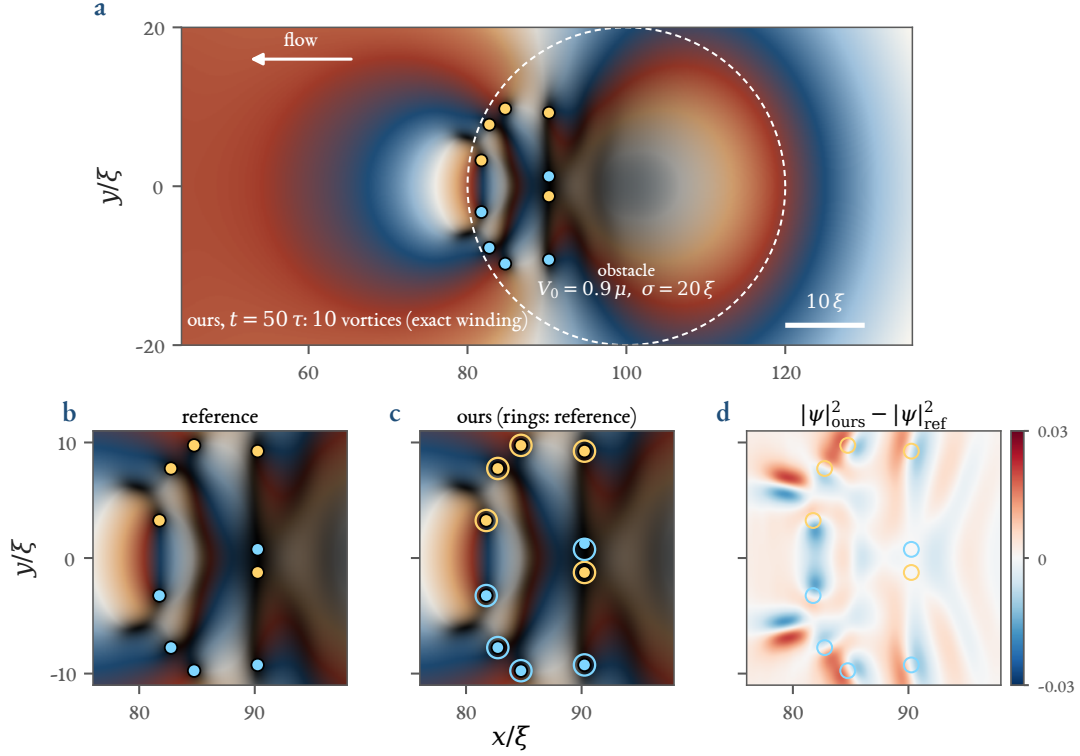


Figure 9.1: The wake at $t = 50 \tau$. Hue is the phase of ψ , brightness its modulus (a hole in the fluid is black); the dots are plaquettes of non-zero winding (counter-clockwise gold, clockwise cyan; Section 9.5). **a**, our field; the dashed circle is the $1/e^2$ radius of the obstacle and the fluid streams from right to left. **b**, **c**, the vortex region of the deposited field and of ours at eight times the grid resolution (spectral interpolation); the rings in **c** are the deposited vortices. **d**, $|\psi|_{\text{ours}}^2 - |\psi|_{\text{ref}}^2$ at most 0.0205 on the grid. Script figures/ch09_wake.py.

off by exactly 0.00275ξ , what makes a vortex count an integer. The *solver* is `qf-pgpe::flow`, the obstacle-flow module of the Rust engine, which integrates the same equation as the reference with a different method, in a different precision, on a different machine.

Figure 9.1 is the picture to keep in mind. The broad lobes of colour are the phase of the potential flow around the obstacle. In panel **a** the ten dots are vortices, five pairs, each vortex the mirror image of an antivortex across the axis of the flow. The dark arcs through them are density troughs, lines on which the density is far below its bulk value. The deposited field and ours cannot be told apart at this magnification; panel **d** shows where they differ.

9.2 The deposited run and the model

The archive. The archive that accompanies the paper (Zenodo, version 1.0.0, 2026; code under the MIT licence, processed data to be reused with attribution under CC BY 4.0, as its README says

[28]) holds the processed data behind the paper’s summary figures (drag, lift and Strouhal number against a superfluid Reynolds number), the authors’ GPU code (CuPy) with its configuration file, and one raw output case, for an obstacle of height $V_0 = 0.9 \mu$ and width $\sigma = 20 \xi$ in a flow of 0.55 times the speed of sound. For that case it gives the force on the obstacle every 0.1τ and the output of the code’s vortex detector every 5τ , both up to $t = 100 \tau$, and snapshots of the wave function every 10τ from $t = 0$ to $t = 50 \tau$. The README says what is not there: the full raw trajectories behind the paper’s averages.

What we reproduce, and what we do not. We reproduce the one raw case. We do not reproduce the paper’s results, which rest on many parameter sets, each averaged over a window $2000 \leq t \leq 10000 \tau$ [27], and on a post-processing (time-averaged irrotational flow, Mach-1 contour, effective diameter, Fourier analysis of the force) that we did not implement. Its central claims, a transition from periodic emission of vortex dipoles to the shedding of alternating same-sign vortex clusters at a superfluid Reynolds number $\text{Re}_s \approx 2$ and the collapse of the Strouhal number and of the drag coefficient onto universal curves against Re_s , are quoted as the authors’ and not as ours. For the obstacle of the deposited case the paper reports a critical velocity $v_c \approx 0.18$ for dipole emission and a transition to cluster shedding at $v_{\text{th}} \approx 0.33$ (in units of the speed of sound; every number of the paper quoted in this chapter is that of its arXiv version 1); the deposited run, at 0.55, lies well above both.

The model. In the units in which $\hbar = m = \mu = g = 1$, the unit of length is the healing length $\xi = \hbar / \sqrt{m\mu}$, the unit of time is $\tau = \hbar / \mu$, the speed of sound of the uniform fluid is $c = \sqrt{\mu/m} = 1$, the bulk density is 1, and forces are measured in μ/ξ . The deposited code integrates

$$\partial_t \psi = v(t) \partial_x \psi - (i + \Gamma(x, y)) \left[-\frac{1}{2} \nabla^2 + |\psi|^2 - 1 \right] \psi - i V(x, y) \psi, \quad (9.1)$$

$$V = V_0 \exp \left[-\frac{2((x - x_{\text{obs}})^2 + y^2)}{\sigma^2} \right], \quad v(t) = v_f \min(t/t_{\text{acc}}, 1), \quad (9.2)$$

with $V_0 = 0.9$, $\sigma = 20$, $x_{\text{obs}} = 100$, $v_f = 0.55$ and $t_{\text{acc}} = 0.1$, on a periodic grid of 1000×500 points of spacing 0.5ξ that covers $[-250, 250) \times [-125, 125) \xi$, with the time step $\Delta t = 0.01 \tau$. Read it term by term. Without v , Γ and V it is the Gross–Pitaevskii equation of Chapter 3 with the chemical potential subtracted. The term $v \partial_x \psi$ moves to the frame of the obstacle: far away the fluid is at rest in the laboratory, the obstacle moves along $+x$ at speed v , and in the frame of the obstacle the fluid streams towards $-x$. The phase θ of ψ is the laboratory phase, so the velocity relative to the obstacle is $\mathbf{u} = \nabla \theta - v \mathbf{e}_x$. The box is $500 \xi \times 250 \xi$ and the obstacle sits at $x = 100 \xi$, so that the wake has 350ξ to run in. The damping function Γ is 2.4×10^{-6} at the centre of the obstacle and at most 1.7×10^{-5} within its $1/e^2$ radius, and rises to $\Gamma_0 = 0.1$ across layers that begin 50ξ and 40ξ from the edges of the box; in the layers the factor $(i + \Gamma)$ turns the Hamiltonian evolution into a damped one, which absorbs the sound that the obstacle radiates before it can return through

the periodic boundaries. The obstacle enters only through $-iV\psi$: the deposited code treats V as unitary and does not multiply it by the damping. The obstacle is *penetrable* because $V_0 < \mu$: at rest, the Thomas–Fermi density at its centre is $1 - V_0 = 0.1$ of the bulk, not zero.

Equation (9.1) is what we recovered from the deposited code. The paper’s own equation (1) agrees with it in its Hamiltonian part, and describes the damping layer (the “fringe region”) with other numbers than the code does ($w_x = 25 \xi$, $w_y = 20 \xi$ and a transition scale $d = 3 \xi$, against layers of 50ξ and 40ξ and a scale of 15ξ in the code). We follow the code, because it is what produced the deposited run, and we have not tried to find out whether the two descriptions are equivalent in some convention.

Two schemes. The deposited code advances (9.1) by a splitting of Strang type: half a step of the linear part (kinetic term, moving-frame term, damping of the kinetic term) by a fourth-order Runge–Kutta method, a full step of the local part (nonlinearity, potential, constant offset) integrated exactly, and a second half step of the linear part, in single precision (complex64) on a GPU, with $\Delta t = 0.01 \tau$. Our scheme integrates the whole right-hand side of (9.1) with one explicit fourth-order Runge–Kutta step, evaluates the derivatives spectrally, works in double precision, and uses the same Δt ; we start it from the deposited field at $t = 0$. The two computations therefore share the model and the initial data, and nothing else: not the scheme, not the precision, not the order of operations, not the machine. If they agree, the agreement is about the model and the data, not about a shared discretisation.

The force. The force on the obstacle is the deposited code’s

$$F_x = \iint \partial_x V |\psi|^2 dx dy, \quad (9.3)$$

taken over the interior of the damping layers ($|x| < 200 \xi$, $|y| < 85 \xi$), with central differences for $\partial_x V$ and the trapezoidal rule, exactly as the code does. It is negative, the fluid pushing the obstacle along the flow, and we plot the drag $D = -F_x$. (The paper writes the force on the fluid, which is the opposite.) The lift F_y is the same integral with $\partial_y V$.

9.3 Why a subsonic flow can dissipate

Landau’s criterion. An object of mass M moves at speed v through a fluid at rest. It can slow down only by creating an excitation of momentum $\hbar k$ and energy $\varepsilon(k)$, and conservation of energy and momentum then allows this only if

$$v \geq \frac{\varepsilon(k)}{\hbar k} \quad \text{for some } k, \quad \text{i.e.} \quad v \geq v_L := \min_k \frac{\varepsilon(k)}{\hbar k} \quad (9.4)$$

(for M large against the excitation). Below v_L the object cannot dissipate energy into the fluid [3]. The criterion needs one input, the spectrum $\varepsilon(k)$ of the elementary excitations of the fluid at rest.

GODFRIN THE SPECTRUM BEHIND THE CRITERION

For superfluid ^4He that spectrum is the dispersion curve of the Landau elementary excitations (phonon, maxon, roton) that Godfrin and his collaborators measured by inelastic neutron scattering at the ILL spectrometer IN5, and whose thermodynamic consequences they drew [2]. In the review he wrote with Krotscheck, Landau’s argument takes one paragraph [5]: the creation of excitations by an object displaced in the fluid is impossible unless its velocity exceeds a critical value, and “The linearity of the dispersion relation at low wave vectors is essential for this to happen”; in a Bose gas, whose dispersion is parabolic, the critical velocity is zero. These are statements about a curve that was *measured*. The two lemmas below prove the inequalities behind them; they know nothing about helium, and nothing about the curve.

LEAN 4 HELIUMKINEMATICS

```

theorem landau_velocity_parabolic_zero {a : ℝ} (ha : 0 < a)
  {v : ℝ} (hv : 0 < v) :
  ∃ k : ℝ, 0 < k ∧ a * k ^ 2 / k < v := by
  refine ⟨v / (2 * a), by positivity, ?_⟩
  -- (proof abridged: three more lines)

theorem landau_velocity_ge_of_above_sound_line {ε : ℝ → ℝ}
  {c k : ℝ} (hk : 0 < k) (h : c * k ≤ ε k) : c ≤ ε k / k := by
  rw [le_div_iff hk]; exact h

```

In words: for a parabolic spectrum $\varepsilon = ak^2$ the ratio $\varepsilon/k = ak$ falls below any $v > 0$ (take $k = v/2a$), so $v_L = 0$; a spectrum on or above the sound line ck has $\varepsilon/k \geq c$ at every k . They are `QuantumFluids.HeliumKinematics.landau_velocity_parabolic_zero` and `QuantumFluids.HeliumKinematics.landau_velocity_ge_of_above_sound_line`, formalised from the review’s paragraph, and they are theorems about real numbers, not about the experiment.

The uniform Gross–Pitaevskii fluid. For the model (9.1) the excitations of the uniform fluid are Bogoliubov’s [50]: in our units $\varepsilon(k)^2 = \frac{k^2}{2}(\frac{k^2}{2} + 2)$, that is $\varepsilon(k) = k\sqrt{1 + k^2/4}$ (see Chapter 5). The following two statements are new Lean, written for this chapter (file `book/lean/Ch09_FlowPast.lean`, namespace `QuantumFluids.FlowPast`; they are not part of the library). Compiled with Lean 4.34.0-rc2 and the matching Mathlib; no sorry; axioms `propext`, `Classical.choice`, `Quot.sound`.

LEAN 4 FLOWPAST (NEW FOR THIS BOOK)

```

noncomputable def bogEps (k : ℝ) : ℝ := k * Real.sqrt (1 + k ^ 2 / 4)

theorem bogEps_ge_sound {k : ℝ} (hk : 0 ≤ k) : 1 * k ≤ bogEps k

theorem landau_uniform_gp :
  (∀ k : ℝ, 0 < k → 1 ≤ bogEps k / k) ∧
  (∀ δ : ℝ, 0 < δ → ∃ k : ℝ, 0 < k ∧ bogEps k / k < 1 + δ)

```

The Landau velocity of the uniform Gross–Pitaevskii fluid is exactly the speed of sound (QuantumFluids.FlowPast.landau_uniform_gp): the ratio ε/k never falls below 1, and it comes within any δ of 1, at $k = \sqrt{2\delta}$, where it equals $\sqrt{1 + \delta/2}$ (proof abridged: the first clause is $\sqrt{1 + k^2/4} \geq 1$, the second is that identity and $\sqrt{1 + \delta/2} < 1 + \delta$). The infimum is approached for $k \rightarrow 0$ and never attained. Our obstacle moves at $0.55 < 1$. For the uniform fluid, (9.4) forbids every excitation, and the run nevertheless dissipates. (Chapter 5 proves the same statement for general c and m : QuantumFluids.BogoliubovDispersion.landau_velocity_eq.)

The hypothesis that fails. Landau’s argument is about excitations of a fluid that is uniform, and an obstacle that disturbs it little. An obstacle of height 0.9μ is not a small disturbance. At rest, the density in its core is a tenth of the bulk, and the local speed of sound there is $\sqrt{0.1} = 0.32$; a fluid forced through or around it must speed up. The criterion has to be applied locally, the local density fixing the local speed of sound. That is how Kwon and Shin read it: nucleation sets in when the flow becomes locally supersonic, $v(\mathbf{r}) > c_s(\mathbf{r}) = \sqrt{g\rho(\mathbf{r})/m}$ (their equation (5), which they call the local Landau criterion), and they build their effective diameter, and with it their Reynolds number, on the Mach-1 contour of the time-averaged irrotational flow [27]. The idea is older. Frisch, Pomeau and Rica found in the Gross–Pitaevskii model that vortex pairs are emitted behind an obstacle [94], and Josserand, Pomeau and Rica related the nucleation around a disk to the flow becoming locally supersonic at its edge [95].

The following statements make the local criterion exact. They are again new Lean, in the same file.

LEAN 4 FLOWPAST (NEW FOR THIS BOOK)

```

theorem supersonic_iff {n j : ℝ} (hn : 0 < n) (hj : 0 < j) :
  Real.sqrt n < j / n ↔ n ^ 3 < j ^ 2

theorem sonic_speed_threshold {n u v B : ℝ} (hB : n + u ^ 2 / 2 + v = B) :
  n < u ^ 2 ↔ 2 / 3 * (B - v) < u ^ 2

theorem bernoulli_min {s n : ℝ} (hs : 0 < s) (hn : 0 < n) :
  3 / 2 * s ≤ s ^ 3 / (2 * n ^ 2) + n

```

```

theorem barrier_height_bound {s n V : ℝ} (hs : 0 < s) (hn : 0 < n)
  (hB : s ^ 3 / (2 * n ^ 2) + n + V = 1 + s ^ 3 / 2) :
  V ≤ (1 - s) ^ 2 * (s + 2) / 2

```

The first (QuantumFluids.FlowPast.supersonic_iff) says that a flow of density n and local flux $j = nu$ has $u > \sqrt{n}$, the local speed of sound, exactly when $n^3 < j^2$, that is when the density is below the *sonic density* $j^{2/3}$. It is algebra, and it is the form in which the data will be read below. *The second* (QuantumFluids.FlowPast.sonic_speed_threshold) adds Bernoulli's relation $n + \frac{1}{2}u^2 + V = B$, the stationary hydrodynamic relation without quantum pressure, with $B = 1 + \frac{1}{2}v^2$ for a fluid of density 1 at speed v far from the obstacle: the flow is locally supersonic exactly when $u^2 > \frac{2}{3}(B - V)$, a threshold on the speed that depends on the potential only. For $V = 0$ it is the critical speed $u^2 = \frac{2}{3} + \frac{1}{3}v^2$ that Josserand, Pomeau and Rica obtained for the flow past a disk; here it is 0.876 far from the obstacle, and 0.409 at the centre of the obstacle, where $V = V_0 = 0.9$.

The last two are a one-dimensional picture of the same mechanism. Take a steady flow in a channel, of flux j conserved along the channel, and write $j^2 = s^3$, so that s is the sonic density. Bernoulli's function $j^2/(2n^2) + n$ is bounded below by $\frac{3}{2}s$ (QuantumFluids.FlowPast.bernoulli_min), with equality only at the sonic point $n = s$ (QuantumFluids.FlowPast.bernoulli_eq_iff, in the file). The proof is a factorisation: $\frac{s^3}{2n^2} + n - \frac{3}{2}s = \frac{(n-s)^2(2n+s)}{2n^2} \geq 0$, the identity $2n^3 - 3sn^2 + s^3 = (n-s)^2(2n+s)$ being checked by ring (proof abridged). Consequently, if the fluid upstream has density 1 and speed j , a steady state can exist at a point of the channel where the potential is V only if $V \leq V_c(s) := (1-s)^2(s+2)/2$ (QuantumFluids.FlowPast.barrier_height_bound): there is no steady flow over a barrier higher than that, the flow is *choked*. For $j = 0.55$ the sonic density is $s = 0.671$ and the ceiling is $V_c = 0.144$; the obstacle's $V_0 = 0.9$ is 6.2 times higher. As $j \rightarrow 1$ (so $s \rightarrow 1$) the ceiling V_c tends to 0: at the speed of sound any barrier chokes the flow, which is Landau's criterion again, seen from the other side.

What the one-dimensional theorem is not. The ceiling V_c reaches $V_0 = 0.9$ at $j = 0.017$, a factor of ten below the critical velocity $v_c \approx 0.18$ that Kwon and Shin find for the two-dimensional flow by testing for stationary solutions. The theorem does not predict that threshold, and LL-15, the programme's rule that an analogy is not a result, says why: it needs a flux conserved along the channel, and the flow has none. On the axis of the deposited run at $t = 20 \tau$ the flux at the centre of the obstacle is $j = 0.17$, 32 % of the upstream 0.55: the stream lines have opened up. What survives is the structure of the picture, the sonic point as the minimum of Bernoulli's function and the supersonic region as the part of the flow beyond it. Kwon and Shin's own estimate of the supersonic region (their appendix A) uses Bernoulli and continuity in two dimensions to first order in v/c ; they report that it overestimates the threshold and attribute the difference to the neglected quantum pressure, which they say is crucial in triggering shedding. The data below agree.

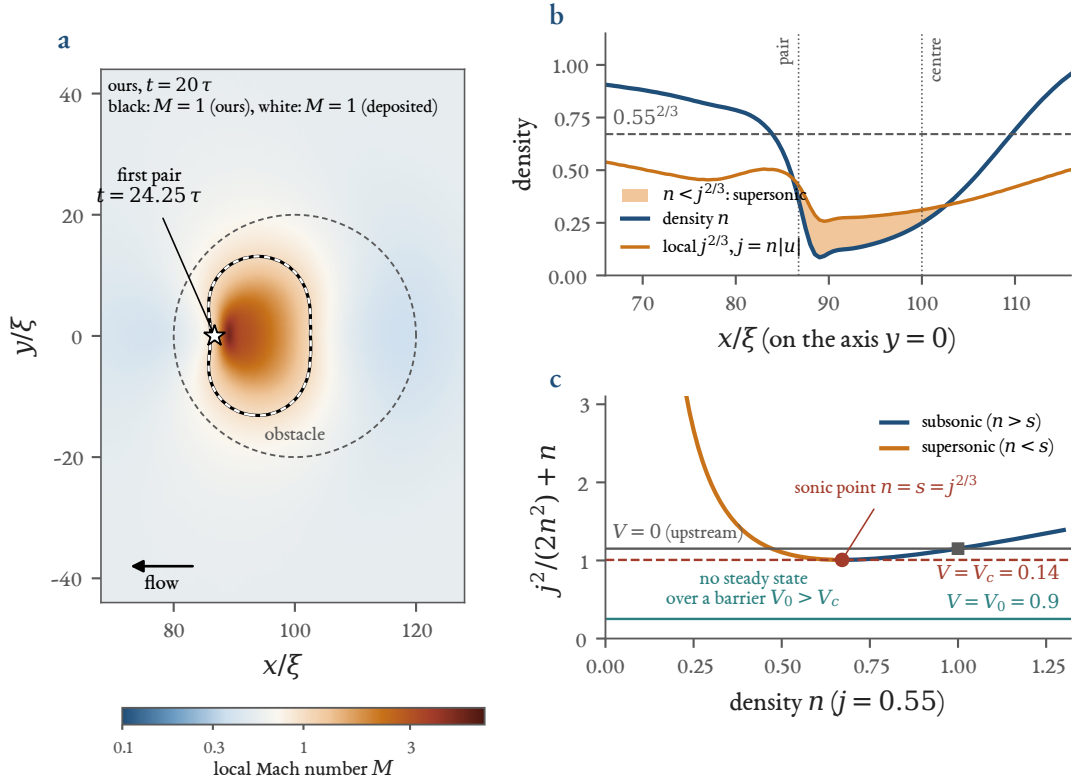


Figure 9.2: The flow is locally supersonic long before the first vortex. **a**, the local Mach number $M = |\nabla\theta - v\mathbf{e}_x|/\sqrt{n}$ of our field at $t = 20 \tau$; black: $M = 1$ (ours), white: $M = 1$ (deposited); the star marks the first vortex pair, $t \approx 24.25 \tau$. **b**, the density and the local sonic density $j^{2/3}$, $j = n|\mathbf{u}|$, on the axis: $M > 1$ exactly where $n < j^{2/3}$ (supersonic iff). **c**, Bernoulli's function $j^2/(2n^2) + n$ of the channel at $j = 0.55$ and the levels $1 + j^2/2 - V$ it must equal (bernoulli_min, barrier_height_bound). Script figures/ch09_mach.py.

Reading the data. Figure 9.2 shows the field at $t = 20 \tau$, four time units before the first pair. The supersonic region is a single blob inside the obstacle, on its downstream half: on the axis it runs from $x = 86.5$ to 102.5 (the obstacle is centred at 100), and it extends at most 13ξ to either side. Its area is $378.0 \xi^2$ in our field and $379.5 \xi^2$ in the deposited one; the Mach number reaches 5.38 and 5.42 at $x = 89$, where the density is 0.086 and the flow runs at $1.58 c$ relative to the obstacle; the two $M = 1$ contours cannot be told apart. The blob existed earlier: its area is 254 and $253 \xi^2$ at $t = 10$ and $192 \xi^2$ at $t = 5$ (maximal M : 2.22 , 2.21 and 1.85). The local criterion is thus satisfied for at least nineteen time units before the first pair appears: necessary, but nowhere near sufficient on a short time scale. The region has to deepen, the rarefaction behind it has to sharpen, and then a hole opens at its downstream end (the star in Figure 9.2a).

The threshold of `sonic_speed_threshold` can be compared with this region without using the density. At $t = 20$ the set where $|\mathbf{u}|^2 > \frac{2}{3}(B - V)$ has an area of $450 \xi^2$ against the measured

$378 \tilde{\epsilon}^2$ and contains 99 % of it (at $t = 5, 10, 15$: 84, 88, 95 %), with an excess of area of 5 % at $t = 5$ and 19 % at $t = 20$. It predicts where the flow is supersonic well and what the density is poorly: Bernoulli's relation implies a density of 0.007 at the centre of the obstacle, where the measured one is 0.249, because it needs a stationary flow (Section 9.6).

9.4 The reproduction

WHAT “REPRODUCE” MEANS HERE

Three things hide in the word. (i) The same input gives the same output, to rounding: the initial field is reproduced this way. (ii) A different scheme gives the same observables within a tolerance: for $t \leq 10$ the force agrees to 4×10^{-5} of its largest value, against a registered tolerance of 10^{-5} that was not met. (iii) The same events happen at the same times: the first vortex pair, the vortex counts. The fourth thing, reproducing the paper, is none of these and is not claimed.

What was run. The flow solver is the module `flow` of the crate `qf-pgpe` (rusty-SUNDIALS, release `v11.6.0`; the Python extension `qf_pgpe` does not expose it). It is driven by the example program `kwon_shin`, which reads the deposited initial field and force, integrates, and writes the force every 0.1τ and ψ every 5τ . The production run to $t = 50 \tau$ took 5000 steps and 2741 s on the shared eight-core machine (load not recorded). Two excerpts of the source follow: the right-hand side (9.1) (`ws.lap` is $\nabla^2 \psi$, `ws.dxp` is $\partial_x \psi$, both spectral) and the velocities of the four stages.

RUSTY-SUNDIALS QF-PGPE::FLOW, THE SOURCE

```
// FlowSolver::rhs, last loop (excerpt)
for i in 0..psi.len() {
    let p = psi[i];
    let h = ws.lap[i] * (-0.5) + p * (p.norm_sqr() - 1.0);
    out[i] = ws.dxp[i] * v
        - (Complex64::new(self.gamma[i], 1.0)) * h
        - Complex64::new(0.0, self.potential[i]) * p;
}
// FlowSolver::step (excerpt): the velocity of the four stages
let (v1, v2, v3, v4) = match ramp {
    Ramp::StepEnd => {
        let v = self.velocity(t + dt);
        (v, v, v, v)
    }
    Ramp::StageTimes => (
        self.velocity(t),
        self.velocity(t + 0.5 * dt),
```

```

    self.velocity(t + 0.5 * dt),
    self.velocity(t + dt),
),
};

```

RUSTY-SUNDIALS QF-PGPE::FLOW, A FRESH RUN

A fresh run of the first time unit, made for this chapter (command, output and two rows of the force file as written; the path is shortened):

```

$ kwon_shin --ref-dir ../20068724/extracted --t-end 1.0 --dt 0.01 --ramp step-
end
step 0/100 t = 0.00 (1 s)
step 100/100 t = 1.00 (1169 s)
force_x: max |ours - reference| = 5.396e-6, relative to max |F| = 8.10e-6; wall
1169 s (100 steps)

t,force_x,force_ref,diff
0.300,-0.178067868131,-0.178068995476,1.127e-6
1.000,-0.665855021696,-0.665849626064,-5.396e-6

```

The hundred steps cost 67 s of CPU time (0.67 s per step, one thread) and 1169 s of wall-clock time: the machine is shared (load average 12.7 at the start, 24.9 at the end), so the wall-clock figure measures the queue and not the code. The force column equals the first rows of the 5000-step run at the printed precision.

The velocity ramp, and a convention that Lean can measure. The deposited driver loops for i in $\text{range}(\text{steps} + 1)$: $tt = i * dt$; if $i > 0$: $\text{evolver.time_evolv}(tt)$, and $\text{time_evolv}(tt)$ uses $v(tt)$ in all of its stages. The step that takes the field from $(i - 1)\Delta t$ to $i\Delta t$ therefore runs at the velocity of its *end*. During the ramp (0.1τ , ten steps) the obstacle moves relative to the fluid by $\sum_{i=1}^{10} v(i\Delta t)\Delta t = 0.03025$ instead of $\int_0^{0.1} v dt = 0.0275$, an overshoot of 0.00275ζ . In general:

LEAN 4 FLOWPAST (NEW FOR THIS BOOK)

```

theorem ramp_overshoot (a h : ℝ) (n : ℕ) :
  ∑ i ∈ range n, a * ((i : ℝ) + 1) * h * h - a * ((n : ℝ) * h) ^ 2 / 2
  = a * h ^ 2 * n / 2

```

Holding the velocity *at* of a linear ramp at the end of each of n steps of size h overshoots the integral $a(nh)^2/2$ by exactly $ah^2n/2$ (`QuantumFluids.FlowPast.ramp_overshoot`; proof: induction on n with one `nlinarith` for the step; abridged); with $a = v_f/t_{acc} = 5.5$, $h = 0.01$ and $n = 10$ this is the 0.00275 above. The solver measures the same thing. With the velocity read at the stage times the

force differs from the deposited one by 3.5×10^{-3} at $t = 0.1 \tau$ (between 2.7×10^{-3} and 3.5×10^{-3} up to $t = 5$); with the reference's reading it differs by 1.1×10^{-6} for $t \leq 0.3$. Halving Δt halves the overshoot, to 0.001375ζ , and the offset goes down to 1.75×10^{-3} ; the offsets are the same multiple of the displacement that separates the two runs, 1.2709 and $1.2713 \mu/\zeta^2$. The convention is a displacement of the obstacle relative to the fluid by the amount that the theorem computes. It also means that halving Δt with the reference's convention is not a convergence test, since the convention depends on Δt . Our own temporal error, with the stage-time reading, is 3.3×10^{-10} in the force over $t \leq 5$ ($\Delta t = 0.01$ against 0.005), negligible against every other number in this chapter.

The library module `HeliumKinematics` does the same kind of small job for the analysis of the neutron time-of-flight data of the 2021 paper: it proves that two printed energy formulas (its equations 12 and 13) are one formula once the flight distance and the incident energy are written through the incident velocity (`QuantumFluids.HeliumKinematics.tof_eq12_eq13`) and that rescaling the energy calibration cannot change whether one measured energy exceeds twice another (`QuantumFluids.HeliumKinematics.plateau_excess_calibration_invariant`): what a convention can and cannot move, settled once. The resemblance is one of method; it says nothing about neutron scattering.

The initial field. The deposited initial field is itself a finding. The code prepares it by imaginary-time propagation from the Thomas–Fermi field, stopping when the integral γ of $|\psi_{\text{new}} - \psi_{\text{old}}|^2$ falls below $\epsilon \delta \tau$, with the deposited tolerance $\epsilon = 5 \times 10^{-4}$ and $\delta \tau = 0.04$, that is below 2×10^{-5} . We re-derived the field in a few lines of numpy from the code's operators. After *one* step $\gamma = 1.39 \times 10^{-5} < 2 \times 10^{-5}$: the loop stops at its first iteration. The stored $\psi(t = 0)$ is therefore the Thomas–Fermi field after one imaginary-time step plus the seeded noise (10^{-4} times integers in $[-5, 5]$ for the real and the imaginary part, relative size 4.1×10^{-4}), not a relaxed stationary state of the obstacle problem. Our array, rounded to `complex64`, equals the stored one bit for bit in 90 % of its 500 000 elements; the other 50 077 (those where the noise has a zero imaginary part) differ by floating-point residues below 2×10^{-16} . In double precision the distance is 2.8×10^{-8} , the rounding of `complex64`. This is a statement about the deposited data and not a criticism of the paper, which describes the initial state as relaxed to the ground state and measures its observables long after any relaxation of the initial field; but anyone who uses the deposited $\psi(0)$ as a ground state should know that the early force contains that relaxation.

Results. Table 9.1 collects the comparisons, Figure 9.3 shows the force. All numbers are computed by `figures/ch09_numbers.py` from the deposited files and our snapshots.

Three things stand out. The agreement is at the level of 10^{-6} for the first time unit and 10^{-4} for the first ten, which is what two accurate integrators of the same equation should give, and it was reached with a scheme, a precision and a machine that have nothing in common with the reference's. Second, the tolerance that the programme had registered for this item of its plan, a relative force agreement of 10^{-5} for $t \leq 10$, is not met: it is 4×10^{-5} of the largest force of that window (it is 6×10^{-6} for $t \leq 0.3$). The criterion was set before the reference's velocity convention was known,

quantity	ours against the deposited run
initial field	bit for bit after rounding to complex64 in 90 % of the elements; 2.8×10^{-8} in double precision
force max $ \Delta F $, $t \leq 0.3$; < 1 ; ≤ 5 ; ≤ 10	1.1×10^{-6} ; 4.0×10^{-6} ; 1.4×10^{-4} ; 2.2×10^{-4} (4.0×10^{-5} of the largest force, 5.45, of that window)
force, windows 10–20, 20–30, 30–40, 40–50	7.9×10^{-3} ; 1.7×10^{-2} ; 1.7×10^{-2} ; 1.6×10^{-2} (0.19 % of the largest force, 8.94)
ψ at $t = 10, 20, 30, 40, 50$ (relative L^2)	2.2×10^{-4} , 5.8×10^{-4} , 1.3×10^{-3} , 2.5×10^{-3} , 4.1×10^{-3} ; after removing a global phase 2.1×10^{-4} , 4.8×10^{-4} , 1.0×10^{-3} , 1.9×10^{-3} , 3.3×10^{-3}
density $ \psi ^2$ (relative L^2)	2.4×10^{-4} , 2.0×10^{-4} , 2.0×10^{-4} , 2.8×10^{-4} , 4.0×10^{-4}
lift F_y at the snapshot times	within 0.4–3.6 % of the deposited values ($ F_y \leq 3.5 \times 10^{-4}$)
vortex census, $t = 30, 40, 50$	25 of 26 vortices on the same plaquette, one within one cell
count of the deposited detector	equal at 9 of 10 times
supersonic region, $t = 20$	area 378.0 against 379.5 ξ^2 ; $M_{\max}$ 5.38 against 5.42
own temporal error	3.3×10^{-10} in the force, $t \leq 5$

Table 9.1: Our Rust run against the deposited run. Every row is recomputed here from the raw files. The rows on the initial field, the force, ψ , the density and the detector’s counts repeat numbers of the programme’s earlier record (docs/designs/PGPE_EXTERNAL_REPRODUCTION.md); the gauge-fixed distances, the lift, the census and the supersonic region are new.

and the plan keeps it as failed. No criterion had been registered for $t > 10$; the second half of the table is reported as found. Third, from $t \approx 10.7$ the force difference changes sign, grows, and levels off at about 1.6×10^{-2} , 0.19 % of the largest force, for $t \geq 26$. Section 9.7 comes back to it, because the first reading of that growth that the programme gave is not supported by the data.

9.5 Counting vortices

What a count is. A vortex is a point where the wave function vanishes and its phase winds by 2π ; a grid sees only ψ at its points. The standard way out, used by many vortex-finding codes, is to add up the *principal differences* of the phase around each plaquette (an elementary square of four grid points) and to divide by 2π . The proof assistant has something exact to say about that procedure: it rests on a statement that is true for a reason, and it fails for a reason.

LEAN 4 VORTEXWINDING

```

noncomputable def pdiff (a b : ℝ) : ℝ := toIocMod two_pi_pos (-π) (b - a)

theorem loop_sum_eq_mul (θ : ℕ → ℝ) (n : ℕ) (hclosed : θ n = θ 0) :
  ∃ m : ℤ, ∑ i ∈ Finset.range n, pdiff (θ i) (θ (i + 1))
    = (m : ℝ) * (2 * π) := by
  -- (proof abridged: pdiff = (b - a) - integer * 2π, then the sum telescopes)

```

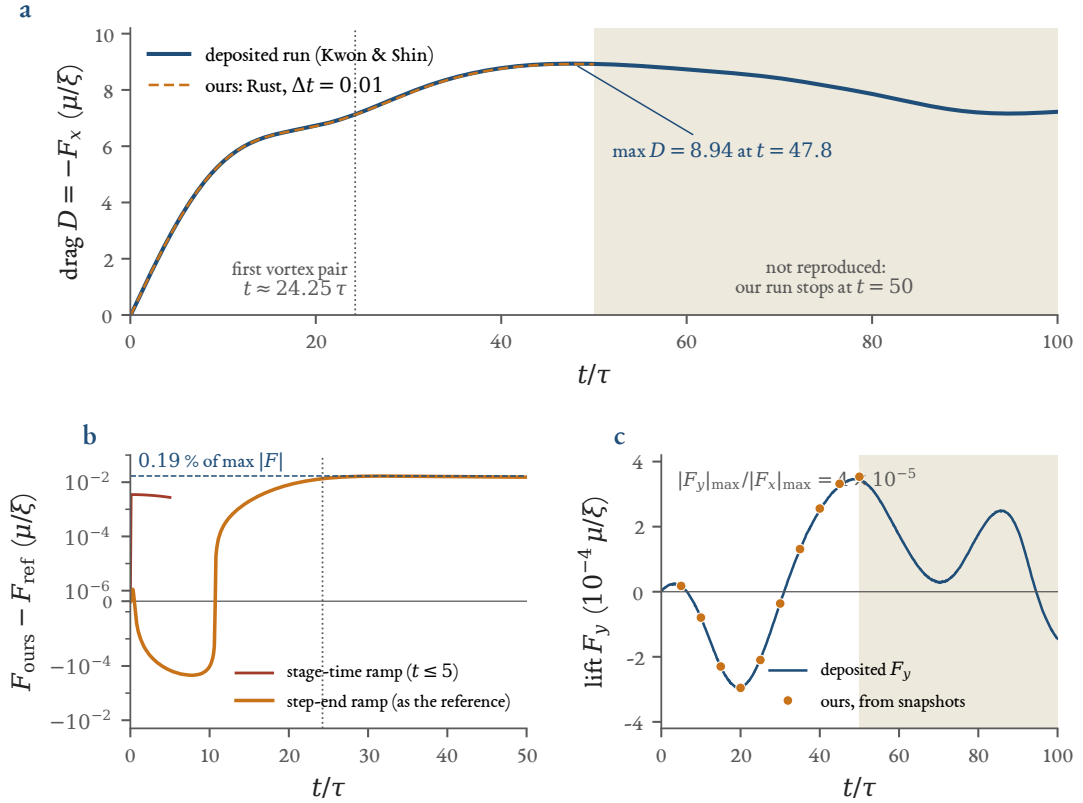


Figure 9.3: The force on the obstacle. **a**, the drag $D = -F_x$ of the deposited run to $t = 100 \tau$ (blue) and of our Rust run to $t = 50 \tau$ (orange dashes). **b**, the difference $F_{\text{ours}} - F_{\text{ref}}$ on a symmetric logarithmic axis, with the reference's step-end reading of the velocity (orange) and with the stage-time reading up to $t = 5$ (red); the dotted line marks the first vortex pair. **c**, the lift F_y of the deposited run and ours at the snapshot times. Script `figures/ch09_force.py`.

```

theorem pdiff_add_rev_eq_two_pi_iff (a b : ℝ) :
  pdiff a b + pdiff b a = 2 * π ↔ (pdiff a b = π ∧ pdiff b a = π)

```

The first theorem, `QuantumFluids.VortexWinding.loop_sum_eq_mul`, says that for *any* closed list of phases $\theta_0, \dots, \theta_n = \theta_0$ the sum of the principal differences along the loop is an integer multiple of 2π (here `pdiff a b` is the representative of $b - a$ in $(-\pi, \pi]$): that a computed winding number is an integer is a theorem and not a numerical accident. The second says exactly when an edge fails to cancel. Walked in both directions an edge gives 0, unless both principal differences equal π , which add up to 2π ; at that single value the cancellation between the two plaquettes that share the edge breaks, and a spurious winding can appear. With no edge at π the oriented sum over a closed surface is zero (`QuantumFluids.VortexWinding.cut_free_of_ne`

`pi`, `QuantumFluids.VortexWinding.balanced_sum_eq_zero`), which for the plaquettes of a periodic grid means that the integer charges add up to zero. What the module does *not* prove, in its own words, is that a field is resolved well enough for the detected winding to be the topological charge of the continuum field: that is a statement about sampling, and it is where data come in. The library's `pdiff` and the Python function that we use are the same function:

```
def pdiff(a, b):
    d = (b - a + np.pi) % (2 * np.pi) - np.pi
    return np.where(d == -np.pi, np.pi, d)

def edge_steps(psi):
    th = np.angle(psi)
    return pdiff(th, np.roll(th, -1, 1)), pdiff(th, np.roll(th, -1, 0))

def plaquette_charge(psi):
    sx, sy = edge_steps(psi)
    s = sx + np.roll(sy, -1, 1) - np.roll(sx, -1, 0) - sy
    q = s / (2 * np.pi)
    return q, np rint(q).astype(int)
```

(from `figures/ch09_common.py`, docstrings omitted; the loop of `s` runs counter-clockwise around the plaquette whose lower-left corner is the grid point.) On the 16 snapshots that we have, the six deposited fields and our ten, the unrounded charge `q` differs from an integer by at most 3×10^{-16} , and the charges of every snapshot add up to zero. The largest principal step on any edge of any of the sixteen snapshots is 0.9993π : the margin to the branch cut is 0.07 % of π , never zero.

Two counts. The deposited run logs a vortex count every 5τ . It is the output of the authors' detector, the function that the same driver also hands to the routine that removes vortices near the boundary of the box. Its rule, from the code: take the local minima of the density along the first array axis (`y`) that are below 0.2 and lie outside a disc of 5ζ around the obstacle; drop every minimum within 3ζ of an earlier one; keep a survivor if the phase circulation on the square loop of half-width 2ζ around it exceeds 0.9π in modulus, steps of 0.3π or more being *dropped* from the sum. We ported this rule line by line (`exploration/external/ks_vortex_count.py`); on the deposited fields at $t = 10, \dots, 50$ it returns the deposited counts 0, 0, 3, 4, 7. Alongside it we use the *exact census*: every plaquette of non-zero winding, with no further rule. The two are compared in Table 9.2 and Figure 9.4.

Three statements follow. *First*, the detector's counts are equal at 9 of the ten times, with the single difference at $t = 45$: 7 in our run, 6 in the deposited file. *Second*, the census is identical at every time where a deposited field exists to compare with: 2, 14 and 10 charged plaquettes at $t = 30, 40, 50$ in both runs, and 25 of the 26 on exactly the same plaquette, the last one within one cell (0.5ζ). This is a much stronger statement than the equal counts: not a number but the positions and signs of the vortices are reproduced. *Third*, the detector is not a census. At $t = 40$ it counts 4 of the 14 vortices that are present; at $t = 30$ it counts 3 where there are 2, because two of its accepted

t/τ	5	10	15	20	25	30	35	40	45	50
reference detector, deposited file	0	0	0	0	2	3	2	4	6	7
same detector, our snapshots	0	0	0	0	2	3	2	4	7	7
exact census, ours	0	0	0	0	2	2	4	14	10	10
exact census, deposited fields	–	0	–	0	–	2	–	14	–	10

Table 9.2: Vortex counts. The first row is the deposited file, the second the same detector run on our snapshots, the last two the exact census of non-zero plaquettes (no deposited field exists at $t = 5, 15, \dots$).

loops enclose the same plaquette, so one vortex is counted twice (Figure 9.4b, c).

A fragile decision. The detector’s loop integral is not quantised, because it drops the large steps, and its decision is fragile. For the same vortex, the integral in the deposited field and in ours differs by up to 0.57π (by 0.24π at $t = 30$), against a threshold of 0.9π , although the two fields agree to better than 4.1×10^{-3} in relative L^2 . At $t = 45$, the one time at which the counts differ, no deposited field exists, so the vortex that the deposited detector missed cannot be identified. In our field the seven accepted vortices have loop integrals of 1.26π or more, except one, at $(x, y) = (82.5, -4.0)$, with 0.951π , which is 5.7 % above the threshold. It is the only candidate for which a perturbation of the size seen between the two runs at other times could flip the decision. That is a plausible explanation of the unit difference. It is not a demonstration.

Mirror symmetry. The exact census is a set of mirror pairs: for each plaquette of charge q at (x, y) there is one of charge $-q$ at $(x, -y)$, in every snapshot of ours and in the deposited fields at $t = 30$ and 40 ; at $t = 50$ the two plaquettes of the deposited field nearest to the axis have their partners one cell away. The relative L^2 distance of ψ from its mirror image $\psi(x, -y)$ is 5.7×10^{-4} at $t = 0$ (the seeded noise), 2.2×10^{-4} and 3.5×10^{-4} at $t = 10$ in the deposited field and in ours, and 1.4×10^{-4} and 1.8×10^{-4} at $t = 50$: it decays instead of growing. Up to $t = 50$ the flow is mirror-symmetric, and so, as far as the lift shows, is the deposited run up to $t = 100$ (Figure 9.3c: the lift never exceeds 4×10^{-5} of the drag).

9.6 The birth of a pair

Zooming in time. The snapshots are five time units apart; the first pair appears between two of them. To see it we continued our field from $t = 20$ to $t = 25$ with the same scheme in numpy (figures/ch09_birth_run.py), storing a $60\zeta \times 50\zeta$ sub-box every 0.1τ . The continuation reproduces our Rust snapshot at $t = 25$ to a relative L^2 distance of 1.2×10^{-15} : two implementations of one scheme, as they should be. Figure 9.5 shows what happens.

What happens. On the axis the density minimum of the sub-box falls from 0.086 at $t = 20$ to 9.1×10^{-3} at $t = 23.3$, where it hesitates, and then, within a time unit, to 9×10^{-5} at $t = 24.2$. At $t = 24.3$ two plaquettes carry the charges ∓ 1 , at $(x, y) = (86.75, \mp 0.25)$: a vortex and an

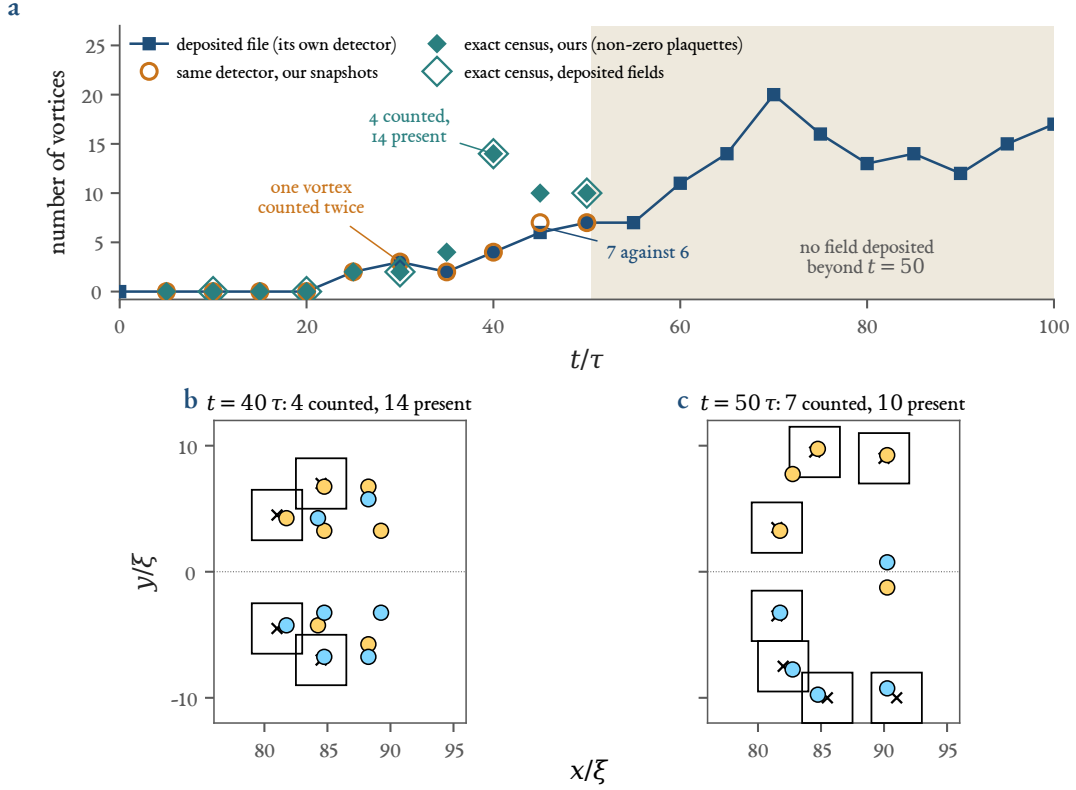


Figure 9.4: How many vortices? **a**, counts against time: the deposited file (blue squares, to $t = 100 \tau$), the same detector on our snapshots (open orange circles), and the exact census of non-zero plaquettes (teal diamonds: ours, filled; deposited fields, open). **b**, **c**, the deposited field at $t = 40$ and 50τ : the plaquettes of non-zero winding (gold counter-clockwise, cyan clockwise) and, as black squares and crosses, the loops and centres of the detector's accepted vortices. Script `figures/ch09_census.py`.

antivortex, 0.5ζ apart, on the axis, 13.25ζ downstream of the centre of the obstacle. Since the flow is mirror-symmetric a single pair has to be born on the axis, and it is. By $t = 24.9$ the two are 2.5ζ apart. The pair appears at the downstream end of the supersonic region, which at $t = 24.3$ runs on the axis from $x = 86.0$ to 102.0 : within a healing length of its edge.

Lean's edge case in the data. The largest principal phase step on any edge of the sub-box is 0.78π at $t = 24.0$, 0.61π at $t = 24.2$ and 0.998π at $t = 24.3$, the first time the pair exists (Figure 9.5c). Two vortices of opposite sign one grid cell apart are the configuration in which the phase differs by almost π across the edge that joins them. The branch cut that `QuantumFluids.VortexWinding.pdiff_add_rev_eq_two_pi_iff` isolates is therefore not a remote possibility: when the pair is born the largest step is within 0.2% of π . No edge is exactly at π , so the census is an exact integer count of the sampled field. What that count means *at birth* is not decided by the theorem. A pair

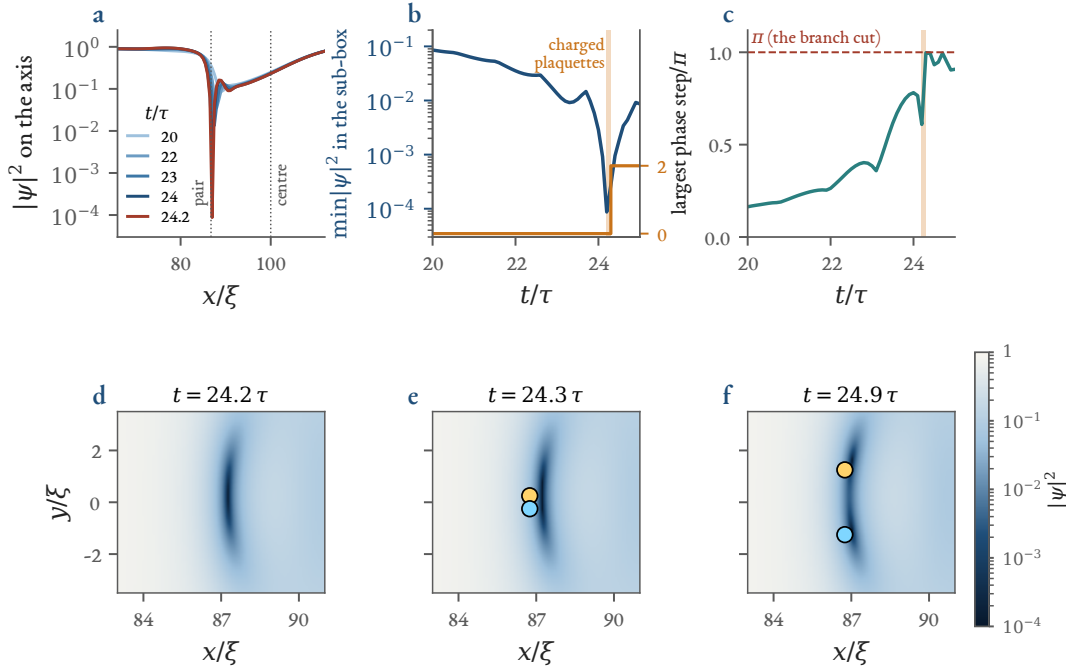


Figure 9.5: The first vortex pair, from the 0.1τ continuation of our field. **a**, the density on the axis $y = 0$ at five times; the dotted lines mark the birth site and the centre of the obstacle. **b**, the smallest density of the sub-box (blue, logarithmic) and the number of plaquettes of non-zero winding (orange); the shaded band is the interval in which the pair is born. **c**, the largest principal phase step on any edge of the sub-box, in units of π . **d–f**, the density (logarithmic colour scale, cubic interpolation) just before the birth, just after it and 0.6τ later, with the charged plaquettes (gold counter-clockwise, cyan clockwise). Script figures/ch09_birth.py.

half a healing length apart, on a grid whose spacing is half a healing length, is at the limit of what the sampled field resolves; resolution, not the theorem, decides whether the two plaquettes are two vortices of the continuum field.

Hydrodynamics on the data. The reading of the run in terms of Bernoulli’s relation and the supersonic region (Section 9.3) rests on the Madelung form of (9.1). Write $\psi = \sqrt{n} e^{i\theta}$, $\mathbf{u} = \nabla\theta - v \mathbf{e}_x$ and $Q = -\nabla^2 \sqrt{n} / (2\sqrt{n})$, the quantum pressure. Where $\Gamma = 0$, the phase obeys

$$-\partial_t \theta = R := \frac{1}{2} |\mathbf{u}|^2 + n + V + Q - 1 - \frac{1}{2} v^2. \quad (9.5)$$

For a stationary flow $R = 0$: that is Bernoulli’s relation with the quantum pressure restored. The algebraic part of the Madelung transformation, the splitting of the kinetic energy density into a quantum-pressure part and a hydrodynamic part, is a theorem of the library (`QuantumFluids.Madelung.kinetic_density_split`); the dynamical identity (9.5) is not, and we check it on the data. At $t = 20.1$ we compute R from a single snapshot of the sub-box (fourth-order differences) and $\partial_t \theta$

from the snapshots at $t = 20.0$ and 20.2 . They agree: the correlation is 0.99999 ($1 - r = 7 \times 10^{-6}$), and the largest difference is 4.2×10^{-3} against an rms of 0.12 for R . The model, its signs, the quantum pressure and the differences are consistent.

The same quantity says how far the flow is from the stationary state that the one-dimensional theorem assumes. On the axis R runs from -0.21 to 0.39 , as large as the terms it balances: the phase is not stationary, and different points of the axis advance in phase at rates that differ by up to $0.6 \mu/\hbar$. This is the phase accumulation across the obstacle that Kwon and Shin describe as the precursor of the phase slip that nucleates dipoles. The quantum pressure is small along most of the axis ($|Q| \leq 0.028$ for $x < 84$ or $x > 92$) and large in the hole: $Q = -0.36$ at $x = 88.5$. That is the quantitative content of the remark of the paper quoted in Section 9.3, that the quantum pressure plays a crucial role in triggering shedding.

9.7 What does not agree, and what we do not know

The plateau of the force difference. The deposited force and ours separate from $t \approx 10.7$. The difference $\Delta F = F_{\text{ours}} - F_{\text{ref}}$ changes sign there, reaches 1.3×10^{-3} at $t = 14$, 8.0×10^{-3} at $t = 20$ and 1.3×10^{-2} at $t = 24$, when the first pair is born and the difference has reached 82 % of its final value, and stays between 1.5×10^{-2} and 1.7×10^{-2} for $t \geq 26$ (Figure 9.3b). Our drag is weaker than the deposited one by 0.19 % of the maximum force. Four things about this difference can be said, and none of them is a cause.

- It starts more than thirteen time units *before* any vortex exists. The programme's design note on the external reproduction reads it as the signature of a perturbation amplified by the unstable wake, and the software paper [21] says that the force separates "once the wake exists". What exists at $t = 11$ is the supersonic region and the rarefaction behind it, not vortices; the first pair arrives when the difference has already grown to 82 % of its final value. That reading is not supported and is corrected here.
- It is smooth and does not oscillate. A constant delay between the two runs does not describe it: fitting $\Delta F = -\delta F'_{\text{ref}}(t)$ window by window gives $\delta = 0.12, 0.17$ and 0.50τ for the windows 20–30, 30–40 and 40–50, with residuals of 4 %, 26 % and 71 % of the difference: a delay would be one number with small residuals.
- The mirror-symmetry-breaking channel is not what grows. The lift of the deposited run stays at 4×10^{-5} of the drag up to $t = 100$, our lift at the snapshot times agrees with it to 3.6 %, and the asymmetry of both fields decays (Section 9.5).
- The fields differ everywhere, mildly: the rms density difference is between 2×10^{-4} and 4×10^{-4} at all five times, and the part of its square sum that lies within 40ζ of the obstacle (about 4 % of the area of the box) is 11 % at $t = 10$, 10 % at $t = 20$, and then 39, 72 and 80 % as the vortices form. The relative L^2 distance of ψ itself, 4.1×10^{-3} at $t = 50$, drops only to 3.3×10^{-3} when a global phase (a gauge, 2.5×10^{-3} at that time) is removed.

The force is a delicate functional. It weighs the density by $\partial_x V$, which is odd about the centre of the obstacle and has the maximum modulus $(2/\sigma) V_0 e^{-1/2} = 0.055$ per healing length; a coherent

density difference of a few 10^{-4} over the region where $\partial_x V$ is large is enough to move it by 10^{-2} .

Candidate causes, none tested. Three differences between the two computations could produce a smooth, slowly growing difference of this size. The reference works in single precision. Its splitting is of second order with $\Delta t = 0.01$, while our scheme has a temporal error of 3.3×10^{-10} in the force at $t \leq 5$ (Section 9.4). And the local step of its code integrates the damping exactly only where $\Gamma > 0.01$ and treats the rest of the box as unitary, while its linear step uses the full Γ . Our scheme has none of the three. The first two are properties of the reference’s arithmetic and discretisation; the third is a difference between what the code does and the model (9.1) that we recovered from it, and would be a defect of our recovery rather than of the reference. What would settle it is a numpy port of the reference’s own stepper, run in double and in single precision from the deposited field: the first run would separate the splitting from the arithmetic. We did not do it.

What was not done. The deposited force and counts go on to $t = 100$; our run stops at $t = 50$, where the deposited snapshots stop, and a longer Rust run costs 46 minutes of wall-clock time per fifty time units on the shared machine. After $t = 50$ the deposited detector’s count rises to 20 at $t = 70$ and then fluctuates between 12 and 17. Nothing in the first hundred time units shows the alternating same-sign clusters that the paper describes at this velocity; they belong to its averaging window of 2000 to 10000 τ , and the lift is still 4×10^{-5} of the drag. We computed no Strouhal number, drag coefficient, Reynolds number or effective diameter. The width of the supersonic region at $t = 20, 26.5 \xi$, is an *instantaneous* quantity of a region that has not yet shed a vortex; it is not the paper’s time-averaged D_{eff} and must not be compared with it. Our run has no vortex removal near the boundary, as the paper’s has; the deposited log records that the code’s routine removed none in the first 100 τ (the two count columns of its file are equal). There is no GPU on this machine, and no comparison with the authors’ code was possible beyond its deposited output.

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved (Lean 4, kernel-checked, standard axioms only): that the Landau velocity of the uniform Gross–Pitaevskii dispersion is exactly the speed of sound; that a flow is locally supersonic iff $n^3 < j^2$, and, with Bernoulli’s relation, iff $u^2 > \frac{2}{3}(B - V)$; that Bernoulli’s function at fixed flux is minimised at the sonic point and that this bounds the height of a barrier that a steady channel flow can cross; that holding a linear ramp at the end of each step overshoots its integral by $ah^2n/2$; that the principal-difference winding around a closed loop is an integer multiple of 2π . These are theorems about numbers and finite sums. They say nothing about the Gross–Pitaevskii dynamics. The library’s Landau lemmas formalise a paragraph of the review of Godfrin and Krotscheck as inequalities between numbers, not as statements about a measured curve.

Simulated and compared (Rust, double precision, against a deposited single-precision run): the force on the obstacle to 2.2×10^{-4} for $t \leq 10$ and to 0.19 % of its maximum for $t \leq 50$; ψ

to 4.1×10^{-3} at $t = 50$; the position and sign of 26 vortices, 25 of them on the same plaque; the area of the supersonic region at $t = 20$ to 0.4 %. Nothing here is a laboratory measurement, and nothing is a reproduction of the paper's results.

Only an analogy: the one-dimensional choked flow as a picture of the two-dimensional threshold. Its premise, a flux conserved along the channel, fails in the data (the flux at the centre of the obstacle is 32 % of the upstream flux), and it puts the threshold a factor ten below the paper's; the ramp theorem's resemblance to the library's calibration lemmas is a resemblance of method.

Failed or unresolved: the registered tolerance 10^{-5} for the force at $t \leq 10$ is not met (4×10^{-5}); the cause of the force difference of 0.19 % is not identified, and the earlier explanation of it is withdrawn; the single difference of the detector's counts at $t = 45$ is explained only as plausible; nothing was run beyond $t = 50$.

9.8 Exercises

Exercise 1. Prove `bernoulli_min` by hand, and then `barrier_height_bound`. Show that $V_c(s) = (1-s)^2(s+2)/2$ decreases on $(0,1)$ and that $V_c \approx \frac{3}{2}(1-s)^2$ near $s = 1$, hence $V_c \approx \frac{2}{3}(1-j)^2$ near $j = 1$; compare with the exact value 0.144 at $j = 0.55$ (the approximation gives 0.135). In two sentences, say why a weak barrier does not choke a flow with $j < 1$ although for $j = 1$ every barrier does. *Hint:* the factorisation $2n^3 - 3sn^2 + s^3 = (n-s)^2(2n+s)$; for the last part, what is $V_c(1)$?

Exercise 2. Load `psi_time_50.0.npy` of the archive, compute the exact census with the functions of Section 9.5, and check that it contains ten plaquettes. Then apply the detector's merging rule, dropping every vortex within 3ζ of an earlier one, to the census. How many vortices remain at $t = 40$ and at $t = 50$, and how does the answer depend on the merging radius? Does any radius make the census reproduce the detector's counts 4 and 7? *Hint:* `scipy.spatial.cKDTree`; the detector also requires a density minimum below 0.2 along the first array axis, which a census does not.

Exercise 3. Suppose the reference had read the velocity at the *start* of each step. State the analogue of `ramp_overshoot` (the sum over $i = 0, \dots, n-1$ of $a(ih)h$) and prove it, in Lean or by induction. Using the sensitivity of the force to a displacement of the obstacle, $1.2709 \mu/\xi^2$, predict the sign and size of the difference between our stage-time run and that hypothetical reference at $t = 0.1$, and the difference between that reference and the actual one. *Hint:* the displacement error is now an undershoot of the same size; the answer for the first is a difference of the size of the offset of Section 9.4 and of opposite sign, and the second is twice as large.

9.9 Notes and further reading

The reproduced run, and all that is quoted of its paper, is that of Kwon and Shin [27], whose open archive [28] (MIT for the code, CC BY 4.0 for the processed data, as its README says) makes a reproduction of this kind possible at all. For the local criterion that organises their analysis, the early studies are the numerical discovery of vortex emission in the Gross–Pitaevskii model by Frisch, Pomeau and Rica [94], and its analysis near the transonic transition by Josserand, Pomeau and Rica [95]; Winiecki and collaborators studied vortex shedding and drag in dilute condensates [96], and the superfluid Reynolds number that the paper adapts to penetrable obstacles comes from Reeves, Billam, Anderson and Bradley [97]. The critical velocity of a Gaussian obstacle is the subject of Kwak, Jung and Shin [98], and the laboratory experiments mentioned at the start are [91–93]. Landau’s argument is in [3]; the statement of it that the Lean library formalises is the paragraph of the review of Godfrin and Krotscheck [5], and the spectrum that enters it, for helium, is the one measured by Godfrin and co-workers [2]. Pitaevskii and Stringari [26] and Donnelly [10] cover the hydrodynamics and the vortices. The engine, its verification and the reproduction are those of the software paper [21]; the library is [12]; the comparison records are in `docs/designs/PGPE_EXTERNAL_REPRODUCTION.md` of the repository and in the ledger entries CLAIM-103 and CLAIM-106. The next steps, in the order we would take them, are the numpy port of the reference’s stepper to separate precision from splitting, a run of the Rust engine to $t = 100$, and a long run of the Rust engine at the deposited parameters, over the paper’s window of 2000τ and more, from which the paper’s time-averaged supersonic region and its D_{eff} could be computed.

Trust: Audit, Failure and the Road Ahead

On 9 October 2026, when the estimators that turn tracked vortex positions into a friction coefficient had just been ported to a second language, the programme’s own known-answer test — which the original implementation had passed, once — was run on 8 seeds instead of one. The test is simple. 8 synthetic tracks of up to 2000 time units (a track stops when a vortex and an antivortex come within a distance 2, the box having side 64) are generated from a model in which the friction of a vortex, $\alpha = 0.02$, has been put in by hand; the estimator receives the tracks and must give α back. When the model vortices do not diffuse, the *energy estimator* returns 0.0200. When they diffuse, with $\eta = 5 \times 10^{-4}$, 10^{-3} and 2×10^{-3} , it returns 0.0183, 0.0163 and 0.0156: too low by 8.5, 18.5 and 22.0 percent. On the one seed with which the test had first been run it had returned 0.0187, inside the registered tolerance of 15 percent, and the gate had been recorded as passed.

The estimator was not wrong in a way that a proof could have caught. The identity on which it rests is a theorem of the library (`DissipativeVortexDynamics.energy_dissipation`, section 10.1), and the theorem is true. It had been applied to data that do not satisfy its hypothesis, and no instrument the programme owned at that moment could say so: not the proof assistant, which had never been asked about the data; not the original implementation, which computed the same quantity in the same way; and not the gate, which had been run once.

That is the subject of this last chapter. The book has used three instruments together — an experiment, a proof assistant, a solver — and each of them vouches for something different and for nothing else. Here we ask what that something is, what it is not, and what happened in the programme behind the book when an instrument was trusted past its warranty. The chapter works the estimator example through, with the proved statement, the solver run and the comparison between them (section 10.1); lays out what each kind of check can and cannot see (section 10.2); reports the audit of the Lean library carried out for this book, including what it corrected in the programme’s own records (section 10.3); tells the failures of the programme from its ledger (section 10.4) and the status of its speed claims (section 10.5); and closes with what a reader can reuse and what is still open (sections 10.6 and 10.7).

10.1 A number that passed once

The model and the two estimators

Chapter 7 develops the motion of a quantized vortex in a thermal bath; we need only its bookkeeping. In the point-vortex description a vortex of charge $q_i = \pm 1$ at position $\mathbf{r}_i$ in a periodic box moves as

$$\dot{\mathbf{r}}_i = (1 - \alpha') \mathbf{v}_{s,i} - \alpha q_i \hat{\mathbf{z}} \times \mathbf{v}_{s,i} + \sqrt{2\eta} \xi_i(t), \quad (10.1)$$

in units $\hbar = m = 1$ (circulation 2π). Here $\mathbf{v}_{s,i}$ is the velocity that the other vortices induce at $\mathbf{r}_i$, α is the longitudinal friction, α' its transverse partner, η the diffusion constant of one vortex and ξ_i unit white noise. Without the noise term this is a gradient flow with a skew part. With H the vortex energy ($H = -\sum_{i<j} q_i q_j \ln r_{ij}^2$ in the plane, the Weiss–McWilliams sum on the torus) it reads $\dot{\mathbf{r}} = A(\nabla H) - \frac{\alpha}{2} \nabla H$, where A is skew at every point and carries α' .

Two estimators of α are applied to tracked positions. The *energy estimator* uses the fact that, without noise, H changes only through the friction: $\alpha = -\dot{H} / (2 \sum_i |\mathbf{v}_{s,i}|^2)$. On a track it is minus the slope of $H(t)$ against the running integral $\int 2 \sum_i |\mathbf{v}_{s,i}|^2 dt$. The *regression estimator* regresses the displacement of every vortex over a fixed lag on two predictors, $\int \mathbf{v}_s dt$ and $\int (-q \hat{\mathbf{z}} \times \mathbf{v}_s) dt$; the two coefficients are $1 - \alpha'$ and α , and the scatter of the residual gives η . The first needs only the energy of the configuration; the second needs the positions, and is the one that also delivers α' and η .

What Lean proves about the energy estimator

LEAN 4 DISSIPATIVE VORTEX DYNAMICS

```
/-- `r' = A(∇H) - γ ∇H` with `A` pointwise skew gives `d/dt H(r) = -γ ||∇H||²`.
  ↪ -/
theorem energy_dissipation (H : X → ℝ) (gradH : X → X) (hH : ∀ x,
  ↪ HasGradientAt H (gradH x) x)
  (A : X → X) (hA : ∀ v, inner ℝ v (A v) = 0) (γ : ℝ) (r : ℝ → X)
  (hr : ∀ t, HasDerivAt r (A (gradH (r t)) - γ • gradH (r t)) t) (t : ℝ) :
  HasDerivAt (fun t => H (r t)) (-γ * ||gradH (r t)|| ^ 2) t := by
have h := (hH (r t)).hasFDerivAt.comp_hasDerivAt t (hr t)
have e : (InnerProductSpace.toDual ℝ X (gradH (r t))) (A (gradH (r t)) - γ •
  ↪ gradH (r t))
  = -γ * ||gradH (r t)|| ^ 2 := by
rw [InnerProductSpace.toDual_apply_apply, inner_sub_right, hA,
  ↪ inner_smul_right,
  real_inner_self_eq_norm_sq]; ring
rw [e] at h
exact h
```

Read the statement as follows. The space X is a real inner-product space (here the configuration space of all the vortices); H has a gradient at every point; A is skew, $\langle v, Av \rangle = 0$; and the path $r(t)$ solves $\dot{r} = A\nabla H - \gamma \nabla H$ at every time. Then $H(r(t))$ is differentiable and its derivative

is $-\gamma\|\nabla H(r(t))\|^2$, whatever the skew part. The proof is the chain rule, the skew symmetry of A and $\langle v, v \rangle = \|v\|^2$; it is the eight lines above, shown whole. Two companions complete it: `DissipativeVortexDynamics.dissipation_indep_of_skew` says that the dissipation rate at a configuration does not depend on the skew part (so the energy estimator knows nothing about α'), and `DissipativeVortexDynamics.energy_antitone` says that for $\gamma \geq 0$ the vortex energy never increases.

The docstring of the module is exact about the scope of what has been formalised: “What is formal is that the estimators used on the data are identities of the model.” That sentence is the point of this section. The identity is a fact about solutions of the model; whether a track extracted from a simulated field is such a solution is a different question, which must be put to the data. The hypothesis `hr` demands a derivative at every time. A vortex with $\eta > 0$ has none: its path is a diffusion. The theorem does not apply to a thermal field, and nothing in the proof assistant objects when a script applies the estimator to one.

What the solver shows

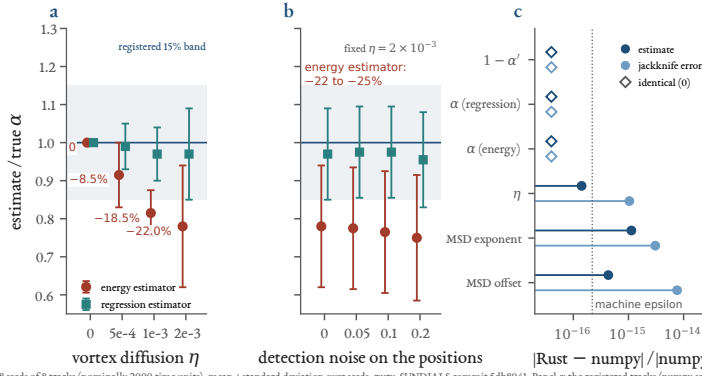
To find out what the estimator does when the hypothesis fails one needs a case in which the truth is known. That is the registered synthetic gate `Go` (Chapter 7 gives its role in the campaign): tracks are generated from the model (10.1) with $\alpha = 0.02$, $\alpha' = 0.10$, $\eta = 2 \times 10^{-3}$ and detection noise added to the positions, and the estimators must return the numbers that were put in. We ran it with the rusty-SUNDIALS example `g0_scan`, a Rust program that contains both the Langevin generator and the estimators of the `qf-pgpe` crate, on seeds $0, \dots, 7$ (the seeds are deterministic, so the run can be repeated exactly).

RUSTY-SUNDIALS THE REGISTERED SYNTHETIC GATE `Go`, MANY SEEDS

```
$ g0_scan 8 --eta-scan      # rusty-SUNDIALS example; detection noise 0, vortex
                             diffusion eta swept; seeds 0..7
```

noise	alpha_energy (mean +- sd)	alpha_regression	1-alpha'	eta	[truth
0.02, 0.90, 2e-3]					
noise 0.00 eta 0e0	0.0200 +- 0.0000	0.0200 +- 0.0000	0.9000	8.76e-11	
noise 0.00 eta 5e-4	0.0183 +- 0.0017	0.0198 +- 0.0012	0.9000	4.98e-4	
noise 0.00 eta 1e-3	0.0163 +- 0.0012	0.0194 +- 0.0014	0.9000	9.87e-4	
noise 0.00 eta 2e-3	0.0156 +- 0.0032	0.0194 +- 0.0024	0.8994	1.93e-3	

Three things are visible in figure 10.1. First, in the noise-free model the energy estimator is exact (0.0200 against 0.02), as the theorem says it must be: the proved identity and the number agree where the hypothesis holds. Second, as soon as the vortices diffuse the estimator is biased low, by 8.5, 18.5 and 22.0 percent at the three diffusions tried, and the registered criterion (within 15 percent) is met on only 2 of 12 seeds in the programme’s own count of this test; the gate had passed on, in the



Panels a, b: Rust example g0_scan, 8 seeds of 8 tracks (nominally 2000 time units), mean $\pm$ standard deviation over seeds; rusty-SUNDIALS commit 54b8041. Panel c: the registered tracks (numpy seed 20261005), analysed by both implementation

Figure 10.1: The energy estimator of the friction fails a known-answer test that it had passed once. **(a)** Estimates divided by the true $\alpha = 0.02$ as the diffusion η of the model vortices grows, mean and standard deviation over 8 seeds of 8 synthetic tracks (Rust g0_scan); the band is the registered 15 percent criterion. The identity of the theorem energy_dissipation is exact at $\eta = 0$ and the energy estimator is exact there; the regression estimator stays near the truth. **(b)** At $\eta = 2 \times 10^{-3}$ the bias does not depend on the detection noise added to the positions. **(c)** The same eight tracks analysed by the numpy estimators and by the Rust estimators: relative difference of every estimate.

ledger’s words, “one lucky seed”. Third, the bias is not caused by the detection noise: it is already there with none, and the energy estimator gives 0.0156, 0.0155, 0.0153 and 0.0150 at detection noise 0, 0.05, 0.1 and 0.2 per coordinate (panel b). The regression estimator, which uses the same tracks, returns 0.0198, 0.0194 and 0.0194 at the same three diffusions and 0.8994 for $1 - \alpha'$ (truth 0.90). Why the energy estimator is biased we do not know. The programme’s note suggests that the stochastic part of $H(t)$ is correlated with the cumulative regressor $\int 2 \sum_i |\mathbf{v}_{s,i}|^2 dt$, which is computed from the same noisy path; that is a hypothesis, and is marked as one in the note.

Panel (c) records what the second implementation did. The numpy estimators of the programme and the Rust estimators of qf -pgpe were run on identical tracks and agree to $1 \text{e-}15$ in all six estimates and to $8 \text{e-}15$ in their jackknife errors; on four real tracks the software paper reports 2×10^{-14} [21]. On the registered tracks themselves (numpy seed 20261005), both implementations return $\alpha_{\text{energy}} = 0.0187$ — the number with which the gate was recorded as passed — and $\alpha_{\text{regression}} = 0.0210$ with $1 - \alpha' = 0.9006$. The agreement is the reason the Rust port could be trusted to expose a defect, and the reason it could not have exposed this one by itself: two implementations of one estimator agree on its bias.

What it cost, and what it did not

The programme’s published friction values were computed with the energy estimator (CLAIM-098 of the ledger). The regression estimator is larger than the energy estimator by 5 to 21 percent in the six measured arms, in exactly the direction of the bias, and the absolute values of α quoted in the first paper [23] were therefore low by roughly 5 to 20 percent. The coefficient of the temperature

law α/T rises from 0.054 (energy) to 0.060 (regression). The registered verdicts are comparisons between arms analysed with the same estimator, so none of them changes; both papers state the correction, the first in its results on the friction [23] and the second in its section on limits [83].

THE RULE

A theorem is true of its hypotheses. Before an identity of the model is used as an estimator on data, test the estimator where the answer is known, *over many realisations*, and with the properties of the data (here: diffusion) switched on. This is the programme’s rule LL-15 — transfer a result with its hypotheses, not just its conclusion — applied to a theorem instead of to a measurement.

10.2 What each check can and cannot see

The programme uses five kinds of check. Table 10.1 sets, for each, an instance in which it caught something and an instance, from the same programme, in which it could not. None of the instances is hypothetical.

Two observations. The checks are not ordered by strength: each answers a different question — is the statement what we think it is, does the code compute what its author intended, does the binary that runs contain that code, was the verdict chosen before the data, does anyone else’s data agree — and passing one is silent about the others. And a check is credited only when it has been *run*. The programme’s habit, visible in every row of the table, is to ask of a result *which* of these it has passed, and to write the answer down before anyone cites the result.

10.3 The audit of the library

A sentence such as “the library has 310 theorems, none uses `sorry`, all use only the three standard axioms” is a claim about a build, and it can be checked only by building. The programme’s records show how easily it drifts. In September an external review of the v1.1.0 tag found that `DualLength.lean` declared 13 theorems while the reported count was 12; looking at all the files showed that 76 theorems existed where 65 carried a `#print axioms` line, so 11 had never been audited and every published count had conflated “theorems” with “theorems carrying an audit line” (RETRACCTIONS.md, R3). The repair was a generator, `scripts/regen_axiom_audit.py`, which rewrites the audit block of every file and has a read-only - - check mode.

For this book we audited the library again, differently. The driver (appendix B; `book/facts/lean_audit.py`) compiles each module, in the order of its imports, against the pinned Mathlib (Lean 4.34.0-rc2, Mathlib tag v4.34.0-rc2), after appending to a scratch copy about twenty lines of Lean — new for this book, printed in appendix B — that walk the kernel environment of the module, ask `Lean.collectAxioms` about *every constant the file declares*, and print the union. It audits the environment, not the text: it needs no `#print axioms` line in the source. Like any auditor it has to be seen to catch something. Run on a file with planted defects it reports a `sorry`, the same `sorry`

check	caught, in this programme	could not see, in this programme
the Lean kernel and the axiom audit (Comparator: two kernels)	A count that had drifted: 76 theorems against 65 audit lines (RETRACTIONS.md, R ₃). A bug of the generator of the challenge files, which cut one statement at a := inside a named argument, found while adding SectorDuality (CLAIM-052).	That the hypothesis of a true theorem fails in the data (section 10.1). That a true statement is empty: the dual-length bound is $x + 1/x \geq 2$ and “could not have come out otherwise”. That a statement is the right physics: “a Comparator pass says the solution proves the challenge’s statement, not that the statement is the right physics” (docs/COMPARATOR_SETUP.md). That the hypotheses of a true theorem are met only by trivial cases: those of KTFLOW (below, and Chapter 6).
a second implementation	A vortex centred on a grid node imprinted with modulus one instead of zero, in the first Rust version. A defect of the Adams order in the CVODE library, exposed by the first Hamiltonian system run through it. The estimator bias, once a seed sweep was cheap.	An error shared by both: numpy and Rust agreed to 2×10^{-14} on four real tracks and both were biased. A platform effect that one platform hides: a $T = 0$ control run that agrees to 10^{-13} on x86-64 differed by 8×10^{-3} in position on an Apple-silicon runner.
a registered test	47 of 113 criteria not met (figure 10.3), each against a threshold written down beforehand.	A criterion that cannot fail for the reason intended: the residual criterion D-G2” was “mis-specified, since the vortex term is 500× the phonon term”. A battery of six criteria that all tested deterministic reproducibility in a chaotic model and none statistical reproducibility (LL-14).
the solver library’s own tests	Nine tests of the Adams method of CVODE, written <i>before</i> the fix of its defect (the method was implicit Euler); eight of them fail on the unfixed code (crates/cvode/tests/adams_order.rs, docs/CVODE_ADAMS_FIX.md).	Which binary a script imports. The Python module installed in the shared environment of this book was built on 26 September, before the fix of 28 September: $y' = -y$ to $t = 10$ at relative tolerance 10^{-10} cost 2 214 597 right-hand-side calls and returned an error of 6.9×10^{-5} , against 616 calls and 1.7×10^{-9} for the build of commit 5db8041. The tests had run on the source. A convergence check in a chapter found the stale binary; no test did.
independent data	Two facts about a published reference run, found by trying to reproduce it: an $O(\Delta t)$ convention in its velocity ramp, and an initial field that is the Thomas–Fermi field after one imaginary-time step plus noise (the stopping rule is met at the first step).	Anything beyond the window reproduced: not $t > 50$, not the Strouhal number, not single precision against discretisation; the model had to be recovered from the code, not from the paper’s text.

Table 10.1: Five kinds of check, each with an instance of what it caught and of what it missed. Sources: RETRACTIONS.md, LEDGER.md (CLAIM-052, 079, 092, 098, 103, 106), LL.md (LL-14), the software paper [21], docs/CVODE_ADAMS_FIX.md of rusty-SUNDIALS, and the probe of appendix B.

a

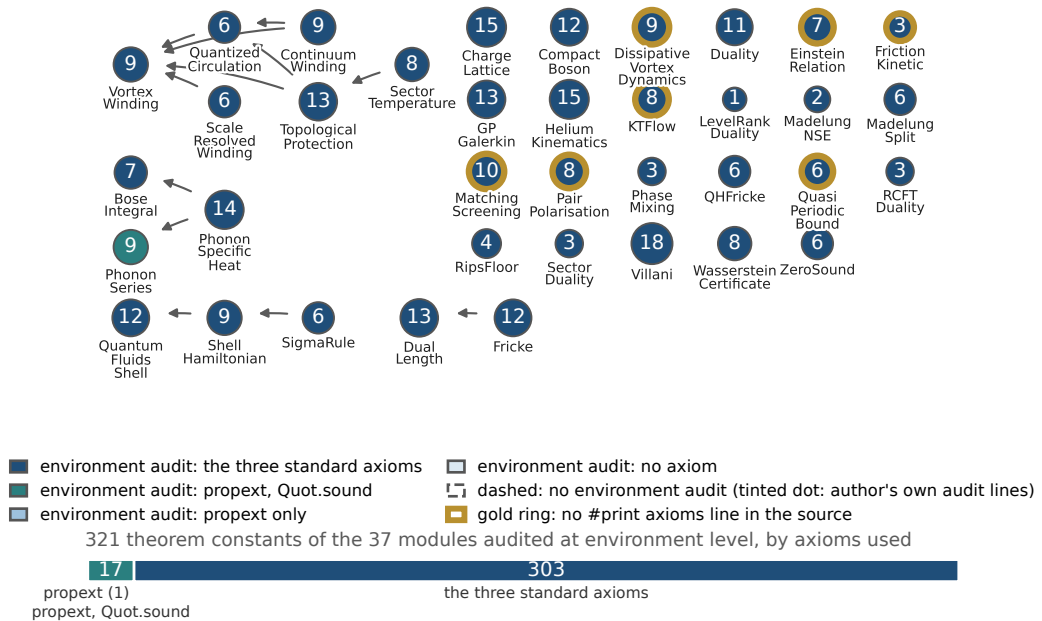


Figure 10.2: The library as audited for this book. **(a)** The 37 modules, placed by the import relation (arrows point from a module to the modules it imports), the size of a disc growing with the number of theorems of the module (printed inside); fill = the axiom footprint found by the environment audit, a dashed outline = no current environment audit (no module when the figure was drawn), a gold ring = the module's source carries no `#print axioms` line of its own (7 modules). **(b)** The axiom footprint of every theorem constant of every module, from the environment audit.

inherited by a theorem whose own proof contains none, a `sorry` in a private theorem, an axiom declared by the module and a use of `native_decide`, and it reports the clean theorems as using at most `propext`, `Classical.choice` and `Quot.sound` (`facts/audit_controls/`). A `grep` for the word `sorry` in the compile log finds two of the three theorems that depend on a `sorry`; the third has no warning of its own.

What the audit found. Five things are worth telling.

The first run of our own driver reported nine failures, and none was real. It compiled each file separately, inside the Mathlib tree; nine modules import a sibling (`import VortexWinding`) whose compiled file was not on that tree's search path, so Lean answered `unknown module prefix`. The driver of this book builds in dependency order and puts the compiled siblings on the path.

Seven modules carry no #print axioms line, and their compile logs are therefore empty: `DissipativeVortexDynamics`, `EinsteinRelation`, `FrictionKinetic`, `KTFLOW`, `Matching Screening`, `PairPolarisation` and `QuasiPeriodicBound`. An empty log of a module that compiles is not a timeout and not a failure; it is a module that was never asked. Asked (table 10.2),

module	theorems	Lean s	sorryAx	axioms used
DissipativeVortexDynamics	9	38	no	propext, choice, quot
EinsteinRelation	7	33	no	propext, choice, quot
FrictionKinetic	3	88	no	propext, choice, quot
KTFlow	8	54	no	propext, choice, quot
MatchingScreening	10	41	no	propext, choice, quot
PairPolarisation	8	55	no	propext, choice, quot
QuasiPeriodicBound	6	55	no	propext, choice, quot

Table 10.2: The modules whose sources carry no `#print axioms` line, audited at environment level: theorem constants declared in the file, Lean wall time in seconds on the shared machine (load 8–35, so an indication only), whether `sorryAx` is reachable, and the axioms used (choice = `Classical.choice`, quot = `Quot.sound`).

each of the seven compiles without error and depends on at most the three standard axioms; no `sorry` is reachable from any of their 51 theorems. The staleness check introduced after R3 (`python3 scripts/regen_axiom_audit.py --check, read-only`) flags exactly these seven library modules (and four scratch files): the drift that R3 repaired reappeared within three weeks of its repair, for the same reason: the check exists, and no script, workflow or git hook of the repository calls it. The programme’s gate script `scripts/verify.sh` builds a single module (`QuantumFluidsShell`) and searches the log for `sorryAx`. The same seven modules are also the ones that the Comparator list (30 modules, 257 theorems, accepted by the Lean kernel and by nanoda [99, 100]) does not contain.

The generator that repaired the count has a blind spot of its own. It recognises a theorem only at the start of a line, so `crossover_apply_mem` and `crossover_apply_not_mem` in `Villani.lean`, declared as `@[simp] theorem`, were never given an audit line: the generator counts 308 theorems where the index of the library lists 310. The generator of the Comparator’s challenge files keeps the theorems that carry an audit line, so it inherits the blind spot: the 257 theorems that the Comparator accepted are the 257 listed ones, out of 259 in the thirty modules it covers (the two missing are the two above). Asked about them, the environment audit finds that both depend on exactly the three standard axioms and on no `sorry`.

One module is outside the build. `FrictionKinetic.lean` appears neither in `lakefile.lean` nor in the umbrella file `QuantumFluids.lean`: `lake build` does not build it, and `import QuantumFluids` does not give its theorems.

The index of the library, from which this book’s citations of theorems were first taken, was wrong about the names. It said that every module lives in the namespace `QuantumFluids.Module`. For 13 of the 37 modules it does not: the seven newest modules sit in bare namespaces (`KTFlow.kt_trapped`), `Duality` sits in `Mathesis.Duality`, `QuantizedCirculation` in `QuantumFluids.Circulation`, and `QuantumFluidsShell`, `ShellHamiltonian` and `SigmaRule` in `QuantumFluids.ShellComplex`. The error was found only when the catalogue of appendix A was generated from the sources instead of typed, and the book’s theorem-citation macro now looks the

namespace up. It is the same kind of error as the drifting count: a statement about the library that nobody had compared with the library.

The result. We audited 37 of the 37 modules at environment level. They declare 321 theorem constants with a source position, against 310 theorem or lemma keywords in their sources, and Lean itself generated 42 more, which have no source position (equation lemmas and the like). No module declares an axiom of its own, sorryAx is reachable from no constant of any of them, and every constant depends on at most propext, Classical.choice and Quot.sound. By the set of axioms a theorem depends on: 303 use all three; 17 use propext and Quot.sound; 1 uses propext only. The surplus of theorem constants over keywords is 11: 10 proof-valued structure fields (such as Balanced.invol of VortexWinding), which Lean records as theorems, and 1 instance whose value is a proof (the lists are in facts/lean_audit.md). Modules whose author-written audit lines are fewer than their theorems: Villani (2 without). The Lean files written for this book (the last section of appendix A) were audited the same way, each compiled against the pinned Mathlib (and, for Ch04_DoSLink, the library modules that the file imports, as built by the audit): 10 of 10 files, 116 theorem constants. None declares an axiom of its own; sorryAx is reachable from no constant of any of them; every constant depends on at most the three standard axioms. The files Ch02_NegativeControl, Ch04_NegativeControl_D15121, Ch04_NegativeControl_L54, written to fail and listed apart in appendix A, were compiled expecting errors: 3 of 3 failed as intended.

What the audit does not do. It reads the environment that Lean’s elaborator produced and trusts Lean’s kernel; the second kernel is the Comparator’s business, and we did not re-run Comparator for this book (its results are those recorded in docs/COMPARATOR_SETUP.md). It says nothing about whether a statement is the right one; the statements are in appendix A for a human to read, and a Comparator pass shows the same statement only because, in the words of the programme’s own note, “‘same statement’ is guaranteed by construction” when the challenge file is generated from the solution. And it is only as complete as the list of modules: the umbrella file and the lakefile are part of what has to be kept in step.

10.4 What failed, plainly

The ledger of the programme (LEDGER.md) numbers its claims; in 56 days, from 2026-08-14 to 2026-10-09, it reached claim 106. Its entries carry statuses, and a good many of the statuses are the words *retracted*, *withdrawn*, *refuted* or *void*. Only 18 of the 106 claims are in the format that the ledger’s own checker (scripts/check_ledger.py) reads; the rest are in a second, tabular format that it does not see — a limitation of the machine check that the ledger itself records (CLAIM-066). What follows is therefore a reading of the text, not a statistic.

A map of the verdicts. Figure 10.3 draws every criterion that the ledger records one by one, for 13 of the programme’s campaigns. There are 113 of them: 53 were met as registered, 47 were not,

11 were mixed, partial or inconclusive, one was voided afterwards and one was not run. The criteria are not independent and not equally important — the kinetic benchmark contributes 25 cells, the finite-size ladder 7 — and the selection rule is the only one available to us: the campaigns whose per-criterion outcomes the ledger gives. It shows how often a criterion written down before the data was not met, among those the ledger itemises: nearly as often as it was met.

September: the literature check. On 20 September the programme pre-registered a hypothesis, the *dual scale* of a quantum fluid, stated it in Lean, and refuted it for helium II by measurement; the release of that day is tagged “stated, proved, and refuted for He-II”. The same day the literature check that the proposal itself had listed as blocking any claim of novelty was run, and it went against the programme on both counts (RETRACTIONS.md). The second invariant of the complexified shell model was a rediscovery of structures in Vladimirova, Shavit and Falkovich’s *Fibonacci turbulence* and in a second paper by overlapping authors on second-harmonic generation as a minimal model of turbulence [101, 102]; on the dyadic lattice the family of conservation laws collapses to a single invariant, so the model has *less* structure than the published one. The dual length $\ell(k) = \varepsilon^2 / (\hbar^2 c^2 k^3)$ was a repackaging of the static response, its “bound” was the inequality between the arithmetic and the geometric mean, it was weaker than the known linear bound of Onsager’s inequality [103] outside a narrow window of wavenumbers, and its interior minimum was dimensionally forced; the “refutation” for helium was structural, since any fluid with a roton violates it. The file states the outcome in one sentence: “No novel scientific contribution survives in the dual-scale programme.” What survived were the Lean proofs, as correct mathematics of known results, the measurements, as correct measurements of textbook physics, and a record of method. The same file lists a repository defect (a symlink to a local disk, committed by mistake, which broke every clone) and the counting error of section 10.3.

Failures of our own design. Not every failure came from outside. Lessons LL-7 to LL-14 of the programme’s LL.md record what the first weeks cost. A table of literature constants entered the roton gap in kelvin and used it as millielectronvolts, overstating it by the Boltzmann-constant factor, about 11.6, and a conversion constant was off by exactly 2; the synthetic tests had been generated with the same wrong constants they checked, and both errors surfaced only against real, digitised data (LL-7). Values of the phonon–roton (Landau) parameters recalled from memory were attributed to two papers that report no such values; they were within about 0.3 percent of the true ones, which is why they survived (LL-10). An exponent fitted to the supremum of the enstrophy was a property of the horizon chosen, because the supremum does not converge in the horizon for a conservative regulator, and every quality signal — $r^2 > 0.96$, disjoint confidence intervals, a timestep-refinement criterion passing at 0.00 percent — was good (LL-11). The thermalisation time of the chaotic model has a coefficient of variation of 23 to 49 percent from one realisation to another at fixed parameters, and four rounds of results had come from a single trajectory (LL-14). And a lesson that is not about physics: a destructive check of the Lean gate, which inserts a sorry into a source file, was launched as one background command, outlived its foreground timeout, and

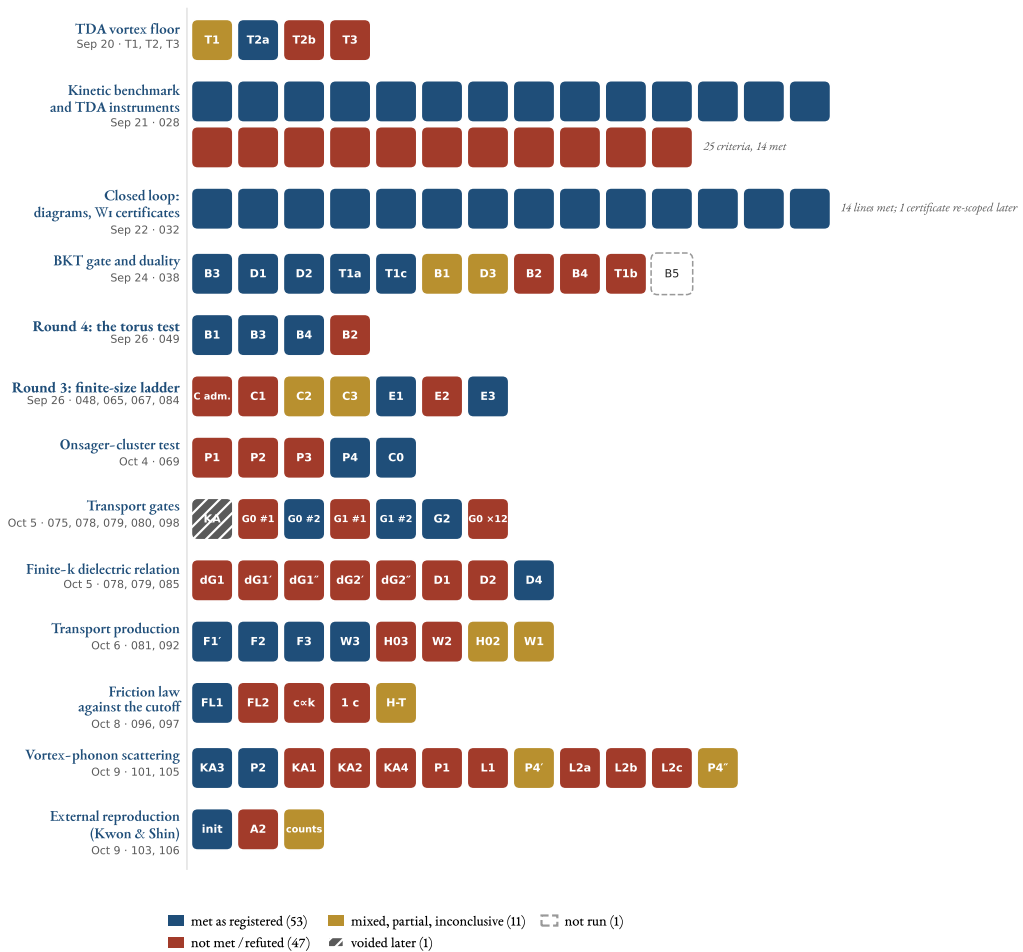


Figure 10.3: Every criterion that the ledger records one by one, by campaign (numbers are ledger claims; figure script figures/ch10_verdicts.py checks each row against LEDGER.md). Blue: met as registered; red: not met or refuted; gold: mixed, partial or inconclusive; hatched: voided afterwards; dashed: not run. Unlabelled cells are criteria the ledger counts but does not name. Labels are those of the registration documents: for instance P1–P3 are the registered predictions of the Onsager-cluster test (all three failed), Go/G1 the gates of the transport campaign (each failed at its first attempt), D1/D2/D4 the criteria of the dielectric test, FL1/FL2 those of the friction-law test, L1/L2a–c those of the vortex-phonon scattering test. The cell G0×12 is the energy criterion of Go re-run over many seeds (section 10.1).

the file still contained the `sorry` when a commit was being prepared; “the margin was one grep” (LL-13).

October: the campaign. The pre-registered campaigns of October produced most of the cells of figure 10.3, and the ledger reads like a lab book of an apparatus that did not work the first time. The friction known-answer gate of CLAIM-075 was recorded as passed on 5 October and withdrawn the same day, when the first gates of the transport campaign were run, because the vortex imprint of the instrument added an amplitude tail four times the Bernoulli depletion and a phase step along a whole line of the box, and the apparent friction at $T = 0$ was as large as the thermal value being measured (CLAIM-078). The counterflow control W2 was refuted, one run of four meeting the no-stall criterion (CLAIM-092). The friction law “ $\alpha \propto \rho_n$ ” was found to be a degeneracy of one cutoff, since $\rho_n \propto T$ there; across three cutoffs the coefficient is 0.31, 0.23 and 0.10 and only α/T is constant (CLAIM-096), and the paper that had published the first reading, together with a “geometric core size of 1.4ξ ” derived from it, had to be corrected in version 2.2. The finite- k dielectric relation, tested mode by mode, held in one run of six; an earlier correlation of 0.996 across runs had been carried by box-scale pairs (CLAIM-085). The vortex–phonon scattering cross-section depended on the pair separation and on the box, and the registered conclusion is that an isolated-vortex $\sigma_{\parallel}(k)$ “is not defined by this observable at these geometries” (CLAIM-105).

Two failures were failures of process rather than of physics. A verdict of “saturation” for the finite-size offset at $L = 192$ (CLAIM-067, admission 3 of 6) was recorded as unresolved, and a longer run (CLAIM-084, admission 6 of 6) showed that it had rested on states that had not equilibrated; the finite-size trend of the whole ladder was then, in the ledger’s words, under the same suspicion. And the first Python attempt at those $L = 192$ runs died silently after about 55 hours, with nothing in its log; the cause could not be diagnosed, an out-of-memory kill being “the leading, unconfirmed hypothesis”. The runs were redone with a Rust port of the integrator, which itself stopped once without an error and was resumed from a checkpoint.

A third process failure happened while this book was being written. The Python module `rusty_sundials` installed in the shared environment had been built on 26 September (the stale module of the probe of appendix B), two days before the Adams method of the solver library was repaired; in that build the method is implicit Euler, and its work grows like $\text{rtol}^{-1/2}$. The library’s tests, written before the repair, fail on the old code, but they test the source, not the file that a script imports. The defect was caught by a convergence check in one chapter. The probe of appendix B (Adams on $y' = -y$ to $t = 10$ at relative tolerance 10^{-10}) separates the two builds: 2214597 right-hand-side calls and an error of 6.9×10^{-5} before, 616 calls and 1.7×10^{-9} after. No computation of this chapter used CVODE through that module; the lesson is the one of section 10.1 applied to a binary: a check certifies the thing it was run on.

Two statements that stayed true. Not everything in the middle of a failed campaign fails. Two Lean statements of the transport campaign illustrate what a proved statement does *not* change.

when	what was withdrawn or corrected	what had failed	rule or repair
Aug 14–15	exponent comparison, ordinal-censoring result (CLAIM-R ₁ , R ₂ , O ₁₂ , O ₁₃)	horizon not converged; one trajectory of a chaotic system	LL-11, LL-14: converge in the horizon; measure the ensemble spread first
Aug	prediction exported to a sister project (CLAIM-O ₁₇)	tested in their code before it was sent: exponent stable to four decimals over 8× horizons	LL-15; retracted the same turn
Sep 20	second invariant, dual length, σ -rule (CLAIM-O ₂₃ , O ₂₄ , O ₂₅)	rediscovery; weaker than the known bound; dimensionally forced	RETRACTIONS.md; no novelty claimed
Sep 20	TDA vortex floor (CLAIM-T ₁ , T ₂ , T ₃)	confounded by the segmentation threshold; split verdict; refuted	threshold-free line tracing; “nothing from this run is citable”
Sep 21	kinetic benchmark and TDA instruments (CLAIM-O ₂₈)	11 of 25 criteria not met; Lean formalisation of nonlinear Landau damping called “out of reach” six days after it had been posted [89]	literature gate before any claim of difficulty
Sep 22	Wasserstein certificate, phase-space pair (CLAIM-O ₃₂ erratum)	computed on 5 points per side instead of the registered 30; the same class of defect as a point list changed between workflow stages, found the same day	all pairs recomputed independently; erratum carried into the next paper
Oct 5	friction known-answer gate (CLAIM-O ₇₅ → O ₇₈)	imprint with an amplitude tail and a phase step	new imprint; gates G ₀ –G ₂
Oct 8	counterflow control W ₂ (CLAIM-O ₉₂ , O ₉₅)	1 of 4 runs met the criterion; a template-filled paragraph contradicted the verdict	explanation withdrawn, observation kept; text corrected
Oct 8–9	$\alpha \propto \rho_n$ and a core size of 1.4ξ (CLAIM-O ₉₆ , O ₉₉)	coefficient depends on the cutoff; $\rho_n \propto T$ degeneracy	paper corrected (v2.2); α/T reported
Oct 9	energy estimator of α (CLAIM-O ₉₈)	biased low by diffusion; gate passes on 2 of 12 seeds	regression estimator; gates run over seeds

Table 10.3: What was withdrawn or corrected, why, and the rule or repair that followed. Every row is an entry of LEDGER.md, RETRACTIONS.md or LL.md.

LEAN 4 QUASIPERIODICBOUND

```

noncomputable def qp (a ω φ : ι → ℝ) (t : ℝ) : ℝ := ∑ k, a k * Real.cos (ω k *
  ↪ t + φ k)

theorem not_qp_of_msd_large (a : ι → ℝ) {J : Type*} [Fintype J] [Nonempty J] (
  ↪ x : ℝ → ℝ) (ts ts : J → ℝ)
  (h : (2 * ∑ k, |a k|) ^ 2 < (∑ j, (x (ts j + ts j) - x (ts j)) ^ 2) /
  ↪ Fintype.card J) :
  ∀ ω φ : ι → ℝ, x ≠ qp a ω φ

```

The first is the logic of a discriminant. A vortex pair whose motion is a regular island has quasi-periodic coordinates, finite sums of cosines; the statement says that if the sampled mean-square increment of a signal exceeds $(2 \sum_k |a_k|)^2$, the signal is not such a sum for *any* frequencies and phases with those amplitudes. The docstring is explicit that the decision rule is “sound in one direction and silent in the other”: growth refutes the island reading, a plateau proves nothing. On the data the registered criterion I2 was read as “not an island” — the separations of the stalled pairs carried a third of their variance in their three largest spectral lines, less than the registered half (CLAIM-o86) — and nothing about the statement had to be revised.

LEAN 4 FRICTIONKINETIC

```

noncomputable def alpha (K : Finset ι) (w f : ι → ℝ) (ps κ : ℝ) : ℝ := (∑ k ∈
  ↪ K, w k * f k) / (ps * κ)

noncomputable def rhoN (K : Finset ι) (w : ι → ℝ) : ℝ := ∑ k ∈ K, w k

theorem coeff_eq_weighted_mean (K : Finset ι) (w f : ι → ℝ) (ρ ps κ : ℝ) (hpn
  ↪ : rhoN K w ≠ 0) (hp : ρ ≠ 0) :
  alpha K w f ps κ / (rhoN K w / ρ) = ρ / (ps * κ) * ((∑ k ∈ K, w k * f k) /
  ↪ ∑ k ∈ K, w k)

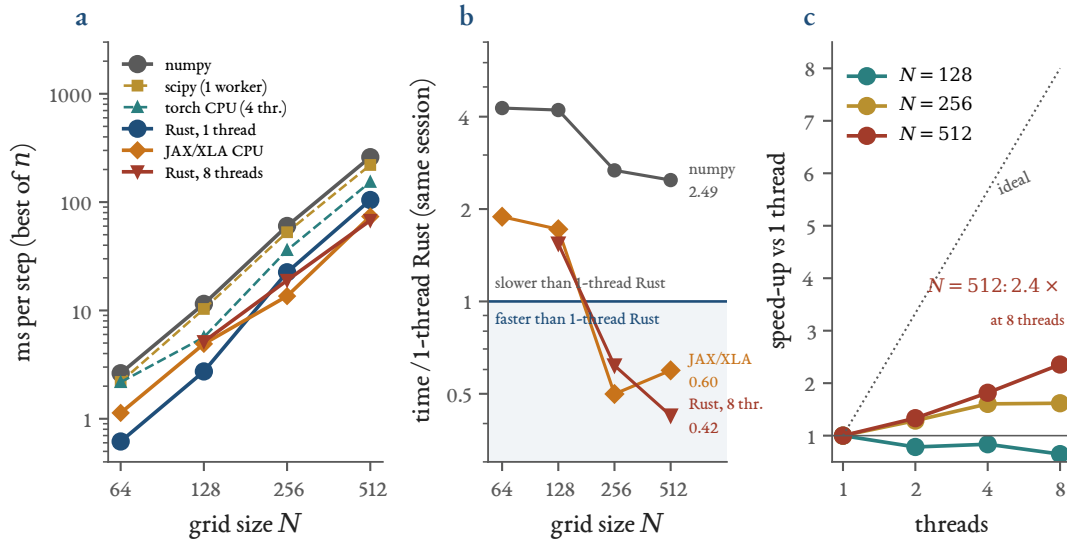
```

The second is an identity of a kinetic model of the friction: if the drag and the normal density are sums over the same bath modes $k \in K$ with the same nonnegative weight w_k , the drag carrying one more factor f_k (group velocity times transport cross-section), then the ratio $\alpha/(\rho_n/\rho)$ is a constant times the w -weighted mean of f over the modes (the proof is algebra: abridged here). What the statement says about the world is only that this coefficient is a property of the *bath*: it can depend on the modes retained. It does not say that it is independent of the cutoff, and when the registered expectation that it nearly is (spread below 0.2) failed — the coefficient was 0.31, 0.23 and 0.10 at the three cutoffs — the identity was not affected. What was withdrawn was a reading of the number measured at one cutoff, a “geometric core size of 1.4ζ ”. The theorem is a statement about the model, the 1.4ζ was a statement about the world, and only the first had been proved.

A statement that was true and nearly empty. A third statement was true, and its trouble was of the opposite kind: in the estimator story a hypothesis failed in the data, here a hypothesis can hardly be met by anything. It was found while Chapter 6 was being prepared. Three theorems of KTFLOW (KTFLOW.kt_invariant, KTFLOW.kt_trapped and KTFLOW.kt_fugacity_bounded) are about a solution (u, y) of the Kosterlitz flow, and the file asks for the two flow equations and for $u > 0$ at every real scale l , negative ones included (the hypotheses hu , hy and $hpos$ are quantified over all $l \in \mathbb{R}$), whereas a renormalisation trajectory is a solution for $l \geq 0$ only. Chapter 6 proves, in a new module of the book, that on the superfluid side these hypotheses leave almost nothing: if $u(0) < \pi/2$ then $y(0) = 0$. (If $y(0) \neq 0$, the invariant forces $y(l)^2 \geq y(0)^2$ for $l \leq 0$, so going backward u would fall at least linearly and be negative before $l = -u(0)/c$, with $c = 4\pi^3 y(0)^2$, which the hypothesis $u > 0$ forbids.) From $y(0) = 0$ the uniqueness of the solution of the linear equation for y gives $y \equiv 0$ and u constant — a standard step, not formalised — so only fixed points of the flow meet the hypotheses there. The three theorems are true, and the kernel certified exactly what had been written; what had been written asked too much of a solution to be applied to one that matters. The library file is unchanged; Chapter 6 restates the theorems under hypotheses on $l \geq 0$ only, and adds the converse.

A name that was copied, not checked. The 2021 paper of Godfrin and his co-authors [2] has seven authors. While this book was written its record at the publisher (Crossref) was compared with the programme's own notes, and two of them gave the list wrongly: the literature ledger wrote the second author's initial as M instead of K, and a design note, like the docstring of the library module PhononSeries, named an eighth author who is not on the paper. No test could catch a name, and none did; the slips were found by looking the record up. They were corrected on 10 October 2026 (in the library the change is a comment, so no theorem is touched); the copy of that module that is generated for the Comparator repeats the old docstring until its generator is run again. It is the lesson of the other paragraphs at the smallest scale: a copy of a copy is not a source.

What the failures have in common. The following is our reading, not a measurement. Most of the entries of table 10.3 fall into three kinds: a result used outside the hypotheses on which it depended (the estimator; the prediction exported to a sister project; the friction law outside one cutoff); one realisation mistaken for an ensemble (the single trajectories of August, the single seed of Go, a single seed per energy at $L = 192$); and a gate or a criterion that could not fail for the reason intended (the battery of LL-14, the imprint that made the known-answer gate pass, the residual criterion D-G2"). They are the same three mistakes at three scales, and each has a cheap check, which is what the rules of the programme are: write down the hypotheses and check them in the target (LL-15), measure the spread before fitting (LL-14), and run the gate where the answer is known with the property switched on (section 10.1). They are also the reason that pre-registration [104] is only half a method: a registered criterion makes the verdict impossible to bend, and does nothing about whether the criterion could have failed for the right reason.



Machine: Intel Core i7-4930MX (2013, 4 cores / 8 threads), shared, load average about 7 while measuring, no GPU/TPU.

Absolute times are pessimistic and thread scaling a lower bound; the same JAX case varied 8.8 vs 4.9 ms between sessions. GPU/TPU: not measured.

Figure 10.4: Time per step of the projected Gross–Pitaevskii integrator in several engines (best of n repetitions). **(a)** Absolute time against grid size N . **(b)** Time relative to the one-thread Rust step measured in the same session, so that load common to both cancels in part; values below one are faster than serial Rust. **(c)** Speed-up of the threaded Rust step against its own one-thread time. All measurements were taken on one shared machine (Intel Core i7-4930MX, 2013, 4 cores and 8 threads, load average ≈ 7 , no GPU); the same JAX case gave 8.8 ms in one session and 4.9 ms in another. Data: `data/generated/pgpe/bench/`.

10.5 Speed is a claim too

The solver of this book is offered as a tool, and a tool has a speed; claims of speed are checked less often than claims of physics. The benchmark of the software paper [21] times one step of the projected Gross–Pitaevskii integrator on an $N \times N$ grid ($L = N/2$, $g = 1$, $\Delta t = 0.01$), identical in every engine; the final states agree in a checksum (to $1e-13$ for JAX against Rust), so the timings are for the same computation.

On one thread the Rust engine takes 0.62, 2.73, 22.52 and 104.60 ms per step at $N = 64, 128, 256, 512$, against 2.6, 11.5, 60.2 and 260.0 ms for numpy: 4.3, 4.2, 2.7 and 2.5 times faster, the advantage shrinking with N . A JAX/XLA version on the CPU is slower than serial Rust at $N = 64$ and 128 and faster at $N \geq 256$; pinned to one core the order reverses, so the XLA lead at large N is threading and not a better algorithm. Threading the Rust step helps at $N \geq 256$ ($2.4\times$ at $N = 512$ on eight threads, $1.6\times$ at $N = 256$) and *hurts* at $N = 128$: 5.15 ms on eight threads against 3.34 ms on one, so for small grids the threading must stay off.

What is not claimed matters as much. There is no comparison with FFTW-based C codes, with GPU codes or with any other published solver, and no measurement on a GPU or a TPU; the

machine was shared, so absolute times are pessimistic and the scaling is a lower bound; the statistic is the best of a few repetitions, taken in the same way for every engine, in two sessions that differ by up to a factor of two for the same case. “Fast” in this book means “faster than numpy on this machine, for this problem, on one thread”. It does not mean “the best solver”.

10.6 What a reader can reuse

The library. The Lean library [12] is 37 modules and 310 theorems and lemmas, with 103 definitions, structures and abbreviations (catalogue in appendix A). It is pinned to Lean 4.34.0-rc2 and to the Mathlib tag of the same name, and is licensed MIT or Apache-2.0. A reader who wants a result, rather than a method, should look first at the modules whose subject is mathematics with a physical reading: `BoseIntegral`, `PhononSeries` and `PhononSpecificHeat` (the coefficients of the phonon specific-heat series), `ZeroSound` (when an undamped zero-sound root exists), `VortexWinding` and `QuantizedCirculation` (that the way simulation codes find vortices measures an integer), `KTFlow` (the Kosterlitz renormalisation flow). Every module is a top-level module of its own: the umbrella file `QuantumFluids` imports all but one.

GODFRIN THE SERIES OF EQ. (22), AND A REMARK OF ITS AUTHORS

In the paper on the dispersion relation of the Landau excitations of superfluid ^4He , measured by inelastic neutron scattering on IN5 at the ILL, Godfrin and co-authors give in Eq. (22) the phonon specific heat $C_V = AT^3 + CT^5 + DT^6 + ET^7 + KT^8 + LT^9$ for a dispersion $\varepsilon = \hbar ck(1 + \alpha_2 k^2 + \dots + \alpha_6 k^6)$ with $\alpha_1 = 0$, and remark that earlier published versions of this series contain errors [2]. The library has derived the six coefficients independently (with sympy) and proved them in Lean (`BoseIntegral`, `PhononSeries`, `PhononSpecificHeat`): a confirmation of the paper’s Eq. (22), not a finding. What is proved is the mathematical statement, a series coefficient; it is not a statement about the experiment.

A series of six terms, each a polynomial in the coefficients of the dispersion relation, is algebra that a machine can check at small cost; but the same remark applies to it as to everything above: the kernel certifies the algebra, and the physics of the dispersion relation is in the measurement.

The solver. The engine `qf-pgpe` of rusty-SUNDIALS (release v11.6.0; software paper [21]) is a pure-Rust projected Gross–Pitaevskii integrator with a Python extension, `qf_pgpe`, whose `Pgpe`, `imprint_v2`, `detect` and `analyse_tracks` are drop-in replacements for the numpy versions of the programme; `CVODE` [4] is available through `rusty_sundials` (take the build of commit `sdb8041` or later, and check it with the probe of appendix B). What is reusable beyond the code is the verification harness: the cross-language comparison on identical inputs, the `CVODE` reference, run with the repaired Adams method, that confirms the fourth order of the step, and a reproduction test of the Kwon and Shin run [28] on its real download, which runs weekly and on demand outside the required checks (Chapter 9).

The discipline. It is the part of the programme that held up under its own failures, and it is small enough to list.

A CHECKLIST, FROM THE PROGRAMME'S PRACTICE AND FROM THE MAKING OF THIS BOOK

1. Register the criteria, the decision rule and the kill rule *before* the data exist, in a file under version control [104].
2. Give every instrument a known-answer test, run it over many realisations and with the property of the data that will matter switched on.
3. Have a second implementation, or a second source of data, for every number that will be quoted.
4. Write down the hypotheses of a result before exporting it, and test each in the target (LL-15).
5. Give every constant that stands for a measurement its source, table and uncertainty in a comment beside it (LL-10).
6. Read the literature for the claim of novelty *and* for the claim of difficulty before either is made.
7. Keep a ledger with statuses that include *refuted* and *withdrawn*, and correct a published number in the next version of the paper that carries it.
8. Look a citation up in the publisher's record before it is printed; a copy of a copy is not a source.
9. Make the record reproducible from files: a script for every figure, a source for every number (appendix B lists the scripts of this book) [105].

10.7 What is still open

The following items are open in the programme's own records; none is a promise.

- *The cause of the energy-estimator bias* is not established; the regression estimator is the unbiased one, and a mechanism has to be derived or ruled out (exercise 1).
- *The temperature law of the friction*, $\alpha/T \approx 0.054$ (energy) or 0.060 (regression), independent of the cutoff over the three cutoffs tried, has no theory; the kinetic reading that was meant to explain it was withdrawn, and the single-pinned-vortex follow-up that would define an isolated-vortex cross-section was not started.
- *The finite-size offset* of the Josephson relation is positive at every size when the states are equilibrated, but the $L = 128$ states are now under the suspicion that the $L = 192$ ones were; the trend with L is not established.
- *Counterflow*: the stall observed in the closed box is a bath-driven stall with a $1/f^{0.5}$ spectrum; the explanation that predicted its box-size dependence was withdrawn when the control W2 was refuted.
- *External reproduction*: one published run has been reproduced, to $t = 50$; the plan's own

criterion (10^{-5} to $t = 10$) was not met (4×10^{-5}); the vortex counts agree at 9 of 10 times; nothing is shown for $t > 50$ or for the Strouhal number.

- *Speed*: no idle-machine repetition, no GPU or TPU measurement; JAX is the proposed path to both.
- *Data*: no open numerical table of the Landau parameters of two-dimensional ^3He has been found, in two independent searches; Chapter 8 proves the existence criterion for zero sound, and a comparison with the film measurements of [1] needs those parameters.
- *Housekeeping*: FrictionKinetic is outside the lakefile; seven modules have no audit line and are outside the Comparator list; the ledger has two formats and its checker reads one; the theorems of KTFLOW should be restated for $l \geq 0$ as in the book's module (Chapter 6).

A last remark on what the book is for. The experiments that frame it — inelastic neutron scattering on two-dimensional liquid ^3He [1] and on superfluid ^4He [2] — end in numbers: a dispersion with a roton-like minimum, a specific-heat series. The programme described here has no measurement of helium of its own to offer. What it can offer is a way of making calculations *around* such numbers answerable: a theorem that can be recompiled, a figure that can be regenerated, a failure that can be read in a ledger. That is the only sense in which a book of this kind can be a tribute without claiming a partnership with the people it admires: it tries to be useful to whoever has next to check a series, rebuild a figure, or decide whether a simulated vortex agrees with a measured one.

WHAT THIS ESTABLISHES, AND WHAT IT DOES NOT

Proved (Lean). The theorem `DissipativeVortexDynamics.energy_dissipation` and its companions are theorems of the point-vortex model; they are not statements about a simulated field or about a vortex in helium. For the modules it covers, the environment audit shows that every theorem constant depends on at most `propext`, `Classical.choice` and `Quot.sound`; appendix A marks, module by module, whether that covers the module. It does not show that a statement is the right one. **Simulated.** The estimator bias, the numpy–Rust agreement and the benchmark are results of runs, on synthetic tracks and on one shared machine; they say nothing about helium. **Not established.** Why the energy estimator is biased; whether each Lean statement is the right physics (a human audit, not a kernel check); anything about a GPU. **Analogy only.** Reading the failures of the programme as three recurring mistakes is our summary of the ledger, not a measurement of how often they occur. **Failed or withdrawn** (sections 10.1 and 10.4): the energy-estimator gate that had passed once; the friction known-answer gate of CLAIM-075; the counterflow control W2; the reading $\alpha \propto \rho_n$ with a core size of 1.4ξ ; the novelty of the dual-scale programme; 47 of the 113 criteria drawn in figure 10.3. Nothing in this chapter says that Henri Godfrin, his collaborators or any laboratory has used, reviewed or endorsed this library.

Exercises

Exercise 1. In the plane the vortex energy is $H = -\sum_{i<j} q_i q_j \ln r_{ij}^2$. Apply Itô’s formula to $H(\mathbf{r}(t))$ for the stochastic model (10.1) and show that the diffusion changes the mean drift of H only through $\eta \Delta H$, which vanishes away from coincidences because the logarithm is harmonic. (On the torus the neutralising background adds a constant.) The mean drift of H is therefore *not* where the bias of the energy estimator comes from. Where could it come from? *Hint:* the estimator is a regression of $H(t)$ on a running integral of $S = \sum_i |\mathbf{v}_{s,i}|^2$; compute the covariance of the noise part of H with that integral.

Exercise 2. Write a Lean command that, for the module it is appended to, prints the number of theorem constants whose axiom set is strictly smaller than `{propext, Classical.choice, Quot.sound}`, and run it on `BoseIntegral` and on `PhononSeries`. Which of the two do you expect to give a positive count, and why? Compare with the footprints listed in appendix A. *Hint:* start from the program in appendix B; `ConstantInfo.isTheorem`, `Lean.collectAxioms` and `Name.isInternal` are all you need. One of the two files proves identities in an arbitrary commutative ring by `linear_combination`, the other is real analysis.

Exercise 3. In figure 10.4c the eight-thread step at $N = 512$ is $2.4\times$ faster than the serial one. Use Amdahl’s law to find the serial fraction s that this implies (the speed-up on p threads is $[s + (1 - s)/p]^{-1}$) and predict the speed-up at $p = 4$; compare with the measured value in `ch10_bench_numbers.json`. At $N = 128$ the same law cannot produce a slowdown: add a per-thread synchronisation cost to the model and estimate the size of the step below which threading cannot pay. Which of your numbers would change on an idle machine? *Hint:* with $s = 0.34$ the model gives 2.4 at $p = 8$; fit $t(p) = t_1[s + (1 - s)/p] + c p$ to the three sizes of the figure.

Notes and further reading

The three instruments of the book are described in their own papers: the Lean library [12] (built on Lean 4 [11] and Mathlib [13]), the solver and its verification [21], and the physical programme [23, 83]. The Comparator [99] judges that a Lean solution proves the statement of a challenge file under a list of permitted axioms, and can pass the exported proof to a second kernel such as nanoda [100]. Pre-registration as a general practice is discussed by Nosek and co-authors [104], and the case for publishing the code and the data from which a number was computed by Peng [105]; both are general discussions, and the checklist of section 10.6 adapts them to this programme. The projected Gross–Pitaevskii method that the solver implements is reviewed by Blakie and co-authors [20]. So far, the only experiment of this book that the programme has reproduced independently is that of Kwon and Shin [27].

The Lean Library: a Complete Catalogue

This appendix lists the whole of the Lean 4 library `QuantumFluids` that the book is about: 37 modules, 310 theorems and lemmas, and 103 definitions, structures and abbreviations. It is *generated* from the sources in `lean_src/by book/facts/make_appA.py` (see appendix B); nothing in it is typed by hand except these paragraphs. Every statement is copied verbatim, line breaks included, up to the `:=` that starts its proof or definition body; the proofs are not reproduced.

How to read an entry. Each module is headed by one line of its own docstring, by the namespace in which its declarations live (for 13 of the 37 modules this is *not* `QuantumFluids.Module`; see table A.2), by the numbers of theorems and of definitions, and by its audit status. A gold $\star$ in the margin followed by chapter numbers marks a declaration that those chapters cite (through `\Lthm` or by copying its statement). The catalogue is a statement of *what is proved*, not of what is measured: to read a statement as physics, go to the chapter that cites it.

What the audit status means. *The audit is of the kernel environment, not of the source text.* For each module the book’s audit driver (`book/facts/lean_audit.py`, appendix B) compiles the module in dependency order against the pinned Mathlib and asks Lean, for *every constant the file declares*, which axioms it depends on (`Lean.collectAxioms`). The status line reports the number of theorem constants found, whether `sorryAx` is reachable, the number of axioms the module declares itself, and the union of the axioms used. The three standard axioms are abbreviated p = `propext`, c = `Classical.choice`, q = `Quot.sound`. At the time this appendix was generated, 37 of 37 modules had such an environment audit on file; 0 more carry only the first-pass compile log (author-written `#print axioms` lines, marked [†] in the table); for the remaining 0 the status reads “not re-checked”. Compiling a module with exit code 0 is not the same as auditing it: 7 modules (`DissipativeVortexDynamics`, `EinsteinRelation`, `FrictionKinetic`, `KTFlow`, `MatchingScreening`, `PairPolarisation`, `QuasiPeriodicBound`) carry no `#print axioms` line in their sources, so a compile log of theirs is empty; the environment audit is what supplies their footprint (chapter 10).

Table A.1: The 37 modules of the library at a glance. *thm*: theorems and lemmas; *def*: definitions, structures, abbreviations; *audit*: axiom footprint (p, c, q as in the text; n/c: not re-checked; *stale*: the file changed after it was audited; *!*: a non-standard axiom or *sorryAx*); *cited by*: chapters that cite a declaration of the module.

module	thm	def	audit	cited by
BoseIntegral	7	2	p c q	1, 4
ChargeLattice	15	4	p c q	—
CompactBoson	12	3	p c q	—
ContinuumWinding	9	1	p c q	3
DissipativeVortexDynamics	9	1	p c q	7, 10
DualLength	13	3	p c q	—
Duality	11	0	p c q	—
EinsteinRelation	7	3	p c q	7
Fricke	12	3	p c q	—
FrictionKinetic	3	2	p c q	7, 10
GPGalerkin	13	7	p c q	2
HeliumKinematics	15	4	p c q	1, 4, 5, 9
KTFlow	8	2	p c q	6, 10
LevelRankDuality	1	2	p c q	—
MadelungNSE	2	3	p c q	3
MadelungSplit	6	1	p c q	3, 9
MatchingScreening	10	3	p c q	6
PairPolarisation	8	2	p c q	6
PhaseMixing	3	0	p c q	8
PhononSeries	9	10	p q	2, 4
PhononSpecificHeat	14	1	p c q	1, 4
QHFricke	6	3	p c q	—
QuantizedCirculation	6	3	p c q	1, 3
QuantumFluidsShell	12	3	p c q	—
QuasiPeriodicBound	6	1	p c q	7, 10
RCFTDuality	3	1	p c q	—
RipsFloor	4	2	p c q	—
ScaleResolvedWinding	6	4	p c q	3
SectorDuality	3	0	p c q	—
SectorTemperature	8	5	p c q	—
ShellHamiltonian	9	2	p c q	—
SigmaRule	6	3	p c q	—
TopologicalProtection	13	2	p c q	—
Villani	18	8	p c q	—
VortexWinding	9	2	p c q	1, 3, 9
WassersteinCertificate	8	6	p c q	—
ZeroSound	6	1	p c q	1, 8

Table A.2: Modules whose declarations do *not* live in `QuantumFluids.Module`. The fully qualified name of a theorem is the namespace below, a dot, and the name printed in the catalogue.

module	namespace in the source
DissipativeVortexDynamics	DissipativeVortexDynamics
Duality	Mathesis.Duality
EinsteinRelation	EinsteinRelation
FrictionKinetic	FrictionKinetic
KTFlow	KTFlow
MadelungSplit	QuantumFluids.Madelung
MatchingScreening	MatchingScreening
PairPolarisation	PairPolarisation
QuantizedCirculation	QuantumFluids.Circulation
QuantumFluidsShell	QuantumFluids.ShellComplex
QuasiPeriodicBound	QuasiPeriodicBound
ShellHamiltonian	QuantumFluids.ShellComplex
SigmaRule	QuantumFluids.ShellComplex

A.1 The catalogue, module by module

A.1.1 BoseIntegral

The Bose integral behind the phonon specific heat of superfluid helium.

Namespace `QuantumFluids.BoseIntegral` · 7 theorems, 2 definitions · *audit*: 7 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: `Classical.choice`, `Quot.sound`, `propext`.

```

★4noncomputable def bose (t : ℝ) : ℂ

★4theorem hasSum_bose {t : ℝ} (ht : t ∈ Ioi (0 : ℝ)) :
  HasSum (fun n : ℕ => (1 : ℂ) * rexp (-(n : ℝ) + 1) * t)) (bose t)

  theorem hasSum_mellin_bose {s : ℂ} (hs : 1 < s.re) :
    HasSum (fun n : ℕ => Complex.Gamma s * 1 / ((n : ℝ) + 1 : ℝ) : ℂ) ^ s) (mellin bose s)

★1,4theorem mellin_bose_four : mellin bose 4 = (π : ℂ) ^ 4 / 15

★4theorem bose_integral_nat {n : ℕ} (hn : 1 ≤ n) :
  mellin bose (n + 1) = (Nat.factorial n : ℂ) * riemannZeta (n + 1)

  theorem bose_integral_even {k : ℕ} (hk : k ≠ 0) :
    mellin bose ((2 * k - 1 : ℕ) + 1) = (Nat.factorial (2 * k - 1) : ℂ) *
      ((-1) ^ (k + 1) * (2 : ℂ) ^ (2 * k - 1) * (π : ℂ) ^ (2 * k) * bernoulli (2 * k)
        / Nat.factorial (2 * k))

★1noncomputable def debyeEnergy (V kB hbar c I T : ℝ) : ℝ

  theorem debyeEnergy_eq (V kB hbar c T : ℝ) :
    debyeEnergy V kB hbar c (π ^ 4 / 15) T = π ^ 2 * V * (kB * T) ^ 4 / (30 * (hbar * c) ^ 3)

★1,4theorem debye_specific_heat (V kB hbar c T : ℝ) :
  HasDerivAt (fun T => debyeEnergy V kB hbar c (π ^ 4 / 15) T)
    (2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3) T

```

A.1.2 ChargeLattice

Two cosmological charge lattices stated so that the kernel reads off the same distinction as CompactBoson.lean: the duality is a relabelling of the sectors; what the physics depends on is a ...

Namespace QuantumFluids.ChargeLattice · 15 theorems, 4 definitions · *audit*: 15 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
def disc (p q :  $\Lambda$ ) :  $\mathbb{Z}$ 

theorem gram_pp (hB : B.IsSymm) (a b :  $\mathbb{Z}$ ) (p q :  $\Lambda$ ) :
  B (a • p + b • q) (a • p + b • q) = a ^ 2 * B p p + 2 * a * b * B p q + b ^ 2 * B q q

theorem gram_pq (hB : B.IsSymm) (a b c d :  $\mathbb{Z}$ ) (p q :  $\Lambda$ ) :
  B (a • p + b • q) (c • p + d • q)
  = a * c * B p p + (a * d + b * c) * B p q + b * d * B q q

theorem disc_sl2 (hB : B.IsSymm) (a b c d :  $\mathbb{Z}$ ) (p q :  $\Lambda$ ) :
  disc B (a • p + b • q) (c • p + d • q) = (a * d - b * c) ^ 2 * disc B p q

theorem disc_sl2_invariant (hB : B.IsSymm) {a b c d :  $\mathbb{Z}$ } (hdet : a * d - b * c = 1) (p q :  $\Lambda$ ) :
  disc B (a • p + b • q) (c • p + d • q) = disc B p q

theorem disc_electric_magnetic_swap (hB : B.IsSymm) (p q :  $\Lambda$ ) : disc B q (-p) = disc B p q

theorem disc_scale (hB : B.IsSymm) (t :  $\mathbb{Z}$ ) (p q :  $\Lambda$ ) : disc B (t • p) (t • q) = t ^ 4 * disc B p q

theorem disc_zero_of_parallel (hB : B.IsSymm) (k :  $\mathbb{Z}$ ) (p :  $\Lambda$ ) : disc B p (k • p) = 0

theorem area_sl2_invariant (hB : B.IsSymm) {a b c d :  $\mathbb{Z}$ } (hdet : a * d - b * c = 1) (p q :  $\Lambda$ ) :
  Real.sqrt (-(disc B (a • p + b • q) (c • p + d • q) :  $\mathbb{R}$ )) = Real.sqrt (-(disc B p q :  $\mathbb{R}$ ))

noncomputable def bpLambda {J :  $\mathbb{N}$ } (Abare :  $\mathbb{R}$ ) (q : Fin J →  $\mathbb{R}$ ) (n : Fin J →  $\mathbb{Z}$ ) :  $\mathbb{R}$ 

def nucleate {J :  $\mathbb{N}$ } (n : Fin J →  $\mathbb{Z}$ ) (i : Fin J) : Fin J →  $\mathbb{Z}$ 

theorem bp_step {J :  $\mathbb{N}$ } (Abare :  $\mathbb{R}$ ) (q : Fin J →  $\mathbb{R}$ ) (n : Fin J →  $\mathbb{Z}$ ) (i : Fin J) :
  bpLambda Abare q (nucleate n i) - bpLambda Abare q n = -((n i :  $\mathbb{R}$ ) - 1 / 2) * q i ^ 2

theorem bp_step_neg_iff {J :  $\mathbb{N}$ } (Abare :  $\mathbb{R}$ ) (q : Fin J →  $\mathbb{R}$ ) (n : Fin J →  $\mathbb{Z}$ ) (i : Fin J) (hq : q i
  ↦ ≠ 0) :
  bpLambda Abare q (nucleate n i) < bpLambda Abare q n ↔ 1 ≤ n i

theorem bp_ge_bare {J :  $\mathbb{N}$ } (Abare :  $\mathbb{R}$ ) (q : Fin J →  $\mathbb{R}$ ) (n : Fin J →  $\mathbb{Z}$ ) :
  bpLambda Abare q 0 ≤ bpLambda Abare q n

theorem bp_min_at_zero {J :  $\mathbb{N}$ } (Abare :  $\mathbb{R}$ ) (q : Fin J →  $\mathbb{R}$ ) (hq : ∀ i, q i ≠ 0) (n : Fin J →  $\mathbb{Z}$ ) :
  bpLambda Abare q n = bpLambda Abare q 0 ↔ n = 0

noncomputable def kaloperV (X 0 :  $\mathbb{R}$ ) (N :  $\mathbb{Z}$ ) :  $\mathbb{R}$ 

theorem kaloper_step (X 0 :  $\mathbb{R}$ ) (N :  $\mathbb{Z}$ ) :
  kaloperV X 0 N - kaloperV X 0 (N + 1) = X * 0 ^ 2 * ((1 :  $\mathbb{R}$ ) - N) - 1 / 2

theorem kaloper_min (X 0 :  $\mathbb{R}$ ) (hX : 0 < X) (h0 : 0 ≠ 0) (N :  $\mathbb{Z}$ ) :
  0 ≤ kaloperV X 0 N ∧ (kaloperV X 0 N = 0 ↔ N = 1)

theorem kaloper_terminates (X 0 :  $\mathbb{R}$ ) (N :  $\mathbb{Z}$ ) (hN : N ≤ 1) :
  kaloperV X 0 (N + ((1 - N).toNat :  $\mathbb{Z}$ )) = 0
```

A.1.3 CompactBoson

The T-duality of the compact boson stated on its SECTORS, so that three things can be read off the kernel: the duality is a reindexing of the sector lattice; the product of the electric and ...

Namespace QuantumFluids.CompactBoson · 12 theorems, 3 definitions · *audit*: 12 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def dim (R : ℝ) (n w : ℤ) : ℝ

def spin (n w : ℤ) : ℤ

theorem dim_dual (R : ℝ) (hR : R ≠ 0) (n w : ℤ) : dim R n w = dim (2 / R) w n

theorem spin_dual (n w : ℤ) : spin n w = spin w n

def swap : ℤ × ℤ ≈ ℤ × ℤ

theorem partition_dual (R : ℝ) (hR : R ≠ 0) (F : ℝ → ℤ → ℝ) :
  ∑' p : ℤ × ℤ, F (dim R p.1 p.2) (spin p.1 p.2) = ∑' p : ℤ × ℤ, F (dim (2 / R) p.1 p.2) (spin p
    ↪ .1 p.2)

theorem dim_electric (R : ℝ) : dim R 1 0 = 1 / R ^ 2

theorem dim_magnetic (R : ℝ) : dim R 0 1 = R ^ 2 / 4

theorem electric_mul_magnetic (R : ℝ) (hR : R ≠ 0) : dim R 1 0 * dim R 0 1 = 1 / 4

theorem eq_of_sq_eq_sq_pos {a b : ℝ} (ha : 0 < a) (hb : 0 < b) (h : a ^ 2 = b ^ 2) : a = b

theorem selfDual_radius {R : ℝ} (hR : 0 < R) : dim R 1 0 = dim R 0 1 ↔ R = Real.sqrt 2

theorem selfDual_value : dim (Real.sqrt 2) 1 0 = 1 / 2 ∧ dim (Real.sqrt 2) 0 1 = 1 / 2

theorem vortex_marginal_radius {R : ℝ} (hR : 0 < R) : dim R 0 1 = 2 ↔ R = 2 * Real.sqrt 2

theorem bkt_not_selfDual : (2 * Real.sqrt 2 : ℝ) ≠ Real.sqrt 2

theorem eta_at_bkt : dim (2 * Real.sqrt 2) 1 0 = 1 / 8

```

A.1.4 ContinuumWinding

"Rome": the discrete winding number equals the continuum degree.

Namespace QuantumFluids.ContinuumWinding · 9 theorems, 1 definition · *audit*: 10 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem pdiff_congr {a a' b b' : ℝ} (ha : (a : Real.Angle) = a') (hb : (b : Real.Angle) = b') :
  pdiff a b = pdiff a' b'

theorem sum_pdiff_telescope (Γ : ℕ → ℝ) (n : ℕ) (h : ∀ i < n, Γ (i + 1) - Γ i ∈ Set.Ioc (-π) π) :
  ∑ i ∈ Finset.range n, pdiff (Γ i) (Γ (i + 1)) = Γ n - Γ 0

theorem degree_int (x y : ℝ) (h : (x : Real.Angle) = y) : ∃ w : ℤ, x - y = (w : ℝ) * (2 * π)

★ noncomputable def sample (n i : ℕ) : I

theorem sample_zero (n : ℕ) : sample n 0 = 0

theorem sample_self (n : ℕ) (hn : 0 < n) : sample n n = 1

theorem sample_val (n i : ℕ) (hi : i ≤ n) (hn : 0 < n) : (sample n i : ℝ) = (i : ℝ) / n

theorem dist_sample (n i : ℕ) (hi : i < n) :
  dist (sample n (i + 1)) (sample n i) = 1 / n

theorem discrete_eq_degree (γ : C(I, Real.Angle)) (hclosed : γ 1 = γ 0) :
  ∃ w : ℤ, ∃ n₀ : ℕ, 0 < n₀ ∧ ∀ n ≥ n₀, ∀ θ : ℕ → ℝ,
    (∀ i ≤ n, (θ i : Real.Angle) = γ (sample n i)) →
    ∑ i ∈ Finset.range n, pdiff (θ i) (θ (i + 1)) = (w : ℝ) * (2 * π)

★ theorem detector_correct (c : C(I, ℂ)) (hne : ∀ t, c t ≠ 0) (hclosed : c 1 = c 0) :
  ∃ w : ℤ, ∃ n₀ : ℕ, 0 < n₀ ∧ ∀ n ≥ n₀,
    ∑ i ∈ Finset.range n, pdiff (Complex.arg (c (sample n i))) (Complex.arg (c (sample n (i + 1))))

```

```

    ↪ )))
    = (w : ℝ) * (2 * π)

```

A.1.5 DissipativeVortexDynamics

The dissipative point-vortex model behind the friction, transverse-force and Einstein-relation measurements (docs/designs/PGPE_FRICTION_PREREG.md amendment A1, PGPE_ALPHAPRIME_PREREG.md, ...

Namespace DissipativeVortexDynamics · 9 theorems, 1 definition · *audit*: 9 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext; the source has no #print axioms line (footprint from the appended environment audit).

```

★7,10 theorem energy_dissipation (H : X → ℝ) (gradH : X → X) (hH : ∀ x, HasGradientAt H (gradH x) x)
  (A : X → X) (hA : ∀ v, inner ℝ v (A v) = 0) (γ : ℝ) (r : ℝ → X)
  (hr : ∀ t, HasDerivAt r (A (gradH (r t)) - γ • gradH (r t)) t) (t : ℝ) :
  HasDerivAt (fun t => H (r t)) (-γ * ||gradH (r t)|| ^ 2) t

★10 theorem energy_antitone (H : X → ℝ) (gradH : X → X) (hH : ∀ x, HasGradientAt H (gradH x) x)
  (A : X → X) (hA : ∀ v, inner ℝ v (A v) = 0) {γ : ℝ} (hγ : 0 ≤ γ) (r : ℝ → X)
  (hr : ∀ t, HasDerivAt r (A (gradH (r t)) - γ • gradH (r t)) t) :
  Antitone (fun t => H (r t))

★7,10 theorem dissipation_indep_of_skew (H : X → ℝ) (gradH : X → X) (hH : ∀ x, HasGradientAt H (gradH x)
  ↪ x)
  (A B : X → X) (hA : ∀ v, inner ℝ v (A v) = 0) (hB : ∀ v, inner ℝ v (B v) = 0) (γ : ℝ)
  (r s : ℝ → X) (hr : ∀ t, HasDerivAt r (A (gradH (r t)) - γ • gradH (r t)) t)
  (hs : ∀ t, HasDerivAt s (B (gradH (s t)) - γ • gradH (s t)) t) (t : ℝ) (hrs : r t = s t) :
  deriv (fun t => H (r t)) t = deriv (fun t => H (s t)) t

★7 noncomputable def r2 (t : ℝ) : ℝ

★7 theorem r2_hasDerivAt (t : ℝ) (ht : t ∈ Icc (0 : ℝ) T) : HasDerivAt (r2 px py mx my) (-4 * α) t

★7 theorem dipole_sq_law (t : ℝ) (ht : t ∈ Icc (0 : ℝ) T) :
  r2 px py mx my t = r2 px py mx my 0 - 4 * α * t

★7 theorem dipole_lifetime_bound (hα : 0 < α) (hT : 0 ≤ T) : T < r2 px py mx my 0 / (4 * α)

★7 theorem centre_velocity_identity (t : ℝ) (ht : t ∈ Icc (0 : ℝ) T) :
  ∃ cx' cy' : ℝ, HasDerivAt (fun s => (px s + mx s) / 2) cx' t ∧ HasDerivAt (fun s => (py s + my
  ↪ s) / 2) cy' t ∧
  cx' * (py t - my t) - cy' * (px t - mx t) = 1 - α' ∧
  cx' * (px t - mx t) + cy' * (py t - my t) = 0 ∧
  (cx' ^ 2 + cy' ^ 2) * r2 px py mx my t = (1 - α') ^ 2

★7 theorem wind_stall {a b c : ℝ} (ha : 0 < a) (hab : a < b) (hc : 0 < c) (d : ℝ → ℝ)
  (hd : ∀ t, HasDerivAt d (-c * (d t - a) * (d t - b) / d t) t) (h0 : b ≤ d 0) :
  ∀ t, 0 ≤ t → b ≤ d t

theorem stall_of_drive_sign {a b c : ℝ} (hab : a < b) (hc : 0 < c) (g : ℝ → ℝ) (hg : ∀ x, a < x →
  ↪ x < b → g x < 0)
  (d : ℝ → ℝ) (hd : ∀ t, HasDerivAt d (-c * g (d t)) t) (h0 : b ≤ d 0) :
  ∀ t, 0 ≤ t → b ≤ d t

```

A.1.6 DualLength

The dual length of a quantum fluid (design memo docs/designs/DUAL_SCALE_QUANTUM_FLUID.md; CLAIM-024).

Namespace QuantumFluids.DualLength · 13 theorems, 3 definitions · *audit*: 13 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def dualLength (hc k epsSq : ℝ) : ℝ

noncomputable def bogoliubovSq (hc ks k : ℝ) : ℝ

noncomputable def ell (ks k : ℝ) : ℝ

theorem dualLength_phonon {hc k : ℝ} (hhc : hc ≠ 0) (hk : k ≠ 0) :
  dualLength hc k ((hc * k) ^ 2) = 1 / k

theorem dualLength_free {hc ks k : ℝ} (hhc : hc ≠ 0) (hk : k ≠ 0) (hks : ks ≠ 0) :
  dualLength hc k ((hc * k ^ 2 / ks) ^ 2) = k / ks ^ 2

theorem dualLength_bogoliubov {hc ks k : ℝ} (hhc : hc ≠ 0) (hk : k ≠ 0) (hks : ks ≠ 0) :
  dualLength hc k (bogoliubovSq hc ks k) = ell ks k

theorem dual_involutive {ks k : ℝ} (hk : k ≠ 0) (hks : ks ≠ 0) :
  ks ^ 2 / (ks ^ 2 / k) = k

theorem ell_dual_invariant {ks k : ℝ} (hk : k ≠ 0) (hks : ks ≠ 0) :
  ell ks (ks ^ 2 / k) = ell ks k

theorem ell_ge {ks k : ℝ} (hk : 0 < k) (hks : 0 < ks) : 2 / ks ≤ ell ks k

theorem ell_eq_iff {ks k : ℝ} (hk : 0 < k) (hks : 0 < ks) : ell ks k = 2 / ks ↔ k = ks

theorem not_bogoliubov_of_lt {hc ks k epsSq : ℝ} (hhc : hc ≠ 0) (hk : 0 < k) (hks : 0 < ks)
  (hmeas : dualLength hc k epsSq < 2 / ks) : epsSq ≠ bogoliubovSq hc ks k

theorem ell_ge_iff_envelope {hc ks k epsSq : ℝ} (hhc : hc ≠ 0) (hk : 0 < k) (hks : 0 < ks) :
  2 / ks ≤ dualLength hc k epsSq ↔ 2 * hc ^ 2 / ks * k ^ 3 ≤ epsSq

theorem bogoliubov_envelope {hc ks k : ℝ} (hhc : hc ≠ 0) (hk : 0 < k) (hks : 0 < ks) :
  2 * hc ^ 2 / ks * k ^ 3 ≤ bogoliubovSq hc ks k

theorem dualLength_feynman {hc ks k S : ℝ} (hhc : hc ≠ 0) (hk : k ≠ 0) (hks : ks ≠ 0) (hS : S ≠ 0)
  ↪ :
  dualLength hc k ((hc * k ^ 2 / (ks * S)) ^ 2) = k / (ks ^ 2 * S ^ 2)

theorem feynman_ge_iff {ks k S : ℝ} (_hk : 0 < k) (hks : 0 < ks) (hS : 0 < S) :
  2 / ks ≤ k / (ks ^ 2 * S ^ 2) ↔ S ^ 2 ≤ k / (2 * ks)

theorem not_bogoliubov_of_structure_factor {hc ks k S epsSq : ℝ}
  (hhc : hc ≠ 0) (hk : 0 < k) (hks : 0 < ks) (hS : 0 < S)
  (hfeyn : epsSq ≤ (hc * k ^ 2 / (ks * S)) ^ 2) -- Feynman's variational bound
  (hpeak : k / (2 * ks) < S ^ 2) : -- measured: S above the dual-scale
  ↪ bound
  epsSq ≠ bogoliubovSq hc ks k

```

A.1.7 Duality

The Abstract Self-Dual Bound and Its First Instances (Rung 0 + Rung 1)

Namespace Mathesis.Duality · 11 theorems, 0 definitions · *audit*: 11 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem sqrt_le_max_of_le_mul {C x y : ℝ} (hx : 0 ≤ x) (hy : 0 ≤ y)
  (h : C ≤ x * y) : Real.sqrt C ≤ max x y

theorem min_le_sqrt_of_mul_le {C x y : ℝ} (hx : 0 ≤ x) (hy : 0 ≤ y)
  (h : x * y ≤ C) : min x y ≤ Real.sqrt C

theorem sqrt_between_duals {C x y : ℝ} (hx : 0 ≤ x) (hy : 0 ≤ y)
  (h : x * y = C) : min x y ≤ Real.sqrt C ∧ Real.sqrt C ≤ max x y

```

```

theorem two_sqrt_mul_le_add {x y : ℝ} (hx : 0 ≤ x) (hy : 0 ≤ y) :
  2 * Real.sqrt (x * y) ≤ x + y

theorem self_dual_fixed_point {C x : ℝ} (hC : 0 < C) (hx : 0 < x) :
  x = C / x ↔ x = Real.sqrt C

theorem Reff_ge_sqrt_of_selfDual {α R : ℝ} (hα : 0 < α) (hR : 0 < R) :
  Real.sqrt α ≤ max R (α / R)

theorem sqrt_le_max_of_le_mul_nat {a b N : ℕ} (h : N ≤ a * b) :
  Real.sqrt (N : ℝ) ≤ max (a : ℝ) (b : ℝ)

theorem eq_lower_bound {D K h Q : ℝ}
  (hD : 0 < D) (hK : 0 < K) (hh : 0 < h) (hQ : 0 < Q) :
  2 * Real.sqrt (D * K * h / 2) ≤ D * K / Q + h * Q / 2

theorem bogoliubov_dual_form {c ks k : ℝ} (hks : 0 < ks) (hk : 0 < k) :
  c ^ 2 * k ^ 2 + c ^ 2 * k ^ 4 / ks ^ 2 = c ^ 2 * k ^ 3 / ks * (k / ks + ks / k)

theorem bogoliubov_selfdual_bound {c ks k : ℝ} (hks : 0 < ks) (hk : 0 < k) :
  2 * c ^ 2 * k ^ 3 / ks ≤ c ^ 2 * k ^ 2 + c ^ 2 * k ^ 4 / ks ^ 2

theorem sinh_selfDual_coupling {K : ℝ} (hK : 0 < K)
  (hfix : Real.sinh (2 * K) * Real.sinh (2 * K) = 1) :
  K = Real.log (1 + Real.sqrt 2) / 2

```

A.1.8 EinsteinRelation

Companion of docs/designs/PGPE_EINSTEIN_PREREG.md.

Namespace EinsteinRelation · 7 theorems, 3 definitions · *audit*: 7 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext; the source has no #print axioms line (footprint from the appended environment audit).

```

★7noncomputable def p (K x y : ℝ) : ℝ

theorem hasDerivAt_p_x (K x y : ℝ) (h : x ^ 2 + y ^ 2 ≠ 0) :
  HasDerivAt (fun x => p K x y) (-K * x / (x ^ 2 + y ^ 2) * p K x y) x

theorem hasDerivAt_p_y (K x y : ℝ) (h : x ^ 2 + y ^ 2 ≠ 0) :
  HasDerivAt (fun y => p K x y) (-K * y / (x ^ 2 + y ^ 2) * p K x y) y

★7noncomputable def Jx (α D K x y : ℝ) : ℝ

noncomputable def Jy (α D K x y : ℝ) : ℝ

theorem Jx_eq (α D K x y : ℝ) : Jx α D K x y = (D * K - 2 * α) * (x / (x ^ 2 + y ^ 2) * p K x y)

theorem Jy_eq (α D K x y : ℝ) : Jy α D K x y = (D * K - 2 * α) * (y / (x ^ 2 + y ^ 2) * p K x y)

★7theorem zero_flux_iff (α D K : ℝ) :
  (∀ x y : ℝ, x ^ 2 + y ^ 2 ≠ 0 → Jx α D K x y = 0 ∧ Jy α D K x y = 0) ↔ D * K = 2 * α

★7theorem einstein_vortex (α η ρ T : ℝ) (hp : 0 < ρ) (hT : 0 < T) :
  (∀ x y : ℝ, x ^ 2 + y ^ 2 ≠ 0 → Jx α (2 * η) (2 * π * ρ / T) x y = 0 ∧ Jy α (2 * η) (2 * π * ρ / T) x y = 0)
  ↔ η = α * T / (2 * π * ρ)

theorem flux_gibbs (E : ℝ → ℝ) (E' μ D T x : ℝ) (hT : T ≠ 0) (hE : HasDerivAt E E' x) :
  ∃ ρ' : ℝ, HasDerivAt (fun x => exp (-E x / T)) ρ' x ∧
  μ * E' * exp (-E x / T) + D * ρ' = (μ - D / T) * E' * exp (-E x / T)

```

A.1.9 Fricke

The group-theoretic core of the "Fricke / K3" gate of paper/wasserstein_slack.tex §6.3, and NOTHING more.

Namespace QuantumFluids.Fricke · 12 theorems, 3 definitions · *audit*: 12 theorem constants audited (+2 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

def W (n : ℤ) : Matrix (Fin 2) (Fin 2) ℤ

theorem W_mul_W (n : ℤ) : W n * W n = (-n) • (1 : Matrix (Fin 2) (Fin 2) ℤ)

def conj (n : ℤ) (g : Matrix (Fin 2) (Fin 2) ℤ) (c' : ℤ) : Matrix (Fin 2) (Fin 2) ℤ

theorem W_mul_eq (n : ℤ) (g : Matrix (Fin 2) (Fin 2) ℤ) (c' : ℤ) (hc : g 1 0 = n * c') :
  W n * g = conj n g c' * W n

theorem det_conj (n : ℤ) (g : Matrix (Fin 2) (Fin 2) ℤ) (c' : ℤ) (hc : g 1 0 = n * c') :
  (conj n g c').det = g.det

theorem conj_apply_one_zero (n : ℤ) (g : Matrix (Fin 2) (Fin 2) ℤ) (c' : ℤ) :
  conj n g c' 1 0 = n * (-(g 0 1))

theorem conj_conj (n : ℤ) (g : Matrix (Fin 2) (Fin 2) ℤ) (c' : ℤ) (hc : g 1 0 = n * c') :
  conj n (conj n g c') (-(g 0 1)) = g

theorem mem_Gamma0_iff_dvd (n : ℕ) [NeZero n] (A : SL(2, ℤ)) :
  A ∈ Gamma0 n ↔ (n : ℤ) ∣ A 1 0

theorem fricke_normalizes (n : ℕ) [NeZero n] (A : SL(2, ℤ)) (hA : A ∈ Gamma0 n) :
  ∃ B : SL(2, ℤ), B ∈ Gamma0 n ∧
    W n * (A : Matrix (Fin 2) (Fin 2) ℤ) = (B : Matrix (Fin 2) (Fin 2) ℤ) * W n

theorem fricke_on_axis (n y : ℝ) (hn : n ≠ 0) (hy : y ≠ 0) :
  ((0 : ℂ) * (Complex.I * y) + (-1)) / ((n : ℂ) * (Complex.I * y) + 0) = Complex.I * (1 / (n * y
    ↪ ))

noncomputable def axis (n y : ℝ) : ℝ

theorem axis_involutive {n y : ℝ} (hn : n ≠ 0) (hy : y ≠ 0) : axis n (axis n y) = y

theorem axis_eq_dual {ks : ℝ} (hks : ks ≠ 0) (k : ℝ) : axis (1 / ks ^ 2) k = ks ^ 2 / k

theorem ell_axis_invariant {ks k : ℝ} (hk : k ≠ 0) (hks : ks ≠ 0) :
  QuantumFluids.DualLength.ell ks (axis (1 / ks ^ 2) k) = QuantumFluids.DualLength.ell ks k

theorem axis_fixed_iff {n y : ℝ} (hn : 0 < n) (hy : 0 < y) :
  axis n y = y ↔ y = 1 / Real.sqrt n

```

A.1.10 FrictionKinetic

The kinetic reading of the friction law (docs/designs/FRICTION_LAW_THEORY_NOTE.md)

Namespace FrictionKinetic · 3 theorems, 2 definitions · *audit*: 3 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext; the source has no #print axioms line (footprint from the appended environment audit).

```

*7.10 noncomputable def alpha (K : Finset ι) (w f : ι → ℝ) (ps κ : ℝ) : ℝ

*7.10 noncomputable def rhoN (K : Finset ι) (w : ι → ℝ) : ℝ

*7.10 theorem coeff_eq_weighted_mean (K : Finset ι) (w f : ι → ℝ) (p ps κ : ℝ) (hpn : rhoN K w ≠ 0) (hp
  ↪ : p ≠ 0) :
  alpha K w f ps κ / (rhoN K w / p) = p / (ps * κ) * ((∑ k ∈ K, w k * f k) / ∑ k ∈ K, w k)

*7 theorem weighted_mean_mem_Icc (K : Finset ι) (w f : ι → ℝ) (hw : ∀ k ∈ K, 0 ≤ w k) (hpos : 0 < ∑ k
  ↪ ∈ K, w k)

```

```
(m M : ℝ) (hm : ∀ k ∈ K, m ≤ f k) (hM : ∀ k ∈ K, f k ≤ M) :  
(∑ k ∈ K, w k * f k) / (∑ k ∈ K, w k) ∈ Set.Icc m M
```

```
*7theorem rayleigh_jeans_mean (K : Finset ι) (f : ι → ℝ) (T c : ℝ) (hT : 0 < T) (hc : 0 < c) :  
(∑ k ∈ K, (T / (2 * c ^ 2)) * f k) / (∑ k ∈ K, (T / (2 * c ^ 2))) = (∑ k ∈ K, f k) / K.card
```

A.1.11 *GPGalerkin*

D3 (docs/OPENAI_NSE_LEVERAGE_FOR_QUANTUM_FLUIDS.md): structure of the Fourier-Galerkin Gross-Pitaevskii nonlinearity that is INDEPENDENT of the truncation set.

Namespace QuantumFluids.GPGalerkin · 13 theorems, 7 definitions · *audit*: 13 theorem constants audited (+4 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
*2noncomputable def nl (Λ : Finset G) (ψ : G → ℂ) (k : G) : ℂ  
  
noncomputable def pairing (Λ : Finset G) (ψ : G → ℂ) : ℂ  
  
noncomputable def ff (ψ : G → ℂ) (p : G × G) : ℂ  
  
noncomputable def dens (Λ : Finset G) (ψ : G → ℂ) (q : G) : ℂ  
  
noncomputable def diffs (Λ : Finset G) : Finset G  
  
theorem pairing_eq_prod (Λ : Finset G) (ψ : G → ℂ) :  
  pairing Λ ψ = ∑ p ∈ Λ ×s Λ, ∑ r ∈ Λ ×s Λ,  
    if p.1 - p.2 = r.1 - r.2 then ff ψ p * (starRingEnd ℂ) (ff ψ r) else 0  
  
theorem dens_mul_conj (Λ : Finset G) (ψ : G → ℂ) (q : G) :  
  dens Λ ψ q * (starRingEnd ℂ) (dens Λ ψ q) = ∑ p ∈ Λ ×s Λ, ∑ r ∈ Λ ×s Λ,  
    if p.1 - p.2 = q ∧ r.1 - r.2 = q then ff ψ p * (starRingEnd ℂ) (ff ψ r) else 0  
  
*2theorem pairing_eq_sum (Λ : Finset G) (ψ : G → ℂ) :  
  pairing Λ ψ = ∑ q ∈ diffs Λ, dens Λ ψ q * (starRingEnd ℂ) (dens Λ ψ q)  
  
noncomputable def pairingRe (Λ : Finset G) (ψ : G → ℂ) : ℝ  
  
theorem pairing_eq_ofReal (Λ : Finset G) (ψ : G → ℂ) :  
  pairing Λ ψ = (pairingRe Λ ψ : ℂ)  
  
*2theorem pairing_nonneg (Λ : Finset G) (ψ : G → ℂ) : 0 ≤ pairingRe Λ ψ  
  
theorem pairing_im_zero (Λ : Finset G) (ψ : G → ℂ) : (pairing Λ ψ).im = 0  
  
*2theorem mass_rate_zero (Λ : Finset G) (ψ : G → ℂ) (ω : G → ℝ) (g : ℝ) :  
  ∑ k ∈ Λ, ((starRingEnd ℂ) (ψ k) * ((ω k : ℂ) * ψ k + (g : ℂ) * nl Λ ψ k)).im = 0  
  
theorem kinetic_le_energy (Λ : Finset G) (ψ : G → ℂ) (ω : G → ℝ) (g E : ℝ) (hg : 0 ≤ g)  
  (hE : E = ∑ k ∈ Λ, ω k * Complex.normSq (ψ k) + g / 2 * pairingRe Λ ψ) :  
  ∑ k ∈ Λ, ω k * Complex.normSq (ψ k) ≤ E  
  
theorem nl_eq_dens (Λ : Finset G) (ψ : G → ℂ) (k : G) :  
  nl Λ ψ k = ∑ b ∈ Λ, ψ b * dens Λ ψ (k - b)  
  
theorem dens_neg (Λ : Finset G) (ψ : G → ℂ) (q : G) :  
  dens Λ ψ (-q) = (starRingEnd ℂ) (dens Λ ψ q)  
  
noncomputable def densVar (Λ : Finset G) (ψ δ : G → ℂ) (q : G) : ℂ  
  
theorem sum_densVar (Λ : Finset G) (ψ δ : G → ℂ) :  
  ∑ q ∈ diffs Λ, (starRingEnd ℂ) (dens Λ ψ q) * densVar Λ ψ δ q  
    = ∑ p ∈ Λ ×s Λ, (starRingEnd ℂ) (dens Λ ψ (p.1 - p.2)) *  
      (δ p.1 * (starRingEnd ℂ) (ψ p.2) + ψ p.1 * (starRingEnd ℂ) (δ p.2))  
  
*2theorem grad_identity (Λ : Finset G) (ψ δ : G → ℂ) :  
  ∑ q ∈ diffs Λ, 2 * ((starRingEnd ℂ) (dens Λ ψ q) * densVar Λ ψ δ q).re
```

```

= 4 * (∑ k ∈ Λ, (starRingEnd C) (δ k) * nL ∧ ψ k).re
★2theorem energy_rate_zero (Λ : Finset G) (ψ : G → C) (ω : G → R) (g : R) :
  ∑ k ∈ Λ, 2 * ω k * ((starRingEnd C) (ψ k) *
    (-Complex.I * ((ω k : C) * ψ k + (g : C) * nL ∧ ψ k))).re
  + g / 2 * ∑ q ∈ diffs Λ, 2 * ((starRingEnd C) (dens ∧ ψ q) *
    densVar ∧ ψ (fun k => -Complex.I * ((ω k : C) * ψ k + (g : C) * nL ∧ ψ k)) q).re = 0

```

A.1.12 HeliumKinematics

Kinematic and consistency identities behind the analysis of neutron-scattering data on superfluid helium-4, formalised from statements made in *Namespace* QuantumFluids.HeliumKinematics · 15 theorems, 4 definitions · *audit*: 15 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

★1,5def phononDisp (c a k : R) : R

★5theorem three_phonon_excess (c a k1 k2 : R) :
  phononDisp c a (k1 + k2) - phononDisp c a k1 - phononDisp c a k2
  = 3 * c * a * k1 * k2 * (k1 + k2)

★1,4,5theorem three_phonon_open_iff {c a k1 k2 : R} (hc : 0 < c) (h1 : 0 < k1) (h2 : 0 < k2) :
  phononDisp c a k1 + phononDisp c a k2 ≤ phononDisp c a (k1 + k2) ↔ 0 ≤ a

★5def seriesDisp (c α2 α3 α4 k : R) : R

★5theorem symmetric_split_excess (c α2 α3 α4 k : R) :
  seriesDisp c α2 α3 α4 k - 2 * seriesDisp c α2 α3 α4 (k / 2)
  = c * k ^ 3 * (3 / 4 * α2 + 7 / 8 * α3 * k + 15 / 16 * α4 * k ^ 2)

★5theorem phase_velocity_excess (c α2 α3 α4 k : R) (hk : k ≠ 0) :
  seriesDisp c α2 α3 α4 k / k - c = c * k ^ 2 * (α2 + α3 * k + α4 * k ^ 2)

★5theorem group_velocity_excess (c α2 α3 α4 k : R) :
  HasDerivAt (seriesDisp c α2 α3 α4)
    (c + c * k ^ 2 * (3 * α2 + 4 * α3 * k + 5 * α4 * k ^ 2)) k

★5theorem two_roton_momentum_le {k1 k2 : V} {kR : R} (h1 : ||k1|| = kR) (h2 : ||k2|| = kR) :
  ||k1 + k2|| ≤ 2 * kR

★5theorem two_roton_parallel (k1 : V) : ||k1 + k1|| = 2 * ||k1||

★5theorem two_roton_antiparallel (k1 : V) : ||k1 + (-k1)|| = 0

def tofEnergy (Ei r : R) : R

★5theorem tofEnergy_rescale (lam Ei r : R) : tofEnergy (lam * Ei) r = lam * tofEnergy Ei r

★9theorem plateau_excess_calibration_invariant {lam : R} (hlam : 0 < lam) (Ei rR : R) :
  2 * tofEnergy (lam * Ei) rR < tofEnergy (lam * Ei) r ↔ 2 * tofEnergy Ei rR < tofEnergy Ei r

★5,9theorem tof_eq12_eq_eq13 {m v tel tin ts : R} (h1 : tel - ts ≠ 0) (h2 : tin - ts ≠ 0) :
  (1 / 2) * m * (v * (tel - ts)) ^ 2 * (1 / (tel - ts) ^ 2 - 1 / (tin - ts) ^ 2)
  = tofEnergy ((1 / 2) * m * v ^ 2) ((tel - ts) / (tin - ts))

★5,9theorem landau_velocity_parabolic_zero {a : R} (ha : 0 < a) {v : R} (hv : 0 < v) :
  ∃ k : R, 0 < k ∧ a * k ^ 2 / k < v

★5,9theorem landau_velocity_ge_of_above_sound_line {ε : R → R} {c k : R} (hk : 0 < k)
  (h : c * k ≤ ε k) : c ≤ ε k / k

def abrahamP (A1 A2 A3 p0 p : R) : R

theorem abraham_sound_speed_sq (A1 A2 A3 p0 p : R) :
  HasDerivAt (abrahamP A1 A2 A3 p0) (A1 + 2 * A2 * (p - p0) + 3 * A3 * (p - p0) ^ 2) p

```

```

theorem debye_mode_count (kD : ℝ) :
  (1 / (2 * Real.pi ^ 2)) * ∫ k in (0 : ℝ)..kD, k ^ 2 = kD ^ 3 / (6 * Real.pi ^ 2)

```

A.1.13 *KTFlow*

The Kosterlitz renormalisation-group flow: an exact conserved quantity, and the trapping of the superfluid side at the universal stiffness.

Namespace `KTFlow` · 8 theorems, 2 definitions · *audit*: 8 theorem constants audited (+2 generated by Lean), 0 sorry, 0 user axioms; axioms used: `Classical.choice`, `Quot.sound`, `propext`; the source has no #print axioms line (footprint from the appended environment audit).

```

*6 noncomputable def f (u : ℝ) : ℝ
*6 noncomputable def H (u y : ℝ) : ℝ

lemma f_antitone {a b : ℝ} (ha : 0 < a) (hab : a ≤ b) (hb : b ≤ π / 2) : f b ≤ f a
lemma hasDerivAt_H (l : ℝ) : HasDerivAt (fun l => H (u l) (y l)) 0 l
*6,10 theorem kt_invariant (l : ℝ) : H (u l) (y l) = H (u 0) (y 0)
lemma f_ge_H (l : ℝ) : H (u 0) (y 0) ≤ f (u l)
*6,10 theorem kt_trapped (h0 : u 0 < π / 2) (hH : f (π / 2) < H (u 0) (y 0)) : ∀ l, 0 ≤ l → u l < π / 2
*6 theorem u_monotone : Monotone u
*6,10 theorem kt_fugacity_bounded (h0 : u 0 < π / 2) (hH : f (π / 2) < H (u 0) (y 0)) (l : ℝ) (hl : 0 ≤
  ↪ l) :
  y l ^ 2 ≤ y 0 ^ 2
*6 theorem kt_units (K : ℝ) : 2 / π < K ↔ 4 < 2 * π * K

```

A.1.14 *LevelRankDuality*

A seventh cross-domain case: level-rank duality of Chern-Simons/WZW theories, `U(N)_K <-> U(K)_N` (Naculich-Schnitzer 2007, DOI 10.1088/1126-6708/2007/06/023; Hsin-Seiberg 2016, DOI ... *Namespace* `QuantumFluids.LevelRankDuality` · 1 theorem, 2 definitions · *audit*: 1 theorem constants audited, 0 sorry, 0 user axioms; axioms used: `Classical.choice`, `Quot.sound`, `propext`.

```

def inBox (a b : ℕ) (μ : YoungDiagram) : Prop
theorem inBox_transpose_iff (a b : ℕ) (μ : YoungDiagram) :
  inBox a b μ ↔ inBox b a μ.transpose
def levelRankRelabeling (a b : ℕ) : { μ : YoungDiagram // inBox a b μ } = { μ : YoungDiagram //
  ↪ inBox b a μ }

```

A.1.15 *MadelungNSE*

Bridge: the Madelung (quantum-fluid) velocity field expressed in OpenAI's Navier-Stokes vocabulary.

Namespace `QuantumFluids.MadelungNSE` · 2 theorems, 3 definitions · *audit*: 2 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: `Classical.choice`, `Quot.sound`, `propext`.

```

noncomputable def scalarLaplacian (p : PressureField) (t : ℝ) (x : Space) : ℝ
def TwiceSpatial (p : PressureField) (t : ℝ) (x : Space) : Prop
theorem divergence_pressureGradient (p : PressureField) (t : ℝ) (x : Space)
  (h : TwiceSpatial p t x) :

```

```

    spatialDivergence (fun q : SpaceTime => pressureGradient p q.1 q.2) t x
      = scalarLaplacian p t x

noncomputable def madelungVelocity (hbar m : ℝ) (S : PressureField) : VelocityField
*3theorem divergence_madelungVelocity (hbar m : ℝ) (S : PressureField) (t : ℝ) (x : Space)
  (h : TwiceSpatial S t x) :
    spatialDivergence (madelungVelocity hbar m S) t x = (hbar / m) * scalarLaplacian S t x

```

A.1.16 MadelungSplit

D4 (docs/OPENAI_NSE_LEVERAGE_FOR_QUANTUM_FLUIDS.md): the Madelung energy split.

Namespace QuantumFluids.Madelung · 6 theorems, 1 definition · *audit*: 6 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def psi {E : Type*} (a S : E → ℝ) (x : E) : ℂ

theorem normSq_phase_mul (θ p a r : ℝ) :
  ||Complex.exp (Complex.I * (θ : ℂ)) * ((p : ℂ) + Complex.I * (a : ℂ) * (r : ℂ))|| ^ 2
    = p ^ 2 + a ^ 2 * r ^ 2

theorem hasDerivAt_line (x v : E) :
  HasDerivAt (fun t : ℝ => x + t • v) v 0

theorem hasDerivAt_psi_line (a S : E → ℝ) (x v : E)
  (ha : DifferentiableAt ℝ a x) (hS : DifferentiableAt ℝ S x) :
  HasDerivAt (fun t : ℝ => psi a S (x + t • v))
    (Complex.exp (Complex.I * (S x : ℂ)) *
      ((fderiv ℝ a x v : ℝ) + Complex.I * (a x : ℂ) * (fderiv ℝ S x v : ℝ))) 0

theorem norm_sq_deriv_psi (a S : E → ℝ) (x v : E)
  (ha : DifferentiableAt ℝ a x) (hS : DifferentiableAt ℝ S x) :
  ||deriv (fun t : ℝ => psi a S (x + t • v)) 0|| ^ 2
    = (fderiv ℝ a x v) ^ 2 + (a x) ^ 2 * (fderiv ℝ S x v) ^ 2

*3theorem madelung_gradient_split {n : ℕ} (a S : EuclideanSpace ℝ (Fin n) → ℝ)
  (x : EuclideanSpace ℝ (Fin n))
  (ha : DifferentiableAt ℝ a x) (hS : DifferentiableAt ℝ S x) :
  ∑ i : Fin n, ||deriv (fun t : ℝ => psi a S (x + t • EuclideanSpace.single i 1)) 0|| ^ 2
    = ∑ i : Fin n, (fderiv ℝ a x (EuclideanSpace.single i 1)) ^ 2
      + (a x) ^ 2 * ∑ i : Fin n, (fderiv ℝ S x (EuclideanSpace.single i 1)) ^ 2

*3,9theorem kinetic_density_split (ħ m a ga gS : ℝ) (hm : m ≠ 0) :
  ħ ^ 2 / (2 * m) * (ga ^ 2 + a ^ 2 * gS ^ 2)
    = ħ ^ 2 / (2 * m) * ga ^ 2 + m / 2 * a ^ 2 * (ħ / m * gS) ^ 2

```

A.1.17 MatchingScreening

The vortex-charge structure factor certifies a long vortex-antivortex matching.

Namespace MatchingScreening · 10 theorems, 3 definitions · *audit*: 10 theorem constants audited (+2 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext; the source has no #print axioms line (footprint from the appended environment audit).

```

lemma norm_exp_sub_exp_le (a b : ℝ) :
  ||exp (I * (a : ℂ)) - exp (I * (b : ℂ))|| ≤ |a - b|

*6noncomputable def wave (k r : E) : ℂ

*6noncomputable def rho {N : ℕ} (k : E) (p m : Fin N → E) : ℂ

```

```

lemma norm_wave_sub_le (k r s : E) : ||wave k r - wave k s|| ≤ ||k|| * ||r - s||
*6theorem norm_rho_le_matching {N : ℕ} (k : E) (p m : Fin N → E) (σ : Fin N = Fin N) :
  ||rho k p m|| ≤ ||k|| * ∑ i, ||p i - m (σ i)||
*6theorem matching_lower_bound {N : ℕ} (k : E) (hk : k ≠ 0) (p m : Fin N → E) (σ : Fin N = Fin N) :
  ||rho k p m|| / ||k|| ≤ ∑ i, ||p i - m (σ i)||

theorem exists_long_pair {N : ℕ} (hN : 0 < N) (k : E) (hk : k ≠ 0) (p m : Fin N → E)
  (σ : Fin N = Fin N) : ∃ i, ||rho k p m|| / (N * ||k||) ≤ ||p i - m (σ i)||

lemma wave_add_period (k r t : E) (n : ℤ) (ht : inner ℝ k t = 2 * Real.pi * n) :
  wave k (r + t) = wave k r

*6theorem norm_rho_le_matching_torus {N : ℕ} (k : E) (p m : Fin N → E) (σ : Fin N = Fin N)
  (t : Fin N → E) (ht : ∀ i, ∃ n : ℤ, inner ℝ k (t i) = 2 * Real.pi * n) :
  ||rho k p m|| ≤ ||k|| * ∑ i, ||p i - (m (σ i) + t i)||

noncomputable def vhat (k₁ k₂ : ℝ) (r : ℂ) : ℂ × ℂ

theorem pointVortex_transverse (k₁ k₂ : ℝ) (r : ℂ) :
  (k₁ : ℂ) * (vhat k₁ k₂ r).1 + (k₂ : ℂ) * (vhat k₁ k₂ r).2 = 0

theorem pointVortex_norm_sq (k₁ k₂ : ℝ) (hk : k₁ ^ 2 + k₂ ^ 2 ≠ 0) (r : ℂ) :
  ||(vhat k₁ k₂ r).1|| ^ 2 + ||(vhat k₁ k₂ r).2|| ^ 2 = (2 * Real.pi) ^ 2 * ||r|| ^ 2 / (k₁ ^ 2 + k₂ ^ 2)
  ↪ 2)

lemma rho_one_pair : rho (E := ℝ) (N := 1) Real.pi (fun _ => 1) (fun _ => 0) = -2

```

A.1.18 PairPolarisation

Companion of docs/designs/PGPE_DIELECTRIC_PREREG.md (finite-k dielectric relation).

Namespace PairPolarisation · 8 theorems, 2 definitions · *audit*: 8 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext; the source has no #print axioms line (footprint from the appended environment audit).

```

noncomputable def wave (k r : E) : ℂ

noncomputable def rho {N : ℕ} (k : E) (p m : Fin N → E) : ℂ

lemma norm_wave (k r : E) : ||wave k r|| = 1

lemma pair_factor (k p m : E) :
  wave k p - wave k m = wave k m * (exp (I * (inner ℝ k (p - m) : ℝ) : ℂ)) - 1)

lemma norm_pair_le (k p m : E) : ||wave k p - wave k m|| ≤ ||k|| * ||p - m||

theorem norm_rho_le_pairs {N : ℕ} (k : E) (p m : Fin N → E) :
  ||rho k p m|| ≤ ||k|| * ∑ i, ||p i - m i||

theorem norm_rho_le_of_size {N : ℕ} (k : E) (p m : Fin N → E) {a : ℝ} (ha : ∀ i, ||p i - m i|| ≤ a)
  ↪ :
  ||rho k p m|| ≤ ||k|| * (N * a)

*6theorem rho_polarisation {N : ℕ} (k : E) (p m : Fin N → E)
  (hk : ∀ i, |inner ℝ k (p i - m i)| ≤ 1) :
  ||rho k p m - I * ∑ i, ((inner ℝ k (p i - m i) : ℝ) : ℂ) * wave k (m i)||
  ≤ ∑ i, (inner ℝ k (p i - m i)) ^ 2

theorem remainder_le {N : ℕ} (k : E) (p m : Fin N → E) :
  ∑ i, (inner ℝ k (p i - m i)) ^ 2 ≤ ||k|| ^ 2 * ∑ i, ||p i - m i|| ^ 2

theorem response_ceiling {N : ℕ} (k : E) (hk : k ≠ 0) (p m : Fin N → E) :
  (2 * Real.pi) ^ 2 * ||rho k p m|| ^ 2 / ||k|| ^ 2 ≤ (2 * Real.pi) ^ 2 * (∑ i, ||p i - m i||) ^ 2

```

A.1.19 PhaseMixing

Free transport forgets its initial density: the linear, field-free core of Landau damping.

Namespace QuantumFluids.PhaseMixing · 3 theorems, 0 definitions · *audit*: 3 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
theorem transport_solution (k v : ℝ) (gv : ℂ) (t x : ℝ) :
  HasDerivAt (fun t => cexp ((k * (x - v * t) : ℝ) * I) * gv)
    (-(v : ℂ) * ((k : ℂ) * I * (cexp ((k * (x - v * t) : ℝ) * I) * gv))) t x
  HasDerivAt (fun x => cexp ((k * (x - v * t) : ℝ) * I) * gv)
    ((k : ℂ) * I * (cexp ((k * (x - v * t) : ℝ) * I) * gv)) x
```

```
★theorem phase_mixing (g : ℝ → ℂ) {k : ℝ} (hk : k ≠ 0) :
  Tendsto (fun t : ℝ => ∫ v : ℝ, cexp (-(k * v * t : ℝ) * I) * g v) atTop (N 0)
```

```
★theorem maxwellian_mode (k t : ℝ) :
  ∫ v : ℝ, cexp (-(k * v * t : ℝ) * I) * ((rexp (-v ^ 2 / 2) / √(2 * π) : ℝ) : ℂ)
    = cexp (-(k * t) ^ 2 / 2 : ℝ)
```

A.1.20 PhononSeries

The algebra behind the phonon specific-heat series of Godfrin, Beauvois, Sultan, Krotscheck, Dawidowski, Faak, Ollivier, PRB 103, 104516 (2021), Eq. (22).

Namespace QuantumFluids.PhononSeries · 9 theorems, 10 definitions · *audit*: 9 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Quot.sound, propext.

```
def kInv (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
def kPow2 (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
★2theorem kPow2_eq (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kInv a1 a2 a3 a4 a5 a6 e * kInv a1 a2 a3 a4 a5 a6 e = kPow2 a1 a2 a3 a4 a5 a6 e
```

```
def kPow3 (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
theorem kPow3_eq (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kPow2 a1 a2 a3 a4 a5 a6 e * kInv a1 a2 a3 a4 a5 a6 e = kPow3 a1 a2 a3 a4 a5 a6 e
```

```
def kPow4 (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
theorem kPow4_eq (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kPow3 a1 a2 a3 a4 a5 a6 e * kInv a1 a2 a3 a4 a5 a6 e = kPow4 a1 a2 a3 a4 a5 a6 e
```

```
def kPow5 (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
theorem kPow5_eq (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kPow4 a1 a2 a3 a4 a5 a6 e * kInv a1 a2 a3 a4 a5 a6 e = kPow5 a1 a2 a3 a4 a5 a6 e
```

```
def kPow6 (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
theorem kPow6_eq (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kPow5 a1 a2 a3 a4 a5 a6 e * kInv a1 a2 a3 a4 a5 a6 e = kPow6 a1 a2 a3 a4 a5 a6 e
```

```
def kPow7 (a1 a2 a3 a4 a5 a6 e : ℝ) : ℝ
```

```
theorem kPow7_eq (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kPow6 a1 a2 a3 a4 a5 a6 e * kInv a1 a2 a3 a4 a5 a6 e = kPow7 a1 a2 a3 a4 a5 a6 e
```

```
★4theorem dispersion_kInv (a1 a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kInv a1 a2 a3 a4 a5 a6 e * (1 + a1 * kInv a1 a2 a3 a4 a5 a6 e + a2 * kInv a1 a2 a3 a4 a5 a6 e
    ↪ ^ 2
    + a3 * kInv a1 a2 a3 a4 a5 a6 e ^ 3 + a4 * kInv a1 a2 a3 a4 a5 a6 e ^ 4
    + a5 * kInv a1 a2 a3 a4 a5 a6 e ^ 5 + a6 * kInv a1 a2 a3 a4 a5 a6 e ^ 6) = e
```

```

*4def kInv0 (a2 a3 a4 a5 a6 e : R) : R
*4def kInv0Deriv (a2 a3 a4 a5 a6 e : R) : R
  def kSq0 (a2 a3 a4 a5 a6 e : R) : R
    theorem kSq0_eq (a2 a3 a4 a5 a6 e : R) (h : e ^ 9 = 0) :
      kInv0 a2 a3 a4 a5 a6 e ^ 2 = kSq0 a2 a3 a4 a5 a6 e
*4theorem density_of_states (a2 a3 a4 a5 a6 e : R) (h : e ^ 9 = 0) :
  kInv0 a2 a3 a4 a5 a6 e ^ 2 * kInv0Deriv a2 a3 a4 a5 a6 e
    = e ^ 2 - 5 * a2 * e ^ 4 - 6 * a3 * e ^ 5 + 7 * (4 * a2 ^ 2 - a4) * e ^ 6
      + 8 * (9 * a2 * a3 - a5) * e ^ 7
      - 3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6) * e ^ 8

```

A.1.2I PhononSpecificHeat

Assembling the phonon specific-heat series of Godfrin et al., PRB 103, 104516 (2021), Eq. (22):

Namespace QuantumFluids.PhononSpecificHeat · 14 theorems, 1 definition · *audit*: 14 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem bernoulli'_five : bernoulli' 5 = 0
theorem bernoulli'_six : bernoulli' 6 = 1 / 42
theorem bernoulli'_seven : bernoulli' 7 = 0
theorem bernoulli'_eight : bernoulli' 8 = -1 / 30
theorem bernoulli'_nine : bernoulli' 9 = 0
theorem bernoulli'_ten : bernoulli' 10 = 5 / 66
theorem riemannZeta_six : riemannZeta 6 = (π : ℂ) ^ 6 / 945
theorem riemannZeta_eight : riemannZeta 8 = (π : ℂ) ^ 8 / 9450
theorem riemannZeta_ten : riemannZeta 10 = (π : ℂ) ^ 10 / 93555
theorem thermal_bose_integral {n : ℕ} (hn : 1 ≤ n) {β : ℝ} (hβ : 0 < β) :
  mellin (fun u => bose (β * u)) (n + 1)
    = (β : ℂ) ^ (-(n : ℂ) + 1) * ((Nat.factorial n : ℂ) * riemannZeta (n + 1))
*4theorem bose_integral_values :
  mellin bose 4 = (π : ℂ) ^ 4 / 15 ∧ mellin bose 6 = 8 * (π : ℂ) ^ 6 / 63 ∧
  mellin bose 7 = 720 * riemannZeta 7 ∧ mellin bose 8 = 8 * (π : ℂ) ^ 8 / 15 ∧
  mellin bose 9 = 40320 * riemannZeta 9 ∧ mellin bose 10 = 128 * (π : ℂ) ^ 10 / 33
*4noncomputable def energyTerm (V kB hbar c g : ℝ) (n : ℕ) (I T : ℝ) : ℝ
  theorem energyTerm_two (V kB hbar c I T : ℝ) :
    energyTerm V kB hbar c 1 2 I T = debyeEnergy V kB hbar c I T
  theorem energyTerm_hasDerivAt (V kB hbar c g : ℝ) (n : ℕ) (I T : ℝ) :
    HasDerivAt (fun T => energyTerm V kB hbar c g n I T)
      (V / (2 * π ^ 2) * g * I * kB ^ (n + 2) / (hbar * c) ^ (n + 1) * ((n + 2 : ℕ) * T ^ (n + 1))
        ↪ ) T
*1,4theorem phonon_specific_heat (V kB hbar c a2 a3 a4 a5 a6 z7 z9 T : ℝ) (hh : hbar ≠ 0) (hc : c ≠ 0)
  ↪ :
  HasDerivAt (fun T =>
    energyTerm V kB hbar c 1 2 (π ^ 4 / 15) T
    + energyTerm V kB hbar c (-5 * a2) 4 (8 * π ^ 6 / 63) T
    + energyTerm V kB hbar c (-6 * a3) 5 (720 * z7) T

```

```

+ energyTerm V kB hbar c (7 * (4 * a2 ^ 2 - a4)) 6 (8 * π ^ 8 / 15) T
+ energyTerm V kB hbar c (8 * (9 * a2 * a3 - a5)) 7 (40320 * z9) T
+ energyTerm V kB hbar c (-3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)) 8
  (128 * π ^ 10 / 33) T)
( 2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3
+ -(40 * (π ^ 4 * a2 * kB ^ 6 * V)) / (21 * (c ^ 5 * hbar ^ 5)) * T ^ 5
+ -(15120 * (a3 * kB ^ 7 * V * z7)) / (π ^ 2 * c ^ 6 * hbar ^ 6) * T ^ 6
+ 224 * π ^ 6 * kB ^ 8 * V * (4 * a2 ^ 2 - a4) / (15 * c ^ 7 * hbar ^ 7) * T ^ 7
+ 1451520 * kB ^ 9 * V * z9 * (9 * a2 * a3 - a5) / (π ^ 2 * c ^ 8 * hbar ^ 8) * T ^ 8
+ -(640 * (π ^ 8 * kB ^ 10 * V * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)))
  / (11 * (c ^ 9 * hbar ^ 9)) * T ^ 9) T

```

A.1.22 QHFricke

Where the Fricke involution of Fricke.lean meets the quantum Hall modular group, and where it stops.

Namespace QuantumFluids.QHFricke · 6 theorems, 3 definitions · *audit*: 6 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def mob (a b c d : ℤ) (z : ℂ) : ℂ
noncomputable def crit : ℂ
def M : SL(2, ℤ)
theorem M_mem_Gamma0 : M ∈ Gamma0 2
theorem M_mul_M : (M : Matrix (Fin 2) (Fin 2) ℤ) * M = -1
theorem M_fixes_crit : mob 1 (-1) 2 (-1) crit = crit
theorem gamma0_two_odd_den (a b c d p q : ℤ) (hdet : a * d - b * c = 1) (hc : Even c) (hq : Odd q)
  ↪ :
  Odd (c * p + d * q)
theorem fricke_crit : mob 0 (-1) 2 0 crit = crit - 1
theorem fricke_plateau_not_plateau (p q r s : ℤ) (hp : p ≠ 0) (hq : Odd q) (hs : Odd s) :
  (r : ℚ) / s ≠ -(q : ℚ) / (2 * p)

```

A.1.23 QuantizedCirculation

Quantized circulation: the defining property of a quantum fluid.

Namespace QuantumFluids.Circulation · 6 theorems, 3 definitions · *audit*: 6 theorem constants audited (+1 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

*3 noncomputable def kappa (hbar m : ℝ) : ℝ
*3 noncomputable def circulation (hbar m : ℝ) (S : ℕ → ℝ) (n : ℕ) : ℝ
*1,3 theorem circulation_quantized (hbar m : ℝ) (S : ℕ → ℝ) (n : ℕ) (hclosed : S n = S 0) :
  ∃ q : ℤ, circulation hbar m S n = (q : ℝ) * kappa hbar m
theorem circulation_eq_zero_iff (hbar m : ℝ) (hhbar : hbar ≠ 0) (hm : m ≠ 0) (S : ℕ → ℝ) (n : ℕ) :
  circulation hbar m S n = 0 ↔ ∑ i ∈ Finset.range n, pdiff (S i) (S (i + 1)) = 0
*3 theorem abs_circulation_ge (hbar m : ℝ) (hhbar : 0 < hbar) (hm : 0 < m) (S : ℕ → ℝ) (n : ℕ)
  (hclosed : S n = S 0) (hne : circulation hbar m S n ≠ 0) :
  kappa hbar m ≤ |circulation hbar m S n|
theorem pdiff_of_mem {a b : ℝ} (h : b - a ∈ Set.Ioc (-π) π) : pdiff a b = b - a

```

```
theorem pdiff_of_wrap {a b : ℝ} (h : b - a + 2 * π ∈ Set.Ioc (-π) π) :  
  pdiff a b = b - a + 2 * π  
  
noncomputable def quarterLoop : ℕ → ℝ  
  
*3theorem circulation_quantum_attained (hbar m : ℝ) :  
  circulation hbar m quarterLoop 4 = kappa hbar m
```

A.1.24 *QuantumFluidsShell*

Formalisation of the conjugated complexification's energy conservation.

Namespace QuantumFluids.ShellComplex · 12 theorems, 3 definitions · *audit*: 12 theorem constants audited (+6 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
noncomputable def shellBc (k : ℕ → ℝ) : ℕ → (ℕ → ℂ) → ℂ  
  
noncomputable def out (k : ℕ → ℝ) (v : ℕ → ℂ) (m : ℕ) : ℝ  
  
theorem re_conj_sq_mul (a b : ℂ) :  
  ((starRingEnd ℂ) a * (starRingEnd ℂ) a * b).re  
  = ((starRingEnd ℂ) b * a * a).re  
  
theorem sum_re_conj_mul_shellBc (k : ℕ → ℝ) (v : ℕ → ℂ) :  
  ∀ N : ℕ, ∑ n ∈ Finset.range (N + 1),  
    ((starRingEnd ℂ) (v n) * shellBc k n v).re = -(out k v N)  
  
theorem shellBc_energy_conservation (k : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ)  
  (hbc : v (N + 1) = 0) :  
  ∑ n ∈ Finset.range (N + 1), ((starRingEnd ℂ) (v n) * shellBc k n v).re = 0  
  
theorem shellBc_real (k : ℕ → ℝ) (v : ℕ → ℂ) (hv : ∀ n, (v n).im = 0) (n : ℕ) :  
  (shellBc k n v).im = 0  
  
noncomputable def rtrace (f : ℂ →ℓ[ℝ] ℂ) : ℝ  
  
theorem rtrace_add (f g : ℂ →ℓ[ℝ] ℂ) : rtrace (f + g) = rtrace f + rtrace g  
  
theorem rtrace_mulRight (c : ℂ) : rtrace (LinearMap.mulRight ℝ c) = 2 * c.re  
  
theorem rtrace_mul_I (d : ℝ) :  
  rtrace (LinearMap.mulRight ℝ ((d : ℂ) * Complex.I)) = 0  
  
theorem rtrace_conj_mul (w : ℂ) :  
  rtrace ((LinearMap.mulRight ℝ w).comp Complex.conjAe.toLinearMap) = 0  
  
theorem shell_divergence_zero (d : ℝ) (w : ℂ) :  
  rtrace ((LinearMap.mulRight ℝ w).comp Complex.conjAe.toLinearMap  
    + LinearMap.mulRight ℝ ((d : ℂ) * Complex.I)) = 0  
  
theorem seam_conserves_iff (k : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (hk : k N ≠ 0) :  
  (∑ n ∈ Finset.range (N + 1), ((starRingEnd ℂ) (v n) * shellBc k n v).re = 0)  
  ↔ ((starRingEnd ℂ) (v N) * (starRingEnd ℂ) (v N) * v (N + 1)).re = 0  
  
theorem seam_zero_conserves (k : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (hk : k N ≠ 0)  
  (hbc : v (N + 1) = 0) :  
  ∑ n ∈ Finset.range (N + 1), ((starRingEnd ℂ) (v n) * shellBc k n v).re = 0  
  
theorem seam_gpe_conserves (k : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (μ : ℝ) (hk : k N ≠ 0)  
  (hbc : v (N + 1) = Complex.I * (μ : ℂ) * (v N * v N)) :  
  ∑ n ∈ Finset.range (N + 1), ((starRingEnd ℂ) (v n) * shellBc k n v).re = 0
```

A.1.25 QuasiPeriodicBound

Companion of amendment E-A1 of docs/designs/PGPE_EINSTEIN_PREREG.md (criterion I2) and of docs/designs/LEVERAGE_DONG2026.md.

Namespace QuasiPeriodicBound · 6 theorems, 1 definition · *audit*: 6 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext; the source has no #print axioms line (footprint from the appended environment audit).

```

★10 noncomputable def qp (a ω φ : ι → ℝ) (t : ℝ) : ℝ

  theorem trig_sum_bound (a ω φ : ι → ℝ) (t : ℝ) : |qp a ω φ t| ≤ ∑ k, |a k|

  theorem increment_bound (a ω φ : ι → ℝ) (t τ : ℝ) :
    |qp a ω φ (t + τ) - qp a ω φ t| ≤ 2 * ∑ k, |a k|

★7 theorem msd_bound (a ω φ : ι → ℝ) {J : Type*} [Fintype J] [Nonempty J] (ts τs : J → ℝ) :
  (∑ j, (qp a ω φ (ts j + τs j) - qp a ω φ (ts j)) ^ 2) / Fintype.card J ≤ (2 * ∑ k, |a k|) ^ 2

  theorem random_walk_exceeds (A c : ℝ) (hc : 0 < c) :
    ∀ τ, 4 * A ^ 2 / c < τ → (2 * A) ^ 2 < c * τ

★7,10 theorem not_qp_of_msd_large (a : ι → ℝ) {J : Type*} [Fintype J] [Nonempty J] (x : ℝ → ℝ) (ts τs :
  ↪ J → ℝ)
  (h : (2 * ∑ k, |a k|) ^ 2 < (∑ j, (x (ts j + τs j) - x (ts j)) ^ 2) / Fintype.card J) :
  ∀ ω φ : ι → ℝ, x ≠ qp a ω φ

  theorem relative_equilibrium_dipole (d0sq : ℝ) :
    ∀ t : ℝ, (d0sq - 4 * (0 : ℝ) * t) = d0sq

```

A.1.26 RCFTDuality

A sixth cross-domain case for the criterion of duality_sector.tex: the modular S-matrix of a rational conformal field theory (RCFT), sharper than SectorDuality.lean's Z_N case.

Namespace QuantumFluids.RCFTDuality · 3 theorems, 1 definition · *audit*: 3 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

def su2Level1_S : Matrix (Fin 2) (Fin 2) ℤ

theorem su2Level1_S_sq : su2Level1_S * su2Level1_S = (2 : ℤ) • (1 : Matrix (Fin 2) (Fin 2) ℤ)

theorem su2Level1_S_symm : su2Level1_S.transpose = su2Level1_S

theorem su2Level1_S_det : su2Level1_S.det = -2

```

A.1.27 RipsFloor

The bridge lemma of workstream T (docs/designs/TDA_VORTEX_FLOOR.md §7).

Namespace QuantumFluids.RipsFloor · 4 theorems, 2 definitions · *audit*: 4 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

def Adj (X : ι → E) (t : ℝ) (i j : ι) : Prop

def Separated (X : ι → E) (d : ℝ) : Prop

theorem no_adj_of_lt {X : ι → E} {d t : ℝ} (hsep : Separated X d) (ht : t < d) (i j : ι) :
  ¬ Adj X t i j

theorem le_of_adj {X : ι → E} {d t : ℝ} (hsep : Separated X d) (h : ∃ i j, Adj X t i j) : d ≤ t

theorem isLeast_edge_scale {X : ι → E} {d : ℝ} (hsep : Separated X d)
  {a b : ι} (hab : a ≠ b) (hd : dist (X a) (X b) = d) :
  IsLeast {t : ℝ | ∃ i j, Adj X t i j} d

```

theorem separated_zero_of_not_injective {X : $\mathfrak{t} \rightarrow E$ } : Separated X $\emptyset$

A.1.28 ScaleResolvedWinding

The winding number seen at scale R is the net topological charge inside R: a discrete Stokes theorem for phase windings, and its two corollaries, "a dipole is invisible from outside" and "a ...

Namespace QuantumFluids.ScaleResolvedWinding · 6 theorems, 4 definitions · *audit*: 11 theorem constants audited (+3 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
structure Region (m :  $\mathbb{N}$ )

noncomputable def Region.step (e : Fin m  $\times$  Fin 4) :  $\mathbb{R}$ 

noncomputable def Region.winding (p : Fin m) :  $\mathbb{R}$ 

noncomputable def Region.boundarySum :  $\mathbb{R}$ 

theorem winding_int (p : Fin m) :  $\exists q : \mathbb{Z}, \text{Rg.winding } p = (q : \mathbb{R}) * (2 * \pi)$ 

theorem interior_sum_eq_zero (hcut :  $\forall e \in \text{Rg.interior}, \text{Rg.step } e \neq \pi$ ) :
   $\sum e \in \text{Rg.interior}, \text{Rg.step } e = 0$ 

★ theorem stokes (hcut :  $\forall e \in \text{Rg.interior}, \text{Rg.step } e \neq \pi$ ) :
   $\text{Rg.boundarySum} = \sum p : \text{Fin } m, \text{Rg.winding } p$ 

theorem boundary_eq_charge (hcut :  $\forall e \in \text{Rg.interior}, \text{Rg.step } e \neq \pi$ ) (q : Fin m  $\rightarrow \mathbb{Z}$ )
  (hq :  $\forall p, \text{Rg.winding } p = (q p : \mathbb{R}) * (2 * \pi)$ ) :
   $\text{Rg.boundarySum} = ((\sum p, q p : \mathbb{Z}) : \mathbb{R}) * (2 * \pi)$ 

★ theorem dipole_invisible (hcut :  $\forall e \in \text{Rg.interior}, \text{Rg.step } e \neq \pi$ ) (q : Fin m  $\rightarrow \mathbb{Z}$ )
  (hq :  $\forall p, \text{Rg.winding } p = (q p : \mathbb{R}) * (2 * \pi)$ ) (hneutral :  $\sum p, q p = 0$ ) :
   $\text{Rg.boundarySum} = 0$ 

theorem single_core_visible (hcut :  $\forall e \in \text{Rg.interior}, \text{Rg.step } e \neq \pi$ ) (q : Fin m  $\rightarrow \mathbb{Z}$ )
  (hq :  $\forall p, \text{Rg.winding } p = (q p : \mathbb{R}) * (2 * \pi)$ ) (p0 : Fin m) (hothers :  $\forall p \neq p_0, q p = 0$ ) :
   $\text{Rg.boundarySum} = (q p_0 : \mathbb{R}) * (2 * \pi)$ 
```

A.1.29 SectorDuality

The finite-cyclic-group counterpart of CompactBoson's duality-as-reindexing statement: on a $\mathbb{Z}_N$ sector group, the duality *is* Mathlib's own discrete Fourier transform (ZMod.dft), and its ...

Namespace QuantumFluids.SectorDuality · 3 theorems, 0 definitions · *audit*: 3 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
theorem sectorDuality_bijective (N :  $\mathbb{N}$ ) [NeZero N] :
  Function.Bijective (ZMod.dft (N := N) (E :=  $\mathbb{C}$ ) : (ZMod N  $\rightarrow \mathbb{C}$ )  $\rightarrow$  (ZMod N  $\rightarrow \mathbb{C}$ ))

theorem sectorDuality_sq_ne_id (N :  $\mathbb{N}$ ) [NeZero N] (hN :  $2 \leq N$ ) :
  (ZMod.dft (N := N) (E :=  $\mathbb{C}$ ))  $\circ$  (ZMod.dft (N := N) (E :=  $\mathbb{C}$ ))  $\neq$  id

theorem kramersWannier_gauging_sq ( $\Phi$  : ZMod 2  $\rightarrow \mathbb{C}$ ) :
  (ZMod.dft (N := 2) (E :=  $\mathbb{C}$ )) ((ZMod.dft (N := 2) (E :=  $\mathbb{C}$ ))  $\Phi$ ) = (2 :  $\mathbb{C}$ )  $\cdot \Phi$ 
```

A.1.30 SectorTemperature

What "topological sectors carry their own temperature" means, exactly, and why a thermometer that averages over sectors cannot read "the" temperature of a mixed state.

Namespace QuantumFluids.SectorTemperature · 8 theorems, 5 definitions · *audit*: 8 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

def sectorCount (ε : ℝ → ℕ) (w : ℝ → ℤ) (W : ℤ) (E : ℕ) : ℕ

def globalCount (ε : ℝ → ℕ) (E : ℕ) : ℕ

def sectors (w : ℝ → ℤ) : Finset ℤ

theorem count_sum (ε : ℝ → ℕ) (w : ℝ → ℤ) (E : ℕ) :
  globalCount ε E = ∑ W ∈ sectors w, sectorCount ε w W E

theorem sectorCount_le (ε : ℝ → ℕ) (w : ℝ → ℤ) (W : ℤ) (E : ℕ) :
  sectorCount ε w W E ≤ globalCount ε E

theorem entropy_ge_sector (ε : ℝ → ℕ) (w : ℝ → ℤ) (W : ℤ) (E : ℕ) :
  Real.log (sectorCount ε w W E) ≤ Real.log (globalCount ε E)

theorem mediant_le {ι : Type*} (s : Finset ι) (a b : ι → ℝ) (r : ℝ) (hs : s.Nonempty)
  (hb : ∀ i ∈ s, 0 < b i) (h : ∀ i ∈ s, a i / b i ≤ r) :
  (∑ i ∈ s, a i) / (∑ i ∈ s, b i) ≤ r

theorem le_mediant {ι : Type*} (s : Finset ι) (a b : ι → ℝ) (r : ℝ) (hs : s.Nonempty)
  (hb : ∀ i ∈ s, 0 < b i) (h : ∀ i ∈ s, r ≤ a i / b i) :
  r ≤ (∑ i ∈ s, a i) / (∑ i ∈ s, b i)

noncomputable def sectorRatio (ε : ℝ → ℕ) (w : ℝ → ℤ) (W : ℤ) (E : ℕ) : ℝ

noncomputable def globalRatio (ε : ℝ → ℕ) (E : ℕ) : ℝ

theorem globalRatio_mem (ε : ℝ → ℕ) (w : ℝ → ℤ) (E : ℕ) (r₁ r₂ : ℝ)
  (hne : (sectors w).Nonempty)
  (hpos : ∀ W ∈ sectors w, 0 < sectorCount ε w W E)
  (hlo : ∀ W ∈ sectors w, r₁ ≤ sectorRatio ε w W E)
  (hhi : ∀ W ∈ sectors w, sectorRatio ε w W E ≤ r₂) :
  r₁ ≤ globalRatio ε E ∧ globalRatio ε E ≤ r₂

theorem global_eq_sector_of_single (ε : ℝ → ℕ) (w : ℝ → ℤ) (W₀ : ℤ) (E : ℕ)
  (hsingle : ∀ x, w x = W₀) :
  globalRatio ε E = sectorRatio ε w W₀ E

theorem sector_invariant (θ : ℝ → ℕ → ℝ) (n : ℕ) {t₀ t₁ : ℝ} (ht : t₀ ≤ t₁)
  (hcont : ∀ i ≤ n, Continuous (fun t => θ t i)) (hclosed : ∀ t, θ t n = θ t 0)
  (hslip : ∀ t ∈ Set.Icc t₀ t₁, ∀ i < n, VortexWinding.pdiff (θ t i) (θ t (i + 1)) ≠ Real.pi)
  (c : ℝ) (h₀ : loopSum θ n t₀ = c) :
  loopSum θ n t₁ = c

```

A.1.31 ShellHamiltonian

The second invariant of the complexified dyadic shell model (design memo docs/designs/DUAL_SCALE_SECOND_INVARIANT.md, addendum A1; CLAIM-023).

Namespace QuantumFluids.ShellComplex · 9 theorems, 2 definitions · *audit*: 9 theorem constants audited (+2 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def dT (k : ℕ → ℝ) (v : ℕ → ℂ) (n : ℕ) : ℝ

noncomputable def flux4 (v : ℕ → ℂ) (n : ℕ) : ℝ

theorem im_core (a b c d : ℂ) (p q r : ℝ) :
  (2 * (starRingEnd ℂ) b * (starRingEnd ℂ) ((p : ℂ) * (a * a) - (q : ℂ) * (starRingEnd ℂ) b * c)
  ↪ * c
  + (starRingEnd ℂ) b * (starRingEnd ℂ) b * ((q : ℂ) * (b * b) - (r : ℂ) * (starRingEnd ℂ) c *
  ↪ d)).im

```

```

= 2 * p * ((starRingEnd C) a * (starRingEnd C) a * (starRingEnd C) b * c).im
- r * ((starRingEnd C) b * (starRingEnd C) b * (starRingEnd C) c * d).im

theorem dT_zero (k : ℕ → ℝ) (v : ℕ → ℂ) : dT k v 0 = -(k 1 * flux4 v 0)

theorem dT_succ (k : ℕ → ℝ) (v : ℕ → ℂ) (m : ℕ) :
  dT k v (m + 1) = 2 * k m * flux4 v m - k (m + 2) * flux4 v (m + 1)

theorem sum_dT (k g : ℕ → ℝ) (v : ℕ → ℂ) (hg : ∀ n, g (n + 1) * (2 * k n) = g n * k (n + 1)) :
  ∀ N : ℕ, ∑ n ∈ Finset.range (N + 1), g n * dT k v n = -(g N * k (N + 1) * flux4 v N)

theorem hamiltonian_rate_zero (k g : ℕ → ℝ) (v : ℕ → ℂ)
  (hg : ∀ n, g (n + 1) * (2 * k n) = g n * k (n + 1)) (N : ℕ) (hbc : v (N + 2) = 0) :
  ∑ n ∈ Finset.range (N + 1), g n * dT k v n = 0

theorem weights_ok (k : ℕ → ℝ) (n : ℕ) :
  ((1 / 2 : ℝ) ^ (n + 1) * k (n + 1)) * (2 * k n) = ((1 / 2 : ℝ) ^ n * k n) * k (n + 1)

theorem T_real (v : ℕ → ℂ) (hv : ∀ n, (v n).im = 0) (n : ℕ) :
  ((starRingEnd C) (v n) * (starRingEnd C) (v n) * v (n + 1)).im = 0

theorem cubic_bound (c : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (hc : ∀ n, |c n| ≤ 1) :

theorem dispersive_norm_le (q c : ℕ → ℝ) (v : ℕ → ℂ) (N : ℕ) (Hval : ℝ) (hc : ∀ n, |c n| ≤ 1)
  (hH : Hval = ∑ n ∈ Finset.range (N + 2), q n * Complex.normSq (v n)
    + ∑ n ∈ Finset.range (N + 1), c n * ((starRingEnd C) (v n) * (starRingEnd C) (v n) * v (n +
    ↪ 1)).im) :
  ∑ n ∈ Finset.range (N + 2), q n * Complex.normSq (v n)
    ≤ Hval + Real.sqrt (∑ n ∈ Finset.range (N + 2), Complex.normSq (v n))
    * ∑ n ∈ Finset.range (N + 2), Complex.normSq (v n)

```

A.1.32 SigmaRule

The sigma-rule instantiated: which dispersive order controls which norm, uniformly in the cutoff.

Namespace QuantumFluids.ShellComplex · 6 theorems, 3 definitions · *audit*: 6 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def kdy (n : ℕ) : ℝ

noncomputable def gw (n : ℕ) : ℝ

theorem gw_mul_kdy (n : ℕ) : gw n * kdy n = 1

theorem gw_mul_kdy_pow (n s : ℕ) : gw n * kdy n ^ (s + 1) = kdy n ^ s

theorem sigma_two_weight (D : ℝ) (n : ℕ) : gw n * (D * kdy n ^ 2) = D * kdy n

theorem sigma_three_weight (D : ℝ) (n : ℕ) : gw n * (D * kdy n ^ 3) = D * kdy n ^ 2

noncomputable def enstrophy (v : ℕ → ℂ) (M : ℕ) : ℝ

theorem enstrophy_le_sigma_three (D : ℝ) (v : ℕ → ℂ) (N : ℕ) (Hval : ℝ)
  (hH : Hval = ∑ n ∈ Finset.range (N + 2), (gw n * (D * kdy n ^ 3)) * Complex.normSq (v n)
    + ∑ n ∈ Finset.range (N + 1), (gw n * kdy n) *
      ((starRingEnd C) (v n) * (starRingEnd C) (v n) * v (n + 1)).im) :
  2 * D * enstrophy v (N + 2)
    ≤ Hval + Real.sqrt (∑ n ∈ Finset.range (N + 2), Complex.normSq (v n))
    * ∑ n ∈ Finset.range (N + 2), Complex.normSq (v n)

theorem halfNorm_le_sigma_two (D : ℝ) (v : ℕ → ℂ) (N : ℕ) (Hval : ℝ)
  (hH : Hval = ∑ n ∈ Finset.range (N + 2), (gw n * (D * kdy n ^ 2)) * Complex.normSq (v n)
    + ∑ n ∈ Finset.range (N + 1), (gw n * kdy n) *

```

```

((starRingEnd ℂ) (v n) * (starRingEnd ℂ) (v n) * v (n + 1)).im) :
  ∑ n ∈ Finset.range (N + 2), (D * kdy n) * Complex.normSq (v n)
  ≤ Hval + Real.sqrt (∑ n ∈ Finset.range (N + 2), Complex.normSq (v n))
  * ∑ n ∈ Finset.range (N + 2), Complex.normSq (v n)

```

A.1.33 TopologicalProtection

Why a winding number can act as a CAUSE: it is conserved by every evolution that avoids a phase slip, so it is a memory the dynamics cannot erase continuously.

Namespace QuantumFluids.TopologicalProtection · *13* theorems, 2 definitions · *audit*: *13* theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem pdiff_shift (a b da db : ℝ) (h : pdiff a b + (db - da) ∈ Set.Ioc (-π) π) :
  pdiff (a + da) (b + db) = pdiff a b + (db - da)

theorem loop_sum_stable (θ δ : ℕ → ℝ) (n : ℕ) (hδ : δ n = δ 0)
  (h : ∀ i < n, pdiff (θ i) (θ (i + 1)) + (δ (i + 1) - δ i) ∈ Set.Ioc (-π) π) :
  ∑ i ∈ Finset.range n, pdiff (θ i + δ i) (θ (i + 1) + δ (i + 1))
  = ∑ i ∈ Finset.range n, pdiff (θ i) (θ (i + 1))

theorem loop_sum_stable_of_small (θ δ : ℕ → ℝ) (n : ℕ) (ε : ℝ) (hδ : δ n = δ 0)
  (hsmall : ∀ i ≤ n, |δ i| < ε) (hedge : ∀ i < n, |pdiff (θ i) (θ (i + 1))| < π - 2 * ε) :
  ∑ i ∈ Finset.range n, pdiff (θ i + δ i) (θ (i + 1) + δ (i + 1))
  = ∑ i ∈ Finset.range n, pdiff (θ i) (θ (i + 1))

theorem circulation_conserved (hbar m : ℝ) (θ δ : ℕ → ℝ) (n : ℕ) (hδ : δ n = δ 0)
  (h : ∀ i < n, pdiff (θ i) (θ (i + 1)) + (δ (i + 1) - δ i) ∈ Set.Ioc (-π) π) :
  Circulation.circulation hbar m (fun i => θ i + δ i) n = Circulation.circulation hbar m θ n

theorem continuousAt_pdiff {a b : ℝ} (h : pdiff a b ≠ π) :
  ContinuousAt (fun p : ℝ × ℝ => pdiff p.1 p.2) (a, b)

noncomputable def loopSum (θ : ℝ → ℕ → ℝ) (n : ℕ) (t : ℝ) : ℝ

theorem loop_sum_const_of_no_slip (θ : ℝ → ℕ → ℝ) (n : ℕ) {t₀ t₁ : ℝ} (ht : t₀ ≤ t₁)
  (hcont : ∀ i ≤ n, Continuous (fun t => θ t i))
  (hclosed : ∀ t, θ t n = θ t 0)
  (hslip : ∀ t ∈ Set.Icc t₀ t₁, ∀ i < n, pdiff (θ t i) (θ t (i + 1)) ≠ π) :
  loopSum θ n t₁ = loopSum θ n t₀

theorem slip_of_winding_change (θ : ℝ → ℕ → ℝ) (n : ℕ) {t₀ t₁ : ℝ} (ht : t₀ ≤ t₁)
  (hcont : ∀ i ≤ n, Continuous (fun t => θ t i)) (hclosed : ∀ t, θ t n = θ t 0)
  (hchange : loopSum θ n t₁ ≠ loopSum θ n t₀) :
  ∃ t ∈ Set.Icc t₀ t₁, ∃ i < n, pdiff (θ t i) (θ t (i + 1)) = π

noncomputable def xyEnergy (J : ℝ) (θ : ℕ → ℝ) (n : ℕ) : ℝ

theorem cos_pdiff (a b : ℝ) : Real.cos (pdiff a b) = Real.cos (b - a)

theorem slip_energy_ge (J : ℝ) (hJ : 0 ≤ J) (θ : ℕ → ℝ) (n j : ℕ) (hj : j < n)
  (hn : pdiff (θ j) (θ (j + 1)) = π) :
  -((n : ℝ) - 2) * J ≤ xyEnergy J θ n

theorem mountain_pass (J : ℝ) (hJ : 0 ≤ J) (θ : ℝ → ℕ → ℝ) (n : ℕ) {t₀ t₁ : ℝ} (ht : t₀ ≤ t₁)
  (hcont : ∀ i ≤ n, Continuous (fun t => θ t i)) (hclosed : ∀ t, θ t n = θ t 0)
  (hchange : loopSum θ n t₁ ≠ loopSum θ n t₀) :
  ∃ t ∈ Set.Icc t₀ t₁, -((n : ℝ) - 2) * J ≤ xyEnergy J (θ t) n

theorem twisted_loopSum (n : ℕ) (hn : 2 ≤ n) :
  ∑ i ∈ Finset.range n, pdiff (2 * π * i / n) (2 * π * (i + 1 : ℕ) / n) = 2 * π

theorem twisted_energy (J : ℝ) (n : ℕ) :

```

```
xyEnergy J (fun i => 2 * π * i / n) n = -(n : ℝ) * J * Real.cos (2 * π / n)
theorem barrier_pos (J : ℝ) (hJ : 0 < J) (n : ℕ) (hn : 10 ≤ n) :
  xyEnergy J (fun i => 2 * π * i / n) n < -((n : ℝ) - 2) * J
```

A.1.34 Villani

A foundation for formalizing Cédric Villani's work, offered as a tribute.

Namespace QuantumFluids.Villani · 18 theorems, 8 definitions · *audit*: 18 theorem constants audited (+4 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```
abbrev Cube (N : ℕ)

def diff (a b : Cube N) : Finset (Fin N)
def dist (a b : Cube N) : ℕ

theorem mem_diff_iff {a b : Cube N} {i : Fin N} : i ∈ diff a b ↔ a i ≠ b i
def IsMidpoint (a b m : Cube N) : Prop

noncomputable def midpointSet (A B : Finset (Cube N)) : Finset (Cube N)
def crossover (c : Finset (Fin N)) (a b : Cube N) : Cube N

@[simp] theorem crossover_apply_mem {c : Finset (Fin N)} {a b : Cube N} {i : Fin N} (h : i ∈ c) :
  crossover c a b i = a i
@[simp] theorem crossover_apply_not_mem {c : Finset (Fin N)} {a b : Cube N} {i : Fin N}
  (h : i ∉ c) : crossover c a b i = b i

theorem diff_crossover_left (c : Finset (Fin N)) (a b : Cube N) :
  diff (crossover c a b) a = diff a b \ c
theorem diff_crossover_right {c : Finset (Fin N)} {a b : Cube N} (hc : c ⊆ diff a b) :
  diff (crossover c a b) b = c

theorem dist_crossover_left (c : Finset (Fin N)) (a b : Cube N) :
  dist (crossover c a b) a = (diff a b \ c).card
theorem dist_crossover_right {c : Finset (Fin N)} {a b : Cube N} (hc : c ⊆ diff a b) :
  dist (crossover c a b) b = c.card

theorem isMidpoint_crossover {c : Finset (Fin N)} {a b : Cube N} (hc : c ⊆ diff a b)
  (h1 : 2 * c.card ≤ dist a b + 1) (h2 : dist a b ≤ 2 * c.card + 1) :
  IsMidpoint a b (crossover c a b)

def takeHalf (s : Finset (Fin N)) : Finset (Fin N)
theorem takeHalf_subset (s : Finset (Fin N)) : takeHalf s ⊆ s
theorem card_takeHalf (s : Finset (Fin N)) : (takeHalf s).card = s.card / 2
theorem takeHalf_card_bounds (a b : Cube N) :
  2 * (takeHalf (diff a b)).card ≤ dist a b + 1 ∧
  dist a b ≤ 2 * (takeHalf (diff a b)).card + 1

def encode (a b : Cube N) : Cube N × Cube N
theorem encodefst_isMidpoint (a b : Cube N) : IsMidpoint a b (encode a b).1
theorem encodesnd_isMidpoint (a b : Cube N) : IsMidpoint a b (encode a b).2
theorem encode_mem_midpointSet {A B : Finset (Cube N)} {a b : Cube N} (ha : a ∈ A) (hb : b ∈ B) :
  (encode a b).1 ∈ midpointSet A B ∧ (encode a b).2 ∈ midpointSet A B
theorem diff_crossover_crossover_compl {c : Finset (Fin N)} {a b : Cube N} (hc : c ⊆ diff a b) :
  diff (crossover c a b) (crossover (diff a b \ c) a b) = diff a b
```

```

theorem encode_injective {a b a' b' : Cube N} (h : encode a b = encode a' b') :
  a = a'  $\wedge$  b = b'

theorem card_sq_ge (A B : Finset (Cube N)) :
  A.card * B.card  $\leq$  (midpointSet A B).card * (midpointSet A B).card

theorem kLDiv_nonincreasing_to_invariant ( $\kappa$  : Kernel  $\mathcal{X} \mathcal{X}$ ) [IsMarkovKernel  $\kappa$ ]
  ( $\mu$   $\nu$  : Measure  $\mathcal{X}$ ) [IsFiniteMeasure  $\mu$ ] [IsFiniteMeasure  $\nu$ ] (hv :  $\kappa \circ_m \nu = \nu$ ) :
  kLDiv ( $\kappa \circ_m \mu$ )  $\nu \leq$  kLDiv  $\mu \nu$ 

```

A.1.35 VortexWinding

Correctness of phase-winding vortex detection.

Namespace QuantumFluids.VortexWinding · 9 theorems, 2 definitions · *audit*: 13 theorem constants audited (+4 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem two_pi_pos : (0 :  $\mathbb{R}$ ) < 2 *  $\pi$ 

★1,3,9noncomputable def pdiff (a b :  $\mathbb{R}$ ) :  $\mathbb{R}$ 

theorem pdiff_mem (a b :  $\mathbb{R}$ ) : pdiff a b  $\in$  Set.Ioc (- $\pi$ )  $\pi$ 

★1,3theorem pdiff_eq (a b :  $\mathbb{R}$ ) :
  pdiff a b = (b - a) - (toIocDiv two_pi_pos (- $\pi$ ) (b - a)) * (2 *  $\pi$ )

★1,3,9theorem loop_sum_eq_mul ( $\theta$  :  $\mathbb{N} \rightarrow \mathbb{R}$ ) (n :  $\mathbb{N}$ ) (hclosed :  $\theta n = \theta 0$ ) :
   $\exists m : \mathbb{Z}, \sum i \in \text{Finset.range } n, \text{pdiff } (\theta i) (\theta (i + 1)) = (m : \mathbb{R}) * (2 * \pi)$ 

theorem pdiff_add_rev_eq (a b :  $\mathbb{R}$ ) :
   $\exists m : \mathbb{Z}, \text{pdiff } a b + \text{pdiff } b a = (m : \mathbb{R}) * (2 * \pi)$ 

theorem pdiff_add_rev (a b :  $\mathbb{R}$ ) :
  pdiff a b + pdiff b a = 0  $\vee$  pdiff a b + pdiff b a = 2 *  $\pi$ 

★1,3,9theorem pdiff_add_rev_eq_two_pi_iff (a b :  $\mathbb{R}$ ) :
  pdiff a b + pdiff b a = 2 *  $\pi \leftrightarrow$  (pdiff a b =  $\pi \wedge$  pdiff b a =  $\pi$ )

structure Balanced {n :  $\mathbb{N}$ } (u v : Fin n  $\rightarrow \mathbb{R}$ )

★9theorem balanced_sum_eq_zero {n :  $\mathbb{N}$ } {u v : Fin n  $\rightarrow \mathbb{R}$ } (B : Balanced u v)
  (hcut :  $\forall i, \text{pdiff } (u i) (v i) + \text{pdiff } (v i) (u i) = 0$ ) :
   $\sum i : \text{Fin } n, \text{pdiff } (u i) (v i) = 0$ 

★9theorem cut_free_of_ne_pi {a b :  $\mathbb{R}$ } (h : pdiff a b  $\neq \pi$ ) : pdiff a b + pdiff b a = 0

```

A.1.36 WassersteinCertificate

P6 of the closed-loop project (docs/designs/CLOSED_LOOP_PREREG.md, docs/designs/CLOSED_LOOP_RESULTS.md).

Namespace QuantumFluids.WassersteinCertificate · 8 theorems, 6 definitions · *audit*: 8 theorem constants audited (+6 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem weak_duality (C :  $\iota \rightarrow \kappa \rightarrow \mathbb{R}$ ) (u :  $\iota \rightarrow \mathbb{R}$ ) (v :  $\kappa \rightarrow \mathbb{R}$ )
  (hfeas :  $\forall i j, u i + v j \leq C i j$ ) ( $\sigma : \iota = \kappa$ ) :
   $\sum i, u i + \sum j, v j \leq \sum i, C i (\sigma i)$ 

theorem certificate (C :  $\iota \rightarrow \kappa \rightarrow \mathbb{R}$ ) (u :  $\iota \rightarrow \mathbb{R}$ ) (v :  $\kappa \rightarrow \mathbb{R}$ )
  (hfeas :  $\forall i j, u i + v j \leq C i j$ ) ( $\sigma : \iota = \kappa$ )
  (htight :  $\sum i, C i (\sigma i) = \sum i, u i + \sum j, v j$ ) ( $\tau : \iota = \kappa$ ) :
   $\sum i, C i (\sigma i) \leq \sum i, C i (\tau i)$ 

noncomputable def C : Fin 6  $\rightarrow$  Fin 6  $\rightarrow \mathbb{R}$ 

```

```

theorem C_eq (i j : Fin 6) : C i j = ![![0, 1, 2, 3/2, 1000, 1000],
    ![1, 0, 3/2, 1000, 3/2, 1000],
    ![2, 1, 5/2, 1000, 1000, 3/2],
    ![3/2, 1000, 1000, 0, 0, 0],
    ![1000, 3/2, 1000, 0, 0, 0],
    ![1000, 1000, 1/4, 0, 0, 0]] i j

noncomputable def u : Fin 6 → ℝ
noncomputable def v : Fin 6 → ℝ
def ofun : Fin 6 → Fin 6
noncomputable def σ : Fin 6 ≈ Fin 6
theorem feasible : ∀ i j, u i + v j ≤ C i j
theorem tight : ∑ i, C i (σ i) = ∑ i, u i + ∑ j, v j
theorem toy_certificate (τ : Fin 6 ≈ Fin 6) : ∑ i, C i (σ i) ≤ ∑ i, C i (τ i)
theorem toy_cost_eq : ∑ i, C i (σ i) = 7 / 4
noncomputable def v_broken : Fin 6 → ℝ
theorem broken_infeasible : ¬ (∀ i j, u i + v_broken j ≤ C i j)

```

A.1.37 ZeroSound

When does a Fermi liquid support undamped zero sound?

Namespace QuantumFluids.ZeroSound · 6 theorems, 1 definition · *audit*: 7 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

*8 noncomputable def g3 (s : ℝ) : ℝ
    theorem g3_pos {s : ℝ} (hs : 1 < s) : 0 < g3 s
*8 theorem g3_le {s : ℝ} (hs : 1 < s) : g3 s ≤ 1 / (s - 1)
*8 theorem g3_ge {s : ℝ} (hs : 1 < s) : 1 / 2 * log (2 / (s - 1)) - 1 ≤ g3 s
*8 theorem continuous0n_g3 {a b : ℝ} (ha : 1 < a) : ContinuousOn g3 (Icc a b)
*8 theorem zero_sound_iff (F : ℝ) : (∃ s, 1 < s ∧ g3 s = 1 / F) ↔ 0 < F
*1,8 theorem zero_sound_2d_iff (F : ℝ) :
    (∃ s, 1 < s ∧ 1 + F * (1 - s / √(s ^ 2 - 1)) = 0) ↔ 0 < F

```

A.2 Lean written for this book

The files below are *new*: they were written for the book, are not part of the QuantumFluids library, and live in `book/lean/`. Each is compiled by its chapter author against the same pinned Mathlib (Lean 4.34.0-rc2), together with the library modules it imports when it says so, and, as the book's rules require, ends with `#print axioms` lines. Their statements are listed here in the same way as the library's; the status line says whether the book's audit driver has checked them. A file that is *meant to fail* (a negative control quoted in a chapter) is never counted as a theorem file or as a failure: such files are listed apart, at the end of this section. One more piece of Lean written for the book is not a module: the twenty-odd lines of meta-program that the audit driver appends to a scratch copy of every library module (printed in appendix B); they define no theorem, and they were compiled as

part of each of those runs.

A.2.1 Ch01_NeutronWindow

What a neutron can reach.

File book/lean/Ch01_NeutronWindow.lean · namespace QuantumFluids.NeutronWindow · 3 theorems, 1 definitions · 3 #print axioms lines in the file · audit: 3 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

★1 noncomputable def energy (hbar m : ℝ) (k : V) : ℝ

  theorem q_bounds (ki kf : V) :
    ||ki|| - ||kf|| ≤ ||ki - kf|| ∧ ||ki - kf|| ≤ ||ki|| + ||kf||

  theorem transfer_le_window (hbar m : ℝ) (hm : 0 < m) (ki kf : V) :
    energy hbar m ki - energy hbar m kf
      ≤ hbar ^ 2 * (2 * ||ki|| * ||ki - kf|| - ||ki - kf|| ^ 2) / (2 * m)

★1 theorem min_incident_wavevector (hbar m : ℝ) (hbar0 : 0 < hbar) (hm : 0 < m) (ki kf : V) (hQ : 0 <
  ↳ ||ki - kf||) :
    ||ki - kf|| / 2 + m * (energy hbar m ki - energy hbar m kf) / (hbar ^ 2 * ||ki - kf||) ≤ ||ki||

```

A.2.2 Ch02_Galerkin

NEW Lean written for Chapter 2 of the book "Quantum Fluids in Lean 4".

File book/lean/Ch02_Galerkin.lean · namespace QuantumFluids.Ch02Galerkin · 20 theorems, 6 definitions · 10 #print axioms lines in the file · audit: 20 theorem constants audited (+6 generated by Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

  noncomputable def n1 (Λ : Finset G) (ψ : G → ℂ) (k : G) : ℂ

★2 noncomputable def planeWave (a₀ : ℂ) (ω t : ℝ) : ℂ

  theorem norm_planeWave (a₀ : ℂ) (ω t : ℝ) : ||planeWave a₀ ω t|| = ||a₀||

  theorem planeWave_hasDerivAt (a₀ : ℂ) (ω t : ℝ) :
    HasDerivAt (planeWave a₀ ω) (-(Complex.I * ω) * planeWave a₀ ω t) t

  theorem n1_single (Λ : Finset G) (k0 : G) (hk0 : k0 ∈ Λ) (a : ℂ) (k : G) :
    n1 Λ (fun x => if x = k0 then a else 0) k = if k = k0 then a * (starRingEnd ℂ) a * a else 0

★2 noncomputable def planeState (k0 : G) (a₀ : ℂ) (Ω t : ℝ) : G → ℂ

★2 noncomputable def galerkinField (Λ : Finset G) (ω : G → ℝ) (g : ℝ) (ψ : G → ℂ) (k : G) : ℂ

★2 theorem planeState_solves_galerkin (Λ : Finset G) (k0 : G) (hk0 : k0 ∈ Λ) (ω : G → ℝ) (g : ℝ)
  (a₀ : ℂ) (k : G) (t : ℝ) :
    HasDerivAt (fun s : ℝ => planeState k0 a₀ (ω k0 + g * ||a₀|| ^ 2) s k)
      (galerkinField Λ ω g (planeState k0 a₀ (ω k0 + g * ||a₀|| ^ 2) t) k) t

  noncomputable def F (ψ : G → ℂ) (a b c d : G) : ℂ

  noncomputable def S4 (Λ : Finset G) (f : G → G → G → G → ℂ) : ℂ

  theorem S4_congr (Λ : Finset G) {f g : G → G → G → G → ℂ} (h : ∀ a b c d, f a b c d = g a b c d) :
    S4 Λ f = S4 Λ g

  theorem S4_add (Λ : Finset G) (f g : G → G → G → G → ℂ) :
    S4 Λ (fun a b c d => f a b c d + g a b c d) = S4 Λ f + S4 Λ g

  theorem S4_conj (Λ : Finset G) (f : G → G → G → G → ℂ) :
    (starRingEnd ℂ) (S4 Λ f) = S4 Λ (fun a b c d => (starRingEnd ℂ) (f a b c d))

  theorem S4_swap_ac (Λ : Finset G) (f : G → G → G → G → ℂ) :

```

```

S4  $\wedge$  f = S4  $\wedge$  (fun a b c d => f c b a d)

theorem S4_swap_bd ( $\Lambda$  : Finset G) (f : G  $\rightarrow$  G  $\rightarrow$  G  $\rightarrow$  G  $\rightarrow$   $\mathbb{C}$ ) :
  S4  $\wedge$  f = S4  $\wedge$  (fun a b c d => f a d c b)

theorem S4_swap_pairs ( $\Lambda$  : Finset G) (f : G  $\rightarrow$  G  $\rightarrow$  G  $\rightarrow$  G  $\rightarrow$   $\mathbb{C}$ ) :
  S4  $\wedge$  f = S4  $\wedge$  (fun a b c d => f b a d c)

theorem F_swap_ac ( $\psi$  : G  $\rightarrow$   $\mathbb{C}$ ) (a b c d : G) : F  $\psi$  c b a d = F  $\psi$  a b c d

theorem F_swap_bd ( $\psi$  : G  $\rightarrow$   $\mathbb{C}$ ) (a b c d : G) : F  $\psi$  a d c b = F  $\psi$  a b c d

theorem F_conj ( $\psi$  : G  $\rightarrow$   $\mathbb{C}$ ) (a b c d : G) : (starRingEnd  $\mathbb{C}$ ) (F  $\psi$  a b c d) = F  $\psi$  b a d c

theorem weighted_pairing_eq_S4 ( $\Lambda$  : Finset G) ( $\psi$  : G  $\rightarrow$   $\mathbb{C}$ ) (p : G  $\rightarrow$   $\mathbb{R}$ ) :
   $\sum k \in \Lambda, (p\ k : \mathbb{C}) * ((\text{starRingEnd } \mathbb{C}) (\psi\ k) * n\ell\ \Lambda\ \psi\ k)$ 
  = S4  $\wedge$  (fun a b c d => (p a :  $\mathbb{C}$ ) * F  $\psi$  a b c d)

theorem weighted_pairing_real ( $\Lambda$  : Finset G) ( $\psi$  : G  $\rightarrow$   $\mathbb{C}$ ) (p : G  $\rightarrow$   $\mathbb{R}$ ) :
  ( $\sum k \in \Lambda, (p\ k : \mathbb{C}) * ((\text{starRingEnd } \mathbb{C}) (\psi\ k) * n\ell\ \Lambda\ \psi\ k)$ ).im = 0

*2 theorem momentum_rate_zero ( $\Lambda$  : Finset G) ( $\psi$  : G  $\rightarrow$   $\mathbb{C}$ ) ( $\omega$  : G  $\rightarrow$   $\mathbb{R}$ ) (g :  $\mathbb{R}$ ) (p : G  $\rightarrow$   $\mathbb{R}$ ) :
   $\sum k \in \Lambda, p\ k * ((\text{starRingEnd } \mathbb{C}) (\psi\ k) * ((\omega\ k : \mathbb{C}) * \psi\ k + (g : \mathbb{C}) * n\ell\ \Lambda\ \psi\ k)).im = 0$ 

theorem momentum_rate_zero_lattice ( $\Lambda$  : Finset ( $\mathbb{Z} \times \mathbb{Z}$ )) ( $\psi$  :  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{C}$ ) ( $\omega$  :  $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{R}$ ) (g :  $\mathbb{R}$ ) :
   $\sum k \in \Lambda, (k.1 : \mathbb{R}) * ((\text{starRingEnd } \mathbb{C}) (\psi\ k) * ((\omega\ k : \mathbb{C}) * \psi\ k + (g : \mathbb{C}) * n\ell\ \Lambda\ \psi\ k)).im = 0$ 

theorem additive_map_vanishes_on_torsion {H : Type*} [AddCommGroup H] (p : H  $\rightarrow$   $\mathbb{R}$ ) (x : H) (n :  $\mathbb{N}$ )
  (hn : n  $\neq$  0) (hx : n • x = 0) : p x = 0

*2 theorem no_momentum_on_torus (N :  $\mathbb{N}$ ) [NeZero N] (p : (ZMod N  $\times$  ZMod N)  $\rightarrow$   $\mathbb{R}$ ) (x : ZMod N  $\times$  ZMod N)
   $\hookrightarrow$  :
  p x = 0

theorem no_additive_wavenumber (N :  $\mathbb{N}$ ) [NeZero N] :  $\neg \exists p : \text{ZMod } N \rightarrow \mathbb{R}, p\ 1 = 1$ 

```

A.2.3 Ch02_Primer

(1) types, propositions and proofs in four lines; (2) Lean also runs programs: a computation on Float and a theorem about $\mathbb{R}$ are different things; (3) linear_combination checks a certificate ...

File book/lean/Ch02_Primer.lean · namespace QuantumFluids.Ch02Primer · 4 theorems, 0 definitions · 4 #print axioms lines in the file · audit: 4 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

*2 theorem and_swap' (p q : Prop) (hp : p) (hq : q) : q  $\wedge$  p

*2 theorem two_add_two : (2 :  $\mathbb{R}$ ) + 2 = 4

theorem tenth_plus_fifth : (1 / 10 + 2 / 10 :  $\mathbb{R}$ ) = 3 / 10

*2 theorem one_add_nilpotent_inv {R : Type*} [CommRing R] (e : R) (h : e ^ 2 = 0) :
  (1 + e) * (1 - e) = 1

```

A.2.4 Ch03_PointVortexPair

The reduced model against which the Gross-Pitaevskii vortex pair is compared in the chapter: point vortices in the plane, the velocity induced at z by a vortex of circulation Γ at w being i ...

File book/lean/Ch03_PointVortexPair.lean · namespace QuantumFluids.Ch03PointVortexPair · 8 theorems, 1 definitions · 7 #print axioms lines in the file · audit: 8 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

★3noncomputable def induced (Γ : ℝ) (z w : ℂ) : ℂ
  theorem cancel_aux (a b c : ℂ) (hb : b ≠ 0) (hc : c ≠ 0) : a / (b * c) * c = a / b
★3theorem induced_mul_conj (Γ : ℝ) {z w : ℂ} (h : z ≠ w) :
  induced Γ z w * (starRingEnd ℂ) (z - w) = ((Γ / (2 * Real.pi) : ℝ) : ℂ) * Complex.I
  theorem induced_perp (Γ : ℝ) {z w : ℂ} (h : z ≠ w) :
    (induced Γ z w * (starRingEnd ℂ) (z - w)).re = 0
  theorem norm_induced_mul (Γ : ℝ) {z w : ℂ} (h : z ≠ w) :
    ||induced Γ z w|| * ||z - w|| = |Γ| / (2 * Real.pi)
  theorem norm_induced (Γ : ℝ) {z w : ℂ} (h : z ≠ w) :
    ||induced Γ z w|| = |Γ| / (2 * Real.pi * ||z - w||)
★3theorem pair_velocities_equal (Γ : ℝ) (z₁ z₂ : ℂ) :
  induced (-Γ) z₁ z₂ = induced Γ z₂ z₁
★3theorem separation_const (Γ : ℝ) (z₁ z₂ : ℝ → ℂ)
  (h₁ : ∀ t, HasDerivAt z₁ (induced (-Γ) (z₁ t) (z₂ t)) t)
  (h₂ : ∀ t, HasDerivAt z₂ (induced Γ (z₂ t) (z₁ t)) t) (t : ℝ) :
  z₁ t - z₂ t = z₁ 0 - z₂ 0
★3theorem pair_speed_quantum (ħ m : ℝ) (ħh : 0 < ħ) (hm : 0 < m) {z w : ℂ} (h : z ≠ w) :
  ||induced (2 * Real.pi * ħ / m) z w|| = ħ / (m * ||z - w||)

```

A.2.5 Ch04_DosLink

Why.

File book/lean/Ch04_DosLink.lean · namespace QuantumFluids.DosLink · 13 theorems, 4 definitions · 7 #print axioms lines in the file · audit: 13 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

theorem kInv_zero (a2 a3 a4 a5 a6 e : ℝ) :
  kInv 0 a2 a3 a4 a5 a6 e = kInv0 a2 a3 a4 a5 a6 e
theorem kInv0_inverts (a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 8 = 0) :
  kInv0 a2 a3 a4 a5 a6 e * (1 + a2 * kInv0 a2 a3 a4 a5 a6 e ^ 2 + a3 * kInv0 a2 a3 a4 a5 a6 e ^
  ↪ 3
    + a4 * kInv0 a2 a3 a4 a5 a6 e ^ 4 + a5 * kInv0 a2 a3 a4 a5 a6 e ^ 5
    + a6 * kInv0 a2 a3 a4 a5 a6 e ^ 6) = e
def dosRhs (a2 a3 a4 a5 a6 e : ℝ) : ℝ
theorem dos_eq (a2 a3 a4 a5 a6 e : ℝ) (h : e ^ 9 = 0) :
  kInv0 a2 a3 a4 a5 a6 e ^ 2 * kInv0Deriv a2 a3 a4 a5 a6 e = dosRhs a2 a3 a4 a5 a6 e
theorem map_kInv0 {S : Type*} [CommRing S] (f : R →+* S) (a2 a3 a4 a5 a6 e : ℝ) :
  f (kInv0 a2 a3 a4 a5 a6 e) = kInv0 (f a2) (f a3) (f a4) (f a5) (f a6) (f e)
theorem map_kInv0Deriv {S : Type*} [CommRing S] (f : R →+* S) (a2 a3 a4 a5 a6 e : ℝ) :
  f (kInv0Deriv a2 a3 a4 a5 a6 e) = kInv0Deriv (f a2) (f a3) (f a4) (f a5) (f a6) (f e)
theorem map_dosRhs {S : Type*} [CommRing S] (f : R →+* S) (a2 a3 a4 a5 a6 e : ℝ) :
  f (dosRhs a2 a3 a4 a5 a6 e) = dosRhs (f a2) (f a3) (f a4) (f a5) (f a6) (f e)
★4noncomputable def kPoly (a2 a3 a4 a5 a6 : ℝ) : ℝ[X]
★4theorem derivative_kPoly (a2 a3 a4 a5 a6 : ℝ) :
  derivative (kPoly a2 a3 a4 a5 a6) = kInv0Deriv (C a2) (C a3) (C a4) (C a5) (C a6) X
★4noncomputable def dosPoly (a2 a3 a4 a5 a6 : ℝ) : ℝ[X]
  theorem dosRhs_poly (a2 a3 a4 a5 a6 : ℝ) :

```

```

dosRhs (C a2) (C a3) (C a4) (C a5) (C a6) (X : ℝ[X]) =
  X ^ 2 - C (5 * a2) * X ^ 4 - C (6 * a3) * X ^ 5 + C (7 * (4 * a2 ^ 2 - a4)) * X ^ 6
  + C (8 * (9 * a2 * a3 - a5)) * X ^ 7
  - C (3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)) * X ^ 8

★4theorem dos_coeffs (a2 a3 a4 a5 a6 : ℝ) :
  (dosPoly a2 a3 a4 a5 a6).coeff 2 = 1 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 3 = 0 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 4 = -5 * a2 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 5 = -6 * a3 ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 6 = 7 * (4 * a2 ^ 2 - a4) ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 7 = 8 * (9 * a2 * a3 - a5) ∧
  (dosPoly a2 a3 a4 a5 a6).coeff 8 = -3 * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)

theorem phonon_specific_heat_from_dos (V kB hbar c a2 a3 a4 a5 a6 z7 z9 T : ℝ) (hh : hbar ≠ 0) (
  ↪ hc0 : c ≠ 0) :
  HasDerivAt (fun T =>
    energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 2) 2 (π ^ 4 / 15) T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 4) 4 (8 * π ^ 6 / 63) T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 5) 5 (720 * z7) T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 6) 6 (8 * π ^ 8 / 15) T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 7) 7 (40320 * z9) T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 8) 8 (128 * π ^ 10 / 33) T)
    ( 2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3
    + -(40 * (π ^ 4 * a2 * kB ^ 6 * V)) / (21 * (c ^ 5 * hbar ^ 5)) * T ^ 5
    + -(15120 * (a3 * kB ^ 7 * V * z7)) / (π ^ 2 * c ^ 6 * hbar ^ 6) * T ^ 6
    + 224 * π ^ 6 * kB ^ 8 * V * (4 * a2 ^ 2 - a4) / (15 * c ^ 7 * hbar ^ 7) * T ^ 7
    + 1451520 * kB ^ 9 * V * z9 * (9 * a2 * a3 - a5) / (π ^ 2 * c ^ 8 * hbar ^ 8) * T ^ 8
    + -(640 * (π ^ 8 * kB ^ 10 * V * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)))
    / (11 * (c ^ 9 * hbar ^ 9)) * T ^ 9) T

★4theorem phonon_specific_heat_end_to_end (V kB hbar c a2 a3 a4 a5 a6 T : ℝ) (hh : hbar ≠ 0) (hc0 :
  ↪ c ≠ 0) :
  HasDerivAt (fun T =>
    energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 2) 2 (mellin bose 4).re T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 4) 4 (mellin bose 6).re T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 5) 5 (mellin bose 7).re T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 6) 6 (mellin bose 8).re T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 7) 7 (mellin bose 9).re T
    + energyTerm V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 8) 8 (mellin bose 10).re T)
    ( 2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3
    + -(40 * (π ^ 4 * a2 * kB ^ 6 * V)) / (21 * (c ^ 5 * hbar ^ 5)) * T ^ 5
    + -(15120 * (a3 * kB ^ 7 * V * (riemannZeta 7).re)) / (π ^ 2 * c ^ 6 * hbar ^ 6) * T ^ 6
    + 224 * π ^ 6 * kB ^ 8 * V * (4 * a2 ^ 2 - a4) / (15 * c ^ 7 * hbar ^ 7) * T ^ 7
    + 1451520 * kB ^ 9 * V * (riemannZeta 9).re * (9 * a2 * a3 - a5) / (π ^ 2 * c ^ 8 * hbar ^
  ↪ 8) * T ^ 8
    + -(640 * (π ^ 8 * kB ^ 10 * V * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)))
    / (11 * (c ^ 9 * hbar ^ 9)) * T ^ 9) T

noncomputable def energyMonomial (V kB hbar c g : ℝ) (n : ℕ) (T : ℝ) : ℝ

★4theorem energyMonomial_eq (V kB hbar c g : ℝ) (n : ℕ) {T : ℝ} (hbc : 0 < hbar * c) (hkT : 0 < kB *
  ↪ T) :
  energyMonomial V kB hbar c g n T
  = energyTerm V kB hbar c g n (mellin bose ((n : ℂ) + 2)).re T

★4theorem phonon_specific_heat_integral (V kB hbar c a2 a3 a4 a5 a6 T : ℝ)
  (hh : 0 < hbar) (hc : 0 < c) (hk : 0 < kB) (hT : 0 < T) :

```

```

HasDerivAt (fun T =>
  energyMonomial V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 2) 2 T
+ energyMonomial V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 4) 4 T
+ energyMonomial V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 5) 5 T
+ energyMonomial V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 6) 6 T
+ energyMonomial V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 7) 7 T
+ energyMonomial V kB hbar c ((dosPoly a2 a3 a4 a5 a6).coeff 8) 8 T)
( 2 * π ^ 2 * kB ^ 4 * V / (15 * c ^ 3 * hbar ^ 3) * T ^ 3
+ -(40 * (π ^ 4 * a2 * kB ^ 6 * V)) / (21 * (c ^ 5 * hbar ^ 5)) * T ^ 5
+ -(15120 * (a3 * kB ^ 7 * V * (riemannZeta 7).re)) / (π ^ 2 * c ^ 6 * hbar ^ 6) * T ^ 6
+ 224 * π ^ 6 * kB ^ 8 * V * (4 * a2 ^ 2 - a4) / (15 * c ^ 7 * hbar ^ 7) * T ^ 7
+ 1451520 * kB ^ 9 * V * (riemannZeta 9).re * (9 * a2 * a3 - a5) / (π ^ 2 * c ^ 8 * hbar ^
↪ 8) * T ^ 8
+ -(640 * (π ^ 8 * kB ^ 10 * V * (55 * a2 ^ 3 - 30 * a2 * a4 - 15 * a3 ^ 2 + 3 * a6)))
/ (11 * (c ^ 9 * hbar ^ 9)) * T ^ 9) T

```

A.2.6 Ch05_BogoliubovDispersion

The Bogoliubov spectrum of a weakly interacting Bose gas (units $\hbar = 1$),
File book/lean/Ch05_BogoliubovDispersion.lean · *namespace* QuantumFluids.BogoliubovDispersion · 24
 theorems, 3 definitions · 15 #print axioms lines in the file · *audit*: 26 theorem constants audited (+1 generated by
 Lean), 0 sorry, 0 user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

*5noncomputable def bogEps (c m k : ℝ) : ℝ

*5noncomputable def alpha2 (c m : ℝ) : ℝ

def phononDisp (c a k : ℝ) : ℝ

theorem bogEps_sq (c m k : ℝ) :
  bogEps c m k ^ 2 = c ^ 2 * k ^ 2 + (k ^ 2 / (2 * m)) ^ 2

theorem bogEps_zero (c m : ℝ) : bogEps c m 0 = 0

theorem bogEps_nonneg (c m k : ℝ) : 0 ≤ bogEps c m k

theorem bogEps_pos {c m k : ℝ} (hc : 0 < c) (hk : 0 < k) : 0 < bogEps c m k

theorem bogEps_neg (c m k : ℝ) : bogEps c m (-k) = bogEps c m k

theorem bogEps_eq_mul {c m k : ℝ} (hm : 0 < m) (hk : 0 ≤ k) :
  bogEps c m k = k * Real.sqrt (c ^ 2 + k ^ 2 / (4 * m ^ 2))

*5theorem bogEps_strictMono0n {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  StrictMono0n (bogEps c m) (Set.Ici 0)

theorem sound_line_le (c m k : ℝ) : c * k ≤ bogEps c m k

*5theorem bogEps_le_phononDisp {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk : 0 ≤ k) :
  bogEps c m k ≤ phononDisp c (alpha2 c m) k

theorem phononDisp_sub_le_bogEps {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk : 0 ≤ k) :
  phononDisp c (alpha2 c m) k - c * (alpha2 c m) ^ 2 * k ^ 5 / 2 ≤ bogEps c m k

theorem sound_speed_limit {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  Tendsto (fun k => bogEps c m k / k) (N[>] 0) (N c)

*5theorem alpha2_limit {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  Tendsto (fun k => (bogEps c m k / (c * k) - 1) / k ^ 2) (N[>] 0) (N (alpha2 c m))

theorem alpha2_pos {c m : ℝ} (hc : 0 < c) (hm : 0 < m) : 0 < alpha2 c m

*5theorem sound_lt_phase_velocity {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk : 0 < k) :
  c < bogEps c m k / k

```

```

*5,9 theorem landau_velocity_eq {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  IsGLB ((fun k => bogEps c m k / k) '' Set.Ioi 0) c

theorem landau_velocity_sInf {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  sInf ((fun k => bogEps c m k / k) '' Set.Ioi 0) = c

*5 theorem bogEps_superadditive {c m k₁ k₂ : ℝ} (hc : 0 < c) (hm : 0 < m) (h₁ : 0 < k₁) (h₂ : 0 < k₂)
  ↪ :
  bogEps c m k₁ + bogEps c m k₂ < bogEps c m (k₁ + k₂)

theorem three_phonon_excess (c a k₁ k₂ : ℝ) :
  phononDisp c a (k₁ + k₂) - phononDisp c a k₁ - phononDisp c a k₂
  = 3 * c * a * k₁ * k₂ * (k₁ + k₂)

theorem three_phonon_open_iff {c a k₁ k₂ : ℝ} (hc : 0 < c) (h₁ : 0 < k₁) (h₂ : 0 < k₂) :
  phononDisp c a k₁ + phononDisp c a k₂ ≤ phononDisp c a (k₁ + k₂) ↔ 0 ≤ a

theorem bogoliubov_lowk_three_phonon_open {c m k₁ k₂ : ℝ} (hc : 0 < c) (hm : 0 < m) (h₁ : 0 < k₁)
  ↪ (h₂ : 0 < k₂) :
  phononDisp c (alpha2 c m) k₁ + phononDisp c (alpha2 c m) k₂ ≤ phononDisp c (alpha2 c m) (k₁ +
  ↪ k₂) ∧
  phononDisp c (alpha2 c m) (k₁ + k₂) - phononDisp c (alpha2 c m) k₁ - phononDisp c (alpha2 c m)
  ↪ k₂
  = 3 * c * alpha2 c m * k₁ * k₂ * (k₁ + k₂)

*5 theorem abs_bogEps_sub_phononDisp_le {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk : 0 ≤ k) :
  |bogEps c m k - phononDisp c (alpha2 c m) k| ≤ c * (alpha2 c m) ^ 2 * k ^ 5 / 2

*5 theorem bogEps_sub_sound_strictMonoOn {c m : ℝ} (hc : 0 < c) (hm : 0 < m) :
  StrictMonoOn (fun k => bogEps c m k - c * k) (Set.Ici 0)

*5 theorem phase_velocity_universal {c m k : ℝ} (hc : 0 < c) (hm : 0 < m) (hk : 0 < k) :
  bogEps c m k / (c * k) = Real.sqrt (1 + (k / (2 * m * c)) ^ 2)

*5 theorem bogEps_sq_gp {g n k : ℝ} (hg : 0 ≤ g) (hn : 0 ≤ n) :
  bogEps (Real.sqrt (g * n)) 1 k ^ 2 = (1 / 2) * k ^ 2 * ((1 / 2) * k ^ 2 + 2 * (g * n))

```

A.2.7 Cho6_KTForward

Why it exists.

File book/lean/Ch06_KTForward.lean · namespace QuantumFluids.KTForward · 18 theorems, 2 definitions · 18
 #print axioms lines in the file · audit: 18 theorem constants audited (+2 generated by Lean), o sorry, o user axioms;
 axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def f (u : ℝ) : ℝ

noncomputable def H (u y : ℝ) : ℝ

lemma f_antitone {a b : ℝ} (ha : 0 < a) (hab : a ≤ b) (hb : b ≤ π / 2) : f b ≤ f a

lemma f_gt_fc {u : ℝ} (hu : 0 < u) (hne : u ≠ π / 2) : f (π / 2) < f u

lemma f_ge_fc {u : ℝ} (hu : 0 < u) : f (π / 2) ≤ f u

lemma hasDerivWithinAt_H (l : ℝ) (hl : 0 ≤ l) :
  HasDerivWithinAt (fun l => H (u l) (y l)) 0 (Set.Ici 0) l

*6 theorem kt_invariant (l : ℝ) (hl : 0 ≤ l) : H (u l) (y l) = H (u 0) (y 0)

lemma f_ge_H (l : ℝ) (hl : 0 ≤ l) : H (u 0) (y 0) ≤ f (u l)

theorem kt_trapped (h0 : u 0 < π / 2) (hH : f (π / 2) < H (u 0) (y 0)) : ∀ l, 0 ≤ l → u l < π / 2

theorem u_monotoneOn : MonotoneOn u (Set.Ici 0)

```

```

theorem kt_fugacity_bounded (h0 : u 0 <  $\pi / 2$ ) (hH : f ( $\pi / 2$ ) < H (u 0) (y 0)) (l :  $\mathbb{R}$ ) (hl : 0  $\leq$ 
   $\hookrightarrow$  l) :
  y l  $^2 \leq$  y 0  $^2$ 

theorem kt_scalar (l :  $\mathbb{R}$ ) (hl : 0  $\leq$  l) :
  HasDerivWithinAt u (2 * (f (u l) - H (u 0) (y 0))) (Set.Ici 0) l

*6theorem kt_escape (l :  $\mathbb{R}$ ) (hl : 0  $\leq$  l) :
  u 0 + 2 * (f ( $\pi / 2$ ) - H (u 0) (y 0)) * l  $\leq$  u l

*6theorem kt_escape_time (hH : H (u 0) (y 0) < f ( $\pi / 2$ )) :
   $\exists$  l, 0  $\leq$  l  $\wedge$   $\pi / 2 \leq$  u l  $\wedge$ 
  l  $\leq$  max 0 (( $\pi / 2$  - u 0) / (2 * (f ( $\pi / 2$ ) - H (u 0) (y 0))))

theorem kt_stiffness_vanishes (hH : H (u 0) (y 0) < f ( $\pi / 2$ )) ( $\varepsilon$  :  $\mathbb{R}$ ) (h $\varepsilon$  : 0 <  $\varepsilon$ ) :
   $\exists$  L, 0  $\leq$  L  $\wedge$   $\forall$  l, L  $\leq$  l  $\rightarrow$  1 / u l <  $\varepsilon$ 

*6theorem kt_dichotomy (h0 : u 0 <  $\pi / 2$ ) (hne : H (u 0) (y 0)  $\neq$  f ( $\pi / 2$ )) :
  ( $\forall$  l, 0  $\leq$  l  $\rightarrow$  u l <  $\pi / 2$ )  $\vee$   $\exists$  l, 0  $\leq$  l  $\wedge$   $\pi / 2 \leq$  u l

lemma hasDerivAt_H_global (l :  $\mathbb{R}$ ) : HasDerivAt (fun l  $\Rightarrow$  H (u l) (y l)) 0 l

lemma kt_invariant_global (l :  $\mathbb{R}$ ) : H (u l) (y l) = H (u 0) (y 0)

lemma u_monotone_global : Monotone u

*6theorem kt_eternal_trivial (h0 : u 0 <  $\pi / 2$ ) : y 0 = 0

```

A.2.8 Ch07_DissipativePairs

Two additions to DissipativeVortexDynamics.lean, in the same model (hbar = m = 1, circulation 2 pi, normal fluid at rest): a vortex of charge q moves at $v = (1 - \alpha')$ $v_s - \alpha q z x$

...

File book/lean/Ch07_DissipativePairs.lean · namespace QuantumFluids.DissipativePairs · 7 theorems, 3 definitions · 7 #print axioms lines in the file · audit: 7 theorem constants audited (+1 generated by Lean), o sorry, o user axioms; axioms used: Classical.choice, Quot.sound, propext.

```

noncomputable def r2 (t :  $\mathbb{R}$ ) :  $\mathbb{R}$ 

theorem r2_hasDerivAt (t :  $\mathbb{R}$ ) (ht : t  $\in$  Icc (0 :  $\mathbb{R}$ ) T) : HasDerivAt (r2 x1 y1 x2 y2) (4 *  $\alpha$ ) t

*7theorem corot_sq_law (t :  $\mathbb{R}$ ) (ht : t  $\in$  Icc (0 :  $\mathbb{R}$ ) T) :
  r2 x1 y1 x2 y2 t = r2 x1 y1 x2 y2 0 + 4 *  $\alpha$  * t

noncomputable def stallPoint (w d0 :  $\mathbb{R}$ ) :  $\mathbb{R}$ 

noncomputable def lowerRoot (w d0 :  $\mathbb{R}$ ) :  $\mathbb{R}$ 

theorem roots_spec {w d0 :  $\mathbb{R}$ } (hw : 0 < w) (hd0 : 0 < d0) (h4 : 4 < w * d0  $^2$ ) :
  0 < lowerRoot w d0  $\wedge$  lowerRoot w d0 < stallPoint w d0  $\wedge$ 
  lowerRoot w d0 + stallPoint w d0 = d0  $\wedge$  lowerRoot w d0 * stallPoint w d0 = 1 / w

theorem stall_fence {a b c :  $\mathbb{R}$ } (hab : a < b) (hc : 0 < c) (g :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hg :  $\forall$  x, a < x  $\rightarrow$  x < b  $\rightarrow$ 
   $\hookrightarrow$  g x < 0)
  (d :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hd :  $\forall$  t, HasDerivAt d (-c * g (d t)) t) (h0 : b  $\leq$  d 0) :
   $\forall$  t, 0  $\leq$  t  $\rightarrow$  b  $\leq$  d t

*7theorem wind_stall_at_b { $\alpha$  w d0 :  $\mathbb{R}$ } (h $\alpha$  : 0 <  $\alpha$ ) (hw : 0 < w) (hd0 : 0 < d0) (h4 : 4 < w * d0  $^2$ 
   $\hookrightarrow$  2)
  (d :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (hd :  $\forall$  t, HasDerivAt d (-(2 *  $\alpha$ ) * (1 / d t - w * (d0 - d t))) t)
  (h0 : stallPoint w d0  $\leq$  d 0) :  $\forall$  t, 0  $\leq$  t  $\rightarrow$  stallPoint w d0  $\leq$  d t

*7theorem wind_no_zero {w d0 :  $\mathbb{R}$ } (hw : 0 < w) (h4 : w * d0  $^2$  < 4) {x :  $\mathbb{R}$ } (hx : 0 < x) :
  0 < 1 / x - w * (d0 - x)

```

```

theorem stall_needs_start {α w d0 : ℝ} (hw : 0 < w) (hd0 : 0 < d0) (h4 : 4 < w * d0 ^ 2) :
  (∀ t : ℝ, HasDerivAt (fun _ : ℝ => lowerRoot w d0)
    (-(2 * α) * (1 / lowerRoot w d0 - w * (d0 - lowerRoot w d0))) t) ∧
    lowerRoot w d0 < stallPoint w d0

```

A.2.9 Cho8_ZeroSound2D

Three small additions around ZeroSound.lean (which proves: an undamped zero-sound root exists iff the Landau parameter F is positive, in 2D and 3D, with the explicit 2D root $s = \dots$

File book/lean/Ch08_ZeroSound2D.lean · *namespace* QuantumFluids.ZeroSound2D · 9 theorems, 2 definitions · 9

#print axioms lines in the file · *audit*: 9 theorem constants audited, 0 sorry, 0 user axioms; axioms used:

Classical.choice, Quot.sound, propext.

```

*8 theorem ph_energy_window {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  {m kF : ℝ} (hm : 0 < m) (k q : E) (hk : ||k|| ≤ kF) :
  (||q|| ^ 2 - 2 * kF * ||q||) / (2 * m) ≤ (||k + q|| ^ 2 - ||k|| ^ 2) / (2 * m) ∧
  (||k + q|| ^ 2 - ||k|| ^ 2) / (2 * m) ≤ (||q|| ^ 2 + 2 * kF * ||q||) / (2 * m)

*8 theorem ph_gap {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  {m kF : ℝ} (hm : 0 < m) (hkF : 0 ≤ kF) (k q : E) (hk : ||k|| ≤ kF) (hq : 2 * kF < ||q||) :
  0 < ||q|| * (||q|| - 2 * kF) / (2 * m) ∧
  ||q|| * (||q|| - 2 * kF) / (2 * m) ≤ (||k + q|| ^ 2 - ||k|| ^ 2) / (2 * m)

*8 theorem ph_edge_attained {E : Type*} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  {kF : ℝ} (hkF : 0 ≤ kF) (q : E) (hq0 : q ≠ 0) (hq : 2 * kF ≤ ||q||) :
  ∃ k : E, ||k|| = kF ∧ kF ≤ ||k + q|| ∧ ||k + q|| ^ 2 - ||k|| ^ 2 = ||q|| ^ 2 - 2 * kF * ||q||

*8 noncomputable def omega2 (s : ℝ) : ℝ

*8 noncomputable def s0 (F : ℝ) : ℝ

*8 theorem zero_sound_2d_unique {F s : ℝ} (hs : 1 < s) (h : 1 + F * omega2 s = 0) :
  0 < F ∧ s = s0 F

*8 theorem s0_sq_sub_one {F : ℝ} (hF : 0 < F) : s0 F ^ 2 - 1 = F ^ 2 / (1 + 2 * F)

theorem one_lt_s0 {F : ℝ} (hF : 0 < F) : 1 < s0 F

*8 theorem omega2_hasDerivAt {s : ℝ} (hs : 1 < s) :
  HasDerivAt omega2 (1 / ((s ^ 2 - 1) * √(s ^ 2 - 1))) s

*8 theorem pole_weight {F : ℝ} (hF : 0 < F) :
  1 / (F ^ 2 * deriv omega2 (s0 F)) = F / ((1 + 2 * F) * √(1 + 2 * F))

*8 theorem pole_fraction {F : ℝ} (hF : 0 < F) :
  4 * s0 F * (1 / (F ^ 2 * deriv omega2 (s0 F))) = 1 - 1 / (1 + 2 * F) ^ 2

```

A.2.10 Ch09_FlowPast

Elementary statements about the model whose reproduction the chapter reports (units $\hbar = m = \mu = 1$, density at infinity $n = 1$, hence sound speed $c = 1$):

File book/lean/Ch09_FlowPast.lean · *namespace* QuantumFluids.FlowPast · 8 theorems, 1 definitions · 8 #print axioms lines in the file · *audit*: 8 theorem constants audited, 0 sorry, 0 user axioms; axioms used: Classical.choice,

Quot.sound, propext.

```

*9 noncomputable def bogEps (k : ℝ) : ℝ

*9 theorem bogEps_ge_sound {k : ℝ} (hk : 0 ≤ k) : 1 * k ≤ bogEps k

*9 theorem landau_uniform_gp :
  (∀ k : ℝ, 0 < k → 1 ≤ bogEps k / k) ∧

```

```

(∀ δ : ℝ, 0 < δ → ∃ k : ℝ, 0 < k ∧ bogEps k / k < 1 + δ)
★9theorem supersonic_iff {n j : ℝ} (hn : 0 < n) (hj : 0 < j) :
  Real.sqrt n < j / n ↔ n ^ 3 < j ^ 2
★9theorem sonic_speed_threshold {n u V B : ℝ} (hB : n + u ^ 2 / 2 + V = B) :
  n < u ^ 2 ↔ 2 / 3 * (B - V) < u ^ 2
★9theorem bernoulli_min {s n : ℝ} (hs : 0 < s) (hn : 0 < n) :
  3 / 2 * s ≤ s ^ 3 / (2 * n ^ 2) + n
★9theorem bernoulli_eq_iff {s n : ℝ} (hs : 0 < s) (hn : 0 < n) :
  s ^ 3 / (2 * n ^ 2) + n = 3 / 2 * s ↔ n = s
★9theorem barrier_height_bound {s n V : ℝ} (hs : 0 < s) (hn : 0 < n)
  (hB : s ^ 3 / (2 * n ^ 2) + n + V = 1 + s ^ 3 / 2) :
  V ≤ (1 - s) ^ 2 * (s + 2) / 2
★9theorem ramp_overshoot (a h : ℝ) (n : ℕ) :
  ∑ i ∈ range n, a * ((i : ℝ) + 1) * h * h - a * ((n : ℝ) * h) ^ 2 / 2 = a * h ^ 2 * n / 2

```

Files that are not counted

Cho2_NegativeControl (deliberate negative control). `book/lean/Ch02_NegativeControl.lean`: this file is MEANT to fail. It contains two false statements; the compiler’s refusal (quoted in Chapter 2) is the point. It is not part of any build. *Expected result*: a compile error. *Observed*: compiled by the audit driver expecting errors: exit code 1, 2 errors; it failed, as intended. It declares no theorem, is not audited for axioms, and is counted neither as a failure nor as a file of the book’s Lean.

Cho4_NegativeControl_D15121 (deliberate negative control). `book/lean/Ch04_NegativeControl_D15121.lean`: This file is MEANT TO FAIL. It is a negative control quoted in Chapter 4 of the book “Quantum Fluids in Lean 4: a tribute to Henri Godfrin”: a proof checker that cannot reject a wrong statement proves nothing. Please exclude it from audits of theorem files. *Expected result*: a compile error. *Observed*: compiled by the audit driver expecting errors: exit code 1, 1 error; it failed, as intended. It declares no theorem, is not audited for axioms, and is counted neither as a failure nor as a file of the book’s Lean.

Cho4_NegativeControl_L54 (deliberate negative control). `book/lean/Ch04_NegativeControl_L54.lean`: This file is MEANT TO FAIL. It is a negative control quoted in Chapter 4 of the book “Quantum Fluids in Lean 4: a tribute to Henri Godfrin”: a proof checker that cannot reject a wrong statement proves nothing. Please exclude it from audits of theorem files. *Expected result*: a compile error. *Observed*: compiled by the audit driver expecting errors: exit code 1, 1 error; it failed, as intended. It declares no theorem, is not audited for axioms, and is counted neither as a failure nor as a file of the book’s Lean.

Cho6_KTEscape (empty placeholder). `book/lean/Ch06_KTEscape.lean` says in its header that it is superseded and holds no declaration; it exists only because files cannot be deleted in the working environment of the book. It is not counted.

Reproducibility: Rebuilding the Figures and Running the Audit

The figures of this book are written by scripts in `book/figures/`, and the numbers a chapter quotes are meant to come from files named in its numbers file. This appendix says where each piece is, how to rerun it, and what it costs. It is itself partly generated: the figure table, the software versions and the listing of the audit program are written by `book/facts/make_appB.py` from the files as they are when it runs, so they cannot drift from the code. Appendix A (the catalogue of the library) is likewise generated, by `book/facts/make_appA.py`; *both scripts are meant to be rerun* after the last chapter has been written, which refreshes the marks of cited theorems in the catalogue, the citation check (`facts/appA_citation_check.md`) and the table of figures below.

B.1 The machine and the software

The computations of the book were run on one machine that was shared with other work (load average between about 8 and 35 while the audit and the chapter runs were queued), so timings are indications and not benchmarks; the benchmark of chapter 10 carries its own caveats. Table B.1 lists what was used.

component	version
CPU	Intel(R) Core(TM) i7-4930MX CPU @ 3.00GHz, 8 hardware threads, no GPU
Python (project venv)	3.12.3; numpy 2.5.3, scipy 1.18.1, matplotlib 3.11.2
JAX (benchmark only)	0.11.2
Lean 4	4.34.0-rc2, x86_64-unknown-linux-gnu, commit 6a10ac8c22beadecabdbb0919c2b50214762f91d, Release
Mathlib	tag v4.34.0-rc2, commit 85e3a25e006c
rusty-SUNDIALS (working tree used)	v11.5.0-6-g5db8041 (5db8041)
typesetting	LuaHBTeX, Version 1.17.0 (TeX Live 2023/Debian)

Table B.1: Software and hardware used. Read from the machine by `make_appB.py`. The Lean and Mathlib versions are the ones pinned in `lean_src/lakefile.lean` and `lean-toolchain`; the library is never rebuilt with `lake update`.

B.2 Rebuilding a figure

A figure is written by a Python script in `book/figures/`. From the repository root, with the project environment,

```
.venv/bin/python book/figures/chNN_name.py      # writes book/figures/chNN_name.
pdf and .png
PYTHONPATH=/mnt/data/xdev-cache/qf_ext ...      # only for scripts that call the
Rust extension qf_pgpe
```

The convention of the book is that the scripts of a chapter save the numbers that its figures and its text use in `figures/chNN_numbers.json` (a value and its source for each number); the list at the end of this section shows which chapters have such a file. Heavy runs — more than a minute of computation — were serialised on the shared machine with `flock` on one lock file: 9 of the 109 scripts of `book/figures/` call it themselves, and the others were launched under it by the command line of their author. Table B.2 lists, for the chapters present when this appendix was generated, each figure and the script that writes it (matched by the name of the file it saves).

Table B.2: Figures of the chapters present when this appendix was generated, and the scripts that write them (the first line of the docstring of each script, shortened; a script whose name differs from the figure’s is named in the last column). The table is set in blocks so that it can break across pages.

ch.	figure file (written by the script of the same name)	first line of the script’s docstring
1	ch01_bench	Chapter 1, ‘the bench’: what a cold neutron can reach on the measured ^4He dispersion curve.
1	ch01_budget	Chapter 1, ‘speed budget’: where does the translation speed of a vortex-antivortex pair in ...
1	ch01_timeline	Chapter 1, timeline: the landmarks this book revisits. Rows are in chronological order and ...
1	ch01_triptych	Chapter 1, triptych: ONE vortex-antivortex pair in a two-dimensional superfluid seen by the ...
2	ch02_bridge	Figures cho2_bridge and cho2_rates: where a theorem about the Galerkin nonlinearity ...

ch. figure file (written by the script of the same name)	first line of the script's docstring
2 ch02_planewave	Figure cho2_planewave: the exact plane wave of the projected GP equation (Lean: ...
2 ch02_rates	[script ch02_bridge.py] Figures cho2_bridge and cho2_rates: where a theorem about the Galerkin nonlinearity ...
2 ch02_stability	Figure cho2_stability: regions of absolute stability of the two multistep families of ...
2 ch02_workprecision	Figure cho2_workprecision: what the Python module rusty_sundials. CcodeSolver does on ...
3 ch03_circulation	Figure cho3_circulation: counting quanta on the solver field (pair $d = 12$, $t = 20$, loops ...

ch. figure file (written by the script of the same name)	first line of the script's docstring
3 ch03_energy	Figure cho3_energy: the kinetic energy of the field splits into a local quantum-pressure ...
3 ch03_leapfrog	Figure cho3_leapfrog: two vortex pairs of the same sense leapfrog (qfpgpe, 120 time units) ...
3 ch03_pairspeed	Figure cho3_pairspeed: the speed of a vortex pair in the periodic box against its ...
3 ch03_phasefield	Figure cho3_phasefield: one wave function, three faces. A vortex-antivortex pair of the ...
4 ch04_asymptotic	Figure cho4_asymptotic: the series continues to every order and is asymptotic, not ...

ch. figure file (written by the script of the same name)	first line of the script's docstring
4 ch04_bose	Figure cho4_bose: the six Bose integrals of Lean 'PhononSpecificHeat. bose_integral_values' ...
4 ch04_budget	Figure cho4_budget: Eq. (22) against the numerical integral of the measured dispersion, and ...
4 ch04_dispersion	Figure cho4_dispersion: the measured phonon branch of superfluid ^4He ($P = 0$, $T < 0.1 \text{ K}$) and ...
4 ch04_heatmap	Figure cho4_heatmap: which excitations carry the heat? The distribution of the specific ...
4 ch04_ladder	Figure cho4_ladder: the solver against the proved series. The specific heat of the MODEL ...

ch. figure file (written by the script of the same name)	first line of the script's docstring
5 ch05_curve	Figure cho5_curve: the dispersion relation of superfluid ^4He at saturated vapour pressure, ...
5 ch05_rings	Chapter 5: a localised density pulse in the projected GPE spreads as a *dispersive* ring ...
5 ch05_solver	[script ch05_solver_fig.py] Figures cho5_solver and cho5_universal from the broadband runs of cho5_dispersion_run.py ...
5 ch05_thresholds	Figure cho5_thresholds: where do the three-phonon decay channels of ^4He close? ...
5 ch05_universal	[script ch05_solver_fig.py] Figures cho5_solver and cho5_universal from the broadband runs of cho5_dispersion_run.py ...

ch. figure file (written by the script of the same name)	first line of the script's docstring
6 ch06_coulomb	Figure cho6_coulomb: the Coulomb gas inside the solver.
6 ch06_field	Figure cho6_field: the simulated classical Bose field near the Kosterlitz-Thouless ...
6 ch06_jump	Figure cho6_jump: the universal jump of the stiffness and the essential singularity, in the ...
6 ch06_portrait	Figure cho6_portrait: the Kosterlitz flow integrated with CVODE (rusty-SUNDIALS) against ...
6 ch06_screening	Figure cho6_screening: the two Lean statements about vortex positions, tested on vortex ...

ch. figure file (written by the script of the same name)	first line of the script's docstring
7 ch07_dipole	Chapter 7, Figure 2: the dissipative point-vortex model integrated with CVODE ...
7 ch07_failures	Chapter 7, Figure 7: two theorems that were true and did not describe what they were ...
7 ch07_fields	Chapter 7, Figure 4 (the striking one): two antiparallel vortex pairs imprinted in a ...
7 ch07_friction	Chapter 7, Figure 5: measuring the friction in the field.
7 ch07_imprint	Chapter 7, Figure 3: the defective imprint of the first instrument, and the $T = 0$ control ...

ch.	figure file (written by the script of the same name)	first line of the script's docstring
8	ch08_fermisurface	Chapter 8, figure (cho8_fermisurface): the Fermi circle as the dynamical variable, $F = 0$...
8	ch08_timedomain	Chapter 8, figure (cho8_timedomain): the collisionless Landau kinetic equation of a 2D ...
8	ch08_vlasov	Chapter 8, figure (cho8_vlasov): the plasma cousin of zero sound, solved by the crate ...
8	ch08_window	Chapter 8, figure (cho8_window): the (q, ω) plane of a Fermi liquid, in the units of ...
8	ch08_zerosound	Chapter 8, figure 1 (cho8_zerosound): the zero-sound root $s(F)$, the weights of the pole and ...
ch.	figure file (written by the script of the same name)	first line of the script's docstring
9	ch09_bir th	
9	ch09_census	
9	ch09_force	
9	ch09_mach	
9	ch09_wake	
ch.	figure file (written by the script of the same name)	first line of the script's docstring
10	ch10_bench	Chapter 10, benchmark panel: time per IF-RK4 step of the PGPE problem ($N \times N$, $L = N/2$, $g = \dots$)
10	ch10_estimator	Chapter 10, figure 'estimator': the registered synthetic gate G_0 (known $\alpha = 0.02$) run ...
10	ch10_library	Chapter 10, figure 'library': the QuantumFluids Lean library as audited for this book.
10	ch10_verdicts	Chapter 10, figure 'verdict map': every criterion that LEDGER.md records one by one for the ...

Other files by chapter. Besides figure scripts a chapter may own: a numbers file (`figures/chNN_numbers.json`), a bibliography file (`refs_chNN.bib`, merged into `refs_all.bib` by `build_refs.py`), Lean files written for the book (`lean/ChNN_*.lean`, compiled against the same pinned Mathlib and listed in appendix A), Rust sources (`rust/`) and a report of what was verified (`facts/chNN_report.md`). Present when this appendix was generated:

- chapter 1: numbers; bib; Lean: `Cho1_NeutronWindow`; report
- chapter 2: numbers; bib; Lean: `Cho2_Galerkin`, `Cho2_Primer`; negative control (meant to fail, not counted): `Cho2_NegativeControl`; Rust; report
- chapter 3: numbers; bib; Lean: `Cho3_PointVortexPair`; report
- chapter 4: numbers; bib; Lean: `Cho4_DosLink`; negative control (meant to fail, not counted): `Cho4_NegativeControl_D15121`, `Cho4_NegativeControl_L54`
- chapter 5: numbers; bib; Lean: `Cho5_BogoliubovDispersion`; report

- chapter 6: numbers; bib; Lean: Cho6_KTForward; empty placeholder (not counted): Cho6_KTEscape; report
- chapter 7: numbers; bib; Lean: Cho7_DissipativePairs
- chapter 8: numbers; bib; Lean: Cho8_ZeroSound2D; Rust; report
- chapter 9: numbers; bib; Lean: Cho9_FlowPast; report
- chapter 10: numbers; bib; report

B.3 CVOICE: which build is imported

The solver library has a history that the book's CVOICE results depend on. Until the repair of 28 September 2026 its Adams method was, in effect, implicit Euler (docs/CVOICE_ADAMS_FIX.md of rusty-SUNDIALS): the order never rose above one, so the work grew like $\text{rtol}^{-1/2}$. The Python module `rusty_sundials` that sits in the project's virtual environment was built on 26 September, before the repair, and is *stale*. Every CVOICE result of this book must come from rusty-SUNDIALS commit `5db8041fd2e3840defc2b4a8c26c1cbb2467fd` (the `VII.6.0` line) or later. The recipe:

```
# 1. build the module from the checkout into a scratch directory (about 3 minutes;
    nothing is written to the checkout)
flock /mnt/data/xdev-cache/tmp/qf_heavy.lock nice book/rust/ch02_build_py.sh OUTDIR
    # writes OUTDIR/py/rusty_sundials.so, OUTDIR/commit.txt
# 2. a stable copy of the build used for the book (commit 5
    db8041fd2e3840defc2b4a8c26c1cbb2467fd):
#
    /mnt/data/xdev-cache/rs_py_5db8041/rusty_sundials.so  sha256 0
    bdb1b4a504fb4817738c483f797e6a109b99121b666f1e51ad8e05c2fb996d7
# 3. run with the repaired module FIRST on the path, then the quantum-fluids
    extension:
PYTHONPATH=/mnt/data/xdev-cache/rs_py_5db8041:/mnt/data/xdev-cache/qf_ext .venv/bin
    /python book/figures/chNN_script.py
# 4. check which build a script imports, whenever in doubt:
.venv/bin/python book/figures/ch10_cvoice_probe.py stale      # the module in the
    virtual environment
PYTHONPATH=/mnt/data/xdev-cache/rs_py_5db8041:/mnt/data/xdev-cache/qf_ext .venv/bin/
    python book/figures/ch10_cvoice_probe.py fixed
```

The probe integrates $y' = -y$ on $[0, 10]$ with absolute tolerance 10^{-14} and counts the right-hand-side calls and the relative error of $y(10)$. It tells the two builds apart at once (table B.3): in the stale one the Adams error falls only as the square root of the tolerance and the cost explodes. A result obtained with the stale module is wrong in a way that no convergence *within* the module reveals, and no test of the source tree can: the tests ran on the source (chapter 10, section 10.4). The rule for the book's scripts is therefore to check the probe once per session, and to record the commit and the checksum of the module in the numbers file of the chapter.

method	rtol	stale module (built 2026-09-26 15:00)	rel. error	repaired build (commit 5db8041)	rel. error
		RHS calls		RHS calls	
adams	10^{-4}	2 386	6.5×10^{-2}	168	5.9×10^{-4}
adams	10^{-6}	23 750	6.4×10^{-3}	359	1.9×10^{-5}
adams	10^{-8}	237 022	6.4×10^{-4}	511	2.0×10^{-7}
adams	10^{-10}	2 214 597	6.9×10^{-5}	616	1.7×10^{-9}
bdf	10^{-4}	335	9.7×10^{-4}	246	7.1×10^{-4}
bdf	10^{-6}	759	2.3×10^{-5}	492	2.0×10^{-5}
bdf	10^{-8}	1 327	4.9×10^{-7}	933	4.5×10^{-7}
bdf	10^{-10}	1 895	1.2×10^{-8}	1 618	1.2×10^{-8}

Table B.3: The probe $y' = -y$, $t \in [0, 10]$, atol 10^{-14} , run with the module of the virtual environment (shared object 9fbdo9e985c2...) and with the repaired build (obdb1b4a504f...). The BDF method is shown for comparison; it differs little between the builds.

B.4 Running the Lean audit

The audit of chapter 10 has three parts, all driven by `book/facts/lean_audit.py`; nothing is written into the Lean project trees, and `lake update` is never run. From the repository root:

```
python3 book/facts/lean_audit.py          # compile every module in import order,
    audit its environment; ~1 h on a loaded machine
python3 book/facts/lean_audit.py --resume # skip the modules whose facts/audit/<
    Module>.json exists and was made on the present code
python3 book/facts/lean_audit.py --book --resume # the same program on the Lean
    files written for the book (book/lean/*.lean); a file that is meant to fail is
    compiled expecting errors
python3 book/facts/lean_audit.py --report # only rebuild facts/lean_audit.md from
    the stored JSON
python3 book/facts/lean_audit.py --controls # negative control: run the auditor on a
    file with planted defects (seconds)
python3 scripts/regen_axiom_audit.py --check # the library's own staleness check of
    the #print axioms blocks (read-only)
python3 book/facts/make_appA.py           # regenerate the catalogue, the
    citation check and the namespace table
```

The driver copies each module to a scratch directory (`QF_AUDIT_WORK`, default `/mnt/data/xdev-cache/tmp/qf_audit`), appends the program below, and compiles it with `lake env lean -R <scratch> -o <scratch>/<Module>.olean` inside the pinned Mathlib tree, with the scratch directory on `LEAN_PATH` so that a module finds its compiled siblings. It takes the shared lock of the machine (`QF_LOCK`) for a short batch only: it starts a new module while the hold is younger than `QF_BATCH_SECONDS` (the run for this book used 100 s, so one to three modules per hold), and then stays away from the lock for `QF_YIELD_SECONDS` (45 s) so that a job that is waiting takes it first. An audit counts as current only if it was made on the present *code* of the file: the driver stores a hash of the source with comments and white space removed, so that a corrected docstring does

not invalidate an audit and any change of code does (appendix A then prints stale). Its output is one log and one JSON file per module in `facts/audit/`, the table `facts/lean_audit.md`, and `facts/audit/summary.json`. The program that is appended — new for this book, and tested on the file with planted defects before it was trusted — is:

```

/-! ## Book audit (appended by book/facts/lean_audit.py to a scratch copy; not
    ↪ part of the library) -/
open Lean Elab Command in
run_cmd do
  let env ← getEnv
  let locals := env.checked.get.constants.foldStage2
    (fun (acc : Array (Name × ConstantInfo)) n ci => acc.push (n, ci)) #[]
  let userName (n : Name) : Name := (privateToUserName? n).getD n
  let isNamed (n : Name) : Bool := !(userName n).isInternal
  let namedThms := locals.filter (fun (n, ci) => ci.isTheorem && isNamed n)
  let namedDefs := locals.filter (fun (n, ci) => (ci.matches .defnInfo _) &&
    ↪ isNamed n)
  let axDecls := locals.filter (fun (_, ci) => ci.matches .axiomInfo _)
  let thmNames : NameSet := namedThms.foldl (fun s (n, _) => s.insert n) {}
  let mut union : NameSet := {}
  let mut rows : Array String := #[]
  for (n, _) in locals do
    let axs ← collectAxioms n
    for a in axs do union := union.insert a
    if thmNames.contains n then
      let line ← match (← findDeclarationRanges? n) with
      | some r => pure (toString r.range.pos.line)
      | none => pure "-1"
      rows := rows.push s!"QFAUDIT|THM|{userName n}|{line}|{" , ".intercalate (axs.
        ↪ toList.map toString)}"
  IO.println s!"QFAUDIT|SUMMARY|{locals.size}|{namedThms.size}|{namedDefs.size}|{
    ↪ axDecls.size}|{union.contains ``sorryAx}|{" , ".intercalate (union.toList.
    ↪ map toString)}"
  for (n, _) in axDecls do IO.println s!"QFAUDIT|AXIOMDECL|{n}"
  for r in rows do IO.println r

```

Reading it: `foldStage2` lists the constants that this file added to the environment; `collectAxioms` returns the axioms a constant depends on, through the whole chain of lemmas it uses; the union over all constants is the footprint of the module; a constant whose footprint contains `sorryAx` contains a `sorry` somewhere below it, even if its own proof does not. The negative control (`facts/audit_controls/AuditControls.lean`, output in `AuditControls.out`) plants a `sorry`, the same `sorry` inherited, a `sorry` in a private theorem, an axiom of its own and a `native_decide`; the auditor reports each.

B.5 Limits of what can be reproduced

- Timings depend on the machine and on its load; the benchmark of chapter 10 was taken on a shared machine and is to be repeated on an idle one.
- The long campaign runs behind chapters 6 to 9 (the pre-registered runs of tens of hours) are not rerun by the book's scripts; their saved outputs are under `data/generated/pgpe/`: 2.8G on the machine of the author; the directory `data/generated/` is git-ignored, and only 189 small analysis files of it were committed (measured when this appendix was generated), and the scripts of the chapters read those outputs.
- No computation of this book used a GPU or a TPU.
- The environment audit trusts Lean's own elaborator and kernel; the second kernel (nanoda) is run through the Comparator, whose recorded results are in `docs/COMPARATOR_SETUP.md` and were not rerun for the book.

Bibliography

- [1] H. Godfrin, M. Meschke, H.-J. Lauter, A. Sultan, H. M. Böhm, E. Krotscheck, and M. Panholzer. Observation of a roton collective mode in a two-dimensional Fermi liquid. *Nature*, 483:576–579, 2012. doi: 10.1038/nature10919. Open-access author copy: <https://hal.science/hal-00920422>.
- [2] H. Godfrin, K. Beauvois, A. Sultan, E. Krotscheck, J. Dawidowski, B. Fåk, and J. Ollivier. Dispersion relation of Landau elementary excitations and thermodynamic properties of superfluid ^4He . *Phys. Rev. B*, 103:104516, 2021. doi: 10.1103/PhysRevB.103.104516. arXiv:2012.09067.
- [3] L. D. Landau. Theory of the superfluidity of helium II. *Phys. Rev.*, 60:356–358, 1941.
- [4] A. C. Hindmarsh, P. N. Brown, K. E. Grant, S. L. Lee, R. Serban, D. E. Shumaker, and C. S. Woodward. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. *ACM Trans. Math. Softw.*, 31:363–396, 2005.
- [5] H. Godfrin and E. Krotscheck. The dynamics of quantum fluids. arXiv:2206.06039, 2022.
- [6] V. F. Sears. Neutron scattering lengths and cross sections. *Neutron News*, 3(3):26–37, 1992. Source of the NIST NCNR tables of neutron scattering lengths and cross sections.
- [7] G. L. Squires. *Introduction to the Theory of Thermal Neutron Scattering*. Cambridge University Press, 3 edition, 2012. First edition 1978.
- [8] H. R. Glyde. *Excitations in Liquid and Solid Helium*, volume 9 of *Oxford Series on Neutron Scattering in Condensed Matter*. Oxford University Press, 1994.
- [9] Institut Laue–Langevin. IN5: direct-geometry time-of-flight spectrometer for cold neutrons (instrument characteristics). Web page, consulted 2026-10-09; <https://www.ill.eu/users/instruments/instruments-list/in5/characteristics>, 2026.
- [10] R. J. Donnelly. *Quantized Vortices in Helium II*. Cambridge University Press, 1991.
- [11] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, 2021.
- [12] X. Callens. QuantumFluids: a Lean 4 library on the mathematical structure of quantum fluids. Zenodo, doi:10.5281/zenodo.23268302 (v1.19.0); <https://github.com/xaviercallens/SocrateAI-Scientific-QuantumFluids>, 2026.

- [13] The mathlib Community. The Lean mathematical library. CPP 2020; <https://github.com/leanprover-community/mathlib4>, 2020.
- [14] G. Baym and C. Pethick. *Landau Fermi-Liquid Theory*. Wiley, 1991.
- [15] L. Onsager. Statistical hydrodynamics. *Nuovo Cimento Suppl.*, 6:279–287, 1949.
- [16] R. P. Feynman. Application of quantum mechanics to liquid helium. In *Progress in Low Temperature Physics*, volume 1, pages 17–53. North-Holland, 1955.
- [17] E. P. Gross. Structure of a quantized vortex in boson systems. *Nuovo Cimento*, 20:454–477, 1961.
- [18] L. P. Pitaevskii. Vortex lines in an imperfect Bose gas. *Sov. Phys. JETP*, 13:451–454, 1961.
- [19] M. J. Davis, S. A. Morgan, and K. Burnett. Simulations of Bose fields at finite temperature. *Phys. Rev. Lett.*, 87:160402, 2001.
- [20] P. B. Blakie, A. S. Bradley, M. J. Davis, R. J. Ballagh, and C. W. Gardiner. Dynamics and statistical mechanics of ultra-cold Bose gases using c-field techniques. *Adv. Phys.*, 57:363–455, 2008.
- [21] X. Callens. qf-pgpe: a verified pure-Rust projected Gross–Pitaevskii engine for quantum-fluid experiments, with a Python extension. Zenodo, doi:10.5281/zenodo.23269715, 2026.
- [22] J. B. Weiss and J. C. McWilliams. Nonergodicity of point vortices. *Phys. Fluids A*, 3(5): 835–844, 1991. doi: 10.1063/1.858014.
- [23] X. Callens. Vortex transport and screening in a closed two-dimensional Bose field (version 2.2). Zenodo, doi:10.5281/zenodo.23262143, 2026.
- [24] C. A. Jones and P. H. Roberts. Motions in a Bose condensate. IV. axisymmetric solitary waves. *J. Phys. A: Math. Gen.*, 15(8):2599, 1982.
- [25] L. D. Landau. The theory of a Fermi liquid. *Sov. Phys. JETP*, 3:920–925, 1957.
- [26] L. Pitaevskii and S. Stringari. *Bose–Einstein Condensation and Superfluidity*. Oxford University Press, 2016.
- [27] Junhwan Kwon and Yong-il Shin. Dynamic similarity of vortex shedding in a superfluid flowing past a penetrable obstacle. *Phys. Rev. Research*, 8(2):023246, 2026. doi:10.1103/wlft-tc8w; arXiv:2602.03518.
- [28] Junhwan Kwon and Yong-il Shin. Data and analysis scripts for “Dynamic similarity of vortex shedding in a superfluid flowing past a penetrable obstacle”. Zenodo, version 1.0.0, 7 May 2026, doi:10.5281/zenodo.20068724, 2026.

- [29] S. A. Orszag. On the elimination of aliasing in finite-difference schemes by filtering high-wavenumber components. *J. Atmos. Sci.*, 28:1074, 1971.
- [30] G. G. Dahlquist. A special stability problem for linear multistep methods. *BIT*, 3:27–43, 1963.
- [31] E. Hairer and G. Wanner. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, volume 14 of *Springer Series in Computational Mathematics*. Springer, 2nd revised edition, 1996.
- [32] A. Prothero and A. Robinson. On the stability and accuracy of one-step methods for solving stiff systems of ordinary differential equations. *Math. Comp.*, 28(125):145–162, 1974.
- [33] J. Avigad, L. de Moura, S. Kong, and S. Ullrich. Theorem proving in Lean 4. online book, https://lean-lang.org/theorem_proving_in_lean4/, consulted 2026-10-09, 2026.
- [34] J. Avigad and P. Massot. Mathematics in Lean. online book, https://leanprover-community.github.io/mathematics_in_lean/, consulted 2026-10-09, 2026.
- [35] E. Hairer, S. P. Nørsett, and G. Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, 2nd revised edition, 1993.
- [36] W. F. Vinen. The detection of single quanta of circulation in liquid helium II. *Proc. R. Soc. Lond. A*, 260:218–236, 1961. doi: 10.1098/rspa.1961.0029.
- [37] R. J. Zieve. Sixty years of quantized circulation. arXiv:2306.15528, 2023.
- [38] E. J. Yarmchuk, M. J. V. Gordon, and R. E. Packard. Observation of stationary vortex arrays in rotating superfluid helium. *Phys. Rev. Lett.*, 43:214, 1979.
- [39] D. J. Bishop and J. D. Reppy. Study of the superfluid transition in two-dimensional ^4He films. *Phys. Rev. Lett.*, 40:1727–1730, 1978.
- [40] J. M. Kosterlitz and D. J. Thouless. Ordering, metastability and phase transitions in two-dimensional systems. *J. Phys. C*, 6:1181–1203, 1973.
- [41] E. Madelung. Quantentheorie in hydrodynamischer Form. *Z. Phys.*, 40:322, 1927.
- [42] H. Helmholtz. Über Integrale der hydrodynamischen Gleichungen, welche den Wirbelbewegungen entsprechen. *J. reine angew. Math.*, 55:25–55, 1858.
- [43] N. E. Phillips, C. G. Waterfield, and J. K. Hoffer. Calorimetric evidence for positive phonon dispersion in liquid helium-4. *Phys. Rev. Lett.*, 25:1260–1262, 1970. doi:10.1103/PhysRevLett.25.1260.

- [44] D. S. Greywall. Specific heat and phonon dispersion of liquid ^4He . *Phys. Rev. B*, 18: 2127–2144, 1978. doi:10.1103/PhysRevB.18.2127; erratum: *Phys. Rev. B* 21, 1329–1331 (1980), doi:10.1103/PhysRevB.21.1329.
- [45] C. M. Bender and S. A. Orszag. *Advanced Mathematical Methods for Scientists and Engineers I*. Springer, New York, 1999. doi:10.1007/978-1-4757-3069-2.
- [46] L. D. Landau. On the theory of superfluidity of helium II. *J. Phys. (USSR)*, 11:91, 1947.
- [47] R. P. Feynman and M. Cohen. Energy spectrum of the excitations in liquid helium. *Phys. Rev.*, 102:1189–1204, 1956.
- [48] H. Palevsky, K. Otnes, K. E. Larsson, R. Pauli, and R. Stedman. Excitation of rotons in helium II by cold neutrons. *Phys. Rev.*, 108:1346–1347, 1957.
- [49] D. Rugar and J. S. Foster. Accurate measurement of low-energy phonon dispersion in liquid ^4He . *Phys. Rev. B*, 30:2595–2602, 1984.
- [50] N. N. Bogoliubov. On the theory of superfluidity. *J. Phys. (USSR)*, 11:23–32, 1947.
- [51] F. Dalfovo, S. Giorgini, L. P. Pitaevskii, and S. Stringari. Theory of Bose–Einstein condensation in trapped gases. *Rev. Mod. Phys.*, 71:463–512, 1999.
- [52] R. P. Feynman. Atomic theory of the two-fluid model of liquid helium. *Phys. Rev.*, 94: 262–277, 1954.
- [53] L. Santos, G. V. Shlyapnikov, and M. Lewenstein. Roton-maxon spectrum and stability of trapped dipolar Bose–Einstein condensates. *Phys. Rev. Lett.*, 90:250403, 2003.
- [54] L. Chomaz, R. M. W. van Bijnen, D. Petter, G. Faraoni, S. Baier, J. H. Becher, M. J. Mark, F. Wächtler, L. Santos, and F. Ferlaino. Observation of roton mode population in a dipolar quantum gas. *Nature Phys.*, 14:442–446, 2018. doi:10.1038/s41567-018-0054-7, arXiv:1705.06914.
- [55] X. Callens. Pre-registration of the projected Gross–Pitaevskii study: known answers K1–K7 and amendment A1. Programme document docs/designs/PGPE_BKT_PREREG.md, repository SocrateAI-Scientific-QuantumFluids; archived in the software record doi:10.5281/zenodo.2285581, 2026.
- [56] H. R. Glyde. Excitations in the quantum liquid ^4He : a review. *Rep. Prog. Phys.*, 81:014501, 2017.
- [57] R. A. Cowley and A. D. B. Woods. Inelastic scattering of thermal neutrons from liquid helium. *Can. J. Phys.*, 49:177–200, 1971.

-
- [58] H. J. Maris. Phonon-phonon interactions in liquid helium. *Rev. Mod. Phys.*, 49:341–359, 1977.
- [59] D. R. Nelson and J. M. Kosterlitz. Universal jump in the superfluid density of two-dimensional superfluids. *Phys. Rev. Lett.*, 39:1201–1205, 1977. doi: 10.1103/PhysRevLett.39.1201.
- [60] V. Ambegaokar, B. I. Halperin, D. R. Nelson, and E. D. Siggia. Dynamics of superfluid films. *Phys. Rev. B*, 21:1806–1826, 1980. doi: 10.1103/PhysRevB.21.1806.
- [61] J. M. Kosterlitz. The critical properties of the two-dimensional xy model. *J. Phys. C: Solid State Phys.*, 7:1046–1060, 1974. doi: 10.1088/0022-3719/7/6/005.
- [62] A. Sultan, M. Meschke, H.-J. Lauter, and H. Godfrin. Static structure factor of two-dimensional liquid ^3He adsorbed on graphite. *J. Low Temp. Phys.*, 169(5–6):367–376, 2012. doi: 10.1007/s10909-012-0649-9.
- [63] N. D. Mermin and H. Wagner. Absence of ferromagnetism or antiferromagnetism in one- or two-dimensional isotropic Heisenberg models. *Phys. Rev. Lett.*, 17:1133–1136, 1966. doi: 10.1103/PhysRevLett.17.1133.
- [64] P. C. Hohenberg. Existence of long-range order in one and two dimensions. *Phys. Rev.*, 158:383–386, 1967. doi: 10.1103/PhysRev.158.383.
- [65] V. L. Berezinskii. Destruction of long-range order in one-dimensional and two-dimensional systems having a continuous symmetry group. I. Classical systems. *Sov. Phys. JETP*, 32:493, 1971. English translation of *Zh. Eksp. Teor. Fiz.* 59, 907; first page as given by the JETP archive record.
- [66] N. Prokofev and B. Svistunov. Two-dimensional weakly interacting Bose gas in the fluctuation region. *Phys. Rev. A*, 66:043608, 2002. doi: 10.1103/PhysRevA.66.043608.
- [67] N. Prokofev, O. Ruebenacker, and B. Svistunov. Critical point of a weakly interacting two-dimensional Bose gas. *Phys. Rev. Lett.*, 87:270402, 2001. doi: 10.1103/PhysRevLett.87.270402.
- [68] V. Ambegaokar, B. I. Halperin, D. R. Nelson, and E. D. Siggia. Dissipation in two-dimensional superfluids. *Phys. Rev. Lett.*, 40:783–786, 1978. doi: 10.1103/PhysRevLett.40.783.
- [69] P. Minnhagen. The two-dimensional Coulomb gas, vortex unbinding, and superfluid-superconducting films. *Rev. Mod. Phys.*, 59:1001–1066, 1987. doi: 10.1103/RevModPhys.59.1001.

- [70] Z. Hadzibabic, P. Krüger, M. Cheneau, B. Battelier, and J. Dalibard. Berezinskii–Kosterlitz–Thouless crossover in a trapped atomic gas. *Nature*, 441:1118–1121, 2006. doi: 10.1038/nature04851.
- [71] T. P. Simula and P. B. Blakie. Thermal activation of vortex-antivortex pairs in quasi-two-dimensional Bose-Einstein condensates. *Phys. Rev. Lett.*, 96:020404, 2006. doi: 10.1103/PhysRevLett.96.020404.
- [72] C. J. Foster, P. B. Blakie, and M. J. Davis. Vortex pairing in two-dimensional Bose gases. *Phys. Rev. A*, 81:023623, 2010. doi: 10.1103/PhysRevA.81.023623.
- [73] M. Hasenbusch. The two-dimensional XY model at the transition temperature: a high-precision Monte Carlo study. *J. Phys. A: Math. Gen.*, 38:5869–5883, 2005. doi: 10.1088/0305-4470/38/26/003.
- [74] G. Moon, W. J. Kwon, H. Lee, and Y. Shin. Thermal friction on quantum vortices in a Bose–Einstein condensate. *Phys. Rev. A*, 92:051601(R), 2015. arXiv:1509.02236.
- [75] T. W. Neely, G. Gauthier, C. Glasspool, M. J. Davis, and M. T. Reeves. Melting of a vortex matter Wigner crystal. arXiv:2402.09920, 2024.
- [76] Yu. A. Sergeev. Mutual friction in bosonic superfluids: a review. *J. Low Temp. Phys.*, 212: 251–305, 2023.
- [77] V. Shukla, M. Brachet, and R. Pandit. Superfluid mutual-friction coefficients from vortex dynamics in the two-dimensional Galerkin-truncated Gross–Pitaevskii equation. arXiv:1412.0706, 2014.
- [78] D. J. Thouless, P. Ao, and Q. Niu. Transverse force on a quantized vortex in a superfluid. *Phys. Rev. Lett.*, 76:3758–3761, 1996.
- [79] S. V. Iordanskii. On the mutual friction between the normal and superfluid components in a rotating Bose gas. *Ann. Phys. (N.Y.)*, 29:335–349, 1964.
- [80] E. B. Sonin. The Magnus force in superfluids and superconductors. *Phys. Rev. B*, 55:485, 1997.
- [81] Z. Mehdi, J. J. Hope, S. S. Szigeti, and A. S. Bradley. Mutual friction and diffusion of two-dimensional quantum vortices. *Phys. Rev. Research*, 5:013184, 2023. arXiv:2205.04065.
- [82] C. Rorai, K. R. Sreenivasan, and M. E. Fisher. Propagating and annihilating vortex dipoles in the Gross–Pitaevskii equation. *Phys. Rev. B*, 88:134522, 2013.
- [83] X. Callens. The friction on a quantized vortex in a truncated Gross–Pitaevskii field follows the temperature, not the normal density. Zenodo, doi:10.5281/zenodo.23262132, 2026.

- [84] G. Krstulovic and M. Brachet. Energy cascade with small-scales thermalization, counterflow metastability and anomalous velocity of vortex rings in Fourier-truncated Gross–Pitaevskii equation. *Phys. Rev. E*, 83:066311, 2011.
- [85] N. Grani, D. Hernández-Rajkov, C. Daix, P. Pieri, M. Pini, P. Magierski, G. Wlazłowski, M. Frómeta Fernández, F. Scazza, G. Del Pace, and G. Roati. Mutual friction and vortex Hall angle in a strongly interacting Fermi superfluid. *Nat. Commun.*, 16:10245, 2025.
- [86] K. Modin and M. Viviani. Integrability of point-vortex dynamics via symplectic reduction: a survey. *Arnold Math. J.*, 7:357–385, 2021. arXiv:2003.00716.
- [87] H. Godfrin, M. Meschke, H.-J. Lauter, H. M. Böhm, E. Krotscheck, and M. Panholzer. Observation of zero-sound at atomic wave-vectors in a monolayer of liquid ^3He . *J. Low Temp. Phys.*, 158(1–2):147–154, 2010. doi:10.1007/s10909-009-9952-5.
- [88] C. Mouhot and C. Villani. On Landau damping. *Acta Math.*, 207(1):29–201, 2011. doi:10.1007/s11511-011-0068-9.
- [89] J. Bedrossian. Formalization of Landau damping in the Vlasov–Poisson equations in Lean. arXiv:2609.16801 (submitted 15 September 2026), 2026.
- [90] W. R. Abel, A. C. Anderson, and J. C. Wheatley. Propagation of zero sound in liquid ^3He at low temperatures. *Phys. Rev. Lett.*, 17:74–78, 1966.
- [91] C. Raman, M. Köhl, R. Onofrio, D. S. Durfee, C. E. Kuklewicz, Z. Hadzibabic, and W. Ketterle. Evidence for a critical velocity in a Bose–Einstein condensed gas. *Phys. Rev. Lett.*, 83:2502, 1999.
- [92] W. J. Kwon, S. W. Seo, and Y. Shin. Periodic shedding of vortex dipoles from a moving penetrable obstacle in a Bose–Einstein condensate. *Phys. Rev. A*, 92:033613, 2015.
- [93] W. J. Kwon, J. H. Kim, S. W. Seo, and Y. Shin. Observation of von Kármán vortex street in an atomic superfluid gas. *Phys. Rev. Lett.*, 117:245301, 2016.
- [94] T. Frisch, Y. Pomeau, and S. Rica. Transition to dissipation in a model of superflow. *Phys. Rev. Lett.*, 69:1644–1647, 1992.
- [95] C. Josserand, Y. Pomeau, and S. Rica. Vortex shedding in a model of superflow. *Physica D*, 134:111, 1999. arXiv:physics/9901055.
- [96] T. Winiecki, B. Jackson, J. F. McCann, and C. S. Adams. Vortex shedding and drag in dilute Bose–Einstein condensates. *J. Phys. B: At. Mol. Opt. Phys.*, 33:4069, 2000.
- [97] M. T. Reeves, T. P. Billam, B. P. Anderson, and A. S. Bradley. Identifying a superfluid Reynolds number via dynamical similarity. *Phys. Rev. Lett.*, 114:155302, 2015.

- [98] H. Kwak, J. H. Jung, and Y. Shin. Minimum critical velocity of a Gaussian obstacle in a Bose–Einstein condensate. *Phys. Rev. A*, 107:023310, 2023.
- [99] leanprover (GitHub organisation). Comparator: a trustworthy judge for Lean proofs. <https://github.com/leanprover/comparator>, n.d. Repository page read 2026-10-09.
- [100] ammkrn (GitHub user). nanoda_lib: an external type checker for Lean 4. https://github.com/ammkrm/nanoda_lib, n.d. Repository page read 2026-10-09.
- [101] N. Vladimirova, M. Shavit, and G. Falkovich. Fibonacci turbulence. *Phys. Rev. X*, 11:021063, 2021. arXiv:2101.10418; reference verified 2026-10-09 on the arXiv abstract page.
- [102] N. Vladimirova, M. Shavit, S. Belan, and G. Falkovich. Second harmonic generation as a minimal model of turbulence. *Phys. Rev. E*, 104:014129, 2021. arXiv:2103.15468; reference verified 2026-10-09 on the arXiv abstract page.
- [103] W. F. Wreszinski and Jr. M. A. da Silva. Onsager’s inequality, the Landau–Feynman ansatz and superfluidity. *J. Phys. A: Math. Gen.*, 38:6293–6310, 2005. arXiv:cond-mat/0411640; reference verified 2026-10-09 on the arXiv abstract page.
- [104] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The preregistration revolution. *Proc. Natl. Acad. Sci. USA*, 115(11):2600–2606, 2018. Verified 2026-10-09 against the Crossref record of doi:10.1073/pnas.1708274114.
- [105] R. D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. Verified 2026-10-09 against the Crossref record of doi:10.1126/science.1213847.